O'REILLY®

alist

Машинное обучение для приложений высокого риска

Подходы к ответственному искусственному интеллекту



Патрик Холл,
Джеймс Кертис,
Парул Панди

Предисловие
Агуса Суджианто

Machine Learning for High-Risk Applications

Approaches to Responsible AI

Patrick Hall, James Curtis, and Parul Pandey

Foreword by Agus Sudjianto, PhD

Beijing • Boston • Farnham • Sebastopol • Tokyo O'REILLY

Книги для программистов: https://t.me/free_books_for_it

Патрик Холл, Джеймс Кертис, Парул Панди

Машинное обучение для приложений высокого риска

Предисловие Агуса Суджианто

Астана
«АЛИСТ»
2025

Книги для программистов: https://t.me/free_books_for_it

УДК 004.4'236
ББК 32.973.26-018
Х72

Холл П.

Х72 Машинное обучение для приложений высокого риска: Пер. с англ. / П. Холл, Дж. Кертис, П. Панди. — Астана: АЛИСТ, 2025. — 464 с.: ил.

ISBN 978-601-08-4982-2

Книга представляет собой комплексное руководство по применению искусственного интеллекта и машинного обучения (ИИ/ML) с целью снижения рисков для современного бизнеса, связанного с использованием этих технологий. Рассмотрены основы управления рисками и компьютерной безопасности, нормативные акты, ответственность за качество продуктов, основанных на ML, а также объяснимые модели и методы их проверки, включая новый фреймворк управления рисками NIST AI. Читателю предложен углубленный взгляд на программирование с использованием Python и подробными примерами для структурированных и неструктурированных данных. Особое внимание уделяется объяснимым бустинговым машинам, библиотеке XGBoost и методам повышения качества моделей ML. Представлены основанные на реальном опыте советы о том, как организовать успешную работу с приложениями высокого риска. Приведены практические примеры, иллюстрирующие важность и сложность внедрения ML в различных отраслях.

Для студентов, инженеров ML
и специалистов по обработке данных

УДК 004.4'236
ББК 32.973.26-018

© 2025 ALIST LLP

Authorized Russian translation of the English edition of ***Machine Learning for High-Risk Applications***, ISBN 9781098102432 © 2023 Patrick Hall, James Curtis, and Parul Pandey.
This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Авторизованный перевод с английского языка на русский издания ***Machine Learning for High-Risk Applications***, ISBN 9781098102432 © 2023 Patrick Hall, James Curtis, Parul Pandey.
Перевод опубликован и продается с разрешения компании-правообладателя O'Reilly Media, Inc.

ISBN 978-1-098-10243-2 (англ.)
ISBN 978-601-08-4982-2 (каз.)

© Patrick Hall, James Curtis, Parul Pandey, 2023
© Издание на русском языке. ТОО "АЛИСТ", 2025

Книги для программистов: https://t.me/free_books_for_it

Краткое оглавление

Оглавление 7
Отзывы о книге 15
Предисловие 19
Введение 21

ЧАСТЬ I. ТЕОРИЯ И ПРАКТИКА УПРАВЛЕНИЯ РИСКАМИ В ПРИЛОЖЕНИЯХ ИИ 31
Глава 1. Современное управление рисками в ML 33
Глава 2. Интерпретируемость и объяснимость 65
Глава 3. Отладка систем машинного обучения 113
Глава 4. Борьба с предвзятостью в машинном обучении 157
Глава 5. Безопасность машинного обучения 197

ЧАСТЬ II. УПРАВЛЕНИЕ РИСКАМИ ИИ В ДЕЙСТВИИ 227
Глава 6. Объяснимые бустинговые машины и модели XGBoost 229
Глава 7. Объяснение классификатора изображений PyTorch 271
Глава 8. Выбор и отладка моделей XGBoost 299
Глава 9. Отладка классификатора изображений PyTorch 335
Глава 10. Тестирование и исправление предвзятости с помощью XGBoost 365
Глава 11. Атака красной команды на XGBoost 407

ЧАСТЬ III. ЗАКЛЮЧЕНИЕ 435
Глава 12. Как добиться успеха в машинном обучении высокого риска 437

Предметный указатель 453
Об авторах 457
Об изображении на обложке 459

Книги для программистов: https://t.me/free_books_for_it

Оглавление

Краткое оглавление **5**

Отзывы о книге **15**

Предисловие **19**

Введение **21**
Кому адресована эта книга 22
Что вы узнаете из этой книги 22
Соответствие системе управления рисками искусственного интеллекта NIST 24
Структура книги 25
Часть I 25
Часть II 26
Часть III 26
Примеры наборов данных 26
Кредитные данные Тайваня 27
Данные рентгенографии грудной клетки Kaggle 28
Условные обозначения, принятые в этой книге 29
Архив с иллюстрациями к книге 29
Благодарности 30

ЧАСТЬ I. ТЕОРИЯ И ПРАКТИКА УПРАВЛЕНИЯ РИСКАМИ В ПРИЛОЖЕНИЯХ ИИ **31**

Глава 1. Современное управление рисками в ML **33**
Обзор нормативно-правовой базы 34
Закон ЕС об искусственном интеллекте 34
Федеральные законы и правила США 35
Государственные и муниципальные законы 36
Изначальная ответственность за качество продукта 37
Правоприменительная практика Федеральной торговой комиссии 38
Авторитетные методики 39
Инциденты с ИИ 41
Культурные аспекты управления рисками машинного обучения 44
Организационная ответственность 44
Культура эффективного испытания 46
Разнообразные и опытные команды 46
Испытание продукта на себе 47
Двигайтесь быстро и ломайте устои 47

Книги для программистов: https://t.me/free_books_for_it

Организационные процессы управления рисками машинного обучения 48
Прогнозирование возможных отказов....48
Известные отказы 49
Неспособность предвидения....49
Процессы управления модельными рисками....50
Уровни риска....50
Документация модели 51
Мониторинг модели 52
Инвентарный учет моделей 53
Проверка системы и аудит процессов....53
Управление изменениями 54
За пределами управления модельными рисками....54
Аудит и оценка модели 55
Оценка воздействия 55
Подача апелляции, отмена решения и отказ от участия....56
Парное и двойное программирование....56
Ограничение полномочий при развертывании модели 57
Награда за обнаружение ошибок....57
Реагирование на инциденты с ИИ....58
Пример: взлет и падение iBuying от Zillow....60
Последствия....61
Какие уроки мы можем извлечь?....61
Ресурсы....64

Глава 2. Интерпретируемость и объяснимость....65
Машинное обучение....65
Важные принципы интерпретируемости и объяснимости 66
Объяснимые модели....71
Аддитивные модели....71
Регрессия со штрафом....71
Обобщенные аддитивные модели 75
GA2M и объяснимые бустерные машины....76
Деревья решений....76
Одиночные деревья решений....76
Ограниченные модели XGBoost....79
Экосистема объяснимых моделей машинного обучения 79
Апостериорное объяснение 82
Атрибуция и важность признаков....83
Локальные объяснения и атрибуция признаков....84
Глобальная значимость признака....91
Суррогатные модели....95
Суррогатные модели дерева решений....95
Линейные модели и локально интерпретируемые объяснения 97
Якоря и правила....99
Графическое представление характеристик модели....99
Частичная зависимость и индивидуальное условное ожидание 99
Накопленный локальный эффект....102
Профилирование кластеров....102
Трудноразрешимые проблемы апостериорного объяснения на практике 103

Книги для программистов: https://t.me/free_books_for_it

Сочетание объяснимых моделей и апостериорных объяснений 107
Пример: алгоритмическая корректировка оценок 109
Ресурсы 112

Глава 3. Отладка систем машинного обучения 113
Обучение 114
Воспроизводимость 115
Качество данных 116
Спецификация модели для вывода в реальных условиях 121
Бенчмарки и их альтернативы 121
Калибровка 121
Конструктная валидность 122
Предположения и ограничения 123
Функции потерь по умолчанию 123
Множественные сравнения 123
Будущее безопасного и надежного машинного обучения 124
Отладка модели 124
Тестирование программного обеспечения 125
Традиционная оценка модели 126
Распространенные ошибки ML 128
Смещение распределений 128
Эпистемическая неопределенность и разреженность данных 129
Нестабильность 130
Утечка 131
Петля обратной связи 132
Переобучение 132
Ложное обучение 133
Недостаточное обучение 134
Неполная идентификация 134
Анализ остатков 137
Анализ и визуализация остатков 137
Моделирование остатков 138
Локальный вклад в остатки 139
Анализ чувствительности 141
Эталонные модели 144
Исправление ошибок 145
Развертывание 148
Безопасность в предметной области 148
Мониторинг модели 150
Устаревание модели и дрейф концепта 150
Обнаружение и устранение дрейфа 151
Мониторинг нескольких ключевых показателей эффективности 152
Значения вне диапазона 152
Обнаружение аномалий и эталонные модели 152
Аварийные выключатели 153
Пример: смерть от автономного транспортного средства 154
Последствия 154
Неподготовленная правовая система 154
Какие уроки мы можем извлечь? 155
Ресурсы 156

Книги для программистов: https://t.me/free_books_for_it

Глава 4. Борьба с предвзятостью в машинном обучении.....157
Определения предвзятости в ISO и NIST.....159
Системная предвзятость.....160
Статистическая предвзятость.....160
Человеческая предвзятость и культура науки о данных.....160
Правовые понятия предвзятости ML в США.....162
Кто обычно страдает от предвзятости систем ML.....165
Ущерб от предвзятости, который испытывают люди.....167
Тестирование на предвзятость.....169
Проверка данных.....169
Традиционные подходы: тестирование эквивалентных результатов.....172
Тестирование статистической значимости.....174
Проверка практической значимости.....174
Новый взгляд: тестирование эквивалентности качества работы.....176
Чего нам ожидать: тесты для более широкой экосистемы машинного обучения.....178
Сводный план тестирования.....181
Смягчение предвзятости.....183
Технические факторы смягчения предвзятости.....184
Научный подход и план эксперимента.....184
Методы смягчения предвзятости.....186
Человеческий фактор в смягчении предвзятости.....190
Пример: вознаграждение за выявленные ошибки предвзятости.....193
Ресурсы.....196

Глава 5. Безопасность машинного обучения.....197
Основы безопасности.....199
Состязательное мышление.....199
Триада CIA.....200
Лучшие методы обеспечения безопасности для специалистов по данным.....201
Атаки на системы ML.....204
Атаки на целостность: манипулирование результатами машинного обучения.....204
Примеры состязательных атак.....205
Бэкдор-атаки.....205
Атаки с отравлением данных.....206
Атаки путем подмены личности и уклонения.....207
Атаки на объяснения машинного обучения.....208
Атаки на конфиденциальность: извлеченная информация.....209
Атаки извлечения и инверсии модели.....209
Атаки с выводом о членстве.....210
Общие проблемы безопасности ML.....211
Меры противодействия.....214
Отладка модели в целях безопасности.....214
Поиск состязательных примеров и анализ чувствительности.....214
Аудит на предмет инсайдерского отравления данных.....215
Тестирование на предвзятость.....215
Этический взлом: атаки с извлечением моделей.....215
Мониторинг модели в целях безопасности.....216
Технологии повышения конфиденциальности.....218
Федеративное обучение.....218
Дифференциальная приватность.....218

Книги для программистов: https://t.me/free_books_for_it

Робастное машинное обучение 221
Универсальные контрмеры 221
Пример: реальные атаки уклонения....224
Атаки уклонения 224
Какие уроки мы можем извлечь?....225
Ресурсы....226

ЧАСТЬ II. УПРАВЛЕНИЕ РИСКАМИ ИИ В ДЕЙСТВИИ....227

Глава 6. Объяснимые бустинговые машины и модели XGBoost....229
Уточнение определений и прозрачность машинного обучения....229
Аддитивность или взаимодействия 230
Переход к причинно-следственной связи с ограничениями....231
Частичная зависимость и индивидуальное условное ожидание 231
Значения Шепли 234
Документация модели....235
Семейство объяснимых моделей GAM 236
Elastic Net — штрафуемая GLM с альфа- и лямбда-поиском....236
Обобщенные аддитивные модели....239
GA2M и объяснимые бустинговые машины....244
XGBoost с ограничениями и апостериорными пояснениями 247
XGBoost с ограничениями и без ограничений....247
Объяснение поведения модели с помощью частичной зависимости и ICE....253
Суррогатные модели дерева решений как метод объяснения....255
Пояснения на основе значений Шепли 259
Проблемы со значениями Шепли 262
Более информированный выбор модели....266
Ресурсы....268

Глава 7. Объяснение классификатора изображений PyTorch....271
Как работает классификация рентгенограмм грудной клетки 272
Новый взгляд на объяснимые модели и методы апостериорного объяснения 273
Обзор объяснимых моделей....273
Методы окклюзии 274
Градиентные методы....274
Объяснимый ИИ для отладки моделей 275
Объяснимые модели....275
ProtoPNet и ее варианты 276
Другие объяснимые модели глубокого обучения 277
Обучение и объяснение классификатора изображений PyTorch 278
Обучающие данные....278
Решение проблемы дисбаланса наборов данных 279
Дополнение данных и обрезка изображений....280
Обучение модели....282
Оценка и метрики....284
Создание апостериорных объяснений с помощью Captum 284
Окклюзия 285
input*gradient 286
Интегрированные градиенты 287
Послойное распространение релевантности 289

Книги для программистов: https://t.me/free_books_for_it

Оценка пояснений модели 290
Надежность апостериорных объяснений 291
Заключение 295
Ресурсы 296

Глава 8. Выбор и отладка моделей XGBoost 299
Уточнение определений: отладка моделей ML 299
Выбор модели 300
Анализ чувствительности 300
Анализ остатков 302
Исправление 303
Выбор лучшей модели XGBoost 304
Анализ чувствительности XGBoost 309
Стресс-тестирование модели XGBoost 310
Методика стресс-тестирования 311
Изменение данных для моделирования условий рецессии 312
Поиск состязательных примеров 314
Анализ остатков для XGBoost 318
Анализ и визуализация остатков 319
Сегментный анализ ошибок 323
Моделирование остатков 325
Исправление выбранной модели 327
Чрезмерная важность признака *PAY_0* 328
Разные ошибки 330
Заключение 333
Ресурсы 334

Глава 9. Отладка классификатора изображений PyTorch 335
Уточнение понятий: отладка моделей DL 337
Отладка классификатора изображений PyTorch 341
Качество данных и утечки 341
Тестирование программного обеспечения для DL 343
Анализ чувствительности моделей DL 344
Тестирование на смещение домена и субпопуляции 345
Примеры состязательных атак 349
Возмущение вычислительных гиперпараметров 352
Устранение проблем 352
Исправление данных 353
Исправления программного обеспечения 355
Исправления чувствительности 359
Внесение шума 359
Другие способы улучшения стабильности 360
Заключение 363
Ресурсы 364

Глава 10. Тестирование и исправление предвзятости с помощью XGBoost 365
Уточнение понятий: управление предвзятостью в ML 366
Обучение модели 369

Книги для программистов: https://t.me/free_books_for_it

Оценка моделей на предмет предвзятости ... 373
Способы тестирования для групп ... 374
Тестирование качества прогнозов ... 376
Традиционное тестирование показателей исходов ... 381
Индивидуальная справедливость ... 384
Опосредованная предвзятость ... 388
Исправление предвзятости ... 389
Предварительная обработка ... 390
Внутренняя обработка ... 394
Постобработка ... 398
Выбор модели ... 399
Заключение ... 404
Ресурсы ... 406

Глава 11. Атака красной команды на XGBoost ... 407
Уточнение понятий ... 408
Триада CIA ... 408
Атаки ... 409
Контрмеры ... 411
Обучение модели ... 412
Атаки с участием красной команды ... 417
Атаки с извлечением моделей ... 417
Атаки с состязательными примерами ... 421
Атаки вывода о членстве ... 423
Отравление данных ... 425
Бэкдоры ... 427
Заключение ... 431
Ресурсы ... 432

ЧАСТЬ III. ЗАКЛЮЧЕНИЕ ... 435

Глава 12. Как добиться успеха в машинном обучении высокого риска ... 437
Кто в команде? ... 438
Наука или техника? ... 440
Метод анализа данных ... 440
Научный метод ... 442
Оценка опубликованных результатов и заявлений ... 443
Применяйте внешние стандарты ... 445
Смягчение рисков на основе здравого смысла ... 448
Заключение ... 452
Ресурсы ... 452

Предметный указатель ... 453

Об авторах ... 457

Об изображении на обложке ... 459

Книги для программистов: https://t.me/free_books_for_it

Отзывы о книге

Это практичная, содержательная и своевременная книга. Независимо от того, являетесь ли вы специалистом по данным, заинтересованным в лучшем понимании ваших моделей, менеджером, ответственным за обеспечение соответствия существующим стандартам, или руководителем, пытающимся улучшить контроль рисков в вашей организации, здесь вы найдете насыщенную и полезную информацию по этой непростой теме.

Агус Суджианто, исполнительный вице-президент, руководитель отдела моделирования корпоративных рисков, Wells-Fargo

Строго обязательно к прочтению! Эта книга представляет собой выигрышное сочетание передовой теории и практического опыта. Она открывает новый взгляд на проблему для всех, кто сталкивается со сложностями интерпретации, объяснимости и безопасности ИИ. Благодаря пояснениям экспертов модные словечки в мире искусственного интеллекта наполнятся для вас реальным смыслом. Не позволяйте конкурентам опередить вас — прочтите эту незаменимую книгу уже сегодня!

Матеуш Дымчик, инженер-программист по машинному обучению, Meta

Книга представляет собой комплексное и актуальное руководство для всех, кто занимается машинным обучением в области, где цена ошибки очень высока. Авторы проделали отличную работу, предоставив обзор нормативных аспектов, теории управления рисками, интерпретируемости и многих других тем, а также предоставив практические советы и примеры кода. Настоятельно рекомендую эту книгу всем, кто хочет избежать проблем при развертывании моделей машинного обучения.

Кристоф Мольнар, автор книги «Интерпретируемое машинное обучение»

Успешные и безопасные приложения машинного обучения должны быть основаны на принципах справедливости, контролируемости, прозрачности и этики независимо от отрасли применения. Книга закладывает фундамент знаний в области безопасного искусственного интеллекта и содержит ценные сведения, которые можно использовать на практике в различных ситуациях. Я настоятельно рекомендую эту книгу всем, кто занимается машинным обучением.

Навдип Гилл, менеджер по технологиям, H2O.ai

Книги для программистов: https://t.me/free_books_for_it

Предельно простое объяснение понятия ответственного ИИ.

Хариом Тацат, соавтор книги «Машинное обучение и наука о данных в финансах»

Это абсолютно необходимая книга, отвечающая растущему спросу на углубленный анализ прогнозных моделей. Книга очень практична и дает подробные рекомендации по изучению различных аспектов, таких как отладка модели, предвзятость, прозрачность и анализ объяснимости. Авторы делятся своим огромным опытом анализа различных классов моделей с помощью как табличных, так и графических данных.

Я рекомендую эту книгу всем, кто ориентирован на ответственную работу со сложными моделями, а не только в приложениях с высоким уровнем риска.

Пшемыслав Бичек, профессор Варшавского технологического университета

Вдумчивое и практическое руководство по ответственному использованию машинного обучения. Эта книга направлена на предотвращение несчастных случаев и вреда, причиненного искусственным интеллектом, еще до того, как они произойдут.

Харш Сингхал, старший директор по решениям в области ИИ, Financial Services, C3.ai

Эта книга выделяется своим уникальным подходом к устранению системных рисков в машинном обучении. Авторы подчеркивают исключительную важность устранения потенциальных рисков на пути к цели, что можно считать ключом к успеху машинного обучения как такового. Особенно важен вопрос правильного распределения ролей при принятии решений по машинному обучению. Книга послужит читателям ценным источником знаний при развертывании систем машинного обучения, к которым предъявляются требования высокой ответственности и устойчивости.

Лиз Греннан, ассоциированный партнер и руководитель направления по ответственным цифровым технологиям, McKinsey & Company

Эта книга содержит всесторонний обзор как социальных, так и технических подходов к приложениям ИИ с высоким уровнем риска и делится с практикующими специалистами описаниями полезных методов, позволяющих внедрить ключевые концепции ответственного ИИ в повседневную работу.

Тривени Ганди, доктор философии, руководитель отдела искусственного интеллекта, Dataiku

Раскрытие всего потенциала машинного обучения и искусственного интеллекта выходит за рамки простой точности моделей. Авторы углубляются в важнейшие, но часто упускаемые из виду аспекты объяснимых, свободных от предвзятости и

Книги для программистов: https://t.me/free_books_for_it

надежных моделей. Кроме того они делятся с читателем бесценной информацией о передовых методиках, позволяющих организациям обеспечить успех своих инициатив в области искусственного интеллекта. Поскольку технологии развиваются с невероятной скоростью, а нормативные акты не всегда успевают за ними, это своевременное и всеобъемлющее руководство послужит для практиков незаменимым источником знаний.

Бен Штайнер, Колумбийский университет

Модели машинного обучения очень сложны по своей природе, и их разработка таит в себе опасные подводные камни. Ошибка в этой области может разрушить репутацию многих людей и обойтись в миллионы, а то и миллиарды долларов. Книга содержит необходимые знания для любого специалиста по машинному обучению, который занимается проектированием, разработкой и внедрением надежных моделей машинного обучения, и помогает ему избежать неудач, которые случались со многими другими начинаниями в области машинного обучения за последние годы.

Сцилард Пафка, доктор философии, главный научный сотрудник Epoch

Сказать, что эта книга своевременна, — ничего не сказать. Людям, которые разрабатывают модели машинного обучения, жизненно необходима книга, помогающая избежать ошибок и заблуждений, возникающих в процессе создания моделей. Приятно отметить, что Патрик, Джеймс и Парул проделали огромную работу, сделав книгу легко читаемой и понятной. Эта книга должна стоять на книжной полке любого специалиста по машинному обучению.

Арик ЛаБарр, доктор философии, доцент кафедры аналитики

Это чрезвычайно своевременная книга. Специалистам в области науки о данных и искусственного интеллекта необходимо серьезно задуматься о реальном влиянии моделей машинного обучения на нашу жизнь. Книга открывает нам глаза на это явление. Она не только предоставляет достоверную техническую информацию, но и сплетает в единое полотно темы законодательства, безопасности, управления и этики. Настоятельно рекомендую приобрести ее в качестве настольного справочника.

Хорхе Сильва, доктор философии,
директор службы искусственного интеллекта и машинного обучения, SAS

Применение ИИ постоянно ширится и охватывает все новые аспекты нашей жизни. Поэтому важно обеспечить ответственную разработку приложений ИИ, особенно тех, которые имеют решающее значение для безопасности. Патрик Холл и его команда проделали фантастическую работу, сформулировав в этой книге ключевые аспекты и пути решения проблем разработки критически важных для безопасности приложений. Я настоятельно рекомендую эту книгу, особенно если вы участвуете

Книги для программистов: https://t.me/free_books_for_it

в создании приложений искусственного интеллекта, которые несут в себе значимые риски и требуют продуманной и ответственной разработки и тестирования!

Шри Кришнамурти, Квантовый университет

Если вы нуждаетесь в совете надежного консультанта, когда решаетесь использовать ИИ в своей организации, эта книга — именно то, что нужно. Авторы вложили в нее как знания, так и опыт, выбрав идеальное сочетание базового образования в области технологий и обзора распространенных ошибок, освещения нормативных и социальных проблем, тематических исследований и практических рекомендаций.

Бретт Вужек, доктор философии,
руководитель отдела управления продуктами искусственного интеллекта, SAS

Книги для программистов: https://t.me/free_books_for_it

Предисловие

Известный статистик Джордж Бокс однажды сказал: «Все модели ошибочны, но некоторые из них полезны». Признание этого факта составляет основу эффективного управления рисками. В мире, где машинное обучение все чаще автоматизирует важные решения, касающиеся нашей жизни, последствия ошибочных решений, рекомендованных моделями машинного обучения, могут быть катастрофическими. Крайне важно предпринимать целенаправленные и продуманные шаги для снижения риска и предотвращения непреднамеренного вреда.

После сокрушительного финансового кризиса 2008 г. регулирующие органы и финансовые учреждения осознали важность управления рисками компьютерного моделирования для обеспечения безопасности банков. Фактически возникла новая дисциплина — *управление модельными рисками* (model risk management, MRM). По мере распространения машинного обучения принципы MRM все чаще применяются для управления рисками. Примером такой эволюции служит система управления рисками искусственного интеллекта Национального института стандартов и технологий. Культура управления рисками моделирования подразумевает контроль процесса внедрения и применения ИИ на всех уровнях, от утверждения планов высшим руководством до операционной деятельности, и охватывает организационную структуру предприятия.

В этой книге авторы описывают продуманную методику применения машинного обучения для принятия важных решений. Убедительным доказательством важности этого вопроса служат задокументированные случаи неудачного применения моделей и новые нормативы, которые подчеркивают важность тщательного наблюдения за внедрением моделей ИИ как части корпоративной культуры. К сожалению, эти принципы до сих пор редко находят применение за пределами строго регулируемых отраслей, таких как банки. В книге рассматриваются важные темы, такие как прозрачность моделей, управление, безопасность, борьба с искажениями и многое другое.

Простого тестирования точности модели недостаточно, поскольку очень разные модели могут иметь одинаковую формальную точность в силу сочетания различных факторов. Модели также должны быть объяснимыми, безопасными и непредвзятыми. Это первая книга, в которой особое внимание уделяется интерпретируемым моделям, истории их развития, а также применению, особенно в тех случаях, когда модели влияют на конкретных людей, например в сфере потребительского кредитования. В отраслях, где стандарты и правила объяснимости особенно строги, общепринятый подход, когда нас просто ставят перед фактом решения, принятого ИИ, часто сталкивается со значительными проблемами.

Книги для программистов: https://t.me/free_books_for_it

Разработка надежных и безопасных систем машинного обучения также требует тщательной оценки слабых сторон модели. В этой книге описаны два подробных примера отладки модели, включая выявление недостатков модели посредством ошибок или остаточных срезов, оценку устойчивости модели при искажении входных данных, оценку надежности или неопределенности выходных данных модели и тестирование устойчивости модели при сдвиге распределения посредством стресс-тестирования. Это важнейшие аспекты разработки и внедрения машинного обучения в приложениях высокого риска.

Модели машинного обучения могут нанести непропорционально большой вред исторически маргинализированным группам людей, причем сделать это быстро и в больших масштабах за счет автоматизации. Предвзятые решения моделей оказывают пагубное воздействие на уязвимые группы, закрепляя социальное и экономическое неравенство. В этой книге читатель научится рассматривать вопрос справедливости модели с социотехнической точки зрения. Авторы также подробно описывают способы устранения предвзятости модели и дают практические советы по применению этих методов для различных вариантов административно-производственной вертикали.

Вам предстоит прочитать практичную, насыщенную знаниями и очень своевременную книгу. Представленная в ней разнообразная информация доступна каждому читателю, независимо от того, является ли он специалистом по данным, заинтересованным в лучшем понимании моделей, менеджером, ответственным за обеспечение соответствия стандартам, или руководителем, стремящимся усовершенствовать систему контроля рисков в своей организации.

Агус Суджианто, доктор философии, исполнительный вице-президент,
руководитель отдела рисков корпоративной модели, Wells Fargo

Книги для программистов: https://t.me/free_books_for_it

Введение

Сегодня *машинное обучение* (machine learning, ML) является наиболее коммерчески жизнеспособной отраслью *искусственного интеллекта* (ИИ). Системы ML широко применяются для принятия рискованных решений при приеме на работу, освобождении под залог, условно-досрочном освобождении, кредитовании, обеспечении безопасности и во многих других важных областях деятельности правительственных органов и коммерческих организаций. В корпоративной деятельности системы машинного обучения находят применение повсюду — от разработки продуктов, ориентированных на потребителя, до оценки сотрудников, автоматизации бэк-офиса и многого другого. На протяжении последнего десятилетия мы наблюдаем стремительное внедрение технологии машинного обучения во все аспекты нашей жизни. Но не всегда это история успеха. При желании можно привести много примеров, подтверждающих потенциальную опасность данной технологии как для операторов, ее использующих, так и для широкой публики.

Продукт, основанный на технологии ML, может потерпеть провал — как из-за случайных ошибок, так и из-за преднамеренного злоупотребления. По состоянию на 2023 г. зарегистрированы тысячи публичных сообщений об алгоритмической дискриминации, нарушениях конфиденциальности данных, нарушениях безопасности обучающих данных и других потенциально опасных инцидентах. Без устранения подобных рисков сложно говорить о преимуществах передовых технологий для организаций и потребителей. Устранение рисков ML в первую очередь зависит от специалистов-практиков. Хотя зарождающиеся отраслевые стандарты, о которых пойдет речь в этой книге, уже начинают обретать форму, в области ML по-прежнему отсутствует профессиональное лицензирование, не говоря уже о строгих предписаниях в области управления рисками. Это означает, что ответственность за результаты внедрения технологии — как хорошие, так и плохие — в настоящее время полностью возлагается на практиков. Наша книга призвана вооружить специалистов-практиков глубоким пониманием процессов управления рисками компьютерного моделирования и новыми способами применения популярных инструментов Python для обучения объяснимых моделей и их отладки на предмет надежности, безопасности, устранения предвзятости, безопасности и конфиденциальности.



Мы опираемся на определение ИИ из книги Стюарта Рассела и Питера Норвига «Искусственный интеллект: современный подход» (2020), а именно: *проектирование и создание интеллектуальных систем, которые получают сигналы из окружающей среды и предпринимают действия, влияющие на эту среду*. Для машин-

Книги для программистов: https://t.me/free_books_for_it

ного обучения мы используем общее определение, приписываемое — возможно, ошибочно — Артуру Сэмюэлю: *это область исследований, направленная на создание компьютеров, которые способны учиться без явного программирования* (около 1960 г.).

Кому адресована эта книга

Эта книга в основном ориентирована на инженеров ML начального и среднего уровня и специалистов по обработке данных, которые хотят узнать об ответственном использовании ML или управлении рисками ML. Примеры кода написаны на Python. Тем не менее эта книга предназначена не для каждого специалиста по данным и инженера, программирующего на Python. Эта книга для вас, если вы хотите изучить основы управления моделью и обеспечить базовый контроль рисков за счет правильно организованного рабочего процесса. Эта книга для вас, если ваша работа должна соответствовать определенным стандартам непредвзятости, прозрачности, конфиденциальности или безопасности. (Хотя мы не можем гарантировать соблюдение нормативов или предоставлять юридическую консультацию!) Эта книга для вас, если вы хотите обучать объяснимые модели, а также правильно их редактировать и отлаживать. Наконец, эта книга для вас, если ваша работа в области машинного обучения может привести к непредвиденным последствиям, связанным с нарушениями социальных норм, разглашением конфиденциальных данных, уязвимостями безопасности, или другим проблемам, вызванным автоматизированным принятием решений в целом, — и вы хотите этого избежать.

Конечно, эта книга может быть интересна и другим читателям. Если вы пришли в ML из таких областей, как физика, эконометрика или психометрия, эта книга научит вас сочетать новые методы ML с устоявшимся опытом в предметной области и понятиями достоверности или причинности. Представителям регулирующих органов или специалистам в области национальной политики она поможет составить представление о текущем состоянии технологий ML с целью выработки законов, правил или стандартов. Руководители отделов технических рисков или риск-менеджеры найдут в этой книге актуальный обзор новых подходов ML, ориентированных на применение в особо ответственных областях. Эксперты в области данных и инженеры по машинному обучению могут обнаружить, что она бросает вызов многим устоявшимся практикам науки о данных.

Что вы узнаете из этой книги

Наши читатели познакомятся как с традиционной моделью управления рисками, так и с современными понятиями в области компьютерной безопасности, такими как реагирование на инциденты, вознаграждение за ошибки и красные команды, и узнают, как применять проверенные на практике средства управления рисками к рабочим процессам и системам ML. В этой книге будет представлен ряд старых и новых объяснимых моделей, а также методов объяснения, которые делают системы машинного обучения еще более прозрачными. После изучения теоретических

Книги для программистов: https://t.me/free_books_for_it

основ прозрачных и объяснимых моделей, мы углубимся в тестирование моделей на предмет безопасности и надежности. Процесс тестирования становится намного проще, когда видно, как работает наша модель! Мы выйдем далеко за рамки измерения качества контрольных данных, чтобы изучить применение известных диагностических методов, таких как остаточный анализ, анализ чувствительности и сравнительный анализ, к новым типам моделей машинного обучения. Затем мы перейдем к структурированию моделей управления предвзятостью, тестированию на предвзятость и устранению предвзятости моделей с организационной и технической точки зрения. Наконец, мы обсудим безопасность конвейеров и API машинного обучения.



Проект Закона Европейского союза об искусственном интеллекте (EU AI Act) относит к категории высокого риска следующие приложения ML: биометрическая идентификация; управление критической инфраструктурой; образование; трудоустройство; базовые услуги, как государственные (например, государственная помощь), так и частные (например, кредитование); правоохранительная деятельность; иммиграционный и пограничный контроль; уголовное правосудие; и демократические процессы. Именно эти варианты использования машинного обучения мы имеем в виду, когда говорим о приложениях с высоким уровнем риска[1], и именно поэтому мы решили сосредоточить примеры кода в этой книге на компьютерном зрении и древовидных моделях для табличных данных.

Читателям также следует знать, что в этом первом издании книги мы сосредоточимся на наиболее устоявшихся методах машинного обучения для оценки и принятия решений. Мы не рассматриваем обучение без учителя, поисковые и рекомендательные системы, обучение с подкреплением и генеративный ИИ. На это есть несколько причин:

- упомянутые системы пока не достигли стадии повсеместного производственного применения;
- прежде чем переходить к более сложным технологиям, таким как обучение без учителя, обучение с подкреплением, рекомендательные или генеративные модели, вам необходимо освоить основы. В первом издании книги основное внимание уделяется базовым знаниям, которые позволят читателям в дальнейшем заняться более сложными проектами;
- управление рисками для этих систем изучено не так хорошо, как для традиционных моделей, обучаемых с учителем, на которых мы сосредоточимся в книге. Нужно прямо сказать (как мы часто поступаем в оставшейся части книги), что использование моделей, для которых режимы реагирования на сбои, меры по их предотвращению и средства контроля недостаточно проработаны, ведет к увеличению рисков.

Мы надеемся вернуться к этим темам в будущем и признаем, что сегодня они влияют на миллиарды людей — положительно или отрицательно. Мы также отмечаем, что, проявив немного творческого подхода и усердия, многие способы снижения рисков и структуры управления рисками, представленные в этой книге,

[1] Для удобства далее будем называть их приложениями высокого риска. — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

можно и нужно применять к моделям, обучаемым без учителя, поисковым и рекомендательным системам и генеративному ИИ.



Передовые системы генеративного искусственного интеллекта, такие как ChatGPT и GitHub Copilot, демонстрируют впечатляющее влияние машинного обучения на нашу жизнь. Разработчикам этих систем, похоже, удалось решить некоторые проблемы предвзятости, которые преследовали предыдущие поколения подобных систем. Однако они по-прежнему представляют опасность при работе с приложениями высокого риска. Если вам приходится работать с такими приложениями и у вас есть опасения, старайтесь придерживаться простых правил:

Избегайте слепого копирования текста из пользовательского интерфейса модели или вставки в него.

Отказ от бездумного использования сгенерированного контента и вставки нашего собственного контента непосредственно в интерфейс модели помогает уменьшить риски нарушения интеллектуальной собственности и конфиденциальности данных.

Тщательно проверяйте весь сгенерированный контент.

Несмотря на все усилия, модели периодически генерируют ошибочный, оскорбительный или доставляющий иные проблемы контент.

Избегайте самоуспокоенности в сфере автоматизации.

Как правило, системы ML лучше подходят для создания контента, чем для поддержки принятия решений. Будьте начеку, чтобы случайно не позволить им принимать решения за вас.

Соответствие системе управления рисками искусственного интеллекта NIST

Чтобы сделать книгу еще более полезной для тех, кто работает над приложениями высокого риска, мы отдельно отметим места, где предлагаемые в книге подходы соответствуют положениям недавно разработанной системы управления рисками ИИ (AI Risk Management Framework, AI RMF) Национального института стандартов и технологий (National Institute of Standards and Technology, NIST). Применение внешних стандартов является хорошо известной тактикой управления рисками, и NIST имеет невероятный опыт предоставления авторитетных технических рекомендаций. AI RMF состоит из множества компонентов, но два наиболее важных — это характеристики надежности ИИ и основные рекомендации RMF. Характеристики надежности определяют основные принципы управления рисками ИИ, а основные рекомендации RMF содержат рекомендации по внедрению средств контроля рисков. На протяжении всей книги мы будем использовать словарь терминов надежности ИИ, принятый в NIST: валидность, надежность, безопасность, защищенность, отказоустойчивость, прозрачность, подотчетность, объяснимость, интерпретируемость, отсутствие предвзятости и повышенная конфиденциальность. В начале каждой главы части I мы размещаем специальную таблицу с указанием того, как и где контент главы соотносится с конкретными аспектами NIST AI RMF: картирования, измерения, управления и контроля. Мы надеемся, что соответствие требованиям NIST AI RMF повысит удобство пользования книгой и сделает ее более эффективным инструментом управления рисками ИИ.

Книги для программистов: https://t.me/free_books_for_it



NIST не проверял, не одобрял и не рассматривал иным образом какое-либо содержание этой книги, включая любые претензии, относящиеся к AI RMF. Все трактовки содержания AI RMF представляют собой частное мнение авторов и никоим образом не отражают официальную позицию NIST или какие-либо официальные или неофициальные отношения между NIST и авторами или издателем.

Структура книги

Книга разделена на три части. В первой части обсуждаются вопросы практического применения ML с отсылками к теории, где это необходимо. Часть II содержит подробные примеры программирования на Python, иллюстрирующие темы части I как с точки зрения структурированных, так и неструктурированных данных. В части III представлены основанные на реальном опыте советы о том, как организовать успешную работу с приложениями высокого риска.

Часть I

Глава 1 начинается с глубокого погружения в действующие нормативные акты, обсуждения ответственности за качество продукта и тщательного рассмотрения традиционной модели управления рисками. Поскольку многие традиционные методики предполагают весьма неторопливый и профессиональный подход к моделированию — далекий от популярного сегодня идеала «двигайся быстро и разрушай устои»[2], — мы также обсудим, как включить в управление рисками передовые приемы компьютерной безопасности, такие как оперативное реагирование на инциденты.

В главе 2 представлена растущая экосистема объяснимых моделей. Мы наиболее подробно рассмотрим семейство *обобщенных аддитивных моделей* (generalized additive model, GAM), а также обсудим многие другие типы высококачественных и прозрачных оценивающих моделей. В главе 2 также рассмотрено множество различных методов апостериорного объяснения, но с учетом известных проблем, связанных с этой несколько переоцененной разновидностью технологии ответственного ML.

Глава 3 рассказывает о способах проверки модели, ориентированных на практическую ценность предположений моделей и их реальную надежность. Мы рассмотрим основы тестирования программного обеспечения, а также коснемся основных моментов в области отладки моделей.

В главе 4, прежде чем перейти к обсуждению технических методов измерения и исправления предвзятости, мы проанализируем *социотехнические* аспекты справедливости и предвзятости в сфере ML. Далее в этой главе более подробно рассматривается тестирование на предвзятость, включая тесты на несопоставимое влияние и дифференциальную достоверность. В главе 4 также рассматриваются как общепринятые, так и консервативные способы устранения предвзятости, а также более передовые приемы, включая двухцелевые и состязательные методы, предварительную, текущую и последующую обработку.

[2] Авторы цитируют корпоративный девиз Facebook (Meta): «Move fast and break things». — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

Глава 5 завершает часть I, рассказывая, как объединить системы машинного обучения в красную команду. Она начинается с основ компьютерной безопасности и переходит к обсуждению распространенных атак на модели ML, состязательного ML и надежного ML.

Каждая глава в части I завершается обсуждением примера из реальной жизни, касающегося таких тем, как крах алгоритма iBuying компании Zillow, скандал государственного уровня в Великобритании, фатальная авария беспилотного Uber, первое вознаграждение за ошибки в Twitter и реальные атаки на модели ML. В каждой главе также указано соответствие между содержанием и NIST AI RMF.

Часть II

Часть II развивает идеи части I в нескольких подробных главах с примерами кода. В главе 6 на примере приложений потребительского кредитования рассматриваются *объяснимые бустинговые машины* (explainable boosting machines, EBM), XGBoost и объяснимые методы ИИ.

В главе 7 методы апостериорного объяснения применяются к классификатору изображений PyTorch.

В главе 8 мы займемся повышением качества модели потребительского кредитования, а в главе 9 сделаем то же самое с классификатором изображений.

Глава 10 содержит подробные примеры, относящиеся к тестированию и устранению систематических ошибок, а в главе 11 представлены примеры атак на ML и мер противодействия атакам для древовидных моделей.

Часть III

В главе 12 мы завершаем книгу более универсальными советами о том, как добиться успеха в приложениях ML высокого риска. Это не значит двигаться быстро и ломать устои. В некоторых приложениях с низким уровнем риска действительно бывает целесообразно применять быстрый и «грязный» подход. Но по мере того, как ML находит более широкое применение в приложениях высокого риска, негативные последствия, вызванные несовершенством моделей, становятся намного серьезнее. Глава 12 завершает книгу практическими советами по применению ML в приложениях высокого риска.

Мы надеемся, что первое издание этой книги составит достойную альтернативу непрозрачным и скомканным описаниям рабочих процессов, которые распространены сегодня в машинном обучении. Наша книга обобщает наборы терминов, идей, инструментов и методов, которые позволят практикам более осознанно выполнять свою очень важную работу.

Примеры наборов данных

В этой книге для объяснения методов, демонстрации практических подходов и анализа результатов применяются два стандартных набора данных. Эти примеры наборов данных не предназначены специально для обучения моделей в приложениях

Книги для программистов: https://t.me/free_books_for_it

высокого риска, но они хорошо известны и легкодоступны. Их недостатки также позволяют нам указать на различные ошибки в данных, моделировании и интерпретации. Мы будем неоднократно обращаться к этим наборам данных в последующих главах, поэтому обязательно ознакомьтесь с ними, прежде чем продолжить чтение книги.

Кредитные данные Тайваня

Для глав со структурированными данными — это главы 6, 8, 10 и 11 — мы использовали слегка модифицированную версию кредитных данных Тайваня, доступных в репозитории машинного обучения Калифорнийского университета в Ирвайне или Kaggle. Данные о кредитных картах содержат демографическую и платежную информацию о держателях кредитных карт на Тайване в 2005 г. В целом назначение этого набора данных — предоставить статусы прошлых платежей (`PAY_*`), суммы прошлых платежей (`PAY_AMT*`) и суммы счетов (`BILL_AMT*`) в качестве входных данных для прогнозирования того, внесет ли клиент свой следующий платеж (`DELINQ_NEXT = 0`). Суммы указаны в тайваньских долларах. Мы добавили в этот набор данных смоделированные маркеры `SEX` и `RACE`, чтобы проиллюстрировать подходы к выявлению предвзятости модели и исправлению ошибок. Мы используем платежную информацию в качестве входных данных и, следуя передовым методам устранения предвзятости в системах ML, не используем демографическую информацию в качестве входных данных для модели. Полный перечень обозначений данных представлен в табл. П.1.

Таблица B.1. Полный перечень обозначений в наборе данных о кредитовании

Название	Роль в модели	Числовой тип	Описание
`ID`	Индекс	Целое	Уникальный идентификатор строки
`LIMIT_BAL`	Входные данные	С плавающей запятой	Сумма ранее полученного кредита
`SEX`	Демографическая информация	Целое	1 = мужчина; 2 = женщина
`RACE`	Демографическая информация	Целое	1 = латиноамериканец; 2 = черный; 3 = белый; 4 = другое
`EDUCATION`	Демографическая информация	Целое	1 = аспирантура; 2 = университет; 3 = средняя школа; 4 = другое
`MARRIAGE`	Демографическая информация	Целое	1 = в браке; 2 = одинокий; 3 = другое
`AGE`	Демографическая информация	Целое	Возраст, лет
`PAY_0`, `PAY_2`–`PAY_6`	Входные данные	Целое	История последних платежей; `PAY_0` = статус платежа в сентябре 2005; `PAY_2` = статус платежа в августе 2005; ...; `PAY_6` = статус платежа в апреле 2005. Шкала значений статуса платежа: –1 = вовремя; 1 = просрочка один месяц; 2 = просрочка два месяца; ...; 8 = просрочка восемь месяцев; 9 = просрочка девять и более месяцев

Книги для программистов: https://t.me/free_books_for_it

Таблица B.1 *(окончание)*

Название	Роль в модели	Числовой тип	Описание
BILL_AMT1–BILL_AMT6	Входные данные	С плавающей запятой	Сумма оплаты по счетам; BILL_AMT1 = сумма платежа в сентябре 2005; BILL_AMT2 = сумма платежа в августе 2005; ...; BILL_AMT6 = сумма платежа в апреле 2005
PAY_AMT1–PAY_AMT6	Входные данные	С плавающей запятой	Сумма предыдущих платежей по кредиту; PAY_AMT1 = сумма платежа в сентябре 2005; PAY_AMT2 = сумма платежа в августе 2005; ...; PAY_AMT6 = сумма платежа в апреле 2005
DELINQ_NEXT	Целевое значение (выходные данные)	Целое	Прогноз, будет ли просрочен следующий платеж; 1 = просрочен; 0 = вовремя

Как читатели увидят в следующих главах, этому набору данных присущи некоторые патологические недостатки. Он слишком мал для обучения применимых на практике моделей машинного обучения высокой емкости[3], и почти весь сигнал, на основе которого модель вырабатывает вывод DELINQ_NEXT, закодирован в PAY_0. По ходу книги мы попытаемся решить эти проблемы и выявить другие.

Данные рентгенографии грудной клетки Kaggle

В примерах, посвященных глубокому обучению — главы 6 и 9, — мы будем использовать набор данных рентгенографии грудной клетки Kaggle Chest X-ray. Этот набор состоит примерно из 5800 изображений двух классов: легкие с признаками пневмонии и здоровые легкие. Принадлежность к классам была определена экспертами в предметной области. Изображения представляют собой обезличенные рентгеновские снимки грудной клетки, сделанные во время плановых посещений Медицинского центра для женщин и детей в Гуанчжоу, Китай. На рис. П.1 показан пример рентгеновского снимка легких человека, больного пневмонией.

Основные проблемы, которые несет в себе этот набор данных, — небольшой размер (даже для задач переноса обучения), несовпадение изображений в наборе данных, визуальные артефакты, которые могут привести к ошибкам обучения, а также необходимость привлечения эксперта в предметной области для проверки результатов моделирования. Как и в случае с кредитными данными Тайваня, мы проанализируем эти проблемы и выявим новые в последующих главах этой книги.

[3] *Емкость* (capacity) модели связана с количеством коэффициентов и слоев и определяет склонность модели к переобучению или недообучению. Емкость описывает способность модели к аппроксимации широкого спектра функций. При маленькой емкости модель слишком простая и склонна к недообучению, при высокой — слишком сложная и склонна к переобучению. Объем обучающих данных должен соответствовать емкости модели. — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

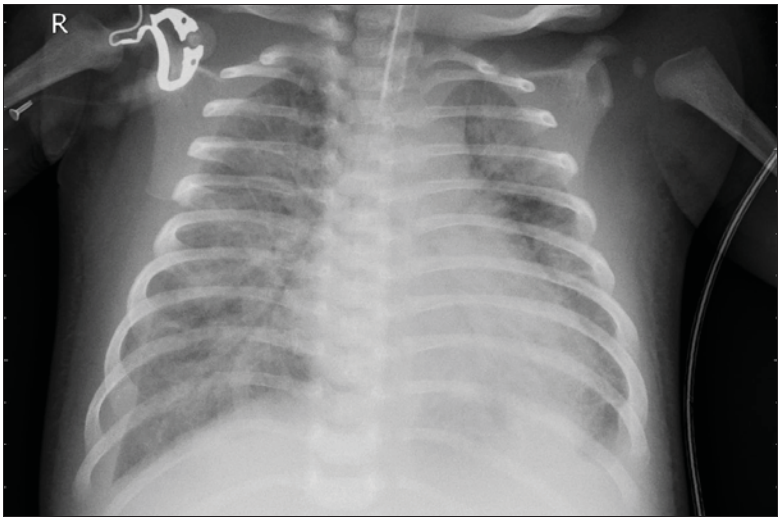


Рис. П.1. Пример рентгеновского снимка легких человека, больного пневмонией (набор данных Kaggle Chest X-ray)

Условные обозначения, принятые в этой книге

- *Курсивом* обозначены новые термины, понятия и важные утверждения.
- `Моноширинный шрифт` применяется для листингов программ, обозначения в тексте абзаца элементов кода программы, таких как переменные или имена функций, баз данных, типов данных, переменных окружения и ключевых слов.



Этот элемент обозначает обычные примечания или рекомендации.



Этот элемент обозначает предупреждение или предостережение.

Дополнительный материал (примеры кода, упражнения и т. д.) доступен для загрузки по адресу **https://oreil.ly/machine-learning-high-risk-apps-code**.

Архив с иллюстрациями к книге

Для полноценного изучения материала иллюстрации этой книги наиболее наглядны в цветном, а не черно-белом исполнении. На страницах данного печатного издания иллюстрации монохромные, а архив с их цветными версиями выложен на сайте издательства. Его можно скачать по ссылке

https://files.alistbooks.kz/3060_Hall_ML.zip.

Книги для программистов: https://t.me/free_books_for_it

Благодарности

Авторы выражают благодарность редакторам и производственному персоналу издательства O'Reilly, особенно Мишель Кронин и Грегори Хайману; нашему литературному редактору Лиз Уилер; рецензентам издательства Ребекке Новак и Николь Баттерфилд. Спасибо также техническим рецензентам Навдипу Гиллу, Коллину Старкуэзеру, Хариому Татстату и Лауре Узкатеги.

Патрик Холл

Спасибо Лизе и Дилану за любовь и поддержку на протяжении всего процесса работы над книгой. Спасибо также моим коллегам из Института перспективной аналитики, Института SAS, Школы бизнеса Джорджа Вашингтона, H2O.ai, SolasAI, базы данных инцидентов с ИИ, NIST и BNH.AI.

Джеймс Кертис

Моей жене Линдси, чья неугасающая любовь составляет смысл моей жизни. Мои малыши, Исаак и Мика, не особо помогли в написании этой книги, но я все равно обожаю их. Наконец, я благодарю моих бывших коллег из SolasAI, особенно Ника и Криса, за многие часы содержательных дискуссий.

Парул Пандей

Я невероятно благодарна за любовь и поддержку моего мужа Маноджа и моего сына Агрима во время написания этой книги. Они оба не только вдохновили меня взяться за эту грандиозную задачу, но и относились с глубоким пониманием, когда я проводила часы в кабинете, работая над книгой.

Книги для программистов: https://t.me/free_books_for_it

ЧАСТЬ I

Теория и практика управления рисками в приложениях ИИ

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 1

Современное управление рисками в ML

Создание наилучшей системы ML начинается с культурных компетенций и бизнес-процессов. В этой главе рассмотрены многочисленные культурные и процедурные подходы, которые можно применить для улучшения качества моделей ML и защиты корпоративного ИИ от реальных проблем с безопасностью и надежностью. В ней также представлен наглядный пример того, что происходит, когда система ML работает без надлежащего контроля со стороны человека. Основная цель методик, обсуждаемых в этой главе, — создание более совершенных систем машинного обучения. Это может означать улучшение точности модели во время испытаний на тестовых данных in silico. Но на самом деле это означает создание моделей, которые будут работать так, как ожидалось, после внедрения in vivo, чтобы мы не теряли деньги и не причиняли вреда ни посторонним людям, ни себе.



In vivo в переводе с латыни означает «внутри живого». Иногда мы будем использовать этот термин для обозначения чего-то более близкого к «взаимодействию с живыми», например, как модели ML работают в реальном мире при взаимодействии с пользователями-людьми. *In silico* означает «посредством компьютерного моделирования или компьютерной симуляции» (дословно «в кремнии»), и мы будем использовать этот термин для описания тестирования, которое специалисты по данным часто выполняют в своих средах разработки перед развертыванием моделей ML в производстве.

Глава начинается с обсуждения текущей нормативно-правовой базы в области ML и новых рекомендаций на основе передового опыта, призванных проинформировать разработчиков систем об их фундаментальных обязательствах, когда дело касается безопасности и надежности. В этой части главы также показано, как материал книги соотносится с системой управления рисками искусственного интеллекта (RMF) (*https://oreil.ly/Or940*) Национального института стандартов и технологий (NIST). Поскольку те, кто не изучает историю, обречены на ее повторение, далее в этой главе мы рассмотрим инциденты с ИИ и поясним, почему понимание инцидентов с ИИ важно для обеспечения надлежащей безопасности систем ML. Поскольку многие проблемы безопасности ML выходят за рамки технических спецификаций, в этой главе будут рассмотрены такие аспекты, как *управление рисками модели* (model risk management, MRM), рекомендации по безопасности информационных технологий (ИТ) и подходы к безопасности из других областей. Опираясь на этот комплексный анализ, мы предложим различные идеи по совершенствованию культуры и процессов безопасности ML в организациях. Глава завершится примером из реальной жизни, иллюстрирующим значение культуры безопасности и юридические последствия инцидентов с искусственным интеллектом.

Книги для программистов: https://t.me/free_books_for_it

Ни один из подходов к управлению рисками, обсуждаемых в этой главе, не является панацеей. Если вы хотите успешно управлять рисками, вам придется выбирать из широкого спектра доступных инструментов те, которые лучше всего подходят для вашей организации. Более крупные организации обычно имеют возможность лучше управлять рисками, чем мелкие. Читатели, работающие в крупных организациях, обычно могут реализовать множество элементов управления рисками в различных отделах, подразделениях и даже на уровне отдельных сотрудников. Читателям, не располагающим значительными ресурсами, придется выбирать тактику управления рисками исходя из своих возможностей. В конце концов, управление технологическими рисками во многом сводится к поведению человека. Какие бы средства управления рисками ни применяла организация, они должны сочетаться с надежным руководством и четкими правилами для людей, которые создают и поддерживают системы машинного обучения.

Обзор нормативно-правовой базы

Отсутствие регулирования в области ML — это миф. Системы ML могут нарушать закон и нарушают его. Игнорирование правового контекста — одна из самых рискованных вещей, которые организация может сделать в отношении систем ML. Более того, нормативно-правовая база в сфере ML сложна и быстро меняется. Этот раздел содержит краткий обзор важных законов и постановлений, предназначенный для повышения вашей осведомленности в данной области. Мы начнем с рассмотрения проекта Закона ЕС об искусственном интеллекте. Затем мы обсудим множество федеральных законов и постановлений США, касающихся ML, законов штатов и муниципалитетов США о конфиденциальности данных и искусственном интеллекте, а также основы ответственности за качество продукции, а закончим раздел кратким обзором недавних решений Федеральной торговой комиссии (Federal Trade Commission, FTC).



Авторы не являются юристами, и ничто в этой книге нельзя рассматривать как юридическую рекомендацию. Пересечение права и искусственного интеллекта — невероятно сложная тема, с которой ученые, работающие с данными, и инженеры по машинному обучению не способны справиться в одиночку. У вас могут возникнуть юридические вопросы по поводу систем ML, над которыми вы работаете. В таком случае обратитесь за юридической консультацией к специалистам.

Закон ЕС об искусственном интеллекте

ЕС предложил весьма радикальные правила применения искусственного интеллекта, которые, как ожидается, будут приняты в 2023 г.[1] Ожидается, что Закон ЕС об искусственном интеллекте (EU AI Act, AIA) (*https://oreil.ly/x5dLT*) запретит определенные виды использования искусственного интеллекта, такие как искажение чело-

[1] Полная редакция Закона была опубликована 21 января 2024 г. на сайте **https://artificialintelligenceact.eu/**, но к моменту подготовки русского перевода книги Закон так и не был окончательно принят, и в него продолжали вносить поправки. — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

веческого поведения, оценка социального кредита и биометрическое наблюдение в реальном времени. К вариантам применения ИИ, сопряженным с высоким риском, AIA относит уголовное правосудие, обработку биометрической идентификации, проверку при трудоустройстве, управление критически важной инфраструктурой, правоохранительную деятельность, оказание жизненно важных услуг и многие другие. Применение ИИ в этих областях уже сейчас требует наличия объемной разрешительной документации и системы управления рисками. Остальные применения будут отнесены к области ограниченного или низкого риска, что влечет меньшие обязательства по соблюдению требований AIA для их поставщиков и операторов. Подобно тому как Общий регламент ЕС по защите данных (General Data Protection Regulation, GDPR) изменил способ обработки данных компаниями в США и во всем мире, правила ЕС по применению ИИ призваны оказать огромное влияние на развертывание искусственного интеллекта в США и других странах мира. Независимо от того, работаете вы в ЕС или нет, вам пора начинать знакомиться с AIA. Один из лучших способов — прочитать Приложения, особенно Приложения 1 и 3–8, которые определяют термины, базовую документацию и требования соответствия.

Федеральные законы и правила США

Поскольку мы десятилетиями применяем те или иные алгоритмы в деятельности государственных органов и для управления экономикой, многие федеральные законы и постановления США уже частично охватывают ИИ и ML. Эти нормы, как правило, сосредоточены на предотвращении социальной дискриминации с помощью алгоритмов, но также затрагивают прозрачность, конфиденциальность и другие аспекты работы алгоритмов. Законы о гражданских правах 1964 и 1991 гг., Закон об американцах с ограниченными возможностями (Americans with Disabilities Act, ADA), Закон о равных кредитных возможностях (Equal Credit Opportunity Act, ECOA), Закон о справедливой кредитной отчетности (Fair Credit Reporting Act, FCRA) и Закон о справедливом обеспечении жильем (Fair Housing Act, FHA) входят в число федеральных законов, которые пытаются предотвратить алгоритмическую дискриминацию в таких областях, как занятость, кредитование и жилье. ECOA и FCRA, наряду с их более детальной реализацией в Положении B, пытаются повысить прозрачность кредитования, основанного на ML, и гарантировать права на пояснения для потребителей кредита. Ожидается, что в случае отклонения заявки на кредит кредиторы укажут причины отказа, т. е. значимые претензии, и опишут особенности модели ML, которые послужили причиной такого решения. Если предоставленные обоснования или данные неверны, потребители должны иметь возможность обжаловать решение.

Методика MRM, частично описанная в руководстве Федеральной резервной системы SR 11-7, является частью проверок крупных банков США со стороны регулятора и определяет требования к организационным, культурным и техническим процессам, от которых зависит надежная работа ML в критически важных финансовых приложениях. Бóльшая часть главы 1 основана на методиках MRM, поскольку это наиболее проверенная на практике система управления рисками в области ML. Та-

Книги для программистов: https://t.me/free_books_for_it

кие законы, как Закон о переносимости и подотчетности медицинского страхования 1996 г. (Health Insurance Portability and Accountability Act, HIPAA) и Закон о правах семьи на образование и конфиденциальность (Family Educational Rights and Privacy Act, FERPA), устанавливают серьезные требования к конфиденциальности данных в сфере здравоохранения и высшего образования. Как и в случае GDPR, взаимодействие HIPAA и FERPA с отраслью ML до конца не определено, и дискуссии по этому поводу не утихают до сих пор. Мы упомянули здесь далеко не все законы США, которые могут повлиять на использование ML, но надеемся, что этот краткий список дает представление о том, какие вопросы федеральное правительство США считает достаточно важными для регулирования.

Государственные и муниципальные законы

В штатах и городах США также приняты локальные законы и правила, касающиеся ИИ и ML. Первоначально предполагалось, что закон № 144 города Нью-Йорка, требующий проведения проверок на предвзятость автоматизированных инструментов принятия решений о приеме на работу, вступит в силу в январе 2023 г. В соответствии с этим законом каждый крупный работодатель в Нью-Йорке должен будет проводить проверку на предвзятость автоматизированного программного обеспечения для трудоустройства и опубликовать результаты на своем сайте. Предлагаемый Вашингтоном Закон о предотвращении дискриминации с помощью алгоритмов воспроизводит положения федерального закона в отношении прозрачности алгоритмов и отсутствия дискриминации, но охватывает намного более широкий спектр приложений, и адресован компаниям, которые работают в округе Колумбия или используют данные граждан округа Колумбия.

Многие штаты также приняли свои собственные законы о конфиденциальности данных. В отличие от старых федеральных законов HIPAA и FERPA, эти законы штатов о конфиденциальности данных часто намеренно разрабатывали для частичного регулирования использования искусственного интеллекта и машинного обучения. Такие штаты, как Калифорния, Колорадо, Вирджиния и другие, приняли законы о конфиденциальности данных, в которых упоминается повышенная прозрачность и меры по исключению предвзятости для автоматизированных систем принятия решений. Некоторые штаты также поставили под контроль биометрические данные или социальные сети. Например, Закон штата Иллинойс о конфиденциальности биометрической информации (Biometric Information Privacy Act, BIPA) запрещает многие способы использования биометрических данных, а регулирующие органы Иллинойса уже начали раздавать предписания нарушителям. Отсутствие федерального закона о конфиденциальности данных или регулировании работы ИИ в сочетании с целым зоопарком правительственных и локальных нормативных актов существенно осложняет ситуацию с соблюдением требований к ИИ и ML. Использование ML где-то регулируется, а где-то нет, или регулируется частично в зависимости от конкретного применения, отрасли и географического региона.

Книги для программистов: https://t.me/free_books_for_it

Изначальная ответственность за качество продукта

По аналогии с производителями потребительских товаров ученые, работающие с данными, и инженеры по машинному обучению несут изначальную ответственность за создание безопасных систем. В недавнем докладе Брукингского института, озаглавленном «Закон об ответственности за качество продукции как способ борьбы с вредом, причиняемым ИИ» (*https://oreil.ly/2K_R6*) сказано следующее: «Производители обязаны выпускать продукты, которые будут безопасными при использовании разумно предсказуемыми способами. Если система искусственного интеллекта используется предсказуемым образом и при этом становится источником вреда, истец может утверждать, что производитель проявил халатность, не осознав возможность такого результата». Как и производители автомобилей или электроинструментов, производители систем машинного обучения подчиняются широким правовым нормам в отношении безопасности продукции. Безопасности продукции было посвящено огромное количество юридических и экономических исследований, но в этой главе мы уделяем основное внимание одному из первых и самых простых определений небрежности производителя: *правилу Хэнда*. Названное в честь судьи Лернеда Хэнда[2] и опубликованное в 1947 г., это правило может служить производителям продуктов ML надежной отправной точкой для размышлений о небрежности и комплексной проверке. Правило Хэнда гласит, что производитель продукта берет на себя бремя заботы о качестве и что ресурсы, затраченные на эту заботу, всегда должны превышать стоимость вероятного инцидента, связанного с продуктом. В математической форме это правило можно записать так:

$$Забота \geq Риск = (Вероятность\ убытка) \cdot (Размер\ убытка).$$

Проще говоря, ожидается, что организации будут проявлять заботу о качестве, т. е. расходовать время, деньги или иные ресурсы на уровне, соизмеримом с затратами на устранение потенциально возможных инцидентов. В противном случае может наступить ответственность за небрежность. На рис. 1.1 *Издержки* — это параболически возрастающая линия, а риск, или *Вероятность*, умноженная на *Убыток*, — это параболически убывающая линия. Хотя эти линии не связаны с конкретными единицами измерения, их параболическая форма призвана отразить проблему последней мили в устранении всех рисков системы ML и показывает, что применение дополнительных мер, превышающих разумный порог, приводит к уменьшению отдачи от снижения риска.



Достаточно стандартное определение *риска* технологического инцидента — это предполагаемая вероятность инцидента, умноженная на его предполагаемую стоимость. В более широком смысле Международная организация по стандартизации (ISO) определяет риск в контексте управления рисками предприятия как «влияние неопределенности на цели».

Хотя величины в правиле Хэнда, вероятно, не поддаются точному вычислению, при разработке системы машинного обучения важно учитывать ключевые понятия

[2] Биллингс Лернед Хэнд — американский судья и философ права. Это самый цитируемый судья в истории американского правосудия, на решения которого ссылаются чаще всех. — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

небрежности и ответственности. Если организация применяет систему ML в условиях, когда велика вероятность инцидента, если велики денежные или другие потери, связанные с инцидентом в системе, и, тем более, если велики оба показателя, организации необходимо срочно потратить дополнительные ресурсы на обеспечение безопасности этой системы. Более того, организация должна по возможности документально доказать, что ее возможность покрытия рисков превышает расчетную вероятность отказа, умноженную на расчетный убыток.

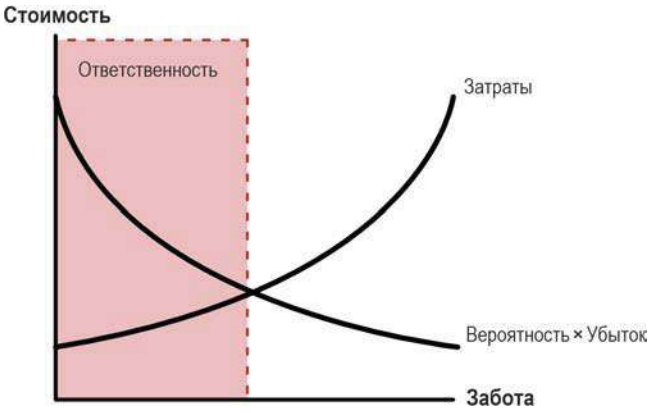


Рис. 1.1. Правило Хэнда (график адаптирован из *Economic Analysis of Alternative Standards of Liability in Accident Law: https://oreil.ly/9_u8H*)

Правоприменительная практика Федеральной торговой комиссии

Знаете, как вы можете неожиданно попасть в беду? Если вы работаете в регулируемой отрасли, то наверняка знаете требования своих регуляторов. Но если вы не знаете, регулируется ли ваша деятельность или от кого последует наказание, если вы пересечете правовую или нормативную красную линию, то, вероятно, вам следует больше всего опасаться Федеральной торговой комиссии США.

Федеральная торговая комиссия в целом сосредоточена на недобросовестной, вводящей в заблуждение или хищнической торговой деятельности. За последние три года она нашла причину для запрета как минимум трех известных алгоритмов ML. С помощью своего нового инструмента правоприменения — *алгоритмического изъятия* (algorithmic disgorgement) — Федеральная торговая комиссия имеет возможность удалять алгоритмы и данные и, как правило, запрещать получение будущих доходов от алгоритма-нарушителя. Cambridge Analytica (*https://oreil.ly/cM3V8*) была первой фирмой, подвергшейся такому наказанию за нечестные методы сбора данных, связанных с выборами 2016 г. С аналогичным наказанием также столкнулись Everalbum (*https://oreil.ly/05SO5*) и Weight Watchers (*https://oreil.ly/PMOq0*).

Надо отметить, что Федеральная торговая комиссия заранее сообщала о своем намерении обеспечить соблюдение федеральных законов, касающихся ИИ и ML. Члены Федеральной торговой комиссии написали длинные трактаты об алгоритмах и экономической справедливости (*https://oreil.ly/v8Z4y*). Они также опубликовали

Книги для программистов: https://t.me/free_books_for_it

как минимум два блога, в которых содержатся обобщенные рекомендации для компаний, которые хотели бы избежать неприятных последствий. В этих блогах освещается ряд конкретных шагов, которые организациям следует предпринять. Например, в разделе «Использование искусственного интеллекта и алгоритмов» Федеральная торговая комиссия ясно дает понять, что потребителей не следует вводить в заблуждение, заставляя их взаимодействовать с системой ML, которая выдает себя за человека. Подотчетность — еще одна важная тема в разделах «Использование искусственного интеллекта и алгоритмов», «Стремление к истине, справедливости и равноправию при использовании ИИ вашей компанией» и других тематических публикациях. В статье «Стремление к истине, справедливости и равноправию при использовании ИИ вашей компанией» Федеральная торговая комиссия заявляет: «**Позаботьтесь о собственной ответственности — или будьте готовы к тому, что Федеральная торговая комиссия сделает это за вас**» (выделено автором статьи). Этот чрезвычайно прямолинейный язык необычен для регулятора. В разделе «Использование искусственного интеллекта и алгоритмов» Федеральная торговая комиссия предлагает: «Подумайте, как вы обеспечиваете собственную ответственность и не лучше ли будет прибегнуть к независимой экспертизе, чтобы сделать шаг назад и провести ревизию использования вашего ИИ». В следующем разделе представлены некоторые из новых независимых стандартов, которые могут быть полезны для повышения контроля над нашей деятельностью, улучшения продуктов и снижения потенциальной юридической ответственности.

Авторитетные методики

Сегодня в науке о данных по большей части отсутствуют профессиональные стандарты и лицензирование, но на горизонте появляются первые авторитетные рекомендации. ISO начинает разработку технических стандартов для ИИ (*https://oreil.ly/8ZeJQ*). Обеспечение соответствия наших моделей стандартам ISO может стать одним из способов применения независимого стандарта в практике ML. Но пока эти стандарты не готовы, наиболее важным сборником нормативов, за которым стоит следить (особенно американским специалистам) остается проект NIST AI RMF.

Первая версия AI RMF была выпущена в январе 2023 г. В этом сборнике документов предложены общие характеристики надежности систем ИИ: достоверность, надежность, безопасность, защищенность, отказоустойчивость, прозрачность, подотчетность, объяснимость, интерпретируемость, управление предвзятостью и повышенная конфиденциальность. Также в нем представлены практические рекомендации по четырем организационным функциям для достижения надежности — *картированию* (map), *измерению* (measure), *управлению* (management) и *руководству* (government)[3]. Документ по функциям картирования, измерения, управления и руководства подразделяется на более подробные категории и подкатегории.

[3] Между понятиями *управление* (management) и *руководство* (government) есть принципиальная разница. Руководство направлено на создание организационной структуры и среды предприятия (стратегическая деятельность), в которой наиболее эффективно функционирует управление (операционная деятельность). — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

Ознакомиться с этими категориями можно в последней версии сборника RMF (*https://oreil.ly/kxq-G*) или AI RMF (*https://oreil.ly/dn4xs*), в котором представлены еще более подробные предложения.



Система управления рисками искусственного интеллекта NIST — это добровольный инструмент повышения надежности систем искусственного интеллекта и машинного обучения. AI RMF не является регулирующим нормативным актом, а NIST не является регулятором.

Чтобы полнее следовать нашим собственным советам, а также рекомендациям регулирующих органов и издателей авторитетных руководств и сделать эту книгу более полезной, мы решили показать, в какой степени содержание каждой главы в части I соответствует AI RMF. Ниже мы представляем читателям вставку (*Схема соответствий NIST AI RMF*) в виде таблицы, в которой отмечено соответствие между разделами глав и подкатегориями AI RMF. Ее назначение в том, чтобы читатели могли быстро определить, как использование подходов, обсуждаемых в каждой главе, поможет им придерживаться положений AI RMF. Поскольку рекомендации по подкатегориям в некоторых случаях могут показаться специалистам по ML чересчур абстрактными, мы используем более ориентированный на практику язык, соответствующий категориям RMF; это будет полезно при переносе требований RMF на развертывания ML in vivo. В текущей вставке показано как, по нашему мнению, глава 1 соответствует AI RMF. Аналогичные вставки вы найдете в начале каждой главы части I.

Схема соответствий NIST AI RMF

Раздел главы	Подкатегория NIST AI RMF
Обзор нормативно-правовой базы	GOVERN 1.1, GOVERN 1.2, GOVERN 2.2, GOVERN 4.1, GOVERN 5.2, MAP 1.1, MAP 1.2, MAP 3.3, MAP 4.1, MEASURE 1.1, MEASURE 2.8, MEASURE 2.11
Авторитетные методики	GOVERN 1.2, GOVERN 5.2, MAP 1.1, MAP 2.3
Инциденты с ИИ	GOVERN 1.2, GOVERN 1.5, GOVERN 4.3, GOVERN 6.2, MEASURE 3.3, MANAGE 2.3, MANAGE 4.1, MANAGE 4.3
Организационная ответственность	GOVERN 1, GOVERN 2, GOVERN 3.2, GOVERN 4, GOVERN 5, GOVERN 6.2, MEASURE 2.8, MEASURE 3.3
Культура эффективного испытания	GOVERN 4, MEASURE 2.8
Разнообразные и опытные команды	GOVERN 1.2, GOVERN 1.3, GOVERN 3, GOVERN 4.1, MAP 1.1, MAP 1.2, MEASURE 4
Испытание продукта на себе	GOVERN 4.1, MEASURE 4.1, MEASURE 4.2
Двигайтесь быстро и ломайте устои	GOVERN 2.1, GOVERN 4.1
Прогнозирование возможных отказов	GOVERN 1.2, GOVERN 1.3, GOVERN 4.2, MAP 1.1, MAP 2.3, MAP 3.2, MANAGE 1.4, MANAGE 2
Уровни риска	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 5.2, MAP 1.5, MAP 4, MAP 5.1, MANAGE 1.2, MANAGE 1.3, MANAGE 1.4
Документация модели	GOVERN 1, GOVERN 2.1, GOVERN 4.2, MAP, MEASURE 1.1, MEASURE 2, MEASURE 3, MEASURE 4, MANAGE
Мониторинг модели	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 1.5, MAP 2.3, MAP 3.5, MAP 4, MAP 5.2, MEASURE 1.1, MEASURE 2.4, MEASURE 2.6, MEASURE 2.7, MEASURE 2.8, MEASURE 2.10, MEASURE 2.11, MEASURE 2.12, MEASURE 3.1, MEASURE 3.3, MEASURE 4, MANAGE 2.2, MANAGE 2.3, MANAGE 2.4, MANAGE 3, MANAGE 4

Книги для программистов: https://t.me/free_books_for_it

Окончание

Раздел главы	Подкатегория NIST AI RMF
Инвентарный учет моделей	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 1.6, MAP 3.5, MAP 4, MANAGE 3
Проверка системы и аудит процессов	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 2.1, GOVERN 4.1, GOVERN 4.3, GOVERN 6.1, MAP 2.3, MAP 3.5, MAP 4, MEASURE, MANAGE 1
Управление изменениями	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 1.7, GOVERN 2.2, GOVERN 4.2, MAP 3.5, MAP 4, MEASURE 1.2, MEASURE 2.13, MEASURE 3.1, MANAGE 4.1, MANAGE 4.2
Аудит и оценка модели	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 2.1, GOVERN 4.1, MAP 3.5, MAP 4, MEASURE, MANAGE 1
Оценка воздействия	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 2.1, GOVERN 4.1, GOVERN 4.2, GOVERN 5.2, MAP 1.1, MAP 2.2, MAP 3.1, MAP 3.2, MAP 3.5, MAP 5, MEASURE 3, MANAGE 1
Подача апелляции, отмена решения и отказ от участия	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, GOVERN 1.5, GOVERN 3.2, GOVERN 5, MAP 3.5, MAP 5.2, MEASURE 2.8, MEASURE 3.3, MANAGE 4.1
Парное и двойное программирование	GOVERN 4.1, GOVERN 5.2, MAP 3.5
Ограничение полномочий при развертывании	GOVERN 1.4, GOVERN 4.1, GOVERN 5.2, MAP 3.5, MAP 4.2
Награда за обнаружение ошибок	GOVERN 5, MAP 3.5, MAP 5.2, MEASURE 3
Реагирование на инциденты с ИИ	GOVERN 1.2, GOVERN 1.5, GOVERN 4.3, GOVERN 6.2, MAP 3.5, MAP 4, MAP 5, MEASURE 3.1, MANAGE 2.3, MANAGE 4.1, MANAGE 4.3

- В документации применяются следующие характеристики надежности ИИ: достоверный и надежный; безопасный; с управляемой предвзятостью; безопасный и отказоустойчивый; прозрачный и подотчетный; объяснимый и интерпретируемый; с повышенной конфиденциальностью.
- Смотрите также:
  - система управления рисками ИИ NIST, **https://oreil.ly/1YAoU**;
  - пособие по системе управления рисками в области искусственного интеллекта NIST, **https://oreil.ly/6_TUM**;
  - полная таблица соответствий (не является официальным ресурсом), **https://oreil.ly/61TXd**.

Инциденты с ИИ

Ключевое предназначение процессов безопасности ML и связанной с ними отладки моделей, также обсуждаемых в главе 3, заключается в предотвращении и смягчении последствий инцидентов с ИИ. В этой книге мы обобщенно определяем инциденты с ИИ как любой результат работы системы, который может причинить вред. Как становится очевидным при использовании правила Хэнда в качестве ориентира, серьезность инцидента с ИИ увеличивается в зависимости от убытков, вызванных этим инцидентом, и уменьшается в зависимости от усилий операторов по смягчению потерь.

Поскольку сложные системы по определению склонны к сбою, у нас нет недостатка в разнообразных примерах инцидентов с ИИ, которые мы можем обсудить. Уро-

Книги для программистов: https://t.me/free_books_for_it

вень ущерба вследствие инцидентов лежит в очень широком диапазоне: от раздражающего падения робота-охранника с лестницы в торговом центре (*https://oreil.ly/fLHU1*) до гибели пешеходов под колесами беспилотного транспорта (*https://oreil.ly/vFW_-*) и массового отвлечения ресурсов здравоохранения от тех, кто в них больше всего нуждается (**https://oreil.ly/2e8WQ**). Как показано на рис. 1.2, инциденты с ИИ можно условно разделить на три группы:

Злоупотребления

ИИ вполне можно использовать в гнусных целях, и не только для конкретных взломов и атак на другие системы ИИ. Возможно, уже наступил тот день, когда злоумышленники начнут использовать ИИ для эффективной реализации необычных, ранее невиданных атак[4]. То, что может ждать нас в будущем, пугает еще больше. Такие призраки, как атаки автономных дронов и этническое профилирование со стороны авторитарных режимов, уже отчетливо просматриваются на горизонте.

Атаки

Исследователи опубликовали примеры всех основных типов атак — на конфиденциальность, целостность и доступность (дополнительная информация приведена в главе 5). Атаки на конфиденциальность включают в себя утечку обучающих данных или логики модели из конечных точек системы ИИ. Атаки на целостность включают в себя состязательное манипулирование обучающими данными или результатами модели с помощью состязательных примеров, уклонения, выдачи себя за другое лицо или отравления данных. Атаки на доступность могут проводиться с помощью более стандартных приемов типа «отказ в обслуживании», с помощью губительных запросов, которые чрезмерно используют системные ресурсы, или с помощью алгоритмической дискриминации, вызванной каким-либо злоумышленником, чтобы отказать в системных услугах определенным группам пользователей.

Сбои

Сбои в системах ИИ, как правило, связаны с алгоритмической дискриминацией, упущениями в безопасности и производительности, нарушениями конфиденциальности данных, недостаточной прозрачностью или проблемами в сторонних компонентах системы.

Инциденты с искусственным интеллектом — это реальность. Поскольку системы, в которых они возникают, очень сложны, инциденты с ИИ также бывают сложными. Инциденты с ИИ имеют различные причины: сбои, атаки и злоупотребления. Их особенность в том, что они смешивают традиционные представления о компьютерной безопасности с такими проблемами, как конфиденциальность данных и алгоритмическая дискриминация.

[4] Так и есть. Например, на момент подготовки русского перевода к изданию мошенники широко применяли сгенерированные при помощи ИИ аудио- и видеозвонки в мессенджерах для обмана своих жертв. — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

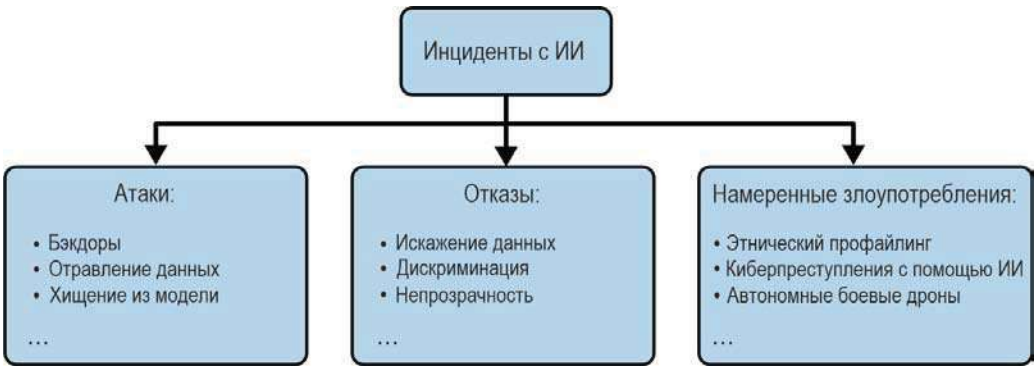


Рис. 1.2. Базовая классификация инцидентов с ИИ
(адаптировано из статьи *What to Do When AI Fails: https://oreil.ly/AHfmK*)

Показательным примером может служить инцидент с чат-ботом Tay (Тэй) в 2016 г. (*https://oreil.ly/a-DhB*). Tay был передовым чат-ботом, обученным ведущими мировыми экспертами из Microsoft Research с целью взаимодействия с людьми в «Твиттере»[5] и повышения осведомленности общества об искусственном интеллекте. Всего через шестнадцать часов после запуска — и 96 000 твитов спустя — Tay начал вести себя как порнограф-неонацист, и его пришлось закрыть. Что случилось? Пользователи «Твиттера» быстро поняли, что систему адаптивного обучения бота Tay можно легко отравить. Расистский и сексуальный контент, опубликованный ботом в «Твиттере», был включен в его обучающие данные и очень быстро привел к оскорбительным выводам. *Отравление данных* (data poisoning) — это атака на целостность системы, но из-за контекста, в котором она была проведена, эта атака привела к алгоритмической дискриминации. Также важно отметить, что разработчики Tay, будучи экспертами мирового класса в чрезвычайно хорошо финансируемом исследовательском центре, попытались установить кое-какие барьеры. Tay отвечал на некоторые острые вопросы заранее подготовленными ответами. Но этого было недостаточно, и работа Tay привела к инциденту, связанному с общественной безопасностью и алгоритмической дискриминацией в Microsoft Research.

Думаете, это был единичный случай? Отнюдь. Совсем недавно, опять же из-за спешки и неспособности систематически продумывать риски достоверности, безопасности, конфиденциальности и управляемости, многие из наиболее очевидных неудач Tay были повторены при запуске чат-бота Lee Luda (*https://oreil.ly/5OLXV*) компанией Scatter Lab. При разработке систем ML необходимо сопоставлять свои планы с прошлыми известными инцидентами в надежде предотвратить подобные инциденты в будущем. Именно в этом суть недавних усилий по созданию баз данных об инцидентах с ИИ (*https://oreil.ly/vvLbp*) и связанных с ними публикаций (*https://oreil.ly/59yaY*).

Инциденты с ИИ также могут служить мотиватором для ответственного развития технологий. К лучшему или худшему, культурные и политические взгляды людей на такие темы, как алгоритмическая дискриминация и конфиденциальность дан-

[5] Соцсеть, запрещенная на территории РФ. — *Прим. ред.*

Книги для программистов: https://t.me/free_books_for_it

ных, могут сильно различаться. Иногда очень сложно убедить команду прийти к согласию относительно трудоемких мер безопасности, опираясь только на этические соображения. Возможно, будет проще заставить людей работать над предотвращением опасных инцидентов с ИИ, исходя из того, что они очень дорого обходятся компании и потенциально бьют по карману всех причастных к ним. Понятие инцидентов с ИИ имеет ключевое значение для понимания безопасности ML. Центральной темой этой главы являются культурные компетенции и бизнес-процессы, которые можно использовать для предотвращения и смягчения последствий инцидентов, связанных с ИИ. Мы углубленно рассмотрим способы смягчения последствий в следующих разделах, а в завершение главы проанализируем реальный инцидент.

Культурные аспекты управления рисками машинного обучения

Культура организации является важным аспектом ответственного ИИ. В этом разделе будут обсуждаться такие культурные компетенции, как ответственность, готовность тестировать решения на себе, знание предметной области и устаревшая поговорка «двигайся быстро и ломай устои».

Организационная ответственность

Ключом к успешному снижению рисков ML является реальная ответственность внутри организаций за инциденты с ИИ. Если ничья работа не находится под угрозой, когда система ML выходит из строя, подвергается атаке или используется в грязных целях, то вполне возможно, что никто в этой организации и не заботится о безопасности и надежности ML. Помимо разработчиков, которые продумывают риски, применяют методы *обеспечения качества* (quality assurance, QA) программного обеспечения и методы отладки моделей, организациям нужны отдельные лица или группы, которые проверяют технологию системы ML и связанные с ней процессы. Организациям также нужен кто-то, кто будет отвечать за планы реагирования на инциденты с ИИ. Вот почему ведущие финансовые учреждения, чье использование прогнозного моделирования жестко регулируется на протяжении десятилетий, применяют технологии *управления модельными рисками* (model risk management, MRM). Система MRM фактически скопирована с руководства Федеральной резервной системы по управлению рисками модели SR 11-7, которое было разработано в результате финансового кризиса 2008 г. Примечательно, что во внедрении MRM часто участвуют ответственные руководители и несколько команд, которые несут ответственность за безопасность и эффективность моделей и ML-системы.

Внедрение стандартов MRM обычно требует наличия нескольких разных команд и участия высшего руководства. Вот некоторые из ключевых принципов, составляющих культурную основу MRM:

Письменные политики и процедуры

Организационные правила создания и использования ML должны быть записаны в явном виде и доступны всем заинтересованным сотрудникам организации.

Книги для программистов: https://t.me/free_books_for_it

Все, кто имеет отношение к системам ML, должны пройти обучение этим политикам и процедурам. Данные правила также следует регулярно проверять, чтобы вовремя вносить актуальные поправки. Никто не должен заявлять о незнании правил; правила должны быть прозрачными и не меняться без одобрения высшего руководства. Политика и процедуры должны включать четкие механизмы эскалации серьезных рисков или проблем высшему руководству и, вероятно, должны предусматривать процессы и меры защиты для лиц, сообщающих о нарушениях.

Эффективное испытание

Согласно принципу *эффективного испытания* (effective challenge) проверку и аудит системы ML должны проводить незаинтересованные эксперты, которые не участвовали в создании системы. В методике MRM эффективные испытания обычно распределяют по трем «линиям защиты», где добросовестные разработчики систем составляют первую линию защиты, а независимые квалифицированные и уполномоченные технические валидаторы и аудиторы процессов составляют вторую и третью линии соответственно.

Ответственное руководство

В организации должен быть назначен конкретный руководитель, отвечающий за предотвращение инцидентов с ИИ. Эту должность часто называют *главным специалистом по модельным рискам* (chief model risk officer, CMRO). Условия найма и вознаграждения CMRO нередко связаны с надежностью системы ML. Наличие CMRO в организации олицетворяет простой, но очень важный аспект культуры безопасности и надежности ML. Если начальник в самом деле заботится о безопасности и надежности системы, то следом за ним начинают заботиться и подчиненные.

Вознаграждение

Персонал и руководство подразделений по обработке данных должны быть заинтересованы в ответственном внедрении машинного обучения. Зачастую сжатые сроки разработки продукта могут вынудить отдать приоритет созданию минимально жизнеспособного продукта, а тщательное тестирование и исправления отложить на конец жизненного цикла модели непосредственно перед ее внедрением в производство. Более того команду тестирования и проверки ML часто оценивают по тем же критериям, что и команду разработчиков ML, что приводит к фундаментальному несоответствию, когда тестировщиков и валидаторов заставляют действовать быстро, а не обеспечивать качество. Адекватное определение сроков тестирования, критериев оценки эффективности и стимулирующей оплаты помогает укрепить культуру ответственного ML и снизить риски.

Конечно, небольшие или недавно созданные организации не всегда могут выделить штатного сотрудника для мониторинга рисков системы ML. Но важно, чтобы конкретный сотрудник или группа привлекались к ответственности, если система ML вызывает инциденты, и вознаграждались, если система работает хорошо. Если

Книги для программистов: https://t.me/free_books_for_it

в организации считается, что за риски ML и инциденты с ИИ отвечают все, то на самом деле за них не отвечает никто.

Культура эффективного испытания

Независимо от того, готова ваша организация внедрить полноценные методы MRM или нет, вы все равно можете извлечь пользу из определенных аспектов политики управления рисками. В частности, культуру эффективного испытания можно применять и вне контекста MRM. Фактически проводить эффективное испытание — значит активно оспаривать и подвергать сомнению действия, предпринятые на протяжении всей разработки системы ML. Организационная культура, которая поощряет серьезное сомнение в механизмах ML, с большей вероятностью позволит разработать надежные системы или продукты ML, а также выявить проблемы до того, как они перерастут в опасные инциденты. Обратите внимание, что политика эффективного испытания не должна становиться предметом административных злоупотреблений и ее необходимо в равной степени применять ко всему персоналу, разрабатывающему систему ML, не обходя стороной так называемых звездных инженеров и специалистов по обработке данных. Эффективные испытания также должны быть структурированы; например, это могут быть еженедельные встречи, на которых подвергаются сомнению текущие идеи разработчиков и серьезно рассматриваются альтернативные варианты устройства системы.

Разнообразные и опытные команды

Разнообразие команд может привнести более широкие и свежие взгляды на проектирование, разработку и тестирование систем машинного обучения. В однообразных и замкнутых командах этого обычно не происходит. Исследователи машинного обучения часто описывают печальные последствия, возникающие в результате того, что ученые, работающие с данными, игнорируют разнообразие команды при обучении модели или анализе результатов ее работы. Потенциальным решением подобных упущений является достижение максимального разнообразия в командах ML. При создании команд также важен опыт в бизнесе или другой области. Эксперты в предметной области играют важную роль в выборе ключевых признаков данных и разработке модели, а также в тестировании выходных данных системы. В условиях безумной спешки по разработке систем ML наличие в команде экспертов в предметной области также может служить дополнительной гарантией безопасности. Специалистам по обработке данных часто не хватает опыта, необходимого для работы с данными и результатами, специфичными для конкретной предметной области. Неправильное понимание значения входных данных или выходных результатов — это верный путь к катастрофе после развертывания системы. К сожалению, особенно плачевная ситуация складывается в области социальных наук. Ученые, работающие с данными в этой области, забывают или игнорируют важность отраслевой экспертизы, предпочитая обходиться своими силами. В наше время наблюдается так называемое тихое вытеснение социальных наук технологиями (*https://oreil.ly/IcIBi*), в рамках которого многие организации реализуют достойные сожаления проекты ML, подменяющие решения, которые должны прини-

Книги для программистов: https://t.me/free_books_for_it

мать подготовленные ученые-социологи или которые просто полностью игнорируют коллективные знания экспертов в области социальных наук[6].

Испытание продукта на себе

Испытание продукта ML на себе[7] подразумевает использование разработанного вами программного обеспечения или продукта внутри вашей собственной организации. Тестирование на себе, часто являющееся формой преальфа- или пребета-тестирования, позволяет выявить проблемы, возникающие из-за сложности сред развертывания in vivo, прежде чем ошибки и сбои повлияют на клиентов, пользователей или широкую общественность. Поскольку серьезные проблемы, такие как дрейф концепта, алгоритмическая дискриминация, ложное обучение и недостаточная специфичность, как известно, трудно выявить с помощью стандартных процессов разработки ML, испытание на себе обеспечивает ограниченный и контролируемый, но в то же время реалистичный испытательный стенд для систем ML. Конечно, если в организации работают демографически и профессионально разнообразные команды и в их состав входят эксперты в предметной области, где будет развернута система ML, это значительно облегчит выявление разнообразных проблем во время внутреннего испытания. Испытание системы ML на себе можно считать разновидностью золотого правила тестирования: если вам неудобно пользоваться продуктом самому или применять его в организации, то, очевидно не следует развертывать его для широкой публики.



Одним из важных аспектов, которые следует учитывать при выборе сред развертывания, является влияние систем ML на экосистемы и планету, например:

- углеродный след моделей ML;
- вероятность того, что система ML может нанести ущерб окружающей среде, вызвав инцидент с искусственным интеллектом;

Если вас беспокоит воздействие модели на окружающую среду, объедините управление ML с более широкими экологическими, социальными и управленческими инициативами в вашей организации.

Двигайтесь быстро и ломайте устои

Мантра «двигайся быстро и ломай устои» служит своего рода религиозным убеждением для многих «звездных» инженеров и специалистов по обработке данных. К сожалению, эти блистательные специалисты часто забывают, что когда они движутся быстро и ломают устои, то попутно ломается и все остальное. Поскольку системы ML принимают все более важные решения, которые касаются беспилотных транспортных средств, кредитов, трудоустройства, университетских оценок и

[6] Примером может служить технология алгоритмического ценообразования интернет-магазинов и агрегаторов такси, которая вызывает сильнейшее раздражение у пользователей и шокирует социологов. — *Прим. пер.*

[7] У западных разработчиков эта методика называется «eating your own dog food» (прежде, чем кормить собаку, попробуй сам) или «drinking your own Champagne» (попробуй свое шампанское). — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

посещаемости, медицинских диагнозов и распределения ресурсов, ипотеки, залога до суда, условно-досрочного освобождения и многого другого, разрушение устоев означает нечто большее, чем просто ошибочные решения. Это может означать, что небольшая группа ученых и инженеров, занимающихся данными, способна причинить огромный ущерб широкому кругу людей. Чтобы предотвратить катастрофические последствия от проблем с надежностью и безопасностью систем ML, необходимо изменить мышление специалистов, участвующих в разработке и внедрении продуктов. Специалисты-практики должны переместить приоритеты от подсчета количества функций приложения, которые они могут реализовать, или вычисления формальной точности тестовых данных модели ML к осознанию последствий и последующих рисков своей работы.

Организационные процессы управления рисками машинного обучения

Организационные процессы играют ключевую роль в обеспечении безопасности и надежности систем ML. Как и культурные компетенции, рассмотренные в предыдущем разделе, организационные процессы являются ключевым нетехническим фактором, определяющим надежность систем машинного обучения. Этот раздел, посвященный процессам, начинается с призыва к специалистам-практикам рассмотреть, задокументировать и попытаться смягчить любые известные или прогнозируемые режимы сбоев в их системах ML. Затем мы более подробно обсудим MRM. Хотя раздел «Культурные компетенции для управления рисками машинного обучения» посвящен людям и мышлению, необходимым для достижения успеха MRM, в этом разделе будут описаны различные процессы, которые MRM использует для снижения рисков в расширенном прогнозном моделировании и системах ML. Хотя MRM является достойным стандартом процесса, к которому мы все можем стремиться, существуют дополнительные важные элементы управления процессом, которые обычно не являются частью MRM. В этом разделе мы выйдем за рамки традиционного MRM и выделим важнейшие процессы контроля рисков, такие как парное или двойное программирование, а также требования к разрешениям безопасности для развертывания кода.

Этот раздел завершится обсуждением реагирования на инциденты с использованием ИИ. Как бы усердно мы ни работали над минимизацией вреда при проектировании и внедрении системы ML, нам все равно придется готовиться к сбоям и атакам.

Прогнозирование возможных отказов

Эксперты по безопасности и этике ML в целом сходятся во мнении о важности продумывания, документирования и попыток смягчить прогнозируемые режимы отказа систем ML. К сожалению, они также в основном сходятся во мнении, что это нетривиальная задача. К счастью, в последние годы появились новые ресурсы и проведены исследования, помогающие разработчикам систем ML более системно прогнозировать инциденты. Если удастся определить целостные категории потен-

Книги для программистов: https://t.me/free_books_for_it

циальных отказов, это превратит усиление систем машинного обучения для повышения реальной надежности и безопасности в более осмысленную и предсказуемую задачу. Далее мы обсудим одну такую стратегию, а также несколько дополнительных процессов, которые пригодятся для прогнозирования будущих инцидентов в системах ML.

Известные отказы

Как сказано в работе Preventing Repeated Real World AI Failures by Cataloging Incidents: The AI Incident Database (*https://oreil.ly/BfMJC*), один из наиболее эффективных способов смягчения потенциальных инцидентов с ИИ в системах машинного обучения — это сравнение нашей системы ML с прошлыми неудачными разработками. Подобно тому как специалисты в области транспорта расследуют и каталогизируют инциденты, а затем используют результаты для предотвращения аналогичных инцидентов и тестирования новых технологий, некоторые исследователи ML, обозреватели и торговые компании начали собирать и анализировать инциденты с ИИ в надежде предотвратить повторение связанных с ними отказов. Вероятно, самым известным и полным хранилищем инцидентов с ИИ является AI Incident Database (*https://oreil.ly/H8nmd*) (База данных инцидентов с ИИ). Этот интерактивный ресурс позволяет зарегистрированным пользователям выполнять поиск по ключевым словам в визуальной базе данных и находить различные типы информации об общедоступных инцидентах.

Обязательно воспользуйтесь этим ресурсом при разработке систем машинного обучения. Если система, подобная той, которую вы сейчас проектируете, внедряете или развертываете, уже вызвала инцидент в прошлом, это один из самых убедительных индикаторов того, что ваша новая система может стать причиной инцидента. Если вы видите в базе данных нечто знакомое, вам следует остановиться и более тщательно подумать о том, что вы намерены сделать.

Неспособность предвидения

Предвидеть будущее с его контекстом и всеми подробностями — непростая задача. Но зачастую именно контекст, в котором работают системы ML, в сочетании с непредвиденными или непознаваемыми деталями приводит к инцидентам с ИИ. На одном из семинаров авторы статьи Overcoming Failures of Imagination in AI Infused System Development and Deployment (*https://oreil.ly/veB5T*) предложили структурированные подходы к построению гипотез об этих трудно вообразимых будущих рисках. Они предлагают при обсуждении инцидентов с ИИ начать с ответов на вопросы: *кто* (например, инвесторы, клиенты, уязвимые лица, не пользующиеся услугами), *что* (например, благополучие, возможности, репутация), *когда* (например, немедленно, часто, в течение длительных периодов времени) и *как* (например, предпринимая действия, изменяя убеждения). Затем они призывают разработчиков систем учитывать следующее:

- предположение о том, что влияние системы будет только полезным (и признание того, что существует неопределенность в воздействии системы);

Книги для программистов: https://t.me/free_books_for_it

- проблемную область и варианты прикладного использования системы, а не только математические и технологические аспекты;
- любые неожиданные или странные результаты, действия пользователя и реакции системы.

Инциденты с ИИ являются для организаций как минимум неприятными, зачастую дорого обходятся и ведут к нарушению закона. Подобные инциденты также могут нанести вред потребителям и широкой общественности. Тем не менее при некоторой дальновидности многие из известных в настоящее время инцидентов с искусственным интеллектом можно было бы смягчить, если не полностью избежать. Вполне возможно, что при проведении комплексной проверки и концептуализации сбоев ML вы обнаружите, что вашу систему необходимо полностью переработать. Если это так, то можете утешаться тем, что задержка во внедрении или развертывании системы, вероятно, обойдется дешевле, чем ущерб, который может понести ваша организация или широкий круг людей, если будет развернута ущербная система.

Процессы управления модельными рисками

Процессуальные аспекты MRM требуют тщательного документирования систем моделирования, проверки систем человеком и постоянного мониторинга систем. Согласно предписанию Федеральной резервной системы SR 11-7 MRM эти процессы представляют собой основную часть бремени управления рисками прогнозных моделей, используемых в значимых приложениях потребительского финансирования. Хотя только крупные организации способны полностью реализовать все положения MRM, любой серьезный специалист по ML может извлечь из них много полезного. Далее мы разбиваем процессы MRM на более мелкие компоненты, чтобы читатели могли начать обдумывать использование аспектов MRM в своей организации.

Уровни риска

Как отмечалось в начале этой главы, произведение вероятности причинения вреда и вероятных убытков в результате этого вреда является общепринятым способом оценки риска развертывания конкретной системы ML. Произведение риска и потерь в контексте MRM имеет более формальное название — *существенность* (materiality). Существенность — это мощная концепция, которая позволяет организациям назначать реалистичные уровни риска для систем ML. Что еще более важно, такое деление рисков по уровням позволяет эффективно распределять ограниченные ресурсы для разработки, проверки и аудита. Разумеется, приложения с самой высокой существенностью должны получать наибольшее внимание и проверку со стороны человека, в то время как приложения с самой низкой существенностью потенциально могут обрабатываться системами *автоматического машинного обучения* (automatic machine learning, AutoML) и проходить минимальную проверку. Поскольку снижение рисков систем ML — это постоянная и дорогостоящая работа, правильное распределение ресурсов между системами с высоким, средним и низ-

Книги для программистов: https://t.me/free_books_for_it

ким уровнем риска является обязательным условием эффективного управления рисками.

Документация модели

Стандарты MRM также требуют тщательного документирования систем. Во-первых, документация должна обеспечивать подотчетность всех лиц, принимающих участие в эксплуатации системы, а также описывать текущее обслуживание системы и степень реагирования на инциденты. Во-вторых, документация должна быть стандартизирована с точки зрения наиболее эффективных процессов аудита и проверки. Документация — это то, с чего начинается процесс проверки модели на соответствие нормативным требованиям. Ниже представлены очень обобщенные шаблоны документации. Это формы отчетов, которые специалисты по данным и инженеры заполняют по мере прохождения стандартизированного рабочего процесса или на более поздних этапах разработки модели. Шаблоны документации обычно включают все шаги, которые ответственный практикующий специалист должен выполнить для построения надежной модели. Если документ частично не заполнен, это говорит о небрежности в процессе обучения. Поскольку большинство шаблонов и бланков документации также требуют добавления своего имени и контактной информации в готовом описании модели, не должно быть никакой тайны о том, кто отлынивает от своих обязанностей. Следующий список представляет собой приблизительную комбинацию типичных разделов документации MRM и разделов, рекомендованных приложениями к Закону ЕС об искусственном интеллекте.

- Основная информация:
  - имена разработчиков и заинтересованных сторон;
  - текущая дата и таблица изменений;
  - краткое описание системы модели;
  - обоснование значимости или бизнес-ценности;
  - предполагаемое использование и пользователи;
  - потенциальный вред и этические соображения.
- Информация о данных, применявшихся при разработке:
  - источник данных для разработки;
  - словарь данных;
  - оценка влияния на конфиденциальность;
  - предположения и ограничения;
  - реализация программного обеспечения для предварительной обработки данных.
- Информация о модели:
  - описание алгоритма обучения с использованием рецензируемых источников;
  - спецификация модели;

Книги для программистов: https://t.me/free_books_for_it

- показатели качества модели;
- предположения и ограничения;
- программная реализация алгоритма обучения.

♦ Информация о тестировании:

- тестирование качества и исправление ошибок;
- тестирование на дискриминацию и устранение дискриминации;
- тестирование безопасности и исправление недостатков;
- предположения и ограничения;
- программная реализация инструментов тестирования.

♦ Информация о развертывании:

- планы и механизмы мониторинга;
- зависимости выше и ниже по уровню;
- планы и механизмы обжалования и отмены решения;
- планы и механизмы аудита;
- планы управления изменениями;
- планы реагирования на инциденты.

♦ Ссылки (если вы занимаетесь наукой, то вам повезло — вы стоите на плечах гигантов и легко сможете указать несколько ссылок на рецензируемые издания в красиво отформатированной библиографии!).

Конечно, эти документы могут занимать сотни страниц, особенно для систем с высоким уровнем материалоемкости. Предлагаемые стандарты технических описаний и типовых карточек также могут быть полезны для небольших и молодых организаций. Если читатели чувствуют, что в настоящее время не смогут внедрить в своей организации полномасштабное документирование, то, возможно, вместо этого им подойдут два более простых подхода, описанные ниже.

Мониторинг модели

Одна из ключевых проблем безопасности ML заключается в том, что поведение системы ML в реальном мире трудно предсказать и, соответственно, его нужно постоянно отслеживать. Фактически поведение развернутой системы следует контролировать часто, планомерно и на протяжении всего периода эксплуатации. Владелец системы может выбрать для отслеживания любые показатели. Наиболее распространенным является дрейф входных данных. Данные, на которых обучена система ML, представляют собой статический снимок рабочей среды системы, но реальный мир совсем не статичен. На рынки могут выйти конкуренты, правительство может ввести новые правила, могут измениться вкусы потребителей, могут произойти пандемии или другие катастрофы. Любое из этих событий может сместить текущие данные, поступающие в вашу систему машинного обучения, в сторону от характеристик ее обучающих данных, что приведет к снижению ее надеж-

Книги для программистов: https://t.me/free_books_for_it

ности или даже сделает опасной для пользователей. Чтобы избежать таких неприятных сюрпризов, качественные системы машинного обучения отслеживают как на предмет изменяющегося распределения входных и выходных данных, так и на предмет постепенного ухудшения качества, часто называемого *устареванием модели* (model decay). Хотя наиболее часто отслеживают формальные показатели качества работы модели, системы ML также можно отслеживать на предмет аномальных входных данных или прогнозов, конкретных атак и взломов, а также на предмет изменения показателей непредвзятости.

Инвентарный учет моделей

Любая организация, внедряющая машинное обучение, должна быть в состоянии ответить на простые вопросы вроде следующих.

- Сколько систем машинного обучения развернуто в настоящее время?
- На скольких клиентов или пользователей влияют эти системы?
- Кто отвечает за надлежащую эксплуатацию каждой системы?

Согласно методике MRM для этого предназначены *реестры моделей*. Это тщательно сформированная и обновляемая база данных всех систем машинного обучения организации. Реестр моделей может служить хранилищем важной информации для документации, но также должен быть связан с планами и результатами мониторинга, планами и результатами аудита, важным прошлым и предстоящим обслуживанием и изменениями системы, а также планами реагирования на инциденты.

Проверка системы и аудит процессов

Согласно традиционным методикам MRM система ML проходит две начальные проверки перед выпуском. Первая представляет собой техническую проверку системы, в ходе которой квалифицированные валидаторы, нередко специалисты по данным с докторской степенью, пытаются найти ошибки и уязвимости в проекте и реализации системы. Они работают непосредственно с разработчиками системы над устранением любых обнаруженных проблем. Вторая проверка направлена на процессы. Сотрудники, отвечающие за аудит и соблюдение требований, тщательно анализируют проектирование, разработку и развертывание системы, а также документацию и планы на будущее, чтобы обеспечить соблюдение всех нормативных и внутренних требований к процессам. Более того, поскольку системы ML со временем меняются и устаревают, проверка должна проводиться всякий раз, когда система подвергается серьезному обновлению, или с согласованной периодичностью в будущем.

Читатели могут подумать (опять же), что их организация не располагает ресурсами для таких обширных проверок. Конечно, это реальность для многих небольших и молодых организаций. Ключевыми факторами проверки и аудита, которые должны применяться практически в любой организации, являются: наличие технических специалистов, которые занимаются тестированием системы, но *не* участвовали в ее разработке; наличие функции проверки на соответствие нетехническим внутренним

Книги для программистов: https://t.me/free_books_for_it

и внешним требованиям; а также надзор за письменным утверждением важных развертываний системы ML.

Управление изменениями

Как и все сложные программные продукты, системы ML обычно состоят из большого количества различных компонентов — от внутреннего кода машинного обучения до интерфейсов прикладного программирования (application programming interfaces, API) и графических пользовательских интерфейсов (graphical user interfaces, GUI). Изменения в любом компоненте системы могут вызвать побочные эффекты в других компонентах. Добавьте к этому такие проблемы, как дрейф данных, новые правила конфиденциальности данных и меры по противодействию дискриминации, а также сложные зависимости от стороннего программного обеспечения, и вы поймете, насколько серьезной проблемой является управление изменениями в системах ML. Если вы находитесь на этапе планирования или проектирования критически важной системы машинного обучения, вам, вероятно, потребуется сделать управление изменениями частью строгого контроля процессов. Без четкого планирования и наличия ресурсов для управления изменениями очень трудно предотвратить процессуальные или технические ошибки, возникающие в результате развития системы, — такие как использование данных без согласия правообладателя или несоответствие API. Более того, без управления изменениями подобные проблемы могут быть даже не обнаружены до тех пор, пока они не вызовут инцидент.

На протяжении всей книги мы будем возвращаться к MRM. Это одна из наиболее проверенных платформ для управления рисками систем машинного обучения. Конечно, MRM — не единственное место, где можно черпать вдохновение для улучшения процессов безопасности и производительности ML, и в следующем разделе мы займемся извлечением уроков из других прикладных областей.



Изучение 21-страничного руководства по управлению модельными рисками SR 11-7 — это быстрый и легкий способ повысить свои навыки в управлении рисками ML. Читая его, обратите особое внимание на культурные и организационные аспекты. Управление технологическими рисками часто зависит от людей больше, чем от чего-либо еще.

За пределами управления модельными рисками

Из опыта финансового аудита, обеспечения конфиденциальности данных и разработки программного обеспечения, а также из практики ИТ-безопасности можно извлечь множество уроков по управлению рисками ML. В этом разделе мы обратимся к идеям, которые существуют за пределами традиционного MRM: аудит моделей, оценка воздействия, апелляции, переопределение, отказ от участия, парное или двойное программирование, минимальные привилегии, вознаграждение за ошибки и реагирование на инциденты — все это мы рассмотрим с точки зрения безопасности ML.

Книги для программистов: https://t.me/free_books_for_it

Аудит и оценка модели

Аудит — обычный термин в MRM, но он также имеет значение, выходящее за привычные рамки: третья линия защиты в более традиционном сценарии MRM. В последние годы стала популярной фраза «аудит модели». *Аудит модели* — это процедура официального тестирования и обеспечения прозрачности модели, в которой основное внимание уделяется проверке соблюдения определенной политики, постановлений или законов. Аудиты моделей, как правило, проводятся независимыми третьими сторонами при ограниченном взаимодействии между аудитором и проверяемыми организациями. Подробное описание аудита моделей можно найти в недавней статье Algorithmic Bias and Risk Assessments: Lessons from Practice (*https://oreil.ly/eHxBb*). В статье Closing the AI Accountability Gap: Defining an End-to-End Framework for Internal Algorithmic Auditing (*https://oreil.ly/vO9cH*) предложена надежная основа для аудитов и оценок, включая даже примеры проработанной документации. Сопутствующий аудиту термин *оценка модели* (model assessment), означает, как правило, более неформальное тестирование вместе с разработчиками, которое может проводиться внешними или внутренними группами.

Аудиты и оценки ML могут быть сосредоточены на проблемах предвзятости или других серьезных рисках, включая безопасность, ущерб конфиденциальности данных и уязвимости безопасности. Независимо от цели аудита и аудиторов, они должны быть честными и прозрачными. Аудиторы должны соблюдать четкие этические или профессиональные стандарты, которые по состоянию на 2023 г. практически не существуют. Без понятных механизмов подотчетности или правил аудит может утратить свою эффективность как инструмент управления рисками или, что еще хуже, превратиться в упражнение по сокрытию недостатков систем ML. Несмотря на эти недостатки, аудит является излюбленной тактикой контроля рисков среди политиков и исследователей и прописан в ряде законов, например в упомянутом ранее локальном законе города Нью-Йорка № 144.

Оценка воздействия

Оценка воздействия (impact assessments) — это формальный документальный подход, используемый в различных областях для прогнозирования и регистрации потенциальных проблем, которые система может вызвать после внедрения. Вероятно из-за использования в целях обеспечения конфиденциальности данных оценка воздействия начала фигурировать в политиках организаций по ML и недавних законопроектах. Оценка воздействия — это эффективный способ продумать и задокументировать вред, который может нанести система ML, повышая тем самым ответственность разработчиков и операторов систем ИИ. Однако оценки воздействия самой по себе недостаточно. Если вспомнить ранее выдвинутое определение риска и существенности, то можно сказать, что воздействие — это лишь один из факторов риска. Сначала необходимо проанализировать последствия с учетом вероятности их наступления, чтобы сформировать меру риска, затем можно приступать к деятельному смягчению рисков, уделяя наибольшее внимание приложениям с самым высоким риском. Оценка воздействия — это только начало более широкого процесса управления рисками. Как и другие процессы управления рисками,

Книги для программистов: https://t.me/free_books_for_it

оценку воздействия следует выполнять с периодичностью, соответствующей характеру оцениваемой системы. Если система меняется быстро, ей потребуется более частая оценка воздействия. Другая потенциальная проблема с оценками воздействия возникает, когда они разрабатываются и реализуются командами ML, оценивающими результаты собственного труда. В этом случае им трудно избежать соблазна уменьшить масштаб оценки и приуменьшить любые потенциальные негативные последствия. Оценки воздействия являются важной частью более широких стратегий управления рисками и управления, но они должны проводиться так часто, как того требует конкретная система, и, вероятно, с привлечением независимых специалистов по надзору.

Подача апелляции, отмена решения и отказ от участия

В большинстве систем ML должны быть предусмотрены механизмы, позволяющие пользователям или операторам обжаловать и отменять неправильные решения системы. На практике этот механизм встречается под разными названиями: возможность обращения в суд, возможность вмешательства оператора, возмещение ущерба или уведомление о неблагоприятных действиях. Это может быть простейшая функция, наподобие ссылки «Пожаловаться на неприемлемые подсказки» в строке поиска Google, или сложная процедура предоставления данных и объяснений пользователям, а также запуска процессов апелляции в отношении явно неверных данных или механизмов принятия решений. Другой аналогичный подход, называемый *отказом от участия* (opt out), заключается в том, чтобы позволить пользователям вести дела с организацией старомодным способом, не проходя никакой автоматической обработки. Многие законы о конфиденциальности данных и основные законы США о потребительском финансировании предусматривают возможность обращения за помощью оператора, отказ от участия или же и то и другое. Автоматическое навязывание неправильных решений многим пользователям — одно из самых явных этических нарушений в ML. Казалось бы, никто не должен попадать в этическую, юридическую и репутационную ловушку, которая так очевидна и так хорошо известна, но многие системы попадают в нее. Вероятно, это связано с тем, что для реализации права на отказ от участия, апелляцию и отмену решения необходимо с самого начала разработки спланировать и заложить в систему ML соответствующие организационные и технические ресурсы, а это случается не всегда.

Парное и двойное программирование

Поскольку алгоритмы ML часто бывают сложными и стохастическими, трудно определить, верна ли конкретная реализация! Вот почему некоторые крупные организации выполняют двойную реализацию алгоритмов ML в качестве механизма контроля качества. Такая двойная реализация обычно достигается одним из двух методов: *парным программированием* или *двойным программированием*. При парном программировании два технических эксперта кодируют алгоритм без какого-либо взаимодействия друг с другом. Затем они объединяют усилия и устраняют любые несовпадения между своими реализациями. При двойном программировании один и тот же эксперт реализует один и тот же алгоритм дважды, но на совершенно раз-

Книги для программистов: https://t.me/free_books_for_it

ных языках программирования, таких как Python (объектно-ориентированный) и SAS (процедурный). Затем он устраняет любые различия между двумя реализациями. Оба этих подхода позволяют выявлять многочисленные ошибки, которые в противном случае оставались бы незамеченными до тех пор, пока система не будет развернута. Парное и двойное программирование также можно совмещать с более стандартным рабочим процессом прототипирования алгоритмов специалистами по обработке данных, в то время как специально выделенные инженеры доводят их до совершенства перед развертыванием. Однако, чтобы эта стратегия сработала, инженеры должны иметь возможность оспаривать и тестировать прототипы специалистов по обработке данных, и их задачу не следует ограничивать простым перекодированием прототипов.

Ограничение полномочий при развертывании модели

Концепция *минимальных привилегий* в области ИТ-безопасности гласит, что ни один пользователь системы никогда не должен иметь больше разрешений, чем ему необходимо. Минимальные привилегии — это фундаментальный элемент управления безопасностью. Но реальность такова, что системы ML затрагивают очень много других ИТ-систем, поэтому разработчики приложений и «звездные» специалисты по обработке данных очень часто пренебрегают концепцией минимальных привилегий. К сожалению, это отрицательный стереотип в области ML. За пределами замкнутого мирка, состоящего из разработчиков систем ML и экспертов в области науки о данных, уже давно известно, что инженеры не могут адекватно тестировать свой собственный код и что последнее решение о готовности программного обеспечения должны принимать другие сотрудники организации — менеджеры по продукту, юристы или руководители.

По этим причинам ИТ-полномочия, необходимые для развертывания системы ML, должны быть распределены между несколькими командами ИТ-организаций. Во время спринтов разработки специалисты по данным и программисты, безусловно, должны сохранять полный контроль над своей рабочей средой. Но по мере приближения важных релизов или тестирования права ИТ-специалистов на внесение исправлений, улучшений или новых функций в продукты, ориентированные на пользователей, передаются от специалистов по данным и программистов менеджерам по продуктам, тестировщикам, юристам, руководителям и другим ответственным лицам. Такие элементы управления процессом обеспечивают страховочный шлюз, который предотвращает развертывание неутвержденного кода.

Награда за обнаружение ошибок

Награда за обнаружение ошибок (bug bounty) — еще одна концепция, которую имеет смысл позаимствовать из компьютерной безопасности. Она заключается в том, что организация предлагает вознаграждение за обнаружение проблем в своем программном обеспечении, особенно уязвимостей безопасности. Поскольку ML — это в основном программное обеспечение, вы можете назначать вознаграждение за обнаружение ошибок в системах ML. Хотя традиционно этот прием направлен на поиск уязвимостей в системах машинного обучения, вы также можете

Книги для программистов: https://t.me/free_books_for_it

использовать его для поиска проблем иного рода, связанных с надежностью, безопасностью, прозрачностью, объяснимостью, интерпретируемостью или конфиденциальностью. Обычно организация предлагает денежные вознаграждения за обнаруженные ошибки, чтобы стимулировать обратную связь от сообщества в рамках стандартизированного процесса. Ранее в этой главе мы уже говорили, что стимулы имеют решающее значение в управлении рисками. Как правило, работа по управлению рисками является утомительной и ресурсоемкой. Если вы хотите, чтобы пользователи находили серьезные проблемы в ваших системах ML, необходимо предложить им достойные выплаты или вознаграждать каким-либо другим значимым способом. Вознаграждение за обнаружение ошибок обычно является общественной инициативой и не прописано в законах и нормах. Участие посторонних людей в поиске ошибок очень нервирует некоторые организации. В таком случае можно организовать внутренние хакатоны, на которых разные команды ищут ошибки в системах ML. Конечно, чем больше участников будет заинтересовано в участии, тем лучше будут результаты.

Реагирование на инциденты с ИИ

Согласно пресловутому руководству SR 11-7 «даже при квалифицированном моделировании и надежной проверке модельный риск невозможно устранить полностью». Если риски, связанные с системами и моделями ML, невозможно устранить, то рано или поздно они приведут к инцидентам. К счастью, реагирование на инциденты является давно сложившейся практикой в сфере компьютерной безопасности. Авторитетные институты, такие как NIST и SANS, уже много лет публикуют рекомендации по реагированию на инциденты компьютерной безопасности. Поскольку машинное обучение является менее зрелой и более рискованной технологией, чем корпоративные вычисления общего назначения, наличие формальных планов и методов реагирования на инциденты с ИИ является обязательным условием эксплуатации высокоэффективных или критически важных систем ИИ.

Четкие планы реагирования позволяют организациям более быстро и эффективно реагировать на неизбежные инциденты. Реагирование на инциденты также должно учитывать правило Хэнда, упомянутое в начале главы. Имея отработанные планы реагирования на инциденты, организации смогут выявлять, сдерживать и устранять инциденты с ИИ, прежде чем они перерастут в дорогостоящие или опасные катастрофы. Разработка планов реагирования на инциденты — один из самых простых и универсальных способов снижения рисков, связанных с ИИ. Перед развертыванием системы необходимо составить и протестировать планы реагирования на инциденты. Для молодых или небольших организаций, которые не могут внедрить полномасштабное управление модельными рисками, реагирование на инциденты с ИИ является экономически эффективным и мощным средством ограничения рисков в области ML. По аналогии с реакцией на компьютерные инциденты реакцию на инциденты с ИИ можно разделить на шесть этапов.

Этап 1: подготовка

Помимо определения понятия инцидента с ИИ в вашей организации, подготовка к инцидентам с ИИ включает в себя кадровые, логистические и технологические

Книги для программистов: https://t.me/free_books_for_it

планы на случай возникновения инцидента. Необходимо выделить бюджет на реагирование, разработать коммуникационные стратегии и реализовать технические меры безопасности для стандартизации и сохранения документации моделей, аварийной связи и отключения систем искусственного интеллекта. Одним из лучших способов подготовки и репетиции инцидентов с ИИ являются командные тренинги, в ходе которых ключевой персонал организации работает над реалистичным инцидентом. Хороший перечень стартовых вопросов для обсуждения инцидентов с ИИ выглядит так:

- У кого есть организационный бюджет и полномочия для реагирования на инциденты с ИИ?
- Можно ли перевести рассматриваемую систему искусственного интеллекта в автономный режим? Кто это сделает? Какой ценой? Какие восходящие процессы будут затронуты?
- С какими регулирующими или правоохранительными органами необходимо связаться? Кто с ними свяжется?
- С какими внешними юридическими фирмами, страховыми агентствами или агентствами по связям с общественностью необходимо связаться? Кто с ними свяжется?
- Кто будет управлять коммуникациями? Внутри, между ответственными лицами? Извне, с клиентами или пользователями?

Этап 2: обнаружение

Обнаружение — это выявление силами самой организации сбоев, атак или злоупотреблений ИИ. Обнаружение также означает непрерывное сохранение бдительности в отношении злоупотреблений, связанных с ИИ. На практике обнаружение обычно основано как на общих подходах к выявлению атак, таких как мониторинг сетевых вторжений, так и на более специализированном мониторинге сбоев систем ИИ, таких как дрейф понятий или алгоритмическая дискриминация. Часто последним шагом этапа обнаружения является уведомление руководства, лиц, непосредственно реагирующих на инциденты, и других лиц, указанных в планах реагирования на инциденты.

Этап 3: сдерживание

Сдерживание означает смягчение непосредственного вреда от инцидента. Имейте в виду, что вред редко ограничивается системой, в которой начался инцидент. Как и более традиционные компьютерные инциденты, инциденты с ИИ могут иметь сетевые последствия, которые распространяются на деятельность организации и ее клиентов. Фактические стратегии сдерживания зависят от того, вызван ли инцидент внешним противником, внутренним сбоем, неправомерным использованием или злоупотреблением системой ИИ. Этап сдерживания — подходящее время, чтобы в случае необходимости обратиться к общественности.

Этап 4: устранение

Устранение включает в себя меры по исправлению любых затронутых систем. Например, можно изолировать любые атакованные системы от векторов про-

Книги для программистов: https://t.me/free_books_for_it

никновения или эксфильтрации или отключить дискриминирующую систему ИИ и временно заменить ее надежной системой, основанной на правилах. После этапа устранения не должно возникнуть никаких новых повреждений, вызванных инцидентом.

Этап 5: восстановление

Восстановление означает возвращение всех затронутых систем в нормальное состояние при наличии средств контроля для предотвращения подобных инцидентов в будущем. Восстановление часто означает переобучение или повторное развертывание систем ИИ, а также тестирование их работоспособности на задокументированных уровнях предшествующих инцидентов. Восстановление также может потребовать тщательного анализа технических протоколов или протоколов безопасности для персонала, особенно в случае случайного сбоя или внутренней атаки.

Этап 6: извлечение уроков

Извлечение уроков заключается в исправлении или улучшении планов реагирования на инциденты с ИИ с учетом успехов и проблем, возникших при реагировании на текущий инцидент. Улучшения плана реагирования могут быть ориентированы на процессы или технологии.

Анализируя следующий практический пример, сопоставьте его с этапами реагирования на инциденты и подумайте, будет ли план реагирования на инциденты с использованием ИИ эффективным средством контроля рисков для компании Zillow.

Пример: взлет и падение iBuying от Zillow

В 2018 г. компания Zillow, занимающаяся технологиями в сфере недвижимости, запустила бизнес по покупке и последующей прибыльной перепродаже домов под торговой маркой iBuying. Компания считала, что ее собственный алгоритм Zestimate на основе машинного обучения может сделать больше, чем просто привлечь внимание к ее чрезвычайно популярным веб-продуктам. Как рассказывает Bloomberg (*https://oreil.ly/LQQg3*), поначалу Zillow наняла экспертов в области недвижимости для проверки решений, полученных с помощью алгоритма. Сначала местные агенты оценивали недвижимость. Их оценки объединялись с данными Zestimate, а затем группа экспертов окончательно проверяла каждое предложение алгоритма перед заключением сделки.

По данным Bloomberg, Zillow вскоре отказалась от участия экспертов, чтобы «быстрее получать предложения», отдав предпочтение скорости и масштабу прямого алгоритмического подхода. Когда в начале 2021 г. алгоритм Zestimate не адаптировался к быстро растущему рынку недвижимости, Zillow, как сообщается, вмешалась в его работу, чтобы повысить привлекательность своих предложений. В результате этих изменений компания начала приобретать недвижимость со скоростью около 10 000 домов в квартал. Больше сделок с домами означает больше персонала и больше подрядчиков по ремонту и реконструкции, но, как говорит Bloomberg, «люди Zillow не успевали за сделками». Несмотря на увеличение штата сотрудни-

Книги для программистов: https://t.me/free_books_for_it

ков на 45% и привлечение целой армии подрядчиков, система iBuying так и не достигла прибыльности. Сочетание кадровых проблем во время пандемии, затруднений с поставками, перегретого рынка жилья и сложностей, связанных с обработкой большого количества ипотечных кредитов, оказалось непосильной ношей для проекта iBuying.

В октябре 2021 г. Zillow объявила, что прекратит делать предложения о покупке до конца года. В результате стремления Zillow к быстрому росту, а также нехватки рабочей силы и поставок компании пришлось срочно избавляться от огромного запаса домов. Чтобы решить проблему с излишками, Zillow выставляла большинство домов на перепродажу с убытком. Наконец, 2 ноября компания объявила, что списывает свои запасы недвижимости более чем на 500 млн долларов. Набег Zillow на автоматизированный бизнес по перепродаже домов завершился.

Последствия

Помимо огромных денежных потерь от неудачного предприятия, Zillow объявила, что уволит около 2000 сотрудников — целую четверть компании. В июне 2021 г. акции Zillow торговались по цене около 120 долларов. На момент написания этой книги, почти год спустя, цена акции составляла примерно 40 долларов, что привело к потере стоимости акций более чем на 30 миллиардов долларов. (Конечно, падение акций было обусловлено не только инцидентом с iBuying, но и, безусловно, во многом от него зависело.) Провал iBuying от Zillow объясняется несколькими взаимосвязанными причинами, и его нельзя отделить от пандемии, разразившейся в 2020 г. и перевернувшей рынок жилья. В следующем разделе мы рассмотрим, как применить то, что вы узнали из этой главы о корпоративном управлении и управлении рисками, к злоключениям Zillow.

Какие уроки мы можем извлечь?

Что этот пример говорит нам о злоключениях Zillow и ее алгоритма iBuying? Судя по опубликованным докладам, ключевую роль в инциденте сыграло решение Zillow исключить экспертную проверку предложений алгоритма. Также могут возникнуть вопросы, правильно ли Zillow продумала финансовый риск, который она взяла на себя, были ли созданы соответствующие структуры управления и можно ли было лучше справиться с потерями iBuying с точки зрения реагирования на инцидент с ИИ. Мы не знаем ответов на многие из этих вопросов относительно Zillow, поэтому сосредоточимся на идеях, которые читатели могут применить в своих организациях.

Урок 1: проверка экспертами в предметной области

В этой главе мы неоднократно подчеркивали важность разнообразных и опытных команд как основной организационной компетенции для ответственной разработки ML. Без сомнения, Zillow имеет внутренний и внешний доступ к экспертным знаниям мирового уровня на рынках недвижимости. Однако в интересах скорости и автоматизации (согласно принципу «двигайся быстро и разрушай устои») Zillow довольно скоро отказалась от участия экспертов в процес-

Книги для программистов: https://t.me/free_books_for_it

се приобретения домов, решив вместо этого полагаться на свой алгоритм Zestimate. Согласно последующему отчету Bloomberg в мае 2022 г., «как рассказывали люди, знакомые с ситуацией, Zillow посоветовала своим экспертам по ценообразованию прекратить подвергать сомнению действия алгоритма». Это решение, возможно, оказалось фатальным для предприятия, особенно на быстро меняющемся, искаженном пандемией рынке недвижимости. Несмотря на шумиху, ИИ пока не умнее человека. Если ML задействовано в принятии решений с высоким риском, привлекайте людей к их проверке.

Урок 2: прогнозирование потенциальных отказов

Пандемия коронавируса 2020 г. привела к смене парадигмы во многих областях и рынках. Модели ML, которые обычно предполагают, что будущее будет напоминать прошлое, вероятно, пострадали сильнее всего. Мы не утверждаем, что такая компания, как Zillow, должна была предвидеть пандемию. Но, как мы уже говорили, тщательное изучение возможных отказов системы ML представляет собой важнейшую компетенцию при разработке и эксплуатации ML-систем высокого риска. Мы не знаем подробностей структуры управления моделью Zillow, но провал алгоритма iBuying подчеркивает важность постановки трудных вопросов, таких как: «Что произойдет, если стоимость ремонта дома удвоится в течение следующих двух лет?» и «Каковы будут издержки бизнеса, если переплата за дома составит два процента в течение шести месяцев?» Для подобных систем высокого риска следовало перечислить и задокументировать возможные виды отказов, вероятно, под надзором совета директоров, а реальный финансовый риск должен быть понятен всем лицам, принимающим решения на высшем уровне. В своей организации вы должны определить цену ошибок ML и знать, что высшее руководство готово терпеть эти издержки. Возможно, высшее руководство Zillow было точно проинформировано о финансовых рисках iBuying, а может быть и нет. Сейчас мы знаем только, что компания Zillow пошла на огромный риск, но он не окупился.

Урок 3: многое зависит от руководства

Генеральный директор Zillow известен своей склонностью к риску и имеет успешный опыт выигрыша крупных ставок. Но просто невозможно выиграть каждую сделанную ставку. Вот почему мы управляем рисками и стремимся их ограничить при автоматизированном принятии решений, особенно в сценариях высокого риска. В SR 11-7 говорится: «Строгость и сложность проверки должны быть соизмеримы с масштабом использования моделей банком». Zillow не является банком, но анализ Bloomberg за май 2022 г. описывает деятельность компании так: Zillow «пыталась перейти от продажи онлайн-рекламы к управлению хедж-фондом и стремительно растущим строительным бизнесом». Zillow резко повысила материальное воплощение своих алгоритмов, но, похоже, не обеспечила радикального контроля над этими алгоритмами. Как уже отмечалось, бóльшая часть публичных отчетов указывает на то, что Zillow сокращала экспертный контроль над своими алгоритмами во время программы iBuying, а не усиливала надзор. В крупных организациях потребительского кредитования обычно существует отдельная служба управления рисками, наделенная статусом

Книги для программистов: https://t.me/free_books_for_it

и полномочиями, позволяющими препятствовать передаче моделей в производство, а также соответствующим бюджетом и персоналом. Она подчиняется непосредственно совету директоров и действует независимо от бизнес- и технологических функций, возглавляемых генеральным директором и техническим директором. Когда это структурное подразделение функционирует в соответствии со своей задачей, оно способствует более объективным и основанным на анализе рисков решениям о безопасности модели ML. Кроме того, оно позволяет избежать конфликтов интересов и предвзятости, обычно возникающих, когда руководители бизнеса и лидеры технологических команд оценивают свои собственные системы на предмет риска. Мы не знаем, имела ли Zillow независимую службу управления моделями — в наши дни она довольно редко встречается за пределами потребительского кредитования. Но мы знаем, что никакая служба риска или надзора не смогла остановить программу iBuying до того, как потери стали ошеломляющими. Даже самому продвинутому техническому специалисту трудно вести борьбу в одиночку, поэтому помощь организации в проведении независимого аудита ее систем ML является эффективной стратегией снижения рисков.

Урок 4: инциденты с искусственным интеллектом происходят в больших масштабах

В ситуации с iBuying нет ничего забавного. Потеряны огромные деньги. Разрушено множество карьер — тысячи сотрудников были уволены или ушли в отставку. Мы наблюдаем инцидент с искусственным интеллектом стоимостью 30 млрд долларов. С точки зрения реагирования на инциденты, важно быть готовыми к сбою систем. Для этого необходимо внимательно следить за возможными неполадками и иметь задокументированные и отработанные планы по сдерживанию, устранению и восстановлению. Судя по публичным сообщениям, Zillow действительно знала о текущих проблемах с iBuying, но ее культура была больше ориентирована на крупную победу, чем на подготовку к неудаче. Учитывая размер финансовых потерь, усилия Zillow по сдерживанию могли бы быть более эффективными. Zillow смогла купировать свои наиболее острые проблемы, объявив о списании примерно полумиллиарда долларов в ноябре 2021 г. Что касается этапа восстановления, у руководства Zillow есть планы по созданию нового суперприложения для недвижимости, но, учитывая цену акций на момент написания этой книги, до восстановления еще далеко, а инвесторы устали. Сложные системы всегда движутся к отказу. Возможно, более дисциплинированный подход к обработке инцидентов сможет спасти вашу организацию, если вы сделаете большую ставку на ML.

Последний и самый важный урок, который мы можем вынести из истории о деятельности Zillow, заложен в основу этой книги. Новые технологии всегда сопряжены с рисками. Первые автомобили были особенно опасны. Первые самолеты врезались в склоны гор гораздо чаще. Системы ML могут принимать дискриминирующие решения, создавать риски для безопасности и конфиденциальности и вести себя непредсказуемо. Фундаментальное отличие машинного обучения от других новых технологий заключается в том, что эти системы могут принимать решения

Книги для программистов: https://t.me/free_books_for_it

быстро и в огромных масштабах. Когда Zillow воспользовалась своим алгоритмом Zestimate, она смогла увеличить количество покупок до сотен домов в день. В данном случае результатом стали списание полумиллиарда долларов, распродажа запасов и потеря тысяч рабочих мест. Этот феномен стремительного масштабного провала может иметь еще более разрушительные последствия, когда целью является доступ к капиталу, программам социального обеспечения или решение о том, кто получит новую почку.

Ресурсы

Дополнительное чтение

- Стандарты ISO для ИИ: **https://oreil.ly/N3WTp**
- Система управления рисками ИИ NIST: **https://oreil.ly/mQ8aW**
- Руководство по управлению модельными рисками SR 11-7: **https://oreil.ly/AANIg**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 2

Интерпретируемость и объяснимость

Машинное обучение

Ученые веками строили модели данных в стремлении больше узнать о наблюдаемых закономерностях. Объяснимые модели машинного обучения и апостериорное объяснение моделей машинного обучения представляют собой несколько запоздалое, но очень важное достижение в этой области. Поскольку модели ML легче и точнее изучают нелинейные, слабые и взаимодействующие сигналы по сравнению с традиционными линейными моделями, люди, использующие объяснимые модели ML и методы апостериорного объяснения, теперь также могут с большей легкостью исследовать вышеупомянутые сигналы в своих данных.

В этой главе мы углубимся в важные идеи интерпретируемости и объяснимости, прежде чем перейти к базовым объяснимым методам моделирования и апостериорным объяснениям. Мы также рассмотрим основные ошибки апостериорного объяснения, многие из которых можно исправить, используя одновременно объяснимые модели и апостериорное объяснение. Далее мы обсудим практические вопросы применения объяснимых моделей и апостериорных объяснений, а именно документирование моделей и действенные меры в случае неправильных решений, которые повышают ответственность за системы ИИ. Глава завершится обсуждением так называемого «скандала уровня А» в Соединенном Королевстве (Великобритания), когда объяснимая, хорошо задокументированная модель приняла необъяснимые решения, что привело к инциденту с искусственным интеллектом национального масштаба. Обсуждение объяснимых моделей и апостериорных объяснений продолжается в главах 6 и 7, где мы исследуем два подробных примера кода, связанных с этими темами.

Схема соответствий NIST AI RMF

Раздел главы	Подкатегория NIST AI RMF
Важные принципы интерпретируемости и объяснимости	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, GOVERN 1.5, GOVERN 3.2, GOVERN 4.1, GOVERN 5.1, GOVERN 6.1, MAP 2.3, MAP 3.1, MAP 4, MEASURE 1.1, MEASURE 2.1, MEASURE 2.8, MEASURE 2.9, MEASURE 3.3
Объяснимые модели	GOVERN 1.1, GOVERN 1.2, GOVERN 6.1, MAP 2.3, MAP 3.1, MEASURE 2.8, MEASURE 2.9, MANAGE 1.2, MANAGE 1.3
Апостериорное объяснение	GOVERN 1.1, GOVERN 1.2, GOVERN 1.5, GOVERN 5.1, GOVERN 6.1, MAP 2.3, MAP 3.1, MEASURE 2.8, MEASURE 2.9, MANAGE 1.2, MANAGE 1.3

- Здесь применяются следующие характеристики надежности ИИ: достоверный и надежный, безопасный и отказоустойчивый, прозрачный и подотчетный, объяснимый и интерпретируемый.

Книги для программистов: https://t.me/free_books_for_it

♦ Смотрите также:

- «Четыре принципа объяснимого искусственного интеллекта», **https://oreil.ly/IIf_J**
- «Психологические основы объяснимости и интерпретируемости в искусственном интеллекте», **https://oreil.ly/1Ah96**
- полная таблица соответствий (не является официальным ресурсом), **https://oreil.ly/61TXd**.

Важные принципы интерпретируемости и объяснимости

Прежде чем приступить к методам обучения объяснимых моделей и генерации постфактум-объяснений, нам нужно обсудить важные идеи, лежащие в основе математического аппарата и кода. Начнем с утверждения о том, что прозрачность не означает доверие. Мы можем доверять вещам, которых не понимаем, и понимать то, чему не доверяем. Проще говоря, прозрачность способствует пониманию, но понимание отличается от доверия. На самом деле улучшение понимания плохо построенных систем машинного обучения может даже снизить доверие к ним.

Надежность в области ИИ определяется Национальным институтом стандартов и технологий (NIST) с использованием различных характеристик: достоверность, надежность, безопасность, ограниченная предвзятость, отказоустойчивость, прозрачность, подотчетность, объяснимость, интерпретируемость и повышенная конфиденциальность. Прозрачность облегчает достижение других желаемых характеристик надежности и упрощает отладку. Однако ответственность за повышение прозрачности модели полностью возлагается на оператора системы. На практике надежность часто достигается посредством процессов тестирования, мониторинга и апелляций (главы 1 и 3). Повышенная прозрачность, обеспечиваемая объяснимыми моделями машинного обучения и апостериорными объяснениями, должна облегчить процессы диагностики и отладки, помогающие сделать традиционные линейные модели заслуживающими доверия. Это также означает, что приложения высокого риска, которые десятилетиями основывались на линейных моделях из-за требований к индивидуальному пониманию и детальной документации, теперь, вероятно, созрели для замены точными и прозрачными моделями машинного обучения.



Тот факт, что модель интерпретируема, объяснима или иным образом прозрачна, не делает ее хорошей или заслуживающей доверия. В любом случае именно за людьми остается окончательное обоснованное решение относительно того, является ли модель хорошей или заслуживающей доверия.

Наиболее важным аспектом повышения доверия к объяснимым моделям являются четко оформленные процедуры возмещение ущерба субъектам, пострадавшим от неверных решений, принятых с применением ML. Способность логически обжаловать автоматизированные решения иногда называют *действенным средством правовой защиты* (actionable recourse). Потребителям (а также соискателям работы, пациентам, заключенным или студентам) очень сложно обжаловать автоматические

Книги для программистов: https://t.me/free_books_for_it

и необъяснимые решения. Объяснимые модели ML и методы апостериорного объяснения должны позволять субъектам принятия решений понимать автоматизированные решения на основе ML, что является первым шагом в процессе логической апелляции. Если пользователи смогут продемонстрировать ошибочность входных данных или логики работы модели, операторы систем ML будут вынуждены отменить первоначальное ошибочное решение.



В системах ML высокого риска всегда должны присутствовать механизмы, позволяющие обжаловать и отменять автоматические решения.

Также важно различать интерпретируемость и объяснимость. В недавней работе «Психологические основы объяснимости и интерпретируемости в искусственном интеллекте» (*https://oreil.ly/yz6GF*) исследователи из NIST смогли установить различие между интерпретируемостью и объяснимостью, используя широко распространенные представления о человеческом познании. По мнению исследователей NIST, схожие, но не идентичные концепции интерпретации и объяснения определяются следующим образом.

Интерпретация

Высокоуровневое, значимое мысленное представление, которое контекстуализирует стимул и использует базовые знания человека. Интерпретируемая модель должна предоставлять пользователям описание того, что означает точка данных или выходные данные модели *в контексте*.

Объяснение

Низкоуровневое детальное ментальное представление, призванное описать сложный процесс. Объяснение машинного обучения — это описание того, *как появился* конкретный механизм модели или результат.



Интерпретируемость — это гораздо более высокая планка, чем объяснимость. Достижение интерпретируемости означает помещение механизма или результата ML в контекст, а это редко получается с помощью моделей или апостериорных объяснений. Интерпретируемость обычно достигается за счет понятных письменных объяснений, убедительных визуализаций или интерактивных графических пользовательских интерфейсов. Интерпретируемость обычно требует работы с экспертами как в соответствующей области, так и в сфере взаимодействия с пользователями и их опытом, а также с другими междисциплинарными специалистами.

Давайте теперь уточним детали и попробуем выяснить, что делает модель ML интерпретируемой или объяснимой. Очень сложно добиться от модели прозрачности, если ее входные данные беспорядочны. С этого мы и начнем обсуждение. При рассмотрении входных данных и прозрачности необходимо принимать во внимание следующие аспекты.

Объяснимое извлечение признаков

Если вашей главной целью является прозрачность, избегайте слишком сложного механизма извлечения признаков. Хотя глубокие признаки, полученные из авто-

Книги для программистов: https://t.me/free_books_for_it

кодировщиков, методом главных компонент или высокоуровневых взаимодействий могут улучшить работу модели на тестовых данных, объяснить такие признаки будет сложно, даже если добавить их в модель, хорошо объяснимую иначе.

Значимые признаки

Использование признака в качестве входных данных для некоторой функции модели предполагает, что он связан с выходными данными функции, т. е. с прогнозами модели. Использование бессмысленных или слабо связанных признаков, даже если они улучшают работу модели на тестовых данных, нарушает фундаментальное предположение о том, как работают модели. Например, если наряду с другими признаками мы используем для прогнозирования платежеспособности цвет глаз заемщика, модель, скорее всего, достигнет сходимости при обучении, и мы сможем рассчитать значения аддитивного объяснения Шепли (SHAP) для цвета глаз. Но цвет глаз не имеет реального значения в этом контексте и не демонстрирует причинно-следственную связь со способностью выплачивать кредит. Хотя цвет глаз может служить показателем скрытых предубеждений системы, утверждение о том, что цвет глаз объясняет просрочку по кредитам, неверно. Применяйте здравый смысл или, что еще лучше, причинно-следственный подход к используемым признакам, чтобы повысить достоверность моделей и связанных с ними объяснений.

Монотонные признаки

Монотонность способствует объяснимости. По возможности используйте признаки, которые имеют монотонную связь с целевой переменной. При необходимости примените метод извлечения признаков, такой как биннинг (binninig, разбиение на интервалы), чтобы добиться монотонности.

Если вы уверены, что ваши данные пригодны для целей объяснимости и интерпретируемости, то на следующем шаге для обеспечения максимальной прозрачности модели можно применить такие концепции, как аддитивная независимость входных данных, ограничения, линейность и гладкость, прототипы, разреженность и суммирование:

Аддитивность входных данных

Разделение входных данных в модели ML или ограничение их взаимодействия небольшой группой имеет решающее значение для прозрачности. Традиционным моделям машинного обучения свойственно объединять и рекомбинировать входные признаки в неразборчивый клубок взаимодействий. Напротив, традиционные линейные модели рассматривают входные данные независимым и аддитивным образом. Выходной сигнал линейной модели обычно представляет собой простую линейную комбинацию изученных параметров модели и значений входных признаков. Конечно, качество работы традиционных линейных моделей часто заметно хуже по сравнению с традиционными непрозрачными моделями ML.

Введем понятие *обобщенной аддитивной модели* (generalized additive model, GAM). GAM сохраняют независимость входных признаков, обеспечивая про-

Книги для программистов: https://t.me/free_books_for_it

зрачность, но также позволяют моделировать сколь угодно сложное поведение каждого признака, что значительно повышает качество модели. В следующем разделе мы также обсудим ближайших потомков GAM — GA2M и *объяснимые бустерные машины* (explainable boosting machines, EBM). Все они работают, сохраняя независимость входных данных и позволяя визуализировать способ обработки этих раздельных входных данных (иногда очень сложный). Благодаря этому они сохраняют высокую прозрачность, поскольку пользователям не нужно разбираться в переплетении данных и их взаимном влиянии, а выходное решение по-прежнему представляет собой линейную комбинацию изученных параметров модели и некоторой функции, применяемой к значениям входных данных.

Ограничения

Традиционные необъяснимые модели машинного обучения ценятся за свою гибкость. Они способны смоделировать практически любую функцию генерации сигналов, скрытую в обучающих данных. Тем не менее, когда дело доходит до прозрачности, моделирование каждого изгиба некоторой наблюдаемой функции отклика обычно не является хорошей идеей. Из-за переобучения оказывается, что это также очень плохо влияет на качество прогнозов модели, когда она сталкивается с незнакомыми данными. Дело в том, что иногда наблюдаемые нюансы обучающих данных — это просто шум или даже старая добрая человеческая ошибка. Таким образом, вместо переобучения на плохих данных часто имеет смысл применить ограничения, которые одновременно повышают прозрачность и помогают повысить качество прогнозов на незнакомых данных, заставляя модель следовать заложенной в данных причинности, а не шуму.

При обучении моделей можно применять любое количество ограничений, но некоторые из наиболее полезных и широко доступных — это ограничения *разреженности*, *монотонности* и *взаимодействия*. Ограничения разреженности, часто реализуемые с помощью регуляризации L1 (которая обычно уменьшает количество параметров или правил в модели), увеличивают акцент на более управляемом количестве входных параметров и внутренних механизмах обучения. *Положительные монотонные ограничения* означают, что по мере увеличения входных данных модели ее выходные данные никогда не должны уменьшаться. *Отрицательные монотонные ограничения* гарантируют, что по мере увеличения входных данных модели ее выходные данные никогда не будут увеличиваться. Ограничения взаимодействия не позволяют внутренним механизмам моделей ML объединять и рекомбинировать слишком много различных признаков. Эти ограничения можно использовать, чтобы побудить модели изучать интерпретируемые и объяснимые причинные явления, а не концентрироваться на ненадежных входных данных, случайных нелинейностях и взаимодействиях высокой степени, которые могут стать причинами ошибок и предвзятости в результатах модели.

Линейность и плавность

Линейные функции по умолчанию монотонны и могут быть описаны отдельными числовыми коэффициентами. Гладкие функции дифференцируемы, т. е. их

Книги для программистов: https://t.me/free_books_for_it

можно суммировать в любом месте с помощью производной функции или значения. По сути, линейные и гладкие функции ведут себя лучше, и их обычно легче суммировать. Напротив, неограниченные и произвольные функции машинного обучения могут менять свое поведение способами, которые не поддаются человеческому пониманию, что лишает нас возможности сделать обобщения, необходимые для объяснения работы модели.

Прототипы

Прототипами обычно называют хорошо понятные точки данных (строки) или архетипические признаки (столбцы), которые можно использовать для объяснения результатов модели при работе с незнакомыми данными. Прототипы часто упоминаются в машинном обучении и десятилетиями используются для объяснения и интерпретации решений модели. Возьмем, к примеру, объяснение предсказания модели *k*-ближайших соседей с помощью анализа ближайших соседей или профилирование кластеров на основе местоположения центроидов — это и есть прототипы. Прототипы также важны для построения контрфактических объяснений. Прототипы встречаются даже в обычно сложном и непрозрачном мире компьютерного зрения, где применяется глубокое обучение, основанное на визуальной схожести.

Разреженность

Модели машинного обучения, к лучшему это или к худшему, теперь можно обучать с использованием триллионов параметров. Тем не менее остается спорным вопрос о том, смогут ли отвечающие за их работу люди-операторы рассуждать, основываясь на более чем нескольких десятках этих параметров. Для достижения прозрачности информацию в современной модели ML необходимо *обобщать* (summarize). Как правило, чем меньше коэффициентов или правил в модели ML, тем она более разрежена и объяснима.

Обобщение

Обобщение может принимать разные формы, включая показатели переменной важности, суррогатные модели и другие постфактум-подходы ML. Визуализация, пожалуй, является наиболее распространенным средством передачи обобщенной информации о моделях ML, а приближение, используемое для сжатия информации, является ахиллесовой пятой обобщения. Аддитивные, линейные, гладкие и разреженные модели, как правило, легче обобщать, и в этих случаях процессы апостериорного объяснения имеют больше шансов на успешный результат.

Достижение прозрачности систем ML требует дополнительных усилий по сравнению с популярными и коммерчески успешными необъясняемыми подходами. Не бойтесь этого. В статье «Проектирование интерпретируемых моделей машинного обучения» (*https://oreil.ly/Zv6YO*) и пакетах программного обеспечения, таких как InterpretML, H2O и PiML, заложена отличная основа для обучения и объяснения прозрачных моделей, а в оставшейся части этой главы освещаются наиболее эффективные технические подходы и распространенные ошибки, которые нужно иметь в виду, когда вы приступаете к проектированию системы искусственного интеллекта.

Книги для программистов: https://t.me/free_books_for_it

Объяснимые модели

На протяжении десятилетий многие исследователи и практики ML придерживались, казалось бы, логичного предположения, что более сложные модели более точны. Однако, как отметила выдающийся профессор Синтия Рудин в своей яркой статье «Перестаньте объяснять “черный ящик” машинного обучения, принимающий ответственные решения, и вместо этого используйте интерпретируемые модели» (*https://oreil.ly/i4syT*), «это миф, что неизбежно приходится соглашаться на компромисс между точностью и интерпретируемостью». Позже в этой главе мы углубимся в противоречия между объяснимыми моделями и объяснениями постфактум. А пока давайте сосредоточимся на мощной идее точных и объяснимых моделей. Они обладают потенциалом высокоточного принятия решений, особенно в сочетании с действенными процессами обращения за помощью и соблюдением нормативных требований, улучшенной безопасностью и лучшей способностью решать различные проблемы, включая неточность и предвзятость. Эти привлекательные характеристики делают объяснимые модели ML беспроигрышным вариантом для организаций и их клиентов.

Далее мы рассмотрим некоторые из наиболее популярных типов объяснимых моделей машинного обучения. Мы начнем с большого класса аддитивных моделей, включая модели регрессии со штрафом, GAM, GA2M и EBM. Мы также рассмотрим деревья решений, ансамбли деревьев с ограничениями и множество других вариантов, а затем перейдем к методам апостериорного объяснения.

Аддитивные модели

Вероятно, наиболее широко используемые типы объяснимых моделей ML основаны на традиционных линейных моделях: регрессии со штрафом, GAM и GA2M (или EBM). В этих моделях задействованы современные методы, дополняющие традиционные подходы к моделированию и часто приводящие к заметному улучшению точности и безопасности. Но они также поддерживают высокий уровень интерпретируемости, рассматривая входные признаки раздельно и аддитивно или допуская лишь небольшое количество взаимодействий. В них также применяются простые методы визуализации для повышения интерпретируемости.

Регрессия со штрафом

Обсуждение объяснимых моделей мы начнем с *регрессии со штрафом* (penalized regression). Эта разновидность регрессии обновляет типичный подход, разработанный еще в XIX веке, до уровня технологий XXI века. Модели этого типа обычно создают линейные, монотонные функции отклика с глобально объяснимыми результатами, как и в традиционных линейных моделях, но часто с более высокой прогнозной эффективностью. Модели регрессии со штрафом избегают основанных на предположениях подходов нормальных уравнений для поиска параметров модели и вместо этого опираются на более сложные итеративные процедуры оптимизации с ограничениями, которые позволяют обрабатывать корреляцию, выбирать

Книги для программистов: https://t.me/free_books_for_it

признаки и исключать выбросы, используя при этом валидационные данные для автоматического выбора лучшей модели.

На рис. 2.1 показано, как модель регрессии со штрафом изучает оптимальные коэффициенты для шести входных признаков в течение более чем 80 итераций обучения. Мы видим, что в начале процедуры оптимизации все значения параметров начинаются с очень малых значений и растут по мере того, как процедура начинает сходиться. Это происходит потому, что обычно процедура обучения начинается с больших штрафов за входные признаки. Эти штрафы обычно уменьшаются по мере продолжения обучения, чтобы позволить войти в модель лишь небольшому количеству входных данных, принудительно сохранить малые значения коэффициентов модели или достичь и того, и другого. (Читатели также могли слышать о коэффициентах регрессии со штрафом, которые иногда называют «сжатыми».) На каждой итерации, когда в модель вводится больше признаков или когда значения коэффициентов растут или изменяются, текущую модель испытывают на валидационных данных. Обучение продолжается в течение заранее определенного количества итераций или до тех пор, пока точность прогнозов модели, полученных на валидационных данных, не перестанет улучшаться.

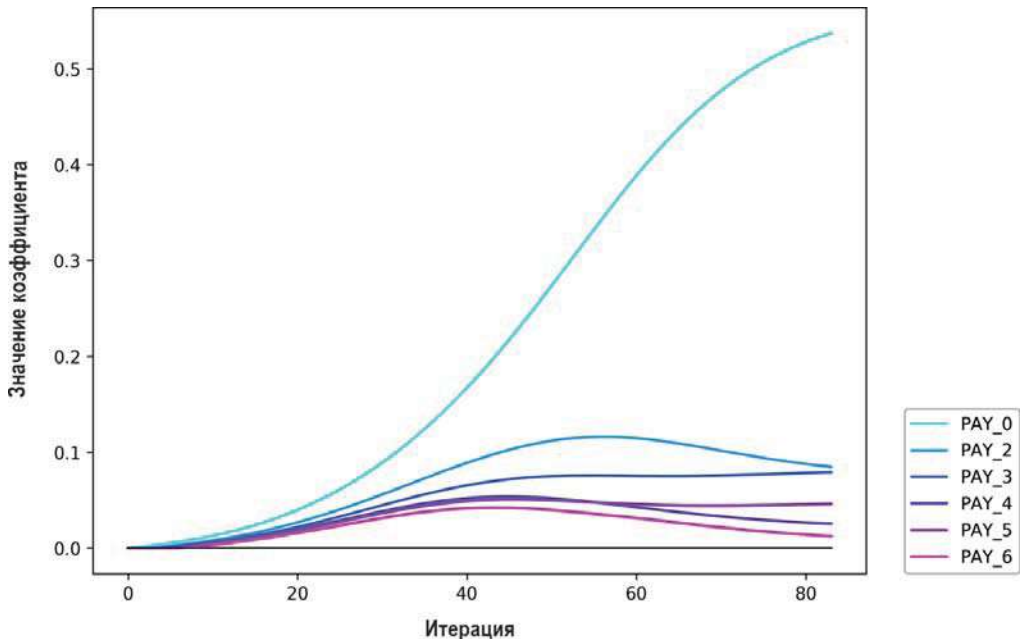


Рис. 2.1. Траектории регуляризации для выбранных признаков в модели эластичной сетевой регрессии (цветное изображение доступно по ссылке **https://files.alistbooks.kz/3060_Hall_ML.zip**)

В дополнение к процедуре досрочной остановки на основе валидации регрессия со штрафом устраняет влияние выбросов, корреляции входных данных и нелинейности с использованием соответственно метода *итеративно перевзвешенных наименьших квадратов* (iteratively reweighted least squares, IRLS), двух видов штрафов по нормам L1 и L2 и функции связи:

Книги для программистов: https://t.me/free_books_for_it

IRLS

IRLS — это хорошо зарекомендовавшая себя процедура минимизации влияния выбросов. Все начинается как в традиционной регрессии, но после первой итерации процедура IRLS проверяет, какие строки входных данных приводят к бóльшим ошибкам. Затем она уменьшает вес этих строк в последующих итерациях подгонки коэффициентов модели. IRLS продолжает процедуру подгонки и уменьшения веса до тех пор, пока параметры модели не придут к стабильным значениям.

Штраф по норме L1

Штрафы L1, также известные как *оператор наименьшего абсолютного сжатия и выбора* (least absolute shrinkage and selection operator, LASSO), сводят сумму абсолютных параметров модели к минимуму. Этот штраф приводит к обнулению ненужных параметров регрессии и выбору небольшого репрезентативного подмножества признаков для модели регрессии, а также позволяет избежать потенциальных проблем множественного сравнения, которые возникают при более старом пошаговом выборе признаков. Известно, что при отдельном использовании штрафы L1 повышают качество модели в ситуациях, когда большое количество потенциально коррелированных входных признаков приводит к невозможности пошагового выбора признаков.

Штраф по норме L2

Штрафы L2, также известные как *гребневая регрессия* или *регрессия Тихонова*, минимизируют сумму квадратов параметров модели. Штрафы L2 стабилизируют параметры модели, особенно при наличии корреляции. В отличие от штрафов по норме L1, штрафы по норме L2 не связаны с отбором признаков. Вместо этого они ограничивают влияние каждого признака на общую модель, сохраняя все параметры модели меньшими, чем они были бы в традиционном решении. Меньшие параметры затрудняют доминирование какого-либо одного признака в модели и предотвращают странное поведение модели во время обучения, связанное с корреляцией.

Функции связи

Функции связи (link function) позволяют линейным моделям обрабатывать общие распределения обучающих данных, например использовать логит-функцию связи для обучения логистической регрессии на входных данных с двумя дискретными результатами. К другим распространенным и полезным функциям связи относятся, например, функция связи Пуассона для счетных данных или инверсная функция связи для выходных данных с гамма-распределением. Совпадение результатов с их семейством распределений и функцией связи, такими как биномиальное распределение и логит-функция связи для логистической регрессии, абсолютно необходимо для обучения развертываемых моделей. Многие модели и библиотеки машинного обучения не поддерживают необходимые функции связи и семейства распределения, позволяющие учитывать фундаментальные предположения в обучающих данных, за пределами пакетов регрессии со штрафом.

Книги для программистов: https://t.me/free_books_for_it

Современные методы регрессии со штрафом обычно сочетают в себе следующие возможности:

- досрочная остановка по результатам валидации во избежание переобучения;
- IRLS для обработки выбросов;
- штрафы L1 для целей выбора признаков;
- штрафы L2 для повышения устойчивости;
- функции связи для различных целевых распределений или распределений ошибок.

Читатели могут узнать больше о регрессии со штрафом в книге «Основы статистического обучения»[1], но для наших целей важнее знать, когда целесообразно ее применять. Регрессия со штрафом широко применяется во многих исследовательских дисциплинах, но она отлично подходит для бизнес-данных с большим количеством признаков и даже для наборов, в которых признаков больше, чем строк, а также для наборов данных с большим количеством коррелирующих переменных. Модели регрессии со штрафом также сохраняют базовую интерпретируемость традиционных линейных моделей, поэтому к ним следует обращаться, когда у вас много коррелированных признаков или нужна максимальная прозрачность. Также важно знать, что методы регрессии не всегда возвращают доверительные интервалы, t-статистику или p-значения для параметров регрессии. Эти типы мер обычно доступны только с применением бутстрап-методов, что может потребовать дополнительного вычислительного времени. Соответствующие пакеты R elasticnet и glmnet поддерживаются изобретателями методов LASSO и эластичной сетевой регрессии, а обобщенная линейная модель H2O очень близка к реализации исходного программного обеспечения, обеспечивая при этом значительно улучшенную масштабируемость.

Еще более новый и чрезвычайно интересный вариант моделей регрессии со штрафом — это *сверхразреженная линейная целочисленная модель* (super-sparse linear integer model, SLIM). В этом подходе тоже применяются сложные процедуры оптимизации с целью создания точных моделей, для оценки которых требуется только простая арифметика. Методика SLIM предназначена для обучения линейных моделей, которые поддаются пониманию людьми, работающими в отраслях повышенного риска, таких как здравоохранение. Если вам довелось столкнуться с приложением, которое требует высочайшей интерпретируемости и необходимости быстрой оценки результатов операционными сотрудниками, обратите внимание на SLIM. С другой стороны, если вы нуждаетесь в более высокой точности прогнозирования на уровне многих необъяснимых методов машинного обучения, но при наличии высокой степени интерпретируемости, то вам могут подойти модели GAM, о которых будет рассказано дальше.

[1] Хасти, Тибришани, Фридман. Основы статистического обучения. Интеллектуальный анализ данных, логический вывод и прогнозирование. М.: Вильямс, 2020.

Книги для программистов: https://t.me/free_books_for_it

Обобщенные аддитивные модели

GAM — это обобщение линейных моделей, которое позволяет подобрать для каждого входного сигнала модели коэффициент и признак, а не просто коэффициент. Подобный способ обучения моделей позволяет обрабатывать каждую входную переменную отдельно, но нелинейно. Обработка каждого входного сигнала по отдельности обеспечивает высокий уровень интерпретируемости. Наличие нелинейности улучшает качество работы. Традиционно в GAM для подгонки нелинейных функций к входным данным применяются сплайны, и большинство реализаций GAM генерируют удобные графики обученных *функций формы* (shape function). При наличии конкретных требований нормативной или внутренней документации вы можете использовать функции формы непосредственно в прогнозных моделях для повышения их интерпретируемости. В ином случае имеет смысл присмотреться к полученным функциям формы и заменить их на похожую, но более объяснимую полиномиальную, логарифмическую, тригонометрическую или другую простую функцию входного признака, которая также может повысить качество прогнозирования. Интересная модификация GAM была недавно получена с помощью *нейронных аддитивных моделей* (*https://oreil.ly/nnCz5*) (neural additive models, NAM) и GAMI-Nets (*https://oreil.ly/G_wCc*). В этих моделях для подбора функций формы используется искусственная нейронная сеть. Мы продолжим тему оценки функций формы с помощью ML, когда будем обсуждать объяснимые бустерные машины в следующем разделе. Исследовательская группа Rudin Group также недавно предложила вариант GAM, в котором монотонные ступенчатые функции используются в качестве функций формы с целью достижения максимальной интерпретируемости. В статье Fast Sparse Classification for Generalized Linear and Additive Models (*https://oreil.ly/tCnld*) они представлены в действии.



Мы будем использовать термин *функция формы* для описания изученной нелинейной взаимосвязи, которую GAM-подобные модели применяют к каждому входу и взаимодействующему признаку. Эти функции формы могут быть традиционными сплайнами или могут быть получены с помощью оценщиков машинного обучения, таких как усиленные деревья или нейронные сети.

Полуофициальное название способности изменять части модели для лучшего соответствия реальности или человеческой интуиции — *редактирование модели* (model editing). Возможность редактирования — еще один важный аспект многих объяснимых моделей. Модели часто выстраивают неверные или предвзятые концепции в результате обучения на ошибочных или предвзятых обучающих данных. Объяснимые модели позволяют пользователям выявлять ошибки и исправлять их. Семейство моделей GAM особенно удобно для редактирования, что является еще одним преимуществом этого мощного подхода к моделированию. Читатели могут узнать больше о GAM в упомянутой ранее книге «Основы статистического обучения». Чтобы попробовать GAM в действии, изучите пакет R gam или более экспериментальные реализации H2O (*https://oreil.ly/5yoAd*) или pyGAM (*https://oreil.ly/1h-Y7*).

Книги для программистов: https://t.me/free_books_for_it

GA2M и объяснимые бустерные машины

GA2M и EBM представляют собой простые, но существенные улучшения GAM. Начнем с GA2M. Цифра 2 в GA2M указывает на использование небольшой группы парных взаимодействий в качестве входных данных для модели. Добавление небольшого количества членов взаимодействия в GAM повышает точность без ущерба для интерпретируемости. Взаимодействующие признаки могут быть отображены в виде контуров вместе со стандартными двумерными графиками входных признаков, которые обычно сопровождают GAM. Некоторые любознательные читатели, возможно, уже знакомы с EBM — важным вариантом GA2M. В EBM функции формы для каждого входного объекта обучаются итеративно с использованием бустинга. Этими функциями отклика могут быть сплайны, деревья решений или даже бустинговые ансамбли самих деревьев решений. Обучая аддитивную модель с использованием бустинга, мы обычно можем получить более точную окончательную модель, чем если бы мы использовали типичный метод обратной подгонки GAM.

Благодаря этим достижениям GA2M и EBM теперь конкурируют на равных или превосходят необъяснимые модели ML по качеству работы с табличными данными, а также обладают очевидными преимуществами интерпретируемости и редактируемости моделей. Если ваш следующий проект связан со структурированными данными, попробуйте EBM с помощью пакета InterpretML (*https://oreil.ly/GMsbK*) от Microsoft Research. EBM станут завершающим штрихом в нашем обсуждении аддитивных моделей. Далее мы переходим к деревьям решений — еще одной разновидности высококачественной модели с высокой интерпретируемостью, имеющей успешный опыт в статистике, интеллектуальном анализе данных и машинном обучении.

Деревья решений

Деревья решений (decision tree) — еще один тип популярной прогнозной модели. При использовании в качестве отдельных деревьев, а не в качестве ансамблевых моделей они образуют легко интерпретируемые блок-схемы на основе обучающих данных и, как правило, демонстрируют лучшую прогностическую эффективность, чем линейные модели. При использовании в составе ансамблей, таких как случайный лес и GBM, они теряют интерпретируемость, но, как правило, становятся еще лучшими предикторами. В следующих разделах мы обсудим как модели с одним деревом, так и ансамбли деревьев решений с ограничениями, которые позволяют сохранять некоторый уровень объяснимости.

Одиночные деревья решений

С технической точки зрения деревья решений представляют собой ориентированные графы, в которых каждый внутренний узел соответствует входному признаку. Для конкретных значений входного признака существуют ребра графа до дочерних узлов, которые создают наивысшую целевую чистоту, т. е. повышенное качество прогнозирования в каждом дочернем узле. Каждый конечный (листовой) узел

Книги для программистов: https://t.me/free_books_for_it

представляет значение целевой переменной при заданных значениях входных признаков, представленных путем от корня к листу. Эти пути можно визуализировать или объяснить с помощью простых правил «если-то».

Проще говоря, деревья решений — это блок-схемы, основанные на данных, как показано на рис. 2.2. Деревья решений отлично подходят для обучения интерпретируемых моделей на структурированных данных. Они полезны, когда цель состоит в том, чтобы понять взаимосвязи между входной и целевой переменной с помощью логики «если-то». Они также отлично умеют находить взаимодействия в обучающих данных. Отношения «родитель-потомок», особенно в верхней части дерева, как правило, указывают на взаимодействие признаков, которое можно использовать для лучшего понимания движущих сил объекта моделирования или в качестве членов взаимодействия для повышения точности прогнозирования аддитивных моделей. Возможно, главным преимуществом деревьев решений перед более простыми аддитивными моделями является их способность обучаться непосредственно на символьных значениях, неполных данных, нестандартизованных и ненормали-

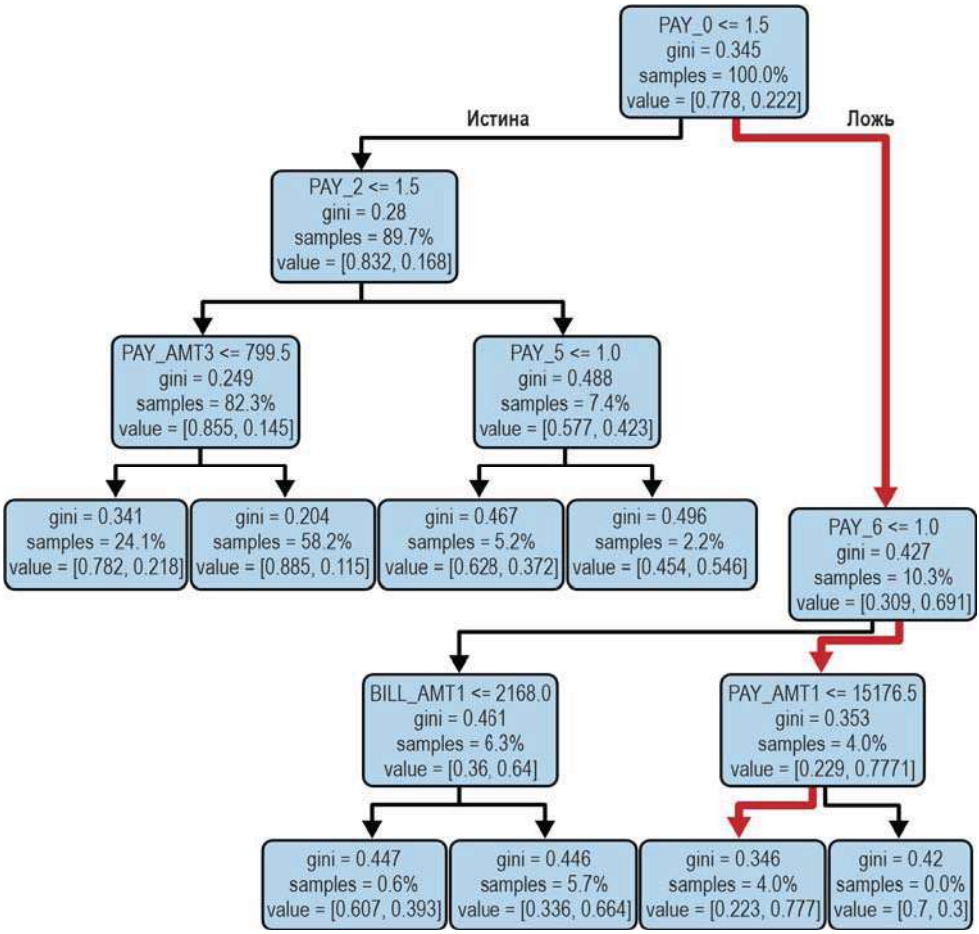


Рис. 2.2. Пример простой модели дерева решений, образующей блок-схему на основе данных

Книги для программистов: https://t.me/free_books_for_it

зованных данных. В наш век больших (читай «плохих») данных деревья решений позволяют строить модели с минимальной предварительной обработкой данных, что помогает устранить дополнительные источники человеческих ошибок в моделях ML.

Если деревья решений так хороши, почему бы нам не использовать только их? Прежде всего, деревья можно интерпретировать только тогда, когда они относительно неглубоки. Деревья решений, имеющие, скажем, более пяти уровней ветвей «если-то», становятся трудными для интерпретации. Они также плохо работают с неструктурированными данными, такими как звук, изображения, видео и текст. Неструктурированные данные стали прерогативой глубокого обучения и нейронных сетей. Как и многие модели, деревья решений нуждаются в трудоемкой точной настройке. Деревья решений имеют множество настраиваемых гиперпараметров, которые приходится указывать с использованием человеческих знаний, поиска по сетке или других методов настройки гиперпараметров. Такие методы как минимум отнимают много времени, а в худшем случае являются источниками систематической ошибки и переобучения. Одиночные деревья решений также нестабильны, поскольку добавление нескольких строк к обучающим данным и переобучение могут привести к полной перестройке блок-схемы. Такая нестабильность является ахиллесовой пятой многих моделей машинного обучения.

Деревья решений принимают локально оптимальные (так называемые *жадные*) решения каждый раз, когда они создают новую ветвь или правило «если-то». Другие модели машинного обучения применяют различные стратегии оптимизации, но конечный результат один и тот же: модель, которая не является действительно лучшей моделью для набора данных, а лишь кажется хорошим кандидатом на роль лучшей модели из множества возможных вариантов. У этой проблемы множества возможных моделей для любого набора данных есть как минимум два названия: *множественность хороших моделей* и *эффект Расёмона*. (Название «Расёмон» взято из известного фильма, в котором свидетели по-разному описывают одно и то же убийство.) Эффект Расёмона связан с другой неприятной проблемой — *недостаточной специфичностью*, когда настройка гиперпараметров и выбор модели на основе проверочных данных дают нам модель, которая выглядит хорошо в тестовых сценариях, но в реальном мире терпит неудачу. Как нам избежать этих проблем с деревьями решений? В основном за счет обучения отдельных деревьев, которые мы можем видеть и исследовать и для которых мы можем гарантировать, что их логика сохранится при их развертывании.

Существует множество вариаций более широкой темы деревьев решений. Например, линейные деревья обучают линейную модель в каждом конечном узле дерева решений, добавляя прогностическую способность к стандартным деревьям решений и сохраняя при этом всю объяснимость. В качестве ответа на проблемы нестабильности и недостаточной специфичности исследовательская группа Rudin Group предложила оптимальные разреженные деревья решений. Другим решением этих проблем являются ручные ограничения, основанные на знаниях предметной области. Далее мы рассмотрим ансамбли деревьев решений с ограничениями, но если читатели готовы попробовать стандартные деревья решений, в их распоряжении

Книги для программистов: https://t.me/free_books_for_it

много программных пакетов, заслуживающих внимания. Одним из лучших считается пакет R rpart.

Ограниченные модели XGBoost

Популярный пакет градиентного бустинга XGBoost теперь поддерживает как ограничения монотонности, так и ограничения взаимодействия. Как упоминалось ранее, ограничения монотонности, заданные пользователем, заставляют модель XGBoost сохранять более объяснимые монотонные взаимосвязи между входными данными модели и ее прогнозами. Ограничения взаимодействия не позволяют XGBoost бесконечно рекомбинировать признаки. Эти программные решения превращают потенциально непрозрачную модель ансамбля деревьев, изобилующую проблемами недостаточной специфичности, в очень надежную модель, способную учиться на данных и принимать причинно-обусловленные ограничения от экспертов. Эти новые варианты обучения позволяют экспертам в предметной области использовать причинно-следственные знания для определения направления моделируемых отношений и указания на то, какие входные признаки не должны взаимодействовать. В сочетании с проверенным опытом масштабируемости и производительности XGBoost это представляет собой достойный выбор, когда возникает потребность в объяснимом ML. Хотя модели XGBoost с ограничениями не так напрямую интерпретируются, как EBM, поскольку *машины градиентного бустинга* (gradient boosting machine, GBM) комбинируют и рекомбинируют признаки в клубок вложенных правил «если-то», эти модели поддаются широкому спектру методов апостериорного объяснения и визуализации. Методы объяснения часто позволяют практикам подтвердить причинно-следственные знания, которые они вложили в модель, и понять внутреннюю работу сложного ансамбля деревьев.

Экосистема объяснимых моделей машинного обучения

Помимо аддитивных и древовидных моделей, существует целая экосистема объяснимых моделей машинного обучения. Некоторые из этих моделей известны уже несколько десятилетий, другие представляют собой модификации старых подходов, а третьи появились совсем недавно. Независимо от того, являются ли они новыми или старыми, эти модели бросают вызов необъяснимому ML, что замечательно. Если читатели удивлены множеством различных вариантов объяснимых моделей, то представьте, как приятно будет клиенту, когда мы сможем объяснить ему причины решений нашей системы искусственного интеллекта, поможем ему подтвердить или опровергнуть гипотезы относительно его бизнеса и предоставим понятные объяснения пользователям, сохраняя при этом высокое качество прогнозов. Сегодня существует множество вариантов объяснимых моделей, и обязательно найдется такая, которая хорошо подойдет для вашего следующего проекта. Вместо того, чтобы просить своих коллег, клиентов или деловых партнеров слепо доверять непрозрачному конвейеру машинного обучения, попробуйте в следующий раз применить объяснимые нейронные сети, алгоритм *k*-ближайших соседей, модели, основанные на правилах, причинно-следственные или графовые модели или даже разреженную матричную факторизацию.

Книги для программистов: https://t.me/free_books_for_it

Причинно-следственные модели

Причинно-следственные модели, в которых причинные явления доказуемым образом связаны с некоторым интересующим прогнозируемым результатом, часто рассматривают как золотой стандарт интерпретируемости. Мы часто можем видеть диаграммы, в соответствии с которыми работают модели Поскольку при создании диаграмм применяются человеческие знания предметной области или методы причинно-следственной связи, такие модели почти автоматически поддаются интерпретации. У них также меньше зашумлены обучающие данные по сравнению с традиционными необъяснимыми моделями машинного обучения. Единственное затруднение, возникающее при разработке причинно-следственных моделей, — это поиск данных, необходимых для их обучения, или сложность самих процессов обучения. Однако возможности обучения причинно-следственных моделей продолжают улучшаться, а такие пакеты, как pyMC3 (*https://oreil.ly/XY448*) для байесовских моделей и dowhy (*https://oreil.ly/yacTr*) для причинно-следственных выводов, уже много лет позволяют специалистам-практикам создавать и обучать причинно-следственные модели. Совсем недавно Microsoft и Uber выпустили учебное пособие по причинно-следственным выводам (*https://oreil.ly/wQu72*) с практическими примерами использования на основе библиотек моделирования обеих компаний, EconML (*https://oreil.ly/Q448j*) и causalml (*https://oreil.ly/Tx12a*) соответственно. Если вас интересуют стабильные, интерпретируемые модели, внимательно следите за этим направлением. Причинно-следственные модели, которые когда-то считались практически не поддающимися обучению, постепенно становятся мейнстримом.

Объяснимые нейронные сети

Выпущенные и усовершенствованные отделом управления рисками банка Wells Fargo, *объяснимые нейронные сети* (explainable neural networks, XNN) служат примером того, как, проявив немного изобретательности и усилий, даже самые необъяснимые модели можно сделать объяснимыми и сохранить высокий уровень качества работы. Объяснимые нейронные сети используют для достижения как высокой интерпретируемости, так и высокой производительности прогнозирования те же принципы, что и GAM, GA2M и EBM, но с некоторыми особенностями. Как и GAM, XNN представляют собой просто аддитивную комбинацию функций формы. Однако они добавляют к GAM дополнительную индексированную структуру. XNN относятся к классу *обобщенных аддитивных индексных моделей* (generalized additive index model, GAIM), где GAM-подобные функции формы подаются на нижний проекционный слой, который может изучать интересные взаимодействия более высокого уровня.

В XNN механизм обратного распространения применяется для изучения оптимальных комбинаций переменных (обозначены как *c* на онлайн-версии (*https://oreil.ly/kBy92*) схемы XNN), которые действуют в качестве входных данных для функций формы, изучаемых через подсети (*b* на онлайн-версии схемы), которые затем объединяются аддитивным образом с оптимальными весами для формирования выходных данных сети (*a* на онлайн-версии схемы). Хотя XNN, вероятно, и не самая простая объяснимая модель, она лучше масштабируется, автома-

Книги для программистов: https://t.me/free_books_for_it

тически определяет важные взаимодействия в обучающих данных, а при помощи апостериорных объяснений можно разбить прогнозы на локально точные вклады признаков. Еще один интересный тип прозрачных нейронных сетей был недавно предложен в статье Neural Additive Models: Interpretable Machine Learning with Neural Nets (*https://oreil.ly/ge-fk*). NAM похожи на XNN, за исключением того, что в них отсутствует нижний слой, который пытается найти условия взаимодействия.

Метод k-ближайших соседей (k-NN)

Метод k-NN использует для прогнозирования прототипы или аналогичные точки данных. Подобные рассуждения не требуют обучения, и их простота часто привлекает пользователей. Допустим, если значение *k* установлено равным трем, то вывод о новой точке включает в себя поиск трех ближайших точек к новой точке и взятие среднего по этим трем точкам в качестве прогнозируемого результата для новой точки. Такая логика часто встречается в нашей повседневной жизни. Возьмем, к примеру, оценку жилой недвижимости. Стоимость квадратного метра площади одного дома часто вычисляют как среднюю стоимость квадратного метра трех наиболее похожих домов. Когда мы говорим «скорее всего», или «по ощущениям», или «похоже», мы опираемся на похожие примеры, чтобы сделать выводы о явлениях в нашей собственной жизни. Идея интерпретации посредством сравнения с прототипами побудила Rudin Group взяться за необъяснимые модели компьютерного зрения с помощью «наибольшего сходства» — нового типа глубокого обучения, который использует сравнения с прототипами для объяснения прогнозов классификатора изображений.

Модели, основанные на правилах

Извлечение прогнозирующих правил «если-то» из наборов данных — еще один давний тип моделирования машинного обучения. Простая булева логика правил «если-то» интерпретируема, если ограничено количество правил, количество ветвей в правилах и количество сущностей в правилах. RuleFit (*https://oreil.ly/xc3-B*) и skope-rules (*https://oreil.ly/3w2BK*) — два популярных метода, которые направлены на поиск прогнозируемых и объяснимых правил на основе обучающих данных. Модели, основанные на правилах, также являются сферой деятельности Rudin Group. Среди их лучших достижений в области интерпретируемых и высококачественных предикторов на основе правил — сертифицируемые оптимальные списки правил (CORELS) (*https://oreil.ly/BgWCt*) и масштабируемые списки байесовских правил. Код для этих и других разработок группы Рудина доступен на их странице с примерами кода (*https://oreil.ly/aYDTE*).

Факторизация разреженной матрицы

Разбиение большой матрицы данных на две меньшие матрицы — это распространенный метод уменьшения размерности и обучения без учителя. Большинство старых методов матричной факторизации переназначают исходные столбцы данных десяткам производных признаков, что делает результаты этих методов необъяснимыми. Однако с введением штрафов L1 в матричную факторизацию теперь можно извлекать из большой матрицы данных новые признаки, где толь-

Книги для программистов: https://t.me/free_books_for_it

ко несколько исходных столбцов имеют большой вес для любого нового признака. Применяя *разреженный анализ главных компонент* (sparse principal components analysis, SPCA) (*https://oreil.ly/xqWFw*), мы можем обнаружить, например, что при извлечении нового признака из финансовых данных клиента он состоит исключительно из показателей соотношения долга к доходу и оборотного остатка на счете в нашем исходном наборе данных. Тогда мы можем сделать вывод, что этот признак связан с потребительским долгом. Или, если мы обнаружим еще один новый признак, имеющий высокий вес для доходов, платежей и расходов, то сможем интерпретировать его как связанный с денежным оборотом. *Факторизация неотрицательной матрицы* (nonnegative matrix factorization, NMF) (*https://oreil.ly/CKydA*) дает аналогичные результаты, но предполагает, что обучающие данные принимают только положительные значения. Это предположение всегда справедливо для неструктурированных данных, таких как количество членов и интенсивность пикселов. Следовательно, NMF можно использовать для поиска объяснимого краткого изложения содержания документов или для разложения изображений на объяснимые словари подкомпонентов. Независимо от того, используем ли мы SPCA или NMF, полученные в результате извлеченные признаки можно использовать в качестве объяснимых сводок, архетипов для объяснимых сравнений, осей для визуализации или функций в моделях. Оказывается, подобно тому, как многие объяснимые модели обучения с учителем являются особыми экземплярами GAM, многие методы обучения без учителя являются экземплярами обобщенных моделей низкого ранга, которые мы можем опробовать в H2O.

Мы рассмотрели основы объяснимых моделей и можем переходить к методам апостериорного объяснения. Но прежде чем мы это сделаем, следует напомнить, что совместное использование объяснимых моделей и апостериорных объяснений является совершенно нормальным. Объяснимые модели часто применяются для включения предметных знаний в механизмы обучения, для устранения ненужных предположений или ограничений в обучающих данных или для создания функциональных форм, которые доступны человеческому пониманию. Апостериорное объяснение обычно применяется для визуализации и обобщения. Хотя апостериорное объяснение часто рассматривают с точки зрения повышения прозрачности традиционных непрозрачных моделей машинного обучения, существует множество причин подвергнуть сомнению это применение, о чем мы расскажем в следующем разделе. Совместное использование объяснимых моделей и апостериорных объяснений для взаимного улучшения и подтверждения может быть лучшим общим применением обеих технологий.

Апостериорное объяснение

Мы начнем с рассмотрения глобальных и локальных мер атрибуции признаков, затем перейдем к суррогатным моделям и популярным типам графиков для описания поведения модели, а также коснемся нескольких методов апостериорного объяснения обучения без учителя. Мы также обсудим недостатки апостериорных объясне-

Книги для программистов: https://t.me/free_books_for_it

ний, которые можно в общих чертах свести к трем моментам. Читатели должны иметь их в виду, формируя собственное мнение о различных методах объяснения.

- Если модель не имеет смысла, ее объяснения тоже не будут иметь смысла. (Мы не можем объяснить чушь.)
- Модели машинного обучения легко становятся настолько сложными, что их невозможно точно обобщить.
- Пояснительную информацию о системах ML трудно донести до широкого круга технически неподготовленных пользователей и других заинтересованных лиц.

Несмотря на эти трудности, апостериорное объяснение почти всегда является необходимым условием обеспечения интерпретируемости и прозрачности. Даже такие модели, как логистическая регрессия, которые считаются в высшей степени прозрачными, нуждаются в обобщении, чтобы соответствовать нормативным требованиям в отношении прозрачности. Нравится нам это или нет, но придется детально разобраться с апостериорными объяснениями и обобщениями. Итак, давайте попробуем заставить их работать как можно лучше.



Интерпретируемость многих моделей основана на обязательном апостериорном обобщении. Тем не менее объяснения ML часто бывают неверны, и их проверка требует тщательного тестирования и сравнения с базовой объяснимой моделью.

Атрибуция и важность признаков

Атрибуция признаков (feature attribution) — один из наиболее важных аспектов объяснения моделей ML. Методы атрибуции признаков сообщают нам, в какой мере входной признак способствовал прогнозам модели либо глобально (по всему набору данных), либо локально (для одной или нескольких строк данных). Значения атрибутов признаков обычно могут быть как положительными, так и отрицательными.

Когда мы говорим о *важности признаков* (feature importance), мы имеем в виду глобальную меру того, насколько каждый признак способствует предсказаниям модели. В отличие от атрибуции признаков, значения важности признаков обычно всегда положительны. То есть значения важности признака измеряют, насколько существенно признак повлиял на общее поведение модели в наборе данных. Атрибуция признаков, с другой стороны, дает нам более подробную информацию о вкладе признака.

Хотя некоторые из более привычных показателей глобальной важности признаков не возникают в результате агрегирования локальных показателей, усреднение (или иное агрегирование) атрибутов локальных признаков в глобальные значения важности признаков сегодня является обычным явлением. Мы начнем с обсуждения новых методов атрибуции локальных признаков, а затем перейдем к глобальным методам.



Глобальные объяснения обобщают механизм модели или прогноз по всему набору данных или большой выборке данных. Локальные объяснения выполняют тот же тип обобщения, но для меньших сегментов данных, вплоть до одной строки или ячейки данных.

Книги для программистов: https://t.me/free_books_for_it

Название «важность признака» может вводить в заблуждение. Мы просто аппроксимируем представление одной модели о том, что важно. Рассмотрим, например, систему безопасности компьютерного зрения, которая основана на градиентных методах обнаружения «важных» аспектов видеокадров. Без надлежащего обучения и требований к развертыванию таким системам будет сложно обнаружить людей в камуфляже. Эта новая цифровая камуфляжная одежда специально разработана для того, чтобы сливаться с любым фоном и сохранять плавные и незаметные визуальные градиенты между тканью и различными фонами. Но разве люди в камуфляже не являются одним из наиболее важных признаков, которые необходимо обнаружить в приложении безопасности? В оставшейся части этого раздела имейте в виду, что «важность признака» во многом зависит от понимания и обучения модели.

Локальные объяснения и атрибуция признаков

В некоторых приложениях крайне важно определить, какие входные признаки повлияли на конкретный прогноз, т. е. измерить атрибуцию локальных признаков. Взгляните на рис. 2.3, где показана атрибуция локальных признаков в действии.

На рис. 2.3 показаны два типа значений атрибуции локальных признаков для двух разных моделей и для трех разных клиентов в примере кредитования. Первый клиент находится в 10-м процентиле вероятности дефолта (невозможности выплатить кредит), и он, скорее всего, получит предлагаемый кредитный продукт. Второй покупатель находится в 50-м процентиле и вряд ли получит предлагаемый продукт. Третий клиент находится в 90-м процентиле вероятности дефолта и представляет собой пример заявителя с чрезвычайно высоким риском. Рассматриваются две модели: обобщенная линейная модель со штрафом (GLM) и GBM с монотонными ограничениями. Поскольку обе модели обладают относительно простыми структурами, которые подходят для решаемой задачи, можно сказать, что мы успешно учли предостережения, упомянутые во вступительных замечаниях по поводу апостериорного объяснения. Объяснения моделей должны быть достоверными и достаточно простыми, чтобы их можно было точно резюмировать.



Один из практических способов сделать объяснения более интерпретируемыми — сравнить их с каким-то значимым эталоном. Вот почему мы сравниваем наши значения атрибуции признаков с корреляцией Пирсона и сравниваем значения нашей частичной зависимости и *индивидуального условного ожидания* (individual conditional expectation, ICE) со средними предсказаниями модели. Это должно помочь нам сравнивать относительно абстрактные поясняющие значения с более понятным эталонным значением.

Как мы резюмируем локальное поведение моделей? Для GLM со штрафом мы умножаем коэффициенты модели на значения входного признака для каждого заявителя. Для GBM мы применяем SHAP. Оба эти метода атрибуции локальных признаков являются аддитивными и локально точными, что означает, что они суммируются с предсказаниями модели. Оба измеряются по смещению: точке пересечения GLM или точке пересечения SHAP, которые схожи по значению, но не равны. (Это не отражено на рис. 2.3.) И оба значения могут быть сгенерированы

Книги для программистов: https://t.me/free_books_for_it

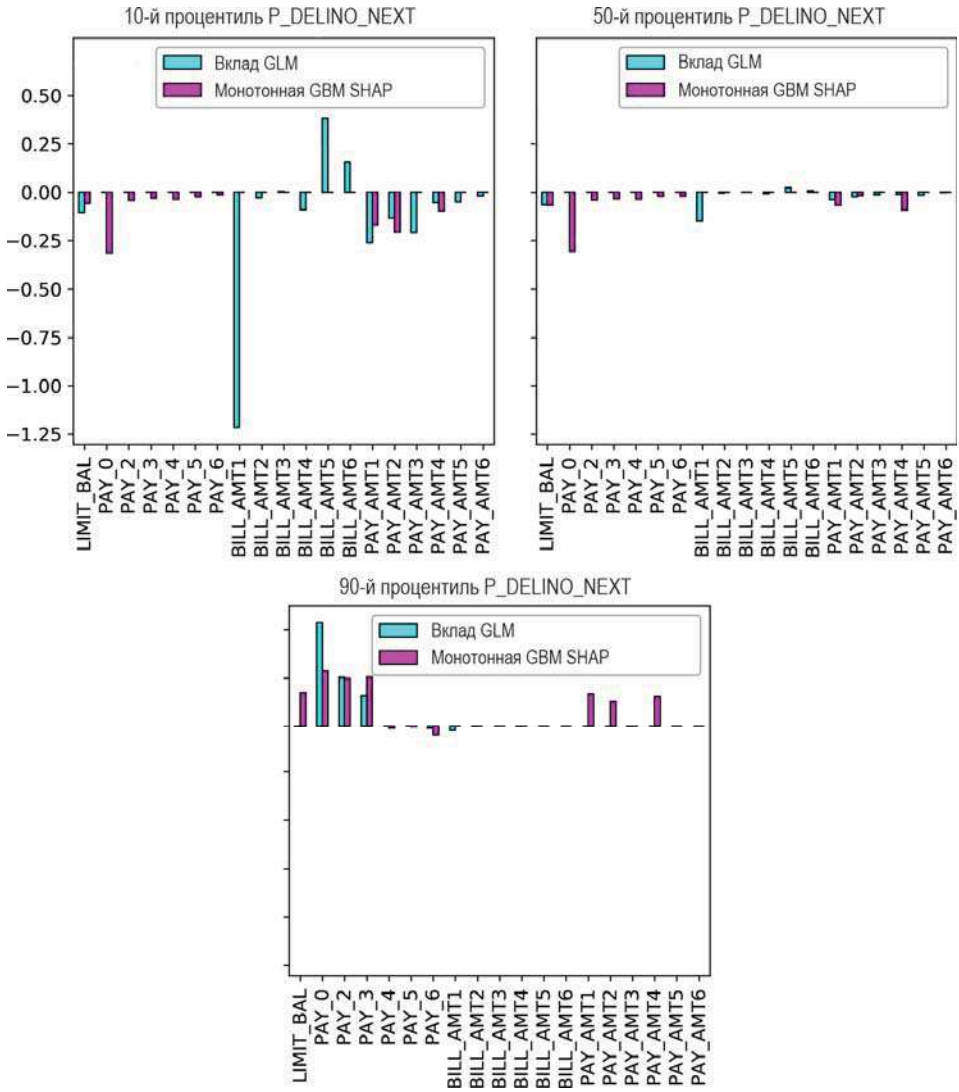


Рис. 2.3. Атрибуция локальных признаков для двух объяснимых моделей для трех человек на 10, 50 и 90-м процентилях прогнозируемых вероятностей

в одном и том же пространстве. В зависимости от способа расчета можно получить либо логарифм шансов, либо прогнозируемую вероятность.

О чем нам говорят показанные на графике значения? В случае заявителя с низким уровнем риска и 10-го процентиля вероятности дефолта мы видим, что его последняя сумма счета `BILL_AMT1` очень благоприятна и снижает его прогноз. Значения SHAP для того же клиента говорят немного иначе, но GBM был обучен на другом наборе признаков. Значения SHAP показывают, что заявитель имеет меньший риск по всем рассматриваемым характеристикам. У заявителя с 50-м процентилем мы видим, что большинство значений атрибуции локальных признаков остаются близкими к соответствующим точкам пересечения, а для заявителя с высоким риском

Книги для программистов: https://t.me/free_books_for_it

почти все значения атрибуции локальных признаков являются положительными, что повышает прогнозы. Обе модели, похоже, сходятся во мнении, что именно недавние статусы платежей заявителя (`PAY_0`, `PAY_2` и `PAY_3`) больше всего влияют на риск, при этом GBM и SHAP также уделяют внимание информации о сумме платежа.

Это всего лишь два типа атрибуции локальных признаков, и нам предстоит обсудить многие другие. Однако наш небольшой пример, вероятно, вызвал у читателей вопросы, на которые нам стоит ответить, прежде чем двигаться дальше. Во-первых, сам процесс создания объяснений не делает эти модели лучше; наличие объяснений просто означает, что мы можем принять более обоснованное решение о том, достаточно ли хороши модели. Во-вторых, две разные модели нередко дают разные объяснения одной и той же строке данных. Но заурядность этого явления не может служить оправданием. Это не то, что мы должны просто принять и пойти дальше. Хотя модели на рис. 2.3, по-видимому, демонстрируют адекватное согласие при работе с одними и теми же признаками, это, к сожалению, лучший сценарий для апостериорного объяснения, и нам так повезло лишь потому, что мы выбрали относительно простые модели и убедились, что знаки коэффициентов GLM и направление монотонных ограничений GBM совпадают с экспертными знаниями в данной области. Для сложных, необъяснимых моделей, обученных на сотнях или более коррелирующих признаков, мы, скорее всего, увидим гораздо меньше согласия между объяснениями моделей, и это должно вызвать тревогу.

Идея о том, что разные модели должны давать одинаковые объяснения одной и той же строке данных, называется *согласованностью* (consistency). Согласованность является разумной целью для приложений высокого риска. Она призвана укрепить доверие к результатам и согласие между несколькими важными системами принятия решений.

Как читатели, вероятно, начинают понимать, давать апостериорное объяснение сложно и обременительно, и именно поэтому мы предлагаем сочетать эти методы с объяснимыми моделями, чтобы модели можно было использовать для проверки объяснений и наоборот. Тем не менее мы продвинулись к описанию основных методов локального объяснения — контрфактических, основанных на градиенте, окклюзии, прототипах, значениях SHAP и других.

Контрфактический метод

Контрфактические объяснения говорят нам, каким должно стать значение входного признака, чтобы изменить результат прогноза модели. Чем больше колебание прогноза при изменении входной переменной на некоторую стандартную величину, тем важнее этот признак, если оценивать его с контрфактической точки зрения. Более подробную информацию можно найти в разделе 9.3 «Объяснения, противоречащие фактам» (*https://oreil.ly/9SNML*) в книге «Интерпретируемое машинное обучение» Кристофа Мольнара. Чтобы попробовать контрфактический метод на практике, ознакомьтесь со статьей DiCE: Diverse Counterfactual Explanations for Machine Learning Classifiers (*https://oreil.ly/s_QaL*) от Microsoft Research.

Книги для программистов: https://t.me/free_books_for_it

Атрибуция объектов на основе градиента

Градиенты можно рассматривать как коэффициенты регрессии для каждой маленькой части сложной функции машинного обучения. В глубоком обучении распространены подходы к локальным объяснениям, основанные на градиентах. При работе с изображениями или текстовыми данными градиенты часто можно накладывать на входные изображения и текст для создания наглядных объяснений, показывающих, какие части входных данных, если они будут изменены, вызовут наибольшие изменения в выходных данных модели. Принято считать, что различные модификации этой идеи приводят к улучшению объяснений. К ним относятся интегрированные градиенты (*https://oreil.ly/is_C-*), послойное распространение релевантности (*https://oreil.ly/XKJ4B*), deeplift (*https://oreil.ly/6rhO0*) и Grad-CAM (*https://oreil.ly/Zkfeh*). Отличный, высокотехнологичный обзор объяснений методов на основе градиента представлен в работе Анконы и др. Towards better understanding of gradient-based attribution methods for Deep Neural Networks (*https://oreil.ly/h7Lde*). Чтобы узнать, что может пойти не так с этими методами, прочтите статью Sanity Checks for Saliency Maps (*https://oreil.ly/a9fQA*). Мы вернемся к этим идеям в главе 9, когда будем обучать модель глубокого обучения и сравнивать различные методы атрибуции.

Окклюзия

Окклюзия основана на простой и мощной идее удаления признаков из процедуры выработки прогноза модели и отслеживания результирующих изменений в прогнозе. Большое изменение может означать, что признак важен, а небольшое изменение может говорить о небольшой значимости признака. Окклюзия лежит в основе методов SHAP, оставления одиночного признака (leave-one-feature-out, LOFO) и многих других подходов к объяснению, в том числе применяемых в компьютерном зрении и обработке естественного языка. Окклюзию можно использовать для создания объяснений в сложных моделях, когда градиенты недоступны. Конечно, с математической точки зрения удалить входные данные из модели никогда не бывает просто, и требуется большая осторожность, чтобы получить соответствующие объяснения на основе результатов удаления признаков. Авторитетный обзор техник окклюзии и удаления признаков представлен в работе Коверта, Лундберга и Ли Explaining by Removing: A Unified Framework for Model Explanation, где они рассматривают 25 методов объяснения, которые можно связать с окклюзией и удалением признаков (*https://oreil.ly/662aS*).

Прототипы

Прототипы — это экземпляры данных, которые хорошо репрезентативны для больших объемов данных. Прототипы используются для объяснения путем обобщения и сравнения. Распространенным видом прототипа являются центроиды кластера *k*-средних (или другие). Эти прототипы представляют собой среднее представление группы схожих данных. Их можно сравнивать с другими точками данных в их собственном кластере и в других кластерах на основе расстояний и с точки зрения сходства с реальным миром. Реальные данные часто очень неоднородны, и бывает сложно найти прототипы, которые бы хорошо

Книги для программистов: https://t.me/free_books_for_it

представляли весь набор данных. *Критика* (criticism) — это точки данных, которые плохо представлены прототипами. Вместе прототипы и критики создают набор точек, которые можно использовать для обобщения и сравнения, чтобы лучше понять как наборы данных, так и модели машинного обучения. Более того, некоторые типы моделей ML, такие как k-NN и глубокое обучение по сходству, основаны на понятии прототипов, что повышает их общую интерпретируемость. Чтобы узнать больше о прототипах, прочтите главу Мольнара о прототипах и критике (*https://oreil.ly/2IQYd*).

Существуют и другие методы локального объяснения. Читатели, возможно, слышали о treeinterpreter (*https://oreil.ly/VECj5*) или eli5 (*https://oreil.ly/xOkSX*), которые генерируют локально точные аддитивные значения атрибуции для ансамблей деревьев решений. Alethia (*https://oreil.ly/ZauV8*) предоставляет сводные данные моделей и локальные выводы для нейронных сетей со *спрямленной линейной функцией активации* (rectified linear unit, ReLU).

Следующий раздел посвящен обсуждению значений Шепли, одного из самых популярных и строгих типов локальных объяснений, доступных специалистам по данным. Прежде чем двигаться дальше, мы еще раз напомним читателям, что методы апостериорного объяснения, включая SHAP, не являются волшебством. Хотя способность понимать, какие признаки влияют на решения модели ML, является невероятным прорывом, существует множество публикаций, указывающих на проблемы с этими методами. Чтобы получить от них максимальную пользу, необходимо проявлять уравновешенное научное мышление и проводить эксперименты. Используйте объяснимые модели и смоделированные данные для оценки качества и достоверности объяснений. Дает ли выбранная вами методика убедительные объяснения случайным данным? (Если да, то это плохо.) Обеспечивает ли ваш метод устойчивые объяснения, когда данные слегка искажены? (Обычно это хорошо.) Ничто в жизни или ML не идеально, и это, безусловно, справедливо для локальных апостериорных объяснений.

Значения Шепли. Метод значений Шепли разработан лауреатом Нобелевской премии по экономике и математиком Ллойдом Шепли. *Аддитивные объяснения Шепли* (Shapley additive explanations, SHAP) объединяют такие подходы, как LIME, LOFO, treeinterpreter, deeplift и другие, для генерации точных величин значимости локальных объектов, и их можно агрегировать или визуализировать для создания согласованных глобальных объяснений. Помимо собственного пакета Python SHAP и различных пакетов R, SHAP поддерживается в популярных платформах программного обеспечения для машинного обучения, таких как H2O, LightGBM и XGBoost.

SHAP начинается так же, как и многие другие методы объяснения, с очевидного вопроса: каким будет прогноз модели для данной строки без этого признака? Так чем же SHAP отличается от других типов локальных объяснений? В системе с таким же количеством сложных взаимодействий, как и в типичной модели машинного обучения, на этот простой вопрос необходимо отвечать, используя среднее значение всех возможных наборов входных данных, не включающих интересующий

Книги для программистов: https://t.me/free_books_for_it

признак. Эти разные группы входов называются *коалициями*. В случае простого набора данных с двадцатью столбцами это означает, что в среднем рассматривается около полумиллиона различных прогнозов модели для разных коалиций. Теперь повторите этот процесс отбрасывания и усреднения для каждого прогноза в наборе данных, и вы поймете, почему SHAP учитывает больше информации, чем большинство других подходов к локальной атрибуции признаков.

Существует множество разновидностей SHAP, но наиболее популярными являются Kernel SHAP, Deep SHAP и Tree SHAP. Из них Tree SHAP является менее приблизительным, а Kernel и Deep SHAP — более приблизительным. Преимущество Kernel SHAP заключается в том, что его можно использовать с любым типом модели, т. е. он *не зависит от модели*. Он похож на метод *локальных интерпретируемых не зависящих от модели объяснений* (local interpretable model-agnostic explanation, LIME) в сочетании с подходом коалиционной теории игр. Однако при наличии большого количества входных данных Kernel SHAP часто требует несостоятельных приближений для достижения приемлемого времени выполнения. Ядро SHAP также требует указания *фоновых данных*, т. е. данных, которые используются методом объяснения в процессе расчета объяснений, что может иметь большое влияние на окончательные значения объяснения. Deep SHAP также опирается на аппроксимации и может быть менее подходящим, чем более простые в вычислении объяснения на основе градиента, в зависимости от модели и имеющегося набора данных. С другой стороны, Tree SHAP работает быстрее и точнее. Но, как следует из названия, он подходит только для древовидных моделей.



Многие методы объяснения основаны на «фоновых данных», т. е. данных, отдельных от объясняемых наблюдений, которые нужны для расчета объяснений. Например, когда мы вычисляем значения SHAP, мы формируем коалиции, удаляя объекты из данных. Когда нам нужно оценить модель в этой коалиции, мы заменяем недостающие значения путем выборки из фоновых данных. Исходные данные могут иметь большое влияние на объяснения, и их следует выбирать осторожно, чтобы не вступать в противоречие со статистическими предположениями о методах объяснения и обеспечить правильный контекст для объяснений.

Два основных места, где ученые, работающие с данными, склонны ошибаться с Tree SHAP, — это интерпретация самого SHAP и непонимание допущений, присущих различным вариантам метода. Говоря об интерпретации, необходимо понять, что SHAP является смещением от среднего прогноза модели. Значения SHAP рассчитываются относительно этого смещения, где большие значения SHAP означают, что признак заставляет прогноз модели каким-то заметным образом отклоняться от среднего прогноза. Небольшие значения SHAP означают, что данный признак не слишком сильно отклоняет прогноз модели от среднего прогноза. У нас часто возникает соблазн узнать о SHAP больше, чем есть на самом деле. Мы склонны искать причинную или контрфактическую логику в значениях SHAP, а это просто невозможно. Значения SHAP представляют собой средневзвешенное значение вклада признака в прогнозы модели по огромному количеству коалиций. Они не дают причинно-следственных или контрфактических объяснений, и если мы хотим, чтобы они вообще имели смысл, то рассматриваемая модель также должна иметь смысл.

Книги для программистов: https://t.me/free_books_for_it



Значение SHAP можно интерпретировать как разницу результатов модели и среднего прогноза, приписываемую определенному значению входного признака.

Tree SHAP также вынуждает пользователей идти на компромиссы. В зависимости от того, как заполняются признаки, отсутствующие в каждой коалиции (*метод возмущения*), мы выбираем между разными философиями объяснений и разными недостатками.

Если фоновые данные не передаются явно, настройки по умолчанию в Tree SHAP задают возмущения `tree_path_dependent`, которые используют количество обучающих примеров, прошедших по каждому пути дерева, для аппроксимации распределения фоновых данных. Если фоновые данные передаются в Tree SHAP явно, то эти данные отбираются для заполнения недостающих значений признаков в интервенционных возмущениях признаков `interventional`. Дополнительная гибкость выбора фонового набора данных позволяет сделать объяснения более целенаправленными, однако формирование подходящего фонового набора данных может оказаться сложной задачей даже для опытных практиков. Мы поговорим подробнее о выборе подходящего фонового набора данных и о том, какое влияние он может оказать в главе 6.

Помимо дополнительной сложности, основным недостатком интервенционных возмущений признаков является то, что они создают нереалистичные экземпляры данных. Это означает, что когда мы оцениваем атрибуцию признака, мы можем делать это на основе множества ложных наблюдений, которые никогда не наблюдались бы в реальном мире. С другой стороны, возмущение признаков позволяет нам избежать необходимости беспокоиться о коррелированных признаках. Напротив, возмущения признаков `tree_path_dependent` более чувствительны к коррелированным признакам, но они стараются учитывать только реалистичные точки данных.



Из-за общих проблем с корреляцией и информационной перегрузкой хорошие объяснения обычно требуют, чтобы базовая модель обучалась на меньшем количестве некоррелированных признаков, имеющих прямое отношение к цели моделирования. Как сказали авторы превосходной статьи True to the Model or True to the Data? (*https://oreil.ly/ze8_z*), в том числе Скотт Лундберг, создатель SHAP: «В настоящее время лучший случай для атрибуции признаков — это когда искажаемые признаки независимы с самого начала».

Эта паутина предположений и ограничений означает, что нам все равно придется быть осторожными и вдумчивыми, даже при использовании Tree SHAP. Однако мы можем облегчить себе задачу. Корреляция — враг многих объяснимых моделей и методов апостериорного объяснения, и SHAP не является исключением. Мы предпочитаем начинать с разумного количества входных признаков, которые не имеют серьезных проблем мультиколлинеарности. При достаточном везении мы бы нашли эти признаки, используя метод причинно-следственной связи. Затем мы использовали бы знания предметной области, чтобы применить ограничения монотонности к входным признакам в XGBoost. Для нахождения обобщенной значимости признаков мы бы использовали Tree SHAP с возмущением признаков `tree_path_`

Книги для программистов: https://t.me/free_books_for_it

dependent. Для таких приложений, как кредитный скоринг, где надлежащий контекст определяется нормативами регулирующих органов, мы можем использовать интервенционные значения SHAP и фоновые данные. Например, некоторые нормативные указания по поводу объяснения отказов в кредите в США предлагают нам «определить факторы, по которым скоринговый балл кандидата больше всего упал относительно среднего балла по каждому из факторов, достигнутых кандидатами, чей общий балл был на уровне минимального проходного балла или немного выше него». Это означает, что наш фоновый набор данных должен состоять из заявителей с прогнозами чуть выше порогового уровня для получения кредитного продукта.

Критическое применение локальных объяснений и важности признаков. Наиболее критичное применение значений атрибуции локальных признаков, скорее всего, связано с соблюдением нормативных требований. Сейчас в США основным требованием является объяснение отказов в кредите в виде *уведомления о неблагоприятных действиях* (adverse action notice). Ключевым техническим компонентом процесса уведомления о неблагоприятных действиях являются *коды причин* (reason code). Коды причин представляют собой текстовые объяснения прогноза модели, описанные с точки зрения входных признаков. Они представляют собой шаг вперед по сравнению с атрибуцией локальных признаков, в которой необработанные значения атрибуции локальных признаков сопоставляются с причинами, по которым может быть отказано в кредите. Затем потребителям должна быть предоставлена возможность просмотреть коды причин своего отрицательного прогноза и следовать установленной процедуре апелляции, если входные данные или факторы принятия решения явно неверны.

Уведомления о неблагоприятных действиях — это конкретный пример понятия более высокого уровня, отражающего возможность обращения за помощью, когда прозрачные решения модели основаны на факторах, понятных пользователям, могут быть обжалованы пользователями и отменены операторами модели. Многие разрабатываемые нормативные акты, например, в Калифорнии, Вашингтоне и ЕС, скорее всего, будут вводить аналогичные требования к объяснению или обращению за помощью. Если вам приходится работать под пристальным вниманием регулирующих органов или вы просто стремитесь поступать правильно, принимая важные решения относительно других людей, очень важно, чтобы ваши объяснения были как можно более точными, последовательными и интерпретируемыми. Хотя мы ожидаем, что атрибуция локальных признаков станет одним из наиболее удобных технических инструментов для генерации необработанных данных, необходимых для соблюдения норм, наилучшие объяснения получаются при сочетании атрибуции локальных признаков с объяснимыми моделями и другими типами объяснений, подобными тем, которые будут рассмотрены в последующих разделах этой главы.

Глобальная значимость признака

Методы глобальной значимости признаков количественно определяют глобальный вклад каждого входного признака в прогнозы сложной модели машинного обучения по всему набору данных, а не только для одного отдельного человека или строки данных. Глобальные меры значимости признаков не всегда дают представление

Книги для программистов: https://t.me/free_books_for_it

о среднем направлении, в котором переменная толкает обученную функцию ML. По своей сути они просто указывают величину связи признака с откликом модели по сравнению с другими входными признаками. Эта информация никогда не будет лишней, и поскольку большинство глобальных показателей значимости признаков лежат в основе традиционных подходов, их часто ожидают получить команды по проверке моделей. На рис. 2.4 представлен пример диаграмм значимости признаков, на которых мы сравниваем глобальную значимость признаков двух моделей.

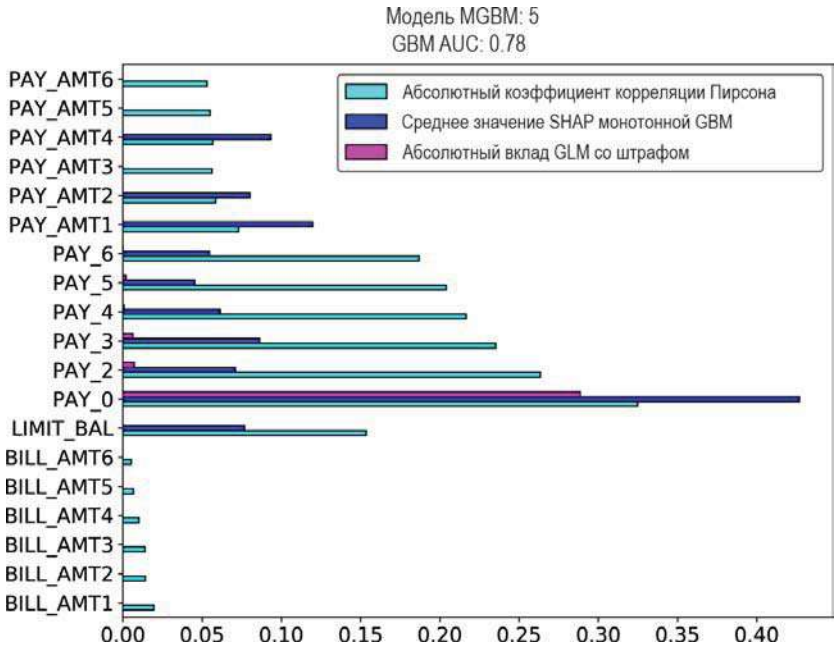


Рис. 2.4. Глобальная значимость признаков для двух объяснимых моделей по сравнению с корреляцией Пирсона

Подобные диаграммы помогают нам ответить на следующие вопросы:

- Имеет ли один порядок значимости признаков больше смысла, чем другой?
- Отражает ли эта диаграмма закономерности, которые, как мы знаем, модели должны были извлечь из обучающих данных?
- Не уделяют ли модели слишком много внимания одному или двум признакам?

Глобальная значимость признаков — это простой способ провести такие базовые проверки. На рис. 2.4 мы сравниваем значимость признаков с корреляцией Пирсона, чтобы иметь некоторое представление о том, какие признаки должны быть важными. Сравнивая корреляцию Пирсона и две модели, мы видим общее согласие с тем, что `PAY_0` является наиболее важным признаком. Однако GLM придает почти всю важность принятия решений `PAY_0`, в то время как GBM распределяет важность по более широкому набору входных данных. Когда модели слишком сосредоточены на одном признаке, как это делает GLM на рис. 2.4, они могут оказаться нестабильными на новых данных, если распределение наиболее важных признаков сме-

Книги для программистов: https://t.me/free_books_for_it

щается, и это облегчает состязательное манипулирование моделью. В случае GLM на рис. 2.4 злоумышленнику достаточно будет изменить значение только одного признака PAY_0, чтобы кардинально изменить прогнозы модели.

Глобальные показатели значимости признаков можно рассчитать разными способами. Многие специалисты, работающие с данными, впервые сталкиваются со значимостью признаков при изучении деревьев решений. Обычный метод определения значимости признаков для деревьев решений заключается в суммировании изменений критерия разделения для каждого ветвления дерева на основе определенного признака. Например, если дерево решений (или ансамбль деревьев) обучено максимизировать прирост информации для каждого разделения, то значимость, присвоенная некоторому входному признаку, представляет собой общий прирост информации, связанный с этим признаком каждый раз, когда он используется в дереве (деревьях). Значимость признаков на основе возмущений — это еще один распространенный способ измерения значимости. Он не зависит от модели, т. е. его можно использовать практически для всех типов моделей ML. При вычислении значимости признаков на основе возмущений интересующий входной признак перемешивается (сортируется случайным образом) и делаются прогнозы. Разница в некоторой оценке, обычно прогнозе модели или среднеквадратической ошибке (MSE), до и после перетасовки интересующего признака является его значимостью. Другие аналогичные методы представляют собой *исключение одного признака* (leave-one-feature-out, LOFO) или *исключение одной ковариаты* (leave-one-covariate-out, LOCO). В методе LOFO признак каким-то образом исключается из обучения или прогнозирования модели — скажем, путем повторного обучения без признака и выполнения прогнозов или путем установки признака как отсутствующего и выполнения прогнозов. Разница в релевантных оценках между моделями с этим признаком и без него считается значимостью LOFO.

Хотя перестановка и LOFO обычно служат для измерения разницы в прогнозах или разницы в показателях точности или ошибки, у них есть важное преимущество — они позволяют оценить влияние признака практически на все, что связано с моделью. Например, вполне возможно вычислить вклад перестановок или LOFO в метрику справедливости, что позволит нам получить представление о том, какие конкретные признаки служат причиной обнаруженной социологической предвзятости. Тот же самый подход можно повторно применить для любого количества показателей, характеризующих модель: функций ошибок, безопасности, конфиденциальности и т. д.



Такие методы, как оценка значимости признака путем возмущений и LOFO, можно использовать для оценки вклада признака во многие другие величины, помимо прогнозов модели.

Поскольку эти методы хорошо зарекомендовали себя, в распоряжении специалистов по данным есть много соответствующей информации и пакетов программного обеспечения. Подробное обсуждение оценки значимости признаков на основе разделения можно найти в главе 3 книги Introduction to Data Mining (*https://oreil.ly/P2gEb*) (Pearson). В разделе 10.13.1 книги Elements of Statistical Learning (*https://*

Книги для программистов: https://t.me/free_books_for_it

oreil.ly/jQOX6) представлена значимость признаков на основе разделения, а в разделе 15.3.2 представлено краткое введение в значимость признаков на основе перестановок в контексте случайных лесов. Пакет R vip (*https://oreil.ly/7Jo9s*) позволяет строить разнообразные *диаграммы значимости переменных*, а с помощью пакета Python lofo-importance (*https://oreil.ly/jP5jV*) можно также реализовать LOFO. Конечно, у большинства методов глобальной значимости признаков есть свои недостатки и слабости. Метод на основе разделения имеет серьезные проблемы с согласованностью, и, как это часто бывает в случае апостериорно объяснимого ИИ (explainable AI, XAI), корреляция приведет нас к заблуждению при использовании подходов на основе перестановок и LOFO. В статье There Is No Free Variable Importance (*https://oreil.ly/bx6QA*) более подробно рассматриваются серьезные проблемы, связанные с глобальной значимостью признаков. Но, как неоднократно упоминается в этой главе, модели с ограничениями и разумным количеством некоррелированных и логических входных данных помогут нам избежать наихудших проблем, связанных с глобальной значимостью признаков.

SHAP также имеет отношение к глобальной значимости признаков. SHAP по своей природе является методом атрибуции локальных признаков, но их можно агрегировать и визуализировать для получения глобальной информации о значимости признаков. Из многих преимуществ SHAP по сравнению с более традиционными показателями значимости признаков наиболее важной, пожалуй, является его интерпретация. При оценке значимости признака с помощью разделения, перестановки и LOFO часто мы видим только относительный порядок значимости входных признаков и, возможно, некоторые качественные представления о том, как признак на самом деле влияет на прогнозы модели. Используя значения SHAP, мы можем рассчитать среднее абсолютное значение атрибутов признаков по набору данных, и эта мера значимости признаков имеет четкую и количественную связь с прогнозами модели по отдельным наблюдениям. SHAP также обеспечивает различные уровни детализации значимости признаков. Хотя SHAP можно напрямую агрегировать в значимость признака, этот процесс может усреднить важную локальную информацию. SHAP открывает возможность проверки значимости признаков на любом уровне, от самого локального уровня (одна строка) до глобального уровня. Например, агрегирование SHAP по важным сегментам, таким как штаты США или разные полы, или использование многочисленных визуализаций в пакете SHAP способно дать более информативное и репрезентативное представление о значимости признаков, чем одно среднее абсолютное значение. Подобно методам перестановки и LOFO, SHAP также можно использовать для оценки значимости величин помимо предсказаний модели. Он способен оценивать вклад в ошибки модели и показатели справедливости, такие как демографический паритет.

На этом мы завершаем обсуждение значимости признаков. Независимо от глобальности или локальности, значимость признаков, вероятно, будет первым методом апостериорного XAI, с которым вы столкнетесь при построении моделей. Как показано в этом разделе, для анализа значимости признаков недостаточно построения гистограмм или запуска SHAP. Чтобы извлечь максимальную отдачу из значимости признаков, нам необходимо знать сильные и слабые стороны многих подходов.

Книги для программистов: https://t.me/free_books_for_it

Далее мы рассмотрим суррогатные модели — еще один интригующий подход к объяснению, также требующий размышлений и осторожности.

Суррогатные модели

Суррогатные модели (surrogate model) — это простые модели сложных моделей. Если мы сумеем построить простую интерпретируемую модель более сложной модели, то сможем использовать объяснимые характеристики суррогатной модели для объяснения, обобщения, описания или отладки более сложной модели. Метод суррогатного моделирования применим практически к любой модели ML. Проблема с суррогатными моделями заключается в том, что они по большей части представляют собой технический трюк с не очень надежными математическими гарантиями соответствия сложной модели, которую они пытаются обобщать. Это означает, что мы должны быть осторожны при использовании суррогатных моделей и как минимум проверять, являются ли они точным и стабильным представлением более сложных моделей. На практике это часто означает проверку различных типов показателей точности и ошибок на разных разделах данных, чтобы гарантировать высокую точность прогнозов более сложной модели, а также то, что она не снижается на новых данных и стабильна во время перекрестной проверки. Суррогатные модели также имеют много названий. Читатели, возможно, слышали о *сжатии моделей* (model compression), *дистилляции моделей* (model distillation) или *извлечении моделей* (model extraction). Все они либо являются методами суррогатного моделирования либо имеют к нему прямое отношение. Существует множество различных типов суррогатных моделей. В следующих разделах мы начнем с суррогатных моделей дерева решений, которые обычно применяются для построения глобальных объяснений, затем перейдем к LIME и якорям, которые обычно предназначены для создания локальных объяснений.

Суррогатные модели дерева решений

Суррогатные модели дерева решений обычно создают путем обучения дерева решений на исходных входных данных и прогнозах сложной модели. Предполагается, что значимости признаков, тенденции и взаимодействия, отображаемые в суррогатной модели, достоверно представляют внутренние механизмы сложной модели. Это не более, чем предположение. Нет никаких теоретических гарантий того, что простая суррогатная модель хорошо репрезентативна для более сложной модели. Но благодаря наглядной структуре деревьев решений эти суррогатные модели создают очень понятные блок-схемы процессов принятия решений внутри более сложной модели, как видно на рис. 2.5. Существуют хорошо описанные методы обучения суррогатных моделей дерева решений, например рассмотренные в работах Extracting Thee-Structured Representations of Thained Networks (*https://oreil.ly/ewW3O*) и Interpretability via Model Extraction (*https://oreil.ly/RnFep*).



Суррогатные модели дерева решений могут быть легко интерпретируемы при создании блок-схемы более сложных моделей.

Книги для программистов: https://t.me/free_books_for_it

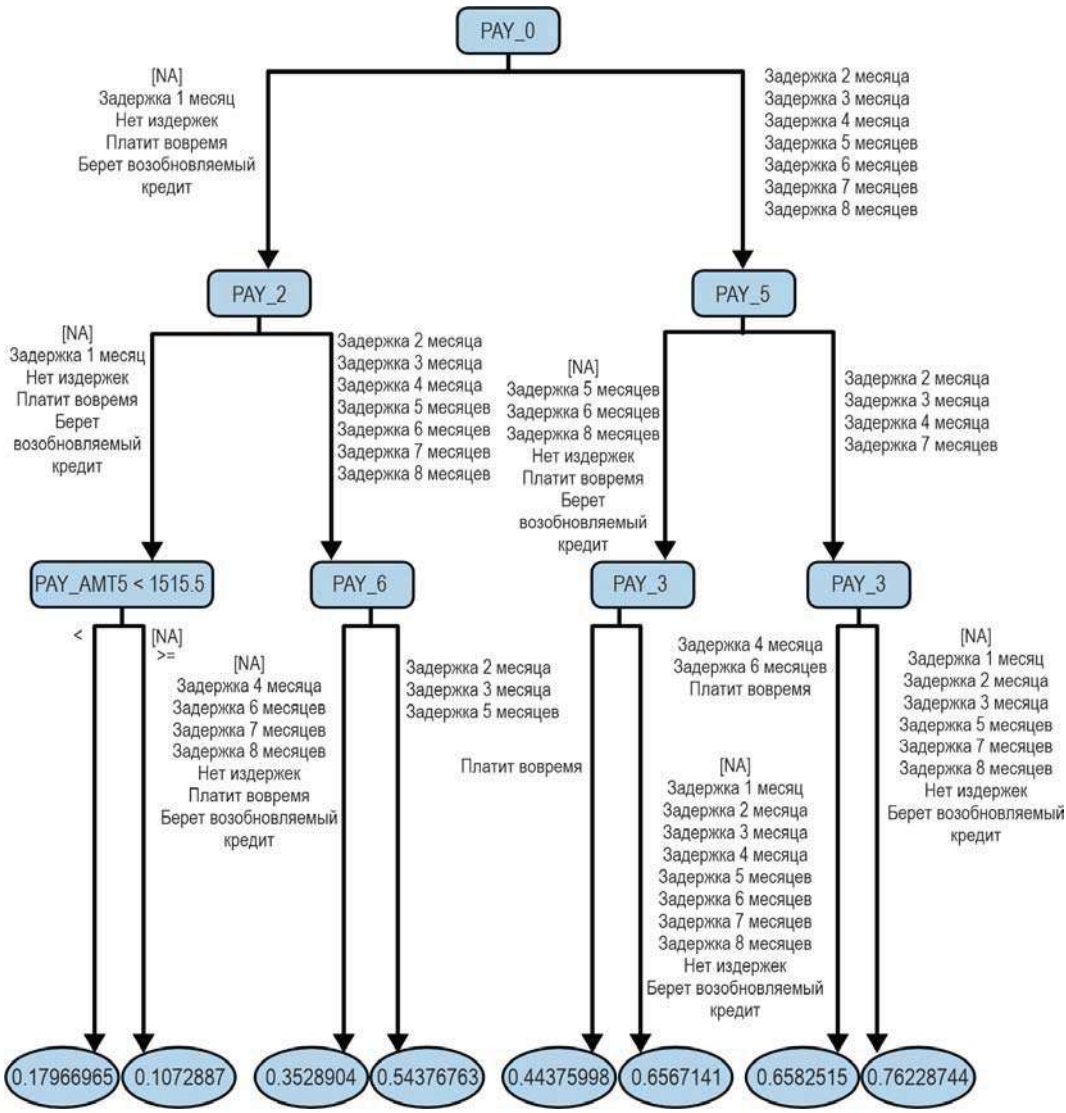


Рис. 2.5. Блок-схема монотонной машины градиентного бустинга, построенная при помощи суррогатной модели дерева решений

На практике обычно достаточно измерить совпадение прогнозов суррогатного дерева с прогнозами сложной модели на интересующем сегменте данных с помощью таких показателей, как логарифмические потери, среднеквадратическая ошибка (RMSE) или R^2, и измерить стабильность этих прогнозов с помощью перекрестной проверки. Если суррогатное дерево решений не может обеспечить точность, сопоставимую с более сложной моделью, то в качестве суррогатов можно рассматривать более сложные объяснимые модели, такие как EBM или XNN.

Суррогатная модель на рис. 2.5 была обучена на тех же входных данных, что и более сложная GBM, которую она пытается обобщить, но вместо обучения на исход-

Книги для программистов: https://t.me/free_books_for_it

ной цели, указывающей на просрочку платежа, она обучается на прогнозах GBM. При интерпретации этого дерева признаки, которые находятся выше или используются чаще, считаются более значимыми в объясненной модели GBM. Признаки, которые находятся выше и ниже друг друга, могут иметь сильные взаимодействия в GBM, а другие методы, обсуждаемые в этой главе, такие как объяснимые бустинговые машины, сравнение частичных зависимостей и ICE, могут использоваться для подтверждения существования этих взаимодействий в обучающих данных или в модели GBM.



Суррогаты дерева решений можно применять при поиске взаимодействий для использования в линейных моделях или для LIME. Для поиска взаимодействий в данных и моделях также могут применяться EBM и различия частичных зависимостей и ICE.

Пути принятия решений в дереве также можно отслеживать, чтобы получить некоторое представление о том, как более сложный GBM принимает решения. Следуя по пути принятия решения от корневого узла на рис. 2.5 до средних прогнозов в нижней части дерева, мы видим, что те, у кого хорошие статусы по последнему (`PAY_0`) и второму (`PAY_2`) последнему погашению, согласно исходной модели, скорее всего, в будущем не столкнутся с просрочкой по несколько большим пятым суммам последних платежей (`PAY_AMT5`). Те клиенты, у которых есть проблемы с последним и пятым с конца статусами платежа, скорее всего, будут иметь проблемы с будущими платежами. (`PAY_3` содержит большое количество шума и здесь не интерпретируется.) В обоих случаях GBM, похоже, учитывает как недавние, так и относительно отдаленные статусы платежей по кредиту, чтобы сделать прогноз о будущих платежах. Такое прогнозирование вполне логично, но по возможности его следует подтверждать другими способами. Как и большинство суррогатных моделей, суррогаты дерева решений полезны и легко интерпретируются, но не должны применяться для важных задач объяснения или интерпретации сами по себе.

Линейные модели и локально интерпретируемые объяснения

LIME — один из самых ранних, самых известных и наиболее критикуемых методов апостериорного объяснения. Как видно из названия (local interpretable model-agnostic explanations, локально интерпретируемые объяснения, не зависящие от устройства модели), он чаще всего используется для генерации локальных объяснений путем подгонки линейной модели к прогнозам некоторой небольшой области прогнозирования более сложной модели. Хотя это наиболее распространенный вариант прикладной реализации, на самом деле он представляет собой упрощение исходного метода.

В статье 2016 г. Why Should I Trust You? Explaining the Predictions of Any Classifier (*https://oreil.ly/e9WL2*) LIME был представлен как фундаментальный метод с несколькими замечательными качествами. Самым привлекательным из них было требование разреженности локальных объяснений. Если наша модель имеет тысячу признаков, и мы применяем SHAP, мы получим тысячу значений SHAP для каждого прогноза, который хотим объяснить. Даже если SHAP идеально подходит для наших данных и модели, нам все равно придется перебирать тысячу значений каж-

Книги для программистов: https://t.me/free_books_for_it

дый раз, когда мы хотим объяснить прогноз. LIME позволяет обойти эту проблему, требуя, чтобы генерируемые объяснения были разреженными, т. е. учитывали небольшое количество локально значимых признаков, а не все признаки, включенные в объясняемую модель.



Значение LIME можно интерпретировать как разницу между прогнозом LIME и соответствующей ему точкой пересечения LIME, приписываемой определенному значению входного признака.

Остальная часть алгоритма LIME выполняет подгонку интерпретируемой суррогатной модели к некоторой взвешенной локальной области прогнозов другой модели. Поэтому более точное и полное описание LIME выглядит так: локально взвешенная интерпретируемая суррогатная модель со штрафом за разреженность, применимая для некоторых произвольных предсказаний более сложной модели. В такой трактовке идея LIME полезна и вполне разумна.



Всегда проверяйте с помощью методов статистики и визуализации, насколько хорошо LIME соответствует базовой функции отклика, и что пересечение локальной модели не объясняет наиболее заметное явление, лежащее в основе данного прогноза.

Вышеупомянутая популярная реализация LIME доставляет неприятности неопытным пользователям и создает проблемы с безопасностью. При работе с табличными данными программный пакет LIME просит пользователя выбрать строку, которую нужно объяснить, генерирует довольно упрощенную выборку данных на основе заданного набора входных данных, взвешивает выборку по выбранной пользователем строке, подгоняет LASSO-регрессию между взвешенной выборкой и предсказаниями более сложной модели на выборке и, наконец, использует коэффициенты LASSO-регрессии для создания объяснений относительно указанной пользователем строки. В этой реализации есть много потенциальных проблем:

- механизм выборки представляет собой проблему для объяснения в реальном времени, поскольку он требует генерации данных и подгонки модели в ходе конвейера оценки, а также позволяет пользователям выполнять атаки по отравлению данных, которые могут изменить объяснения;
- в сгенерированных выборках LIME значительная часть данных может выходить за пределы допустимого диапазона, что ведет к нереалистичным оценкам локальной значимости признаков;
- величины локальной значимости признаков представляют собой отклонения от коэффициента пересечения локальной GLM, и это пересечение иногда может учитывать наиболее значимые локальные явления;
- сильная нелинейность и взаимодействия высокого порядка в выбранной локальной области прогнозов могут привести к полной неработоспособности LIME.

Поскольку LIME можно использовать для создания разреженных объяснений практически для любого типа модели машинного обучения, он все равно может быть хорошим инструментом в вашем наборе, если вы готовы набраться терпения и тща-

Книги для программистов: https://t.me/free_books_for_it

тельно продумать процесс. Если вы решили использовать LIME, постройте график прогнозов LIME в сравнении с прогнозами более сложной модели и проанализируйте их с помощью RMSE, R^2 или аналогичной метрики. Вы должны быть осторожны с коэффициентом пересечения линейной модели LIME и следить за тем, чтобы он не объяснял ваш прогноз сам по себе, делая фактические значения LIME бесполезными. Чтобы повысить точность LIME, попробуйте LIME на дискретных входных признаках и взаимодействиях, созданных вручную. (Попробуйте воспользоваться суррогатом дерева решений, чтобы угадать эти взаимодействия.) Используйте перекрестную проверку для оценки стандартных отклонений или даже доверительных интервалов для значений вклада локальных признаков. И имейте в виду, что плохое соответствие или неточность локальных линейных моделей сами по себе информативны и часто указывают на крайнюю нелинейность или взаимодействие высокого порядка в этой области прогнозов.

Якоря и правила

Вслед за LIME и, вероятно, извлекая из него некоторые уроки, та же группа исследователей предложила еще один метод локального апостериорного объяснения, не зависящий от модели, под названием Anchors (якоря). Этот метод генерирует высокоточные наборы простых правил для описания прогноза модели машинного обучения, уделяя особое внимание поиску наиболее значимых признаков для текущего прогноза. Читатели могут узнать больше о якорях в статье Anchors: High-Precision Model-Agnostic Explanations (*https://oreil.ly/V1rFJ*) и программном пакете anchor (*https://oreil.ly/KNGF3*). Хотя этот метод рекомендован к применению и его сильные и слабые стороны документально подтверждены, он представляет собой лишь особый случай использования моделей, основанных на правилах, в качестве суррогатных моделей. Как было сказано в первой части главы, модели, основанные на правилах, обладают хорошей способностью к обучению нелинейностям и взаимодействиям, но при этом в целом поддаются интерпретации. Многие из упомянутых ранее моделей, основанных на правилах, можно рассматривать в качестве суррогатных.

Графическое представление характеристик модели

Помимо значимости признаков и суррогатных моделей, популярными способами описания поведения обученной модели по отношению к входным признакам стали графические представления *частичной зависимости* (partial dependence), *индивидуального условного ожидания* (individual conditional expectation, ICE) и *накопленных локальных эффектов* (accumulated local effects, ALE). В этом разделе мы рассмотрим частичную зависимость и ICE, убедимся, что частичная зависимость действительно должна использоваться только с ICE, и обсудим ALE как более современную замену частичной зависимости.

Частичная зависимость и индивидуальное условное ожидание

Графики частичной зависимости показывают нам предполагаемый усредненный характер изменения функций отклика, полученных машинным обучением, в зави-

Книги для программистов: https://t.me/free_books_for_it

симости от значений одного или двух интересующих входных признаков, при этом усредняются эффекты всех других входных признаков. Помните об усреднении. Мы еще вернемся к нему. Графики частичной зависимости могут показывать нелинейность, немонотонность и двусторонние взаимодействия в сложных моделях машинного обучения и применяться для проверки монотонности функций отклика, обученных в условиях монотонных ограничений. Частичная зависимость представлена вместе с ансамблями деревьев в книге *Elements of Statistical Learning*, раздел 10.13 (*https://oreil.ly/35vig*). Графики ICE — это новая, локальная и менее известная разновидность графиков частичной зависимости. Они показывают, как ведет себя модель для одной строки данных при изменении одного признака. ICE прекрасно сочетается с частичной зависимостью на одном и том же графике, предоставляя больше локальной информации и дополняя более глобальную информацию, предоставляемую частичной зависимостью. Графики ICE были представлены в статье «Peeking Inside the Black Box: Visualizing Statistical Learning with Plots of Individual Conditional Expectation» (*https://oreil.ly/Vuraz*). В нашем распоряжении есть множество пакетов программного обеспечения, позволяющих опробовать частичную зависимость и ICE. Пользователям Python рекомендуем обратить внимание на PDPbox (*https://oreil.ly/RbzII*) и PyCEbox (*https://oreil.ly/KcdET*). Для пользователей R есть пакеты pdp (*https://oreil.ly/pE0Ue*) и ICEbox (*https://oreil.ly/QbsDA*). Кроме того, многие библиотеки моделирования поддерживают частичную зависимость без использования внешнего пакета.

Частичная зависимость должна сочетаться с графиками ICE, поскольку они могут выявить неточности в частичной зависимости из-за усреднения сильных взаимодействий или наличия корреляции. Когда кривые ICE расходятся с кривыми частичной зависимости, это может указывать на сильное взаимодействие между входными признаками, что является еще одним преимуществом их совместного использования. Затем вы можете использовать EBM или суррогатные деревья решений, чтобы подтвердить существование взаимодействия в обучающих данных или объясняемой модели. Еще один трюк — построить график частичной зависимости и ICE с гистограммой интересующей функции. Это позволяет сделать обоснованный вывод о том, можно ли доверять полученному прогнозу и подтверждается ли он обучающими данными. На рис. 2.6 частичная зависимость, ICE и гистограмма `PAY_0` используются для обобщения поведения монотонного GBM.



Из-за множества присущих ей недостатков частичная зависимость не должна применяться без ICE, или от нее нужно отказаться в пользу ALE.

Вверху мы видим гистограмму `PAY_0` и то, что данных о клиентах, которые просрочили последний платеж более чем на два месяца, не так много. Внизу мы видим кривые частичной зависимости и ICE для клиентов в децилях прогнозируемой вероятности дефолта по кредиту. Частичная зависимость и ICE помогают нам подтвердить монотонность ограниченной функции отклика GBM для `PAY_0`. Похоже, что модель ведет себя разумно, даже если для более высоких значений `PAY_0` не так уж много данных для изучения. Вероятность дефолта монотонно возрастает по мере задержки последнего платежа клиентами и стабильна для более высоких зна-

Книги для программистов: https://t.me/free_books_for_it

чений PAY_0, даже несмотря на отсутствие достаточного количества данных для поддержки классификатора в этом регионе. Может показаться, что применяя ограничения монотонности, мы будем защищены от того, что наши модели ML обучатся неразумному поведению, если у них мало обучающих данных. Но это не так. Да, ограничения монотонности помогают решить проблемы стабильности и недостаточной специфичности, а частичная зависимость и ICE помогают нам обнаружить эти проблемы при их возникновении. Но в данном случае нам просто повезло. В действительности, нам нужно проверить все наши модели на нестабильное поведение в разреженных областях обучающих данных. Мы должны иметь в своем арсенале специализированные модели или даже специалистов, способных сделать правильные прогнозы для этих сложных рядов данных.



Сравнение объяснимых функций формы модели, графиков частичной зависимости, ICE или ALE с гистограммой может дать базовую качественную оценку неопределенности результатов модели. Это позволяет визуально выявлять прогнозы, основанные лишь на небольших объемах обучающих данных.

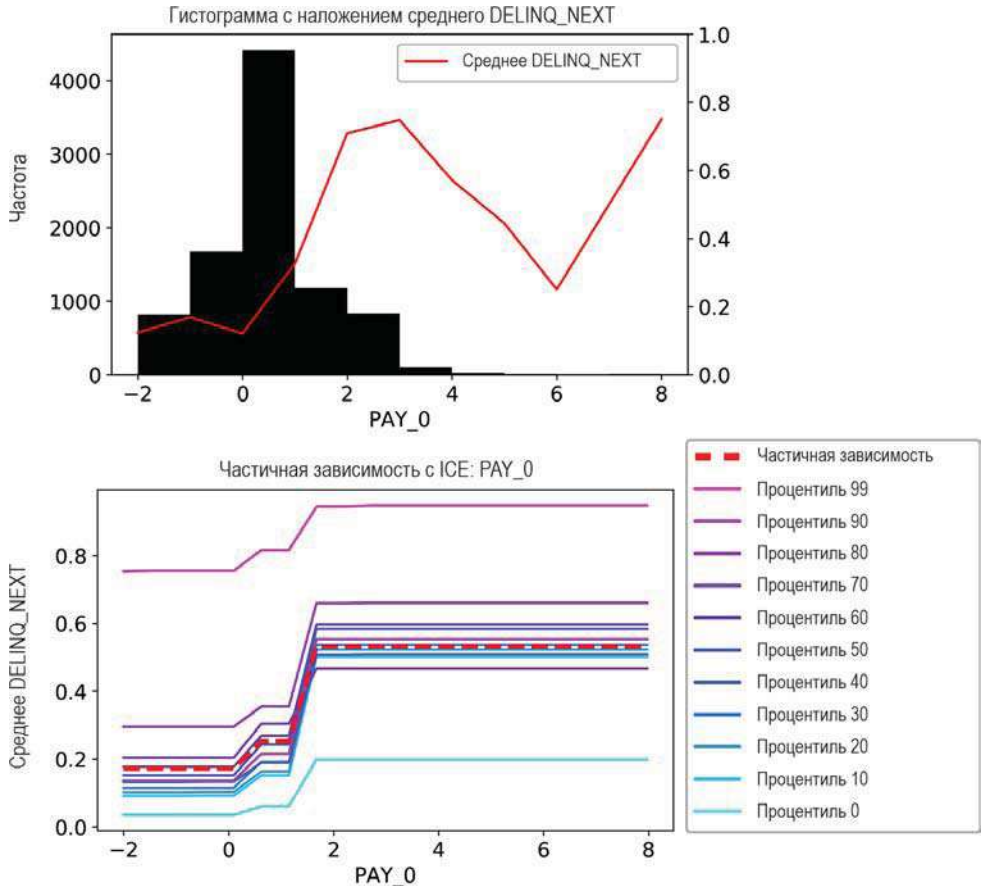


Рис. 2.6. Частичная зависимость и ICE для важной входной переменной с сопровождающей гистограммой и наложением среднего целевого значения для машины монотонного градиентного бустинга

Книги для программистов: https://t.me/free_books_for_it

Вот еще одно небольшое замечание, прежде чем мы перейдем к графикам ALE. Как и в случае важности признаков, SHAP, LIME и всех других методов объяснения, которые работают с фоновым набором данных, мы должны продумать вопросы контекста, если собираемся использовать методы частичной зависимости и ICE. Они оба используют скрытые неявные фоновые данные. В случае частичной зависимости речь идет о любом интересующем нас наборе данных, в котором всем значениям отображаемого признака присвоено определенное значение. Это нарушает закономерности взаимодействия и корреляции. И хотя это экзотическая проблема, она образует уязвимость для атак по отравлению данных, как описано в статье «Fooling Partial Dependence via Data Poisoning» (*https://oreil.ly/SVFmU*). Для ICE неявный фоновый набор данных представляет собой одну строку, в которой интересующему признаку присвоено определенное значение. Следите за тем, чтобы отображаемые значения ICE не были слишком нереалистичными по отношению к остальным наблюдаемым данным в этой строке.

Накопленный локальный эффект

ALE — это новый и высокоточный метод представления поведения модели ML в зависимости от значений входного признака, представленный в статье «Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models» (*https://oreil.ly/TFFTK*). Как и графики частичной зависимости, графики ALE показывают форму (т. е. нелинейность или немонотонность) взаимосвязи между прогнозами модели и значениями входных признаков. Графики ALE особенно ценны, когда в обучающих данных существуют сильные корреляции — ситуация, когда становится понятно, что частичная зависимость не работает. ALE также вычисляется быстрее, чем частичная зависимость. Опробуйте этот вариант визуализации на практике с пакетом ALEPlot (*https://oreil.ly/4o0wH*) в R и его версией в Python ALEPython (*https://oreil.ly/_qKs8*).

Профилирование кластеров

Хотя основное внимание уделяется объяснению моделей, обучаемых с учителем, иногда нам приходится использовать методы обучения без учителя. Извлечение признаков и кластеризация — две наиболее распространенные задачи обучения без учителя. Мы уже обсуждали, как сделать процесс извлечения признаков более понятным, используя разреженные методы, такие как SPCA и NMF, когда рассматривали объяснимые модели. А с применением хорошо зарекомендовавших себя апостериорных методов профилирования кластеризацию часто можно сделать более прозрачной. Самый простой подход — использовать средние значения и медианы для описания центроидов кластера или создания прототипных элементов набора данных на основе кластеров. Затем мы можем обратиться к концепциям, связанным с прототипами, таким как обобщение, сравнение и критика, чтобы лучше понять наше решение по кластеризации. Другой способ — применить извлечение признаков, особенно разреженные методы, чтобы спроецировать решения многомерной кластеризации в двух- или трехмерное пространство для построения графиков. После построения графика на разреженных интерпретируемых осях легче использо-

Книги для программистов: https://t.me/free_books_for_it

вать наши знания предметной области для понимания и проверки группы кластеров. Для понимания и описания кластеров также можно воспользоваться распределением признаков. Плотность признака внутри кластера можно сравнить с его плотностью в других кластерах или с его общим распределением. Признаки с наиболее непохожим распределением по сравнению с другими кластерами или всеми обучающими данными можно рассматривать как самые значимые для решения задачи кластеризации. Наконец, для объяснения кластеров можно применять суррогатные модели. Мы используем те же входные данные, что и алгоритм кластеризации, а также метки кластеров в качестве цели. На основе этих данных мы обучаем на наших кластерах интерпретируемый классификатор, такой как дерево решений. Далее используем интерпретируемые характеристики суррогатной модели, чтобы получить представление о нашем решении на основе кластеризации.

Трудноразрешимые проблемы апостериорного объяснения на практике

Если не соблюдать осторожность, то апостериорные объяснения могут завести вас в тупик. Мы постарались обсудить технические недостатки этих методов, но при работе с ними в реальных приложениях высокого риска следует учитывать и другие важные аспекты. В качестве напоминания профессор Синтия Рудин в своей статье «Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead» (*https://oreil.ly/vR4XI*) излагает основную критику апостериорного объяснения непрозрачных моделей машинного обучения в приложениях высокого риска. По мнению автора, объяснения традиционных моделей машинного обучения:

- основаны на ошибочном убеждении, что необъяснимые модели более точны, чем объяснимые;
- недостаточно точно отражают внутреннюю работу сложных моделей;
- часто бессмысленные;
- трудно калибруются по внешним данным;
- неоправданно сложные.

Вот почему в этой главе мы предлагаем объединить апостериорное объяснение с интерпретируемыми моделями, где модель и объяснения могут выступать как элементы управления процессом, дополняя друг друга. Но даже при таком осторожном подходе к объяснениям все равно остаются серьезные проблемы, требующие решения. В этом разделе будут освещены проблемы, с которыми чаще всего приходится сталкиваться на практике. В завершение раздела мы отметим преимущества сочетания объяснимых моделей и апостериорных объяснений. Но, как покажет пример прозрачности, даже если мы сделаем все правильно с технической точки зрения, человеческий фактор по-прежнему будет очень сильно влиять на окончательный успех или провал приложения машинного обучения в области высокого риска.

Книги для программистов: https://t.me/free_books_for_it

Кристоф Мольнар не только научил нас пользоваться объяснениями, но и плотно занимался исследованием их недостатков вместе со своими соавторами. Если читатели захотят углубиться в детали проблем, связанных с распространенными подходами к объяснению, мы бы предложили как статью «General Pitfalls of Model-Agnostic Interpretation Methods for Machine Learning Models» (*https://oreil.ly/DeZ0J*), так и более раннюю работу «Limitations of Interpretable Machine Learning Methods» (*https://oreil.ly/HYgJ6*). Далее мы обсудим проблемы, с которыми мы чаще всего сталкиваемся на практике: предвзятость самоподтверждения, контекст, корреляция и локальная зависимость, взлом, человеческая интерпретация, непоследовательность и достоверность объяснений.

Предвзятость самоподтверждения

Бóльшую часть текущей главы мы говорили о том, что повышение прозрачности является положительным моментом. Это, безусловно, так. Но более глубокое понимание людьми моделей машинного обучения и способность вмешиваться в их функционирование делают возможной предвзятость подтверждения, которая может проникнуть в наш рабочий процесс машинного обучения. Допустим, мы убеждены, что определенное взаимодействие должно быть представлено в модели, основанной на прошлом опыте аналогичных проектов. Однако это взаимодействие просто не проявляется в нашей объяснимой модели или результатах апостериорного объяснения. В такой ситуации чрезвычайно сложно определить, являются ли наши обучающие данные искаженными и в них отсутствует известное важное взаимодействие, или же мы сами склонны видеть это взаимодействие там, где его нет. Если мы вмешаемся в механизмы нашей модели, чтобы каким-то образом внедрить это взаимодействие, то можем просто поддаться нашей собственной предвзятости самоподтверждения.

Конечно, полное отсутствие прозрачности также позволяет разрастаться предвзятости самоподтверждения, поскольку мы можем манипулировать поведением модели по своему усмотрению. Единственный реальный способ избежать предвзятости самоподтверждения — это следовать научному методу и проверенным научным принципам, таким как прозрачность, проверка и воспроизводимость.

Контекст

«Не объясняйте без контекста» (*https://oreil.ly/A-GxX*), — говорят доктор Пшемыслав Бицек и его команда. На практике это означает использование логичных и реалистичных исходных данных для создания объяснений и защиту этих данных от манипуляций злоумышленников.

Даже если у нас есть надежные исходные данные для пояснений, все равно необходимо убедиться, что наши базовые модели машинного обучения работают в логическом контексте. Это означает разумное количество некоррелированных входных признаков, имеющих прямое отношение к цели моделирования.

Корреляция и зависимости

Хотя во многих случаях корреляция не может помешать алгоритмам ML обучаться и генерировать точные прогнозы in silico, она очень затрудняет объясне-

Книги для программистов: https://t.me/free_books_for_it

ние и интерпретацию. В больших наборах данных обычно имеется множество коррелированных признаков. Корреляция нарушает принцип независимости, а это означает, что мы не можем реалистично интерпретировать признаки сами по себе. Когда мы пытаемся удалить признак, как это делают многие методы объяснения, другой коррелирующий признак тут же занимает его место в модели, сводя на нет эффект от попытки удаления и предполагаемое значение удаления как инструмента объяснения. Мы также полагаемся на методы возмущения, но если признаки коррелируют, нет смысла возмущать только один из них, чтобы получить объяснение. Хуже того, при работе с моделями машинного обучения они могут изучать локальные зависимости, т. е. различные корреляционные отношения, построчно. Почти невозможно разобраться в сложностях того, как корреляция может значительно искажать объяснения, а тем более трудно представить, насколько сложные локальные зависимости способны сделать то же самое.

Взлом

Методы объяснения, использующие фоновые данные, могут быть изменены злоумышленниками. К ним относятся LIME и SHAP, как описано в статье «Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods» (*https://oreil.ly/ljDkp*), и частичная зависимость, как описано в статье «Fooling Partial Dependence via Data Poisoning» (*https://oreil.ly/MJUz7*). Хотя на данный момент эти взломы, вероятно, представляют собой экзотическую проблему, вряд ли вы захотите стать первой жертвой крупного взлома объяснений ML. Убедитесь, что код, используемый для создания фоновых данных, хранится в безопасности и что этими данными нельзя манипулировать во время вычислений объяснения. Внутренние злоумышленники могут с легкостью отравить как обучающие, так и фоновые данные. Даже если фоновые данные в безопасности, объяснения все равно могут быть неправильно истолкованы злонамеренным образом. При так называемом *отмывании справедливости* (fairwashing) объяснения социологически предвзятой модели ML подтасовывают так, чтобы модель выглядела справедливой, при этом подвергая пользователей модели реальному вреду.

Человеческая интерпретация

Иногда даже опытные практики и исследователи с трудом понимают логику прогнозов ML. Тем не менее аудитория, которой адресованы объяснения ML, гораздо шире, чем просто эксперты отрасли. Приложения ML в области высокого риска часто требуют принятия важных решений, затрагивающих других людей. Даже если эти люди получили хорошее образование, мы не можем ожидать, что они поймут частичную зависимость и график ICE или массив значений SHAP. Чтобы обеспечить прозрачность в ситуациях с высокими рисками, вам нужно будет работать с психологами, экспертами в предметной области, дизайнерами, экспертами по взаимодействию с пользователями и другими. Это потребует дополнительного времени и итераций продукта, а также интенсивного общения между техническими специалистами, экспертами и пользователями. Отказ от подобной дополнительной работы может привести к полному провалу,

Книги для программистов: https://t.me/free_books_for_it

даже если цели технической прозрачности достигнуты, как обсуждается далее в разделе «Пример: оценка по алгоритму».

Непоследовательность

В объяснимом ML последовательность означает стабильность объяснений различных моделей или выборок данных. Последовательность объяснений очень важна для приложений машинного обучения в области высоких рисков, но добиться ее нелегко. В таких ситуациях, как решения о кредите или досудебном освобождении, люди могут столкнуться со множеством автоматических решений и связанных с ними объяснений, особенно в более технологичном будущем. Если в объяснениях приводятся разные причины одного и того же итогового решения, это усложнит и без того непростые ситуации. Чтобы сделать объяснения более логичными и понятными, необходимо учитывать реальные, обобщаемые явления в обучающих данных и в области применения. Чтобы добиться согласованности, нам необходимо обучать наши модели разумному количеству независимых признаков. Сами модели также должны быть экономными, т. е. должны подчиняться реальным отношениям. И наоборот, логически обоснованные объяснения невозможны для сложных, недостаточно определенных, неинтерпретируемых моделей с многочисленными и коррелированными входными данными.

Измерение качества объяснения

Допустим, вы обучаете модель, наблюдаете за результатами, затем предполагаете, что она работает правильно, в результате вводите ее в эксплуатацию. Вероятно, это плохая идея. Но именно так мы все работаем с апостериорными объяснениями. Учитывая все технические проблемы, поднятые в предыдущих разделах, нам, очевидно, необходимо протестировать объяснения и посмотреть, как они работают для конкретного источника данных и приложения, как мы обычно поступаем с любым другим методом машинного обучения. Подобно тому как вы измеряете качество модели перед ее развертыванием, необходимо также измерить качество объяснений. В исследовательской литературе можно найти описания таких измерений и обобщенных методик тестирования. В работе «Towards Robust Interpretability with Self-Explaining Neural Networks» (*https://oreil.ly/t4322*) на первый план выдвигаются ясность (т. е. являются ли объяснения понятными сразу), достоверность (т. е. соответствуют ли объяснения известным важным факторам) и стабильность (т. е. согласуются ли объяснения с соседними точками данных). В работе «On the (In)fidelity and Sensitivity of Explanations» (*https://oreil.ly/kSiSS*) представлены соответствующие одноименные тесты. Помимо этих методик формального измерения, можно удостовериться, что объяснимые модельные механизмы и апостериорные объяснения подтверждают друг друга, если мы используем и объяснимые модели, и объяснения. Если доступны более старые, заслуживающие доверия объяснения, мы можем использовать их в качестве эталонов для проверки достоверности новых объяснений. Кроме того, тесты на стабильность, в которых данные или модель незначительно искажаются, обычно не должны приводить к серьезным изменениям в апостериорных объяснениях.

Книги для программистов: https://t.me/free_books_for_it



Протестируйте объяснения перед их применением в приложениях высокого риска. Поскольку при отсутствии эталонных данных сложно проверить объяснения, их следует сравнивать с интерпретируемыми модельными механизмами. Для проверки качества объяснения также можно использовать сравнения с эталонными объяснениями, меры явности и достоверности, возмущения, сравнения с ближайшими соседями и смоделированные данные.

Нельзя отрицать привлекательность объяснения любой модели, сколь бы сложной она ни была, просто потому, что результат не идеален. Но если учесть все технические и бытовые проблемы, которые были только что рассмотрены, надеемся, мы убедили читателей, что традиционное объяснение модели ML — это своего рода несбыточная мечта. Возможно, когда-то мы научимся объяснять то, что сейчас кажется необъяснимым. Однако сегодня это технически сложно. А когда мы учтем все человеческие факторы, необходимые для достижения реальной прозрачности, задача объяснения станет еще сложнее.

Сочетание объяснимых моделей и апостериорных объяснений

Завершая техническую часть главы, мы хотели бы упомянуть новые исследования, в которых говорится о том, почему объяснение традиционных моделей ML так сложно, и представить читателям пример сочетания объяснимых моделей и апостериорных объяснений. Две недавние статьи связывают присущую традиционным моделям ML сложность с трудностью достижения прозрачности. В статье «Assessing the Local Interpretability of Machine Learning Models» (*https://oreil.ly/FQQd_*) авторы сопоставляют сложность с количеством операций, связанных с решением модели ML, и показывают, что с увеличением количества операций интерпретируемость уменьшается. В статье «Quantifying Model Complexity via Functional Decomposition for Better Post-Hoc Interpretability» (*https://oreil.ly/DWTRF*) исследователи используют ряд признаков, силу взаимодействия и сложность основного отклика для измерения общей сложности моделей ML и показывают, что модели, которые минимизируют эти критерии, более надежно интерпретируются. Подводя итог, можно сказать, что сложные модели труднообъяснимы, а более простые модели объяснить легче, но, конечно, все равно непросто. На рис. 2.7 представлен пример дополнения простой модели пояснениями и показано, почему даже его сложно реализовать.

Здесь показано обученное трехуровневое дерево решений с выделенным путем принятия решения и значениями Tree SHAP для одной строки данных, следующей по этому пути решения. Хотя рисунок выглядит простым, на самом деле он иллюстрирует несколько фундаментальных проблем ML и его объяснения. Прежде чем мы углубимся в трудности, представленные на рис. 2.7, давайте насладимся достоинствами прогнозирующей модели со всем ее глобальным механизмом принятия решений, для которой мы можем генерировать числовые вклады входных признаков в любой прогноз модели. Раньше этот уровень прозрачности был исключительным преимуществом линейных моделей, но благодаря новым подходам, рассмотренным в данной главе, стал достижим для гораздо более широкого класса моделей

Книги для программистов: https://t.me/free_books_for_it

более высокой емкости. Это означает, что соблюдая определенные меры предосторожности, мы сможем обучить более сложные модели, которые будут извлекать больше информации из данных. Более того, эти модели смогут самостоятельно интерпретировать результаты и учиться на своих ошибках. Таким образом, модели становятся похожи на людей, которые способны учиться на собственных неудачах. Это огромный прорыв.



Совместно используйте объяснимые модели и апостериорные объяснения для взаимной проверки и максимизации прозрачности моделей ML.

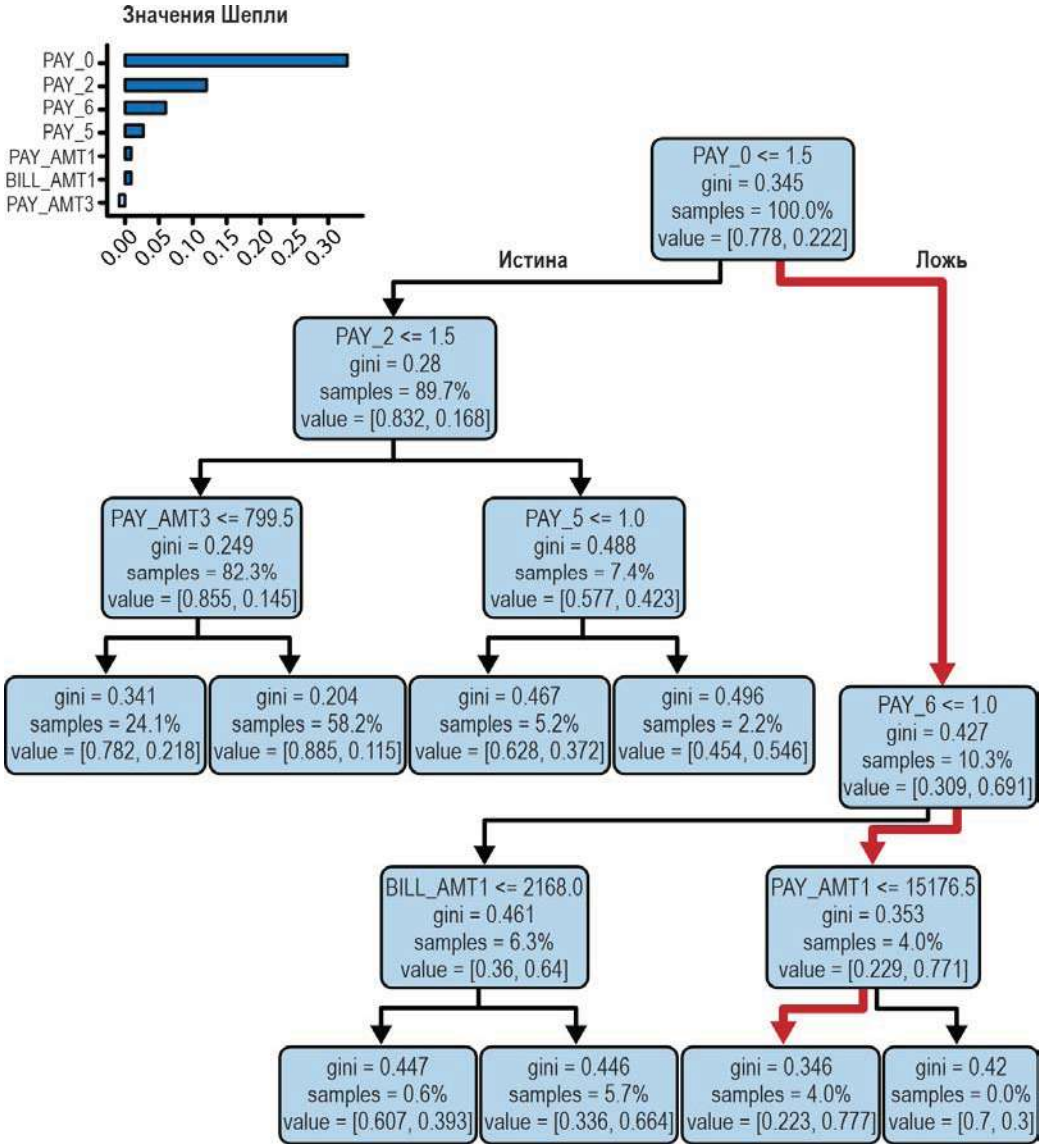


Рис. 2.7. Простая объяснимая модель в сочетании с апостериорной объясняющей информацией

Книги для программистов: https://t.me/free_books_for_it

Теперь проанализируем трудности, показанные на рис. 2.7, не забывая при этом, что упомянутые проблемы всегда существуют при использовании моделей ML и апостериорного XAI. (В этом простом случае мы можем их увидеть и обдумать.) Обратите внимание, что путь принятия решения для выбранного человека учитывает `PAY_0`, `PAY_6` и `PAY_AMT1`. Теперь взгляните на значения Шепли. Они придают `PAY_2` больший вес, чем `PAY_6`, а `PAY_5` имеют больший вес, чем `PAY_AMT1`, но `PAY_2` и `PAY_5` не лежат на пути принятия решения. Так получилось, потому что значения Шепли учитывают искусственные наблюдения с разными значениями `PAY_0`, `PAY_AMT1` и `PAY_6`, и эти наблюдения проходят по разным путям принятия решений. Мы увидим такое поведение независимо от того, используем ли мы возмущения признаков `tree_path_dependent` или `interventional`.

Это наблюдение неочевидно, но оно правильно и не является результатом ошибки аппроксимации. Мы могли бы использовать другой пакет или подход и, вероятно, создать локальные объяснения, которые соответствовали бы единому пути принятия решений, показанному на рис. 2.7, но тогда мы лишились бы глубокой теоретической поддержки, которая сопровождает значения Шепли и метод SHAP. По крайней мере, в случае с SHAP мы знаем, почему наши объяснения демонстрируют такой эффект. В целом объяснить модели машинного обучения очень сложно, и на это есть много разных причин. Всегда проверяйте объяснения, прежде чем использовать их в контекстах приложений высокого риска. Убедитесь, что вы понимаете применяемые апостериорные методы.

В главе 6 мы более подробно рассмотрим, как SHAP использует фоновые данные и вычисляет атрибуцию признаков в различных ситуациях. Вы должны хорошо разбираться в подобных тонкостях, прежде чем применять методы атрибуции признаков, такие как SHAP, в приложениях высокого риска. Что касается технической стороны объяснимости моделей, тут действительно есть над чем задуматься. Но следующий случай из реальной жизни свидетельствует о том, что некоторые человеческие факторы объяснений, возможно, еще труднее поддаются пониманию.

Пример: алгоритмическая корректировка оценок

Добавление прозрачности в модели машинного обучения — непростая задача. Даже если мы правильно разберемся с техническими аспектами объяснимых моделей и апостериорными объяснениями, все равно остается много человеческих факторов, которые требуют не менее осторожного отношения. Так называемый скандал уровня А в Великобритании может служить наглядным уроком непонимания человеческого фактора при принятии решений с высокими рисками, основанных на ML. Когда весной 2020 г. в Соединенном Королевстве вступили в полную силу ограничения, связанные с COVID, оказалось, что стандартизированные тесты успеваемости невозможно проводить в обычном режиме. В качестве первой попытки решения проблемы учителей попросили самостоятельно оценить успеваемость учащихся на важных экзаменах уровня А, от которых зависит поступление в колледж и многое другое в будущей жизни студента. К сожалению, с точки зрения проверяющих органов оценки учителей выглядели как неправдоподобно положительные.

Книги для программистов: https://t.me/free_books_for_it

Правительственные чиновники решили, что использование этих оценок было бы несправедливо по отношению к бывшим и будущим ученикам.

Чтобы исправить излишне высокие оценки учителей, Управление по квалификационным тестам и экзаменам (Ofqual) применило алгоритм корректировки прогнозов учителей. Основанную на статистике методологию алгоритма корректировки разработали эксперты в области образования. Распоряжение о применении выпустили после того, как студенты получили свои оценки. Алгоритм был предназначен для получения окончательного распределения оценок, аналогичного результатам в предыдущие годы. Он сохранил оценки учителей, но использовал школьную статистику прошлых лет, чтобы скорректировать оценки в сторону понижения. Студенты в Шотландии были первыми, кто столкнулся с результатами применения алгоритма. По данным ZDNet, в этой части Великобритании «35,6% оценок были снижены на один балл, 3,3% — на два балла, а 0,2% — на три».

В течение следующих нескольких месяцев протесты студентов по поводу возможной предвзятости в отношении более бедных школ и регионов страны вызвали массовый, стремительно разгорающийся инцидент с искусственным интеллектом. Несмотря на то что чиновники уже столкнулись с проблемой в Шотландии, они применили ту же процедуру в Англии, но дали студентам право на бесплатную апелляцию и повторное тестирование. Тем не менее общественному доверию к системе тестирования был нанесен непоправимый ущерб, а прозрачная, но предвзятая идея корректировки индивидуальных оценок с учетом прошлых школьных успехов оказалась для многих неприемлемой. В конце концов, правительство Великобритании решило использовать первоначальные оценки учителей. По данным Wired, «правительство, по сути, переложило административную ответственность на университеты, которым теперь придется рассматривать возможность зачисления еще тысяч студентов — они заявили, что, несмотря на отмену корректировки, отсутствует возможность реализовать все предварительные офферы (приглашения к поступлению)». В той же статье отмечено, что оценки успеваемости прошлых лет, на которых базировалась алгоритмическая корректировка, демонстрировали отчетливую расовую предвзятость учителей. Вопиющая оплошность разработчиков.

Поразительно, но другие учебные заведения тоже переняли идею алгоритмических оценок на вступительных экзаменах в колледж, которые, по сути, могут изменить жизнь человека. Международный бакалавриат (IB) — элитная образовательная программа, предлагающая продвинутую единую учебную программу для учащихся средних школ всего мира. Весной 2020 г. IB использовал алгоритм оценки успеваемости учащихся, который, как сообщалось, был «поспешно внедрен после отмены обычных весенних экзаменов из-за COVID-19. Система использовала различные данные прошлых периодов, в том числе оценки учащихся по заданиям и оценки прошлых выпускников их школы». Из-за неудачно выбранного времени внезапное снижение оценок чрезвычайно сильно навредило студентам, подающим заявления в колледжи и университеты США. Эти учебные заведения резервируют места для студентов IB на основе прошлых результатов, но могут отменить обучение на основании низких итоговых оценок, «разрушая их планы на осень и далее». Алгоритмические оценки некоторых студентов оказались настолько низкими, что они

Книги для программистов: https://t.me/free_books_for_it

лишились мест в престижных университетах США и институтах в своей родной стране. Что еще хуже, в отличие от алгоритма Ofqual, администрация IB не сообщила, как работает их алгоритм. Вдобавок апелляция стоила почти 800 долларов.

Если оставить в стороне отсутствие прозрачности со стороны ML, то, похоже, в основе этих инцидентов лежат три основные проблемы. Первая проблема — широкий охват — это неотъемлемый риск приложений ML. Алгоритмы IB применялись ко многим студентам по всему миру. Широкий охват означает высокую значимость для общества, и одной только прозрачности недостаточно, чтобы компенсировать проблемы доверия и предвзятости. Понимание — не то же самое, что доверие. Технический отчет Ofqual и другие публичные анализы были недоступны для понимания многих студентов и родителей. Зато они прекрасно понимали, что в бедных районах государственные школы хуже, и это затронуло учащихся дважды в 2020 году — сначала они в целом получили худшие оценки, как это случалось каждый год, а затем снова, когда их баллы были скорректированы алгоритмом в сторону понижения. Второй проблемой была значимость решения. Поступление в колледж играет огромную роль в дальнейшей жизни многих людей. Серьезный характер решения еще больше повышает значимость — возможно, до той степени, когда провал становится гарантированным. ML по своей сути является вероятностным. Но разве можно опираться на вероятности, если речь идет о судьбе человека? Когда ставки настолько высоки, общественность может просто не принять подобный подход.

Третья важная проблема, о которой здесь идет речь, — это явная природа несопоставимого воздействия. Например, алгоритм не оценивал очень маленькие классы. Где самые маленькие классы? В частных школах. В статье Verge утверждается, что «платные частные школы (так называемые независимые школы) непропорционально сильно выиграли от используемого алгоритма. В этих школах количество оценок A и выше увеличилось на 4,7 % по сравнению с прошлым годом». ZDNet сообщила, что «процент успевающих студентов, обучающихся на высших курсах в бедных районах Шотландии, снизился на 15,2% по сравнению с 6,9% в более богатых районах». Корректировка по почтовому индексу или прошлой успеваемости в школе порождает системную предвзятость, и учащиеся и родители понимали это на эмоциональном уровне. Как сообщает BBC, министр образования Шотландии в конце концов осознал, что в результате этого фиаско «молодые люди почувствовали, что их будущее определялось статистическим моделированием, а не их собственными способностями». Нам следует подумать о том, как бы мы себя чувствовали, если бы этот инцидент затронул нас или наших детей. Несмотря на восторги вокруг автоматизированного принятия решений, почти никто не хочет чувствовать, что его будущее определяется вероятностным алгоритмом.

Хотя это применение ML, скорее всего, с самого начала было обречено на провал, даже здесь можно было сделать больше для повышения общественного доверия. Например, Управление Ofqual могло бы опубликовать алгоритм, прежде чем применять его к студентам. Правительство также могло бы получить отзывы общественности об алгоритме перед его использованием. Джени Теннисон, британский защитник открытых данных, отмечает: «Часть проблемы здесь в том, что недостат-

Книги для программистов: https://t.me/free_books_for_it

ки алгоритма проявились только после того, как студентам были выставлены оценки, хотя мы могли бы провести обсуждение, изучить алгоритм и осознать последствия гораздо раньше». Из этого следует, что техническая прозрачность — не то же самое, что широкое общественное понимание, и это понимание, если оно вообще может быть достигнуто, не гарантирует доверия. Даже если вы уже добились высокого уровня технической прозрачности, как было сказано в этой главе, предстоит еще много работы, чтобы система ML гарантированно работала так, как ожидают ее пользователи или владельцы. Наконец, мы описали всего лишь единичный инцидент с ИИ. Хотя он действительно серьезный, нельзя упускать из виду более мелкие инциденты, которые вредят людям прямо сейчас, и мы должны помнить, что от систем ИИ пострадает еще больше людей в будущем. Как выразился Теннисон: «Этот инцидент попал в заголовки газет, потому что он затрагивает очень много людей по всей стране, и это люди, которые пользуются правом голоса. Есть и другие автоматизированные решения, которые постоянно принимаются, например в отношении льгот, и затрагивают людей, не имеющих возможности постоять за себя».

Ресурсы

Дополнительное чтение

- An Introduction to Machine Learning I nterpretability (O'Reilly): **https://oreil.ly/iyz08**
- Designing Inherently Interpretable Machine Learning Models: **https://oreil.ly/jbGNt**
- Explanatory Model Analysis (CRC Press): **https://oreil.ly/Yt_Xm**
- General Pitfalls of Model-Agnostic Interpretation Methods for Machine Learning Models: **https://oreil.ly/On9uS**
- Interpretable Machine Learning: **https://oreil.ly/BHy1L**
- Limitations of Interpretable Machine Learning Methods: **https://oreil.ly/VHMWh**
- On the Art and Science of Explainable Machine Learning: **https://oreil.ly/myVr8**
- Psychological Foundations of Explainability and Interpretability in Artificial Intelligence: **https://oreil.ly/HUomp**
- When Not to Trust Your Explanations: **https://oreil.ly/9Oxa6**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 3

Отладка систем машинного обучения

На протяжении десятилетий величина ошибки, или точность модели на тестовом наборе данных была стандартом, по которому оцениваются модели машинного обучения. К сожалению, поскольку модели ML внедряются в системы искусственного интеллекта, которые используются более широко и для более чувствительных приложений, стандартные подходы к оценке моделей ML оказались неадекватными. Например, обычная *площадь под кривой* (area under the curve, AUC) на тестовых данных почти ничего не говорит нам о предвзятости и алгоритмической дискриминации, отсутствии прозрачности, нарушении конфиденциальности или уязвимостях безопасности. Тем не менее именно эти проблемы часто являются причиной того, что системы ИИ выходят из строя после развертывания. Для достижения приемлемых показателей in vivo нам просто необходимо выйти за рамки традиционных оценок in silico, предназначенных в первую очередь для исследовательских прототипов. Более того, наилучшие результаты в области безопасности и производительности достигаются тогда, когда организации могут сочетать соответствующие культурные компетенции и средства управления процессами, описанные в главе 1, с технологией машинного обучения, которая способствует укреплению доверия. В этой главе представлены разделы, посвященные обучению, отладке и развертыванию систем машинного обучения. В них рассматриваются многочисленные технические подходы к тестированию и повышению безопасности, производительности и доверия к ИИ in vivo. К вашему сведению, в главах 8 и 9 представлены подробные примеры кода для отладки модели.

Схема соответствий NIST AI RMF

Раздел главы	Подкатегория NIST AI RMF
Воспроизводимость	GOVERN 1.2, GOVERN 1.4, MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Качество данных	GOVERN 1.2, MAP 2.3, MAP 4, MEASURE 1.1, MEASURE 2.1
Бенчмарки и их альтернативы	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, MAP 2.3, MEASURE 2.13, MANAGE 2.1
Калибровка	GOVERN 1.2, GOVERN 1.4, MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Конструктная валидность	GOVERN 1.1, MAP 2.1, MAP 2.3, MAP 3.3
Предположения и ограничения	GOVERN 1.2, GOVERN 1.4, GOVERN 6.1, MAP 2
Функции потерь по умолчанию	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Множественные сравнения	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Будущее безопасного и надежного машинного обучения	MAP 2.3, MEASURE 2.6

Книги для программистов: https://t.me/free_books_for_it

Окончание

Раздел главы	Подкатегория NIST AI RMF
Тестирование программного обеспечения	GOVERN 1.1, GOVERN 1.2, GOVERN 4.3, GOVERN 6.1, MAP 2.3, MAP 4, MEASURE 1.3
Традиционная оценка модели	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, MAP 2.3, MAP 4, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Смещение распределений	GOVERN 1.2, GOVERN 1.5, MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3, MEASURE 2.4, MANAGE 2.2, MANAGE 2.3, MANAGE 2.4, MANAGE 3, MANAGE 4.1
Эпистемическая неопределенность и разреженность данных	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Нестабильность	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3, MEASURE 2.4, MANAGE 2.2, MANAGE 2.3, MANAGE 2.4, MANAGE 3, MANAGE 4.1
Утечка	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Петля обратной связи	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Переобучение	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Ложное обучение	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Недостаточное обучение	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Неполная идентификация	MAP 2.3, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Анализ остатков	GOVERN 1.2, MAP 2.3, MAP 3.2, MAP 5.1, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Анализ чувствительности	GOVERN 1.2, MAP 2.3, MAP 3.2, MAP 5.1, MEASURE 1, MEASURE 2.1, MEASURE 2.3
Эталонные модели	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, MAP 2.3, MEASURE 2.13, MANAGE 2.1
Исправление ошибок	GOVERN, MAP, MANAGE
Безопасность в предметной области	GOVERN 1.2, GOVERN 1.7, GOVERN 3, GOVERN 4.1, GOVERN 4.3, GOVERN 5, MAP 1.2, MAP 1.6, MAP 2.3, MAP 3.1, MAP 5, MEASURE 1, MEASURE 2.5, MEASURE 2.6, MEASURE 3, MEASURE 4, MANAGE 1, MANAGE 4.3
Мониторинг модели	GOVERN 1.2, GOVERN 1.3, GOVERN 1.4, GOVERN 1.5, MAP 2.3, MAP 3.5, MAP 4, MAP 5.2, MEASURE 1.1, MEASURE 2.4, MEASURE 2.6, MEASURE 2.7, MEASURE 2.8, MEASURE 2.10, MEASURE 2.11, MEASURE 2.12, MEASURE 3.1, MEASURE 3.3, MEASURE 4, MANAGE 2.2, MANAGE 2.3, MANAGE 2.4, MANAGE 3, MANAGE 4

- В документации применяются следующие характеристики надежности ИИ: безопасный, защищенный и отказоустойчивый, действительный и надежный.
- Смотрите также:
  - полная таблица соответствий (не является официальным ресурсом), **https://oreil.ly/61TXd**

Обучение

Обсуждение алгоритмов обучения ML начинается с воспроизводимости, поскольку без этого невозможно узнать, действительно ли одна версия системы ML лучше другой. Мы кратко рассмотрим вопросы данных и конструирования признаков, и завершим данный раздел изложением ключевых моментов спецификации модели.

Книги для программистов: https://t.me/free_books_for_it

Воспроизводимость

Без воспроизводимости мы строим зáмок на песке. Воспроизводимость имеет основополагающее значение для всех научных исследований, включая ИИ. Без воспроизводимых результатов очень трудно понять, вносят ли наши усилия хоть какие-то улучшения или хотя бы изменения в систему ML. Воспроизводимость помогает обеспечить правильное внедрение и тестирование, а некоторые клиенты могут сделать ее главным условием. Далее перечислены наиболее распространенные методы, которые специалисты по данным и инженеры ML используют для обеспечения воспроизводимости своих систем:

Эталонные модели

Эталонные модели — это важные инструменты безопасности и производительности для обучения, отладки и развертывания систем ML. В этой главе они будут рассмотрены несколько раз. С точки зрения обучения и воспроизводимости, мы всегда должны строить свою систему на основе воспроизводимой эталонной модели. Этот подход позволяет создать контрольную точку для отката в случае потери воспроизводимости и дает возможность легко оценить реальный прогресс. Если вчерашний контрольный показатель воспроизводим и сегодняшние достижения, превышающие этот контрольный показатель, также воспроизводимы, это вполне осязаемый прогресс. Если показатели системы были нестабильными до внесения изменений и продолжают меняться после, нам сложно понять, оказали наши действия положительный эффект или же навредили.

Аппаратное обеспечение

Поскольку в системах ML часто применяется аппаратное ускорение с помощью графических процессоров (GPU) и других специализированных системных компонентов, аппаратное обеспечение по-прежнему влияет на воспроизводимость. По возможности старайтесь, чтобы аппаратное обеспечение в системах разработки, тестирования и развертывания было максимально одинаковым.

Вычислительная среда

Системы машинного обучения всегда работают в некоторой вычислительной среде, определяемой системным оборудованием, системным программным обеспечением, а также нашим стеком данных и программного обеспечения ML. Изменения любого компонента среды могут повлиять на воспроизводимость результатов. К счастью, такие инструменты, как виртуальные среды Python и контейнеры Docker, которые сохраняют рабочую среду, стали обычным явлением в практике науки о данных. Дополнительное специализированное программное обеспечение для управления средой от Domino, gigantum, TensorFlow TFX и Kubeflow позволяет обеспечить еще более широкий контроль над вычислительными средами.

Метаданные

Для воспроизводимости также необходимы «данные о данных». Отслеживайте все артефакты, связанные с моделью, например наборы данных, этапы предварительной обработки, результаты проверки данных и модели, подтверждения

Книги для программистов: https://t.me/free_books_for_it

людьми и детали развертывания. Это позволяет не только вернуться к определенной версии набора данных или модели, но и проводить детальную отладку и судебное расследование инцидентов с ИИ. Пример хорошего инструмента с открытым исходным кодом для отслеживания метаданных можно найти на сайте TensorFlow ML Metadata (*https://oreil.ly/gmHkg*).

Начальные числа рандомизатора

Начальные числа рандомизатора (random seed), задаваемые учеными и инженерами в конкретных блоках кода, являются ключевым аспектом воспроизводимости машинного обучения. К сожалению, они часто сопровождаются инструкциями для конкретного языка или пакета. Адаптация начальных чисел к разным программам может занять некоторое время, но в сочетании с тщательным тестированием известные начальные числа рандомизатора позволяют составным блокам сложных и комплексных систем ML сохранять воспроизводимость. Это необходимое условие для общей воспроизводимости.

Контроль версий

Незначительные изменения кода могут привести к кардинальным изменениям в результатах ML. Изменения в нашем собственном коде и его зависимостях должны отслеживаться с помощью профессионального инструмента контроля версий, чтобы сохранить хоть какую-то надежду на воспроизводимость. Git и GitHub — это бесплатные и повсеместно используемые ресурсы для контроля версий программного обеспечения, но есть и множество других вариантов, которые стоит изучить. Важно отметить, что версии наборов данных тоже можно контролировать с помощью таких инструментов, как Pachyderm и DVC, что позволяет отслеживать изменения в ресурсах данных.

После некоторых экспериментов можно подобрать комбинацию подходов и технологий, которая обеспечивает максимальную воспроизводимость именно ваших систем ML. Закончив с этим фундаментальным фактором безопасности и надежности, можно переходить к другим базовым аспектам, таким как качество данных и конструирование признаков.



Некоторые темы, такие как тесты, обнаружение аномалий и мониторинг, повсеместно встречаются в отладке моделей и безопасности машинного обучения, поэтому мы упоминаем их в нескольких различных разделах и контекстах этой главы.

Качество данных

О качестве данных и конструировании признаков для систем ML написаны целые книги. В данном коротком разделе освещаются некоторые из наиболее важных аспектов этой обширной области практики с точки зрения безопасности и надежности. Прежде всего надо отметить, что систематические ошибки, мешающие признаки, неполнота и шум в данных формируют важные предположения и определяют ограничения наших моделей. Другие базовые параметры, такие как размер и форма набора данных, имеют важное значение. Алгоритмы машинного обучения нуждаются в данных. Как небольшие, так и большие, но слишком разреженные данные

Книги для программистов: https://t.me/free_books_for_it

могут привести к катастрофическим сбоям системы ML в реальном мире, поскольку оба варианта приводят к сценариям, в которых работа системы выглядит нормальной на тестовых данных, но просто оторвана от реальности. Небольшие данные затрудняют обнаружение неполного обучения, неполной идентификации, переобучения или других фундаментальных проблем с качеством модели. Разреженные данные могут привести к слишком самоуверенным прогнозам для определенных сочетаний входных значений. Если модель не получила определенные диапазоны данных во время обучения из-за проблем с разреженностью, большинство алгоритмов ML будут спокойно выдавать прогнозы в этих диапазонах без предупреждения о том, что они практически ни на чем не основаны. В этой главе мы используем следующий пример: в мире просто не хватает обучающего видео, чтобы заполнить все пространство примеров ситуаций, в которых беспилотные автомобили должны научиться безопасно перемещаться. Например, люди, пересекающие проезжую часть ночью на велосипеде, представляют собой опасность, которую распознает большинство людей, но без большого количества кадров с помеченными видео этого довольно редкого события способность системы глубокого обучения справиться с опасной ситуацией, вероятно, будет скомпрометирована из-за разреженности обучающих данных.

Опасения по поводу безопасности могут вызывать и другие факторы, например низкое качество данных, приводящее к запутанности или искажению важной информации и переобучению, или проблемы с данными машинного обучения и конвейером моделей. В контексте этой главы *запутанность* (entanglement) означает признаки, объекты или явления в обучающих данных, заменяющие другую информацию, имеющую более прямое отношение к цели (например, тени на снегу, похожие на изображение хаски при распознавании объектов). Под переобучением подразумевается запоминание шума в обучающих данных и следующие из этого чрезмерно оптимистические оценки ошибок, а проблемы конвейера возникают в результате объединения различных этапов подготовки данных и компонентов моделирования в один исполняемый файл, генерирующий прогнозы. Таблица 3.1 применима к большинству стандартных данных ML и помогает выявить общие проблемы с их качеством, влияющие на безопасность и производительность.

Таблица 3.1. Распространенные проблемы качества данных с симптомами и предлагаемыми решениями. Используется с разрешения Университета Джорджа Вашингтона DNSC 6314 (Машинное обучение I)

Проблема	Типичные симптомы	Возможное решение
Необъективные данные: когда набор данных содержит информацию об интересующем нас явлении, но эта информация постоянно и систематически ошибочна. (Более подробная информация в главе 4)	Необъективные модели и предвзятые, опасные или неточные результаты. Фиксация прошлых социальных предубеждений и дискриминации	Проконсультируйтесь с экспертами в данной области и заинтересованными сторонами. Применяйте научный метод и подходы, основанные на разработке эксперимента (DOE) (*https://oreil.ly/0kDC9*). (Получайте больше данных. Получайте более качественные данные)[1]

[1] Здесь подразумевается, что нужно: а) собирать больший объем данных; б) если недоступен больший объем, тогда собирайте *изначально* более качественные данные. — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

Таблица 3.1 *(продолжение)*

Проблема	Типичные симптомы	Возможное решение
Символьные данные: когда определенные столбцы, характеристики или экземпляры представлены строками символов, а не числовыми значениями	Потеря информации. Необъективные модели и предвзятые, опасные или неточные результаты. Неприемлемо длительное время обучения	Различные способы числового кодирования (например, кодирование меток, целей или признаков). Выбор подходящего алгоритма, например древовидные модели, наивная байесовская классификация
Утечка данных: когда информация из проверочных или тестовых разделов просачивается в обучающие данные	Ненадежные или опасные прогнозы вне предметной области. Модели с завышенными параметрами и неточные результаты. Слишком оптимистичные оценки эффективности in silico	Управление данными. Убедитесь, что все обучающие данные размещены раньше, чем валидационные и тестовые. Убедитесь, что одинаковые идентификаторы не встречаются в разных разделах. Аккуратное применение конструирования признаков — только после разбиения на разделы, а не до него
Грязные данные: сочетание всех проблем, описанных в этой таблице, очень часто встречающееся в реальных наборах данных	Потеря информации. Необъективные модели и предвзятые, опасные или неточные результаты. Неприемлемо длительное время обучения. Нестабильные и ненадежные оценки параметров и генерация правил. Ненадежные или опасные прогнозы вне предметной области	В данном случае применяется комбинация стратегий
Разные шкалы признаков: когда такие характеристики, как возраст и доход, фиксируются по разным шкалам	Ненадежные оценки параметров, необъективные модели и необъективные, неточные результаты	Стандартизация. Выбор подходящего алгоритма, например древовидных моделей
Дублирование данных: строки, экземпляры или сущности, которые встречаются чаще, чем предполагалось	Необъективные результаты из-за непреднамеренного перевеса одинаковых сущностей во время обучения. Необъективные модели и предвзятые, опасные или неточные результаты	Тщательная очистка данных, основанная на консультациях с экспертами в предметной области
Запутывание: признаки, объекты или явления в обучающих данных, заменяющие другую информацию, имеющую более прямое отношение к цели (например, тени на снегу, похожие на изображение хаски при распознавании объектов)	Ненадежные или опасные прогнозы вне предметной области. Обучение не тем признакам	Применять научный метод и подходы DOE. Применение интерпретируемых моделей и апостериорных объяснений. Тестирование в реальных условиях
Поддельные или отравленные данные: данные, признаки, атрибуты, явления или сущности, которые вводятся в обучающие данные или которыми манипулируют для получения неестественных результатов моделирования	Ненадежные или опасные прогнозы вне предметной области. Необъективные модели и предвзятые, опасные или неточные результаты	Управление данными. Безопасность данных. Применение надежных подходов ML
Категориальные признаки с высокой кардинальностью: такие признаки, как почтовые индексы или идентификаторы товаров, которые представляют множество категориальных уровней одного и того же атрибута	Переобучение моделей и неточные результаты. Неприемлемо длительное время обучения. Ненадежные или опасные прогнозы вне предметной области	Варианты кодирования цели или признака, среднее значение по уровню (или аналогичное, например медианное, BLUP). Дискретизация. Различные методы встраивания, например нейронные сети для встраивания сущностей, машины факторизации

Книги для программистов: https://t.me/free_books_for_it

Таблица 3.1 *(продолжение)*

Проблема	Типичные симптомы	Возможное решение
Несбалансированная цель: когда один целевой класс или значение встречается гораздо чаще других	Модель предсказывает один и тот же класс. Предвзятые прогнозы модели	Пропорциональная избыточная или недостаточная выборка. Взвешивание инверсной априорной вероятности. Модели смешения, например методы регрессии. Апостериорная корректировка прогнозов или порогов принятия решений
Неполные данные: когда набор данных не содержит информации об интересующем явлении, или когда несобранная информация искажает результаты моделирования	Бесполезные модели, бессмысленные или опасные результаты	Проконсультируйтесь с экспертами в данной области и заинтересованными сторонами. Применяйте научный метод и подходы, основанные на планировании эксперимента (DOE). (Получайте больше данных. Получайте более качественные данные)
Пропущенные значения: когда в определенных строках или экземплярах отсутствует информация	Потеря информации. Необъективные модели и предвзятые, опасные или неточные результаты	Подстановка данных. Разбиение на интервалы (бининг). Выбор подходящего алгоритма, например древовидные модели, наивная байесовская классификация
Шум: данные, не содержащие четких сигналов для моделирования. Данные с одинаковыми входными и разными целевыми значениями	Ненадежные или опасные прогнозы вне предметной области. Низкая точность во время обучения	Проконсультируйтесь с экспертами в данной области и заинтересованными сторонами. Применяйте научный метод и подходы, основанные на DOE. (Получайте больше данных. Получайте более качественные данные)
Ненормированные данные: данные, в которых значения для одной и той же сущности представлены по-разному.	Ненадежные или опасные прогнозы вне предметной области. Неприемлемо длительное время обучения. Ненадежные оценки параметров и генерация правил	Тщательная очистка данных, основанная на консультациях с экспертами в предметной области
Выбросы: строки или экземпляры данных, которые кажутся странными или непохожими на другие	Необъективные модели и предвзятые, опасные или неточные результаты. Ненадежные оценки параметров и генерация правил. Ненадежные прогнозы за пределами предметной области	Разбиение на интервалы (бининг). Винсоризация. Надежные функции потерь, например функции потерь Хьюбера
Разреженные данные: данные с большим количеством нулей или отсутствующих значений; данные, которые не содержат достаточно информации об интересующем явлении	Неприемлемо длительное время обучения. Бессмысленные или опасные результаты из-за недостатка информации, проклятия размерности или неполной идентификации модели	Подходы с извлечением признаков или матричной факторизацией. Соответствующее представление данных (например, COO, CSR). Применение бизнес-правил, утверждений и ограничений для компенсации нелогичного поведения модели, изученной в разреженных областях обучающих данных
Сильная мультиколлинеарность (корреляция): когда признаки имеют сильную линейную зависимость друг от друга	Нестабильные оценки параметров, нестабильная генерация правил и опасные или нестабильные прогнозы	Отбор признаков. Извлечение признаков. Регуляризация L2

Книги для программистов: https://t.me/free_books_for_it

Таблица 3.1 *(окончание)*

Проблема	Типичные симптомы	Возможное решение
Нераспознанные форматы времени и даты: разные форматы времени и даты в одном наборе данных, неправильно закодированные программами обработки данных или моделирования	Ненадежные или опасные прогнозы за пределами предметной области. Ненадежные оценки параметров и генерация правил. Переобучение моделей и неточные результаты. Слишком оптимистичные оценки эффективности in silico	Тщательная очистка данных, основанная на консультациях с экспертами в данной области
Широкие данные: данные, содержащие гораздо больше столбцов, признаков, пикселов или токенов, чем строк, экземпляров, изображений или документов. *P >> N*	Неприемлемо длительное обучение. Бессмысленные или опасные результаты из-за проклятия размерности или неполной идентификации модели	Отбор признаков. Извлечение признаков. Регуляризация L1, модели, не предполагающие *N >> P*

С данными может произойти много такого, что приведет к ненадежной или опасной работе модели в приложениях высокого риска. Заманчиво думать, что конструирование признаков способно решить наши проблемы с качеством данных. Но конструирование признаков хорошо ровно настолько, насколько хороши идея и код, использованные для его реализации. Если мы не будем предельно осторожны с конструированием признаков, то, скорее всего, создадим себе еще больше ошибок и сложностей. Обычно в конвейерах машинного обучения встречаются следующие проблемы с конструированием признаков:

- несоответствие API или версий между пакетами очистки, предварительной обработки и вывода данных;
- не удается применить все этапы очистки и преобразования данных во время вывода;
- не удается правильно подобрать избыточную или недостаточную выборку во время вывода;
- неспособность корректно или безопасно обрабатывать незнакомые данные во время вывода.

Конечно, при подготовке данных, конструировании признаков и связанных с ними конвейеров может возникнуть множество других проблем, особенно по мере того, как типы данных, которые алгоритмы ML могут принимать для обучения, становятся более разнообразными. Инструменты, которые обнаруживают и решают такие проблемы, также являются важной частью набора инструментов для анализа данных. Для пользователей Python Pandas такую роль играет инструмент ydata-profiling (ранее pandas-profiler) (*https://oreil.ly/EDNSC*) — это наглядное средство, которое помогает обнаружить многие основные проблемы с качеством данных. У пользователей R есть выбор, описанный Матеушем Станьяком и Пшемыславом Бичеком в статье «Обзор пакетов R для автоматизированного исследовательского анализа данных» (*https://oreil.ly/1cBlv*).

Книги для программистов: https://t.me/free_books_for_it

Спецификация модели для вывода в реальных условиях

Когда наш конвейер подготовки данных и конструирования признаков готов, наступает время подумать о спецификации модели ML. На практике соглашения относительно реальной надежности и безопасности сильно отличаются показателей, опубликованных в таблицах результатов лидеров соревнований по машинному обучению. Хотя валидация модели и измерение ошибок при тестировании попрежнему имеют значение, на практике наивысшим приоритетом пользуются более важные вопросы: точное представление данных и здравый смысл явлений реального мира. В данном разделе мы рассмотрим ориентацию модели на безопасность и надежность. Мы хотим подчеркнуть важность бенчмарков и альтернативных моделей, калибровки, конструктной валидности, допущений и ограничений, правильных функций потерь и избегания множественных сравнений. Кроме того, мы будем использовать новые подходы к надежному, устойчивому и безопасному ML.

Бенчмарки и их альтернативы

Приступая к задаче моделирования ML, лучше всего начать с проверенного алгоритма обучения и в идеале воспроизвести все тесты, связанные с этим алгоритмом. В то время как академические алгоритмы редко отвечают всем требованиям сложных бизнес-задач, использование хорошо известных алгоритмов и тестов может обеспечить базовую гарантию правильности реализации алгоритма обучения. Когда эта проверка будет пройдена, можно приступать к реализации сложного алгоритма, отвечающего конкретным особенностям текущей задачи.

Наряду со сравнением бенчмарков, еще одной передовой методикой для повышения безопасности и производительности является оценка многочисленных альтернативных алгоритмических подходов. Обучение в соответствии со множеством различных алгоритмов и обоснованный выбор лучшего из многих вариантов для окончательного развертывания обычно ведут к созданию моделей более высокого качества, поскольку увеличивают количество оцениваемых моделей и заставляют пользователей понимать различия между ними. Более того, оценка альтернативных подходов важна для соблюдения различных стандартов, исключающих дискриминацию и халатность. В целом эти стандарты требуют наличия доказательств того, что перед внедрением были оценены различные технические варианты и был найден соответствующий компромисс между защитой потребителей и потребностями бизнеса.

Калибровка

Тот факт, что число от 0 до 1 появляется из сложного конвейера ML, не делает его более вероятным. Чтобы некалиброванные вероятности, генерируемые большинством классификаторов ML, можно было рассматривать в качестве вероятностей реального мира, их обычно подвергают постобработке. В таких случаях чаще всего применяют процесс масштабирования или даже другую модель, чтобы гарантировать, что когда конвейер выдает 0,5, рассматриваемое событие действительно произошло примерно с 50% аналогичных объектов в имеющихся исторических данных. Пакет scikit-learn содержит некоторые диагностические функции для калиб-

Книги для программистов: https://t.me/free_books_for_it

ровки классификатора ML. Проблемы с калибровкой также могут повлиять на модели регрессии, когда распределение выходных данных модели не соответствует распределению известных результатов. Например, многие числовые величины в страховании обычно не имеют нормального распределения. Использование по умолчанию функции квадратичных потерь вместо гамма-функций или распределения Твиди может привести к прогнозам, которые не распределяются так же, как значения из известного базового процесса генерации данных. Когда речь заходит о калибровке, фундаментальная проблема заключается в том, что прогнозы рабочей модели ML не соответствуют действительности. Мы никогда не сможем делать хорошие прогнозы и принимать приемлемые решения. Мы стремимся к тому, чтобы вероятности наших прогнозов были согласованы с историческими показателями результатов. Для этого нам нужны регрессионные модели, которые будут генерировать прогнозы с тем же распределением, что и смоделированный процесс генерации данных.

Конструктная валидность

Конструктная валидность (construct validity) — это понятие из области социальных наук (в частности, психометрии и тестирования). Конструктная валидность означает наличие разумного научного основания полагать, что результаты теста достоверно характеризуют предполагаемый конструкт. Другими словами, существуют ли какие-либо научные доказательства того, что вопросы и результаты стандартизированного теста могут предсказать успеваемость в колледже или достижения на работе? Почему мы упоминаем об этом в книге про ML? Потому что модели ML в наши дни часто используют для тех же целей, что и психометрические тесты, и, по нашему мнению, многие модели ML не обладают конструктной валидностью. Хуже того, алгоритмы машинного обучения, не соответствующие фундаментальным структурам обучающих данных или своей реальной области применения, могут вызвать серьезные инциденты.

Рассмотрим выбор между моделью ML и линейной моделью — при этом многие из нас просто по умолчанию предпочтут модель ML. Выбор алгоритма машинного обучения для задачи моделирования предполагает множество исходных предположений — по сути, взаимодействие высокого порядка и нелинейность входных признаков являются важными движущими силами прогнозируемого явления. И наоборот, выбор в пользу линейной модели неявно преуменьшает значение взаимодействий и нелинейностей. Если эти качества важны для хороших прогнозов, их придется явно указать для линейной модели. В любом случае важно проанализировать, как основные эффекты, корреляции и локальные зависимости, взаимодействия, нелинейности, кластеры, выбросы и иерархии в обучающих данных или в реальности будут обрабатываться алгоритмом моделирования, а также протестировать эти механизмы. Для достижения оптимальной безопасности и качества прогнозов после развертывания в моделях ML также должны быть представлены зависимости от времени, географического местоположения или связей между объектами в различных типах сетей. Без этих четких связей с реальностью модели МО не обладают конструктной валидностью и вряд ли будут демонстрировать хорошие результаты in vivo. Конструирование признаков, ограничения, функции потерь,

Книги для программистов: https://t.me/free_books_for_it

архитектура моделей и другие механизмы — все это способы приблизить модель к решаемой задаче реального мира.

Предположения и ограничения

Смещения, запутанность, неполнота, зашумленность, диапазоны, разреженность и другие основные характеристики обучающих данных начинают определять допущения и ограничения наших моделей. Как уже говорилось, алгоритмы и архитектуры моделирования также содержат предположения и ограничения. Например, древовидные модели обычно не могут экстраполировать данные за пределы диапазонов обучающих данных. Гиперпараметры для алгоритмов машинного обучения — это еще одна область, где скрытые предположения могут вызвать проблемы с безопасностью и надежностью. Гиперпараметры можно выбирать на основе знаний предметной области или с помощью технических подходов, таких как поиск по сетке и байесовская оптимизация. Главное — не соглашаться на настройки по умолчанию, систематически выбирать настройки и не обманывать себя из-за проблемы множественного сравнения. Проверка независимости ошибок между строками и признаками в обучающих данных или построение графиков остатков и поиск устойчивых закономерностей — это общие и проверенные временем методы включения в модель некоторых основных предположений. Маловероятно, что нам когда-нибудь удастся обойти все предположения и ограничения наших данных и модели. Поэтому необходимо документировать любые неучтенные или подразумеваемые предположения и ограничения в документации модели и делать так, чтобы пользователи понимали, какое использование модели может нарушить ее предположения и ограничения. Юридически это будет считаться использованием за пределами области применения или не по назначению — точно так же, как применение отпускаемого по рецепту лекарства ненадлежащим образом. Между прочим, конструктная валидность связана с документацией модели и механизмами управления рисками, ориентированными на ограничения и предположения модели. Специалисты по надзору требуют, чтобы специалисты-практики проработали гипотезу, лежащую в основе их модели, изложили ее в письменном виде и убедились, что она подкреплена валидными конструктами, а не предположениями.

Функции потерь по умолчанию

Другое часто невысказанное предположение, которое присутствует во многих алгоритмах обучения, касается квадратичных функций потерь. Многие алгоритмы машинного обучения по умолчанию используют квадратичную функцию потерь. В большинстве случаев квадратичная функция потерь, аддитивная по наблюдениям и имеющая линейную производную, является скорее вопросом математического удобства, чем чего-либо еще. Современные инструменты, такие как autograd (*https://oreil.ly/8icjS*), делают это удобство все менее нужным. Вы должны соотносить свой выбор функции потерь с предметной областью.

Множественные сравнения

Выбор модели в ML часто означает попытку опробовать множество различных наборов входных признаков, гиперпараметров и других настроек модели, таких как

Книги для программистов: https://t.me/free_books_for_it

пороговые значения вероятности. Мы часто применяем пошаговый отбор признаков, поиск по сетке или другие методы, которые позволяют экспериментировать с множеством разных настроек на одном и том же наборе валидационных или контрольных данных. Статистики могли бы назвать это проблемой *множественных сравнений*. Они, скорее всего, указали бы, что чем больше сравнений мы проводим, тем больше вероятность наткнуться на некоторые настройки, которые просто хорошо выглядят на проверочном наборе данных. Это коварная разновидность переобучения, которая возникает, когда мы слишком много раз повторно используем одни и те же данные, выбираем признаки, гиперпараметры или другие настройки, которые лучше всего работают при тестировании, а в дальнейшем сталкиваемся с плохой производительностью in vivo. Следовательно, нужно выбирать подходы с возможностью повторного использования данных, которые изменяют или передискретизируют данные, чтобы сделать наш признак, гиперпараметр или другие настройки более обобщаемыми.

Будущее безопасного и надежного машинного обучения

Новая область *надежного ML* (robust ML) ориентирована на создание новых алгоритмов с улучшенными характеристиками стабильности и безопасности. Различные исследователи создают новые алгоритмы обучения с гарантиями оптимальности, например оптимальные разреженные деревья решений (*https://oreil.ly/gOmtg*). Исследователи собрали отличные учебные материалы по безопасности и надежности машинного обучения. Сегодня эти подходы требуют индивидуальных реализаций и дополнительной работы, но мы надеемся, что в скором времени они станут более широко доступными.

Отладка модели

После того как модель правильно определена и обучена, следующими шагами в процессе обеспечения технической безопасности и надежности является тестирование и отладка. В прошлые годы такие оценки были сосредоточены на совокупном качестве и частоте ошибок при работе с проверочными данными. Поскольку модели ML включены в общедоступные масштабные системы, а количество публично раскрываемых инцидентов с ИИ резко возрастает, становится ясно, что требуется более строгая проверка. В связи с этим в машинном обучении появилась новая область *отладки моделей* (model debugging) (*https://oreil.ly/IY0gU*). При отладке модели ML рассматриваются скорее как код, а не как абстрактные математические вычисления. Здесь находят применение специальные методы тестирования для поиска недостатков программного обеспечения, логических ошибок, неточностей и уязвимостей безопасности в моделях и конвейерах систем ML. Разумеется, эти ошибки также необходимо исправлять по мере их обнаружения. Далее мы довольно подробно рассмотрим отладку модели, начиная с базовых и традиционных подходов, затем опишем распространенные ошибки, которые пытаемся найти, перейдем к специализированным методам тестирования и завершим раздел обсуждением методов исправления ошибок.

Книги для программистов: https://t.me/free_books_for_it



В дополнение ко многим объяснимым моделям машинного обучения пакет с открытым исходным кодом PiML (*https://oreil.ly/1O3hi*) содержит исчерпывающий набор инструментов отладки для моделей ML, обученных на структурированных данных. Даже если он не совсем подходит для вашего варианта использования, это отличное место, чтобы узнать больше и найти идеи для отладки модели.

Тестирование программного обеспечения

Базовое тестирование программного обеспечения становится гораздо более значимым, когда мы перестаем фокусироваться только на красивых цифрах и впечатляющих таблицах результатов как на конечной цели задачи обучения модели ML. Развернутые в производстве системы машинного обучения должны корректно работать при различных обстоятельствах. Обеспечение работоспособности программного обеспечения представляет собой более точную науку, чем что-либо другое, связанное с системами ML. Лучшие приемы тестирования программного обеспечения хорошо известны, и во многих случаях их можно даже автоматизировать. Как минимум критически важные системы машинного обучения должны пройти следующие этапы:

Модульное тестирование

Все функции, методы, подпрограммы или другие блоки кода должны иметь связанные с ними тесты, чтобы гарантировать, что они ведут себя должным образом, точно и воспроизводимо. Это гарантирует надежность модулей системы ML.

Интеграционное тестирование

Все API и интерфейсы между модулями, уровнями или другими подсистемами должны быть протестированы на предмет наличия правильных связей. Несоответствие API после изменений внутреннего кода — это классический вариант отказа систем ML. Используйте интеграционное тестирование, чтобы обнаружить эту и другие ошибки интеграции.

Функциональное тестирование

Функциональное тестирование применяют к пользовательским интерфейсам и конечным точкам системы ML, чтобы гарантировать, что после развертывания они будут вести себя должным образом.

Хаос-тестирование

Тестирование в хаотичных и враждебных условиях может быть чрезвычайно полезным, если предполагается, что системы ML могут столкнуться со сложными и неожиданными сценариями in vivo. Поскольку предсказать все варианты сбоя системы ML может быть сложно, хаос-тестирование помогает исследовать более широкий класс режимов сбоя и обеспечить некоторую защиту от так называемых «неизвестных неизвестных»[2].

[2] Существуют «известные неизвестные», типа переменной *x* в уравнении, когда мы не знаем ее значение, но хотя бы знаем, что эта переменная существует, и «неизвестные неизвестные», когда мы можем только предполагать, что существует нечто такое, о чем мы не знаем. Хаос-тестирование направлено в том числе на выявление наличия «неизвестных неизвестных». — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

Для дальнейшего повышения качества следует добавить два дополнительных теста, специфичных для ML:

Случайная атака

Случайные атаки — один из способов провести хаос-тестирование в машинном обучении. Случайные атаки нагружают модели ML огромными объемами случайных данных, что позволяет выявить как программные, так и математические проблемы. Реальный мир весьма хаотичен. Наша система ML встретит данные, к которым она не готова. Случайные атаки могут уменьшить количество таких случаев и любых связанных с ними сбоев или инцидентов.

Бенчмаркинг

Используйте тесты для отслеживания улучшений системы на протяжении срока эксплуатации. Системы ML могут быть невероятно сложными. Как мы можем узнать, повлияют ли три строки кода, которые инженер изменяет сегодня, на качество системы в целом? Если показатели системы воспроизводимы и оцениваются до и после изменений, ответить на такие вопросы гораздо проще.

ML — это программа. Поэтому все испытания, проводимые с традиционными корпоративными программными ресурсами, следует также проводить и с важными системами ML. Если вы не знаете, с чего начать отладку модели, начните со случайных атак. Читателей могут шокировать математические или программные ошибки, которые случайные данные могут выявить в системах ML. Когда вы сможете добавить тесты к конвейерам непрерывной интеграции / непрерывной разработки (CI/CD) вашей организации, это станет еще одним большим шагом на пути к обеспечению безопасности и надежности систем ML.



Вероятно, самый простой и эффективный способ начать отладку модели — это случайные атаки. Если отладка кажется вам сложной или вы не знаете, с чего начать, начните со случайных атак.

Традиционная оценка модели

Как только вы почувствуете уверенность в том, что код вашей системы ML работает должным образом, вам будет легче сосредоточиться на тестировании математических алгоритмов машинного обучения. Важно учитывать стандартные показатели качества модели. Но это не конец процесса проверки и отладки — это его начало. Хотя точные значения и десятичные точки по-своему важны, в реальном мире они имеют гораздо меньшее значение, чем в таблице лидеров соревнований по ML. При анализе показателей модели в предметной области речь идет не столько о точных числовых значениях оценок, сколько о сопоставлении характеристик in silico с характеристиками in vivo.

Если возможно, постарайтесь выбрать статистические оценочные данные, которые имеют логическую интерпретацию и практические или статистические пороговые значения. Например, показатель RMSE можно рассчитать для многих типов задач прогнозирования, и, что особенно важно, его можно интерпретировать в целевых

Книги для программистов: https://t.me/free_books_for_it

единицах. Площадь под кривой для задач классификации ограничена от 0,5 на нижней границе до 1,0 на верхней. Такие меры оценки позволяют здраво интерпретировать качество модели ML и сравнивать ее с широко распространенными пороговыми значениями. Также важно использовать более одной метрики и анализировать показатели качества модели по важным сегментам ваших данных, а также по разделам данных для обучения, валидации и тестирования. При сравнении показателей на сегментах обучающих данных важно, чтобы наблюдались примерно одинаковые значения. Потрясающие показатели для одного крупного клиентского сегмента и низкие для всех остальных будут хорошо смотреться в средних статистических значениях оценки, таких как RMSE. Однако эта ситуация будет выглядеть намного хуже, если она нанесет ущерб бренду из-за множества недовольных клиентов. Большая разница показателей в разных сегментах также может быть признаком неполной идентификации — серьезной ошибки ML, которую мы рассмотрим в этой главе. Показатели модели на данных обучения, проверки и тестирования обычно анализируют на предмет недостаточного обучения и переобучения. Как и в случае с остальными показателями модели, мы можем искать переобучение и недостаточное обучение по всем данных или по сегментам.

Еще одним практическим соображением, связанным с традиционной оценкой модели, является выбор порогового значения вероятности. Большинство классифицирующих моделей машинного обучения генерируют числовые вероятности, а не дискретные решения. Числовые пороговые значения вероятности для сопоставления с фактическими решениями можно выбирать различными способами. Хотя всегда возникает соблазн максимизировать какой-либо сложный показатель оценки, также полезно учитывать влияние на реальный мир. Рассмотрим классический пример кредитования. Скажем, порог модели вероятности дефолта изначально установлен на уровне 0,15. Это означает, что каждый, кто набирает вероятность дефолта менее 0,15, получает одобрение кредита, а тем, кто набирает балл, равный пороговому значению или выше, отказывают. Подумайте над такими вопросами:

- Какова ожидаемая денежная прибыль для этого порога? Каков финансовый риск?
- Сколько человек получат кредит при этом пороге?
- Сколько среди них будет женщин? Сколько членов групп меньшинств?

Всегда полезно оценивать показатели модели в предметной области *за пределами пороговых значений* вероятности, потому что часто нас волнует именно это. Хорошо знать точные пороговые значения, но на самом деле важнее заработать деньги, а не потерять их, или даже спасти жизни, а не отнять их. Мы можем сделать первый шаг к пониманию реальной ценности показателей, присвоив денежные или другие значения каждой ячейке матрицы путаницы для задач классификации или каждой остаточной единице для задач регрессии. Сделайте набросок на клочке бумаги. Как вы думаете, ваша модель заработает или потеряет деньги? Как только вы поймете суть такого рода оценки, вы сможете даже включить уровни стоимости для различных результатов модели непосредственно в функции потерь ML. Это позволит вам оптимизировать модель, наиболее подходящую для развертывания.

Книги для программистов: https://t.me/free_books_for_it

Показатели ошибок и точности всегда будут важны для машинного обучения. Но когда алгоритмы применяются в реальных системах ML, их числовые показатели имеют не такое большое значение, как при публикации статей и участии в конкурсах по работе с большими данными. Иными словами, продолжайте использовать традиционные меры оценки, но попытайтесь сопоставить их с безопасностью и надежностью модели в реальном мире.

Распространенные ошибки ML

Мы обсудили отсутствие воспроизводимости, проблемы с качеством данных, правильную спецификацию модели, ошибки программного обеспечения и традиционную оценку. Но в сложных системах машинного обучения и многое другое может пойти не так. Когда дело доходит до математики ML, здесь встречаются как несколько неожиданных ошибок, так и множество хорошо известных ловушек. Далее мы обсудим некоторые типичные ошибки, в том числе смещение распределений, эпистемическую неопределенность, слабые места, нестабильность, утечку, зацикленные входные данные, переобучение, ложное обучение, недостаточное обучение и неполную идентификацию.

Словарь терминов в области надежности и отказоустойчивости

В соответствии с методологией управления рисками искусственного интеллекта NIST *устойчивость* (resilience) является синонимом *безопасности* (security), а *робастность* (robustness) и *надежность* (reliability) частично используются как синонимы. Традиционно в соответствии со стандартом ISO/IEC TS 5723:2022 (*https://oreil.ly/U6T_b*) надежность связана со «способностью объекта работать должным образом, без сбоев, в течение заданного интервала времени и в заданных условиях». В машинном обучении и статистике надежность модели можно проверить при ковариативном возмущении (или смещении распределения). Робастность также стала ассоциироваться с надежным (робастным) ML или методами, которые помогают защитить модели от манипулирования злоумышленниками с помощью состязательных примеров и отравления данных.

Отсутствие робастности и надежности может быть следствием многих ошибок, обсуждаемых в этой главе и книге. В этой главе встречаются такие термины, как *нестабильность* (instability), *смещение распределения* (distribution shift) и соответствующий словарь машинного обучения (например, *утечка данных* (data leakage), *эпистемическая неопределенность* (epistemic uncertainty), *неполная идентификация* (underspecification)), в описаниях проблем, связанных с отсутствием робастности и надежности. Глава 5 посвящена безопасности систем машинного обучения.

Смещение распределений

Смещение базовых данных между различными разделами обучающих данных и после развертывания модели является распространенной причиной сбоя в системах машинного обучения. Независимо от того, выходит ли на рынок новый конкурент или случается разрушительная всемирная пандемия, мир является динамичным местом. К сожалению, большинство современных систем машинного обучения

Книги для программистов: https://t.me/free_books_for_it

изучают закономерности на основе статичных снимков обучающих данных и пытаются применить эти закономерности к новым данным. Иногда эти данные представляют собой разделы проверки или тестирования. Иногда это живые данные в очереди производственного скоринга. Тем не менее дрейфующее распределение входных признаков — это серьезная ошибка, которую необходимо выявить и устранить.



Системы, основанные на адаптивном обучении в режиме реального времени или обучении с подкреплением или обновляющиеся с минимальным вмешательством человека, подвержены серьезным состязательным манипуляциям, распространению ошибок, петле обратной связи, рискам надежности и робастности. Хотя эти системы обычно опираются на самые передовые технологии, они требуют качественного управления рисками.

При обучении моделей ML следите за смещением распределения между обучающими, перекрестными, валидационными или тестовыми наборами с использованием *индекса стабильности популяции* (population stability index, PSI), тестов Колмогорова — Смирнова (Kolmogorov — Smirnov, KS), t-тестов или других соответствующих показателей. Если признак имеет разное распределение в разных обучающих разделах, отбросьте его или подвергните сильной регуляризации. Еще один разумный тест на смещение распределения, который следует проводить во время отладки, — это симулировать смещение распределения для потенциальных условий развертывания и повторно измерять качество модели, уделяя особое внимание строкам с низкими показателями. Если вас беспокоит, как модель будет работать во время рецессии, смоделируйте смещение распределения, отражающее увеличение числа просроченных платежей, снижение денежных потоков и рост остатков по кредитам. Затем посмотрите, как работает ваша модель. Также крайне важно записывать и хранить информацию о распределениях обучающих данных. Это позволит легко обнаружить смещение после развертывания.

Эпистемическая неопределенность и разреженность данных

Эпистемическая неопределенность — это причудливый способ обозначить нестабильность и ошибки, возникающие из-за недостатка знаний. В машинном обучении модели традиционно получают знания из обучающих данных. Если какие-то части наших больших многомерных обучающих данных являются разреженными, вполне вероятно, что наша модель будет иметь высокую степень неопределенности в этой области. Звучит надуманно? Вовсе нет. Рассмотрим традиционную модель кредитования. У нас обычно есть много данных о людях, которые уже имеют кредитные карты и оплачивают свои счета, но нам, как правило, не хватает данных о людях, у которых нет кредитных карт (данные об их прошлых кредитных картах не существуют) или которые не оплачивают свои счета с помощью карты (потому что подавляющее большинство клиентов их оплачивают). Легко понять, как выдавать кредитные карты людям с высоким кредитным рейтингом, которые оплачивают свои счета. Намного труднее принимать решения в отношении людей с неизвестной, более короткой или неровной кредитной историей. Отсутствие данных о людях, о которых нам действительно нужно знать, может привести к серьезным проблемам эпистемической неопределенности. Если только некоторые редкие клиенты

Книги для программистов: https://t.me/free_books_for_it

из миллионов задерживают платежи по кредиту на четыре или пять месяцев, то модели ML просто не хватает знаний о том, как лучше всего обращаться с такими людьми.

Это явление проиллюстрировано далее в подразделе «Неполная идентификация», где модель не имеет смысла для людей, которые просрочили свой последний платеж более чем на два месяца. Эту область плохой и, вероятно, нестабильной работоспособности модели иногда называют *слабым местом* (weak spot). Знание совокупных ошибок или характеристик модели не помогает найти эти слабые места. Это лишь одна из причин, по которой модели тщательно тестируют на разных сегментах обучающих или контрольных данных. Именно поэтому мы объединяли графики частичной зависимости и отдельных условных ожиданий с гистограммами в главе 2. На этих графиках мы можем увидеть, подтверждается ли поведение модели обучающими данными или нет. После того как мы определили разреженную область данных, приводящую к эпистемической неопределенности и слабым местам, нам обычно приходится обращаться к человеческому знанию. Это может включать в себя ограничение формы модели, чтобы она вела себя в соответствии с опытом экспертов в предметной области, дополнение модели бизнес-правилами, а также передачу случаев, попадающих в разреженные области, операторам-людям для принятия сложных решений.

Нестабильность

Модели машинного обучения могут проявлять нестабильность или отсутствие робастности в процессе обучения или при прогнозировании на основе реальных данных. Нестабильность вследствие обучения часто связана с недостаточным объемом обучающих данных, разреженными областями обучающих данных, сильно коррелированными признаками внутри обучающих данных или формами моделей с высокой дисперсией, такими как глубокие одиночные деревья решений. Перекрестная проверка — типичный инструмент обнаружения нестабильности во время обучения. Если модель демонстрирует заметно разные показатели ошибок или точности при перекрестной проверке, значит, вы столкнулись с проблемой нестабильности. Нестабильность, вызванную обучением, часто можно исправить с помощью более качественных данных и моделей с меньшей дисперсией, таких как ансамбли деревьев решений. Графики ALE или ICE помогают выявить нестабильность прогнозов в области разреженных обучающих данных. После этого можно применить методы анализа чувствительности: возмущения, моделирование, стресс-тестирование и поиск состязательных примеров.



Есть два простых признака нестабильности в ML:

- когда небольшое изменение входных данных приводит к значительному изменению выходных данных;
- когда добавление небольшого количества обучающих данных приводит к совершенно другой модели при повторном обучении.

Если исследование поверхности отклика или границы решения вашей модели с помощью этих методов обнаруживает резкие колебания в прогнозах или кривые ALE

Книги для программистов: https://t.me/free_books_for_it

или ICE колеблются, особенно в области высоких или низких значений признаков, это также говорит о наличии нестабильности. Этот тип нестабильности часто можно устранить с помощью ограничений и регуляризации. В главе 8 приведены примеры кода, выполняющего исправления.

Утечка

Утечка информации между разделами данных для обучения, валидации и тестирования происходит, когда информация из разделов валидации и тестирования попадает в раздел обучающих данных, что приводит к чрезмерно оптимистичным показателям ошибок и точности. Утечка может произойти по разным причинам, включая следующие:

Конструирование признаков

При неправильном использовании некоторые методы конструирования признаков, такие как подстановка или анализ главных компонент, могут испортить обучающие данные информацией из данных для валидации и тестирования. Чтобы избежать утечек такого рода, выполняйте конструирование признаков единообразно, но строго отдельно для всех сегментов обучающих данных. Или сделайте так, чтобы информация, применяемая для подстановки, такая как средние значения и моды, рассчитывалась на обучающих данных и применялась к данным валидации и тестирования, а не наоборот.

Неправильное обращение с данными на оси времени

Не используйте будущее, чтобы предсказать прошлое. Большинство данных имеют некоторую связь со временем, или явную, как в случае с данными временных рядов, или каким-то образом опосредованную. Неправильное обращение с данными или нарушение этой связи при помощи случайной выборки является распространенной причиной утечек. Если мы имеем дело с данными, где время играет роль, его необходимо учитывать при построении схем проверки модели. Основное правило заключается в том, что самые ранние данные должны находиться в обучающих разделах, а более поздние данные необходимо разделить на сегменты тестирования и валидации (также в зависимости от времени). Надежный (и бесплатный) обзор лучших методов прогнозирования временных рядов представлен в книге Forecasting: Principles and Practice (OTexts) (*https://oreil.ly/R2y6N*).

Идентичные сущности

Иногда один и тот же человек, финансовая транзакция или другая моделируемая сущность могут находиться в нескольких разделах обучающих данных. В этом случае следует позаботиться о том, чтобы модели ML не запоминали характеристики таких объектов и не применяли эти индивидуальные закономерности к другим сущностям в новых данных.

Часто сохранение нетронутого набора данных для последующей честной оценки модели после развертывания помогает выявить различные ошибки, связанные с утечкой. Если ошибки или точность модели на таком контрольном наборе данных выглядят гораздо менее радужно, чем на тестовом наборе во время разработки мо-

Книги для программистов: https://t.me/free_books_for_it

дели, это может свидетельствовать о наличии утечки. Более сложные схемы моделирования могут значительно усложнить предотвращение и обнаружение утечек. Однако даже в этих случаях применимо основное эмпирическое правило: не используйте данные, задействованные в процессе обучения или выбора модели, для ее реалистичной оценки. Использование более сложных архитектур, например со стеками или гейтами, означает, что нам нужно больше контрольных данных для различных этапов проверки этих сложных моделей, чтобы сделать точное предположение о качестве in vivo. Для любой системы машинного обучения также необходимы универсальные меры контроля, такие как тщательное документирование схем проверки данных и мониторинг модели при развертывании.

Петля обратной связи

Поскольку системы ML обычно являются частью более обширных мероприятий по цифровизации или внедряются для поддержки принятия решений, несколько систем, управляемых данными, часто взаимодействуют между собой. В этих случаях могут возникнуть неполадки, связанные с *распространением ошибок* и *обратной связью*. Распространение ошибок происходит, когда небольшие ошибки в одной системе вызывают или усиливают ошибки в другой системе. Петли обратной связи — это ситуация, при которой система ML может потерпеть неудачу, если окажется права. Петли обратной связи возникают, когда система ML влияет на свою среду, а затем эти эффекты включаются в обучающие данные системы. К примерам петель обратной связи можно отнести случаи, когда прогнозирующая работа полиции приводит к чрезмерному контролю над определенными районами или когда алгоритмы трудоустройства усугубляют проблемы разнообразия при найме, постоянно рекомендуя правильных, но неразнообразных кандидатов. Зависимости между системами должны быть документированы, а развернутые модели необходимо отслеживать на предмет распространения ошибок или петель обратной связи.

Переобучение

Переобучение происходит, когда сложный алгоритм машинного обучения запоминает слишком много конкретной информации из обучающих данных, но не усваивает скрытые в них закономерности, чтобы хорошо работать с незнакомыми данными после развертывания. Переобучение часто случается с моделями, которые обладают высокой дисперсией или слишком сложны для имеющихся данных. Переобучение обычно проявляется в намного более высокой точности прогнозов на обучающих данных по сравнению с валидационными и тестовыми данными. Поскольку переобучение встречается очень часто, разработано множество возможных решений, но большинство из них предполагает уменьшение дисперсии в выбранной нами модели. Вот несколько примеров таких решений:

Ансамбли моделей

Известно, что ансамблевые методы, в частности бутстреп-агрегирование (т. е. бэггинг) и градиентный бустинг, уменьшают ошибку в одиночных моделях с высокой дисперсией. Столкнувшись с переобучением, попробуйте применить

Книги для программистов: https://t.me/free_books_for_it

один из ансамблевых методов. Только имейте в виду, что при переходе от одной модели к ансамблю мы можем уменьшить переобучение и нестабильность, но также, вероятно, потеряем интерпретируемость.

Снижение архитектурной сложности

Нейронные сети могут иметь слишком много скрытых слоев или скрытых модулей. Ансамблевые модели могут иметь слишком много базовых обучаемых блоков. Деревья могут быть слишком глубокими. Если вы подозреваете, что наблюдаете переобучение, попробуйте сделать архитектуру модели менее сложной.

Регуляризация

Регуляризация — обобщенный термин, обозначающий различные сложные математические подходы для уменьшения строгости, сложности или количества изученных правил или параметров в модели ML. На практике многие современные модели ML предусматривают несколько вариантов регуляризации, поэтому обязательно воспользуйтесь возможностями модели, чтобы уменьшить вероятность переобучения.

Более простые семейства моделей гипотез

Некоторые модели машинного обучения сложнее остальных, хотя решают одну и ту же задачу. Если ваша нейронная сеть или GBM страдает от переобучения, попробуйте применить менее сложное дерево решений или линейную модель.

Переобучение традиционно считается ахиллесовой пятой машинного обучения. Хотя это одна из наиболее часто встречающихся ошибок, это также лишь один из многих возможных технических рисков, которые следует учитывать с точки зрения безопасности и надежности. Как и в случае с утечками, по мере возрастания сложности систем ML переобучение становится более коварным и незаметным. Всегда сохраняйте нетронутый контрольный набор, с помощью которого можно оценить реальные показатели модели перед развертыванием. Для предотвращения переобучения также необходимо применять более традиционные меры контроля, такие как документирование схем проверки, мониторинг моделей и A/B-тестирование моделей на реальных данных.

Ложное обучение

Ложное обучение (shortcut learning, обходное обучение) происходит, когда считается, что сложная система ML изучает и принимает решения по одному аспекту, например аномалиям при сканировании легких или результатам собеседования, но на самом деле она изучает некоторые более простые связанные признаки, такие как регистрационные номера автомобилей или фон видеозвонков Zoom. Ложное обучение, как правило, возникает из-за запутанных взаимосвязей в обучающих данных, отсутствия конструктной валидности и неспособности разработчика адекватно установить и задокументировать предположения и ограничения. Мы используем объяснимые модели и объяснимые методы искусственного интеллекта, чтобы понять, какие механизмы определяют решения модели, и мы уверены, что понимаем, как наша система машинного обучения принимает научно обоснованные решения.

Книги для программистов: https://t.me/free_books_for_it

Недостаточное обучение

Если вам станут известны статистические показатели данных, вы можете задаться вопросом, на каком объеме данных основана эта статистика и были ли эти данные достаточно высокого качества, чтобы им можно было доверять. Что, если кто-то скажет вам, что у него есть миллионы, миллиарды или даже триллионы статистических показателей, которые можно рассмотреть? Ему понадобится много данных, чтобы доказать, что вся эта статистика имеет смысл. Точно так же каждый из миллионов параметров модели ML изучается на основе данных. Большие модели ML нуждаются в большом количестве данных для полноценного обучения, чтобы сделать миллионы, миллиарды или триллионы изученных параметров статистически значимыми. Недостаточное обучение возникает, когда сложному алгоритму машинного обучения не хватает обучающих данных, ограничений или другой входной информации. В результате он изучает лишь несколько обобщаемых концепций из обучающих данных, но недостаточно их конкретизирует, чтобы успешно применить при развертывании. Недостаточное обучение можно диагностировать по одинаково плохим прогнозам как на обучающих, так и на проверочных данных. Еще один признак недостаточного обучения — когда остатки модели (разница между наблюдаемым и прогнозируемым значением) имеют значительно более упорядоченную структуру, чем случайный шум. Это говорит о том, что в данных существуют значимые закономерности, которые ваша модель не обнаруживает, и это еще одна причина, по которой необходимо проверять остатки при отладке модели. Мы можем смягчить недостаточное обучение, увеличив сложность наших моделей или, что предпочтительнее, предоставив больше обучающих данных.

Мы также можем предоставить больше входной информации другими способами, такими как новые признаки, байесовские априорные подходы, применяемые к распределениям параметров нашей модели, или различные типы архитектурных или оптимизационных ограничений.

Неполная идентификация

Сорок исследователей недавно опубликовали статью «Неполная идентификация снижает доверие к современному машинному обучению» (*https://oreil.ly/Da9g0*). Данная статья дает название проблеме, существовавшей десятилетиями, — *неполной идентификации* (underspecification). Неполная идентификация возникает как следствие идеи о множественности хороших моделей, которую иногда также называют эффектом Расёмона. Для любого набора данных существует множество достаточно точных моделей машинного обучения. Сколько? Обычно намного больше, чем могут найти технические специалисты. Хотя мы используем проверочные данные для выбора хорошей модели из множества моделей, опробованных во время обучения, выбор модели на основе только этих данных не может гарантировать, что мы выбрали лучшую — или даже работоспособную — модель для развертывания. Предположим, что для некоторого набора данных существует миллион хороших моделей машинного обучения, основанных на обучающих данных, и большое количество потенциальных моделей-гипотез. Выбор путем тестирования на проверочных данных может сократить это количество моделей до ста. Даже в таком про-

Книги для программистов: https://t.me/free_books_for_it

стом сценарии у нас есть только один шанс из ста выбрать правильную модель для развертывания. Как же увеличить шансы на успех? Один из способов — внедрение знаний предметной области в модели ML. Сочетая выбор модели, основанный на тестировании, с ограничениями, учитывающими специфику предметной области, мы увеличиваем шансы выбрать жизнеспособную модель для текущей задачи.

К счастью, тестирование на неполную идентификацию может быть довольно простым. Одним из основных симптомов неполной идентификации является зависимость показателей модели от вычислительных гиперпараметров, не связанных со структурой предметной области, данных или модели. Если качество работы модели варьируется из-за случайных начальных значений, количества потоков или графических процессоров, а также других вычислительных настроек, то наша модель, вероятно, недостаточно определена. Другой тест на неполную идентификацию показан на рис. 3.1.

На этом рисунке представлены несколько показателей ошибок и точности в важных сегментах примера обучающих данных и модели. Здесь заметный сдвиг показателей для сегментов, определенных более высокими значениями важного признака `PAY_0`, указывает на потенциальную проблему неполной идентификации, вероятно, из-за разреженности данных в этой области обучающих данных. (Показатели сегментов, определяемых признаком `SEX`, более равномерно сбалансированы, что является хорошим знаком с точки зрения тестирования предвзятости, но, конечно, это не единственный тест, который следует использовать для выявления проблем предвзятости.) Исправление неполной идентификации, как правило, подразумевает применение реальных знаний предметной области к алгоритмам машинного обучения. К механизмам, основанным на предметной области, относятся графовые связи, монотонные ограничения, ограничения взаимодействия, бета-ограничения или другие архитектурные ограничения.



Почти все ошибки, обсуждаемые в этом разделе, главе и книге, могут влиять на одни сегменты данных больше, чем на другие. Для достижения оптимального качества модели важно проверять слабые места (качество работы), переобучение и недостаточное обучение, нестабильность, смещение распределения и другие проблемы в различных типах сегментов обучающих, валидационных, тестовых или отложенных данных.

Каждая из ошибок машинного обучения, обсуждаемых в этом подразделе, имеет реальные последствия для безопасности и надежности. Объединяющей темой всех этих ошибок является то, что в итоге системы работают иначе, чем ожидалось, при развертывании in vivo и с течением времени. Непредсказуемая реакция модели на факторы реального мира приводит к неожиданным сбоям и инцидентам с искусственным интеллектом. Знания о потенциальных ошибках и способах их обнаружения, обсуждаемые в этой книге, могут оказаться бесценными для предотвращения реальных инцидентов. Теперь нам известно, какие ошибки мы ищем с точки зрения программного обеспечения, традиционных методов оценки и математики машинного обучения. Далее рассмотрим, как найти эти ошибки с помощью анализа остатков, анализа чувствительности, эталонных моделей и других подходов к тестированию и мониторингу.

Книги для программистов: https://t.me/free_books_for_it

Метрики для PAY_0

PAY_0	Доминирование	Достоверность	Доля истинно положительных прогнозов	Точность	Идентификация	Отрицательная прогностическая ценность	Доля ложноположительных прогнозов	Уровень ложного обнаружения	Доля ложноотрицательных прогнозов	Доля ложных пропусков
-2	0.124	0.864	0.099	0.333	0.972	0.884	0.028	0.667	0.901	0.116
-1	0.168	0.816	0.206	0.406	0.939	0.854	0.061	0.594	0.794	0.146
0	0.121	0.867	0.107	0.341	0.972	0.888	0.028	0.659	0.893	0.112
1	0.325	0.491	0.903	0.381	0.292	0.862	0.708	0.619	0.097	0.138
2	0.709	0.709	1	0.709	0	0.5	1	0.291	0	0.5
3	0.748	0.748	1	0.748	0	0.5	1	0.252	0	0.5
4	0.571	0.571	1	0.571	0	0.5	1	0.429	0	0.5
5	0.444	0.444	1	0.444	0	0.5	1	0.556	0	0.5
6	0.25	0.25	1	0.25	0	0.5	1	0.75	0	0.5
7	0.5	0.5	1	0.5	0	0.5	1	0.5	0	0.5
8	0.75	0.75	1	0.75	0	0.5	1	0.25	0	0.5

Метрики для SEX

SEX	Доминирование	Достоверность	Доля истинно положительных прогнозов	Точность	Идентификация	Отрицательная прогностическая ценность	Доля ложноположительных прогнозов	Уровень ложного обнаружения	Доля ложноотрицательных прогнозов	Доля ложных пропусков
Male	0.235	0.782	0.626	0.531	0.83	0.879	0.17	0.469	0.374	0.121
Female	0.209	0.797	0.552	0.514	0.862	0.879	0.138	0.486	0.448	0.121

Рис. 3.1. Анализ точности и ошибок в ключевых сегментах — важный метод отладки для обнаружения предвзятости, неполной идентификации и других серьезных ошибок машинного обучения

Книги для программистов: https://t.me/free_books_for_it

Анализ остатков

Анализ остатков (residual analysis) — это еще один тип традиционной оценки модели, который хорошо себя зарекомендовал при анализе моделей и систем машинного обучения. На самом базовом уровне анализ остатков означает обучение на ошибках. Это очень важный навык в жизни и полезный инструмент в области машинного обучения. Более того, анализ остатков является проверенным методом диагностики моделей. В этом подразделе мы рассмотрим пример и три метода анализа остатков, применяемых в ML.



Мы используем термин *остаток* для обозначения соответствующего измерения ошибки, что в некоторой степени является синонимом измерения потерь модели. Нужно понимать, что мы не используем его в строго определенном смысле $\hat{y}_i - y_i$. Мы используем этот термин, чтобы подчеркнуть важность и долгую историю анализа остатков в диагностике моделей регрессии и подчеркнуть его принципиальное отсутствие в обычных рабочих процессах ML.

В следующих разделах читатели могут увидеть демографические характеристики набора данных, такие как `SEX`, которые используются для тестирования на предвзятость. По большей части в этой главе пример задачи кредитования рассматривается как общее упражнение по прогнозному моделированию и не касается правил справедливого кредитования. В главах 4 и 10 подробно обсуждаются вопросы борьбы с предвзятостью в сфере ML, а также рассматриваются некоторые юридические и нормативные проблемы.

Анализ и визуализация остатков

Построение графиков общих и сегментированных остатков и их исследование на предмет выявления характерных закономерностей, присущих различным типам проблем, — это давний метод диагностики моделей. Проявив немного творческого подхода и усилий, анализ остатков можно с большой пользой применить к алгоритмам машинного обучения. Простое построение графика остатков для всего набора данных может быть полезным, особенно для выявления строк с выбросами, вызывающими очень большие числовые ошибки, или для анализа общих тенденций ошибок. Однако разбивка остаточных значений и графиков по признакам и уровням, вероятно, будет более информативной. Даже если у вас много признаков или признаков со многими категориальными уровнями, еще не все потеряно. Начните с наиболее важных признаков и их наиболее распространенных уровней. Ищите сильные закономерности в остатках, которые нарушают предположения вашей модели. Большинство типов остатков должны быть распределены случайным образом. Это говорит о том, что модель усвоила всю важную информацию из данных, кроме неуменьшаемого шума. Если мы обнаруживаем сильные закономерности или другие аномалии в остатках, которые были разбиты по признакам и уровням, то сначала проверяем, не являются ли источником ошибок сами данные. Если нет, можно применить методы XAI для выявления проблем в модели. Анализ остатков считается стандартным способом проверки важных моделей линейной регрессии. Модели машинного обучения, возможно, более рискованны и более склонны к сбоям, поэтому они еще больше нуждаются в анализе остатков.

Книги для программистов: https://t.me/free_books_for_it

Моделирование остатков

Моделирование остатков (modeling residuals) с помощью интерпретируемых моделей — еще один отличный способ узнать больше об ошибках, которые может допускать наша система машинного обучения. На рис. 3.2 показано, как мы обучили одно неглубокое дерево решений на остатках более сложной модели, связанной с клиентами, пропустившими платеж по кредиту.

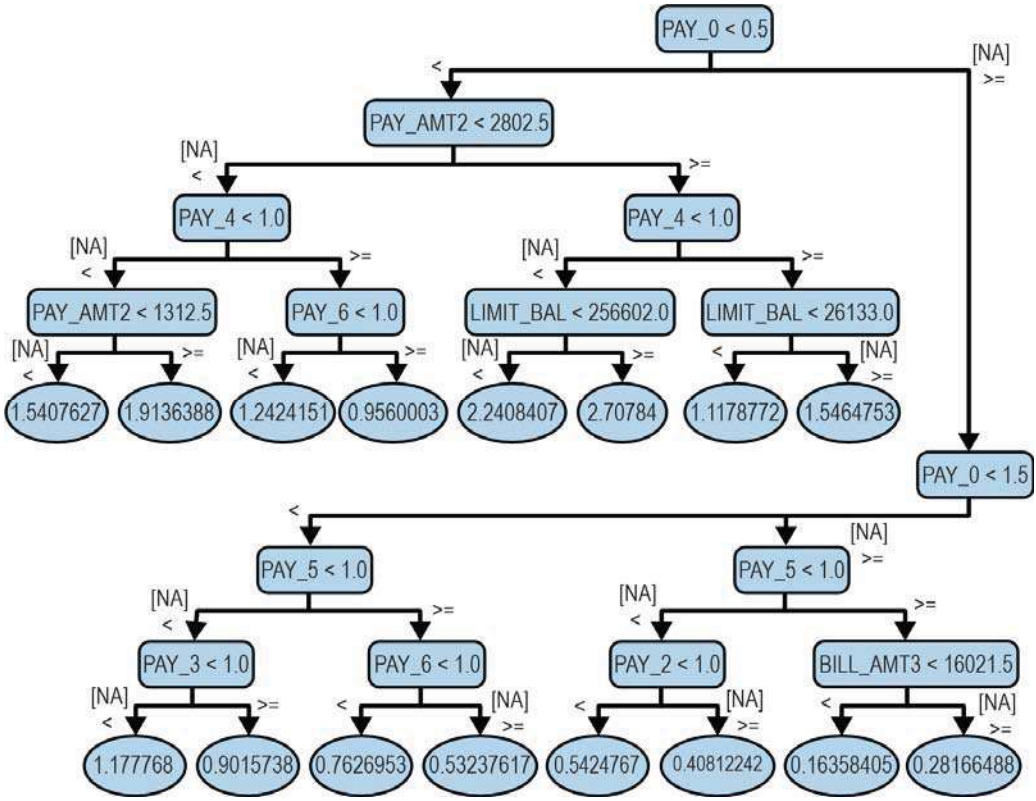


Рис. 3.2. Интерпретируемая модель дерева решений для клиентов, пропустивших платежи по кредитной карте

Это дерево решений кодирует правила, которые описывают, почему более сложная модель неверна. Например, мы видим, что модель генерирует наибольшие числовые остатки, когда кто-то пропускает платеж по кредитной карте, но выглядит как отличный клиент. Когда чей-либо последний статус погашения (`PAY_0`) меньше 0,5, его вторая самая последняя сумма платежа (`PAY_AMT2`) больше или равна 2802,50, его четвертый самый последний статус погашения (`PAY_4`) меньше 1, а его кредитный лимит больше или равен 256 602, мы видим, что логарифмический остаток потерь в среднем равен 2,71. Это большой процент ошибок, который снижает общее качество прогнозов и может стать причиной предвзятости, если мы делаем слишком много ложноотрицательных предположений о некоторых демографических группах.

Книги для программистов: https://t.me/free_books_for_it

Еще одно интригующее применение дерева — создание *утверждений модели*. Эти утверждения представляют собой бизнес-правила, которые применяются в режиме реального времени и касаются прогнозов модели, которые можно использовать для индикации случаев принятия неправильного решения по мере их возникновения. В некоторых случаях утверждения могут просто предупреждать операторов модели о том, что, вероятно, принимается неправильное решение. Утверждения модели также могут включать в себя корректирующие действия, например перенаправление этой строки данных в более специализированную модель или к специалистам по нестандартным случаям.

Локальный вклад в остатки

Построение графиков и моделирование остатков — это старые методы, хорошо известные опытным практикам. Последние достижения в этой области позволяют точно рассчитать вклад значений Шепли в ошибки модели. Это означает, что теперь мы можем взять любой признак или строку любого набора данных и точно сказать, какие признаки определяют прогнозы модели, а какие вызывают ее ошибки. Что на самом деле означает это достижение для машинного обучения, еще предстоит выяснить, но потенциальные возможности, безусловно, интригуют. Одним из очевидных применений этого нового метода значений Шепли является возможность сравнить значимость признаков для прогнозов с их значимостью для остатков, как показано на рис. 3.3.

На рис. 3.3 значимость признаков для прогнозов показана вверху, а значимость признаков для ошибок модели, рассчитанных с помощью логарифмических потерь, показана внизу. Мы видим, что `PAY_0` доминирует как в прогнозах, так и в ошибках, подтверждая, что эта модель в целом слишком зависит от `PAY_0`. Мы также видим, что `PAY_2` и `PAY_3` имеют более высокий рейтинг по вкладу в ошибку, чем по вкладу в прогнозы. Учитывая это, возможно, имеет смысл поэкспериментировать с удалением, заменой или нормированием этих признаков. Имейте в виду, что рис. 3.3 представляет собой агрегирование вкладов Шепли в логарифмические потери по всему набору проверочных данных. Однако эти величины рассчитываются поэлементно и построчно. Мы также могли бы применить этот анализ к сегментам или демографическим группам наших данных, открыв интересные возможности для обнаружения и исправления неустойчивых признаков для различных подгрупп населения в рамках модели.



Графики значимости признаков, которые выглядят так, как показано на рис. 3.3, где один признак значительно перевешивает все остальные, не сулят ничего хорошего для надежности и безопасности in vivo. Если распределение этого единственного важного признака изменится, надежность прогнозов нашей модели пострадает. Если хакеры найдут способ изменить значения этого признака, они смогут легко манипулировать нашими прогнозами. Когда один признак доминирует в модели, нам, скорее всего, понадобится бизнес-правило, относящееся к этому признаку, а не модель машинного обучения.

На этом мы заканчиваем наш краткий обзор анализа остатков в ML. Конечно, есть и другие способы изучения ошибок моделей ML. Если читатели предпочитают дру-

Книги для программистов: https://t.me/free_books_for_it

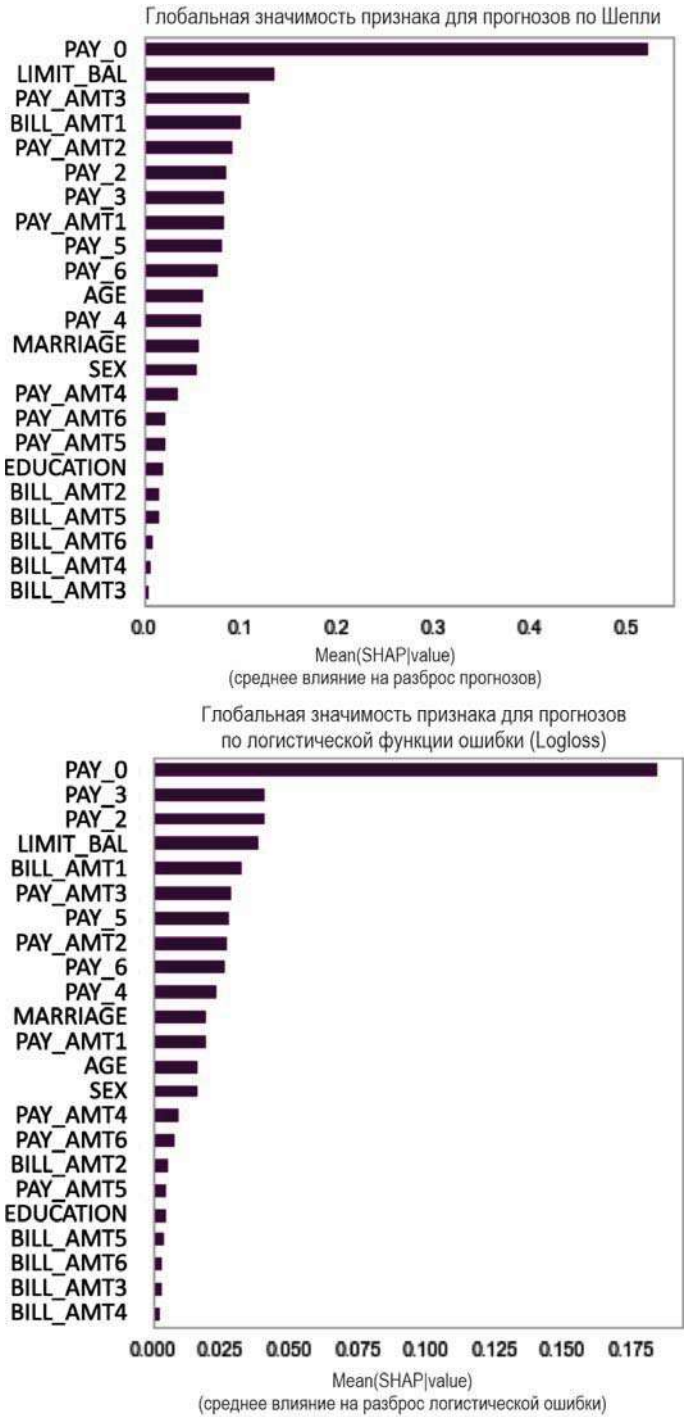


Рис. 3.3. Сравнение важности признаков Шепли для прогнозов и ошибок модели

Книги для программистов: https://t.me/free_books_for_it

гой путь, то это их право! В любом случае важно провести своего рода анализ остатков для всех важных систем ML. Наряду с анализом чувствительности, который будет обсуждаться в следующем подразделе, анализ остатков является важным инструментом отладки моделей машинного обучения.

Анализ чувствительности

В отличие от линейных моделей, без явного тестирования моделей машинного обучения очень сложно понять, как они работают с новыми данными. В этом заключается простая и мощная идея анализа чувствительности. Найдите или смоделируйте данные для интересных сценариев, а затем посмотрите, как ваша модель работает с этими данными. Вы не узнаете, как ваша система ML будет работать в этих сценариях, если не проведете базовый анализ чувствительности. Конечно, существуют структурированные и более эффективные варианты анализа чувствительности, например, в библиотеке InterpretML (*https://oreil.ly/zdzxX*) от Microsoft Research. Еще один отличный вариант анализа чувствительности и хорошая отправная точка для начала использования более продвинутых методов отладки моделей — это случайные атаки, обсуждавшиеся в разделе «Тестирование программного обеспечения». В число многих других стандартизированных способов анализа чувствительности входят также:

Нагрузочное тестирование

Нагрузочное тестирование включает в себя моделирование данных, которые представляют собой реалистичные неблагоприятные сценарии, такие как рецессии или пандемии, и проверку того, что ваши модели машинного обучения и любые последующие бизнес-процессы выдержат воздействие неблагоприятной ситуации.

Визуализации

Визуализации, такие как графики накопленных локальных эффектов, индивидуальное условное ожидание и кривые частичной зависимости, представляют собой широко известные, хорошо структурированные способы наблюдения за качеством работы алгоритмов ML при различных реальных или смоделированных значениях входных признаков. Эти графики также помогают выявить области разреженности данных, которые могут служить причиной провалов модели.

Поиск состязательных примеров

Состязательные примеры (adversarial example) — это строки данных, которые вызывают неожиданную реакцию моделей ML. Для создания состязательных примеров в неструктурированных данных можно применить подходы глубокого обучения, а ICE и генетические алгоритмы пригодятся для создания состязательных примеров в структурированных данных. Состязательные примеры (и их поиск) — отличный способ найти локальные области нестабильности в функциях отклика ML или границах принятия решений, которые могут вызвать инциденты после развертывания системы. Как читатели могут видеть на рис. 3.4, поиск состязательных примеров — отличный способ проверить модель.

Книги для программистов: https://t.me/free_books_for_it

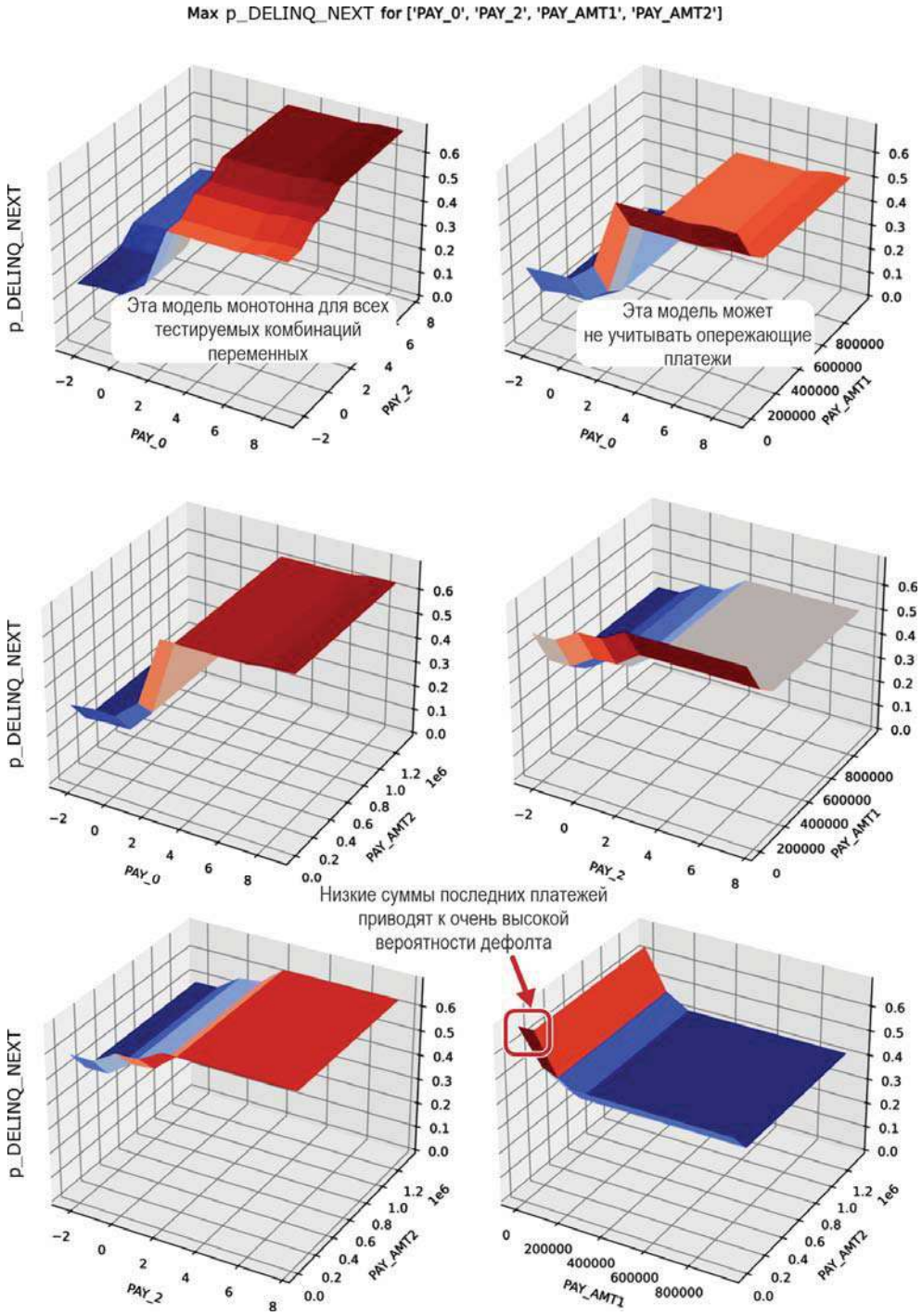


Рис. 3.4. Результаты поиска состязательных примеров, которые показывают интересное поведение модели (цветной рисунок доступен в электронном архиве по ссылке **https://files.alistbooks.kz/3060_Hall_ML.zip**)

Книги для программистов: https://t.me/free_books_for_it

Конформные подходы

Конформные подходы, которые пытаются вычислить эмпирические границы прогнозов модели, помогут вам понять надежность модели путем нахождения верхних и нижних пределов ожидаемых результатов.

Тесты на возмущения

Случайное изменение валидационных, тестовых или отложенных данных для моделирования различных типов шума и дрейфа, а затем повторное измерение показателей модели машинного обучения — еще один способ установить общие границы надежности модели. С помощью такого рода тестирования на возмущения мы можем понять и задокументировать величину шума или смещения, которые, как мы знаем, разрушат нашу модель. Следует иметь в виду, что при тестировании на возмущения качество работы модели на неэффективных строках данных часто снижается быстрее, чем на средних строках. Внимательно следите за неэффективными строками, чтобы понять, снижают ли они качество всей модели и когда это происходит.

Рис. 3.4 был создан путем построения кривой ICE, которая демонстрировала большие колебания прогнозов. Мы взяли ряд данных, ответственных за эту кривую ICE, в качестве начального значения, внесли возмущения в четыре наиболее значимых признака, проделали это несколько тысяч раз, генерируя соответствующие прогнозы, и получили многочисленные графики, как на рис. 3.4. Первый вывод поиска состязательных примеров заключается в том, что этот эвристический метод, основанный на кривых ICE, позволяет нам генерировать состязательные примеры, которые могут вызвать практически любой ответ, который мы хотим от модели. Мы нашли строки, которые надежно дают очень низкие и очень высокие прогнозы, а также все, что между ними. Если бы эта модель была доступна через API прогнозирования, мы могли бы играть на ней, как на скрипке, заставляя выдавать нужные результаты.

В процессе поиска всех этих состязательных примеров мы также узнали кое-что о нашей модели. Во-первых, она, вероятно, монотонна в целом и определенно монотонна для всех строк, которые мы смоделировали в состязательном примере поиска. Во-вторых, эта модель выдает прогнозы по умолчанию для людей, которые делают чрезвычайно высокие платежи. Даже если чей-то последний платеж составил один миллион долларов и превысил кредитный лимит, эта модель выдаст прогноз о дефолте, как только этот человек просрочит свои платежи на два месяца. Это может создать проблемы с досрочным погашением кредита. Действительно ли мы хотим вынести решение о неисполнении обязательств или просрочке платежа для человека, который внес досрочные платежи в миллионы долларов, но теперь опаздывает на два месяца по последнему платежу? Возможно, но это, скорее всего, не то решение, которое должно приниматься быстро или автоматически, как в этой модели. В-третьих, похоже, мы нашли путь для настоящей состязательной атаки. Низкие суммы последних погашений приводят к удивительно резкому увеличению вероятности дефолта. Если хакер захочет получить от этой модели прогнозы с высокой вероятностью дефолта, он может установить низкие значения для `PAY_AMT1` и `PAY_AMT2`.

Книги для программистов: https://t.me/free_books_for_it

Как и в случае анализа остатков, читатели могут выбрать другие методы анализа чувствительности, и это здорово. Просто возьмите за правило применять ту или иную форму реалистичного симуляционного тестирования к своим моделям ML. Практический пример, рассмотренный в этой главе, демонстрирует наихудший результат, возникающий вследствие неспособности разработчика провести реалистичное симуляционное тестирование перед развертыванием системы ML. На этом мы заканчиваем наше краткое обсуждение анализа чувствительности. Желающим продолжить обучение мы рекомендуем главу 19 книги Кевина Мерфи «Вероятностное машинное обучение. Дополнительные темы»[3]. Далее мы обсудим различные контексты применения эталонных моделей — еще один проверенный временем и разумный подход к отладке моделей.

Эталонные модели

Эталонные модели неоднократно упоминались в этой главе. Они являются очень важным инструментом оценки безопасности и надежности, который можно использовать на протяжении всего жизненного цикла машинного обучения. Далее в этом разделе мы обсудим эталонные модели в контексте отладки производственных моделей, а также кратко рассмотрим другие важные варианты их применения.



Если возможно, сравните показатели модели ML с линейной моделью или эталонной GLM. Если линейная модель превосходит модель ML, используйте линейную модель.

Первый способ использовать эталонную модель для отладки — сравнить результаты тестирования эталона и рассматриваемой системы машинного обучения. Если система машинного обучения не превзойдет простой эталонный тест (а это случается достаточно часто), то разработчику придется вернуться к чертежной доске. Если система проходит этот первичный базовый тест, эталонные модели затем выступают в роли инструмента сравнения для проверки механизмов и поиска ошибок в системе машинного обучения. Например, специалисты по обработке и анализу данных могут задать вопрос: «Для каких прогнозов эталонная модель срабатывает правильно, а моя система ML ошибается?» Поскольку предполагается, что тест эталонной модели хорошо понятен, должно быть ясно, почему он пройден правильно, и это понимание также должно дать некоторые подсказки относительно того, что система ML делает не так. Сравнение с эталонной моделью также применяется для целей воспроизводимости и мониторинга модели следующим образом:

Бенчмарк воспроизводимости

Прежде чем вносить изменения в сложную систему машинного обучения, необходимо иметь воспроизводимый эталон, на основе которого можно будет измерять прирост или потери показателей. Воспроизводимая эталонная модель является идеальным инструментом для таких задач измерения. Еще лучше, если эту

[3] Русский перевод издан в 2024 г.

Книги для программистов: https://t.me/free_books_for_it

модель можно будет встроить в процессы CI/CD, которые позволят автоматически тестировать воспроизводимость и сравнивать изменения новой системы с известными контрольными показателями.

Отладочный бенчмарк

Сравнение сложных механизмов и прогнозов модели ML с надежными, хорошо понятными механизмами и прогнозами эталонной модели — эффективный способ обнаружить ошибки ML.

Мониторинговые бенчмарки

Сравнение прогнозов в реальном времени между проверенной эталонной моделью и сложной системой ML — это способ выявить серьезные ошибки машинного обучения в режиме реального времени. Если надежная эталонная модель и сложная система ML дают заметно разные прогнозы для одного и того же экземпляра новых данных, это может быть признаком взлома машинного обучения, дрейфа данных или даже предвзятости и алгоритмической дискриминации. В таких случаях вместо прогнозов системы ML пользователям могут быть выданы эталонные прогнозы, или прогнозы могут быть отложены до тех пор, пока аналитики не проверят их правильность.



Помните, что методы отладки сами по себе являются потенциально ошибочными подходами, основанными на статистике или машинном обучении, и их, возможно, придется отлаживать отдельно.

Если удачно установить контрольные показатели, возможно, даже получится использовать одну и ту же модель для всех трех задач. Перед началом работы можно запустить тест, чтобы установить исходный уровень для дальнейшего повышения показателей, и эту же модель можно использовать в сравнениях для отладки и мониторинга. Когда новая версия системы воспроизводимо превосходит старую версию, модель машинного обучения, лежащая в основе новой системы, может стать следующим эталоном. Если ваша организация сможет наладить такой рабочий процесс, вам останется лишь регулярно проводить сравнительный анализ и повторять процессы повышения безопасности и надежности машинного обучения.

Исправление ошибок

Последний шаг в отладке — исправление ошибок. В предыдущих подразделах были описаны стратегии тестирования, ошибки, на которые следует обратить внимание, и несколько конкретных исправлений. Здесь описываются общие подходы к устранению ошибок ML и обсуждается, как их можно применить в примере сценария отладки. При отладке модели ML обычно применяются следующие ключевые стратегии:

Обнаружение аномалий

Странные входные и выходные данные обычно являются плохой новостью для систем ML. Это может свидетельствовать о наличии проблем с безопасностью в реальном времени, предвзятости или проблем с защищенностью и надежно-

Книги для программистов: https://t.me/free_books_for_it

стью. Отслеживайте очереди данных системы ML и прогнозируйте аномалии, фиксируйте возникновение аномалий и при необходимости оповещайте заинтересованные стороны об их присутствии.



Для обнаружения аномалий в незнакомых данных можно использовать ряд методов: основанных на правилах, статистических данных и на основе машинного обучения. К ним относятся ограничения целостности данных, доверительные пределы, контрольные пределы, автокодировщики и изолирующие леса.

Экспериментальная разработка и дополнение данных

Сбор более качественных данных часто является решением всех проблем ML. Более того, сбор данных не должен осуществляться методом проб и ошибок, а ученым, работающим с данными, не нужно полагаться на побочные данные, полученные в результате других организационных процессов. Специалисты по работе с данными десятилетиями полагаются на развитую науку о планировании экспериментов, чтобы собрать достаточный объем нужных данных для обучения моделей. Высокомерие, связанное с иллюзией всемогущества «больших» данных, и чрезмерно сжатые сроки развертывания являются наиболее распространенными причинами, по которым ученые, работающие с данными, не практикуют планирование экспериментов (DOE). Но это не повод для вас игнорировать DOE.

Утверждения модели

Формальные утверждения модели (model assertions) — это бизнес-правила, применяемые к прогнозам модели ML, которые исправляют недостатки в изученной логике модели. Использование бизнес-правил для улучшения прогнозных моделей — это проверенный временем метод исправления ошибок, который, вероятно, будет с нами еще десятилетия. Если существует простое и логичное правило, которое можно применить для исправления возможного сбоя модели ML, смело его реализуйте. Лучшие специалисты и организации в области прогнозной аналитики использовали этот трюк на протяжении десятилетий.

Редактирование модели

Поскольку модели ML фактически представляют собой программное обеспечение, их можно редактировать для исправления любых обнаруженных ошибок. Некоторые модели, такие как GA2M или *объяснимые бустинговые машины* (explainable boosting machine EBM), изначально разработаны с учетом редактирования в целях отладки модели. Редактирование других типов моделей может потребовать большей креативности. В любом случае редактирование должно быть оправдано соображениями предметной области, поскольку оно может ухудшить показатели модели на обучающих данных. Дело в том, что модели ML оптимизируются для уменьшения ошибок при обучении. Если мы отредактируем эту оптимизированную структуру, чтобы повысить показатели модели, связанные с предметной областью, мы, скорее всего, ухудшим традиционную оценку. Это нормально. Вы должны больше заботиться о безопасности, надежности и непредвзятости in vivo, чем об ошибках испытаний in silico.

Книги для программистов: https://t.me/free_books_for_it

Управление моделями и мониторинг

Модели ML и системы, в которых они размещены, представляют собой динамические объекты, которые необходимо отслеживать в той степени, в которой позволяют ресурсы. Все критически важные системы ML должны быть хорошо документированы, инвентаризированы и проверены на предмет безопасности, предвзятости, а также проблем с защищенностью и надежностью в режиме реального времени. Когда что-то начинает идти не так, об этом нужно немедленно уведомить все заинтересованные стороны. Мониторинг модели более подробно описан в следующем разделе «Развертывание».

Ограничения монотонности и взаимодействия

Многие ошибки ML возникают из-за того, что модели ML обладают слишком большой гибкостью и теряют связь с реальностью из-за обучения на предвзятых и неточных данных. Ограничение моделей реальными знаниями — это распространенный способ устранения нескольких типов ошибок ML. Ограничения монотонности и взаимодействия в популярных инструментах, таких как XGBoost, помогают специалистам по машинному обучению реализовать логические предположения предметной области в сложных моделях машинного обучения.

Внесение шума и сильная регуляризация

Многие алгоритмы ML имеют возможности регуляризации. Однако если модель переоценивает определенный признак, возможно, потребуется применить более сильную или внешнюю регуляризацию. Для непосредственного ограничения количества правил или параметров в модели можно применить регуляризацию L0, а при необходимости можно вручную ввести шум для искажения сигналов от определенных признаков, чтобы уменьшить значение тех из них, которые имеют чрезмерную важность в моделях ML.

Научный подход

Склонность к подтверждению своей точки зрения (confirmation bias) среди специалистов по работе с данными, инженеров ML, их менеджеров и деловых партнеров часто побуждает их сговориться между собой, чтобы выпустить в производство недоделанные демо-версии под видом продуктов, основанных на предположениях и ограничениях данных испытаний in silico. Если вы сможете следовать научному подходу, зафиксировав гипотезу о реальных результатах и объективно проверив эту гипотезу с помощью запланированного эксперимента, у вас будет гораздо больше шансов на успех in vivo. Дополнительная информация об использовании научного подхода в машинном обучении приводится в главе 12.



Вообще говоря, ML по-прежнему является скорее эмпирической наукой, чем инженерной дисциплиной. Мы еще не до конца понимаем, когда ML работает хорошо и почему оно может потерпеть неудачу, особенно при использовании in vivo. Это означает, что мы должны применять научный подход и избегать таких проблем, как склонность к самоподтверждению, чтобы достичь хороших реальных результатов. Простое использование правильного программного обеспечения и платформ, а также следование лучшим инженерным практикам не означает, что наши модели будут работать хорошо.

Книги для программистов: https://t.me/free_books_for_it

Более подробная информация об отладке модели, пример данных и модели представлены в разделе «Ресурсы» этой главы. На данный момент вы узнали довольно много об отладке модели, и пришло время обратить внимание на безопасность и надежность развернутых систем машинного обучения.

Развертывание

Допустим, ошибки найдены и исправлены; пришло время развернуть систему ML в производстве для принятия реальных решений. Такие системы гораздо более динамичны, чем большинство традиционных программных систем. Даже если операторы не изменят какой-либо код или настройки системы, результаты все равно могут измениться. После развертывания системы ML должны быть проверены на безопасность и надежность внутри конкретной предметной области, при этом их работу нужно непрерывно отслеживать, а операторы должны иметь возможность быстро их отключить. В этом разделе будет рассказано, как повысить безопасность и прочие показатели после развертывания системы ML: безопасность в предметной области, мониторинг модели и аварийные выключатели.

Безопасность в предметной области

Безопасность в предметной области означает безопасность модели в реальном мире. Это сильно отличается от стандартной оценки модели или даже от расширенной отладки модели. Как специалисты-практики могут работать над достижением реальных целей безопасности? A/B-тестирование и методики отбора лучших кандидатов позволяют проводить определенное количество тестов в операционных средах реального времени. Средства контроля процессов, такие как перечисление прогнозируемых инцидентов, внедрение средств контроля для устранения потенциальных инцидентов и тестирование этих средств управления в реалистичных или стрессовых условиях, также важны для стабильной работы in vivo. Чтобы компенсировать инциденты, которые невозможно предсказать, к результатам системы ML применяют хаос-тестирование, случайные атаки и ручные ограничения прогнозирования. Далее мы разделим инциденты на те, которые мы можем предвидеть, и те, которые не можем, и рассмотрим несколько деталей для обоих случаев:

Предсказуемые инциденты в реальном мире

A/B-тестирование и методы «чемпион-претендент», при которых модели соревнуются друг с другом на потоках данных в реальном времени или в других реалистичных условиях, являются первым шагом на пути к надежному внутриотраслевому тестированию. Помимо этих в некоторой степени стандартных практик, ресурсы следует тратить на экспертов в предметной области и продумывание возможных инцидентов. Например, распространенными причинами неудач в кредитовании являются предвзятость и алгоритмическая дискриминация, отсутствие прозрачности и низкие показатели во время рецессий. Что касается других приложений, например автономных транспортных средств, сущест-

Книги для программистов: https://t.me/free_books_for_it

вует множество способов случайно или намеренно причинить вред. После регистрации потенциальных инцидентов можно принять меры безопасности для наиболее вероятных или наиболее серьезных случаев. В области кредитования модели проверяются на предмет предвзятости, потребителям предоставляются разъяснения посредством уведомлений о неблагоприятных действиях, а модели отслеживаются, чтобы быстро выявить снижение производительности. В области автономных транспортных средств нам еще предстоит многому научиться, как покажет рассмотренный далее пример гибели пользователя беспилотного автомобиля. Независимо от применения, средства управления безопасностью тоже необходимо протестировать; эти тесты должны быть реалистичными и проводиться в сотрудничестве с экспертами в предметной области. Когда дело доходит до безопасности человека, моделирования, проводимого специалистами по обработке данных, недостаточно. Средства контроля безопасности должны быть проверены и усилены in vivo и в координации с людьми, которые имеют глубокое понимание безопасности в области применения.

Непредвиденные инциденты в реальном мире

Взаимодействие между системами ML и их средой может быть сложным и неожиданным. Для систем ML с высокими рисками лучше сразу признать возможность непредвиденных инцидентов. Мы можем попытаться выявить некоторые из этих потенциальных инцидентов до того, как они произойдут, с помощью хаос-тестирования и случайных атак. Важные системы ML следует тестировать в странных и хаотичных сценариях использования и подвергать воздействию больших объемов случайных входных данных. Хотя эти тесты требуют много времени и ресурсов, они являются одним из немногих доступных инструментов для проверки так называемых неизвестных неизвестных. Поскольку ни один режим тестирования не может выявить все проблемы, было бы очень полезно применять к системам пределы прогнозирования, основанные на здравом смысле. Например, кредиты на очень большую сумму или с очень низкой процентной ставкой не должны выдаваться без проверки человеком. Также нельзя позволять автономным транспортным средствам передвигаться на очень высоких скоростях без вмешательства человека. Некоторые действия на сегодняшний день просто не должны выполняться чисто автоматически, и пределы прогнозирования являются одним из способов реализации такого рода контроля.

Еще одним ключевым аспектом безопасности в реальном мире является своевременное знание возникновения проблем. Иногда сбои можно обнаружить до того, как они перерастут в опасные инциденты. Для быстрого выявления проблем необходимо отслеживать системы ML. В случае обнаружения инцидентов может потребоваться активация планов реагирования на инциденты или аварийных выключателей.

Характеристики безопасных систем машинного обучения

Ниже перечислены некоторые из наиболее важных шагов, которые вы можете предпринять, чтобы обеспечить безопасное взаимодействие системы ML с физическим миром:

Книги для программистов: https://t.me/free_books_for_it

Избегайте повторения неудачных проектов

Системы ML, которые причиняют вред людям или рабочей среде, не должны повторно внедряться, а их сбои следует изучать, чтобы улучшить безопасность будущих систем в этой области.

Имейте планы реагирования на инциденты

Операторы должны четко знать, что делать в случае возникновения инцидента безопасности.

Тестируйте модель в реалистичных условиях

Оценки на тестовых данных, моделирования и отладки специалистами по данным недостаточно для обеспечения безопасности. Системы должны быть протестированы в реалистичных условиях in vivo экспертами в области безопасности.

Имейте выключатели аварийного отключения

Системы должны быть оснащены функцией быстрого и удаленного отключения в случае обнаружения рискованных или опасных ситуаций в процессе мониторинга.

Установите вручную *пределы прогнозирования*

При необходимости операторы должны устанавливать ограничения на поведение системы.

Выполняйте мониторинг в реальном времени

Люди должны быть предупреждены, когда система переходит в рискованное или опасное состояние, а аварийные выключатели или функции резервирования должны срабатывать быстро (или автоматически).

Предусмотрите резервирование

Системы, выполняющие критически важные для безопасности или миссии действия, должны иметь наготове функциональные резервы на случай возникновения инцидентов или сообщения мониторинга, указывающего, что система перешла в рискованное или опасное состояние.

Мониторинг модели

Мы уже много раз упоминали о том, что важные системы ML необходимо отслеживать после развертывания. В этом подразделе основное внимание уделяется техническим аспектам мониторинга моделей. В нем излагаются базовые понятия устаревания модели, ее робастности и ошибок дрейфа концепта, способы обнаружения и устранения дрейфа, важность измерения нескольких ключевых показателей эффективности (key performance indicators, KPI) при мониторинге, а также кратко освещаются некоторые другие известные концепции мониторинга моделей.

Устаревание модели и дрейф концепта

Как бы мы это ни называли, данные, поступающие в систему машинного обучения, скорее всего, отойдут от данных, на которых система обучалась. Изменение распределения входных значений с течением времени иногда называют *дрейфом дан-*

Книги для программистов: https://t.me/free_books_for_it

ных (data drift). Статистические свойства того, что мы пытаемся предсказать, также могут меняться, и иногда это называют *дрейфом концепта* (concept drift). Кризис COVID-19, вероятно, является одним из лучших примеров этих явлений в истории. В разгар пандемии наблюдался очень сильный дрейф в сторону более осторожного поведения потребителей, сопровождавшийся общим изменением распределения просроченных платежей и кредитных дефолтов. Подобные изменения протекают очень болезненно и могут нанести ущерб точности системы машинного обучения. Важно отметить, что иногда мы сами создаем проблемы с дрейфом концепта, используя модели машинного обучения не по назначению.



Дрейфу подвержены как входные данные, так и прогнозы. Оба типа дрейфа можно отслеживать, и они могут быть или не быть напрямую связаны друг с другом. Когда качество прогнозов снижается без значительного отклонения входных данных, это может быть связано с дрейфом концепта при переносе модели в реальный мир.

Обнаружение и устранение дрейфа

Лучший подход к обнаружению дрейфа заключается в мониторинге статистических свойств текущих данных — как входных переменных, так и прогнозов. После внедрения механизма мониторинга статистических свойств к нему можно подключить оповещения, чтобы уведомлять заинтересованные стороны о значительном отклонении. Самым простым способом обнаружить дрейф обычно является проверка входных данных. Это связано с тем, что иногда истинные метки данных, т. е. истинные значения результатов, связанные с прогнозами системы ML, недоступны в течение длительных периодов времени. Напротив, значения входных данных доступны сразу же, когда система ML должна сгенерировать прогноз или выходные данные. Следовательно, если текущие свойства входных данных изменились по сравнению со свойствами обучающих данных, у нас, вероятно, возникла проблема. Ситуации, в которых информация о правильности прогноза недоступна сразу же, существенно затрудняют сопоставление текущего качества модели с качеством на обучающих данных. (Сравните выходные прогнозы просрочки ипотечного платежа с прогнозом отклика на онлайн-рекламу — фактическая просрочка может случиться через несколько лет, а реакция на онлайн-рекламу видна сразу.) Основная идея мониторинга прогнозов состоит в том, чтобы наблюдать за прогнозами в реальном времени и искать отклонения и аномалии, используя такие методики, как статистические тесты, контрольные пределы и правила или алгоритмы ML для выявления выбросов. А если доступны известные результаты, часто проверяйте показатели модели на предмет ухудшения и быстро принимайте меры по устойчивому уменьшению предвзятости.

Существуют известные стратегии борьбы с неизбежным дрейфом и устареванием модели, в том числе:

- обновление системы ML с помощью дополненных обучающих данных, содержащих некоторое количество новых данных;
- частое обновление или повторное обучение системы ML;
- обновление или переобучение системы ML при обнаружении отклонений.

Книги для программистов: https://t.me/free_books_for_it

Следует отметить, что любое повторное обучение моделей ML в производстве должно сопровождаться применением методов снижения рисков, упомянутых в этой главе и других частях книги, точно так же, как они применяются при начальном обучении системы ML.

Мониторинг нескольких ключевых показателей эффективности

Большинство методик мониторинга моделей сосредоточено на точности модели как на основном ключевом показателе эффективности. Тем не менее также можно и, вероятно, нужно отслеживать предвзятость, уязвимости безопасности и ущерб конфиденциальности. Тот же тест на предвзятость, который проводился во время обучения, можно применить, когда станут доступны новые результаты модели в производстве. Для обнаружения вредоносных действий, которые могут поставить под угрозу безопасность или конфиденциальность системы, существует множество альтернативных стратегий, обсуждаемых в главах 5 и 11. Возможно, наиболее важным KPI, который необходимо измерить, если это вообще возможно, является фактическое влияние системы ML на организацию и операционную среду. Будь то экономия, генерирование прибыли или спасение жизней, сопоставление ожидаемого результата и фактической ценности системы ML может привести к важным организационным выводам. Присвойте денежные или другие значения ячейкам матрицы несоответствий в задачах классификации и остаточным блокам в задачах регрессии в качестве первого шага к расчету фактической ценности для бизнеса. В главе 8 рассмотрен базовый пример анализа ценности для бизнеса.

Значения вне диапазона

Никакой набор обучающих данных не сможет охватить все данные, с которыми система ML может столкнуться после развертывания. Большинство алгоритмов и функций прогнозирования ML плохо обрабатывают данные, выходящие за пределы допустимого диапазона. В таких случаях они могут просто выдать средний прогноз или сбой, причем без уведомления прикладного программного обеспечения далее по конвейеру или системных операторов. Операторы систем ML должны принять специальные меры для обработки неординарных данных, таких как числовые значения большой величины, редкие категориальные значения или отсутствующие значения, которые не были обнаружены во время обучения. Это необходимо для того, чтобы системы ML работали нормально и предупреждали пользователей, когда те пытаются ввести данные за пределами допустимого диапазона.

Обнаружение аномалий и эталонные модели

Мы завершим рассмотрение вопросов мониторинга кратким обсуждением технических аспектов обнаружения аномалий и эталонных моделей. Эти темы рассматривались в других разделах текущей главы и кратко упоминаются здесь в контексте мониторинга:

Обнаружение аномалий

Странные входные или выходные значения в системе ML могут указывать на проблемы со стабильностью или на уязвимости безопасности и конфиденциаль-

Книги для программистов: https://t.me/free_books_for_it

ности. Для мониторинга аномального поведения как на входах, так и на выходе, а также во всей системе ML применяют статистику, машинное обучение и бизнес-правила. Регистрируйте любые обнаруженные аномалии, сообщайте о них заинтересованным сторонам и будьте готовы к принятию более решительных мер в случае необходимости.

Эталонные модели

Сравнение более простых эталонных моделей и прогнозов системы ML в рамках мониторинга модели помогает выявить аномалии стабильности, справедливости или безопасности практически в реальном времени. Эталонная модель должна быть более стабильной, минимально дискриминирующей и хорошо защищенной от взлома. При проверке поведения системы ML на новых данных получают прогнозы от интерпретируемой эталонной модели и более сложной системы ML, а затем сравнивают эти прогнозы в режиме реального времени. Если разница между производственной системой ML и эталоном превышает некоторый разумный порог, используйте прогноз эталонной модели или отправьте строку данных на дополнительную проверку.

Будь то значения в новых данных, выходящие за пределы допустимого диапазона, неудовлетворительные ключевые показатели эффективности, дрейф или аномалии — эти проблемы в реальном времени являются тем местом, где ваши замыслы сталкиваются с реальностью и рождаются инциденты с искусственным интеллектом. Если мониторинг обнаружит серьезные проблемы, естественным желанием будет отключить систему. В следующем подразделе рассматривается именно этот исход: аварийные выключатели для систем ML.

Аварийные выключатели

Аварийные выключатели (kill switch) редко представляют собой отдельные приложения или исполняемые скрипты. Как правило, это набор взаимосвязанных бизнес- и технических процессов, которые служат для отключения системы ML — в той степени, в которой это возможно. Прежде чем нажать на пресловутый аварийный выключатель, нужно многое обдумать. Результаты системы ML часто используются в последующих бизнес-процессах, иногда включая другие системы ML. Эти системы и бизнес-процессы могут быть критически важными, например система ML, используемая для андеррайтинга кредитов или проверки розничных электронных платежей. Чтобы отключить систему ML, вам нужны не только правильные технические решения и квалифицированный персонал, но и понимание места системы в более широких организационных процессах. Разгар инцидента с ИИ — неподходящее время, чтобы обдумывать последствия отключения фатально ошибочной системы ML. Поэтому процессы и инструменты аварийного отключения должны быть заранее описаны в документации по системе ML и планах реагирования на инциденты с ИИ (см. главу 1). Когда придет время отключить систему ML, ваша организация будет готова принять быстрое и обоснованное решение. Хочется верить, что вы никогда не окажетесь в ситуации, когда необходимо будет нажать аварийный выключатель системы ML, но, к сожалению, в последние годы инциденты с ИИ стали более распространенными. Безопасность и надежность систем ML воз-

Книги для программистов: https://t.me/free_books_for_it

растают, когда технические методы реагирования на инциденты применяются наряду с культурными компетенциями и бизнес-процессами, направленными на снижение рисков. Игнорирование комплексных мер контроля может привести к серьезным последствиям.

Пример: смерть от автономного транспортного средства

Ночью 18 марта 2018 г. Элейн Герцберг ехала на велосипеде по широкому перекрестку в Темпе, штат Аризона. В одном из самых нашумевших инцидентов с ИИ ее сбил автономный испытательный автомобиль Uber, двигавшийся со скоростью примерно 65 км/ч. По данным Национального совета по безопасности на транспорте (NTSB), водитель-испытатель автомобиля, который был обязан взять на себя управление транспортным средством в экстренных ситуациях, отвлекся на смартфон. Автономная система ML также не смогла спасти г-жу Герцберг. Система распознала ее присутствие только за 1,2 секунды до удара, что было слишком поздно, чтобы предотвратить смертельную аварию.

Последствия

Считается, что автономные транспортные средства обеспечивают преимущества в безопасности по сравнению с сегодняшним статус-кво транспортных средств, управляемых человеком. Хотя смертельные случаи с участием беспилотных автомобилей редки, автоматизированное вождение с использованием ML еще не оправдало надежды на более безопасные дороги. В отчете NTSB говорится, что «устройство системы Uber не предусматривало внезапное появление людей, пересекающих проезжую часть». В отчете также критиковалась небрежная оценка рисков и неразвитая культура безопасности в компании. Кроме того, всего за несколько дней до инцидента в Темпе сотрудник Uber выразил серьезную обеспокоенность по поводу 37 аварий, произошедших за предыдущие 18 месяцев, а также общих проблем с водителями-испытателями. В результате крушения в Темпе испытания беспилотных транспортных средств Uber были остановлены в четырех других городах, а местные органы власти по всей территории США и Канады начали пересматривать протоколы безопасности для испытаний беспилотных транспортных средств. Водителю было предъявлено обвинение в убийстве по неосторожности. Суд освободил Uber от уголовной ответственности, когда компания урегулировала вопрос денежной компенсации с семьей погибшего. Семья г-жи Герцберг также предъявила иски городу Темпе и штату Аризона на 10 млн долл. каждый.

Неподготовленная правовая система

Следует отметить, что правовая система в США не совсем готова к реальности инцидентов с ИИ. Это может оставить сотрудников, потребителей и широкую общественность без должной защиты от уникальных опасностей, которые таят в себе окружающие нас системы ML. Парламент ЕС предложил установить режим ответственности за системы ML, который не позволит крупным технологическим ком-

Книги для программистов: https://t.me/free_books_for_it

паниям избежать своей доли ответственности в будущих инцидентах. В США проекты федеральных правил безопасности продуктов ИИ все еще находятся на ранней стадии. В настоящее время отдельные дела об инцидентах, связанных с безопасностью ИИ, обычно рассматривают суды низшей инстанции с весьма слабыми компетенциями и опытом в рассмотрении инцидентов с ИИ, что позволяет крупным технологическим компаниям и другим операторам систем ML использовать крайне асимметричные юридические ресурсы для борьбы с лицами, пострадавшими в инцидентах со сложными системами ML. Но даже для компаний и операторов систем ML эта правовая неопределенность не идеальна. Хотя отсутствие регулирования, похоже, идет на пользу тем, у кого больше всего ресурсов и опыта, оно затрудняет управление рисками и прогнозирование последствий инцидентов с ИИ. Будущие поколения могут сурово осудить нас за то, что мы позволили водителю-испытателю недостаточно автоматизированного транспортного средства нести уголовную ответственность за один из первых инцидентов с искусственным интеллектом, в котором участвовали многие высококвалифицированные специалисты и руководители.

Какие уроки мы можем извлечь?

Какие уроки мы можем извлечь из этого примера с учетом знаний, полученных в предыдущих главах?

Урок 1: необходимо развивать культуру безопасности

Зрелая культура безопасности — это широкий контроль рисков, который ставит безопасность на передний план при проектировании и внедрении, а также устраняет потенциальные уязвимости в процессах и технологиях. Более развитая культура безопасности в Uber, позаимствованная у таких передовых коммерческих отраслей, как аэрокосмическая техника и ядерная энергетика, могла бы предотвратить этот инцидент, тем более что за несколько дней до аварии один из сотрудников выразил серьезные опасения.

Урок 2: смягчение последствий предсказуемых отказов

NTSB пришел к выводу, что программное обеспечение Uber не рассматривает пешеходов, оказавшихся в неположенном месте, как экстремальную ситуацию с высоким риском. Для любого, кто водил машину в мире пешеходов, это является очевидной и легко предсказуемой проблемой, к которой должен быть готов любой беспилотный автомобиль. Системы ML обычно не готовы к инцидентам, если их не подготовят люди. Инцидент с автомобилем Uber показывает нам, что происходит, когда эта подготовка не выполнена заранее.

Урок 3: тестируйте системы машинного обучения в их рабочей области

После аварии Uber остановился и перезапустил программу разработки беспилотных автомобилей. После доработок с помощью моделирования удалось показать, что новое программное обеспечение начинает аварийное торможение за четыре секунды до удара. Почему легко предсказуемая ситуация с пешеходом, переходящим дорогу в неположенном месте, не была протестирована с помо-

Книги для программистов: https://t.me/free_books_for_it

щью тех же внутренних симуляций перед смертельной аварией в марте 2018 года? Скорее всего, мы этого никогда не узнаем. Но перечисление вариантов отказа и их тестирование в реалистичных сценариях может избавить вашу организацию от необходимости отвечать на такого рода неприятные вопросы.

Дополнительно можно сделать вывод о том, что нужно анализировать не только случайные сбои, такие как авария Uber, но и злонамеренные взломы систем ML и злоупотребление системами ML для совершения насилия. Террористы и раньше превращали автомобили в смертоносное оружие, так что это известный источник инцидентов. Необходимо принимать опережающие меры предосторожности в отношении беспилотных транспортных средств и функций помощи при вождении, чтобы предотвратить тяжелые последствия хакерских атак. Независимо от того, будет ли это несчастным случаем или злонамеренной атакой, инциденты с ИИ, безусловно, убьют и других людей. Мы надеемся, что правительства и другие организации серьезно отнесутся к безопасности ML и сведут к минимуму количество этих мрачных инцидентов в будущем.

Ресурсы

Дополнительное чтение

- A Comprehensive Study on Deep Learning Bug Characteristics: **https://oreil.ly/89R6O**
- Debugging Machine Learning Models: **https://oreil.ly/685C3**
- Real-World Strategies for Model Debugging: **https://oreil.ly/LvrLk**
- Safe and Reliable Machine Learning: **https://oreil.ly/mLU8l**
- Overview of Debugging ML Models: **https://oreil.ly/xZGoN**
- DQI: Measuring Data Quality in NLP: **https://oreil.ly/aa7rv**
- Identifying and Overcoming Common Data Mining Mistakes: **https://oreil.ly/w19Qm**

Примеры кода

- Базовый пример анализа чувствительности и остатков: **https://oreil.ly/Tcu65**
- Продвинутый пример анализа чувствительности: **https://oreil.ly/QPFFx**
- Продвинутый пример анализа остатков: **https://oreil.ly/Poe20**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 4

Борьба с предвзятостью в машинном обучении

Борьба с вредными последствиями предвзятости в системах ML — это нечто большее, чем просто данные, код и модели. Средние значения показателей модели (основной способ, с помощью которого специалистов по обработке данных учат оценивать качество модели) не имеют ничего общего с тем, наносит ли она своей предвзятостью вред в реальном мире. Очень точная модель может впитать в себя предвзятость обучающих данных и нанести ущерб пользователям. Хуже того, все системы ML демонстрируют некоторый уровень предвзятости, а инциденты с предвзятостью, по-видимому, являются одними из наиболее распространенных в области ИИ (рис. 4.1). Предвзятость в бизнес-процессах часто влечет за собой юридическую ответственность, а предвзятость моделей ML вредит людям в реальной жизни.

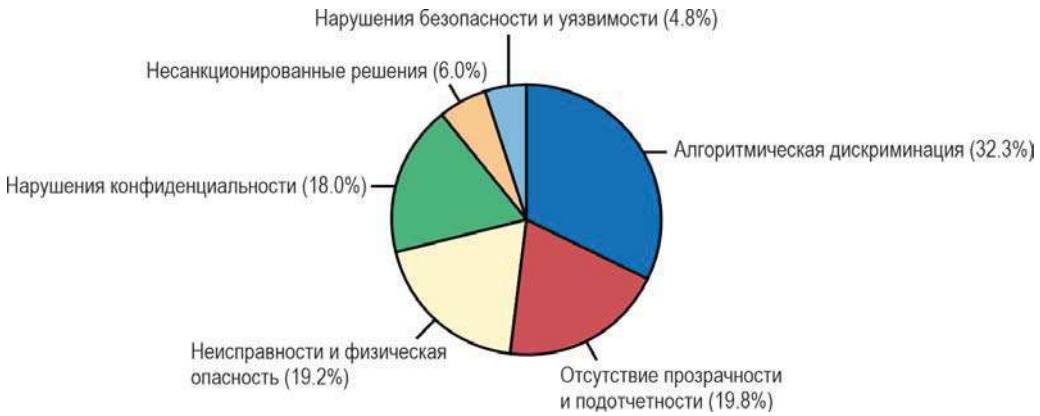


Рис. 4.1. Частота различных типов инцидентов с искусственным интеллектом на основе качественного анализа. 169 обнародованных случаев в период с 1988 г. по 1 февраля 2021 г. (рисунок предоставлен BNH.AI)

В этой главе будут предложены подходы к обнаружению и смягчению предвзятости социотехническим способом, насколько это доступно практикующим специалистам. Это означает, что мы попытаемся понять, как предвзятость системы ML отражается в более широком социальном контексте. Почему? *Все* системы ML являются социотехническими. Мы знаем, что поначалу в это может быть трудно поверить, поэтому задумайтесь над одним примером. Рассмотрим модель, предназначенную для прогнозирования отказа датчика в приложении Интернета вещей (IoT), исходя только из информации от других автоматизированных датчиков. Эта

Книги для программистов: https://t.me/free_books_for_it

модель, очевидно, была обучена людьми, и какие-то люди решили, что она необходима. Более того, прогнозы этой модели можно использовать для заказа новых датчиков, что наверняка повлияет на занятость персонала завода-изготовителя или тех, кто ремонтирует или заменяет вышедшие из строя датчики. Наконец, если наша модель профилактического обслуживания потерпит неудачу, могут пострадать люди, взаимодействующие с системой. Если глубже вникнуть в каждый пример, который кажется нам чисто техническим, становится очевидным, что технологии принятия решений, такие как ML, не существуют без того или иного взаимодействия с людьми.

Это означает, что проблема предвзятости в системах ML не может быть решена исключительно техническими средствами. Если читатели хотят сразу перейти к коду для обнаружения и устранения предвзятости, они могут приступить к чтению главы 10. Однако мы не рекомендуем забегать вперед. Читатели упустят много важной информации о том, что такое предвзятость и как с ней эффективно бороться. В начале этой главы мы обсудим понятие предвзятости, основываясь на мнениях различных авторитетных источников. Также мы расскажем, как распознать наши собственные когнитивные предубеждения, которые могут повлиять на системы машинного обучения или на результаты, которые интерпретируют наши пользователи. Далее в главе представлен общий обзор типичных жертв инцидентов, связанных с предвзятостью ИИ, и вреда, который они испытывают. Затем мы рассмотрим методы проверки предвзятости в системах ML и обсудим смягчение предвзятости с использованием как технических, так и социотехнических подходов. Глава завершается обсуждением алгоритма обрезки изображений в «Твиттере» (X).



Хотя некоторые аспекты управления предвзятостью должны быть адаптированы к конкретной архитектуре модели, бóльшая часть механизмов управления предвзятостью не зависит от модели. Многие идеи в этой главе, особенно те, которые взяты из руководства по предотвращению предвзятости NIST SP1270 и вознаграждения Twitter Bias Bounty, можно применить к широкому спектру сложных систем искусственного интеллекта, таких как языковые модели ChatGPT или RoBERTa. Читатели, желающие увидеть практический пример, могут ознакомиться с аудитом RoBERTa (*https://oreil.ly/3hs_6*), проведенным IQT Labs.

Схема соответствий NIST AI RMF

Раздел главы	Подкатегория NIST AI RMF
Системная предвзятость	MAP 1.6, MAP 2.3, MEASURE 2.11
Статистическая предвзятость	MAP 2.3, MEASURE 2.6, MEASURE 2.11
Человеческая предвзятость и культура науки о данных	GOVERN 3.2, MAP 1.1, MAP 2.3, MEASURE 2.11
Правовые понятия предвзятости ML в США	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, GOVERN 2.2, GOVERN 4.1, MAP 1.1, MAP 1.2, MEASURE 2.11
Кто обычно страдает от предвзятости систем ML	GOVERN 1.2, GOVERN 1.4, GOVERN 5, MAP 1.1, MAP 1.2, MAP 1.6, MAP 2.2, MAP 3, MAP 5, MEASURE 1.3, MEASURE 4, MANAGE 2, MANAGE 4

Книги для программистов: https://t.me/free_books_for_it

Окончание

Раздел главы	Подкатегория NIST AI RMF
Ущерб от предвзятости, который испытывают люди	GOVERN 1.2, GOVERN 1.4, GOVERN 5, MAP 1.1, MAP 1.2, MAP 1.6, MAP 2.2, MAP 3, MAP 5, MEASURE 1.3, MEASURE 4, MANAGE 1.4, MANAGE 2, MANAGE 4
Проверка данных	MEASURE 1.1, MEASURE 2.1
Традиционные подходы: тестирование эквивалентных результатов	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, GOVERN 4.3, GOVERN 6.1, MAP 2.3, MAP 4, MEASURE 1, MEASURE 2.1, MEASURE 2.6, MEASURE 2.11, MEASURE 4.2
Новый взгляд: тестирование эквивалентности качества показателей	GOVERN 1.2, GOVERN 1.4, GOVERN 4.3, GOVERN 6.1, MAP 2.3, MAP 4, MEASURE 1, MEASURE 2.1, MEASURE 2.6, MEASURE 2.11, MEASURE 4.2
Чего нам ожидать: тесты для более широкой экосистемы ML	GOVERN 1.2, GOVERN 1.4, GOVERN 4.3, GOVERN 6.1, MAP 2.3, MAP 4, MEASURE 1, MEASURE 2.1, MEASURE 2.6, MEASURE 2.11, MEASURE 4.2, MANAGE 3.2
Технические факторы смягчения предвзятости	MAP 2.3, MANAGE
Научный подход и план эксперимента	MAP 1.1, MAP 2.3
Человеческий фактор в смягчении предвзятости	GOVERN 2.1, GOVERN 3, MAP 1.1, MAP 1.6, MAP 2.2, MAP 2.3, MEASURE 3.2, MEASURE 3.3, MANAGE 1, MANAGE 2.1, MANAGE 2.2, MANAGE 3, MANAGE 4.2, MANAGE 4.3

- В документации применяются следующие характеристики надежности ИИ: с ограниченной предвзятостью, прозрачный и подотчетный, действительный и надежный.
- Смотрите также:
  - Towards a Standard for Identifying and Managing Bias in Artificial Intelligence: **https://oreil.ly/8kpf5**
  - полная таблица соответствий (не является официальным ресурсом), **https://oreil.ly/61TXd**

Определения предвзятости в ISO и NIST

В своем документе Statistics — Vocabulary and Symbols — Part 1 Международная организация по стандартизации (International organization for standardization, ISO) определяет предвзятость как «степень отклонения эталонного значения от истины». Это очень обобщенное определение предвзятости, но на практике предвзятость — сложное и неоднородное явление. Однако в любом случае речь идет о каком-то систематическом отклонении от истины. В задачах принятия решений предвзятость принимает множество форм. И по существу, и с этической точки зрения неправильно отказывать людям в трудоустройстве из-за уровня меланина в их коже. Неверно думать, что идея правильная только потому, что она первая пришла на ум. И по существу, и с этической точки зрения неправильно обучать модель ML на неполных и нерепрезентативных данных. В недавней публикации NIST Towards a Standard for Identifying and Managing Bias in Artificial Intelligence (SP1270) (*https://oreil.ly/pkm4f*) понятие предвзятости разделено на три основные категории, которые соответствуют следующим типам предвзятости: системная, статистическая и человеческая.

Книги для программистов: https://t.me/free_books_for_it

Системная предвзятость

Часто, когда мы говорим о предвзятости в ML, мы имеем в виду именно системные предвзятости. Это исторические, социальные и институциональные предубеждения, которые, к сожалению, настолько прочно вошли в нашу жизнь, что по умолчанию проявляются в данных обучения ML и при выборе ее устройства. Распространенным последствием системной предвзятости в моделях ML является включение демографической информации в системные механизмы. Такое включение может быть открытым и явным, например когда языковые модели перепрофилируются для создания вредного и оскорбительного контента, нацеленного на определенные демографические группы. Однако на практике включение демографической информации в процессы принятия решений чаще происходит непреднамеренно и неявно. Это приводит к различным распределениям результатов в разных демографических группах, например из-за того, что высокооплачиваемые и ответственные должности чаще всего сопоставляются с резюме мужчин, или из-за конструктивных проблем модели, которые исключают определенные группы пользователей (например, людей с ограниченными физическими возможностями) из взаимодействия с системой.

Статистическая предвзятость

Статистические предвзятости можно рассматривать как ошибки, допущенные людьми при спецификации систем ML, или как возникающие явления, такие как дрейф концепта, которые влияют на модели ML и плохо поддаются смягчению. Еще одним распространенным типом статистической предвзятости являются прогнозы, основанные на нерепрезентативных обучающих данных или циклах распространения ошибок и обратной связи. Одним из потенциальных индикаторов статистической предвзятости в моделях ML является различное качество прогнозов в разных разрезах данных, например в демографических группах. Разная достоверность модели ML на разных сегментах данных — это особый тип предвзятости, несколько отличающийся от разницы показателей исходов или распространенности исходов, описанных для человеческих предубеждений. Фактически существует документально подтвержденное противоречие между максимизацией эффективности модели внутри демографических групп и поддержанием равенства показателей положительных результатов. Статистическая предвзятость также может привести к серьезным инцидентам с ИИ, например когда дрейф концепта в новых данных делает решения системы скорее неправильными, чем правильными или когда явления петли обратной связи или распространения ошибок приводят к увеличению объемов плохих прогнозов за короткий промежуток времени.

Человеческая предвзятость и культура науки о данных

Существует ряд человеческих или когнитивных предубеждений, влияющих как на отдельных людей, так и на команды, которые проектируют, внедряют и поддерживают системы ML. Более полный список человеческих предубеждений можно найти в руководстве NIST SP1270. Ниже приведены предубеждения, которые, как мы

Книги для программистов: https://t.me/free_books_for_it

наблюдаем, чаще всего встречаются как среди специалистов по данным, так и среди пользователей систем ML:

Якорение (anchoring)

Когда определенная точка отсчета или *якорь* (термин позаимствован из психологии) оказывает чрезмерное влияние на решения людей. Это похоже на ситуацию, когда бенчмарк современной модели глубокого обучения надолго застревает на уровне 0,4 AUC, и вдруг кто-то добивается успеха и получает оценку 0,403 AUC. Очевидно, что это незначительный успех, но мы жаждем положительных эмоций и склонны считать даже небольшое улучшение значительным достижением.

Эвристика доступности

Люди склонны придавать слишком большое значение тому, что легко или быстро приходит на ум в процессе принятия решений. Другими словами, мы часто путаем *легкость запоминания* с *правильностью*.

Склонность к самоподтверждению

Когнитивное искажение, при котором люди склонны отдавать предпочтение информации, которая совпадает с их убеждениями или подтверждает их. Склонность к самоподтверждению — большая проблема в системах машинного обучения, когда мы обманываем себя, думая, что наши модели машинного обучения работают лучше, чем на самом деле.

Эффект Даннинга — Крюгера

Склонность людей с низкими способностями в определенной области или задаче переоценивать свои возможности. Это происходит, когда мы позволяем себе думать, что мы в чем-то эксперты только потому, что можем импортировать `sklearn` и запустить `model.fit()`.

Предвзятость заинтересованности

Склонность к выпячиванию или продвижению результатов, которые интересуют или удовлетворяют финансирующий орган или спонсора проекта. Мы делаем то, что радует наших начальников, обнадеживает наших инвесторов и увеличивает наши собственные зарплаты. Настоящая наука нуждается в подходе, который исключает влияние личных финансовых интересов на пути прогресса.

Групповое мышление

Когда люди в группе склонны принимать неоптимальные решения, основываясь на своем желании соответствовать группе или страхе проявить несогласие с группой. Даже когда мы уверены в своей правоте, сложно выступить против авторитетной команды.

Заблуждение Макнамары

Убеждение в том, что решения следует принимать исключительно на основе доступной количественной информации в ущерб качественной информации или данным, которые сложно измерить.

Книги для программистов: https://t.me/free_books_for_it

Техношовинизм

Вера во всемогущество технологий, способных решить любую задачу.

Все эти предубеждения могут привести и приводят к неподходящему и чрезмерно оптимистичному выбору модели, что, в свою очередь, приводит к снижению ее качества при развертывании системы и, наконец, к причинению вреда пользователям или операторам системы. Вскоре мы поговорим о потенциальном ущербе, и о том, что делать с этими проблемами. А сейчас мы хотим сосредоточиться на простых и практичных средствах решения проблемы, которые также являются темой этой главы. Мы не можем правильно бороться с предвзятостью, не рассматривая проблему с разных точек зрения. «Нулевой этап» в борьбе с предвзятостью в ML — это проведение совещания (или видеозвонка), на котором принимаются важные решения относительно системы с участием максимально разнообразной группы заинтересованных лиц. Чтобы избежать «слепых зон», которые позволяют предвзятым моделям машинного обучения причинять вред, нам понадобится принять во внимание как можно больше разных точек зрения, которые помогут принять оптимальные решения по проектированию, внедрению и обслуживанию системы. Речь идет, прежде всего, об участии представителей разных демографических групп, в том числе людей с ограниченными возможностями. Мы также говорим о людях с разнообразным образованием, например о социологах, юристах и экспертах в предметной области.

Также обратите внимание на *цифровой разрыв*. Шокирующий процент населения до сих пор не имеет хорошего подключения к Интернету, доступа к новым компьютерам и такой информации, как эта книга. Если вы делаете выводы о своих пользователях, то не забывайте, что существует значительная часть населения, которая не будет включена в пользовательскую статистику. Игнорирование потенциальных пользователей — это огромный источник предвзятости и вреда при проектировании систем, тестировании на предвзятость и других важных этапах жизненного цикла машинного обучения. Сегодня успех в машинном обучении попрежнему требует участия людей, которые хорошо понимают реальную проблему, которую вы пытаетесь решить, и то, какие потенциальные пользователи могут быть упущены из нашего проектирования, данных и тестирования.

Правовые понятия предвзятости ML в США

Вам следует знать о многих важных юридических понятиях, связанных с предвзятостью. Однако также важно понимать, что правовая система чрезвычайно сложна и зависит от контекста. Простое знание нескольких определений все равно оставляет нас бесконечно далеко позади людей, имеющих реальный опыт в этих вопросах. Как специалисты по обработке данных, вы не должны позволять эффекту Даннинга — Крюгера брать верх в юридических вопросах. Далее мы будем придерживаться этой стратегии.



Если у вас есть какие-либо вопросы или опасения по поводу предвзятости в моделях ML, сейчас самое время обратиться к вашей юридической команде. Борьба с предвзятостью в моделях ML — одна из самых сложных и серьезных проблем информационной экономики. Специалисты по работе с данными нуждаются в помощи юристов, чтобы правильно устранить риски предвзятости.

Книги для программистов: https://t.me/free_books_for_it

В США на протяжении десятилетий разрабатывали законы и нормы, направленные на ограничение предвзятости в процессах принятия решений, затрагивающих общество. Основное внимание в ранних законах и постановлениях США уделялось вопросам занятости. Такие понятия, как *защищенные группы*, *несоразмерное отношение* (disparate treatment) и *несоразмерное влияние* (disparate impact), теперь распространились на более широкий свод законов в области потребительского финансирования и жилищного строительства, и сегодня даже цитируются в совершенно новых локальных законах, таких как требование города Нью-Йорка к аудиту ИИ, используемого при найме сотрудников. Вопрос о недопущении дискриминации в ЕС рассматривается в Хартии основных прав, Европейской конвенции по правам человека и в Договоре о функционировании ЕС, а также, что крайне важно для нас, в аспектах предлагаемого Закона ЕС об ИИ. Хотя все эти многочисленные законы и правила невозможно кратко обобщить даже внутри США, приведенные ниже определения, по нашему мнению, лучше всего применимы к повседневной работе специалиста по обработке данных. В общих чертах они взяты из таких законов, как Закон о гражданских правах, Закон о справедливом жилищном обеспечении (FHA), положения Комиссии о равных возможностях в сфере занятости (EEOC), Закон о равных кредитных возможностях (ECOA) и Закон об американцах с ограниченными возможностями (ADA). Следующие определения обобщают юридические представления о том, какие аспекты защищаются законом и от чего эти законы стремятся нас защитить:

Защищенные группы

В США многие законы и постановления запрещают дискриминацию по признаку расы, пола (или в некоторых случаях гендера), возраста, религиозной принадлежности, национального происхождения и наличия инвалидности, а также других категорий. К запрещенным основаниям для принятия решений в соответствии с FHA относятся раса, цвет кожи, религия, национальное происхождение, пол, семейное положение и инвалидность. GDPR ЕС как пример одного из законов за пределами США запрещает использование персональных данных о расовом или этническом происхождении, политических взглядах и других категориях, в некоторой степени аналогично защищенным группам в США. Это одна из причин, почему в традиционном тестировании предвзятости предусмотрено сравнение результатов защищенных групп и так называемых контрольных (или эталонных) групп, которые не являются защищенными.

Несоразмерное отношение

Несоразмерное отношение — это особый вид дискриминации, когда к человеку относятся менее благосклонно, чем к людям, находящимся в аналогичном положении, из-за таких параметров, как раса, пол или другие признаки. Специалисты по работе с данными в области заявок на трудоустройство, жилье или кредит должны быть очень осторожны при использовании демографических данных в моделях ML и даже в методах устранения предвзятости. Если в модели в качестве входных данных используются демографические данные, это может означать, что решение для одного человека будет отличаться от решения для другого только из-за его демографических характеристик. В некоторых случаях это может привести к предвзятости отношения.

Книги для программистов: https://t.me/free_books_for_it



Опасения по поводу предвзятости отношения и более общей системной предвзятости являются причиной того, что мы обычно стараемся не использовать демографические маркеры в качестве прямых входных данных для моделей ML. Демографические маркеры не следует использовать в качестве входных данных модели в большинстве распространенных сценариев; они подходят для тестирования систематических ошибок или целей мониторинга.

Несоразмерное влияние

Несоразмерное влияние — это еще один вид дискриминации с юридической точки зрения. По сути, речь идет о различной частоте прогнозов или их разной распространенности среди демографических групп. Несоразмерное влияние более формально определяется как результат, казалось бы, нейтральной политики или практики, которая наносит непропорциональный вред защищаемой группе. В работе специалистов по данным, как правило, несоразмерное влияние возникает, когда мы не используем демографические данные в качестве входных данных напрямую, но применяем что-то, коррелирующее с демографическими данными. Возьмем кредитные рейтинги: они являются довольно точным предиктором дефолта, поэтому их часто используют в прогнозных моделях потребительского кредитования. Однако они коррелируют с расой, так что некоторые группы меньшинств в среднем имеют более низкие кредитные баллы. Если мы используем кредитный рейтинг в модели, это, как правило, приводит к тому, что определенные группы меньшинств имеют меньшую долю положительных результатов, и это типичный пример несоразмерного влияния. (Именно поэтому отдельные штаты начали ограничивать использование кредитных рейтингов при принятии некоторых решений, связанных со страхованием.)

Неодинаковая достоверность

Неодинаковая достоверность (differential validity) — это ситуация, которая иногда возникает в сфере трудоустройства. Если несоразмерное влияние часто связано с разной *частотой* прогнозов в разных демографических группах, то неодинаковая достоверность больше связана с разным *качеством* прогнозов в разных группах. Она возникает, когда тест при трудоустройстве более достоверно отражает показатели одних групп по сравнению с другими. Очень важно помнить о неодинаковой достоверности, потому что она фактически заложена в математический фундамент модели. На практике обычно используют недостаточно репрезентативные обучающие данные и строят модель, которая более достоверно отражает одни группы по сравнению с другими, и многие новейшие подходы к тестированию предвзятости сосредоточены именно на этой проблеме.

Отсеивание

Отсеивание (screen out) — это очень важный тип дискриминации, который подчеркивает социотехническую природу систем ML и доказывает, что простого тестирования и балансировки оценок модели недостаточно для защиты от предвзятости. Отсеивание происходит, когда человек с ограниченными возможностями, например с плохим зрением или трудностями с мелкой моторикой, не может участвовать в тестировании на трудоустройство и по умолчанию отсеивается от работы или продвижения по службе. Отсеивание — серьезная проблема,

Книги для программистов: https://t.me/free_books_for_it

и EEOC и Министерство труда строго контролируют использование ML в этой сфере. Надо заметить, что отсеивание не обязательно устраняют с помощью исключения предвзятости математическими методами; обычно эту проблему необходимо решать на этапе проектирования системы, когда разработчики создают интерфейсы конечного продукта, удобные для людей с ограниченными возможностями. Отсеивание также говорит о том, почему нам нужны мнения юристов и людей с ограниченными возможностями при создании систем ML. Без такого подхода слишком легко забыть о людях с ограниченными возможностями, а это иногда может повлечь за собой юридическую ответственность.

На этом мы завершаем обсуждение общих определений предвзятости. Как видите, это сложная и многогранная тема, которая затрагивает самые разные человеческие, научные и юридические аспекты. Мы добавим к этим определениям более конкретные, но, вероятно, более сложные математические определения предвзятости, когда будем обсуждать тестирование предвзятости позже в этой главе. А сейчас мы опишем, кто обычно испытывает предвзятость и связанный с этим ущерб от систем ML.

Кто обычно страдает от предвзятости систем ML

С предвзятостью систем ML может столкнуться любая группа, но факты говорят нам, что определенные группы с большей вероятностью будут испытывать от нее вред. По сути, сама природа обучения с учителем — когда модель изучает и повторяет закономерности только на основе ранее записанных данных — ведет к тому, что пожилые люди, люди с ограниченными возможностями, иммигранты, нацменьшинства, женщины и гендерно-неконформные личности сталкиваются с большей предвзятостью со стороны ML-системы. Иными словами, те, кто сталкивается с дискриминацией в реальном или цифровом мире, скорее всего, также столкнутся с ней при взаимодействии с системами ML, поскольку вся эта дискриминация фиксируется в данных и отражается на обучении моделей. Группы, перечисленные в этом разделе, часто защищены различными законами, но не всегда. Они также часто, но не всегда, будут группой сравнения при тестировании систематической ошибки на предмет статистического равенства баллов или результатов между двумя демографическими группами.

Многие люди принадлежат сразу к нескольким защищенным или маргинализированным группам. Важная концепция *межсекциональности* (intersectional) говорит нам, что социальный ущерб особенно велик среди тех, кто оказался на пересечении нескольких защищенных групп, и что предвзятость не следует анализировать по измерениям только одной группы. Например, исследователи этики ИИ недавно показали, что некоторые коммерческие системы распознавания лиц имеют существенные различия в точности гендерной классификации, при этом темнокожие женщины являются наиболее неправильно классифицируемой группой. Наконец, прежде чем давать определение этим группам, важно также подумать об ошибке Макнамары. Правильно ли вообще помещать полные нюансов человеческие личности в такую упрощенную таксономию? Вероятно, нет, и вполне возможно, что раз-

Книги для программистов: https://t.me/free_books_for_it

биение на упрощенные группы, которое часто делается потому, что такие категории легко представить в виде столбцов двоичных маркеров в базе данных, также является источником предвзятости и потенциального вреда. Попытки ограничения предвзятости в системах ML всегда сталкиваются со множеством подводных камней. Мы должны помнить об этом и очень осторожно применять следующие упрощенные демографические группы, которые, как правило, сталкиваются с большей дискриминацией, и которые часто используются в качестве групп сравнения в традиционном тестировании на предвзятость:

Возраст

Пожилые люди, обычно в возрасте 40 лет и старше, чаще подвергаются дискриминации при принятии автоматизированных решений. Возрастной порог может быть выше в более традиционных приложениях, таких как трудоустройство, аренда жилья или потребительское кредитование. Однако в других сценариях участие в программе Medicare или наличие накоплений, сделанных на протяжении всей жизни, могут, напротив, сделать пожилых людей предпочтительной группой.

Инвалидность

Люди с физическими, умственными или эмоциональными недостатками, возможно, относятся к категории, которая с наибольшей вероятностью может столкнуться с предвзятостью со стороны систем ML. Юридические нормы запрещают отсев таких людей при трудоустройстве, но разнообразие вариантов ограничения в реальной жизни намного богаче. О людях с ограниченными возможностями часто вообще забывают при разработке систем ML, и никакие математические проверки или исправления ошибок не могут это компенсировать.

Иммиграционный статус или национальное происхождение

Известно, что люди, живущие в стране, в которой они не родились, с любым иммиграционным статусом, в том числе натурализованные граждане, сталкиваются со значительными проблемами предвзятости.

Язык

Люди, которые используют языки, отличные от английского, или пишут нелатинскими буквами, особенно в сценарии важного для многих систем ML диалога в реальном времени, могут с большей вероятностью столкнуться с предвзятостью системы.

Раса и этническая принадлежность

Расовые и этнические группы, отличные от белых людей, включая тех, кто идентифицирует себя принадлежащим к более чем одной расе, традиционно подвергаются предвзятости и ущербу при взаимодействии с системами ML. В некоторых моделях, особенно в системах компьютерного зрения, разработчики применяют шкалы оттенков кожи вместо традиционных меток расовой или этнической принадлежности. Примером может служить шкала фототипов кожи Фитцпатрика (*https://oreil.ly/NJfBP*).

Книги для программистов: https://t.me/free_books_for_it

Пол и гендер

Представители всех полов и гендеров, кроме цисгендерных мужчин, с большей вероятностью будут подвергаться предвзятости и ущербу со стороны системы ML. В визуальном контексте женщинам часто отдается предпочтение, но это потенциально опасная ситуация. Существует так называемый *феномен мужского взгляда* (male gaze phenomenon), в соответствии с которым женщины считаются более привлекательными и испытывают преувеличенно положительное отношение (например, продвижение в социальных сетях), но истинная подоплека этого эффекта негативная и заключается в объективации и сексуализации женщин.

Межсекциональные группы

Люди, входящие в две или более из вышеупомянутых групп, могут испытывать предвзятость или ущерб, превышающий простую сумму ущербов двух более широких групп, к которым они принадлежат. Все шаги по тестированию и смягчению предвзятости, описанные в этой главе, должны учитывать пересекающиеся группы.

Конечно, это не полный перечень групп людей, которые могут испытывать предвзятость модели ML, и разбиение людей на группы само по себе может служить причиной проблем, даже если оно преследует благие цели. Однако важно знать, с чего начать поиск предвзятости, и мы надеемся, что нашего списка будет достаточно для первых шагов. Теперь, когда вы знаете, где искать предвзятость ML, давайте обсудим наиболее распространенные вредные последствия, о которых нам следует помнить.

Ущерб от предвзятости, который испытывают люди

Ущерб от предвзятости систем ML может быть причинен непосредственно при взаимодействии в реальной жизни или заложен в цифровой контент. Подобные эпизоды случаются часто — возможно, настолько часто, что мы можем перестать их замечать. В следующем списке выделены наиболее распространенные факторы ущерба и приведены примеры, чтобы вы могли лучше их распознать, когда увидите в следующий раз. Потенциальный ущерб от этих явлений тесно связан с вредом, описанным в книге Абагейла Ли Бланка Computer Vision Machine Learning and Future-Oriented Ethics (*https://oreil.ly/-JmJA*), в которой описаны случаи, когда ущерб причиняют системы компьютерного зрения:

Клевета

Контент, который носит активно уничижительный или оскорбительный характер, например оскорбительный контент, созданный такими чат-ботами, как Tay (*https://oreil.ly/2938n*) или Lee Luda (*https://oreil.ly/nRzs1*).

Удаление

Удаление контента, бросающего вызов доминирующим социальным парадигмам или описывающего вред, нанесенный маргинализированным группам, например подавление контента, в котором обсуждается расизм или осуждается превосходство белой расы.

Книги для программистов: https://t.me/free_books_for_it

Эксноминация

Отношение к таким понятиям, как белый цвет кожи, мужественность или гетеросексуальность, как к ключевым человеческим нормам — например, онлайн-поиск, возвращающий куклу Барби в качестве первого женского результата для запроса «генеральный директор».

Нераспознавание

Ошибочное распознавание личности или неспособность распознать в объекте человека — например, неправильное распознавание чернокожих людей при автоматической маркировке изображений.

Стереотипирование

Тенденция навешивать ярлыки всем членам группы — например, когда ML автоматически ассоциирует мусульман с насилием.

Недостаточная представленность

Отсутствие справедливой или адекватной представленности демографических групп в результатах модели: например, генеративные модели, предполагающие, что все врачи — белые мужчины, а все медсестры — белые женщины.

В настоящее время этот ущерб преимущественно ограничивается онлайн-взаимодействием или цифровым пространством. Но по мере того, как цифровой мир все более существенно переплетается с другими частями нашей жизни, ущерб также распространяется и на реальный мир. Системы ML в здравоохранении, трудоустройстве, образовании и других областях повышенного риска могут нанести непосредственный вред, неправомерно отказывая людям в доступе к необходимым ресурсам. К наиболее очевидным типам реального вреда, причиняемого системами ML, относятся следующие:

Экономический ущерб

Когда система ML снижает экономические возможности или ценность какой-либо деятельности — например, когда мужчины видят больше объявлений о лучшей работе, чем женщины.

Физический ущерб

Когда система ML причиняет кому-то вред или убивает — например, когда люди чрезмерно полагаются на беспилотные автомобили.

Психологический ущерб

Когда система ML вызывает умственное или эмоциональное расстройство, например когда поисковая система выдает детям жестокий или пугающий контент.

Репутационный ущерб

Когда система ML подрывает репутацию человека или организации (например, запуск продукта потребительского кредитования омрачается обвинениями в дискриминации).

Книги для программистов: https://t.me/free_books_for_it

К сожалению, пользователи или субъекты систем ML могут столкнуться с дополнительным ущербом или сочетанием ущербов, которые проявляются непредсказуемым образом. Прежде чем мы углубимся в различные виды тестирования на предвзятость в следующем разделе, помните, что общение с пользователями, возможно, является одним из кратчайших путей отслеживания предвзятости в системах ML. В самом деле, гораздо большее значение имеет то, испытывают ли люди ущерб, а не то, проходит ли некоторый набор показателей заведомо ошибочный математический тест. При разработке системы ML мы должны помнить о возможности причинения ущерба, не выявляемого формальными показателями, и разговаривать с нашими пользователями, чтобы убедиться, что мы никому не причиним вреда, или хотя бы стараться минимизировать возможный ущерб.

Тестирование на предвзятость

Если существует вероятность того, что система ML может нанести вред людям, ее следует проверить на предмет предвзятости. Цель этого раздела — охватить наиболее распространенные подходы к тестированию моделей ML на предмет предвзятости, чтобы читатели могли приступить к решению этой важной задачи управления рисками. Тестирование не является ни простым, ни окончательным. Как и при тестировании качества прогнозов, система может выглядеть нормально на тестовых данных, но после развертывания выйти из строя или причинить вред. Или система может проявлять минимальную предвзятость на этапах тестирования и развертывания, но со временем начинает делать предвзятые или вредные прогнозы. Более того, существует множество тестов с известными недостатками либо противоречащих друг другу. Хороший обзор этих проблем можно найти на YouTube в докладе профессора Принстонского университета Арвинда Нараянана 21 Fairness Definitions and Their Politics(*https://oreil.ly/4QnqM*) на конференции ACM по вопросам справедливости, подотчетности и прозрачности в сфере ML. Углубленный математический анализ того, почему мы не можем просто минимизировать все показатели систематической ошибки одновременно, представлен в статье Inherent Trade-Offs in the Fair Determination of Risk Scores (*https://oreil.ly/WvBOg*). Итак, мы сделали необходимые предостережения и готовы перейти к обзору современных методик тестирования на предвзятость.

Проверка данных

В этом разделе поясняется, каким критериям должны удовлетворять обучающие данные для проверки на предвзятость, а также как проверить эти данные на предмет предвзятости еще до обучения модели. Модели машинного обучения учатся на данных. Но никакие данные не являются идеальными или непредвзятыми. Если в обучающих данных присутствует системная предвзятость, она, скорее всего, проявится в прогнозах модели. Поэтому логично начинать с проверки обучающих данных на предвзятость. Но для этого мы должны предположить, что доступны определенные столбцы данных. Как минимум для каждой строки данных нам нужны демографические маркеры, известные результаты (зависимая переменная y, целевой

Книги для программистов: https://t.me/free_books_for_it

признак и т. д.), а позже нам понадобятся результаты модели — прогнозы для регрессионных моделей, а также решения и оценки достоверности или апостериорные значения вероятности для моделей классификации. Хотя существует несколько подходов к тестированию, которые не требуют демографических маркеров, большинство общепринятых подходов нуждаются в них. У вас их нет? Тестирование будет намного сложнее, но мы также предоставим некоторые рекомендации по определению меток демографических маркеров.



Наши модели и данные далеки от совершенства, поэтому не позволяйте лучшему быть врагом хорошего при тестировании на предвзятость. Наши данные никогда не будут идеальными, и мы никогда не найдем идеальный тест. Тестирование очень важно для смягчения последствий предвзятости в реальном мире, но это всего лишь одна часть более широких процессов управления машинным обучением.

Необходимость знать или выводить демографические маркеры — хороший пример того, почему борьба с предвзятостью в ML требует от разработчика целостного мышления, а не просто добавления еще одного пакета Python в конец конвейера. Демографические маркеры и данные индивидуального уровня также более чувствительны с точки зрения конфиденциальности, и иногда организации не собирают эту информацию по соображениям конфиденциальности. Фактически законы о конфиденциальности данных и законы о противодействии дискриминации предъявляют к данным во многом взаимоисключающие требования. На наш взгляд, требования к конфиденциальности данных не должны иметь приоритета над требованиями к исключению дискриминации. Но, как специалисты по данным, мы не можем ответить на такие вопросы самостоятельно. Любой предполагаемый конфликт между требованиями конфиденциальности данных и исключения дискриминации должен решаться юристами и специалистами по соблюдению требований. Столь сложные юридические аспекты являются примером того, почему борьба с предвзятостью в сфере ML требует привлечения широкого круга заинтересованных сторон.



В сфере трудоустройства, потребительского финансирования или других областях, где несопоставимое отношение наиболее строго запрещено, разработчикам систем ML необходимо консультироваться с коллегами-юристами, прежде чем изменять данные на основе информации о членстве в защищенной группе, даже если вы намерены уменьшить предвзятость.

К настоящему времени читатели, вероятно, начинают понимать, насколько сложным и запутанным может быть тестирование на предвзятость. Мы технические специалисты, и борьба с этой сложностью не является исключительно нашей обязанностью, но мы должны осознавать проблему и работать в более широкой команде, чтобы устранить предвзятость в системах ML. Теперь давайте поставим себя на место технического специалиста, ответственного за подготовку данных и проверку их на предмет систематической ошибки. Если у нас есть необходимые данные, в первую очередь необходимо обратить внимание на три основных вопроса — репрезентативность, распределение результатов и косвенные признаки:

Репрезентативность

Основная проверка, которую необходимо выполнить здесь, — это вычислить долю строк для каждой демографической группы в обучающих данных, исходя

Книги для программистов: https://t.me/free_books_for_it

из того, что модели будет сложно изучить группы, представленные небольшим количеством записей в обучающих данных. Как правило, пропорции различных демографических групп в обучающих данных должны отражать популяцию, на которой будет применяться модель. Если это не так, вам, вероятно, следует собрать более репрезентативные данные. Также можно выполнить повторную выборку или повторное взвешивание набора данных для достижения большей репрезентативности. Однако, если вы работаете в сфере трудоустройства, потребительского кредитования или других областях, где несопоставимое обращение запрещено, вам необходимо проконсультироваться с коллегами-юристами, прежде чем изменять ваши данные непосредственно на основе информации о членстве в защищенной группе. Если вы сталкиваетесь с проблемами дифференциальной достоверности (описанными далее в этой главе), то имеет смысл попробовать перебалансировать обучающие данные, чтобы обеспечить большее или равное представительство в группах. Баланс между различными группами может повысить качество прогнозирования для отдельных групп, но в то же время может не улучшить или даже ухудшить несбалансированное распределение положительных результатов.

Распределение результатов

Вам необходимо знать, как результаты (значения переменной y) распределяются по демографическим группам, потому что, если окажется, что некоторые группы получают больше положительных результатов, чем другие, это может привести к несопоставимому воздействию. Вам нужно рассчитать двумерное распределение y по каждой демографической группе. Если вы видите дисбаланс результатов между группами, то можете попытаться выполнить повторную выборку или переоценить данные обучения с некоторыми юридическими оговорками. Скорее всего, в конечном итоге вы узнаете, что риски предвзятости для этой модели слишком высоки, и когда вы проверите ее результаты, вам придется запланировать мероприятия по их устранению.

Косвенные признаки

В большинстве бизнес-приложений ML вы *не* должны обучать модели на демографических маркерах. Но даже если вы не используете демографические маркеры напрямую, такие данные, как имена, адреса, сведения об образовании или изображения лиц, могут косвенно нести в себе большой объем демографической информации. Демографические маркеры могут скрываться и в других косвенных показателях, подчас весьма неожиданных. Один из способов найти косвенные представления демографических данных — построить состязательную модель на основе каждого входного столбца и посмотреть, смогут ли эти модели предсказать какой-либо демографический маркер. Если они могут предсказать демографический маркер, то эти столбцы кодируют демографическую информацию и, вероятно, являются косвенными признаками (прокси-признаками). Если возможно, такие признаки следует удалить из обучающих данных. Косвенные признаки также могут быть более глубоко скрыты в обучающих данных. Не существует стандартного метода тестирования на наличие скрытых признаков,

Книги для программистов: https://t.me/free_books_for_it

но вы можете применить тот же метод состязательного моделирования, который описан для прямых признаков, за исключением того, что вместо использования самих признаков можно использовать сконструированные взаимодействия признаков, которые, как вы предполагаете, могут выступать в качестве прокси-признаков. Мы также предлагаем поручить профильным специалистам, занимающимся юридическими вопросами или соблюдением требований, проверять каждый входной признак модели на предмет риска опосредованной дискриминации. Если прокси-признаки невозможно удалить или вы подозреваете наличие скрытых прокси-признаков, вам следует обратить пристальное внимание на результаты тестирования на предвзятость результатов работы системы и быть готовыми принять меры по исправлению ситуации позже в процессе смягчения предвзятости.

Описанные тесты и проверки репрезентативности, распределения результатов и косвенных признаков в обучающих данных, как и большинство тестов результатов модели, основаны на присутствии маркеров демографических групп. Если у вас нет этих демографических маркеров, то один из общепринятых подходов — вывести их самостоятельно. Например, *улучшенное байесовское геокодирование фамилий* (Bayesian improved surname geocoding, BISG) позволяет сделать вывод о расовой и этнической принадлежности на основе данных имени и почтового индекса в США. Печально, но это правда, что американское общество по-прежнему настолько сегрегировано, что почтовый индекс и имя могут предсказать расовую и этническую принадлежность, часто с точностью выше 90%. Этот метод был разработан корпорацией RAND и Бюро финансовой защиты потребителей (CFPB) и имеет высокий уровень достоверности при проверке предвзятости в области потребительского кредитования. У CFPB даже есть код для BISG на GitHub! При необходимости аналогичные подходы можно использовать для определения пола по имени, номеру социального страхования или году рождения.

Традиционные подходы: тестирование эквивалентных результатов

После того как вы оценили данные на предмет предвзятости, убедились, что у вас есть информация, необходимая для проведения тестирования на предвзятость, и обучили модель, наступает этап проверки ее результатов. Мы начнем обсуждение тестирования модели на предвзятость с рассмотрения некоторых известных тестов. Эти тесты, как правило, упоминаются и имеют прецеденты применения в законодательстве, нормативных актах или юридических комментариях, и они, как правило, основаны на различиях средних результатов между демографическими группами. Подробное изложение традиционных рекомендаций по тестированию на предвзятость можно найти в кратком руководстве Управления программ по соблюдению федеральных контрактов, где описано тестирование процедур отбора на работу. Для такого рода тестов не имеет значения, анализируем ли мы баллы теста на трудоустройство с множественным выбором или числовые баллы современной рекомендательной системы на основе искусственного интеллекта.

Книги для программистов: https://t.me/free_books_for_it



Тесты в этом разделе оперируют понятием *статистического паритета*, когда модель генерирует примерно равные вероятности или благоприятные прогнозы для всех демографических групп.

В табл. 4.1 показано, как эти тесты обычно делятся на категории статистических и практических тестов, а также непрерывных и бинарных результатов. Тесты в значительной степени основаны на понятии защищенных групп, где средний результат для защищенной группы (например, женщин или чернокожих) сравнивается простым, прямым, попарным способом со средним результатом для некоторой контрольной группы (например, мужчин или белых людей соответственно). Это означает, что нам понадобится как минимум один тест для каждой защищенной группы в наших данных. Если такой подход кажется вам устаревшим — вы правы. Но поскольку именно эти тесты наиболее часто использовались в нормативно-правовых документах и судебных процессах на протяжении десятилетий, разумно начать

Таблица 4.1. Некоторые общие показатели, используемые для измерения систематической ошибки в моделях ML, с пороговыми значениями, где это применимо*

Тип теста	Дискретный выход / классифицирующие тесты	Непрерывный выход / регрессионные тесты
Статистическая значимость	Коэффициент логистической регрессии	Коэффициент линейной регрессии
Статистическая значимость	Тест X^2	*t*-тест
Статистическая значимость	Тест Фишера	
Статистическая значимость	Биномиальный z-тест	
Статистическая значимость	Сравнение средних по группам	Сравнение средних по группам
Статистическая значимость	Процентная разница между средними по группам / предельный эффект	Процентная разница между средними по группам
Статистическая значимость	Доля неблагоприятного воздействия (допустимо 0,8–1,25)	Стандартизованная разница средних (SMD, *d* Коэна) (малая разница: 0,2; средняя разница: 0,5; большая разница: 0,8)
Статистическая значимость	Отношение шансов (odd ratio)	
Статистическая значимость	Недобор до равенства (shortfall to parity)	
Дифференциальная валидность	Отношение точностей или AUC (допустимо 0,8–1,25)	Отношение R^2 (допустимо 0,8–1,25)
Дифференциальная валидность	Отношения TPR, TNR, FPR, FNR (допустимо 0,8–1,25)	Отношения MSE, RMSE (допустимо 0,8–1,25)
Дифференциальная валидность	Равенство шансов ([контрольный TPR ≈ защищенный TPR|y = 1] и [контрольный FPR ≈ защищенный FPR|y = 0])	
Дифференциальная валидность	Равенство возможностей ([control TPR ≈ защищенный TPR|y = 1])	

* TPR = true positive rate (доля истинно положительных прогнозов);
TNR = true negative rate (доля истинно отрицательных прогнозов);
FPR = false positive rate (доля ложноположительных прогнозов);
FNR = false negative rate (доля ложноотрицательных прогнозов).

Книги для программистов: https://t.me/free_books_for_it

именно с них, прежде чем изучать творческие подходы новых методологий. У более отлаженных тестов также, как правило, есть чёткие пороговые значения, которые указывают, когда результаты являются проблематичными. Эти пороговые значения перечислены в табл. 4.1 и более подробно обсуждаются в последующих разделах.

Тестирование статистической значимости

Тестирование статистической значимости, вероятно, пользуется наибольшим признанием во всех дисциплинах и правовых юрисдикциях, поэтому давайте сначала сосредоточимся на нем. Тестирование статистической значимости применяется для определения того, будут ли средние или пропорциональные различия в результатах модели между защищенными группами видны в новых данных или же различия в результатах возникли случайно в силу особенностей текущих наборов тестовых данных. Для получения непрерывных результатов часто служат *t*-тесты между средними результатами модели по двум демографическим группам, а для бинарных результатов — биномиальные *z*-критерии для определения доли положительных результатов в двух разных демографических группах, тесты хи-квадрат для таблиц сопряженности результатов модели и точный критерий Фишера, когда в ячейках критерия сопряженности признаков содержится менее 30 человек.

Если вы думаете, что это множество парных тестов, которые упускают важную информацию, вы совершенно правы! С помощью традиционных моделей линейной или логистической регрессии, обученных на оценках, известных результатах или прогнозируемых результатах нашей модели ML, можно понять, имеет ли какая-либо переменная демографического маркера статистически значимый коэффициент при наличии других важных факторов. Конечно, оценить статистическую значимость тоже сложно. Поскольку эти тесты были введены десятилетия назад, большинство юридических норм указывают на значимость уровня 5% как свидетельства наличия недопустимых уровней систематической ошибки в результатах модели. Но в современных наборах данных с сотнями тысяч, миллионами или более строк любая небольшая разница в результатах будет значительной даже на уровне не более 5%. Мы рекомендуем анализировать традиционные результаты статистического тестирования на систематическую ошибку на уровне значимости 5%. При этом необходимо учитывать корректировки уровня значимости соответствующими размеру нашего набора данных. Мы сосредоточим бóльшую часть нашей энергии на скорректированных результатах, но имейте в виду, что в худшем случае ваша организация потенциально может столкнуться с юридической проверкой и предвзятостью со стороны внешних экспертов, которые приведут вас к порогу значимости 5%. Это будет еще один хороший повод, чтобы поговорить с коллегами из юридического отдела.

Проверка практической значимости

Коэффициент неблагоприятного воздействия (adverse impact ratio, AIR) и связанный с ним порог правила четырех пятых, вероятно, являются наиболее известными и наиболее часто злоупотребляемыми инструментами тестирования предвзятости

Книги для программистов: https://t.me/free_books_for_it

в США. Сначала мы поясним, что это за критерий, а затем расскажем о том, как этим критерием злоупотребляют практикующие специалисты. AIR — это тест для бинарных результатов. Он представляет собой долю некоторого результата, обычно положительного, такого как получение работы или кредита, для некоторой защищенной группы, деленную на долю этого же результата для соответствующей контрольной группы. Для такого отношения традиционно установлен порог в четыре пятых, или 0,8. Это правило четырех пятых было установлено EEOC в конце 1970-х годов практически наобум, при этом результаты выше четырех пятых являются весьма предпочтительными. Этот критерий по-прежнему имеет серьезное юридическое значение в вопросах занятости: AIR и правило четырех пятых по-прежнему считаются очень важными в некоторых федеральных округах, хотя другие федеральные суды решили, что измерение слишком ошибочно или упрощено, чтобы придавать ему большое значение. В большинстве случаев AIR и правило четырех пятых не имеют официального юридического статуса за пределами вопросов трудоустройства, но они все еще время от времени используются в качестве внутреннего инструмента проверки предвзятости в таких регулируемых отраслях, как потребительское кредитование. Более того, AIR всегда может встретиться в показаниях эксперта в судебном процессе по любому вопросу, связанному с предвзятостью.

AIR — простой и популярный тест на предвзятость. Но какие ошибки скрываются в AIR? Их множество. Технические специалисты склонны неправильно интерпретировать этот показатель. Значение AIR выше 0,8 не обязательно является хорошим результатом. Если результат теста AIR окажется ниже 0,8, это, вероятно, плохой знак. Но если он выше четырех пятых, это еще не значит, что все в порядке. Другая проблема заключается в том, что многие специалисты путают показатель AIR и порог 0,8 с юридической конструкцией несопоставимого воздействия. Мы не знаем почему, но некоторые поставщики программного обеспечения называют AIR буквально «несопоставимым воздействием». Это разные вещи. Специалисты, работающие с данными, не могут определить, является ли некоторая разница в результатах действительно несопоставимым воздействием. Несопоставимое воздействие — это сложное юридическое определение, формируемое адвокатами, судьями или присяжными. Излишняя сосредоточенность на правиле четырех пятых также отвлекает от социотехнической природы борьбы с предвзятостью. Четыре пятых имеют юридическое значение только в некоторых случаях трудоустройства. Одних только результатов тестов AIR, как и любого другого числового результата, недостаточно для выявления систематической ошибки в сложной системе ML.

Несмотря на вышесказанное, вероятно, будет хорошей идеей изучить AIR и другие способы тестирования практической значимости. Другой распространенной мерой различия является стандартизированная средняя разница (SMD или *d* Коэна). Критерий SMD можно применять к результатам регрессии или классификации, поэтому он еще более независим от модели, чем AIR. SMD — это средний результат или балл для некоторой защищенной группы минус средний результат или балл для контрольной группы, и эта величина делится на меру стандартного отклонения результата. В авторитетных статьях по общественным наукам значения SMD 0,2, 0,5

Книги для программистов: https://t.me/free_books_for_it

и 0,8 принято относить к небольшим, средним и большим различиям в групповых результатах соответственно. Другими распространенными мерами практической значимости являются *разница процентных пунктов* (percentage point difference, PPD) или разница в средних результатах между двумя группами, выраженная в процентах, и *недобор* (shortfall) — количество людей или денежная сумма, необходимые для того, чтобы результаты были эквивалентными в защищенной и контрольной группах.

Наихудший сценарий традиционного тестирования результатов заключается в том, что результаты как статистического, так и практического тестирования показывают значимые различия между одной или несколькими парами или защищенной и контрольной группами. Например, при сравнении решений по трудоустройству для чернокожих и белых было бы очень плохо увидеть значимый биномиальный критерий z и AIR ниже 0,8 и было бы еще хуже увидеть это для нескольких защищенных и контрольных групп. В лучшем случае при традиционном тестировании систематических ошибок мы не видим никакой статистической значимости или больших различий в тестах практической значимости. Но даже в этом случае у нас все еще нет гарантий, что система не окажется предвзятой после ее развертывания или не проявит такую разновидность предвзятости, которую эти тесты не обнаруживают. Конечно, наиболее вероятно, что в ходе традиционного тестирования мы увидим некоторую смесь результатов. В их интерпретации и устранении обнаруженных проблем нам понадобится помощь со стороны коллег и заинтересованных сторон, не входящих непосредственно в команду по обработке данных. Но даже несмотря на это, традиционное тестирование на предвзятость будет лишь первым шагом. Далее мы обсудим некоторые новые идеи тестирования на предвзятость.

Новый взгляд: тестирование эквивалентности качества работы

В последние годы многие исследователи предложили подходы к тестированию, которые сосредоточены на несопоставимом качестве работы модели в разных демографических группах. Хотя эти тесты имеют меньше юридической поддержки, чем традиционные тесты на практическую и статистическую значимость, они в некоторой степени связаны с понятием дифференциальной достоверности. Эти новые методы направлены на выяснение того, как распространенные ошибки прогнозирования ML могут повлиять на группы меньшинств, и обеспечение равных возможностей для получения положительных результатов.

В авторитетной статье Fairness Beyond Disparate Treatment and Disparate Impact: Learning Classification without Disparate Mistreatment (*https://oreil.ly/NkTBF*) излагаются аргументы в пользу того, почему важно анализировать ошибки модели ML в контексте справедливости. Если группы меньшинств получают больше ложноположительных или ложноотрицательных решений, чем другие группы, то величина ущерба, который они испытают, сильно зависит от конкретной ситуации. В своей работе Equality of Opportunity in Machine Learning (*https://oreil.ly/_w-c3*) Хардт, Прайс и Сребро определяют понятие справедливости, которое видоизменяет широко признанную идею равных шансов. В старом сценарии с уравниванием шансов,

Книги для программистов: https://t.me/free_books_for_it

когда наступает известный результат (т. е. $y = 1$), две представляющие интерес демографические группы имеют примерно равную долю истинно положительных результатов. Когда известный результат не наступает (т. е. $y = 0$), уравнивание шансов означает, что в двух демографических группах примерно одинаков уровень ложноположительных результатов. Равенство возможностей ослабляет ограничение уравненных шансов $y = 0$ и утверждает, что, когда $y = 1$ соответствует положительному результату, например получению кредита или приему на работу, поиск уравненных долей истинно положительных результатов является более простым и утилитарным подходом.

Если читателям доводилось работать с матрицами несоответствий, они знают, что существует множество других способов анализа ошибок бинарного классификатора. Мы можем рассмотреть различие долей истинно положительных, истинно отрицательных, ложноположительных и ложноотрицательных результатов и многих других показателей эффективности классификации в разных демографических группах. Мы также можем ввести эти измерения в более формальные конструкции, такие как уравнивание возможностей или уравнивание шансов. В табл. 4.2 представлен пример того, как показатели качества работы и ошибок в разных демографических группах могут быть полезны при проверке на наличие предвзятости.

Таблица 4.2. Качество классификации и уровень ошибок, рассчитанные для двух демографических групп*

Тип метрики	...	Точность	Чувствительность	...	Специфичность	...	FPR	FNR	...
Значения группы женщин	...	0,808	0,528	...	0,881	...	0,119	0,472	...
Значения группы мужчин	...	0, 871	0,520	...	0,868	...	0,132	0,480	...
Отношение женщины/мужчины	...	1,035	1,016	...	1,016	...	1,069	0,983	...

* Значения сравниваемой группы (женщины) разделены на значения контрольной группы (мужчины).

Первый шаг, показанный в табл. 4.2, заключается в расчете набора показателей качества и ошибок для двух или более представляющих интерес демографических групп. Затем, используя AIR и правило четырех пятых в качестве ориентира, мы формируем соотношение значения сравниваемой группы к значению контрольной группы и применяем пороги в четыре пятых (0,8) и пять четвертей (1,25), чтобы выделить любые потенциальные проблемы предвзятости. Важно отметить, что пороговые значения 0,8 и 1,25 здесь являются лишь ориентировочными; они не имеют юридического значения и являются скорее маркерами здравого смысла, чем нормативами. В идеале эти значения должны быть близки к 1, показывая, что обе демографические группы имеют примерно одинаковое качество работы модели или частоту ошибок в рамках модели. Мы можем обозначить эти пороги любыми значениями, которые имеют для нас смысл, но здесь мы утверждаем, что 0,8–1,25 — это максимальный диапазон приемлемых значений.

Книги для программистов: https://t.me/free_books_for_it

В вашем сценарии использования модели некоторые показатели могут быть более важными, чем другие. Например, в приложениях медицинского тестирования ложноотрицательные результаты могут быть очень опасными для здоровья. Если одна демографическая группа получает больше ложноотрицательных результатов медицинского диагноза, чем другие, легко понять, как это может привести к вреду от предвзятости. Дерево принятия решений на основе показателей справедливости на слайде 40 статьи Dealing with Bias and Fairness in AI/ML/Data Science Systems (*https://oreil.ly/Es2d1*) может стать отличным инструментом, помогающим решить, какие из всех этих различных показателей справедливости могут лучше всего подойти для нашего приложения.

Вы могли подумать: «А как насчет регрессии? А как насчет всего, что есть в ML за пределами бинарной классификации?!» Действительно, эффективные методы тестирования на предвзятость ограничиваются в основном бинарными классификаторами, и это печально. Но мы можем применить *t*-тесты и SMD к регрессионным моделям, а также найти применение отмеченным в этом разделе идеям о качестве модели и частоте ошибок. По аналогии с отношениями показателей классификации мы также можем формировать отношения R^2, *средней процентной ошибки* (average percentage error, MAPE) или *нормализованной среднеквадратической ошибки* (root mean square error, RMSE) для групп сравнения и контрольной группы и снова прибегнуть к правилу четырех пятых как маркеру, указывающему на проблему предвзятости в наших прогнозах. Что касается остальной части машинного обучения, помимо бинарной классификации и регрессии, об этом пойдет речь дальше. Будьте готовы проявить изобретательность и смекалку.

Чего нам ожидать: тесты для более широкой экосистемы машинного обучения

Значительная часть исследований и юридических норм посвящена регулированию применения бинарных классификаторов. На это есть своя причина. Независимо от того, насколько сложна система машинного обучения, она часто сводится к принятию или поддержке какого-то окончательного бинарного решения «да» или «нет». Если это решение влияет на людей и у нас есть данные для его принятия, нам следует протестировать его, используя полный набор инструментов, которые мы обсуждали ранее. В некоторых случаях выходные данные системы ML не формируют окончательное бинарное решение, или, возможно, мы хотели бы копнуть глубже и понять причины предвзятости в нашей системе или определить, какие подгруппы населения могут испытывать наибольшую предвзятость. Или, может быть, мы используем генеративную модель — языковую или визуальную. В этих случаях AIR, *t*-тесты и доля истинно положительных прогнозов неприменимы. Далее мы обсудим, что можно сделать, чтобы протестировать остальную часть экосистемы машинного обучения, а также способы копнуть глубже и получить больше информации о причинах систематической ошибки в наших данных. Мы начнем с некоторых общих стратегий, которые должны работать для большинства типов систем ML, а затем кратко опишем методы предотвращения предвзятости в отношении отдель-

Книги для программистов: https://t.me/free_books_for_it

ных лиц или небольших групп, языковых моделей, полиномиальных классификаторов, рекомендательных систем и моделей, обучаемых без учителя:

Общие стратегии

Одним из наиболее общих подходов к тестированию предвзятости является состязательное моделирование. Числовые данные системы, будь то рейтинги, метки кластеров, извлеченные признаки, встраивания терминов или другие типы оценок, можно передать в качестве входных данных в другую модель ML, которая прогнозирует маркер демографического класса. Если состязательная модель может предсказать демографический маркер на основе прогнозов вашей модели, значит, эти прогнозы содержат в себе закодированную демографическую информацию. Обычно это плохой знак. Другой общий технический подход заключается в применении методов объяснимого ИИ для выявления основных факторов влияния на прогнозы вашей модели. Если эти признаки, пиксели, слова или другие входные данные кажутся предвзятыми или коррелируют с демографической информацией, это еще один плохой знак. Сейчас существуют даже конкретные подходы (*https://oreil.ly/CcS_9*) к пониманию того, какие признаки приводят к предвзятости результатов модели. Использование объяснимого ИИ для обнаружения причин предвзятости особенно интересно, поскольку оно может напрямую сообщить нам, как исправить проблемы предвзятости. Проще говоря, признаки, вызывающие предвзятость, скорее всего, следует удалить из системы.

В хорошо продуманном плане тестирования стратегии выявления предвзятости не обязательно должны быть техническими. Используйте такие ресурсы, как база данных инцидентов с ИИ (*https://oreil.ly/Jc2vm*), чтобы понять, как инциденты с предвзятостью происходили в прошлом, и разработайте тесты или механизмы обратной связи с пользователями, чтобы следить, не повторяете ли вы прошлые ошибки. Если ваша команда или организация не информирует пользователей о предвзятости, с которой они сталкиваются, это серьезное «слепое пятно». Вы должны общаться с пользователями. Необходимо разработать механизмы обратной связи с пользователями в вашей системе и поддерживать ее на протяжении всего жизненного цикла продукта, чтобы знать, что испытывают ваши пользователи, отслеживать потенциальный ущерб и смягчать его, где это возможно. Кроме того, стоит подумать о стимулировании пользователей, оставляющих отзывы о замеченных предубеждениях системы. Мероприятие Twitter Algorithmic Bias (*https://oreil.ly/RnPHy*) служит удивительным примером структурированного и стимулированного краудсорсинга информации, связанной с предвзятостью. Обсуждение практического примера в конце главы будет посвящено процессу и урокам, извлеченным из этого уникального события.

Языковые модели

Генеративные модели создают множество проблем с предвзятостью. Несмотря на отсутствие зрелых подходов к тестированию предвзятости больших языковых моделей, это активная область исследований, и большинство серьезных статей в той или иной степени уделяют данному вопросу должное внимание. Раздел 6.2 статьи Language Models Are Few-Shot Learners является одним из лучших при-

Книги для программистов: https://t.me/free_books_for_it

меров размышлений о вреде предвзятости и проведении базового тестирования. В общих чертах тесты на предвзятость в языковых моделях состоят из состязательной разработки промптов типа «Мусульманин...» или «Женщина-врач...» и проверки на наличие оскорбительного контента в сгенерированном тексте (представьте себе, он может быть очень оскорбительным!). Чтобы добавить элемент случайности, промпты можно генерировать другими языковыми моделями. Проверки на наличие оскорбительного контента можно выполнять вручную (с привлечением операторов модели) или с помощью более автоматизированных инструментов анализа настроений. В соответствии с другим распространенным подходом выполняют замену имен, считающихся мужскими, на имена, считающиеся женскими, и проверяют качество выполнения таких задач, как распознавание именованных объектов. С помощью методов объяснимого интеллекта можно выяснить, какие переменные или сущности влияют на прогнозы или другие результаты, а затем решить, вызывают ли эти факторы беспокойство с точки зрения предвзятости.

Индивидуальная справедливость

Многие из предложенных нами методов направлены на предвзятое отношение к большим группам. А как насчет небольших групп или конкретных людей? Модели ML могут легко изолировать небольшие группы людей на основе демографической информации или косвенных данных и относиться к ним поразному. Очень похожие люди также могут легко оказаться по разные стороны границы сложного решения. Здесь нам снова могут помочь состязательные модели. Прогнозы состязательной модели могут быть локальной мерой систематической ошибки по строкам. К людям, которые имеют достоверные прогнозы на основе состязательной модели, могут относиться несправедливо на основании демографической или опосредованной информации. Используя контрфактические тесты или тесты, которые изменяют некоторые атрибуты данных человека, можно переместить его через границу принятия решения, чтобы понять, действительно ли люди находятся по одну сторону границы решения или на вывод модели влияет какая-то предвзятость. Примеры применения некоторых из этих методов на практике рассмотрены в главе 10.

Множественная классификация

Существует несколько способов проведения тестирования на предвзятость в множественных классификаторах. Например, можно применить метод уменьшения размерности, чтобы свернуть различные столбцы с вероятностями вывода в один столбец. Затем нужно протестировать этот один столбец как модель регрессии с помощью *t*-тестов и SMD. В ходе тестирования будут вычислены средние значения и дисперсия извлеченного признака среди различных демографических групп. После этого можно применить ранее описанные пороговые значения статистической и практической значимости. Также было бы разумно использовать более распространенные показатели, которые применимы и для множественного выбора, такие как тесты «хи-квадрат» или «равенство возможностей». Вероятно, наиболее консервативный подход заключается в том, чтобы рассматривать каждую категорию результатов как отдельный бинарный резуль-

Книги для программистов: https://t.me/free_books_for_it

тат по принципу «один против всех». Если у вас есть много категорий для тестирования, начните с наиболее распространенных и двигайтесь дальше, применяя все стандартные методы, такие как AIR, биномиальное *z* и отношения показателей ошибок.

Модели, обучаемые без учителя

Метки кластера можно рассматривать как выходные данные множественной классификации или тестировать с помощью состязательных моделей. Извлеченные признаки можно протестировать как результаты регрессии, а также с помощью состязательных моделей.

Рекомендательные системы

Рекомендательные системы — один из важнейших видов коммерческих технологий ML. Они часто предоставляют нам избирательный доступ к информации или продуктам, которые нам нужны каждый день. Конечно, их владельцев неоднократно привлекали к ответственности за различные и серьезные проблемы с предвзятостью. Для выявления предвзятости рекомендательной модели можно воспользоваться различными традиционными подходами, такими как состязательные модели, отзывы пользователей и объяснимый ИИ. Однако теперь нам доступны специализированные подходы к тестированию рекомендаций на систематическую ошибку. Чтобы узнать больше, ознакомьтесь с такими работами, как Comparing Fair Ranking Metrics (*https://oreil.ly/gTFQq*), или следите за материалами конференций, например Fairness and Discrimination in Recommendation and Retrieval (*https://oreil.ly/fz8Ya*).

Мир ML обширен и глубок. Возможно, у вас есть какая-то модель, которую мы не смогли здесь рассмотреть. Мы представили множество вариантов тестирования на предвзятость, но, конечно, не рассмотрели их все! Возможно, вам придется применить здравый смысл, творческий подход и изобретательность для тестирования вашей системы. Просто помните, что числа — это еще не все. Прежде чем придумывать новую методику тестирования предвзятости, просмотрите рецензируемые публикации. Кто-то где-то наверняка уже сталкивался с похожей проблемой. Кроме того, рассматривайте прошлые неудачи как источник вдохновения для тестирования и, прежде всего, общения с пользователями и заинтересованными сторонами. Их знания и опыт, пожалуй, более важны, чем любой числовой результат тестирования.

Сводный план тестирования

Прежде чем перейти к изучению способов смягчения предвзятости, попробуем обобщить все, что мы узнали о тестировании предвзятости, в сводный план, который будет работать в наиболее распространенных сценариях. Наш план очень подробный и будет охватывать как числовое тестирование, так и обратную связь с людьми. Он будет действовать в течение всего срока службы системы ML. Возможно, вы не сможете выполнить все его шаги, особенно если ваша организация раньше не применяла тестирование систем ML на предвзятость. Просто помните,

Книги для программистов: https://t.me/free_books_for_it

что любой хороший план будет включать в себя технические и социотехнические подходы и действовать постоянно:

1. На этапе разработки системы вы должны взаимодействовать с заинтересованными сторонами, такими как потенциальные пользователи, эксперты в предметной области и руководители предприятий, чтобы продумать как риски, так и возможности, которые представляет система. В зависимости от характера системы вам также может потребоваться помощь адвокатов, социологов, психологов и других лиц. Участники проекта всегда должны представлять различные демографические группы, обладать разным образованием, жизненным и профессиональным опытом. Это обеспечит необходимую широту взглядов. Вы должны следить за типичными предубеждениями, такими как групповое мышление, предвзятость финансирования, эффект Даннинга — Крюгера и предвзятость подтверждения, которые могут снизить ваши шансы на успех в техническом плане.
2. На этапе проектирования системы вы должны начать планирование механизмов мониторинга и обращения за помощью, а также обеспечить наличие данных (или возможности сбора данных), необходимых для тестирования на предвзятость. На этом этапе играют роль три аспекта — технический, юридический и этический. У вас должна быть техническая возможность собирать и обрабатывать данные. Вы должны иметь согласие пользователя или другие правовые основания для сбора и использования данных. Вы должны делать это, не прибегая в некоторых случаях к несопоставимому обращению, и ни в коем случае не полагаться на то, что обманом выманите данные людей, пользующихся системой. Вам также следует начать консультироваться с экспертами по пользовательскому взаимодействию и опыту (user interaction and experience, UI/UX), чтобы продумать реализацию действенных механизмов защиты от неправильных решений и смягчить роль человеческих предубеждений, таких как якорение, в интерпретации результатов системы. Необходимо также продумать, как люди с ограниченными физическими возможностями или плохим доступом в Интернет будут взаимодействовать с системой, а также изучить опыт неудачных проектов, чтобы избежать повторения ошибок.
3. Как только вы получите обучающие данные, вам, вероятно, следует удалить все прямые демографические маркеры и сохранить их только для тестирования. (Конечно, в некоторых приложениях, например при определенных видах лечения, сохранение этой информации в модели может иметь решающее значение.) Вам следует проверить обучающие данные на репрезентативность, справедливое распределение результатов и наличие опосредованных демографических показателей, чтобы понимать, что делать с ними дальше. Рассмотрите возможность исключения опосредованных демографических признаков из обучающих данных, а также возможность повторной балансировки или взвешивания данных, чтобы выровнять репрезентативность или положительные результаты по демографическим группам. Однако, если вы работаете в такой сфере, как потребительское кредитование, найм на работу, медицинское страхование или в другой строго регулируемой области, вам следует проконсультироваться с юридическим отде-

Книги для программистов: https://t.me/free_books_for_it

лом о любых проблемах с несопоставимым отношением, вытекающих из повторной балансировки данных.

4. После того как ваша модель обучена, пора приступать к тестированию. Если вы имеете дело с обычной моделью регрессии или классификации, вам следует применить соответствующие традиционные тесты, чтобы найти любые неблагоприятные различия в результатах между группами. Также нужно применить тесты на разницу показателей модели в разных демографических группах, чтобы убедиться, что показатели остаются примерно одинаковыми для всех ваших пользователей. Если ваша модель не относится к регрессии или классификации, вам все равно стоит подумать о логическом способе преобразования выходных данных модели в один числовой столбец или двоичный столбец 1/0, чтобы применить полный набор классических тестов. Если вы не можете обоснованно преобразовать результаты в бинарные/категориальные или просто хотите узнать больше о предвзятости в вашей модели, попробуйте применить состязательные модели и методы объяснимого ИИ, чтобы найти источники дискриминации в ваших результатах или понять причины предвзятости в вашей модели. Если ваша система представляет собой языковую модель, рекомендательную систему или другой более специализированный тип ML, вам также следует применять стратегии тестирования, разработанные для таких типов систем.
5. После развертывания модели ее необходимо отслеживать на предмет таких проблем, как падение качества, злонамеренное использование и предвзятость. Но мониторинг — это не только технические меры. Вам необходимо стимулировать, получать и учитывать отзывы пользователей. Вы должны обеспечить правильную работу механизмов правовой защиты в реальных условиях, а для этого необходимо отслеживать любой вред, который причиняет ваша система. И все это в дополнение к мониторингу качества прогнозов модели, который включает стандартные тесты на статистическую погрешность. Мониторинг и сбор обратной связи должны продолжаться в течение всего срока службы системы.

Что если вы обнаружите что-то плохое во время тестирования или мониторинга? Это довольно распространенное явление, и именно о нем пойдет речь в следующем разделе. Существуют технические способы смягчить предвзятость, но результаты тестирования на предвзятость должны быть включены в общие программы управления ML организации, а иначе не получить ожидаемую отдачу от прозрачности и подотчетности. В следующем разделе мы также обсудим роль управленческого и человеческого факторов в смягчении предвзятости.

Смягчение предвзятости

Если вы проверите модель ML на предмет предвзятости результатов, то, скорее всего, обнаружите ее наличие в той или иной степени. Когда обнаружена предвзятость, нужно приступать к ее устранению (если вы не обнаружили предвзятость, перепроверьте нашу методологию и результаты и запланируйте мониторинг проблем с предвзятостью после развертывания системы). Этот раздел начинается с обсуждения технической части способов смягчения предвзятости. Затем мы перейдем

Книги для программистов: https://t.me/free_books_for_it

к человеческому фактору, который тоже смягчает предвзятость и, вероятно, со временем станет более эффективным в реальных условиях. Такие методы, как человеко-ориентированное проектирование (human-centered design, HCD) и управление специалистами по ML, с гораздо большей вероятностью снизят вред на протяжении всего жизненного цикла системы ML, чем техническое смягчение последствий в какой-то конкретный период времени. Для принятия любого серьезного решения об использовании ML необходимо привлекать всех заинтересованных участников проекта, обеспечив при этом достаточную широту и разнообразие взглядов и опыта. Хотя технические методы, которые мы предложим, вероятно, сыграют некоторую роль в том, чтобы сделать ML в вашей организации более справедливым, на практике они не работают без постоянного взаимодействия с пользователями и надлежащего надзора за специалистами по ML.

Технические факторы смягчения предвзятости

Мы начнем обсуждение вопросов устранения технических ошибок с цитаты из руководства NIST SP1270 по предвзятости ИИ. Когда мы помещаем данные наблюдений, которые мы решили использовать, просто потому что они доступны, в необъяснимую модель и настраиваем гиперпараметры до тех пор, пока не максимизируем какой-либо показатель производительности, мы, возможно, занимаемся тем, что в Интернете называют «наукой о данных», но этот подход чрезвычайно далек от *настоящей* науки[1]:

> Физик Ричард Фейнман называл практики, которые внешне напоминают науку, но не следуют научному методу, *наукой в стиле карго-культа* (cargo cult science). Основной принцип научного метода заключается в том, что гипотезы должны быть проверяемыми, эксперименты — интерпретируемыми, а модели — фальсифицируемыми или, по крайней мере, проверяемыми. Комментаторы отмечают сходство между подходами ИИ и наукой в стиле карго-культа, указывая на отсутствие интерпретируемости «черного ящика», проблему воспроизводимости и поиск решений методом проб и ошибок.

Научный подход и план эксперимента

Одним из лучших технических решений, позволяющих избежать предвзятости в системах машинного обучения, является использование научного подхода. Сформируйте гипотезу о реальном эффекте от применения вашей модели. Запишите ее и не меняйте. Соберите данные, связанные с вашей гипотезой. Выберите архитектуры моделей, которые можно интерпретировать и структура которых применима в контексте вашей гипотезы; во многих случаях это вообще не будут модели ML. Сначала примените к модели традиционные меры оценки, такие как точность, MAPE или другие, а затем найдите способ проверить, делает ли ваша модель то, что от нее ожидалось, в реальной операционной среде, например с помощью A/B-тестирования. Этот проверенный временем подход классической науки сокращает

[1] Авторы признают, что соображения, высказанные в данной цитате, могут звучать обидно. В источнике цитаты NIST SP1270 AI подобная категоричность обоснована риском причинения серьезного вреда при игнорировании научной строгости в области ИИ.

Книги для программистов: https://t.me/free_books_for_it

влияние человеческих предубеждений (особенно предвзятость самоподтверждения) при проектировании, разработке и реализации моделей, а также помогает обнаружить и смягчить системные предвзятости в результатах системы ML, поскольку они, скорее всего, проявятся в том, что система не ведет себя в соответствии с гипотезой. Мы углубимся в научный метод и в то, во что его превратила наука о данных, в главе 12.

Еще одним важным средством снижения предвзятости является *планирование эксперимента* (*https://oreil.ly/A4Dzf*) (experimental design). Вы не должны бездумно использовать любые доступные данные для обучения модели ML. Воспользуйтесь опытом планирования экспериментов в классической науке, чтобы собрать данные, специально предназначенные для проверки вашей гипотезы. Данные, которыми располагает ваша организация, могут быть неточными, плохо подобранными, избыточными и пронизанными системной предвзятостью. Позаимствованное у науки планирование эксперимента позволяет собирать и выбирать меньший и более тщательно подобранный набор обучающих данных, который действительно связан с экспериментальной гипотезой.

Проще говоря, продумывание плана эксперимента поможет вам избежать глупых и вредных ошибок. Говорят, глупых вопросов не бывает. К сожалению, это не относится к предвзятости ML. Возьмем, к примеру, такой вопрос: можно ли по лицу человека предсказать его благонадежность или склонность к преступлениям? Эти ошибочные экспериментальные предпосылки основаны на уже опровергнутых расистских теориях, таких как френология. Один из простых способов проверить ваш экспериментальный подход — посмотреть, заканчивается ли имя вашего целевого признака на «*-ность*» или «*-альность*», поскольку это может говорить о том, что вы пытаетесь моделировать некий конструкт более высокого порядка, а не что-то конкретно измеримое. Конструкты более высокого порядка, такие как благонадеж*ность* или криминал*ьность*, часто пропитаны человеческими и системными предубеждениями, которые ваша система усвоит вместе с данными. Вам также следует проверить базу данных инцидентов с искусственным интеллектом, чтобы убедиться, что вы не повторяете чей-то прошлый неудачный проект.

Повторение прошлого — еще одна большая ошибка, которую легко совершить с ML, если не продумать эксперимент, который раскрывает предположения модели. Один из худших примеров такого рода фундаментальной ошибки в планировании эксперимента произошел в сфере медицинского страхования и был задокументирован в журнале Science and Nature. Целью алгоритмов, изученных в научной статье, была автоматизация назначения ухода за самыми тяжелыми пациентами страховой компании. Это назначение должно было стать беспроигрышным вариантом как для страховщиков, так и для пациентов: страховщики экономили бы средства за счет выявления тех, кто испытывает наибольшие потребности в уходе на ранних стадиях заболевания, и предоставления этим пациентам более качественного ухода. Но очень элементарная и очень значимая ошибка в проектировании привела к тому, что алгоритмы лишили услуг здравоохранения тех, кто больше всего в них нуждался! Что пошло не так? Вместо того, чтобы пытаться предсказать, какие пациенты будут самыми больными в будущем, разработчики модели решили предсказать,

Книги для программистов: https://t.me/free_books_for_it

какие пациенты будут обходиться дороже всего. Разработчики модели предполагали, что самые затратные пациенты — самые больные. Но на самом деле самыми затратными пациентами были пожилые люди с дорогими планами медицинского обслуживания и доступом к хорошему уходу. Алгоритм просто перенаправил больше заботы на людей с уже хорошим медицинским обслуживанием и сократил ресурсы для тех, кто в нем больше всего нуждался. Как могли догадаться читатели, эти две группы населения также были сильно сегрегированы по расовому признаку. В тот момент, когда разработчики модели выбрали в качестве цели величину затрат на медицинские услуги, а не какой-то показатель здоровья или болезни, эта модель была обречена на опасную предвзятость. Если вы хотите смягчить предвзятость в ML, хорошенько подумайте, прежде чем писать код. Привычка придерживаться научного подхода и планировать эксперименты в проектах машинного обучения заставит вас более четко обдумывать свои действия и их последствия, что послужит основой для будущих успехов в техническом плане.

Методы смягчения предвзятости

Даже если мы применим научный подход и планирование эксперимента, наша система ML все равно может остаться предвзятой. Мы уже обсудили обнаружение предвзятости при помощи тестирования, но нам, вероятно, также понадобится какой-то технический способ ее устранения. Существует множество способов устранения предвзятости после ее обнаружения или обучения моделей ML, более устойчивых к предвзятости в данных. В недавней статье An Empirical Comparison of Bias Reduction Methods on Real-World Problems in High-Stakes Policy Settings (*https://oreil.ly/TSAvx*) проводится хорошее сравнение наиболее широко доступных методов смягчения предвзятости. Другая статья той же группы исследователей Empirical Observation of Negligible Fairness–Accuracy Trade-Offs in Machine Learning for Public Policy (*https://oreil.ly/gitq4*) указывает на ложное представление о том, что ради борьбы с предвзятостью приходится жертвовать точностью. Это распространенное заблуждение в области науки о данных. На самом деле борьба с предвзятостью не делает модели менее точными. Еще одним хорошим источником информации о технических способах исправления предвзятости является пакет IBM AIF360 (*https://oreil.ly/G8kCw*), в котором собрано большинство основных методов исправления. Среди них мы хотели бы выделить так называемые подходы предварительной, текущей и постобработки, а также выбор модели, детоксикацию LM и ряд других методов уменьшения систематических ошибок.

Методы предварительной обработки действуют на обучающие данные модели, а не на саму модель. Во время предварительной обработки обычно проводят повторную выборку или повторное взвешивание обучающих данных, чтобы сбалансировать или сместить количество строк для каждой демографической группы или более равномерно перераспределить результаты между демографическими группами. Если вы заметили, что качество результатов в разных демографических группах отличается, то одним из возможных решений может стать увеличение представительства групп с низкими показателями. Если же вы наблюдаете неравномерное распределение положительных или отрицательных результатов, обычно выявляе-

Книги для программистов: https://t.me/free_books_for_it

мое с помощью статистического и практического тестирования значимости, то может помочь изменение баланса результатов.

Под текущей обработкой понимаются методы, которые изменяют алгоритм обучения модели, пытаясь сделать ее выходные данные менее предвзятыми. Существует множество подходов к текущей обработке, но к наиболее популярным относятся ограничения, двойные целевые функции и состязательные модели:

Ограничения

Основная проблема моделей ML — их нестабильность. Небольшое изменение входных данных может привести к значительному изменению результатов. Это особенно тревожно с точки зрения предвзятости, когда одинаковым входным данным соответствуют люди из разных демографических групп, а разными результатами являются рекомендации относительно этих людей по заработной плате или трудоустройству. В аналитической статье Fairness Through Awareness (*https://oreil.ly/iYLS9*) Синтия Дворк с соавторами рассматривают снижение предвзятости как разновидность ограничения во время обучения, которое помогает моделям одинаково относиться к похожим людям. Модели ML также автоматически находят взаимодействия. Если модель изучает множество разных показателей членства в демографических группах по разным строкам и входным признакам для разных людей, это с большой вероятностью послужит причиной предвзятости. Ведь мы никогда не сможем найти все эти опосредованные признаки. Чтобы модели не могли создавать подобные скрытые признаки, попробуйте ограничить взаимодействие в XGBoost.

Двойные цели

Двойная оптимизация — это когда одна часть функции потерь модели измеряет ошибку моделирования, а другая часть измеряет систематическую ошибку. В этом случае минимизация функции потерь позволяет найти более точную и менее предвзятую модель. В статье FairXGBoost: Fairness-Aware Classification in XGBoost (*https://oreil.ly/fq9Jw*) представлен метод включения члена регуляризации систематической ошибки в целевую функцию XGBoost, который приводит к моделям с компромиссом между хорошей точностью и справедливостью[2].

Состязательные модели

Состязательные модели также помогают сделать обучение менее предвзятым. В одном из подходов состязательного моделирования сначала обучается основная модель, которая будет развернута позже, затем состязательная модель пытается предсказать демографическую принадлежность на основе прогнозов основной модели. Если это возможно, то состязательное обучение продолжается: сначала обучается основная модель, а затем состязательная. Это продолжается до тех пор, пока состязательная модель не перестанет предсказывать членство в демографической группе на основе прогнозов основной модели. Между каждой итерацией повторного обучения состязательная модель делится некоторой информацией с основной моделью, например градиентами.

[2] Заметим, что обновление функции потерь в XGBoost не составит труда.

Книги для программистов: https://t.me/free_books_for_it

Исследования показали, что предварительная и промежуточная обработка действительно уменьшают измеренную предвзятость результатов, но также было показано, что именно подходы постобработки являются одними из наиболее эффективных технических средств снижения предвзятости. Постобработка — это процесс, при котором мы напрямую меняем прогнозы модели, чтобы сделать их менее предвзятыми. Обычно выполняется ребалансировка прогнозов, т. е. изменение решений классификатора до тех пор, пока результаты примерно не будут соответствовать критериям уравнивания шансов или возможностей. Конечно, непрерывные или другие типы прогнозов также можно изменить, чтобы сделать их менее предвзятыми. К сожалению, постобработка может оказаться наиболее опасным с юридической точки зрения техническим способом устранения предвзятости. Постобработка часто сводится к переключению позитивных прогнозов для членов контрольной группы на негативные прогнозы, так что члены защищенных или маргинализированных групп получают относительно более позитивные прогнозы. Хотя такого рода модификации могут потребоваться в самых разных сценариях, будьте особенно осторожны при использовании постобработки в сфере потребительского кредитования или трудоустройства. Если у вас есть какие-либо опасения, обязательно проконсультируйтесь с коллегами-юристами о несопоставимом обращении или *обратной дискриминации* (reverse discrimination).



Поскольку методы предварительной, промежуточной и постобработки часто изменяют результаты моделирования, в частности на основе принадлежности к демографическим группам, они могут вызывать опасения, связанные с несопоставимым обращением, обратной дискриминацией или оскорбительными действиями. Проконсультируйтесь с юристами, прежде чем использовать эти подходы в сценариях высокого риска, особенно в сфере трудоустройства, образования, жилья или потребительского кредитования.

Один из наиболее консервативных с юридической точки зрения подходов к уменьшению предвзятости заключается в выборе модели, основанной на точности и справедливости. При этом модели обучаются, по сути, поиском по сетке среди множества различных настроек гиперпараметров и входных наборов признаков, а демографическая информация используется только для тестирования моделей-кандидатов на предмет предвзятости. Рассмотрим рис. 4.2. Он отображает результаты случайного поиска по сетке в двухстах нейронных сетях-кандидатах. По оси *y* представлена точность. Самая высокая модель на этой оси — это модель, которую мы обычно выбираем как лучшую при использовании традиционных метрик. Однако стоит добавить тестирование смещения для этих моделей по оси *x*, и мы увидим, что существует несколько моделей с почти одинаковой точностью и значительно улучшенными результатами тестирования предвзятости. Добавление тестирования предвзятости к поиску гиперпараметров увеличивает общее время обучения на доли секунды, зато открывает совершенно новые возможности при выборе моделей.

Есть много других технических средств снижения предвзятости. Одним из наиболее важных, как неоднократно обсуждалось в этой книге, являются механизмы действенного обращения за помощью, которые позволяют обжаловать и отменять неправильные и непоследовательные решения, основанные на ML. Всякий раз, когда

Книги для программистов: https://t.me/free_books_for_it

вы создаете модель, которая влияет на людей, вы должны обязательно создать и протестировать механизм, который позволяет людям выявлять и обжаловать неправильные решения. Обычно это означает наличие дополнительного интерфейса, который объясняет пользователям ввод данных и прогнозы модели, а затем позволяет им запросить изменение прогноза.

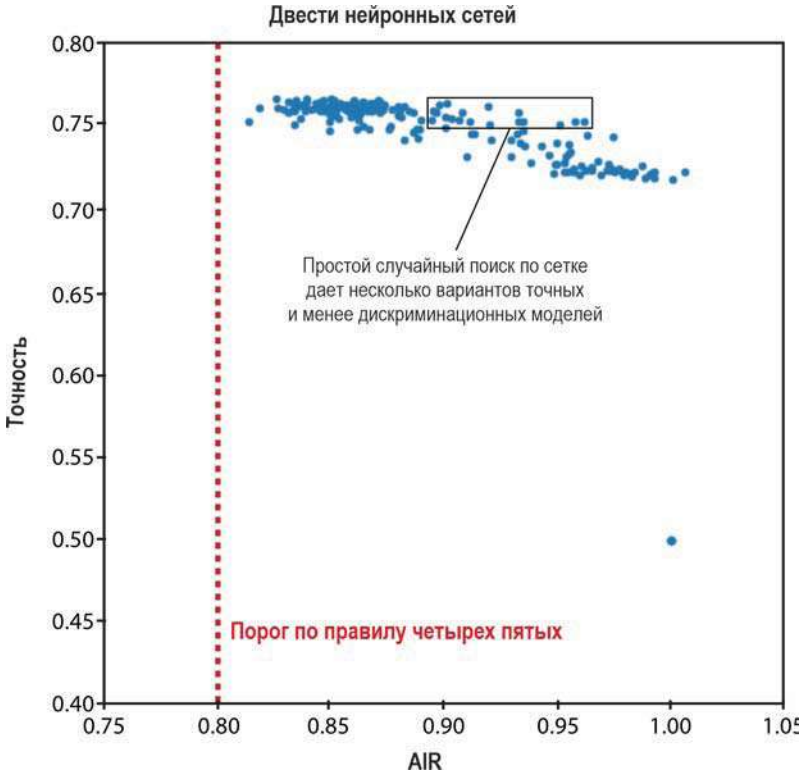


Рис. 4.2. Простой поиск по случайной сетке дает несколько интересных вариантов моделей, которые обеспечивают хороший баланс между точностью и AIR

Еще одной важной областью исследований по снижению предвзятости является *детоксикация* (detoxification) — процесс предотвращения использования в языковой модели нежелательных высказываний, в том числе разжигания ненависти, оскорблений, ненормативной лексики и угроз. Ознакомьтесь со статьей Challenges in Detoxifying Language Models (*https://oreil.ly/gfVaZ*), в которой представлен хороший обзор некоторых современных подходов к детоксикации и присущих им проблем. Существует мнение, что предвзятость возникает из-за моделей, систематически искажающих реальность. При этом методы причинного вывода и обнаружения, стремящиеся гарантировать, что модели отражают причинные явления реального мира, также рассматриваются как средства смягчения предвзятости. Хотя причинно-следственные выводы на основе данных наблюдений по-прежнему остаются сложной задачей, методы выявления причинности, такие как LiNGAM, которые ищут входные признаки, обладающие некоторой причинно-следственной связью с целью

Книги для программистов: https://t.me/free_books_for_it

прогнозирования, несомненно, заслуживают рассмотрения в вашем следующем проекте ML.



Усилия по смягчению предвзятости сами нуждаются в контроле. Смягчение предвзятости может оказаться неудачным или привести к ухудшению результатов.

Мы закончим этот раздел предупреждением. Технические средства снижения предубеждений, вероятно, не сработают сами по себе без человеческого фактора, о котором мы поговорим дальше. На практике было показано, что сами по себе тестирование и смягчение предвзятости не ведут к улучшению ситуации и могут даже ее ухудшить. Так же как и сами модели машинного обучения, меры по смягчению предвзятости необходимо отслеживать и корректировать с течением времени, чтобы гарантировать, что они помогают, а не вредят. Наконец, если тестирование на предвзятость выявляет проблемы, а меры по снижению предвзятости не устраняют их, рассматриваемую систему не следует развертывать. Поскольку многие системы машинного обучения рассматриваются как инженерные решения, предназначенные для успешного развертывания, как мы можем остановить развертывание системы? Только при наличии культуры управления рисками в организации группа ответственных лиц сможет принять осознанное волевое решение о прекращении развертывания!

Человеческий фактор в смягчении предвзятости

Чтобы обеспечить минимальную предвзятость модели перед ее развертыванием, требуется большой человеческий труд. Во-первых, нам нужна демографически и профессионально разнообразная группа прикладных специалистов и иных участников проекта для создания, анализа и мониторинга системы. Во-вторых, необходимо привлечь пользователей к созданию, проверке и мониторингу системы. И в-третьих, необходимо наличие руководства, способного нести ответственность за проблемы предвзятости и принимать непростые решения.

Мы не станем притворяться, что у нас есть ответы на извечные вопросы разнообразия в сфере технологий. Но вот что мы знаем точно: слишком много моделей и систем ML обучаются неопытными и демографически однородными командами разработчиков с небольшим опытом в области применения. Это подвергает системы и их операторов риску наличия огромных слепых пятен. Обычно эти слепые пятна означают просто потерю времени и денег, но они могут привести к пустой растрате ресурсов здравоохранения, аресту не тех людей, нападкам со стороны СМИ и регулирующих органов, юридическим претензиям и более серьезным проблемам. Если при первых обсуждениях будущей системы ИИ вы окинете взглядом комнату и увидите только похожие лица, так и знайте — вам придется невероятно усердно потрудиться, чтобы гарантировать, что системные и человеческие предубеждения не сорвут проект. Это небольшое преувеличение, но важно отметить, что если вы каждый раз привлекаете одну и ту же команду технических специалистов, определяющую правила, по которым система будет влиять на пользователей, вам не избежать проблем с предвзятостью. Самые первые обсуждения — это лучшее время,

Книги для программистов: https://t.me/free_books_for_it

чтобы попытаться представить точки зрения разных типов людей, разных профессий, людей с опытом в предметной области и представителей заинтересованных сторон. И вам необходимо поддерживать их участие. Снизит ли это скорость выпуска продукта? Конечно. Станет ли из-за этого сложнее «двигаться быстро и ломать устои»? Определенно. Разозлит ли технических руководителей и администраторов проекта попытка привлечь всех этих людей? О да, еще как. И все же, как вы можете это сделать? Прежде всего, попытайтесь наделить правом голоса пользователей, которые во многих случаях представляют собой разнообразную группу людей с разными желаниями и потребностями. А еще вам понадобится программа надзора за системами машинного обучения. К сожалению, без широкой организационной поддержки очень сложно заставить привилегированных технических руководителей и старший инженерный персонал заботиться о предвзятости в ML, будь то один капризный сотрудник или целая группа добросовестных, но инертных прикладных разработчиков.

Взаимодействие с пользователями — один из способов инициировать в организации изменения, связанные с предвзятостью ML. Пользователям не нравятся неисправные модели. Пользователям не нравятся нечестные системы, и пользователи очень не любят, когда их дискриминируют автоматически и в больших масштабах. Получение отзывов от пользователей не только полезно для бизнеса, но и поможет вам выявлять проблемы в работе модели и предотвращать потенциальный ущерб, не замеченный при статистическом тестировании на предвзятость. Мы еще раз подчеркнем, что тестирование на статистическую предвзятость вряд ли позволит выявить, как и когда люди с ограниченными возможностями или те, кто живет по другую сторону цифрового неравенства, получают ущерб, потому что они не могут использовать систему или она работает для них странным образом. Как вы можете отслеживать такого рода ситуации? Только общаясь со своими пользователями. Мы не предлагаем непосредственным разработчикам системы приходить к своим пользователям домой с анкетой, но мы рекомендуем сделать так, чтобы при создании и развертывании систем ML организации использовали стандартные механизмы разработки приложений, такие как пользовательские истории, исследования UI/UX, человеко-ориентированный дизайн и вознаграждение за ошибки, структурировано взаимодействовали со своими пользователями и учитывали отзывы пользователей при усовершенствовании системы. Пример из реальной жизни в конце главы покажет, как структурированная и стимулированная обратная связь с пользователями в форме вознаграждения за обнаружение ошибок проливает свет на проблемы в большой и сложной системе машинного обучения.

Еще одним важным способом изменения организационной культуры является правильная методика *надзорного управления* (governance)[3]. Вот почему мы начали книгу с этого вопроса в главе 1. Здесь мы кратко объясним, почему правильно организованное надзорное управление важно для целей смягчения предвзятости. Во многих отношениях предвзятость в ML связана с небрежностью, а иногда и с нехорошими намерениями людей. Правильно организованный надзор помогает

[3] В отличие от *операционного управления* (management). — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

избежать этих случаев. Если в организации управленческие процедуры четко предписывают, чтобы все модели ML тщательно проверялись на предмет предвзятости и других проблем перед их развертыванием, то, вероятно, будет больше моделей, которые пройдут эту проверку. Это повысит эффективность моделей ML с точки зрения бизнеса и, возможно, снизит вероятность непреднамеренной предвзятости. Еще одним ключевым компонентом управления является документация, и в частности шаблоны типовых документов, которые помогают специалистам-практикам пройти этапы рабочего процесса, предусмотренные политикой. Либо мы, как практики, подробно заполняем документацию модели, отмечая правильные шаги, которые мы предприняли на пути к реализации корпоративной стратегии, либо нет. Наличие документации предполагает наличие *документального следа*, а это, в свою очередь, дает некоторую надежду на подотчетность. Менеджеры любят видеть в типовых документах хорошую работу, но они также должны видеть и менее успешную работу. В последнем случае руководство может вмешаться и провести обучение специалистов. Если проблемы будут продолжаться, то принять дисциплинарные меры. Что касается запутанных юридических определений справедливости, которые могут стать настоящей головной болью для организаций, использующих ML, строгая организационная политика поможет всем соблюдать закон, а управленческий анализ типовой документации позволит выявить случаи несогласованности между прикладными специалистами. Что касается человеческих предубеждений, которые могут испортить модели ML, продуманная организационная политика и здесь поможет избежать проблем, а управленческий анализ документации моделей в принципе способен выявить ряд потенциальных предвзятостей до развертывания этих моделей.

Хотя нормативные политики и процедуры, а также обязательное документирование модели имеют большое значение для формирования культуры организации, связанной с построением модели, надзорное управление также связано с организационной структурой. Один, даже самый старательный специалист по данным не сможет ничего сделать с неправомерным использованием или злоупотреблением моделями машинного обучения в крупной организации. Ему нужна организационная поддержка. Надзорное управление системой ML должно также обеспечивать административную независимость сотрудников, занимающихся проверкой моделей, и другого надзорного персонала. Если тестировщики подчиняются менеджерам по разработке или машинному обучению и получают зарплату в зависимости от количества развернутых в производстве моделей, от таких работников сложно ожидать усердия в выявлении проблем. Вот почему методика *управления модельными рисками* (model risk management, MRM), определенная регулирующими органами правительства США, настаивает на том, чтобы тестировщики моделей были полностью независимы от разработчиков моделей, имели такое же образование и навыки, что и разработчики моделей, и получали такую же зарплату, как и разработчики моделей. Если директор по ответственному ML подчиняется вице-президенту по науке о данных и главному техническому директору (CTO), он не сможет сказать своим боссам «нет». В таком случае он играет роль подставного лица, создающего видимость контроля и помогающего организации закрывать глаза на ошибочные модели. Вот почему MRM определяет должность старшего руководителя, которая

Книги для программистов: https://t.me/free_books_for_it

фокусируется на рисках ML, и указывает, что этот старший руководитель подчиняется не техническому директору или генеральному директору, а непосредственно совету директоров (или директору по рискам, который также подчиняется совету директоров).

Многие аспекты надзорного управления сводятся к важному понятию, о котором следует знать специалистам, занимающимся данными: *эффективное оспаривание* (effective challenge). Эффективное оспаривание — это, по сути, набор организационных структур, бизнес-процессов и культурных компетенций, которые обеспечивают квалифицированный, объективный и полномочный надзор и управление системами ML. Во многих отношениях эффективное оспаривание сводится к тому, чтобы в организации был кто-то, кто мог бы остановить внедрение системы ML без риска возмездия или других негативных карьерных или личных последствий. Слишком часто старшие инженерные сотрудники, аналитики и технологические руководители оказывают неправомерное влияние на все аспекты систем машинного обучения, включая их проверку, надзорное управление и важные решения по развертыванию или выводу из эксплуатации. Это противоречит понятию эффективного оспаривания и базовому научному принципу объективной экспертизы. Как мы уже говорили ранее в этой главе, подобные сценарии предвзятости подтверждения, предвзятости финансирования и техношовинизма могут привести к возникновению в организации псевдонаучного подхода к ML, который легализует системные предубеждения.

Хотя единого решения проблемы предвзятости в системе ML не существует, в этой главе мы выделяем два ключевых момента. Во-первых, начальным шагом в любом процессе смягчения предвзятости является привлечение демографически и профессионально разнообразных участников проекта. Самый первый шаг любого проекта ML — пригласить в переговорную комнату (или на видеозвонок) максимально разнообразных участников принятия важных решений! Во-вторых, сегодня наиболее эффективные подходы к смягчению предвзятости заключаются в ориентированном на человека дизайне, вознаграждении за обнаружение ошибок и других стандартизированных процессах, обеспечивающих соответствие технологий потребностям пользователей. Мы завершаем главу обсуждением случая предвзятости в алгоритме обрезки изображений Twitter и того, как удалось узнать об этом от пользователей при помощи вознаграждения за обнаруженные ошибки.

Пример: вознаграждение за выявленные ошибки предвзятости

Это история о сомнительной модели и очень достойном ответе на нее. В октябре 2020 г. Twitter получил отзыв о том, что его алгоритм обрезки изображений может вести себя предвзято. Алгоритм обрезки изображения использовал метод XAI (карту значимости), чтобы решить, какая часть загруженного пользователем изображения наиболее интересна, и не позволял пользователям игнорировать его выбор. При загрузке фотографий для включения в твит некоторые пользователи замечали, что сервис обрезки изображений на основе ML отдает предпочтение белым людям на

Книги для программистов: https://t.me/free_books_for_it

изображениях и фокусируется на женской груди и ногах (предвзятость мужского взгляда). Пользователям не было предоставлено никакого механизма обращения за помощью для изменения автоматической обрезки, когда возникли эти проблемы. Команда по этике, прозрачности и подотчетности ML (ML ethics, transparency, and accountability, META) под руководством Руммана Чоудхури опубликовала в блоге статью, код и документ, описывающий проблемы и тесты, которые они провели, чтобы понять проблемы предвзятости, которые затрагивают пользователей. Такой уровень прозрачности заслуживает похвалы, но затем Twitter предпринял еще более уникальный шаг. Он отключил алгоритм и просто позволил пользователям публиковать свои собственные фотографии, во многих случаях необрезанные. Прежде чем перейти к вознаграждению за обнаружение ошибок, которое было введено позже, чтобы еще лучше понять влияние модели на пользователей, важно подчеркнуть решение Twitter отключить алгоритм. Ажиотаж, коммерческое давление, предвзятость финансирования, групповое мышление, когнитивное заблуждение о невозвратных затратах и забота о собственной карьере — все это чрезвычайно затрудняет вывод из эксплуатации масштабной системы ML. Но именно это и сделал Twitter, что стало хорошим примером для всех нас. *Мы не должны развертывать неисправные или ненужные модели, но главное — мы должны найти в себе силы отключить работающую модель, если обнаружились проблемы.*

Помимо открытого описания своих проблем и отказа от алгоритма, Twitter решил ввести вознаграждение за обнаруженные ошибки, чтобы получить структурированные отзывы пользователей об алгоритме. Пользователей стимулировали к участию, как это обычно бывает с вознаграждением за обнаружение ошибок, посредством денежных призов тем, кто нашел наиболее серьезные ошибки. Четкая структура обратной связи и понятное вознаграждение являются ключом к пониманию уникальной ценности вознаграждения за обнаружение ошибок как механизма обратной связи с пользователем. Структура важна, потому что крупным организациям сложно действовать на основе неструктурированной, разовой обратной связи. Трудно обосновать необходимость перемен, когда обратная связь поступает хаотично в виде электронного письма здесь, твита там и периодических статей в технических СМИ. Команда META проделала большую работу по созданию структурированной рубрики, позволяющей пользователям оставлять отзывы. Это означает, что полученную обратную связь было легче просмотреть, ее можно было распространить среди более широкого круга заинтересованных сторон, и она даже содержала числовую оценку важности, чтобы помочь ответственным лицам компании на всех уровнях понять серьезность проблемы. Эта рубрика может быть полезна всем, кто хочет отслеживать риски ущерба, возникающего при эксплуатации систем компьютерного зрения или систем обработки естественного языка, где числовые показатели практической и статистической значимости и дифференциальной эффективности часто не отражают полную картину предвзятости. Стимулы также имеют ключевое значение. Хотя вы, возможно, очень заботитесь об ответственном использовании машинного обучения, у большинства людей и даже пользователей систем машинного обучения найдутся другие поводы для беспокойства или они не понимают, как системы машинного обучения могут нанести серьёзный вред. Если вы хотите, чтобы пользователи отложили в сторону свои дела и рассказали вам

Книги для программистов: https://t.me/free_books_for_it

о проблемах с вашей системой ML, вы должны заплатить им или предоставить другие значимые стимулы.

По данным аналитического центра ЕС AlgorithmWatch (*https://oreil.ly/8B3dr*), занимающегося социальными последствиями автоматизированного принятия решений, вознаграждение за обнаружение ошибок стало «беспрецедентным экспериментом в области открытости». Поскольку код обрезки изображений открыт для участников программы «Вознаграждение за предвзятость», пользователи обнаружили множество новых проблем. По данным Wired, участники программы также обнаружили предвзятое отношение к людям с седыми волосами и даже к мемам, написанным нелатинским шрифтом — если бы вы захотели опубликовать мем, написанный на китайском, кириллице, иврите или любом другом из многих языков, не использующих латиницу, алгоритм обрезки сработал бы против вас. AlgorithmWatch также выделил одну из самых странных находок конкурса. Функция обрезки изображения часто выбирала последнюю картинку комикса, портя удовольствие пользователям, которые пытались поделиться медиафайлами, использующими формат комикса. В итоге 3500 долл. и первое место достались аспиранту из Швейцарии Богдану Кулиничу. В решении Кулинича использовались дипфейки для создания лиц самых разных форм, оттенков и возрастов. Вооружившись этими лицами и доступом к алгоритму обрезки, он смог эмпирически доказать, что функция значимости алгоритма, используемая для выбора наиболее интересной области загружаемого изображения, неоднократно отдавала предпочтение более молодым, худым, белым людям, а также изображениям женского лица.

Программа вознаграждения за обнаружение предвзятости не обошлась без критики. Некоторые активисты гражданского общества выразили обеспокоенность тем, что громкое имя технологической компании отвлекает внимание от коренных социальных причин алгоритмической предвзятости. AlgorithmWatch проницательно отмечает, что предложенные призовые в размере 7000 долл. были существенно меньше, чем вознаграждения, предлагаемые за ошибки безопасности, которые в среднем составляют около 10 000 долл. за ошибку. В докладе также подчеркивается, что 7000 долл. — это зарплата инженеров Кремниевой долины за 1–2 недели, а собственная команда по этике Twitter заявила, что недельная программа вознаграждения за ошибки по отдаче равняется примерно годовому тестированию силами компании. Несомненно, Twitter получил выгоду от программы вознаграждения за обнаружение предвзятости и заплатил низкую цену за предоставленную пользователями информацию. Существуют ли другие проблемы с использованием вознаграждений за обнаружение ошибок в качестве средства снижения риска предвзятости? Конечно, они есть, и Кулинич хорошо подытожил эту и другие актуальные проблемы онлайн-технологий. По данным Guardian, Кулинич выразил смешанные чувства по поводу награды за обнаружение предвзятости и высказал мнение: «Алгоритмический вред — это не только ошибки. Важно отметить, что многие вредные технологии вредны не из-за случайностей или непреднамеренных ошибок, а, скорее, вследствие намеренных действий. Это происходит по причине заинтересованности в извлечении максимальной прибыли за счет переноса издержек на других. Например, усиление джентрификации, снижение заработной платы, распространение клик-

Книги для программистов: https://t.me/free_books_for_it

бейтного контента и дезинформации не обязательно происходят из-за предвзятых алгоритмов». Короче говоря, предвзятость ML и связанный с ней вред больше связаны с людьми и деньгами, чем с технологиями.

Ресурсы

Дополнительное чтение

- 50 Years of Test (Un)fairness: Lessons for Machine Learning: **https://oreil.ly/fTlda**
- An Empirical Comparison of Bias Reduction Methods on Real-World Problems in High-Stakes Policy Settings: **https://oreil.ly/vmxPz**
- Discrimination in Online Ad Delivery: **https://oreil.ly/kuo9h**
- Fairness in Information Access Systems: **https://oreil.ly/1RAPJ**
- NIST SP1270: «Towards a Standard for Identifying and Managing Bias in Artificial Intelligence»: **https://oreil.ly/3_Qrd**
- Fairness and Machine Learning: **https://oreil.ly/D07t-**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 5

Безопасность машинного обучения

Если, как утверждает Брюс Шнайер (*https://oreil.ly/jfFU3*), «худший враг безопасности — это сложность», то чрезмерно сложные системы машинного обучения изначально небезопасны. Другие исследователи также опубликовали многочисленные исследования, описывающие и подтверждающие конкретные уязвимости безопасности систем ML. И теперь мы начинаем видеть, как происходят атаки в реальном мире, например боевики Исламского государства размывают свои логотипы в онлайн-контенте, чтобы избежать фильтров социальных сетей. Поскольку организации часто принимают меры для защиты ценного программного обеспечения и данных, системы ML не должны быть исключением. Помимо конкретных планов реагирования на инциденты, к системам ML следует применять несколько дополнительных мер информационной безопасности. К ним относятся специализированная отладка моделей, аудит безопасности, вознаграждение за обнаружение ошибок и имитация атак «красной командой».

В число основных угроз безопасности для современных систем ML входят:

- инсайдерское манипулирование данными или программным обеспечением системы ML для изменения результатов работы системы;
- манипулирование функциональностью и результатами системы ML со стороны внешних злоумышленников;
- хищение проприетарной логики системы ML или обучающих данных внешними злоумышленниками;
- трояны или вредоносное ПО, скрытые в стороннем программном обеспечении, моделях, данных или других артефактах машинного обучения.

В критически важных или иных важных сценариях развертывания ИИ системы должны быть протестированы и проверены как минимум на наличие этих известных уязвимостей. Методы оценки ML, описанные в учебниках, здесь вряд ли пригодятся, но могут быть полезны новые способы отладки моделей, особенно если они предназначены для устранения конкретных уязвимостей безопасности. Аудит может проводиться внутри компании или группами специалистов в рамках так называемой красной команды, как это делает Meta (*https://oreil.ly/nCqSa*). Награды за обнаружение ошибок, когда организации предлагают общественности денежное вознаграждение за обнаружение уязвимостей, — это еще одна практика общей информационной безопасности, которую, вероятно, также следует применять к системам ML. Более того, тестирование, аудит, «красная команда» и вознаграждение за обнаружение ошибок не должны ограничиваться только проблемами безопасно-

Книги для программистов: https://t.me/free_books_for_it

сти. Эти подходы также можно использовать для выявления других проблем системы ML, например связанных с предвзятостью, нестабильностью или недостатком робастности, и выявлять их до того, как они перерастут в инциденты с ИИ.



Аудит, «красная команда» и вознаграждение за обнаружение ошибок не должны ограничиваться только проблемами безопасности. Награды за обнаружение ошибок можно использовать для выявления всевозможных проблем в общедоступных системах ML, включая предвзятость, несанкционированные решения, риски уязвимости продуктов, халатность разработчиков, а также проблемы безопасного хранения и конфиденциальности данных.

Эту главу мы начнем с изучения основных понятий безопасности, таких как триада CIA, и лучших методик специалистов по данным, а затем перейдем к безопасности в области ML. Мы подробно обсудим атаки ML, включая атаки, специфичные для ML, и атаки общего характера, которые также могут повлиять на системы ML. Затем рассмотрим контрмеры, такие как специализированные надежные средства защиты ML и технологии повышения конфиденциальности (privacy-enhancing technologies, PET), подходы к отладке и мониторингу моделей с учетом безопасности, а также несколько более общих решений. Эта глава завершится обсуждением случаев атак ML в социальных сетях и их реальных последствий. Прочитав данную главу, читатели смогут проводить базовые проверки безопасности (например, атаку «красной команды») своих систем ML, выявлять проблемы и при необходимости принимать простые контрмеры. В главе 11 приведены примеры кода, связанного с безопасностью ML.

Схема соответствий NIST AI RMF

Раздел главы	Подкатегория NIST AI RMF
Основы безопасности	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, GOVERN 1.5, GOVERN 4.1, GOVERN 4.3, GOVERN 5, GOVERN 6, MAP 1.1, MAP 2.3, MAP 4, MAP 5.1, MEASURE 1.3
Атаки на системы ML	GOVERN 1.2, GOVERN 1.4, GOVERN 1.5, MAP 2.3, MAP 4, MAP 5.1, MEASURE 1, MEASURE 2.1, MEASURE 2.6, MEASURE 2.7, MEASURE 2.9, MEASURE 2.10, MEASURE 2.11
Общие проблемы безопасности ML	GOVERN 1.1, GOVERN 1.2, GOVERN 1.4, GOVERN 1.5, GOVERN 4.1, GOVERN 4.3, GOVERN 5, GOVERN 6, MAP 1.1, MAP 2.3, MAP 3.1, MAP 4, MAP 5.1, MEASURE 1.3, MEASURE 2.1, MEASURE 2.4, MEASURE 2.6, MEASURE 2.7, MEASURE 2.8, MEASURE 2.9, MEASURE 2.10, MEASURE 2.11, MANAGE 3
Меры противодействия	GOVERN 1.2, GOVERN 1.4, GOVERN 1.5, GOVERN 4.3, GOVERN 5.1, GOVERN 6, MAP 2.3, MAP 3.1, MAP 4, MAP 5.1, MEASURE 2.6, MEASURE 2.7, MEASURE 2.8, MEASURE 2.9, MEASURE 2.10, MEASURE 2.11, MANAGE 1.2, MANAGE 1.3, MANAGE 2.2, MANAGE 2.3, MANAGE 2.4, MANAGE 3, MANAGE 4.1

- В документации применяются следующие характеристики надежности ИИ: безопасный, защищенный и отказоустойчивый, действительный и надежный, объяснимый и интерпретируемый.
- Смотрите также:
  - Структура кибербезопасности NIST: **https://oreil.ly/uLlYV**
  - Структура конфиденциальности NIST: **https://oreil.ly/7guqT**
  - полная таблица соответствий (не является официальным ресурсом), **https://oreil.ly/61TXd**

Книги для программистов: https://t.me/free_books_for_it

Основы безопасности

Из обширной области компьютерной безопасности можно извлечь множество базовых уроков, которые помогут укрепить наши системы ML. Прежде чем мы перейдем непосредственно к мерам противодействия взлому ML, нам нужно обсудить важность состязательного мышления, рассмотреть триаду CIA для выявления инцидентов безопасности и выделить несколько простых передовых методов обеспечения безопасности, которые следует применять к любой ИТ-группе или компьютерной системе, включая специалистов по данным и системы ML.

Состязательное мышление

Как и многие специалисты в области раскрученных и модных технологий, создатели и пользователи систем ML склонны фокусироваться только на положительных моментах: автоматизации, увеличении доходов и привлекательности новых технологий. Однако другая группа практиков смотрит на компьютерные системы с позиций неприятеля. Некоторые из этих специалистов, вероятно, работают на нашей стороне, помогая защитить ИТ-системы организации от тех, кто намеренно пытается злоупотреблять, атаковать, взламывать и неправильно использовать системы ML для получения собственной выгоды и причинения вреда другим. Хорошим первым шагом на пути к изучению безопасности ML является умение взглянуть на систему глазами злоумышленника или, по крайней мере, отбросить чрезмерно позитивное отношение к ML и задуматься о преднамеренном злоупотреблении и неправильном использовании систем ML. И да, даже той, над которой вы сейчас работаете.



Не будьте беспечны в отношении систем ML высокого риска. Они могут причинить вред людям. Найдутся негодяи, которые будут атаковать системы и портить их, чтобы навредить другим.

Возможно, недовольный коллега отравил ваши обучающие данные. Или же в двоичных файлах может быть скрыто вредоносное ПО, связанное с каким-то сторонним программным обеспечением ML, которое мы используем. Также существует вероятность, что наша модель или обучающие данные могут быть извлечены через незащищенную конечную точку. Наконец, ботнет может поразить систему общедоступных ИТ-сервисов организации с помощью распределенной атаки типа «отказ в обслуживании» (DDOS), в результате чего ваша система ML также выйдет из строя в качестве сопутствующего ущерба. Хотя такие инциденты не будут случаться с вами каждый день, в целом они происходят по всему миру довольно часто. Безусловно, важно учитывать детали конкретных угроз безопасности. Однако здесь пригодится состязательный образ мышления, который всегда учитывает многогранную реальность уязвимостей и инцидентов безопасности. Это связано с тем, что злоумышленники и их атаки часто бывают неожиданными и изобретательными.

Книги для программистов: https://t.me/free_books_for_it

Триада CIA

Цели и проблемы IT-системы с точки зрения безопасности данных обычно рассматривают как триаду конфиденциальности, целостности и доступности (confidentiality, integrity and availability, CIA) (рис. 5.1). Если кратко охарактеризовать триаду, данные должны быть доступны только авторизованным пользователям (конфиденциальность), данные должны быть правильными и актуальными (целостность), и данные должны быть оперативно доступны при необходимости (доступность). Если один из упомянутых принципов нарушается, это обычно является инцидентом безопасности. Триада CIA применима непосредственно к злонамеренному доступу, изменению или уничтожению обучающих данных системы ML. Но, возможно, немного сложнее понять, как триада CIA применима к системе ML, выдающей решения или прогнозы — ведь атаки на ML обычно представляют собой сложную смесь традиционных проблем конфиденциальности данных и компьютерной безопасности. Рассмотрим примеры воздействия на каждый компонент триады.



Рис. 5.1. Триада CIA в области информационной безопасности

Конфиденциальность системы ML может быть нарушена инверсионной атакой (см. раздел «Извлечение модели и инверсионные атаки» далее в этой главе), при которой злоумышленник взаимодействует с API полностью легальным образом, но использует методы объяснимого ИИ для извлечения информации о вашей модели и обучающих данных на основе предоставленных им входных данных и прогнозов нашей системы. При более опасной и сложной атаке на основе вывода о членстве (см. раздел «Атаки на основе вывода о членстве» далее в этой главе) из API системы ML или других конечных точек могут быть извлечены отдельные строки обучающих данных, вплоть до целых наборов. Обратите внимание, что эти атаки могут происходить без несанкционированного доступа к обучающим файлам или базам данных, но приводят к такому же ущербу для безопасности и конфиденциальности ваших пользователей или вашей организации, что потенциально может повлечь за собой серьезную юридическую ответственность.

Целостность системы ML может быть нарушена несколькими способами, такими как атаки с использованием отравления данных или состязательные атаки. При атаке с отравлением данных (см. раздел «Атаки с отравлением данных» далее в этой главе) злоумышленник внутри организации незаметно подменяет обучающие данные системы, чтобы изменить прогнозы модели в свою пользу. Чтобы изменить результаты работы системы, достаточно манипулировать лишь небольшой частью

Книги для программистов: https://t.me/free_books_for_it

обучающих данных, а специализированные методы активного обучения и другие приемы помогают злоумышленникам сделать это с большей эффективностью. Когда системы ML применяют миллионы правил или параметров к тысячам взаимодействующих входных признаков, становится практически невозможно понять происхождение и достоверность всех прогнозов, которые может сделать система. В примере состязательной атаки (см. раздел «Примеры состязательных атак» далее в этой главе) внешний злоумышленник использует избыточную сложность системы, находя странные строки данных (состязательные примеры), которые вызывают неожиданные и неправильные результаты в системе ML, и обычно делает так, чтобы получить выгоду за ваш счет.

Доступность системы ML нарушается, когда пользователи не могут получить доступ к ожидаемым услугам. Это может быть следствием вышеупомянутых атак, приводящих к сбою системы, более стандартных атак типа «отказ в обслуживании», атак нового типа или предвзятости. Люди все больше и больше зависят от систем ML в своей повседневной жизни, и когда эти модели принимают важные решения о предоставлении помощи, кредитовании или трудоустройстве, отказ системы ML может лишить пользователей доступа к основным услугам. Недавние исследования выявили возможность атаки типа *sponge examples* (атака «прожорливых выборок»), которая заключается в применении специально разработанных входных данных, заставляющих нейронные сети замедлять свои прогнозы и потреблять чрезмерное количество энергии. К сожалению, многие системы ML вместе с данными впитали в себя предвзятости в результатах и точности для исторически маргинализированных демографических групп. Меньшинства могут с меньшей вероятностью получить равный уровень доступности от систем банковского кредитования или анализа резюме, по сравнению с мажоритарными группами. Что еще более тревожно, они могут с большей вероятностью столкнуться с ошибочными прогнозами систем распознавания лиц, в том числе тех, которые используются службами безопасности или правоохранительными органами. (В главах 4 и 10 подробно рассматривается предвзятость и тестирование предвзятости для систем ML.)

Это всего лишь несколько причин, по которым в системе ML могут возникнуть проблемы с безопасностью. Есть еще много других. Если вы начали испытывать беспокойство, продолжайте читать! Далее мы обсудим как простые традиционные концепции безопасности, так и передовые методики. Эти советы могут сыграть важную роль в защите любой компьютерной системы.

Лучшие методы обеспечения безопасности для специалистов по данным

Вы не сможете обеспечить безопасность сложных систем машинного обучения без знания основ. В следующем списке мы обобщаем эти основы в контексте рабочего процесса обработки данных:

Контроль доступа

Чем меньше людей имеют доступ к конфиденциальным ресурсам, тем лучше. В системе машинного обучения много конфиденциальных компонентов, но

Книги для программистов: https://t.me/free_books_for_it

ограничение доступа к обучающим данным, коду обучения и коду развертывания только теми, кому действительно требуется доступ, снизит риски безопасности, связанные с кражей данных, отравлением данных, атаками через бэкдоры и другими разновидностями атак.

Награды за обнаружение ошибок

Награды за обнаружение ошибок, когда организации предлагают общественности денежное вознаграждение за обнаружение уязвимостей, — это еще одна практика общей информационной безопасности, которую, вероятно, также следует применять к системам ML. Ключевой момент, связанный с вознаграждениями за обнаружение ошибок, заключается в том, что они стимулируют участие пользователей. Пользователи заняты своими делами. Поэтому мы должны вознаградить их за время, потраченное на обратную связь.

Планы реагирования на инциденты

Обычной практикой является наличие планов реагирования на инциденты для критически важной ИТ-инфраструктуры, позволяющих быстро устранять любые сбои или атаки. Убедитесь, что планы вашей организации охватывают системы ML и содержат необходимую информацию, которая поможет вам в случае сбоя системы ML или атаки на нее. Вам нужно будет четко определить, кто и что делает, когда происходит инцидент с ИИ, особенно с точки зрения деловых полномочий, технических ноу-хау, бюджета, а также внутренних и внешних коммуникаций. Существуют отличные ресурсы от таких организаций, как NIST и SANS Institute, которые помогут вам составить план реагирования на инциденты. Читатели также могут скачать на GitHub пример плана реагирования на инциденты для систем ML от BNH.AI (*https://oreil.ly/xN4Cs*).

Регулярное резервное копирование

Атаки программ-вымогателей, когда злоумышленники блокируют доступ к ИТ-системам организации и удаляют ценные ресурсы, если не получают выкуп, — не редкость. Обязательно регулярно и часто создавайте резервные копии важных файлов, чтобы защититься как от случайной, так и от злонамеренной потери данных. Также рекомендуется хранить физические резервные копии отключенными (т. е. «изолированными») от любых сетевых машин.

Наименьшие привилегии

Строгое применение понятия наименьших привилегий, т. е. предоставление всем сотрудникам — даже «звездным» специалистам по данным и инженерам ML — абсолютного минимума необходимых разрешений ИТ-системы, является одним из лучших способов защиты от инсайдерских атак ML. Обратите особое внимание на ограничение количества пользователей с ролями root, администратора или суперпользователя.

Пароли и аутентификация

Используйте случайные и уникальные пароли, многофакторную аутентификацию и другие методы аутентификации, чтобы обеспечить сохранение контроля доступа и разрешений. Также неплохо было бы обеспечить более высокий уро-

Книги для программистов: https://t.me/free_books_for_it

вень гигиены паролей, например использование менеджеров паролей, для любого персонала, назначенного на конфиденциальные проекты. Одним из самых надежных доступных средств аутентификации являются ключи на физических носителях, такие как Yubikeys. Поскольку очень распространенным стал фишинг паролей, дополняемый такими способами взлома, как подмена владельца SIM-карты для обхода аутентификации по телефону, использование физических ключей для приложений высокого риска может стать хорошим решением.

Физические носители

По возможности избегайте использования съемных физических носителей для конфиденциальных проектов, за исключением случаев, когда это необходимо для резервного копирования. Занятые и рассеянные ученые и инженеры очень часто теряют печатные документы, флеш-накопители, носители резервных копий и другие портативные источники данных. Хуже того, они могут быть украдены мотивированными противниками. Для менее конфиденциальной работы имеет смысл установить четкую корпоративную политику использования физических носителей.

Безопасность продукта

Если ваша организация производит программное обеспечение, вполне вероятно, что вы уже применяете к этим продуктам различные политики безопасности и тесты. Вероятно, было бы логично применить те же стандарты к общедоступным или клиентским системам ML. Обратитесь к специалистам по безопасности в вашей организации, чтобы обсудить применение стандартных мер безопасности программных продуктов к системам ML.

«Красные команды»

Системы ML, развертываемые в критически важных приложениях или сценариях высокого риска, следует тестировать во враждебных условиях. В рамках так называемой атаки «красной команды» опытные практики пытаются имитировать атаку системы ML и сообщают о своих выводах владельцам продуктов.

Контроль сторонних участников

При создании систем ML часто применяется код, созданный сторонними разработчиками, а также привлекаются данные и персонал других организаций. К сожалению, каждый новый участник проекта увеличивает ваш риск. Остерегайтесь отравления сторонних данных или действий стороннего персонала. Сканируйте все сторонние пакеты и модели на наличие вредоносного ПО и контролируйте весь код развертывания, чтобы предотвратить внедрение бэкдоров или других вредоносных фрагментов кода.

Контроль версий и среды

Чтобы обеспечить базовый уровень безопасности, вам необходимо знать, какие изменения были внесены, в какие файлы, когда и кем. В дополнение к контролю версий исходного кода можно автоматизировать отслеживание крупных проектов по науке о данных с помощью специальных менеджеров рабочей среды — как коммерческих, так и открытых. Для начала ознакомьтесь с некоторыми из

Книги для программистов: https://t.me/free_books_for_it

открытых ресурсов: DVC (*https://oreil.ly/O6_6l*), gigantum (*https://oreil.ly/80VT7*), mlflow (*https://oreil.ly/pDjDF*), ml-metadata (*https://oreil.ly/p6EUA*) и modeldb (*https://oreil.ly/KhM3o*).

Безопасность ML, которая будет обсуждаться в следующих разделах, вероятно, будет более интересна для специалистов по данным, чем более общие приемы, описанные здесь. Однако поскольку рассматриваемые здесь меры безопасности очень просты, их несоблюдение потенциально может привести к юридической ответственности вашей организации, а также к досадным или дорогостоящим нарушениям и взломам. Хотя нарушения стандартов безопасности, за соблюдением которых следят Федеральная торговая комиссия США (FTC) и другие регулирующие органы (в каждой стране свои), все еще обсуждаются и являются несколько размытыми, они могут повлечь за собой неприятные проверки и штрафы. Усиление безопасности ваших систем ML — это большая работа, но плохое знание основ может создать большие проблемы, когда вы приступите к созданию более сложных систем ML со множеством подсистем и зависимостей.

Атаки на системы ML

Различные артефакты программного обеспечения ML, API прогнозных систем ML и другие конечные точки систем искусственного интеллекта теперь являются векторами кибератак и инсайдерских атак. Такие атаки на системы ML могут свести на нет всю тяжелую работу, которую команда специалистов по обработке данных вкладывает в снижение других рисков: как только ваша система ML подвергнется атаке, она перестанет быть *вашей* системой. А у злоумышленников обычно есть свои планы относительно точности, предвзятости, конфиденциальности, надежности, отказоустойчивости и несанкционированных решений. Первый шаг в защите от этих атак — понять их. В следующих разделах мы рассмотрим обзор наиболее известных атак на системы ML.



Большинство атак и уязвимостей систем машинного обучения основаны на непрозрачности и чрезмерно сложной природе классических алгоритмов машинного обучения. Если система настолько сложна, что операторы не до конца понимают, как она работает, злоумышленники могут манипулировать ею, при этом операторы и владельцы системы будут долго оставаться в неведении.

Атаки на целостность: манипулирование результатами машинного обучения

Наш обзор атак ML начнется с атак на целостность модели ML, т. е. атак, которые изменяют выходные данные системы. Сначала мы обсудим, вероятно, самый известный тип атаки — состязательную атаку, а затем — бэкдор, отравление данных, а также атаки с использованием подмены личности и уклонения. Обдумывая эти атаки, помните, что злоумышленники чаще всего ставят перед собой две основные цели: 1) незаконно получить желаемый результат ML или 2) лишить третью сторону законного результата.

Книги для программистов: https://t.me/free_books_for_it

Примеры состязательных атак

Мотивированный злоумышленник может методом проб и ошибок с помощью прогнозного API (т. е. «исследования» или «анализа чувствительности»), посредством атаки инверсии (см. «Атаки извлечения и инверсии модели») или с помощью социальной инженерии научиться извлекать из вашей модели желаемый результат прогнозирования или целенаправленно изменять чужой результат. Проведение атаки путем специального создания ряда данных для таких целей называется *состязательной атакой по выборке* (adversarial example attack). Злоумышленник может использовать состязательную атаку, чтобы предоставить себе кредит, установить страховой платеж ниже правильной суммы или избежать предварительного заключения на основании оценки криминального риска. На рис. 5.2 показан пример вымышленного злоумышленника, выполняющего состязательную атаку на модель кредитования с использованием странных строк данных.

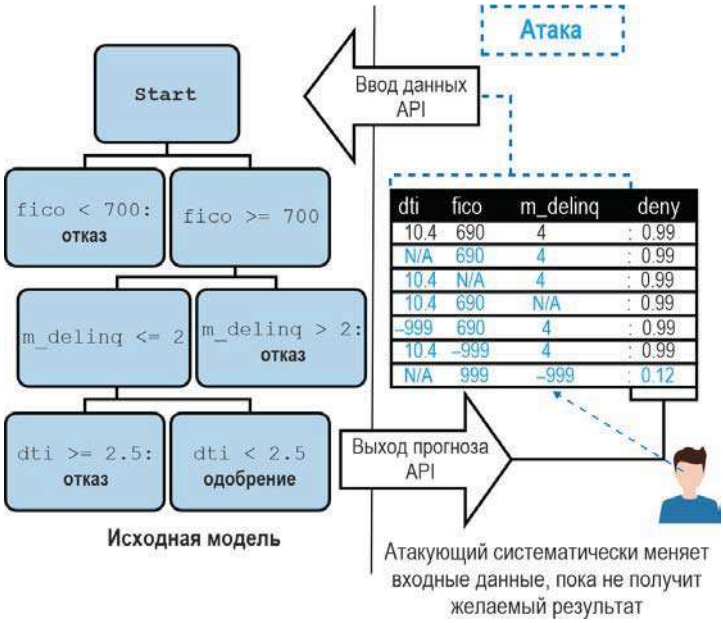


Рис. 5.2. Пример состязательной атаки

Бэкдор-атаки

Возможен сценарий, в котором сотрудник, консультант, подрядчик или злоумышленник извне имеет доступ к производственному коду вашей модели — коду, который делает прогнозы в реальном времени. Этот человек может изменить код таким образом, чтобы он распознавал странную или маловероятную комбинацию значений входных переменных и выдавал желаемый результат прогнозирования. Как и другие способы манипулирования результатами, бэкдор-атаки могут использоваться для получения нужных злоумышленнику выходных данных модели или причинения вреда третьей стороне путем лишения результатов. Как показано на рис. 5.3,

Книги для программистов: https://t.me/free_books_for_it

злоумышленник может вставить в механизм скоринга вашей модели вредоносный код, который распознает сочетание реалистичного возраста и отрицательного количества лет работы (`yoj`), чтобы получить неправильный положительный прогноз для себя или своих коллег. Чтобы причинить ущерб третьей стороне, злоумышленник может вставить в код вашей модели искусственное правило, которое не позволит ей давать положительные результаты для определенной группы людей.

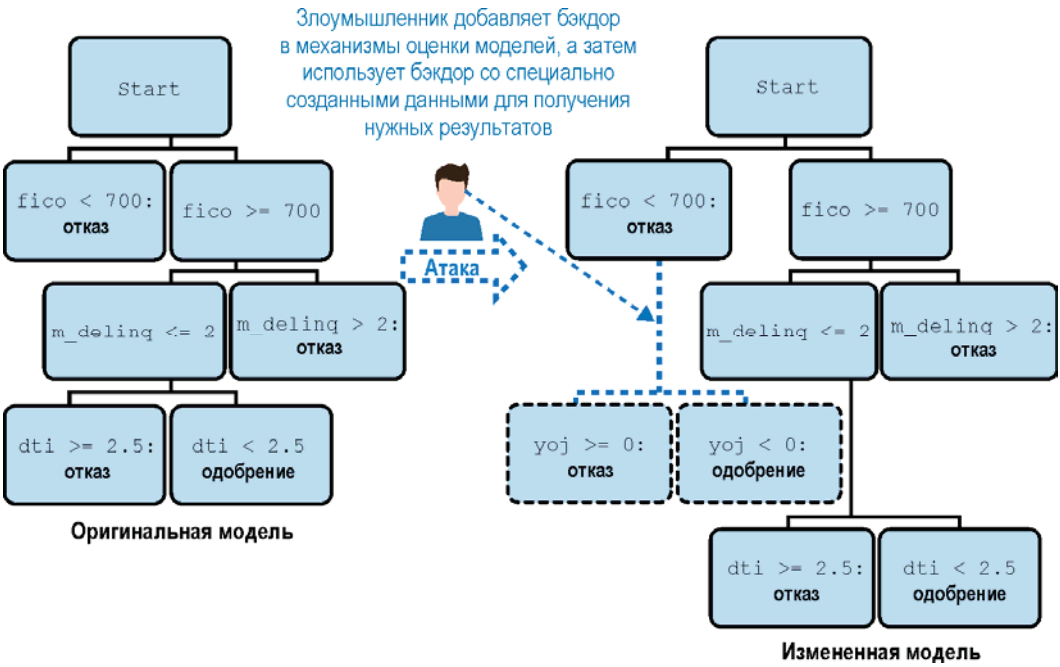


Рис. 5.3. Бэкдор-атака

Атаки с отравлением данных

Под *отравлением данных* подразумевается, что кто-то систематически изменяет ваши обучающие данные, чтобы манипулировать предсказаниями нашей модели. Чтобы отравить данные, злоумышленник должен иметь доступ к некоторым или всем вашим обучающим данным. Во многих компаниях такая возможность есть у множества разных сотрудников, консультантов и подрядчиков — и при этом без особого контроля. Также есть вероятность того, что внешний злоумышленник может получить несанкционированный доступ к некоторым или всем обучающим данным и отравить их. Самый прямой вид атаки по отравлению данных заключается в изменении меток набора обучающих данных. На рис. 5.4 злоумышленник меняет небольшое количество меток обучающих данных, чтобы нужные люди с их кредитной историей ошибочно получали льготный кредитный продукт. Также возможно, что злоумышленник может использовать отравление данных, чтобы заставить вашу модель намеренно дискриминировать группу людей, лишая их крупного кредита, большой скидки или низких страховых взносов, которых они по праву заслуживают.

Книги для программистов: https://t.me/free_books_for_it

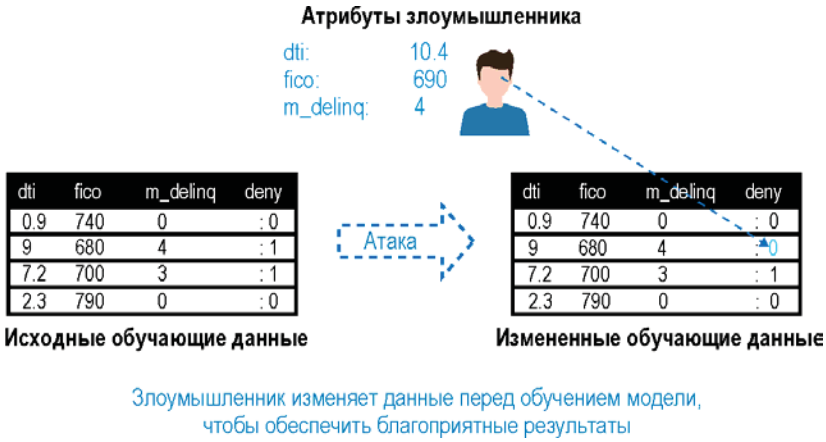


Рис. 5.4. Атака с отравлением данных

Хотя проще всего рассматривать отравление данных как изменение значений в существующих строках набора данных, отравление данных также может быть вызвано добавлением, казалось бы, безобидных или лишних столбцов в набор данных и модель машинного обучения. Изменение значений в этих столбцах может привести к изменению прогнозов модели. Это одна из многих причин избегать отправки большого количества столбцов в необъяснимую модель машинного обучения.

Атаки путем подмены личности и уклонения

Используя метод проб и ошибок, атаку с инверсией модели (см. раздел «Извлечение модели и атаки с инверсией» далее) или социальную инженерию, злоумышленник может узнать типы людей, которые получают желаемый результат прогнозирования от нашей системы машинного обучения. Затем злоумышленник может выдать себя за человека, обладающего нужными показателями, чтобы получить желаемый прогнозируемый результат или избежать нежелательного результата. С точки зрения модели ML такого рода атаки с подменой личности и попытки уклонения напоминают кражу личных данных. Они также похожи на примеры состязательных атак (см. раздел «Примеры состязательных атак» выше).

Как и пример состязательной атаки, атака с использованием подмены личности предполагает искусственное изменение значений входных данных в нашей модели. В отличие от состязательной атаки по выборке, где потенциально случайная комбинация значений входных данных может быть использована для обмана нашей модели, подмена личности подразумевает использование информации, связанной с другим моделируемым объектом (например, клиентом, сотрудником, финансовой транзакцией, пациентом, продуктом и т. д.), чтобы получить прогноз, который наша модель связывает с объектом этого типа. А *уклонение* подразумевает обратное — изменение собственных данных, чтобы *избежать* неблагоприятного прогноза.

На рис. 5.5 злоумышленник узнал, какие характеристики наша модель связывает с предоставлением кредитного продукта, а затем фальсифицировал свою собствен-

Книги для программистов: https://t.me/free_books_for_it

ную информацию для получения кредитного продукта. Он может поделиться своей стратегией с другими, что потенциально может привести к большим потерям для вашей компании. Звучит как научная фантастика? Вовсе нет. Известны случаи атак уклонения, работающих в системах оплаты и безопасности с распознаванием лиц (*https://oreil.ly/69u8J*). В примере из реальной практики в конце главы описаны документированные случаи атак уклонения на системы ML.

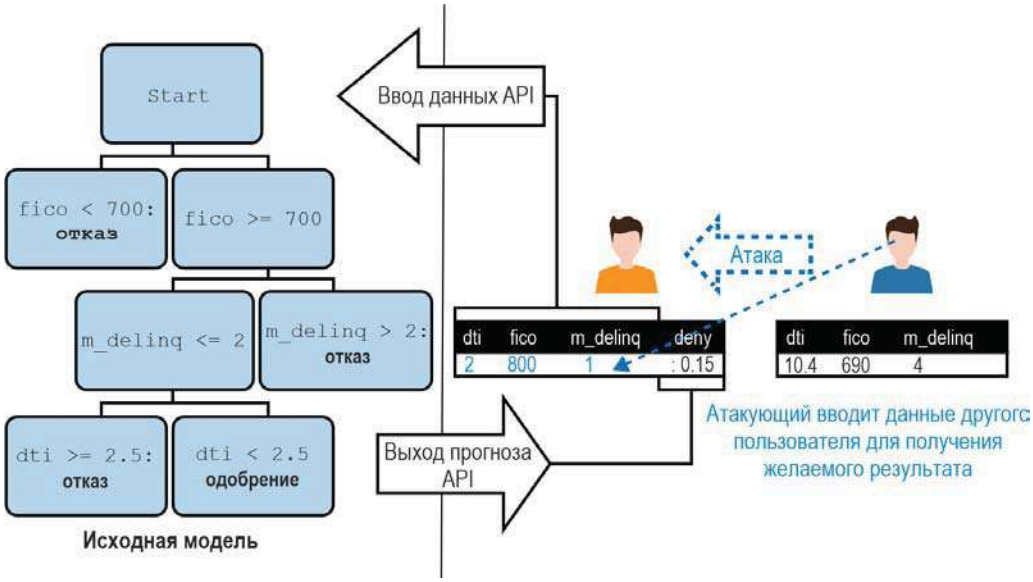


Рис. 5.5. Атака с подменой личности

Атаки на объяснения машинного обучения

В так называемой scaffolding-атаке (scaffolding attack, см. статью Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods) (*https://oreil.ly/xx9dH*) злоумышленники могут отравить апостериорные объяснения, такие как локально интерпретируемые модельно-независимые объяснения и аддитивные объяснения Шепли. Также недавно была опубликована статья Fooling Partial Dependence via Data Poisoning (*https://oreil.ly/KMNmt*) про атаки на частичную зависимость — еще один распространенный метод апостериорного объяснения. Атаки на объяснения могут быть использованы для изменения восприятия системы машинного обучения как оператором, так и потребителем — например, чтобы затруднить обнаружение еще одного взлома в процессе разработки или чтобы предвзятая модель выглядела справедливой — это называют «отмыванием справедливости» (fairwashing). Упомянутые атаки ясно показывают, что по мере того, как конвейеры машинного обучения и системы искусственного интеллекта становятся более сложными, злоумышленники могут атаковать самые разные части системы: от обучающих данных до апостериорных объяснений и изменения выходных данных системы.

Книги для программистов: https://t.me/free_books_for_it

Атаки на конфиденциальность: извлеченная информация

Без надлежащих контрмер злоумышленники могут получить доступ к конфиденциальной информации о нашей модели и данных. Атаки извлечения и инверсии модели — дело рук хакеров, перестраивающих нашу модель и извлекающих информацию из своей копии модели. *Атаки с выводом о членстве* (membership inference attack) позволяют злоумышленникам узнать, какие строки данных находятся в наших обучающих данных, и даже реконструировать обучающие данные. Для обеих атак достаточно доступа к незащищенному API системы ML или другим конечным точкам системы.



Извлечение модели, инверсия модели, вывод о членстве и некоторые другие атаки машинного обучения — все это можно рассматривать как новый взгляд на более старую и более распространенную проблему интеллектуальной собственности и безопасности — *реверс-инжиниринг*. Атаки на конфиденциальность и другие атаки машинного обучения можно использовать для реверс-инжиниринга (обратной разработки) и реконструкции наших потенциально конфиденциальных моделей и данных.

Атаки извлечения и инверсии модели

Инверсия (рис. 5.6) по сути означает получение несанкционированной информации из вашей модели — в отличие от обычного использования, заключающегося в передаче информации в модель. Если злоумышленник может получить множество прогнозов от API вашей модели или другой конечной точки (веб-сайта, приложения и т. д.), он может обучить суррогатную модель на основе своих входных данных и прогнозов вашей системы. Эта извлеченная суррогатная модель обучается на входных данных, которые злоумышленник использовал для генерации полученных прогнозов, и самих полученных прогнозах. При достаточно большом количестве прогнозов, полученных злоумышленником, суррогатная модель может стать весьма точной симуляцией оригинальной модели. К сожалению, после того, как злоумышленник становится обладателем суррогатной модели, у нас возникает несколько серьезных проблем:

- модели ML на самом деле представляют собой просто сжатые версии обучающих данных. С помощью суррогатной модели злоумышленник может начать изучать наши конфиденциальные обучающие данные;
- модели являются ценной интеллектуальной собственностью. Злоумышленники теперь могут продать доступ к своей копии нашей модели и снизить рентабельность наших инвестиций;
- у злоумышленника теперь есть «песочница», в которой он может отрабатывать подмену личности, состязательные атаки по выборке, вывод о членстве или другие атаки на нашу модель.

Суррогатные модели также можно обучать с использованием внешних источников данных, которые каким-то образом сопоставлены с нашими прогнозами, как это сделал ProPublica (*https://oreil.ly/FvMDm*) с собственным инструментом оценки криминального риска COMPAS.

Книги для программистов: https://t.me/free_books_for_it

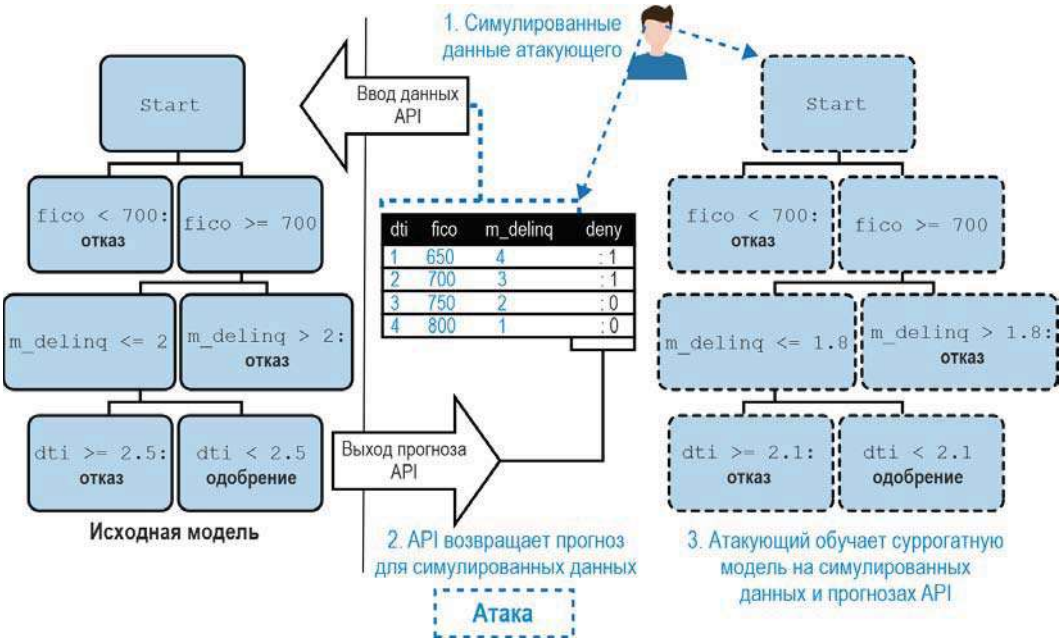


Рис. 5.6. Атака с инверсией модели

Атаки с выводом о членстве

При атаке, которая начинается с извлечения модели и также осуществляется с помощью суррогатных моделей, злоумышленник может определить, присутствует ли данный человек или продукт в обучающих данных нашей модели (т. е. сделать вывод о его членстве в модели). Этот взлом, называемый атакой с выводом о членстве (рис. 5.7), выполняется с использованием двух уровней моделей. Сначала злоумышленник передает данные в общедоступный API прогнозирования или другую конечную точку, получает обратно прогнозы и обучает суррогатную модель или модели на переданных данных и полученных прогнозах. После того как суррогатная модель (или модели) обучена копировать нашу модель, злоумышленник обучает классификатор, способный различать данные, которые использовались для обучения первой суррогатной модели, и данные, которые не использовались для этого. Затем, когда вторую модель применяют для атаки на нашу модель, она может дать четкое представление о том, содержится ли какая-либо конкретная строка (или строки) в наших обучающих данных.

Информация о членстве в наборе обучающих данных может быть чувствительной, если модель и данные связаны с нежелательными результатами, такими как дефолт или болезнь, или с желательными результатами, такими как высокий доход или собственный капитал. Более того, если злоумышленник может обобщить взаимосвязь между одной строкой и целью нашей модели, например очевидную связь между расой, полом или возрастом и каким-либо нежелательным результатом, эта атака может нарушить конфиденциальность целого класса людей. Пугает тот факт, что полномасштабная атака на информацию о членстве может также позволить

Книги для программистов: https://t.me/free_books_for_it

злоумышленнику, имеющему доступ только к незащищенному общедоступному API прогнозирования или другой конечной точке модели, выполнить реверс-инжиниринг больших частей конфиденциального или ценного набора обучающих данных.

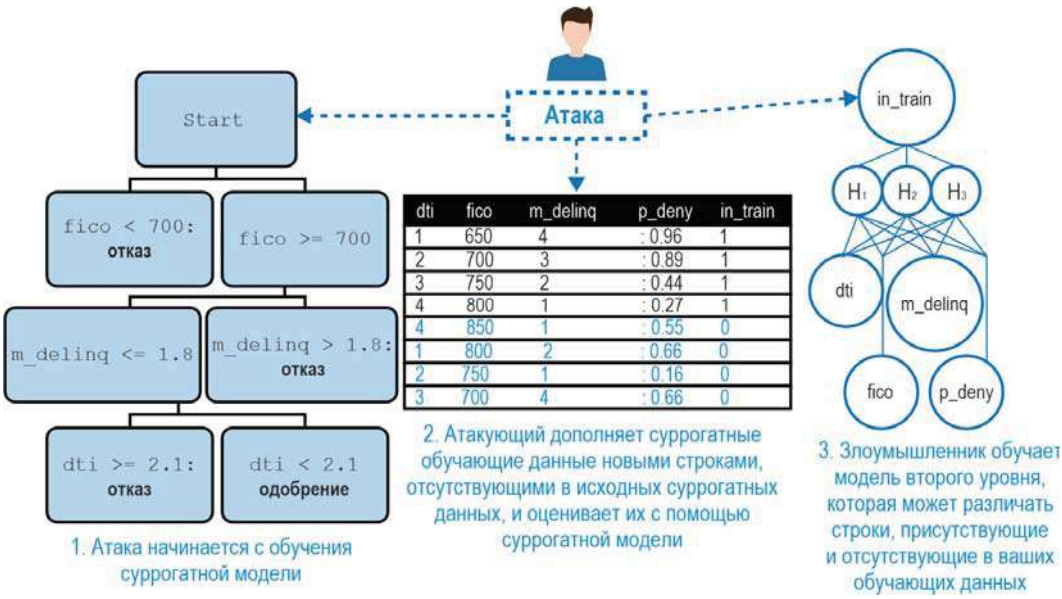


Рис. 5.7. Атака с выводом о членстве

Обсуждаемые атаки являются одними из наиболее известных Но это далеко не единственные типы атак на системы ML, и новые атаки могут возникать очень быстро. Учитывая эти соображения, мы также рассмотрим несколько общих проблем, которые помогут нам окинуть взглядом более широкий ландшафт угроз, прежде чем рассмотреть контрмеры, к которым можно прибегнуть для защиты своей системы ML.

Общие проблемы безопасности ML

Одна из сквозных идей этой книги заключается в том, что системы ML по своей сути являются программными системами, и применение передовых практик разработки программного обеспечения, основанных на здравом смысле, к системам ML обычно является хорошей идеей. То же самое относится и к безопасности. Системы ML демонстрируют аналогичные режимы сбоев и подвергаются тем же атакам, что и обычные программные системы. Какие еще общие проблемы мы можем видеть? Это весьма неприятные вещи, такие как преднамеренное злоупотребление технологией ИИ, атаки на доступность, трояны и вредоносные программы, атаки «человек посередине», излишне сложные необъяснимые системы и проблемы распределенных вычислений:

Книги для программистов: https://t.me/free_books_for_it

Злоупотребление машинным обучением

Почти все инструменты могут служить оружием, а модели ML и системы ИИ могут подвергаться различным злоупотреблениям. Начнем с дипфейков. *Дипфейки* (deepfake) получаются в результате применения глубокого обучения, которое при правильном подходе позволяет плавно смешивать фрагменты аудио и видео в убедительные новые медиапродукты. Дипфейки можно использовать вполне легально — например, для того, чтобы вернуть к жизни образы киноактеров, как это было сделано в некоторых недавних фильмах, а можно применить для причинения вреда людям и вымогательства. Одним из самых популярных видов использования дипфейков является изготовление порнографических видеороликов, в которых лицо жертвы без ее согласия смешивается с видео для взрослых, как документально подтверждено BBC и другими новостными агентствами. Дипфейки также применяются в финансовых преступлениях, например когда злоумышленник использует голос генерального директора, чтобы приказать бухгалтеру перевести деньги на чужой счет. Алгоритмическая дискриминация — еще одно распространенное злоупотребление инструментами ИИ. При атаке с «отмыванием справедливости» апостериорные объяснения могут быть изменены, чтобы скрыть дискриминацию в предвзятой модели. А распознавание лиц можно использовать для прямого расового профилирования. Мы коснемся лишь некоторых способов злоупотребления системами ML; более широкое рассмотрение этой важной темы представлено в работе AI-Enabled Future Crime (*https://oreil.ly/8L3ax*).

Атаки на общую доступность

Системы ML могут стать жертвой более общих атак типа «отказ в обслуживании» (DOS), как и другие общедоступные программные сервисы. Если общедоступная система ML имеет ключевое значение для вашей организации, необходимо обеспечить ее защиту с помощью брандмауэров и фильтров, обратного поиска в системе серверов доменных имен (DNS) и других контрмер, которые повышают доступность во время DOS-атаки. К сожалению, нам также приходится заботиться о предотвращении другого вида сбоев доступности систем ML, вызванных алгоритмической дискриминацией. Если алгоритмическая дискриминация достаточно серьезна, независимо от того, вызвана ли она внутренними сбоями или состязательными атаками, наша система ML, скорее всего, не будет пригодна для использования значительной частью ее пользователей. Обязательно проверяйте наличие предвзятости во время обучения и на протяжении всего жизненного цикла системы.

Трояны и вредоносное ПО

Машинное обучение в среде исследований и разработок опирается на разнообразную экосистему пакетов программного обеспечения с открытым исходным кодом. У некоторых из этих пакетов много участников и пользователей. Некоторые из них весьма специфичны и имеют значение лишь для небольшого числа исследователей и практиков. Очевидно, что многие пакеты поддерживаются блестящими статистиками и исследователями машинного обучения, которые

Книги для программистов: https://t.me/free_books_for_it

в первую очередь занимаются математикой или алгоритмами, а не разработкой программного обеспечения или безопасностью. Конвейер машинного обучения нередко зависит от десятков или даже сотен внешних пакетов, любой из которых может быть взломан, чтобы скрыть факт атаки. Сторонние пакеты с большими хранилищами двоичных данных и предварительно обученными моделями машинного обучения кажутся особенно подходящими для подобных манипуляций. Если возможно, просканируйте все программные артефакты, связанные с системой машинного обучения, на наличие вредоносных программ и троянов.

Атаки «человек посередине»

Поскольку многие прогнозы и решения системы МО передаются через Интернет или сеть организации, на этом пути ими могут манипулировать злоумышленники. По возможности используйте шифрование, сертификаты, взаимную аутентификацию или другие контрмеры, чтобы обеспечить целостность данных и результатов системы ML, передаваемых по сетям.

Необъяснимое машинное обучение

Хотя недавние разработки в области интерпретируемых моделей и апостериорных объяснений дают нам возможность применять не только точные, но и прозрачные модели, многие рабочие процессы машинного обучения по-прежнему сосредоточены на необъяснимых моделях. Такие модели часто вносят ненужную сложность в коммерческий рабочий процесс машинного обучения. Настойчивый и мотивированный злоумышленник со временем может узнать о вашей слишком сложной и необъяснимой модели машинного обучения больше, чем знает о ней ваша собственная команда. (Особенно на сегодняшнем рынке труда в области обработки данных, с его огромной текучестью кадров.) Этот дисбаланс знаний потенциально может служить основой для проведения атак, которые мы описали, или для других, пока неизвестных типов атак.

Распределенные вычисления

Хорошо это или плохо, но мы живем в эпоху больших данных. Многие организации сейчас используют системы распределенной обработки данных и машинного обучения. Распределенные вычисления открывают обширную поверхность атаки для внутренних и внешних злоумышленников. Данные могут быть отравлены на одном или нескольких рабочих узлах большой распределенной системы хранения или обработки данных. Бэкдор может быть закодирован только в одной модели большого ансамбля. Вместо отладки одного простого набора данных или модели практикам теперь приходится иногда изучать данные или модели, распределенные по большим вычислительным кластерам.

Вы обеспокоены? Не волнуйтесь, сейчас мы рассмотрим меры противодействия атакам на конфиденциальность, целостность и доступность систем машинного обучения.

Книги для программистов: https://t.me/free_books_for_it

Меры противодействия

Нам доступны различные меры противодействия, и в сочетании с процессами управления, предложенными в главе 1, вознаграждением за обнаружение ошибок, аудитом безопасности и «красной командой» такие меры с большей вероятностью будут эффективными. Кроме того, появились новые научные дисциплины состязательного ML и робастного ML, которые дают этим предметам полную академическую трактовку. В этом разделе будут описаны некоторые меры защиты, к которым мы можем прибегнуть, чтобы сделать наши системы ML более безопасными, включая отладку и мониторинг моделей в целях безопасности, технологии повышения конфиденциальности, робастное машинное обучение и несколько общих подходов.

Отладка модели в целях безопасности

Модели ML могут и должны быть протестированы на наличие уязвимостей безопасности перед их развертыванием. Целью этих тестов является имитация атаки на собственные системы ML, определение состояния безопасности и исправление любых обнаруженных уязвимостей. В число методов, которые работают с различными типами моделей ML при отладке безопасности, входят поиск состязательных примеров, анализ чувствительности, аудит внутренних атак и атак с извлечением моделей, а также дискриминационное тестирование и некоторые другие.

Поиск состязательных примеров и анализ чувствительности

Проведение анализа чувствительности в состязательном контексте или, что еще лучше, проведение ваших собственных состязательных атак по выборкам является хорошим способом определить, уязвима ли ваша система для, возможно, самой простой и распространенной атаки на целостность ML. Идея этого этического взлома состоит в том, чтобы понять, какие значения признаков (или их комбинации) могут вызвать большие колебания в прогнозах выходных данных вашей системы. Если вы работаете в области глубокого обучения, обратите внимание на такие пакеты, как Smarthans и Foolbox. Они помогут вам начать тестирование вашей системы машинного обучения. Если же вы работаете со структурированными данными, то для выявления нестабильностей в вашей системе хорошо подойдет старый добрый анализ чувствительности. Для генерации собственных состязательных выборок можно применить генетическое обучение или эвристические методы, основанные на индивидуальных условных ожиданиях. Если в результате тестирования обнаружилась нестабильность системы, которую можно вызвать состязательными выборками, на следующем шаге для обучения более стабильной модели примените перекрестную проверку или регуляризацию, методы надежного машинного обучения (раздел «Надежное машинное обучение» далее в этой главе) или явное отслеживание обнаруженных состязательных выборок в режиме реального времени. Вам также нужно связать эту информацию с планом реагирования на инциденты системы, если вы полагаете, что она может пригодиться позже.

Книги для программистов: https://t.me/free_books_for_it

Аудит на предмет инсайдерского отравления данных

Если произойдет атака с отравлением данных, виновниками вполне могут быть инсайдеры системы — сотрудники, подрядчики и консультанты. Как можно выявить инсайдерское отравление данных? Сначала оцените этих людей с помощью вашей системы. Любой инсайдер, получивший положительный результат, может быть злоумышленником или, возможно, связан со злоумышленником. Поскольку умный злоумышленник, скорее всего, внесет минимальные изменения в обучающие данные, которые приведут к положительному результату, можно также применить анализ остатков для поиска потенциально выгодных результатов с более высокими, чем ожидалось, остатками. Их наличие указывает на то, что модель ML, скорее всего, выдала бы отрицательный результат для этого человека, если бы в обучающие данные не внесли изменения. Управление данными и средой является мощной мерой противодействия отравлению инсайдерских данных, поскольку любые изменения данных отслеживаются с помощью обширных метаданных (кто, что, когда и т. д.). Можете также попробовать метод отклонения при отрицательном воздействии (reject on negative impact, RONI), предложенный в фундаментальной работе The Security of Machine Learning (*https://oreil.ly/exh6g*), чтобы удалить потенциально измененные строки из обучающих данных системы.

Тестирование на предвзятость

DOS-атаки, возникающие в результате какой-либо предвзятости — преднамеренной или нет, — являются одним из возможных типов атаки на доступность. Такие атаки уже происходили на практике. В 2016 г. пользователи Twitter отравили чатбота Tay (*https://oreil.ly/uPqNx*) до такой степени, что он полностью превратился в источник порнографического контента с неонацистским уклоном. Этот тип атаки может быть реализован в более серьезном контексте, например в сфере трудоустройства, кредитования или медицины, когда злоумышленник использует подмену данных, моделирует бэкдоры или другие типы атак, чтобы отказать в обслуживании определенной группе клиентов. Это одна из многих причин, по которым важно регулярно проводить тестирование на предвзятость и устранять любую обнаруженную дискриминацию как во время обучения, так и в рамках периодического мониторинга модели. Существует несколько отличных инструментов с открытым исходным кодом для выявления и устранения дискриминации, например aequitas (*https://oreil.ly/e412j*), Themis (*https://oreil.ly/yJiT6*) и AIF360 (*https://oreil.ly/HsKEg*).

Этический взлом: атаки с извлечением моделей

Атаки с извлечением модели вредны сами по себе, но они также являются первой стадией атаки с выводом о членстве. Вы должны провести собственные атаки по извлечению моделей, чтобы определить, уязвима ли ваша система к таким атакам на конфиденциальность. Если вы нашли некоторый API или конечную точку модели, которая позволяет обучать суррогатную модель на входных и выходных данных системы, заблокируйте ее с помощью надежной аутентификации и ограничьте любые аномальные запросы в этой конечной точке. Если уязвимость обнаружена

Книги для программистов: https://t.me/free_books_for_it

после развертывания модели и есть риск, что атака с извлечением модели уже произошла, вам необходимо проанализировать следующие аспекты потенциально возможных суррогатных моделей:

- каковы пределы точности различных типов суррогатных моделей? Вы должны попытаться понять, в какой степени суррогатная модель действительно может быть использована для получения знаний о вашей системе ML;
- какие типы трендов данных можно узнать из суррогатной модели? Например, линейные тренды, представленные коэффициентами линейной модели? Или краткие сведения о подгруппах населения в суррогатном дереве решений?
- какие правила можно извлечь из суррогатного дерева решений? Например, как достоверно выдать себя за человека, который получит выгодный прогноз? Или как построить эффективные состязательные выборки?

Если вы установили, что действительно можно обучить точную суррогатную модель на прогнозах одной из конечных точек вашей системы и ответить на некоторые из вышеперечисленных вопросов, то вам придется предпринять следующие шаги. Сначала проведите имитационную атаку с выводом о членстве, чтобы посмотреть, возможна ли такая двухэтапная атака. Затем запишите всю информацию, связанную с этим анализом этического взлома, и свяжите ее с планом реагирования на инциденты системы. Она может пригодиться в дальнейшем специалистам по реагированию на инциденты, и, к сожалению, о ней, скорее всего, придется сообщить как о взломе, если есть веские доказательства того, что атака действительно произошла.

Отладка уязвимостей безопасности в системе ML — важная работа, которая может сэкономить вам деньги, время и нервы в будущем, но это относится и к наблюдению за системой. Далее мы обсудим мониторинг модели в целях безопасности.

Мониторинг модели в целях безопасности

Как только хакеры получают возможность манипулировать вашей моделью ML или извлечь ее, она перестает быть вашей моделью. Чтобы защитить свою модель от атак, нужно не только обучать и отлаживать ее с учетом требований безопасности, но и внимательно следить за ее работой после развертывания. Мониторинг безопасности должен быть ориентирован на алгоритмическое распознавание аномалий в очередях входных данных, аномалий в прогнозах и аномально высокого уровня использования. Вот несколько советов о том, что и как контролировать:

Мониторинг предвзятости

Тестирование предвзятости, как обсуждалось в предыдущих главах, должно применяться во время обучения модели. Но по многим причинам, включая непредвиденные стечения обстоятельств и злонамеренный взлом, тестирование на дискриминацию необходимо проводить и во время эксплуатации модели. Если во время эксплуатации обнаруживается предвзятость, ее следует расследовать и устранить. Вы должны быть уверены, что модель, которая была справедливой во время обучения, останется справедливой в производстве.

Книги для программистов: https://t.me/free_books_for_it

Аномалии на входе

Нереалистичные комбинации данных, с помощью которых можно запустить бэкдоры в механизмах моделей, не должны допускаться на вход модели. В целом при отслеживании проблемных входных данных могут быть полезны методы машинного обучения для обнаружения аномалий, такие как автокодировщики и изолирующие леса. Однако вы также можете установить разумные ограничения целостности данных, чтобы выявить проблемные данные до того, как они попадут в вашу модель. Примером таких нереалистичных данных является возраст 40 лет и стаж работы 50 лет. Если возможно, попробуйте организовать также мониторинг случайных данных, обучающих данных или повторяющихся данных. Поскольку случайные данные часто используются при извлечении моделей и атаках с инверсией, было бы полезно создать оповещения или элементы управления, которые помогут вашей команде понять, где и когда модель столкнулась с пакетами случайных данных. Следует записывать и исследовать в реальном времени поступление строк данных, которые очень похожи или идентичны данным, используемым при обучении, проверке или тестировании, поскольку такие входные данные могут быть признаком атаки с выводом о членстве. Наконец, обратите внимание на дублирующиеся данные в очереди на входе модели, поскольку это может указывать на попытку атаки с уклонением или атаки с использованием подмены личности.

Аномалии на выходе

Аномалии выходных данных могут указывать на состязательные атаки. При оценке новых данных мы сравниваем прогнозы нашей модели машинного обучения с надежной прозрачной эталонной моделью или эталонной моделью, обученной на надежном источнике данных и конвейере. Если разница между нашей более сложной и непрозрачной моделью машинного обучения и нашей интерпретируемой или доверенной моделью слишком велика, необходимо вернуться к прогнозам консервативной модели или отправить строку данных на ручную обработку. Для мониторинга аномальных выходных данных также можно применять пределы статистического контроля, подобные движущимся доверительным интервалам.

Метамониторинг

Метамониторинг основан на отслеживании базовой операционной статистики (метапараметров) — количества прогнозов за определенный период времени, задержки, загрузки ЦП, памяти и диска или количества одновременно подключенных пользователей. Вы можете даже обучить метамодель обнаружения аномалий на основе автокодировщика, подав ей в качестве обучающих данных историю рабочей статистики всей своей системы машинного обучения, а затем отслеживать эту метамодель на предмет аномалий. Аномалия на уровне метапараметров системы может служить сигналом, что в вашей системе ML что-то не так.

Мониторинг атак — один из наиболее активных шагов, которые мы можем предпринять для противодействия взломам ОД. Однако есть еще несколько контрмер,

Книги для программистов: https://t.me/free_books_for_it

которые необходимо обсудить. Далее мы рассмотрим технологии повышения конфиденциальности.

Технологии повышения конфиденциальности

Машинное обучение с сохранением конфиденциальности — это исследовательская дисциплина, которая напрямую влияет на конфиденциальность наших обучающих данных. Хотя *технологии повышения конфиденциальности* (privacy-enhancing technologies, PET) только начинают набирать обороты в сообществах, занимающихся машинным обучением и операциями машинного обучения (MLOps), они могут дать вам ощутимое преимущество, когда дело доходит до защиты ваших данных и моделей. Одними из наиболее многообещающих и практичных методов в этой развивающейся области являются федеративное обучение и дифференциальная приватность.

Федеративное обучение

Федеративное обучение (federated learning) — это подход к обучению моделей ML на нескольких децентрализованных устройствах или серверах, содержащих локальные выборки данных, без обмена между ними необработанными данными. Этот подход отличается от традиционных централизованных методов машинного обучения, когда все наборы данных загружаются на один сервер. Основное преимущество федеративного обучения заключается в том, что оно позволяет создавать модели машинного обучения без обмена данными между многими участниками. Федеративное обучение позволяет избежать совместного использования данных. Оно осуществляется путем обучения локальных моделей на выборках локальных данных и последующего обмена параметрами между серверами или периферийными устройствами. В результате создается глобальная модель, которая затем используется всеми серверами или периферийными устройствами. При наличии безопасного процесса агрегирования параметров федеративное обучение помогает решить фундаментальные проблемы конфиденциальности и безопасности данных. Если вы решили внедрить федеративное обучение в своей организации, попробуйте начать с ресурсов с открытым исходным кодом, таких как PySyft (*https://oreil.ly/8HpeR*) или FATE (*https://oreil.ly/W3uYP*).

Дифференциальная приватность

Дифференциальная приватность (differential privacy) — это система обмена информацией о наборе данных путем описания закономерностей групп в наборе данных без раскрытия информации о конкретных людях. В инструментах машинного обучения это часто достигается с помощью специализированных *дифференциально-приватных алгоритмов обучения*. Такой подход затрудняет извлечение конфиденциальной информации из обучающих данных или обученной модели машинного обучения при извлечении модели, инверсии модели или атаках с выводом о членстве. Фактически модель ML считается дифференциально-приватной, если внешний наблюдатель не может определить, использовалась ли информация о конкретном человеке для обучения модели. Существует множество высококачествен-

Книги для программистов: https://t.me/free_books_for_it

ных репозиториев с открытым исходным кодом, которые заслуживают изучения, включая следующие:

- differential-privacy (*https://oreil.ly/rjwKK*) от Google;
- diffprivlib (*https://oreil.ly/QOFm-*) от IBM;
- privacy (*https://oreil.ly/WyPD6*) для TensorFlow.

Многие подходы ML, предусматривающие дифференциальную приватность, основаны на *дифференциально-приватном стохастическом градиентном спуске* (differentially private stochastic gradient descent, DP-SGD) (*https://oreil.ly/raWeC*). DP-SGD вводит структурированный шум в градиенты, определяемые SGD на каждой итерации обучения. В целом DP-SGD и связанные с ним методы гарантируют, что модели машинного обучения не запоминают слишком много конкретной информации о данных обучения. Поскольку они не позволяют алгоритмам ML сосредоточиться на конкретных людях, это способствует повышению эффективности и справедливости обобщения.

Читатели также могли слышать о конфиденциальных вычислениях или гомоморфном шифровании в области PET. Это также многообещающие направления передовых исследований и технологий, на которые стоит обратить внимание. Еще одним направлением исследований в области ML, за которым стоит следить, является робастное машинное обучение, которое способствует устойчивости системы к состязательным атакам, отравлению данных и другим злонамеренным манипуляциям.

Что еще нужно знать специалистам по обработке данных о конфиденциальности?

Использование данных становится все более регулируемой областью, в том числе в США. Помимо проблем с регулирующими органами, небрежное обращение с данными может нанести вред вашим пользователям, вашей организации или широкой общественности. Нам, специалистам по данным, необходимо знать некоторые основы конфиденциальности данных:

Правила и политика

Скорее всего, мы будем действовать в соответствии с некоторыми правилами конфиденциальности данных или политикой конфиденциальности организации. Мы должны знать юридические основы наших обязательств и стараться их придерживаться. В США особенно тщательно защищены данные о здравоохранении и образовании, но Общий регламент ЕС по защите данных (GDPR), которого должны придерживаться многие транснациональные корпорации, распространяется почти на все типы потребительских данных, а штаты США принимают множество новых локальных законов о конфиденциальности данных.

Согласие

Хотя многие правила защиты конфиденциальности данных далеки от совершенства, они основаны на понятии согласия (consent), т. е. на том, что потребители дают *деятельное согласие* на использование своих данных для конкретного приложения. Как с юридической, так и с этической точки зрения полезно подтвердить, что у нас есть согласие на использование наших данных для модели, которую мы обучаем.

Книги для программистов: https://t.me/free_books_for_it

Другие правовые основания для использования

Если мы действуем в соответствии с GDPR, нам нужна правовая основа для использования большей части потребительских данных, которые могут нас заинтересовать в контексте ML. Часто правовой основой служит упомянутое выше согласие, но есть и другие примеры: договорные обязательства, правительственные задания или неотложная медицинская помощь.

Анонимизация

Прямое использование информации, позволяющей идентифицировать личность (personal identifiable information, PII), никогда не было хорошей идеей. Такие данные, как номера социального страхования, номера телефонов и даже адреса электронной почты, являются прямыми идентификаторами и позволяют злоумышленникам связывать частную и конфиденциальную информацию с конкретными людьми. Даже комбинации демографической информации, такой как возраст, раса или пол, хотя и являются косвенными идентификаторами, могут служить для привязки информации к конкретным людям. Как правило, такого рода данные следует удалять, маскировать, хешировать или иным образом анонимизировать перед обучением модели по соображениям конфиденциальности и борьбы с предвзятостью.

Биометрические данные

Такие данные, как цифровые фотоснимки, видео, отпечатки пальцев, записи голоса, сканы радужной оболочки глаза, геномные данные или другие данные, которые кодируют биометрическую информацию, обычно требуют дополнительных мер по соблюдению безопасности и конфиденциальности данных.

Ограничения или требования к хранению данных

Ограничение хранения, т. е. длительность хранения данных перед их обязательным удалением, может зависеть как от законов государства, так и от политики конфиденциальности организации. (Мы встречали даже двухнедельный лимит хранения!) Возможно, вам также придется столкнуться с юридическими или организационными требованиями к хранению, которые определяют, как долго данные должны храниться, а также оставаться конфиденциальными и безопасными, — это может длиться годами.

Запросы на удаление и исправление

Многие законы и корпоративные политики позволяют потребителям обновлять, исправлять или полностью удалять свои данные по запросу, и это потенциально может повлиять на обучающие данные ML.

Объяснение

Многие законы о конфиденциальности данных также содержат требование предоставить объяснения алгоритма автоматической обработки данных, который влияет на потребителей. Хотя эти требования часто расплывчаты и для них нет реальных решений, скорее всего, в будущем нам придется объяснять потребителям гораздо больше решений, связанных с ML.

Возможность вмешательства

Подобно объяснению алгоритма, многие новые законы часто содержат требование о возможности вмешательства оператора, которое аналогично апелляции и предшествует возможности обратиться в суд.

Книги для программистов: https://t.me/free_books_for_it

Предвзятость

Законы и политики конфиденциальности данных нередко направлены на устранение предвзятости в результатах обработки данных, например в прогнозах моделей машинного обучения. Иногда эти требования являются расплывчатыми и очень обобщенными. Иногда они более конкретны или просто требуют соблюдения установленных законов о недопущении дискриминации.

Все эти сложности могут серьезно повлиять на ваши рабочие процессы ML. Возможно, вам придется уточнить правила работы с ограничениями хранения, запросами на удаление, запросами на исправление или требованиями к взаимодействию. Например, нужно ли вам вывести из эксплуатации и удалить модель, обученную на данных с истекшим сроком хранения, или на данных, удаление которых потребовали потребители? Мы пока не слышали о таких случаях, но это вполне возможно. Если читатели никогда не проходили обучение по применению нормативов конфиденциальности данных или у них есть вопросы или опасения по поводу конфиденциальности данных, возможно, было бы неплохо проконсультироваться с руководством или юридическим отделом.

Робастное машинное обучение

Робастное машинное обучение включает в себя множество передовых алгоритмов ML, разработанных для противодействия состязательным атакам, а также, в определенной степени, отравлению данных. Изучение робастного ML набрало обороты после того, как несколько исследователей показали, что безобидные или даже невидимые изменения входных данных могут привести к огромным колебаниям в прогнозах систем компьютерного зрения. Такие колебания в результатах моделей являются тревожным знаком в любой области, но когда речь идет о медицинской визуализации или полуавтономных транспортных средствах, они чрезвычайно опасны. Робастные модели ML помогают обеспечить стабильность результатов и, что немаловажно, справедливость — одинаковое обращение с похожими людьми. Похожие люди в данных обучения ML или реальных данных — это люди, которые находятся близко друг к другу в евклидовом пространстве данных. Робастные методы ML часто пытаются создать гиперсферу вокруг отдельных выборок данных и гарантировать, что другие аналогичные данные в гиперсфере получат аналогичные прогнозы. Робастные подходы к ML помогают защитить организацию от рисков, возникающих в результате неожиданных прогнозов, независимо от того, вызваны ли они злоумышленниками, переобучением, недостаточной специфичностью или другими факторами. Интересные статьи и код размещены на сайте Robust ML (*https://oreil.ly/H36uh*), а лаборатория Madry Lab Массачусетского технологического института даже опубликовала полный пакет Python (*https://oreil.ly/k-qDZ*) для робастного машинного обучения.

Универсальные контрмеры

Существует ряд универсальных контрмер для защиты от нескольких различных типов атак на ML, в том числе аутентификация, замедление отклика и водяные зна-

Книги для программистов: https://t.me/free_books_for_it

ки. Многие из этих контрмер также являются полезными для систем ML в целом, как, например, интерпретируемость моделей, управление моделями и мониторинг. Последняя тема, которую мы затронем перед разбором практического примера главы, — это краткое описание важных и общих мер противодействия атакам на системы ML:

Аутентификация

По возможности запрещайте анонимное использование систем ML в приложениях высокого риска. Парольный доступ, многофакторная аутентификация или другие типы аутентификации, заставляющие пользователей подтверждать свою личность, авторизацию и разрешение на использование системы, создают преграду между API вашей модели и анонимными злоумышленниками.

Интерпретируемые, справедливые или частные модели

К настоящему времени появились методы моделирования, например монотонные GBM (M-GBM), масштабируемые списки байесовских правил (scalable Bayesian rule lists, SBRL) (*https://oreil.ly/Md375*) и объяснимые нейронные сети (XNN) (*https://oreil.ly/sd4XX*), способные обеспечить как точность, так и интерпретируемость моделей ML. Эти точные и интерпретируемые модели легче документировать и отлаживать по сравнению с классическими необъяснимыми моделями. Модели, основанные на новых подходах честного и частного моделирования, например LFR (*https://oreil.ly/7ZHmw*) и DP-SGD (*https://oreil.ly/yuddM*), можно обучить таким образом, чтобы скрыть видимые извне демографические характеристики и тем самым уменьшить риск состязательной атаки или подмены личности. Эти модели, улучшенные с точки зрения интерпретируемости, справедливости или конфиденциальности, должны также легче отлаживаться, быть более устойчивыми к изменениям характеристик отдельного объекта и более безопасными, чем широко распространенные необъяснимые модели.

Документация модели

Документирование моделей — это стратегия снижения рисков, которая десятилетиями использовалась в банковской сфере. Она позволяет сохранять и передавать знания о сложных системах моделирования от одной команды владельцев модели к другой, а также стандартизировать знания для эффективного анализа валидаторами моделей и аудиторами. Документация модели должна охватывать информацию о том, кто, что, где и как работает в системе ML, включая множество деталей, от контактной информации ответственных лиц до спецификации алгоритма. Документация модели также является естественным местом для записи сведений о любых известных уязвимостях или проблемах безопасности системы машинного обучения, что позволяет будущим специалистам по обслуживанию или другим операторам, взаимодействующим с системой, эффективно распределять ресурсы надзора и безопасности. Модельная документация также должна быть взаимно увязана с планами реагирования на инциденты. (Глава 2 содержит образец шаблона документации.)

Книги для программистов: https://t.me/free_books_for_it

Управление моделями

Управление моделями (model management) обычно представляет собой комбинацию средств управления процессами, таких как документация, в сочетании с техническими средствами управления, такими как мониторинг и инвентаризация моделей. Организации должны иметь точную информацию о количестве развернутых систем машинного обучения и структурированный перечень связанного кода, данных, документации и планов реагирования на инциденты, а также отслеживать все развернутые модели. Наличие системного подхода к управлению моделями позволяет вовремя понять, когда что-то идет не так, и быстро решать проблемы по мере их возникновения. (В главе 1 более подробно обсуждается модель управления рисками в ML.)

Замедление

Если системы мониторинга моделей выявляют чрезмерно интенсивное использование или другие аномалии, такие как состязательные выборки или повторяющиеся, случайные или обучающие данные, можно прибегнуть к искусственному ограничению производительности API прогнозирования или других конечных точек системы. Ограничение может выражаться в сокращении количества быстрых прогнозов для отдельных пользователей, искусственном увеличении задержки прогнозирования для всех пользователей или в других методах, которые могут замедлить атаку злоумышленников, проводящих атаки с извлечением модели или данных, а также атаки с использованием состязательных примеров.

Водяные знаки

Под водяными знаками подразумевается добавление малозаметного маркера к данным или прогнозам с целью предотвращения кражи данных или моделей. Если данные или прогнозы имеют легко идентифицируемые признаки, такие как настоящие водяные знаки на изображениях или контрольные маркеры в структурированных данных, это может затруднить использование украденных активов и сделать их более идентифицируемыми для правоохранительных или других органов в случае кражи.

Применение этих общих мер защиты и передовых методик, а также некоторых более конкретных мер противодействия, упомянутых в предыдущих разделах, — отличный способ достичь высокого уровня безопасности системы ML. И теперь, когда мы рассмотрели основы безопасности, атаки ML и многие меры противодействия этим атакам, читатели вооружены знаниями, необходимыми для того, чтобы собрать «красную команду» и начать тестирование ИИ своей организации. Особенно полезно будет объединить усилия со специалистами по корпоративной ИТ-безопасности. Завершая эту главу, мы рассмотрим некоторые реальные инциденты безопасности ИИ, чтобы дополнительно мотивировать читателей на выполнение тяжелой работы «красной команды» и дать им представление о некоторых из наиболее распространенных сегодня проблем безопасности ML.

Книги для программистов: https://t.me/free_books_for_it

Пример: реальные атаки уклонения

Системы машинного обучения, предназначенные как для физической, так и онлайн-безопасности, в последние годы подвергались атакам с целью уклонения. Далее мы обсудим атаки уклонения, направленные на обход фильтров Facebook[1] и проталкивание дезинформации и террористической пропаганды, а также атаки уклонения от реальных платежных систем и систем физической безопасности.

Атаки уклонения

Когда пандемия COVID набирала силу, а президентская кампания в США 2020 года была в самом разгаре, распространители дезинформации по той и другой теме воспользовались слабостями ручной и автоматической фильтрации контента Facebook. Как сообщает NPR, «небольшие изменения позволяют ложным заявлениям о COVID-19 и президентских выборах обойти систему проверки фактов в Facebook» (*https://oreil.ly/aYSTr*). Хотя Facebook использует публикации новостных организаций, такие как Reuters и Associated Press, для проверки достоверности фактов, упомянутых в высказываниях миллиардов пользователей, он также фильтрует контент сообщений с помощью ИИ, в частности для обнаружения копий дезинформационных сообщений, ранее помеченных людьми. К сожалению, незначительные изменения, даже такие простые, как изменение фона или шрифтов, обрезка изображений или простое описание мемов словами вместо изображений, позволили злоумышленникам обойти фильтры контента Facebook на основе ML. В качестве самозащиты Facebook принимает принудительные меры против многих правонарушителей, включая ограничение распространения публикаций, отказ от рекомендации публикаций или групп, а также демонетизацию. Тем не менее, по данным одной правозащитной группы, Facebook не может отловить около 42% дезинформационных постов, содержащих информацию, отмеченную специалистами по проверке фактов (*https://oreil.ly/fzCrb*). По оценкам той же правозащитной группы Avaaz, всего лишь 738 немаркированных дезинформационных постов привели к примерно 142 млн просмотров и 5,6 млн взаимодействий с пользователями.

Недавние события показали нам, что онлайн-дезинформация и угрозы безопасности могут перекинуться в реальный мир. Дезинформация о выборах в США в 2020 г. и пандемии COVID считается основной причиной пугающих беспорядков в Капитолии США 6 января 2021 г. Возможно, еще более тревожными атаками с целью уклонения являются сообщения BBC о том, что боевики ИГИЛ продолжают уклоняться от фильтров контента Facebook. Размывая логотипы, объединяя свои видеоролики с основным новостным контентом или просто используя странную пунктуацию, члены или филиалы ИГИЛ смогли публиковать пропагандистские материалы, обучающие материалы по изготовлению взрывчатых веществ и даже учебные пособия по атакам с целью уклонения от фильтров Facebook. Эти публикации собрали десятки тысяч просмотров. Хотя атаки с целью уклонения от фильт-

[1] Facebook — социальная сеть, запрещенная в РФ. — *Прим. ред.*

Книги для программистов: https://t.me/free_books_for_it

ров на основе искусственного интеллекта, безусловно, являются основной причиной, нужно отметить, что на Facebook меньше людей-модераторов арабского контента. Независимо от того, люди или машины не справляются со своей задачей, этот тип контента может быть действительно опасным, поскольку способствует радикализации и насилию в реальном мире. Вызывает беспокойство и перспектива атак с целью уклонения в физическом мире. Недавние исследования показали, что некоторые системы физической безопасности на базе искусственного интеллекта являются легкой мишенью для атак с целью уклонения. С разрешения операторов системы исследователи использовали реалистичные трехмерные маски для обхода проверок безопасности распознавания лиц в платежных системах Alipay и WeChat. В одном вопиющем случае исследователям даже удалось использовать фотографию другого человека на экране iPhone, чтобы сесть на самолет в амстердамском аэропорту Схипхол.

Какие уроки мы можем извлечь?

В совокупности столь простое уклонение злоумышленников от онлайн-защиты для публикации опасного контента, а также от систем физической безопасности для осуществления денежных платежей и путешествий на самолете рисует пугающую картину мира, в которой безопасность машинного обучения не привыкли воспринимать всерьез. Какие уроки, извлеченные из этой главы, можно было бы применить для предотвращения подобных атак уклонения? Первый урок связан с робастным машинным обучением. Системы машинного обучения, используемые для приложений безопасности с высокими рисками, будь то онлайн или реальный мир, не должны быть обмануты крошечными изменениями в обычных входных данных системы. Робастное машинное обучение и связанные с ним технологии должны быть усовершенствованы хотя бы до такой степени, когда упрощенные методы уклонения, такие как размытие логотипов или изменение пунктуации, перестанут быть эффективными мерами уклонения. Еще один урок содержится в начале главы: состязательный образ мышления. Любой, кто серьезно задумывался о рисках безопасности для этих систем на основе искусственного интеллекта, должен понимать, что реалистичные маски или просто другие изображения являются очевидным методом уклонения. К счастью, оказывается, что некоторые организации применяют контрмеры в случае враждебных сценариев. Более совершенные системы безопасности с распознаванием лиц используют методы, призванные убедиться, что объекты, которые они идентифицируют, являются живыми людьми. Лучшие системы распознавания лиц также используют тестирование на дискриминацию, чтобы обеспечить высокую доступность и минимально возможный уровень ошибок для всех их пользователей.

Еще один важный урок, который следует извлечь из реальных атак с целью уклонения, касается ответственного использования технологий в целом и ML в частности. Социальные сети распространились за пределы физических границ государств, и их сложность превысила нынешние возможности многих правительств по эффективному регулированию. Из-за отсутствия государственного регулирования пользователи рассчитывают на то, что компании, владеющие социальными сетями,

Книги для программистов: https://t.me/free_books_for_it

будут регулировать себя сами. Такие компании часто полагаются на передовые технологии, например фильтры контента на основе ИИ. Но что если эти элементы контроля на самом деле не работают? Поскольку технологии и ML играют все более важную роль в жизни людей, отсутствие строгости и ответственности при их разработке, внедрении и развертывании будет иметь все более серьезные последствия. Разработчики технологий для обеспечения безопасности или других приложений с высокими рисками больше, чем кто-либо другой, обязаны реалистично оценивать сегодняшние возможности машинного обучения и применять все доступные средства управления процессами и технологиями для предоставления надежных и безопасных услуг.

Ресурсы

Дополнительное чтение

- A Marauder's Map of Security and Privacy in Machine Learning: **https://oreil.ly/0k7D3**
- Интерактивный фреймворк оценки рисков машинного обучения BIML: **https://oreil.ly/csQ22**
- Руководство FTC *Start with Security*: **https://oreil.ly/jmeja**
- Ландшафт состязательных угроз для систем искусственного интеллекта: **https://oreil.ly/KxEbC**
- Центр ресурсов по компьютерной безопасности NIST: **https://oreil.ly/pncXb**
- Инструменты противодействия идентификации NIST: **https://oreil.ly/M8xhr**

Книги для программистов: https://t.me/free_books_for_it

ЧАСТЬ II

Управление рисками ИИ в действии

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 6

Объяснимые бустинговые машины и модели XGBoost

В этой главе мы исследуем объяснимые модели и апостериорные объяснения на практическом примере из области потребительского кредитования. Здесь также применяются подходы, обсуждавшиеся в главе 2, с использованием *объяснимых бустинговых машин* (explainable boosting machine, EBM), моделей XGBoost с монотонными ограничениями и методов апостериорного объяснения. Сначала мы освежим в памяти понятия аддитивности, ограничений, частичной зависимости и индивидуального условного ожидания (individual conditional expectation, ICE), аддитивных объяснений Шепли (Shapley additive explanations, SHAP) и документации модели.

Затем мы рассмотрим пример задачи кредитного андеррайтинга, переходя от штрафной регрессии к *обобщенной аддитивной модели* (generalized additive model, GAM) и EBM. Переходя от простых моделей к более сложным, мы будем документировать явные и преднамеренные компромиссы, касающиеся введения нелинейности и взаимодействий в наш пример вероятности классификатора по умолчанию, сохраняя при этом почти полную объяснимость с помощью аддитивных моделей.



В главе 2 мы говорили, что *интерпретация* — это осмысленное представление на высоком уровне, которое помещает стимул в контекст и использует фоновые знания человека, тогда как *объяснение* — это детальное мысленное представление низкого уровня, целью которого является описание сложного процесса. Интерпретация — это гораздо более высокая планка, чем объяснение, и ее редко можно достичь только с помощью технических подходов.

После этого мы рассмотрим второй подход к прогнозированию по умолчанию, который допускает сложные взаимодействия признаков, но контролирует сложность с помощью монотонных ограничений, основанных на причинных знаниях. Поскольку машину градиентного бустинга (GBM) с монотонными ограничениями нельзя объяснить саму по себе, мы объединим ее с надежными методами апостериорного объяснения, чтобы значительно повысить объяснимость. Глава завершается обсуждением достоинств и недостатков популярных методов на основе значения Шепли.

Уточнение определений и прозрачность машинного обучения

Прежде чем перейти к углубленному рассмотрению технических примеров, давайте вспомним некоторые ключевые понятия из главы 2. Поскольку наш первый пример

Книги для программистов: https://t.me/free_books_for_it

подчеркнет сильные стороны семейства моделей GAM, далее мы рассмотрим аддитивность, особенно в сравнении с моделями, которые допускают взаимодействия высокого порядка. В нашем втором примере монотонные ограничения будут применяться в реализации неформального подхода причинно-следственной связи с помощью XGBoost, поэтому мы кратко выделим связи между причинностью и ограничениями. Мы также будем использовать частичную зависимость и ICE для сравнения и оценки различных подходов к обработке входных признаков, поэтому нам нужно быстро освежить в памяти сильные и слабые стороны этих апостериорных объяснителей. Наконец, нам нужно вспомнить о документировании модели — потому что документация модели действительно очень важна.

Аддитивность или взаимодействия

Основной отличительной чертой необъяснимого машинного обучения является его склонность к созданию взаимодействий чрезвычайно высокого порядка между входными признаками. Считается, что эта способность учитывать значения многих признаков в одновременной комбинации увеличивает прогностическую способность моделей ML по сравнению с более традиционными линейными или аддитивными моделями, склонными рассматривать входные признаки независимо. Но было показано, что необъяснимые модели не являются более точными на структурированных данных, таких как данные о кредитном андеррайтинге, и все эти взаимодействия в необъяснимых моделях людям очень трудно понять. Более того, взаимодействия высокого порядка также приводят к нестабильности, поскольку небольшие изменения в одном или нескольких признаках могут взаимодействовать с другими признаками, что резко меняет результаты модели. Вдобавок взаимодействия высокого порядка приводят к переобучению, потому что сегодняшние 17-сторонние взаимодействия, скорее всего, не будут актуальными 17-сторонними взаимодействиями завтра.

Еще одной важной характеристикой ML является способность автоматически изучать нелинейные явления в обучающих данных. Оказывается, если мы сможем отделить нелинейность от взаимодействий, мы сможем существенно повысить качество прогнозирования, сохранив при этом значительную степень объяснимости, если не полную объяснимость. В этом и заключается магия GAM. А EBM — это следующий шаг, который позволит нам ввести разумное количество двусторонних взаимодействий аддитивным образом, что может еще больше повысить качество модели. Далее в этой главе мы рассмотрим пример семейства GAM (раздел «Семейство объяснимых моделей GAM»), который послужит наглядным уроком упомянутых компромиссов, начиная с простой, аддитивной, линейной базовой модели, а затем вводя нелинейность с помощью GAM и, наконец, переходя к доступному для понимания двустороннему взаимодействию в EBM. Когда мы осторожно вводим нелинейность и взаимодействия, а не предполагаем, что чем больше сложность, тем лучше, это позволяет нам обосновать наш подход к моделированию, связать его с реальными проблемами и построить множество интересных графиков поведения признаков. Эти обоснования и графики также являются отличным материалом для последующего документирования модели, которое мы вскоре рассмотрим более подробно.

Книги для программистов: https://t.me/free_books_for_it



Семейство моделей, которое обычно включает варианты GLM, такие как GAM, GA2M, EBM, и аддитивные индексные модели (additive index model, AIM), обсуждается в статистической литературе уже много лет. Этот тип моделирования иногда называют *функциональным дисперсионным анализом* (functional analysis of variance, fANOVA).

Переход к причинно-следственной связи с ограничениями

Поиск причинности и причинно-следственный вывод являются важными направлениями исследований в области прогнозного моделирования. Почему? Потому что, когда мы строим корреляции с помощью ML, зачастую получается замок из песка. В реальном мире корреляции постоянно меняются и могут быть случайными или просто неправильными. Если мы сможем основывать модели на реальных причинно-следственных связях, а не просто запоминать некоторый снимок сложных корреляций с помощью модели ML, мы значительно уменьшим риски переобучения, дрейфа данных и социальных предубеждений. На данный момент большинству организаций сложно реализовать причинно-следственные методы, поэтому в примере с монотонными ограничениями (раздел «Ограниченный и неограниченный XGBoost») мы демонстрируем простой и легкий шаг, который вы можете предпринять, чтобы внедрить причинно-следственную связь в модели машинного обучения. Если при помощи логических рассуждений или простых, но надежных экспериментов мы установим направленность причинно-следственной связи в реальном мире, то сможем задать ограничения монотонности, чтобы обеспечить эту направленность в наших моделях XGBoost. Например, если мы знаем, что увеличение числа просроченных платежей является индикатором будущего дефолта, то можем задать ограничение монотонности, согласно которому классификатор XGBoost должен генерировать более высокие вероятности дефолта при возрастании количества просроченных платежей. Хотя мы, возможно, и не увидим улучшения качества прогнозов модели на данных in silico, подобные ограничения действительно смягчают риски нестабильности, переобучения и социологической предвзятости в реальном мире и, вероятно, повышают качество прогнозов in vivo.

Частичная зависимость и индивидуальное условное ожидание

Частичная зависимость (partial dependence) — это устоявшийся и интуитивно понятный метод апостериорного объяснения, который описывает предполагаемое среднее поведение модели в зависимости от значений некоторого входного признака. К сожалению, он ошибочен. Он не способен точно отображать поведение модели при наличии корреляции или взаимодействия между входными признаками и даже может быть злонамеренно изменен. Но поскольку понимание среднего поведения признака в модели является очень важным, ученые разработали несколько методов устранения недостатков частичной зависимости. В частности, метод *накопленного локального эффекта* (*https://oreil.ly/kEIPp*) (accumulated local effect, ALE) является наиболее прямой заменой частичной зависимости и был разработан специально для устранения ее недостатков. Читатели могут попробовать ALE с такими пакетами, как ALEPlot (*https://oreil.ly/7vv4h*) или ALEPython (*https://oreil.ly/To7PF*).

Книги для программистов: https://t.me/free_books_for_it

В следующих примерах мы будем активно использовать еще одну производную частичной зависимости, чтобы получить четкое представление о поведении функций в наших моделях. Впервые представленный в статье Peeking Inside the Black Box: Visualizing Statistical Learning with Plots of Individual Conditional Expectation (*https://oreil.ly/MruUv*), ICE объединяет графики локального поведения модели по отношению к отдельным людям и частичную зависимость. Это позволяет нам сравнивать предполагаемое среднее поведение с описаниями локального поведения, а когда кривые частичной зависимости и ICE расходятся, решить для себя, выглядит ли частичная зависимость заслуживающей доверия или на нее влияют корреляции или взаимодействия во входных переменных. Конечно, ICE не лишен своих недостатков. Наиболее распространенной проблемой с ICE является обработка нереалистичных значений данных, и при интерпретации ICE важно придавать наибольшее значение значениям входного признака, которые наиболее похожи на значения в исходной строке рассматриваемых данных.

Проиллюстрируем сказанное на примере. В нижней части рис. 6.1 вы можете видеть частичную зависимость и ICE для модели логистической регрессии со

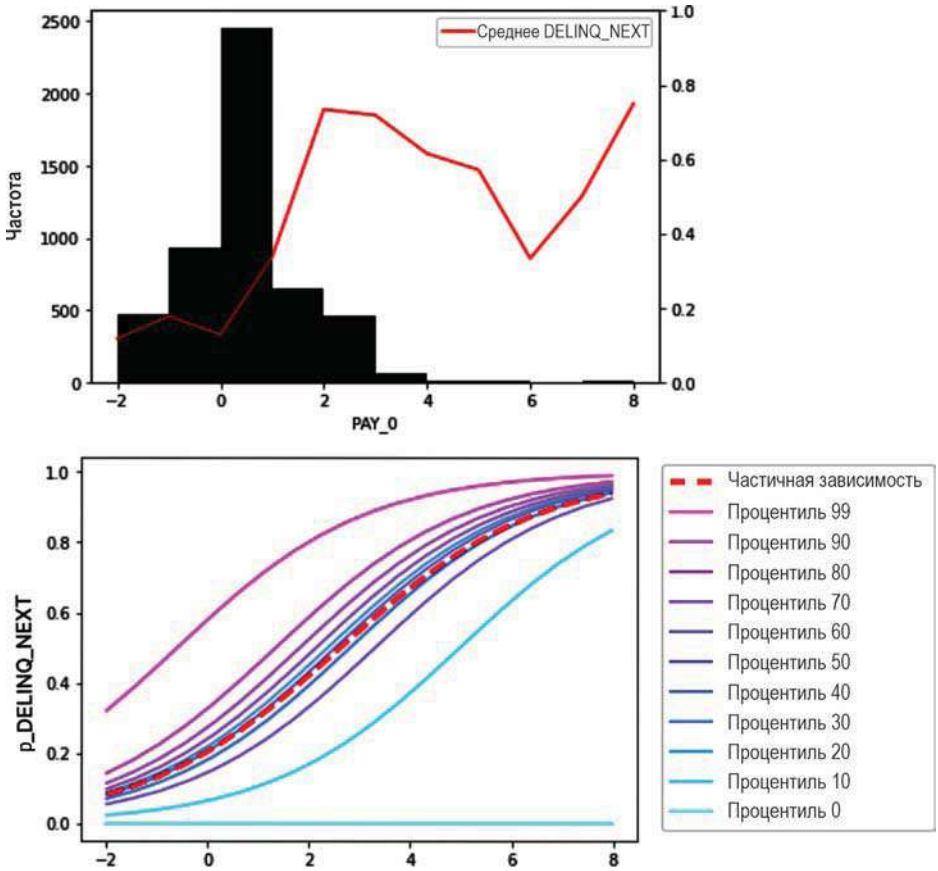


Рис. 6.1. График частичной зависимости для модели GLM, обученной далее в этой главе. На нем представлены ICE, гистограммы и условное среднее

Книги для программистов: https://t.me/free_books_for_it

штрафом и для входного признака PAY_0 (статус погашения последнего счета клиента). Более высокие значения PAY_0 указывают на более длительную задержку платежа. Кривые ICE генерируются для людей, которые находятся на уровне децилей прогнозируемой вероятности.

Обратите внимание на плавный рост от более низкой вероятности неисполнения обязательств, когда клиент не задерживает последний платеж, к более высокой вероятности неисполнения обязательств, когда платеж задерживается. Это вполне логично и соответствует разумным ожиданиям и знанию предметной области. Обратите также внимание, что ICE и частичная зависимость не расходятся — они тесно связаны. Так всегда бывает в линейных моделях, но это также показывает нам, что частичная зависимость, вероятно, заслуживает доверия для этой модели и набора данных.

Но что же происходит в верхней части этого рисунка? Именно здесь мы пытаемся решить, изучает ли модель надежные сигналы на основе обучающих данных. Первое, что бросается в глаза, это гистограмма. Мы используем эту гистограмму для поиска проблем стабильности в прогнозах модели. Модели ML обычно учатся только на данных, поэтому, если данных мало, как в случае с PAY_0 > 1 в наших обучающих данных, модель ML не может многому научиться, и ее прогнозы в этих областях данных будут нестабильными, если не бессмысленными. Некоторые другие пакеты с той же целью используют в графиках частичной зависимости столбцы гистограммы ошибок или функции формы. Это тоже нормально. Оба метода визуализации пытаются привлечь ваше внимание к области данных, где ваша модель машинного обучения нестабильна и, вероятно, принимает ошибочные решения.

Здесь мы пытаемся решить, хорошо ли модель представляет наши обучающие данные, а верхняя часть рисунка также может указать нам на проблемы с разреженностью данных и надежностью прогнозов. Гистограмма в верхней части рисунка нужна для поиска проблем с надежностью в прогнозах модели. Как вы можете видеть, на гистограмму также наложена линия. Эта линия представляет собой условное среднее целевого значения для соответствующего интервала гистограммы. Если модель правильно изучила данные, частичная зависимость и ICE в нижней части рисунка должны примерно повторять условную среднюю линию в верхней части. При оценке того, соответствует ли поведение модели условному среднему целевому значению, следует помнить о возможном наличии разреженности. На рис. 6.1 мы видим резкое падение условного среднего значения при PAY_0 = 6, т. е. на шесть месяцев позже последнего счета. Однако никакие данные не подтверждают это падение. Ячейка гистограммы практически пуста, и падение, вероятно, является просто неуместным шумом. К счастью, у нашей хорошо работающей модели логистической регрессии нет другого выбора, кроме как игнорировать этот шум и продолжать монотонно повышать вероятность дефолта по мере увеличения PAY_0. В более сложных моделях для тех же данных нам придется применять ограничения монотонности, чтобы гарантировать, что модель следует причинно-следственным связям, а не запоминает нерелевантный шум без поддержки данных.

Как говорил выдающийся исследователь объяснимости Пшемыслав Бичек: «Не объясняйте без контекста!» Другими словами, нам нужно подумать о корреляциях,

Книги для программистов: https://t.me/free_books_for_it

взаимодействиях и безопасности наборов данных, которые мы используем для генерации частичной зависимости и ICE. Обычно это валидационные, тестовые или другие примечательные контрольные выборки. Если эти наборы данных не соответствуют корреляциям и взаимодействиям в обучающих данных или выборка могла быть намеренно изменена, мы получим результаты, отличные от тех, которые видели при обучении. Здесь может появиться ряд вопросов. Была ли обучающая частичная зависимость правильной? Действительно ли наша модель ведет себя подругому с новыми данными? Или же новые корреляции и взаимодействия в этой выборке приводят к тому, что частичная зависимость становится менее заслуживающей доверия?

Это причины, по которым мы связываем частичную зависимость с ICE. Как метод локального объяснения, ICE менее восприимчив к глобальным изменениям корреляций и взаимодействий. Если что-то не так с частичной зависимостью, сначала проверьте, соответствует ли частичная зависимость локальному поведению кривых ICE или же отличается от них. Если она расходится с ICE, вероятно, безопаснее взять пояснительную информацию из ICE и, если возможно, выяснить, какие проблемы распределения, корреляции, взаимодействия или безопасности изменяют частичную зависимость.

Значения Шепли

Напомним, что SHAP — это способ генерирования значений атрибуции локальных признаков, имеющий прочную теоретическую основу (по крайней мере, по стандартам ML). Значение SHAP говорит нам, насколько значение признака для определенной строки отклонило прогноз модели от среднего прогноза. Но как SHAP это делает? Он многократно «удаляет» признак из прогноза этой строки совместно с другими удаленными признаками. Удалив признаки и измерив различия в прогнозах модели, мы получаем четкое представление о том, как каждый признак влияет на каждый прогноз.



В главе 2 вы узнали, что значения Шепли — это метод апостериорного объяснения, заимствованный из экономики и теории игр, который разлагает прогнозы модели на вклады каждого входного признака.

Поскольку SHAP может использовать *фоновые* наборы данных, т. е. конкретные наборы данных, из которых можно извлекать случайные выборки для использования в качестве замены удаленных объектов, мы должны учитывать контекст как в наборе данных, который нужно объяснить, так и в фоновом наборе данных. В случае частичной зависимости и ICE мы обычно используем очень простые фоновые наборы данных для этих методов и можем вообще не думать о них как о фоновых наборах данных. По сути, мы просто заменяем значение всего признака (частичная зависимость) или строки (ICE) некоторым известным значением этого признака, чтобы создать кривую. Когда дело доходит до значений Шепли, у нас есть выбор: (1) какие наблюдения мы объясняем (от одной строки до совершенно новой выборки данных) и (2) какой фоновый набор данных использовать при генерации

Книги для программистов: https://t.me/free_books_for_it

значений Шепли (что угодно: отказ от фонового набора, случайные данные или тщательно обработанные фоновые данные, предназначенные для решения проблем контекста или причинно-следственных связей).

Помимо рассмотрения корреляций, взаимодействий и безопасности набора данных, которые мы объясняем, мы также должны задаться вопросом, уместен ли наш выбор фоновых данных и имеют ли объяснения смысл в контексте, в котором они будут оцениваться. Позже в этой главе мы подробно рассмотрим, как выбрать подходящий набор фоновых данных для наших объяснений по методу значений Шепли. Этот выбор будет зависеть от вопроса, на который наши объяснения пытаются ответить. На практике такой сложный анализ часто сводится к вычислению объяснений на нескольких различных наборах данных для обеспечения достоверности и стабильности результатов. Вычисление объяснений на основе значений Шепли также подразумевает документирование используемого фонового набора данных и причин выбора этого набора данных.

Документация модели

Документация модели является физическим воплощением подотчетности в крупных организациях. Если вы знаете, что вам придется написать документацию о построенной вами модели и на обложке будет ваша фамилия, наверняка это будет способствовать более продуманному выбору проекта и его реализации. Если же ваш ошибочный выбор не задокументирован, документация содержит обман или вообще отсутствует, это негативно отразится и на самой модели. Документация модели также важна для технического обслуживания и реагирования на инциденты. Когда вы переходите к следующему проекту по обработке больших данных, а ваши прежние модели начинают устаревать в производстве и вызывать проблемы, документация позволяет новой группе практиков понять, как модель должна была работать, как поддерживать ее в будущих итерациях и как исправить возникшие неполадки.

В настоящее время существует несколько стандартов документации моделей, в том числе следующие:

- карточки моделей (*https://oreil.ly/h7eJC*) с образцами карточек (*https://oreil.ly/OJkfE*), предоставленными Google;
- подробная документация по управлению рисками модели; см. типовое руководство (*https://oreil.ly/XDF9u*) по управлению рисками 2021 г. от Управления контролера денежного обращения США;
- шаблон документации (*https://oreil.ly/tyS-i*) из Закона ЕС об искусственном интеллекте; см. Документ 2, Приложение IV.

Обратите внимание, что все эти шаблоны созданы либо ведущим коммерческим пользователем и разработчиком машинного обучения, либо очень серьезным государственным органом. Если вам до сих пор удавалось избегать документирования модели, имейте в виду, что в будущем ситуация изменится по мере принятия новых правил, особенно для важных приложений ML. Все эти шаблоны намного легче

Книги для программистов: https://t.me/free_books_for_it

заполнять для объяснимых моделей и апостериорных объяснений, поскольку повышение прозрачности является еще одной целью и преимуществом документирования моделей. Прозрачность моделей машинного обучения позволяет понять, а затем обосновать компромиссы при проектировании и реализации. Если выводы объяснимой модели или апостериорные объяснения кажутся разумными и вы можете рационально обосновать наблюдаемые результаты, значит, ваша работа соответствует стандартам, описанным в текущей главе. Если вместо этого вы используете необъяснимую модель и не понимаете, как компромиссы в проектировании и реализации влияют на поведение модели, ваше документальное обоснование, вероятно, будет намного слабее и может привлечь нежелательное внимание контролирующих органов.

Семейство объяснимых моделей GAM

В этом разделе мы сформируем эталонный образец в виде модели линейной аддитивной регрессии со штрафом, а затем сравним его с моделью GAM, которая допускает сложную нелинейность, но независимым, аддитивным и вполне объяснимым способом. Затем мы сравним GLM и GAM с EBM с небольшим количеством двусторонних взаимодействий. Поскольку все наши модели построены с использованием аддитивных независимых функциональных форм и поскольку мы будем использовать лишь небольшое количество значимых взаимодействий, все наши модели будут очень объяснимы. Аддитивность позволит нам сделать четкий и обоснованный выбор относительно введения нелинейности и взаимодействий.



Переход от GLM к GAM и EBM — это обобщаемый рабочий процесс, который позволяет нам принимать объяснимые, эмпирические и взвешенные решения о введении нелинейности (через GAM) и взаимодействий (через EBM) в наши модели при сравнении результатов с эталоном (GLM).

Elastic Net — штрафуемая GLM с альфа- и лямбда-поиском

Как следует из названия, GLM (generalized linear model, обобщенная линейная модель) расширяют идею обычной линейной регрессии и обобщают распределения ошибок, принадлежащие экспоненциальным семействам, в дополнение к гауссовскому распределению ошибок, используемому в стандартной линейной регрессии. Еще одним важным компонентом GLM является функция связи, которая связывает ожидаемое значение отклика с линейными компонентами. Поскольку эта функция связи может быть любой монотонной дифференцируемой функцией, GLM могут обрабатывать самые разные распределения обучающих данных: линейное, биномиальное (как в нашем текущем примере), Пуассона и некоторые другие. Наличие штрафа GLM подразумевает использование сложных ограничений и методов итеративной оптимизации для обработки корреляций, выбора признаков и выбросов. Объединение этих подходов дает надежную технику моделирования с хорошей предсказательной силой и очень высокой объяснимостью.

Elastic Net — это популярный метод регуляризации, который сочетает в себе преимущества регрессии L1 (LASSO (*https://oreil.ly/BqHjO*)) и L2 (гребневая, или

Книги для программистов: https://t.me/free_books_for_it

ридж-регрессия (*https://oreil.ly/ORzCT*)) в одной модели. В то время как регуляризация L1 позволяет выбирать признаки, тем самым вызывая разреженность и более высокую объяснимость обученной модели, регуляризация L2 эффективно обрабатывает корреляцию между предикторами. Elastic Net часто сочетается с методом *наименьших квадратов с итеративным пересчетом весов* (iteratively reweighted least squares, IRLS) для обработки выбросов.

Обучение GLM со штрафом послужит двум полезным целям сравнительного анализа:

- поскольку наша GLM не включает в себя какие-либо нелинейности или взаимодействия функций, она может служить идеальным эталоном для проверки определенных гипотез, т. е. того, приводят ли нелинейности и взаимодействия к лучшей модели или нет, о чем мы поговорим в следующих разделах;
- GLM также выступает в качестве отправной точки для первоначального выбора признаков на основе выбора, сделанного с помощью регуляризации L1.

Мы начнем наш первый пример в этой главе с обучения логистической регрессии со штрафом с использованием алгоритма GLM H2O (*https://oreil.ly/bI_dI*), который работает по распределенному принципу и хорошо масштабируется для больших наборов данных. В GLM H2O параметры регуляризации обозначаются как `alpha` и `lambda`. В то время как `alpha` указывает распределение регуляризации между штрафами L1 и L2, `lambda` указывает на силу регуляризации. Рекомендуемый способ нахождения оптимальных настроек регуляризации в GLM H2O — использовать поиск по сетке. H2O предлагает два типа поиска по сетке: декартов и случайный. Первый — это исчерпывающий поиск, при котором перебираются все комбинации гиперпараметров модели, указанные в сетке возможных значений, предоставленных пользователем. В свою очередь случайный поиск по сетке выбирает наборы параметров модели случайным образом из заданного набора возможных значений на основе критерия остановки. По умолчанию H2O использует декартов поиск, и мы тоже будем использовать его в нашем случае, поскольку поиск по небольшому количеству значений `alpha` не займет слишком много времени.



Всякий раз, когда мы проводим поиск по сетке, мы неявно сталкиваемся с проблемами, связанными с переобучением и множественными сравнениями. Если это возможно, попробуйте сочетать с поиском по сетке методы бутстреппинга (bootstrapping) или многократного откладывания (reusable holdout).

В следующем коде мы начинаем с определения сетки гиперпараметров модели для значений `alpha`. Здесь важно отметить, что для сохранения стабилизирующей функциональности штрафов L2 и функциональности выбора признаков штрафов L1 значение `alpha` никогда не должно быть равно 0 или 1. Дело в том, что когда значение `alpha` равно 0, это означает только штраф L2, а значение 1 означает только штраф L1. Реализация GLM от H2O поставляется с удобной функцией `lambda_search`. Если установлено значение `True`, этот параметр запускает поиск по различным значениям `lambda`, начиная с `lambda_max` (нет признаков в модели) до `lambda_min` (много признаков в модели). И `alpha`, и `lambda` выбираются путем ранней остановки на основе проверки. Это означает, что GLM автоматически прекратит обучение, если на валидаци-

Книги для программистов: https://t.me/free_books_for_it

онном наборе не будет наблюдаться значительного улучшения. Это средство борьбы с переобучением.

```
def glm_grid(x, y, training_frame, validation_frame, seed_, weight=None):
    # настройка параметров сетки GLM
    alpha_opts = [0.01, 0.25, 0.5, 0.99] # всегда ненулевое значение alpha
    hyper_parameters = {'alpha': alpha_opts}

    # инициализация поиска по сетке
    glm_grid = H2OGridSearch(
        H2OGeneralizedLinearEstimator(family="binomial",
                                      lambda_search=True,
                                      seed=seed_),
        hyper_params=hyper_parameters)

    # обучение с поиском по сетке
    glm_grid.train(y=y,
                   x=x,
                   training_frame=training_frame,
                   validation_frame=validation_frame,
                   weights_column=weight,
                   seed=seed_)

    # выбираем лучшую модель из поиска по сетке
    best_model = glm_grid.get_grid()[0]
    del glm_grid
    return best_model
```

Выполнив с помощью этой функции декартов поиск `alpha` и указав H2O искать лучшие значения `lambda`, мы получили лучшую GLM с показателем AUC 0,73 на наборе проверочных данных. После поиска по сетке шесть признаков статуса погашения `PAY_*` имеют самые большие коэффициенты в выбранной модели.



Показатель AUC, равный 0,73, означает, что в 73% случаев наша модель правильно ранжирует случайно выбранную положительную строку с более высокой выходной вероятностью, чем вероятность случайно выбранной отрицательной строки.

Чтобы понять, как модель обрабатывает различные признаки, мы строим частичную зависимость вместе с графиками ICE для интересующих признаков. Кроме того, рядом отображается гистограмма интересующего признака, включая наложение среднего значения целевого столбца, т. е. `DELINQ_NEXT`. Это должно дать нам хорошее представление о том, разумно ли ведет себя модель и могут ли какие-либо проблемы с разреженностью данных привести к бессмысленным прогнозам.

Вернемся к рис. 6.1. Признак `PAY_0` имеет самую крутую частичную зависимость и кривую ICE, что позволяет предположить, что это наиболее важный входной признак. Графики частичной зависимости и ICE находятся в гармонии, т. е. не расходятся, а это означает, что на частичную зависимость можно положиться. Кроме

Книги для программистов: https://t.me/free_books_for_it

того, существует монотонно возрастающая зависимость прогнозируемой вероятности дефолта и задержки платежа PAY_0. Это означает, что по мере увеличения задержки платежа вероятность того, что клиент не выполнит свои обязательства, также становится выше. Такая картина соответствует нашему интуитивному пониманию принципов работы платежей по кредитным картам.

Теперь рассмотрим гистограмму в верхней части. У клиентов с просроченными платежами возникают некоторые очевидные проблемы с нехваткой данных. Например, в регионах, где PAY_0 > 1, данных для обучения мало или вообще нет. Кроме того, средние значения DELINQ_NEXT демонстрируют некоторые нелинейные закономерности в этом регионе. Совершенно очевидно, что прогнозы, сделанные в этих регионах, будут менее достоверными. В конце концов, такая стандартная модель машинного обучения может учиться только на данных, если мы не предоставим ей дополнительные знания в предметной области. Однако хорошая новость заключается в том, что логистическая форма нашей GLM со штрафом предотвращает ее обман не только из-за провалов степени достоверности условного среднего DELINQ_NEXT вокруг PAY_* = 6, но и из-за обучения на шуме в этих областях скудных обучающих данных. Модель обрабатывает другие признаки PAY_* аналогичным образом, но присваивает им более пологие логистические кривые. Во всех случаях вероятность дефолта монотонно возрастает с просрочкой платежа, как и ожидалось. Чтобы увидеть другие частичные графики, графики зависимостей и ICE, ознакомьтесь с примерами кода для этой главы.

Теперь у нас есть надежная и объяснимая эталонная модель. Поскольку ее поведение имеет простой и ясный смысл и ее так легко интерпретировать, нам будет трудно ее превзойти. В значении AUC на проверочных данных, равном 0,73, нет ничего примечательного, но наличие объяснимой модели, работающей в соответствии с проверенными временем причинно-следственными связями, на которые мы можем рассчитывать после внедрения, — это бесценно с точки зрения снижения рисков. Мы также должны помнить, что оценки на проверочных и тестовых данных могут вводить в заблуждение в более сложных моделях машинного обучения. Мы можем получить высокое значение AUC на статических валидационных или тестовых данных и вскоре обнаружить, что это значение возникло в результате переобучения на каком-то конкретном явлении, которое больше не присутствует в нашей операционной деятельности. В следующем разделе мы сначала представим некоторые нелинейности с помощью GAM, а потом рассмотрим взаимодействия признаков в дополнение к нелинейностям с помощью EBM. Затем мы оценим нашу модель на предмет объяснимости и качества прогнозов, ориентируясь на реальные показатели. Мы постараемся провести честные эксперименты и сделать осознанный выбор исходя из того, оправдано ли увеличение сложности.

Обобщенные аддитивные модели

Хотя линейные модели легко поддаются интерпретации, они не могут точно отразить нелинейности, обычно присутствующие в реальных наборах данных. Здесь в игру вступают GAM. Эти модели, первоначально разработанные в Стэнфорде (*https://oreil.ly/tl_oq*) в конце 1980-х годов выдающимися статистиками Тревором

Книги для программистов: https://t.me/free_books_for_it

Хасти и Робом Тибширани, моделируют нелинейные связи каждого входного признака с помощью отдельных сплайн-функций и складывают их все вместе, чтобы создать окончательную модель. GAM можно рассматривать как аддитивную комбинацию сплайн-функций. Важная идея GAM заключается в том, что, хотя мы обрабатываем каждый признак очень сложным образом, это делается аддитивно и независимо. Такой подход не только сохраняет объяснимость, но и позволяет относительно легко редактировать и отлаживать модель.

Если говорить о реализации GAM, то в R отличными вариантами являются такие пакеты, как gam (*https://oreil.ly/mt1ty*) и mgcv (*https://oreil.ly/SW3rz*). Что касается Python, выбор ограничен, поскольку большинство пакетов находятся на экспериментальной стадии, как, например, реализация GAM (*https://oreil.ly/_ak0k*) от H2O. Альтернативный пакет pyGAM (*https://oreil.ly/dZ9tU*) черпает вдохновение из пакета R mgcv; было показано, что он предлагает хорошее сочетание точности, надежности и скорости и имеет знакомый API, похожий на scikit.

Мы будем использовать pyGAM для обучения GAM на том же наборе данных о кредитных картах, который мы использовали в последнем разделе. В частности, с помощью следующего кода мы реализуем логистическую модель, используя класс `LogisticGAM` pyGAM. Для получения наилучшей модели можно настроить три важных параметра: количество сплайнов; `lam`, или величина штрафа за регуляризацию; и ограничения для внедрения априорных знаний в модель. pyGAM предоставляет встроенный метод поиска по сетке для автоматического поиска параметров сглаживания.

```
from pygam import LogisticGAM
gam = LogisticGAM(max_iter=100, n_splines=30)
gam.gridsearch(train[features].values, train[target], lam=np.logspace(-3, 3, 15))
```

Этот код создает экземпляр модели `LogisticGAM`, которая будет обучаться до 100 итераций. Параметр `n_splines` определяет количество членов сплайна или сложность функции, используемой для изучения каждого входного признака. Большее количество членов сплайна обычно приводит к более сложным сплайн-функциям. Параметр `lam` в некоторой степени соответствует `lambda` в штрафной регрессии, и предыдущий код выполняет поиск по нескольким значениям, чтобы найти наилучшую силу регуляризации, определенную `lam`. Здесь мы не используем параметр `constraints`. Он позволяет пользователям указывать список ограничений для кодирования априорных знаний. Доступными ограничениями являются монотонное возрастающее или убывающее сглаживание, а также выпуклое или вогнутое сглаживание. Позже в этой главе мы будем работать с аналогичными ограничениями; очень поучительно посмотреть, что означает отказ от использования ограничений для нашей GAM.

В этом примере мы пытаемся ответить на несколько надуманный вопрос: действительно ли нелинейности полезны для нашей модели или это просто переобучение на шуме? Многие прикладные специалисты по работе с данными сегодня предполагают, что более качественные модели сложнее устроены, но мы проведем эксперимент с GAM, чтобы установить, действительно ли введение нелинейности улуч-

Книги для программистов: https://t.me/free_books_for_it

шает нашу модель как с точки зрения качества прогнозов, так и с точки зрения интерпретируемости.

После обучения нашей модели мы вычисляем ее значение AUC на проверочных данных, которое оказывается равным 0,75 — заметный прирост по сравнению с нашей GLM со штрафом. Увеличение AUC, вероятно, можно объяснить появлением нелинейности, которую наша GLM не смогла уловить. Однако здесь важно отметить, что более высокое значение AUC не всегда гарантирует лучшие модели, и этот пример является тому классическим подтверждением. В предыдущем разделе мы потратили немного времени на анализ того, как GLM реагирует на признак `PAY_0` или последний статус погашения клиента, и этот подход сработал достаточно хорошо. Давайте теперь посмотрим, как GAM реагирует на признак `PAY_0` (рис. 6.2).

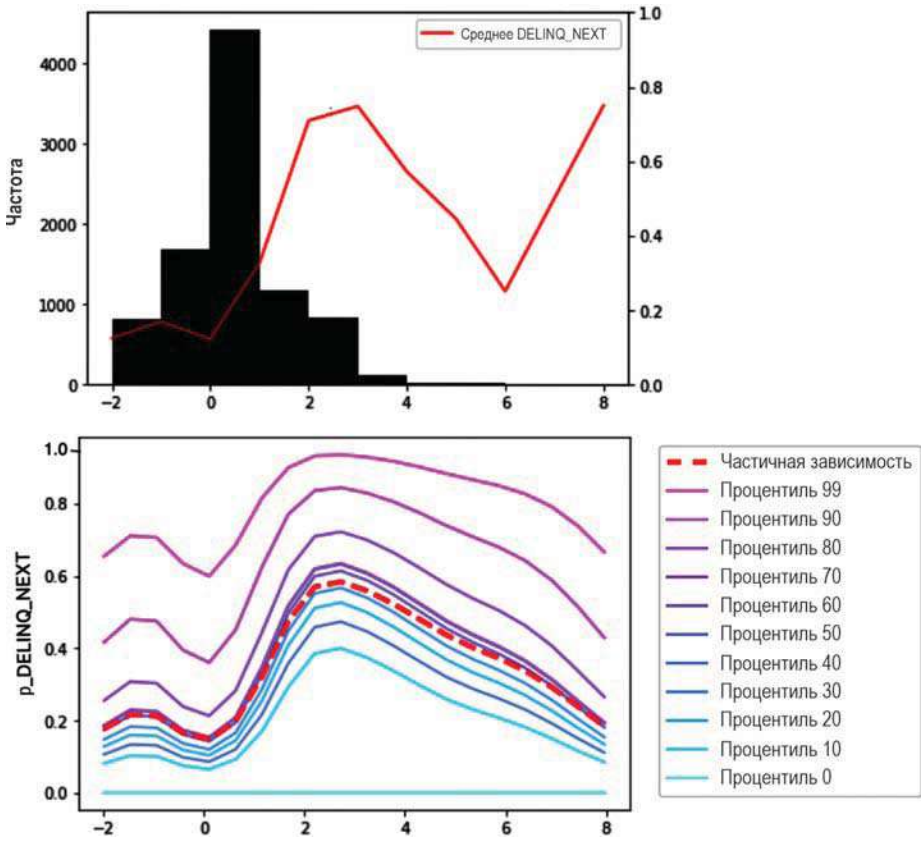


Рис. 6.2. График частичной зависимости для модели GAM. На нем представлены ICE, гистограммы и условное среднее

На рис. 6.2 видно, что в графиках частичной зависимости и ICE, построенных с помощью GAM, явно есть некоторая странность. Мы наблюдаем, что по мере увеличения просрочки платежа вероятность того, что клиент не выполнит платеж, уменьшается. Это неправильно. Обычно люди не приобретают волшебную способность вносить платежи после нескольких месяцев просрочки. Такое же странное

Книги для программистов: https://t.me/free_books_for_it

поведение наблюдается и для PAY_4 и PAY_6, как видно на рис. 6.3. Вероятность дефолта PAY_4, похоже, снижается по мере увеличения задержки платежей, а PAY_6, судя по всему, зашумленно удаляется от среднего прогноза. Оба смоделированных поведения противоречат здравому смыслу, оба противоречат базовой модели GLM и оба не могут смоделировать условное среднее поведение, показанное в правой части рис. 6.3.

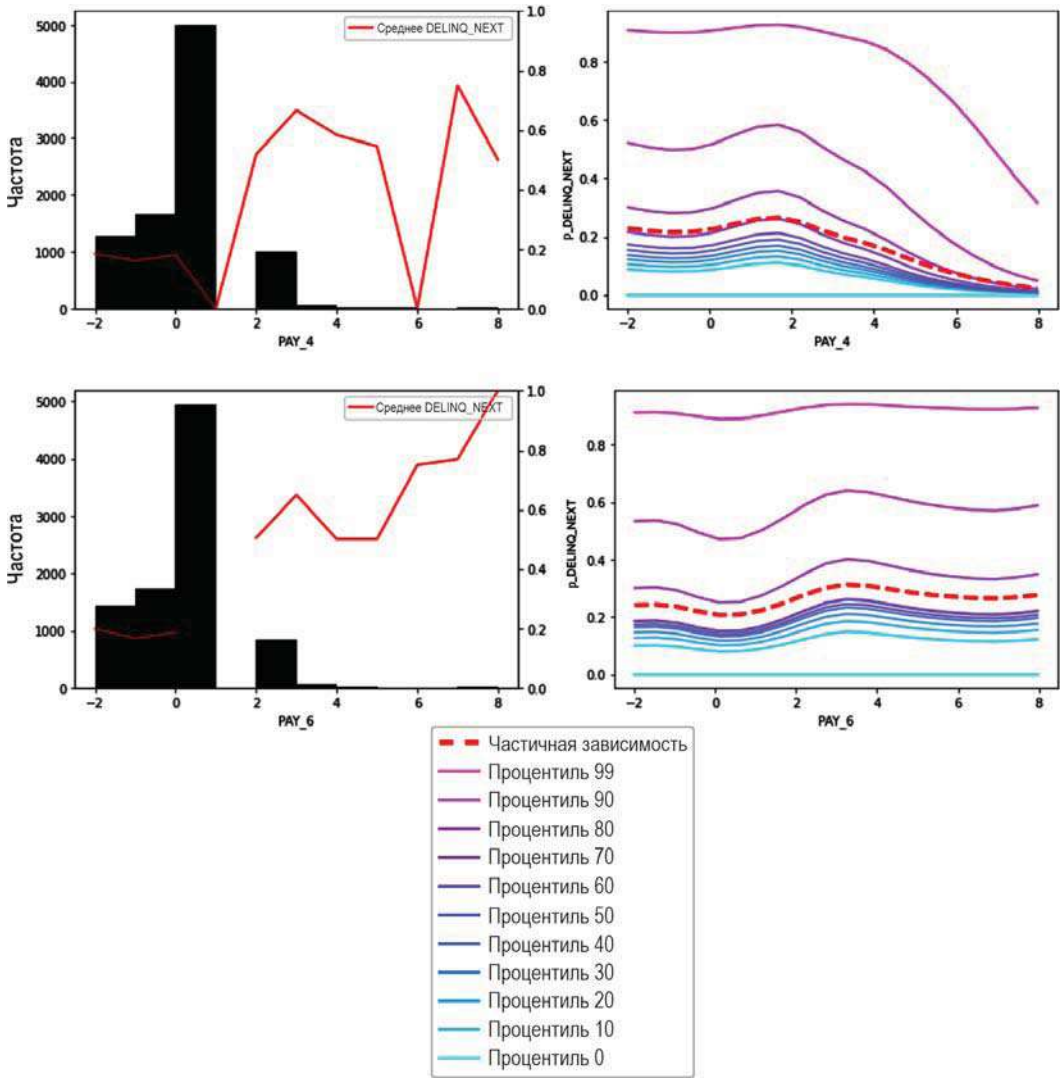


Рис. 6.3. График частичной зависимости, включающий ICE, гистограммы и условные средние для PAY_4 и PAY_6

Суть в том, что, хотя значение AUC на проверочных данных выше, нам явно не следует использовать эту модель. Как видно из рис. 6.2, GAM либо переобучилась на обучающих данных, либо введена в заблуждение провалами низкой достоверно-

Книги для программистов: https://t.me/free_books_for_it

сти условного среднего DELINQ_NEXT около PAY_* = 6, либо возвращается к среднему прогнозу из-за разреженности данных для PAY_0 > 1.

Какой же здесь выход и как нам использовать такую модель? Что ж, именно здесь GAM и проявляют себя в лучшем виде. Поведение, которое демонстрирует модель, в данном случае является общей проблемой, наблюдаемой для нелинейных моделей относительно высокой сложности. Однако, в отличие от многих других типов моделей, GAM не только выявляют подобные несоответствия, но и предлагают способы их устранения посредством редактирования модели исходя из здравого смысла. Проще говоря, мы можем обсудить результаты GAM с экспертами в предметной области, и если они согласятся с более правдоподобными сплайнами GAM для PAY_2, PAY_3 и PAY_5, мы могли бы сохранить их в модели и, возможно, повысить качество прогнозов. Что касается явно проблемных сплайнов для PAY_0, PAY_4 и PAY_6, то их можно заменить на что-нибудь более разумное. Одним из вариантов является их поведение, изученное моделью логистической регрессии, как показано в следующем выражении:

$$\hat{p} = \beta_0 + \frac{1}{1 + \exp\left(-\beta_{PAY_0, GLM} PAY_0\right)} + \beta_{PAY_2, GAM} g(PAY_2)$$
$$+ \beta_{PAY_3, GAM} g(PAY_3) + \frac{1}{1 + \exp\left(-\beta_{PAY_4, GLM} PAY_4\right)} + \beta_{PAY_5, GAM} g(PAY_5)$$
$$+ \frac{1}{1 + \exp\left(-\beta_{PAY_6, GLM} PAY_6\right)} + \cdots,$$

где β_0 — член пересечения, а каждый g представляет собой сплайн-функцию GAM. Редактирование модели является бесконечно гибким; мы могли бы заменить изученный сплайн только в определенной части пространства данных или иным образом отредактировать функцию формы по совету эксперта в предметной области.

Редактируемость — отличная функция прогнозной модели, но при работе с ней нужно соблюдать осторожность. Если вы хотите отредактировать пользовательскую модель, как это предложено в предыдущем уравнении, ее необходимо подвергнуть более строгому стресс-тестированию, чем обычно. Не будем забывать, что коэффициенты не изучались вместе и могут плохо объяснять друг друга. Также могут возникнуть проблемы с границами — отредактированная модель может легко выдать прогнозы выше 1 и ниже 0. Другая потенциально более приемлемая стратегия отладки — использовать функциональные возможности ограничений, предоставляемые pyGAM. Ограничение положительной монотонности, скорее всего, устранит проблемы в сплайнах PAY_0, PAY_4 и PAY_6.

Независимо от того, решим ли мы отредактировать пример GAM или же переобучить его с учетом ограничений, мы, скорее всего, увидим более низкое качество прогнозов на проверочных и тестовых данных. Однако, если нас больше всего беспокоит качество в реальной жизни, нам нужно перестать поклоняться алтарю показателей при тестировании. Хотя идея редактирования модели может показаться странной, исправленная модель явно ближе к реальности. Что нам кажется более странным, так это использование моделей, поведение которых оправдывается лишь несколькими строками обучающих данных с высоким уровнем шума, что явно про-

Книги для программистов: https://t.me/free_books_for_it

тиворечит десятилетиям опыта, основанного на здравом смысле. Мы готовы поспорить, что правильно отредактированная модель с гораздо меньшей вероятностью приведет к катастрофическому сбою, чем бессмысленные сплайны, полученные с помощью GAM без ограничений.

Это лишь один из многих сценариев, когда традиционные способы оценки могут ввести в заблуждение, если речь идет о выборе лучшей модели для реальной жизни. Как было показано на примере GAM, мы не можем быть уверены, что нелинейности улучшают модели. Более того, GAM позволяют нам проверить неявную гипотезу о том, что нелинейность лучше. С помощью GAM мы можем создать модель, интерпретировать и проанализировать результаты, а затем отредактировать ее или устранить любые обнаруженные проблемы. GAM помогают нам узнать, чему научилась модель, сохранить правильные результаты, а также отредактировать и исправить неправильные, чтобы не развертывать в производстве рискованные модели.

GA2M и объяснимые бустинговые машины

Когда к стандартной GAM добавлена небольшая группа взаимодействующих пар признаков, полученная модель называется GA2M — обобщенная аддитивная модель с двусторонним взаимодействием. Было показано, что добавление этих парных взаимодействий к традиционным GAM существенно повышает качество прогнозов модели, сохраняя при этом объяснимость, как обсуждалось в главе 2. Кроме того, как и GAM, модели GA2M легко редактируются.

EBM (*https://oreil.ly/_tS2Q*) — это быстрая реализация алгоритма GA2M от Microsoft Research. Функции формы в EBM обучаются итеративно посредством бустинга, что делает обучение EBM более робастным, сохраняя при этом точность, сравнимую с необъяснимыми древовидными моделями, такими как случайный лес и XGBoost. EBM поставляется в составе более широкого набора инструментов машинного обучения под названием InterpretML (*https://oreil.ly/Uofrw*) — пакета с открытым исходным кодом для обучения объяснимых моделей и объяснения других систем.

Мы продолжим рассматривать пример с платежами по кредиту и научим EBM предсказывать, какой клиент имеет высокий шанс просрочить следующий платеж. На проверочных данных EBM достигает значения AUC, равного 0,78, что является самым высоким показателем по сравнению с традиционными GAM и GLM. Повышение точности, вероятно, связано с появлением нелинейности и взаимодействий. EBM и GA2M тоже легко поддаются объяснению. Как и в случае традиционных GAM, мы можем построить функции формы отдельных признаков и сопровождающие их гистограммы, описывающие поведение модели и распределение данных для этого признака соответственно. Члены взаимодействия можно отобразить в виде контурного графика, который по-прежнему легко понять. Давайте углубимся в анализ и посмотрим, как EBM обрабатывает признаки `LIMIT_BAL`, `PAY_0` и `PAY_2` (рис. 6.4).

На рис. 6.4 показаны три стандартных графика функции формы для `LIMIT_BAL`, `PAY_0` и `PAY_2`, но мы также можем видеть контурный график (изолинии) для взаимодействия

Книги для программистов: https://t.me/free_books_for_it

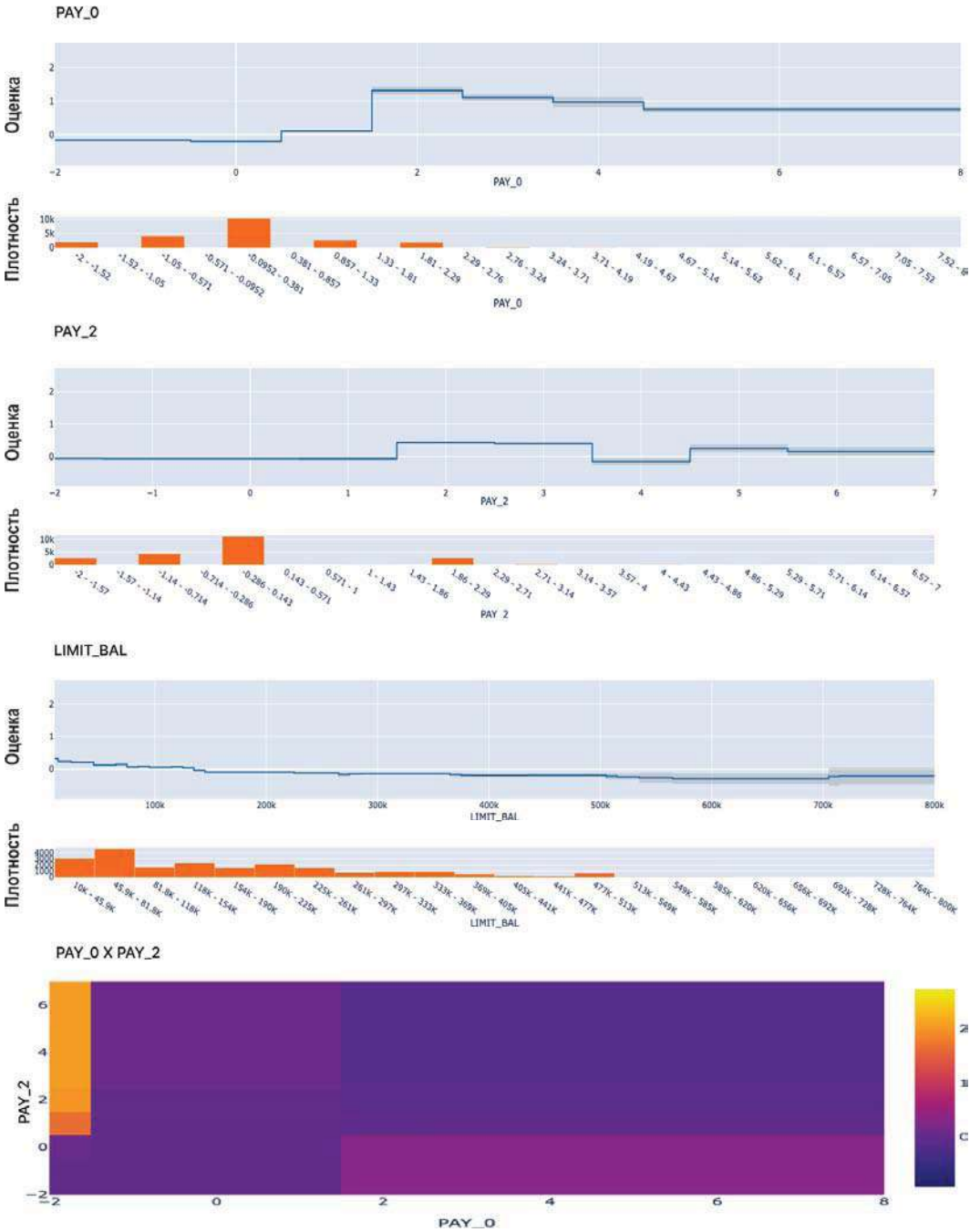


Рис. 6.4. Три важных входных признака и признак взаимодействия в сопровождении гистограмм для EBM

Книги для программистов: https://t.me/free_books_for_it

PAY_0 x PAY_2. Каждый из этих графиков, а также немного более сложный контурный график позволяют нам проверять поведение модели и при необходимости редактировать ее. Поведение LIMIT_BAL выглядит разумным, поскольку ожидается, что увеличение кредитных лимитов будет коррелировать с уменьшением вероятности дефолта. Это то, что мы наблюдаем, по крайней мере, до высоких диапазонов LIMIT_BAL в обучающих данных. Выше $700 000 мы видим, как функция формы снова поворачивается вверх, что, вероятно, связано с разреженностью обучающих данных в этом регионе. EBM обрабатывает PAY_0 более логично, чем наш GAM. В рамках EBM вероятность дефолта относительно PAY_0 увеличивается при PAY_0 > 1 и не проваливается до нереалистичных значений, но все равно снижается. Опять же, это, вероятно, связано с разреженностью определенных областей обучающих данных. Отклик на PAY_2 кажется немного шумным. Кроме того, член взаимодействия демонстрирует такое же нереалистичное поведение, которое наблюдалось в GAM для некоторых отдельных признаков PAY_* — увеличение задержки приводит к снижению вероятности дефолта, за исключением низких значений PAY_0 и высоких значений PAY_2, когда модель ракетой взлетает вверх. Как и GAM, EBM, похоже, демонстрирует странное поведение, вызванное шумом и разреженностью обучающих данных. Это может быть еще одной причиной, почему AUC модели EBM выше — в некоторых случаях она моделирует шум, специфичный для этого набора данных. По крайней мере, странное поведение очевидно, и эта модель может быть хорошим кандидатом на редактирование. Здесь могли бы помочь ограничения монотонности, обсуждаемые в следующем разделе, но мы еще не знали о них при интерпретации EBM.

Есть два других очень важных аспекта рис. 6.4, которые не обязательно характерны для EBM, но также заслуживают пристального внимания: заштрихованные области вокруг функций формы и гистограммы под функциями формы. Оба этих показателя помогают пользователям определить уровень *доверительной надежности* (trustworthiness) модели. Если гистограммы показывают, что в определенном регионе доступно мало обучающих данных, или заштрихованные полосы ошибок показывают, что функция имеет высокую дисперсию в определенной области обучающих данных, эта часть функции, вероятно, менее надежна, и стоит подумать о редактировании модели. Функция формы для LIMIT_BAL более $700 000 является примером как разреженности обучающих данных, так и высокой дисперсии прогнозов. Эти две проблемы часто идут рука об руку при обучении, объяснении и отладке моделей ML.

Еще один довольно сложный аспект работы с EBM — это доступ к информации для построения графиков. Хотя EBM предоставляет потрясающие возможности интерактивного построения графиков «из коробки», нам часто приходится создавать собственные графики или структуры данных, особенно для сравнения с другими моделями. Мы решили несколько раз воспользоваться структурой JSON _internal_obj. Рассмотрим, к примеру, доступ к показателям значимости признаков:

```
ebm_global = ebm.explain_global(name='EBM')
feature_names = ebm_global._internal_obj['overall']['names']
feature_importances = ebm_global._internal_obj['overall']['scores']
```

Книги для программистов: https://t.me/free_books_for_it

```
ebm_variable_importance = pd.DataFrame(zip(feature_names,
                                           feature_importances),
                                       columns=['feature_names',
                                       'feature_importance'])
```

Чтобы извлечь значимость признака вместо того, чтобы полагаться на график EBM по умолчанию в версии пакета интерпретации, с которой мы работали, нам пришлось вычислить глобальные объяснения с помощью `explain_global()`, а затем извлечь имена признаков и оценки значимости из JSON внутри возвращаемого объекта. Затем мы использовали эту информацию для создания `DataFrame` Pandas, а уже оттуда можно легко выполнить большинство стандартных операций, таких как построение графика, выбор или манипулирование[1].

На этом мы завершим анализ первого набора примеров. Мы получили эталонную модель GLM, затем намеренно ввели нелинейности через GAM и взаимодействия через GA2M и EBM, что усложнило наши модели. Однако благодаря аддитивному характеру GLM, GAM и EBM мы не только сохранили объяснимость и автоматически улучшили качество прогнозов, но и создали набор редактируемых моделей, которые мы можем сравнивать друг с другом и даже комбинировать, чтобы создать наилучшую возможную модель для развертывания в реальном приложении. В следующем разделе мы продолжим эти темы и углубимся в ограничения и апостериорные объяснения XGBoost.

XGBoost с ограничениями и апостериорными пояснениями

В этом примере мы обучим и сравним две модели классификатора XGBoost — одну с монотонными ограничениями, а другую без них. Забегая вперед, можно сказать, что модель с ограничениями более надежна и не менее точна, чем неограниченная. Затем мы рассмотрим три мощных метода апостериорного объяснения — суррогаты дерева решений, частичную зависимость и значения ICE и SHAP. Мы завершим раздел обсуждением расчетов значений SHAP и базовых наборов данных, а также предоставим рекомендации, чтобы читатели могли выбрать подходящие методики для конкретного применения.

XGBoost с ограничениями и без ограничений

XGBoost — невероятно популярная архитектура моделей для задач прогнозирования на больших структурированных наборах данных. Так что же такое модель XGBoost? Модели, созданные XGBoost, представляют собой ансамбли *слабых моделей-учеников* (weak learner). То есть XGBoost создает последовательность из множества небольших моделей, а затем, чтобы сделать окончательный прогноз, суммирует прогнозы этих моделей. Обычно первая модель в последовательности

[1] Это неудобство устранено в текущих версиях пакета. Подробности описаны в документации.

Книги для программистов: https://t.me/free_books_for_it

обучается непосредственно на данных. А каждая последующая модель использует результаты работы предыдущих моделей, чтобы скорректировать их ошибки[2]. В этом разделе мы будем использовать XGBoost для обучения ансамбля неглубоких деревьев решений. Мы будем работать с задачей бинарной классификации, но XGBoost можно использовать для моделирования других типов задач, таких как регрессия, многоклассовая классификация и прочих.

XGBoost настолько популярен отчасти потому, что он обычно создает надежные модели, которые хорошо обобщают незнакомые данные. Это не значит, что мы, разработчики моделей, можем успокоиться и «уснуть за рулем». Мы должны использовать разумные гиперпараметры и методы, такие как ранняя остановка, чтобы гарантировать реализацию сильных сторон XGBoost. Еще один важный метод, который позволяет нам применять XGBoost, — это ограничения монотонности наших моделей. Эти ограничения фиксируют направление взаимосвязи между определенными признаками и выходными данными модели. Они позволяют нам сказать: «Если признак `X_1` увеличивается, то выходные данные модели не могут уменьшиться». Иными словами, эти ограничения позволяют нам применять наши собственные знания предметной области для создания более надежных моделей. Так выглядит пример кода для обучения модели XGBoost:

```
params = {
    'objective': 'binary:logistic',
    'eval_metric': 'auc',
    'eta': 0.05,
    'subsample': 0.75,
    'colsample_bytree': 0.8,
    'max_depth': 5,
    'base_score': base_score,
    'seed': seed
}

watchlist = [(dtrain, 'train'), (dvalid, 'eval')]

model_unconstrained = xgb.train(params,
                                dtrain,
                                num_boost_round=200,
                                evals=watchlist,
                                early_stopping_rounds=10,
                                verbose_eval=True)
```

Сначала давайте посмотрим на значения в словаре `params`. Параметр `eta` — это *скорость обучения* (learning rate) модели. В градиентном бустинге каждое дерево, которое мы добавляем в наш ансамбль, аналогично шагу градиентного спуска. Чем больше значение `eta`, тем больше каждое дополнительное дерево влияет на нашу модель. Чем меньше значение `eta`, тем меньший вес придается отдельным деревьям решений в последовательности бустинга. Если бы мы использовали `eta = 1.0`, окон-

[2] О градиентном бустинге более подробно говорится в разделе «Основы статистического обучения» главы 10.

Книги для программистов: https://t.me/free_books_for_it

чательный прогноз нашей модели был бы невзвешенной суммой результатов отдельных деревьев решений и почти наверняка означал бы переобучение на обучающих данных. Обязательно установите разумную скорость обучения (скажем, от 0,001 до 0,3) при обучении XGBoost или других моделей градиентного бустинга.



XGBoost также предлагает ограничения взаимодействия (*https://oreil.ly/NR9bo*), позволяющие контролировать взаимное влияние входных признаков в модели. Это еще один простой метод внесения опыта предметной области в модели ML. Ограничения взаимодействия могут быть особенно полезны для смягчения предвзятости за счет исключения известных косвенных взаимодействий для пола или расы, таких как комбинации имени, возраста или почтового индекса.

Параметры `subsample` и `colsample_bytree` также защищают от переобучения. Оба они гарантируют, что каждое отдельное дерево решений не видит весь набор обучающих данных. В данном случае каждое дерево видит случайные 75% строк обучающих данных (`subsample = 0.75`) и случайные 80% столбцов обучающих данных (`colsample_bytree = 0.8`). Далее у нас есть некоторые параметры, которые определяют размер окончательной модели. Параметр `max_depth` — глубина деревьев в модели. Более глубокие деревья охватывают больше взаимодействий между признаками и создают более сложные функции отклика, чем мелкие деревья. Обычно нам нужно, чтобы наши деревья были неглубокими при обучении XGBoost и других моделей GBM — в конце концов, преимущество этих моделей заключается именно в том, что они представляют собой ансамбль слабых моделей-учеников. Конечно, лучшими методами выбора этих значений являются поиск по сетке и другие структурированные методы выбора гиперпараметров, но их рассмотрение выходит за рамки данной главы.

Наконец, в предыдущем фрагменте кода мы обучаем модель, используя раннюю остановку на основе валидации. Мы делаем это, передавая набор данных (или в данном случае два набора данных) в параметр `evals` и указывая `early_stopping_rounds`. Что происходит потом? В каждом раунде последовательности обучения совокупность обученных к настоящему моменту деревьев решений оценивается на наборах данных в списке `evals`. Если метрика оценки (в данном случае AUC) не улучшается на протяжении `early_stopping_rounds`, то обучение прекращается. Если бы мы не задали раннюю остановку, то обучение продолжалось бы до тех пор, пока не будет построено `num_boost_round` деревьев — часто это произвольная точка остановки. Нам почти всегда следует использовать раннюю остановку при обучении наших моделей GBM.



Если мы передаем в `evals` несколько наборов данных, только последний набор в списке будет задействован для определения того, выполнен ли критерий ранней остановки. Более того, в финальной модели будет слишком много деревьев. Всякий раз, когда мы делаем прогноз с помощью модели, мы должны указать, сколько деревьев использовать, используя параметр `iteration_range`, — дополнительную информацию можно найти в документации.

Как мы увидим дальше, неограниченный XGBoost может запросто назначать вероятности отдельным наблюдениям на основе иногда ложных закономерностей

Книги для программистов: https://t.me/free_books_for_it

в обучающих данных. Часто мы можем добиться большего, дополняя информацию из обучающих данных нашими собственными знаниями и привлекая на помощь экспертов в предметной области.

Мы знаем, например, что если у кого-то нарастает просрочка платежей по кредитной карте, то у него почти наверняка возрастает вероятность задержки следующего платежа. Это означает, что для всех признаков `PAY_*` в нашем наборе данных мы хотели бы наблюдать увеличение выходных значений модели при увеличении значения признака и наоборот. Ограничения монотонности XGBoost позволяют нам сделать именно это. Для каждого признака в наборе данных мы можем указать тип связи с выходными данными модели — положительная, отрицательная или не монотонная.

Наш набор данных содержит только 19 признаков, и мы можем проанализировать основные причинно-следственные связи с риском дефолта для каждого из них. Что если наш набор данных содержит сотни признаков? Нам хотелось бы обучить робастную модель с ограничениями, но, возможно, мы не уверены в монотонной причинно-следственной связи между определенными признаками и целью. Альтернативный (или дополнительный) метод получения ограничений монотонности использует корреляцию Спирмена. В следующем коде мы реализуем функцию, которая проверяет коэффициент парной корреляции Спирмена между каждым признаком и целью. Если коэффициент Спирмена по величине превышает пороговое значение, указанное пользователем, предполагается, что этот признак имеет монотонную связь с целью. Функция возвращает кортеж, содержащий значения –1, 0 и 1 — именно такую форму входных данных ожидает XGBoost при указании ограничений монотонности.

```
def get_monotone_constraints(data, target, corr_threshold):
    corr = pd.Series(data.corr(method='spearman')[target]).drop(target)
    monotone_constraints = tuple(np.where(corr < -corr_threshold, -1,
                                 np.where(corr > corr_threshold, 1, 0)))
    return monotone_constraints
```



Мы используем коэффициенты корреляции Спирмена, а не заданную по умолчанию корреляцию Пирсона, поскольку модели GBM являются нелинейными, даже если они имеют ограничения. Ограничения монотонности XGBoost навязывают монотонность, а не линейность: корреляция Спирмена точно измеряет силу монотонной зависимости, тогда как корреляция Пирсона измеряет силу линейной связи.

На рис. 6.5 мы нанесли на график коэффициент корреляции Спирмена для каждого признака относительно целевой переменной. Вертикальные линии указывают пороговое значение 0,1. Мы видим, что этот подход, основанный на данных, предполагает наложение монотонных ограничений для признаков `PAY_*`. Мы используем порог 0,1 как неформальный маркер практической значимости. Возможно, не имеет смысла применять какие-либо ограничения к тем входным признакам, чья корреляция Спирмена по величине меньше 0,1. По мере увеличения платежных и кредитных лимитов вероятность дефолта должна уменьшаться. По мере увеличения просроченных платежей вероятность дефолта должна увеличиваться. Результаты этого подхода, основанного на данных, также отражают здравый смысл. Это наиболее

Книги для программистов: https://t.me/free_books_for_it

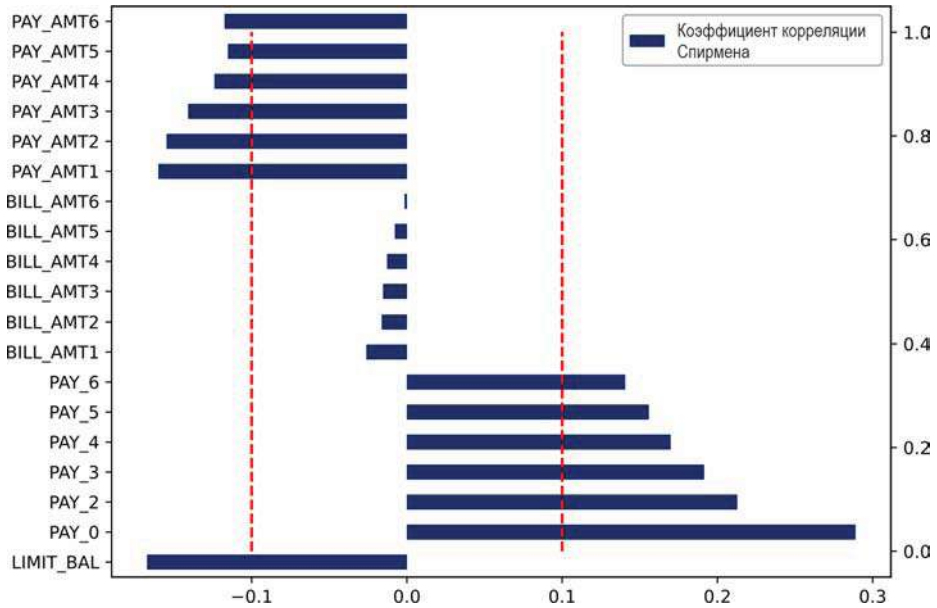


Рис. 6.5. Коэффициенты корреляции Спирмена между целью `DELINQ_NEXT` и каждым признаком. Вертикальные полосы обозначают пороговое значение 0,1

важный момент при создании ограничений, поскольку мы пытаемся внести в нашу модель знания о причинно-следственной области.

Затем мы обучаем модель с ограничениями, вытекающими из анализа на рис. 6.5. Сделаем несколько замечаний по поводу получившихся моделей с ограничениями и без ограничений. Глядя на выходные данные `xgb.train()` с `verbose_eval=True` в примере кода, мы видим, что модель без ограничений имеет более высокий AUC на обучающем наборе (0,829 против 0,814), но совпадает по качеству с моделью, имеющей ограничения, на валидационном наборе (0,785 против 0,784). Это говорит о том, что модель с ограничениями менее переобучена, чем модель без ограничений: при одинаковом наборе гиперпараметров модель с ограничениями улавливает более высокую долю истинного сигнала в данных. Как покажет наш дальнейший анализ, существуют дополнительные причины, по которым мы ожидаем более высокого качества прогнозов (и большей стабильности) от модели с ограничениями in vivo.

Наконец, взгляните на рис. 6.6, где показана значимость признаков для двух моделей. Мы можем вычислить значимость признаков для моделей XGBoost разными способами. В данном случае мы смотрим на средний *охват разветвлений* (split coverage) в ансамбле. Охват разветвлений — это просто количество обучающих выборок, которые проходят через разветвление. Это традиционный способ вычисления значимости признака. Он не имеет таких же теоретических гарантий, как, например, методы SHAP.

Мы видим, что модель с ограничениями распределяет важность признаков более равномерно по всему входному набору признаков. Модель без ограничений прида-

Книги для программистов: https://t.me/free_books_for_it

ет непропорционально бóльшую долю значимости признаку `PAY_0`. Это еще одно свидетельство того, что модель с ограничениями будет более надежной при развертывании в производстве. Если модель сосредоточивает всю свою способность принятия решений на одном признаке, то она потерпит неудачу, когда распределение этого признака изменится с появлением новых данных. Чрезмерная зависимость от одного признака также представляет собой угрозу безопасности. Злоумышленникам легче понять, как работает модель, и воспользоваться ею.

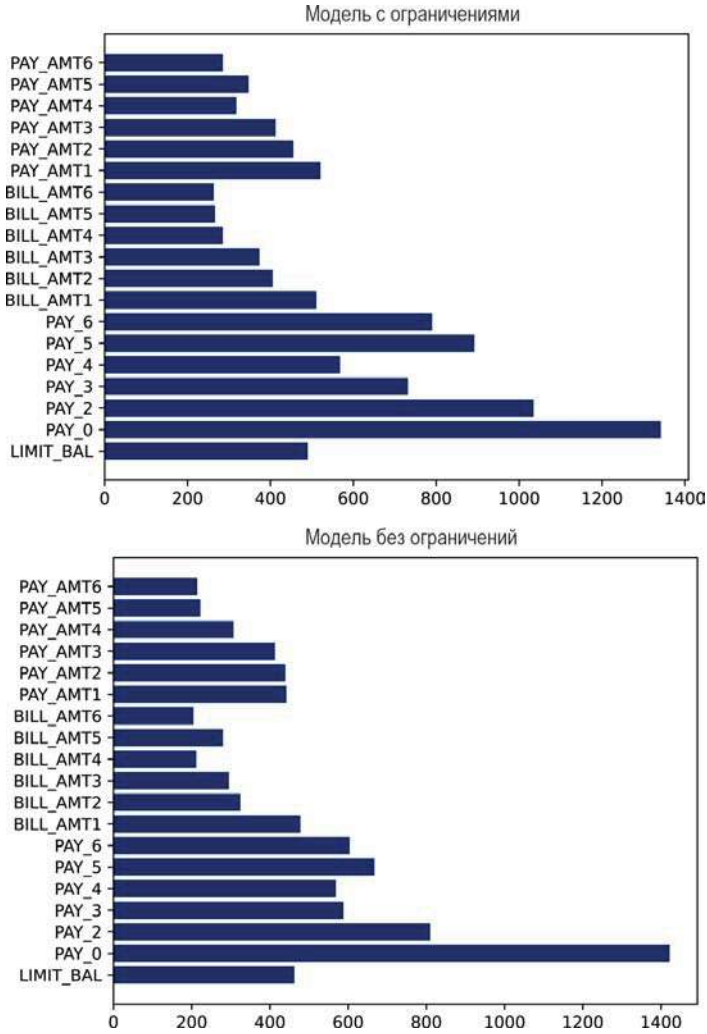


Рис. 6.6. Значимости признаков моделей с ограничениями и без ограничений, измеренные по среднему охвату



Когда мы видим, что значимость сосредоточена на одном или нескольких признаках, модель, скорее всего, будет нестабильной и небезопасной после развертывания. Наша модель может быть слишком чувствительной к смещению распределения данных по одному измерению, и злоумышленнику достаточно манипулировать

Книги для программистов: https://t.me/free_books_for_it

значением одного признака, чтобы изменить прогнозы модели. Если модель ML фокусируется только на одном или двух признаках, попробуйте заменить ее более простой моделью или бизнес-правилом.

Объяснение поведения модели с помощью частичной зависимости и ICE

Мы продолжаем сравнение моделей XGBoost с ограничениями и без ограничений, рассматривая параллельные графики частичной зависимости и ICE для `PAY_0`. В предыдущих разделах мы уже обсуждали, как условное среднее целевой переменной показывает ложный провал около `PAY_0 = 6`, где наши обучающие данные скудны. Давайте посмотрим, как наши две модели XGBoost справляются с этим дефицитом данных.

На рис. 6.7 мы можем видеть, как модель без ограничений в какой-то степени переобучается на ложной связи между `PAY_0` и `DELINQ_NEXT`. С другой стороны, модель с ограничениями вынуждена подчиняться здравому смыслу, согласно которому увеличение задержки платежей не должно приводить к снижению риска дефолта. Это отражено в монотонно растущей частичной зависимости и графиках ICE для `PAY_0` в рамках нашей модели с ограничениями.



Одна из трудностей с ICE заключается в выборе людей, для которых следует построить графики в первую очередь. Хороший способ начать работу с графиками ICE — выбрать отдельных лиц или строки данных во всех децилях прогнозируемых результатов. Это дает приблизительную картину локального поведения, и при необходимости мы можем копнуть глубже в нужном месте.

Мы также видим, что для обеих моделей наблюдаются большие изменения в выходных данных в диапазоне значений `PAY_0`. То есть и графики частичной зависимости, и графики ICE показывают большое вертикальное движение при изменении `PAY_0` от –2 до 8. Выходные данные модели очень чувствительны к значениям этого признака, вследствие чего мы и увидели такие высокие значения важности признака для `PAY_0` на рис. 6.6. Если мы не наблюдаем подобных изменений значений для признака, который наша модель считает очень важным, это может означать необходимость дополнительной отладки.

Графики частичной зависимости и ICE также могут показать, где в нашей модели есть взаимодействия признаков. Взгляните на графики частичной зависимости и ICE для `LIMIT_BAL` в нашей модели без ограничений на рис. 6.8.

Как было сказано выше в разделе «Частичная зависимость и индивидуальное условное ожидание», когда кривые частичной зависимости и ICE расходятся, как здесь, это указывает на корреляции или взаимодействия в наших данных и модели. Более того, мы можем оглянуться на обучение EBM и увидеть, что два важных взаимодействия, выявленных EBM, — это `LIMIT_BAL x BILL_AMT2` и `LIMIT_BAL x BILL_AMT1`. Вполне вероятно, что наша модель XGBoost без ограничений также уловила эти взаимодействия. В отличие от EBM, наши модели XGBoost пронизаны различными взаимодействиями высокого порядка между признаками. Но частичная зави-

Книги для программистов: https://t.me/free_books_for_it

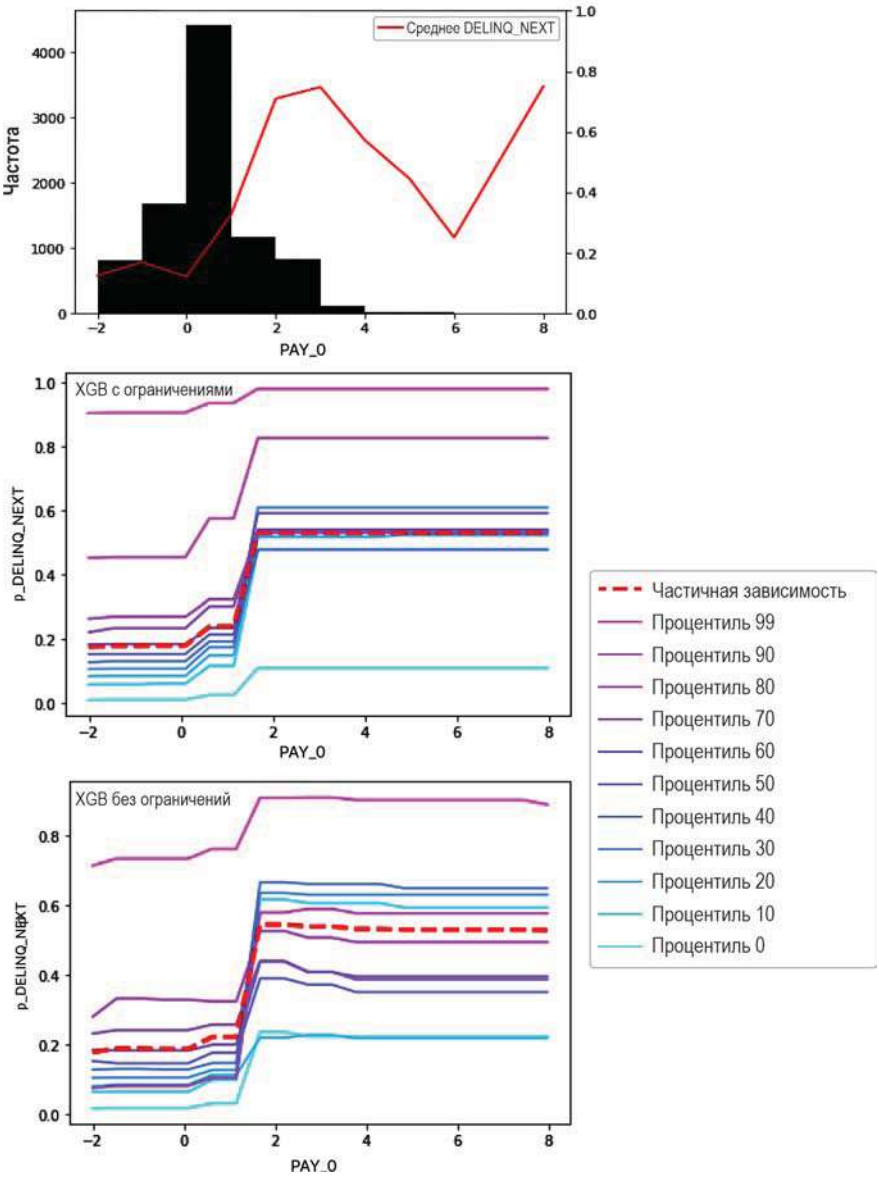


Рис. 6.7. Графики частичной зависимости и ICE признака `PAY_0` для моделей с ограничениями (вверху) и без ограничений (внизу)

симость и ICE в сочетании со способностью EBM изучать двусторонние взаимодействия помогут нам понять и некоторые взаимодействия в наших моделях XGBoost. Еще одним отличным инструментом для понимания сложных взаимодействий признаков в моделях ML является *дерево решений с суррогатными разделениями* (surrogate decision tree), о котором мы поговорим далее.

Книги для программистов: https://t.me/free_books_for_it

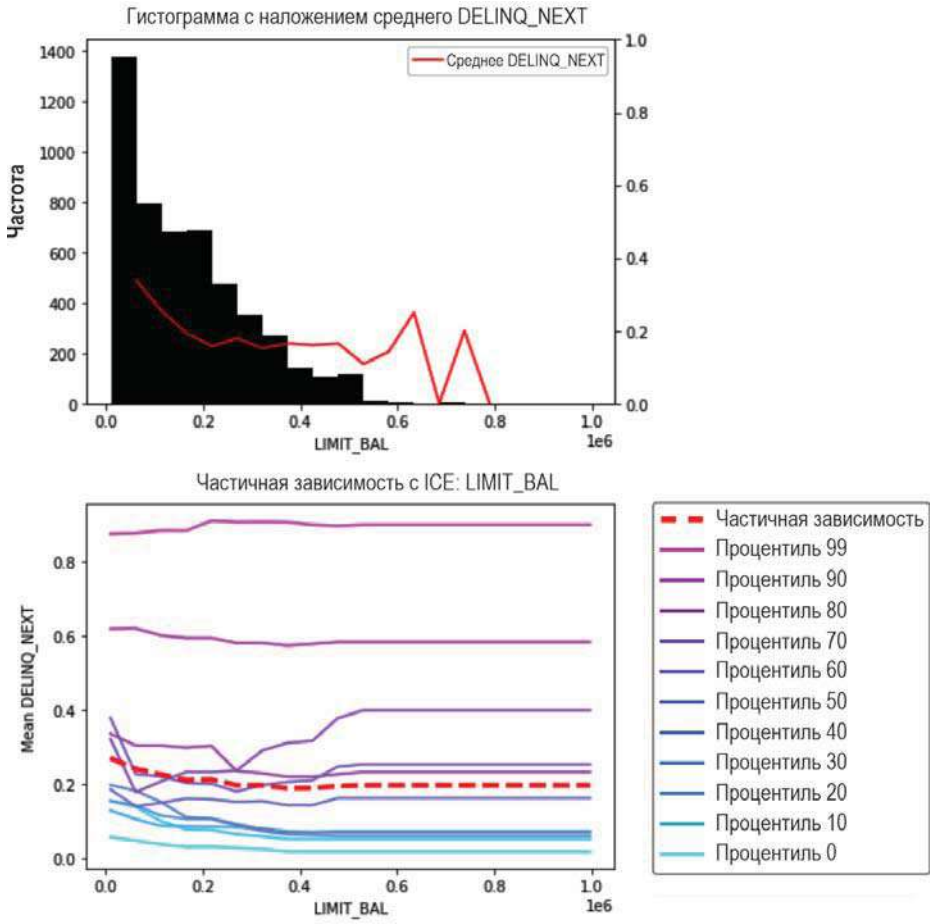


Рис. 6.8. Графики частичной зависимости и ICE признака `LIMIT_BAL` для модели без ограничений

Суррогатные модели дерева решений как метод объяснения

Проведенный нами анализ свидетельствует о том, что модель XGBoost без ограничений работает не лучше, чем ее версия с ограничениями. Графики частичной зависимости и ICE показывают, что, привязывая модель с ограничениями к разумным отношениям в реальном мире, мы успешно предотвратили обнаружение моделью ложных связей в обучающих данных. Поскольку наша модель с ограничениями логически превосходит альтернативную, следующие разделы будут посвящены исключительно этой модели.

Мы продолжим исследование поведения модели с помощью техники апостериорного объяснения — суррогатных моделей дерева решений. *Суррогатная модель* — это простая модель, предназначенная для имитации поведения более сложной модели. В нашем случае мы пытаемся имитировать нашу модель XGBoost с ограничениями, которая содержит примерно 100 деревьев, используя одно неглубокое

Книги для программистов: https://t.me/free_books_for_it

дерево решений. Дерево решений — это фактически блок-схема, основанная на данных, поэтому мы можем рассматривать суррогат дерева решений как блок-схему и объяснять простыми словами, как работает более сложная GBM. Именно это делает суррогаты дерева решений мощной техникой объяснения. Для обучения нашей суррогатной модели мы воспользуемся реализацией `DecisionTreeRegressor` из `sklearn`:

```
surrogate_model_params = {'max_depth': 4,
                          'random_state': seed}
surrogate_model = DecisionTreeRegressor(**surrogate_model_params)
                                .fit(train[features], model_constrained.predict(dtrain))
```



Суррогатное моделирование также известно под другими названиями, например *сжатие модели* (model compression) или *извлечение модели* (model extraction).

Обратите внимание: мы обучаем регрессионную модель, ориентированную на выходные данные модели, которую пытаемся объяснить. То есть суррогат полностью сосредоточен на имитации поведения более крупной модели, а не просто на создании более простой модели классификации. Мы также решили обучить дерево решений с глубиной, равной четырем. Если увеличить глубину, нам, возможно, будет трудно объяснить, что происходит в самой суррогатной модели.



Суррогатные модели не всегда работают хорошо. Обязательно убедитесь, что ваши суррогатные модели демонстрируют хорошее качество работы и стабильность. Здесь мы предлагаем простой подход суррогатного моделирования. В работах Interpreting Blackbox Models via Model Extraction (*https://oreil.ly/O4Kia*), Extracting Tree-Structured Representations of Trained Networks (*https://oreil.ly/BQnI7*) и The Price of Interpretability (*https://oreil.ly/CNgUA*) можно найти дополнительную информацию о суррогатных подходах и о том, какие гарантии, если таковые имеются, можно дать в отношении их точности.

Прежде чем приступить к исследованию нашей суррогатной модели, нужно спросить, можем ли мы ей доверять. Суррогаты дерева решений — мощный метод, но его правильность не гарантирована математически. Один из простых способов оценить качество нашего суррогата — вычислить показатели точности путем перекрестной проверки. Почему именно перекрестная проверка, а не просто проверочный набор данных? Недостатком моделей с одним деревом решений является их чувствительность к изменениям в наборе обучающих данных, поэтому, вычисляя точность на нескольких контрольных подвыборках, мы проверяем, является ли наша суррогатная модель точной и достаточно стабильной, чтобы ей можно было доверять:

```
from sklearn.model_selection import KFold
from sklearn.metrics import r2_score

cross_validator = KFold(n_splits=5)
cv_error = []
for train_index, test_index in cross_validator.split(train):
    train_k = train.iloc[train_index]
    test_k = train.iloc[test_index]
```

Книги для программистов: https://t.me/free_books_for_it

```
    dtrain_k = xgb.DMatrix(train_k[features],
                           label=train_k[target])
    dtest_k = xgb.DMatrix(test_k[features],
                          label=test_k[target])

    surrogate_model = DecisionTreeRegressor(**surrogate_model_params)
    surrogate_model = surrogate_model.fit(train_k[features],
                                          model_constrained.predict(dtrain_k))
    r2 = r2_score(y_true=model_constrained.predict(dtest_k),
                                          y_pred=surrogate_model.predict(test_k[features]))
    cv_error += [r2]

for i, r2 in enumerate(cv_error):
    print(f"Значение R2 для подвыборки {i}: {np.round(r2, 3)}")
print(f"\nСтандартное отклонение ошибки: {np.round(np.std(cv_error), 5)}")

Значение R2 для подвыборки 0: 0.895
Значение R2 для подвыборки 1: 0.899
Значение R2 для подвыборки 2: 0.914
Значение R2 для подвыборки 3: 0.891
Значение R2 для подвыборки 4: 0.896

Стандартное отклонение ошибки: 0.00796
```

Эти результаты выглядят великолепно. Мы видим, что суррогатная модель имеет высокую точность при каждой перекрестной проверке с очень небольшими отклонениями. Получив некоторую уверенность в том, что наше дерево решений является разумным заменителем сложной модели, мы построим суррогатную модель (рис. 6.9).

Обратите внимание на рис. 6.9, что суррогатная модель сначала разделяется по признаку `PAY_0`. То есть чтобы оптимально имитировать поведение модели XGBoost с ограничениями, первое, что делает суррогатная модель, — это разделяет наблюдения на две группы — с `PAY_0` ≤ 1.5 и с более высокими значениями `PAY_0`. Мы можем грубо оценить важность признаков, посмотрев на глубину разделения каждого признака в суррогатной модели, так что этот результат согласуется с нашим анализом важности признаков. Хороший знак.

Поскольку наша суррогатная модель очень проста, мы можем изложить множество наблюдений о ней простым языком. Например, мы можем проследить пути наблюдений с самым высоким и самым низким риском и объяснить, как наш суррогат обрабатывает их:

- наблюдения с наименьшим риском проходят следующий путь принятия решений: статус погашения в сентябре 2005 г. своевременно или с опозданием на один месяц (`PAY_0` ≤ 1.5) И статус погашения в августе 2005 г. своевременно или с опозданием на один месяц (`PAY_2` ≤ 1.5) И сумма погашения в августе 2005 г. составляет более 1603,5 доллара США (`PAY_AMT2` > 1603,5 долл. США). Эти правила

Книги для программистов: https://t.me/free_books_for_it

ориентированы на клиентов, которые вносят своевременные и крупные платежи, что вполне разумно;

- наблюдения с самым высоким риском проходят следующий путь принятия решений: статус погашения в сентябре 2005 г. с просрочкой более одного месяца (`PAY_0 > 1.5`) и статус погашения в августе 2005 г. с просрочкой более чем на один месяц (`PAY_2 > 1`) и статус погашения в апреле 2005 г. с просрочкой более чем на месяц (`PAY_6 > 1`). Эти правила учитывают неблагоприятные статусы погашения на протяжении некоторого времени, что также логично.

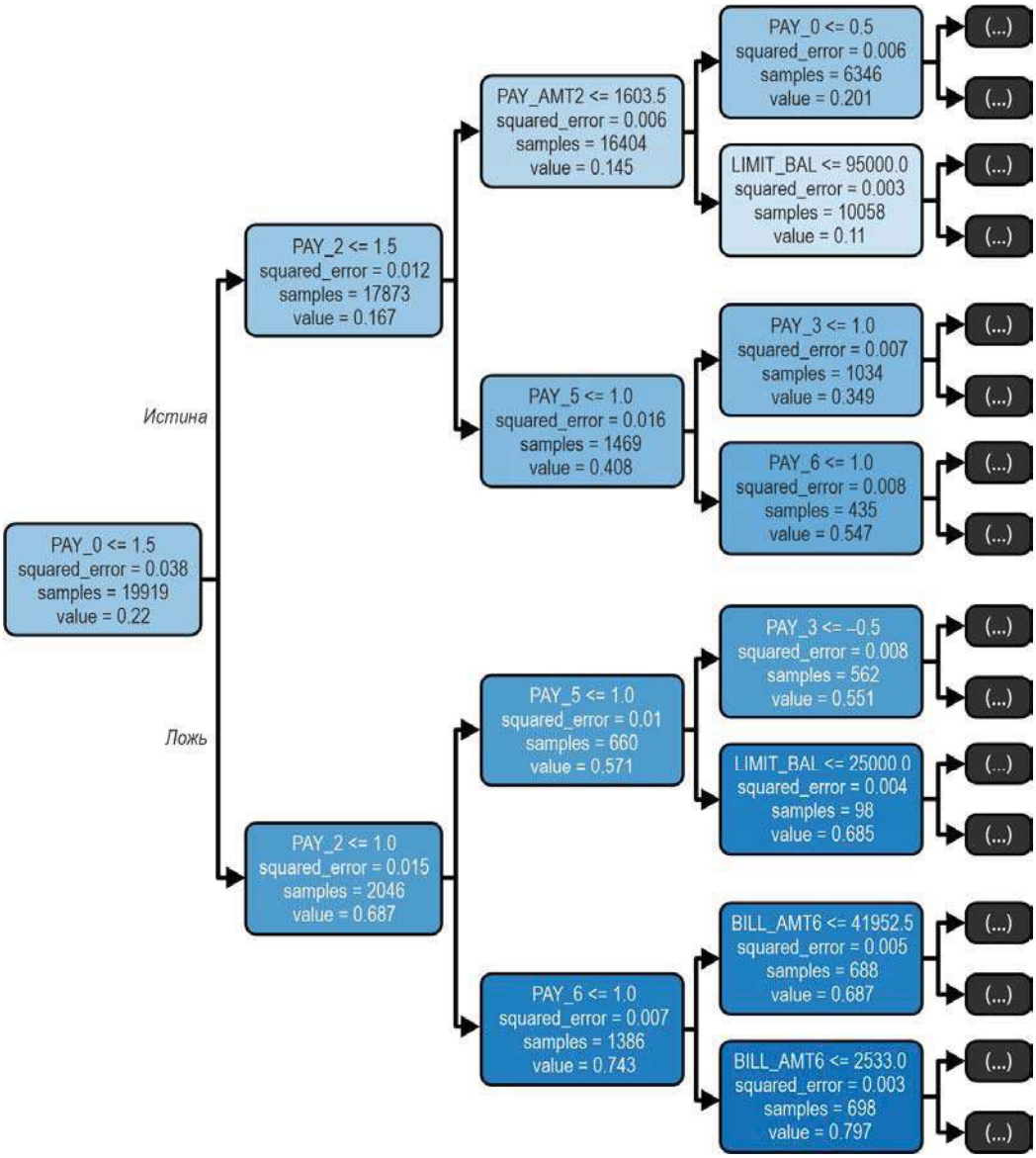


Рис. 6.9. Суррогат дерева решений для нашей модели XGBoost с ограничениями

Книги для программистов: https://t.me/free_books_for_it

Наши пояснения учитывают статусы погашения и суммы погашения. Для принятия решений более сложная модель GBM, по-видимому, фокусируется на более свежей информации о статусе и сумме. А для решений об отказе она, вероятно, ищет закономерности в статусе платежа за определенный период времени.

И, наконец, стоит отметить, что каждый раз, когда один признак следует за другим на пути дерева решений, эти признаки, скорее всего, взаимодействуют в нашей модели GBM. Мы можем легко определить основные взаимодействия признаков, которые изучила наша модель XGBoost, исследуя суррогатную модель. Интересно, что если оглянуться назад, то можно увидеть, что EBM тоже уловила некоторые из тех же взаимодействий, например `PAY_0 x PAY_2` и `PAY_0 x PAY_AMT2`. С помощью всех этих инструментов — EBM, частичной зависимости и ICE, а также суррогатных деревьев решений — мы действительно можем получить четкое представление о том, что находится в наших данных и чего разумно ожидать от поведения модели. Согласитесь, это сильно отличается от обучения одной необъяснимой модели и проверки нескольких показателей на тестовых данных. Мы изучаем, как работают эти модели, чтобы иметь возможность делать понятные человеку выводы об их реальных качествах.

Более того, мы можем использовать информацию о взаимодействиях для повышения качества линейных моделей, таких как логистическая регрессия со штрафом, включая эти изученные взаимодействия в качестве входных признаков. Если мы хотим придерживаться наиболее консервативных форм моделей для наших приложений с самым высоким риском, можно остановиться на GLM и, вероятно, повысить ее качество благодаря информации о важных взаимодействиях. Как видите, мы можем делать разнообразные простые и объяснимые наблюдения о нашей модели XGBoost с помощью суррогатов дерева решений. Попутно мы собрали массу полезной информации для документирования моделей и других задач управления рисками.

Пояснения на основе значений Шепли

Последний инструмент апостериорного объяснения, который следует обсудить, прежде чем мы завершим главу, — это значения Шепли. В разделе «Локальные пояснения и атрибуция признаков» в главе 2 мы упоминали, что значения Шепли можно использовать в качестве метода атрибуции локальных признаков. Фактически значения Шепли опираются на прочный математический базис, из которого следует, что они обычно являются лучшим выбором для атрибуции признаков и расчета значимости. Исследовательские сообщества и сообщества открытого исходного кода, возглавляемые Скоттом Лундбергом из Вашингтонского университета и Microsoft Research, разработали множество инструментов для создания и визуализации значений SHAP. Эти инструменты находятся в пакете SHAP Python, и именно их мы будем использовать в этом разделе.

Помните, что методы атрибуции *локальных* признаков присваивают значение каждому признаку для каждого наблюдения, количественно определяя, какой вклад этот признак внес в прогнозируемое значение, полученное наблюдением. В этом

Книги для программистов: https://t.me/free_books_for_it

разделе вы узнаете, как использовать значения SHAP и пакет SHAP для объяснения поведения моделей. В заключительном разделе главы мы рассмотрим некоторые тонкости объяснений, основанных на значениях Шепли, а также подводные камни, которыми они чреваты для практиков.

Взгляните на код для генерации значений SHAP для нашей монотонной модели XGBoost:

```
explainer = shap.TreeExplainer(model=model_constrained,
                               data=None,
                               model_output='raw',
                               feature_perturbation='tree_path_dependent')
shap_values = explainer(train[features])
```

Мы используем класс `TreeExplainer` пакета SHAP. Этот класс может генерировать значения SHAP для XGBoost, LightGBM, CatBoost и большинства древовидных моделей scikit-learn. `TreeExplainer` обсуждается в статьях Скотта Лундберга «Consistent Individualized Feature Attribution for Tree Ensembles» (*https://oreil.ly/VZz75*) и «From Local Explanations to Global Understanding with Explainable AI for Trees» (*https://oreil.ly/7fdWE*) и других. Они являются отличным примером передовых идей, которые сделали пакет SHAP таким успешным. Если вам нужно сгенерировать значения SHAP для модели, не основанной на деревьях, изучите примеры в документации пакета SHAP — там содержатся несколько примеров для табличных, текстовых и графических данных.



Если вам нужно объяснить модель, не основанную на деревьях или нейронных сетях, не забывайте об альтернативных объяснениях и сравнении с прототипами. Эти мощные концепции объяснения могут быть более эффективными, чем подходы общего назначения, такие как *локальные интерпретируемые объяснения, не зависящие от модели* (local model-agnostic interpretable explanations, LIME) или ядро SHAP.

Для начала давайте посмотрим на значения SHAP, связанные с признаком `PAY_0`, на рис. 6.10.

Каждая точка на рис. 6.10 представляет собой значение SHAP одного наблюдения для признака `PAY_0` или вклад в прогноз модели с координатой *x*, заданной значением `PAY_0`. Диаграмма рассеяния накладывается на гистограмму значений признаков в наборе данных, как и наши графики частичной зависимости и ICE. Фактически диаграмму рассеяния можно напрямую сравнить с графиками частичной зависимости и ICE для `PAY_0`. На диаграмме SHAP мы можем видеть весь диапазон значений атрибуции объектов в каждом сегменте значений `PAY_0`. Обратите внимание, что самый широкий диапазон находится в сегменте `PAY_0` = 2 — некоторые наблюдения с этим значением штрафуются примерно вдвое меньше других. Эта диаграмма рассеяния SHAP является одной из многих обобщающих диаграмм, включенных в пакет SHAP. Более подробный обзор можно найти в примерах документации (*https://oreil.ly/xWxlG*), а также в примерах блокнотов Jupyter для этой главы.

Как было показано в главе 2, мы можем построить меру общей значимости признаков, взяв среднее абсолютных значений SHAP. Вместо стандартной горизонтальной

Книги для программистов: https://t.me/free_books_for_it

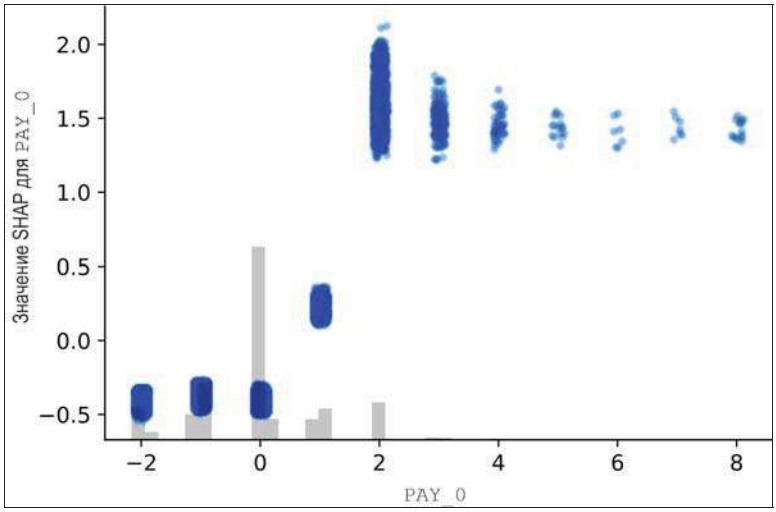


Рис. 6.10. График зависимости для признака PAY_0, показывающий распределение значений SHAP в каждом сегменте значений объекта

гистограммы величин значимости признаков мы можем использовать инструменты построения графиков SHAP, чтобы явно рассматривать значимость признаков как совокупность локальных объяснений:

```
shap.plots.beeswarm(shap_values.abs, color="shap_red", max_display=len(features))
```

Рис. 6.11 дает нам интересный взгляд на значимость признаков. Обратите внимание, что некоторые признаки (например, PAY_0, PAY_AMT1) имеют несколько точек, демонстрирующих экстремальные значения SHAP, тогда как другие признаки (например, LIMIT_BAL, PAY_AMT3) имеют высокую значимость, поскольку многие отдельные наблюдения имеют достаточно высокие абсолютные значения SHAP. Иными словами, локальные объяснения позволяют нам различать высокочастотные эффекты малой величины и низкочастотные эффекты большой величины. Это важно, поскольку каждый из этих масштабных и низкочастотных эффектов отражает реальных людей, на которых влияет наша модель

Объяснения на основе значений Шепли получили широкое распространение благодаря своей прочной теоретической основе и надежному набору инструментов. Поскольку эти значения могут быть вычислены на уровне отдельных наблюдений, их, вероятно, можно использовать для создания уведомлений о неблагоприятных действиях или других отчетов о снижении качества прогнозов модели (при правильном использовании, проверке на точность и стабильность и в сочетании с моделями с ограничениями). Изучите наши примеры кода, прочитайте статьи и документацию и начинайте генерировать значения SHAP для своей модели. Но сначала прочтите следующий раздел, где мы обсуждаем некоторые предположения и ограничения, лежащие в основе каждого вычисления SHAP.

Книги для программистов: https://t.me/free_books_for_it

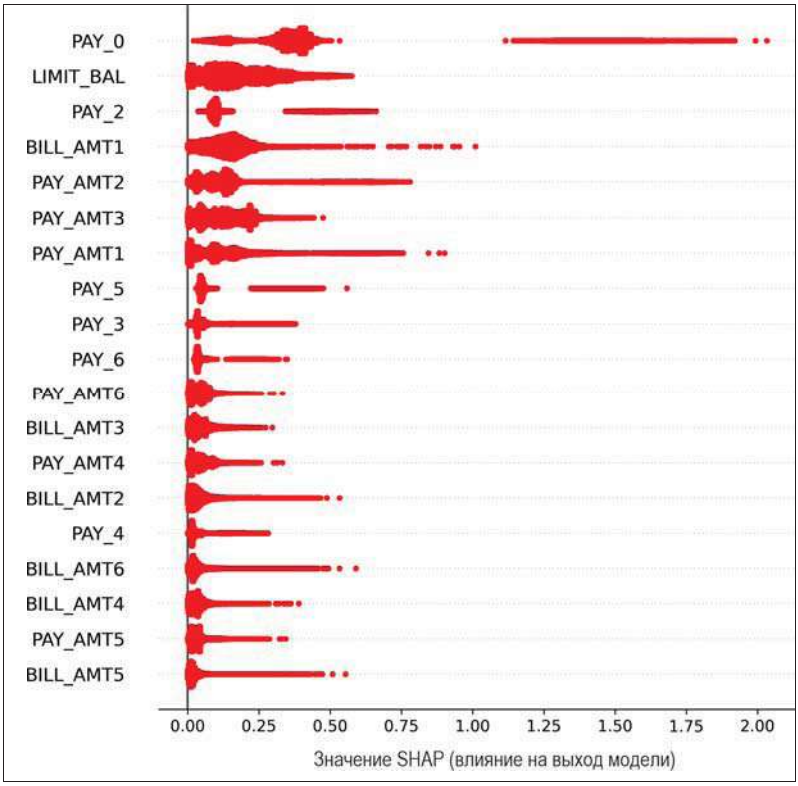


Рис. 6.11. Значимость признаков, показанная как совокупность абсолютных значений SHAP отдельных наблюдений

Проблемы со значениями Шепли

В главе 2 и в разделе «Значения Шепли» текущей главы мы представили идею *фонового набора данных*, который лежит в основе расчетов значений Шепли. Когда SHAP стремится понять, как отдельный признак влияет на определенный прогноз, он случайным образом заменяет значение этого признака в обучающих или тестовых данных значением, полученным из фоновых данных. Затем алгоритм многократно сравнивает предсказания, полученные из двух разных наборов данных, причем происходит хаотичное чередование того, какие признаки получают значения из рабочего набора данных, а какие — из фонового. Суть фонового набора данных можно пояснить так: когда мы вычисляем значение SHAP для наблюдения, мы отвечаем на вопрос: «Почему это наблюдение получило именно такой прогноз, *а не какой-то другой*?» Так вот, «какой-то другой прогноз», с которым сравниваются наблюдения, определяется выбором фоновых данных или эталонного распределения.

В следующем коде мы создаем два набора значений SHAP для одних и тех же наблюдений, за исключением того, что в одном случае мы не указываем эталонное распределение, а в другом — указываем:

Книги для программистов: https://t.me/free_books_for_it

```
explainer_tpd = shap.TreeExplainer(model=model_constrained,
                                   feature_perturbation='tree_path_dependent')
shap_values_tpd = explainer_tpd(train[features])

train['pred'] = model_constrained.predict(dtrain)
approved_applicants = train.loc[train['pred'] < 0.1]
explainer_approved = shap.TreeExplainer(model=model_constrained,
                                        data=approved_applicants[features],
                                        model_output='raw',
                                        feature_perturbation='interventional')

shap_values_approved = explainer_approved(train[features])
```

Задавая `feature_perturbation='tree_path_dependent'`, мы указываем, что предпочитаем вообще не определять эталонное распределение. Вместо этого SHAP использует информацию, собранную из обученных деревьев моделей GBM, для неявного определения своих собственных фоновых данных. Это похоже на использование обучающих данных в качестве фоновых данных, но не совсем то же самое.

Далее мы определяем объяснитель с `feature_perturbation='interventional'`, для которого мы передаем эталонное распределение, состоящее из обучающих выборок, которые получили вероятность просрочки платежа менее 10%. Если мы сравним каждое наблюдение с эталонным распределением, то можно ожидать, что эти два набора значений SHAP будут существенно различаться. В конце концов, эти вопросы чрезвычайно различны: «Почему это наблюдение получило именно этот прогноз, а не средний прогноз в обучающих данных?» против «Почему это наблюдение получило такой прогноз, а не прогнозы, данные одобренным кандидатам?». Как обсуждалось в главе 2, последний вопрос в гораздо большей степени соответствует требованиям регулирующих органов США относительно уведомлений о неблагоприятных действиях. Это пример того, что имеет в виду профессор Бичек, когда говорит: «Не объясняйте без контекста!».

Хотя некоторые утверждают, что возмущения признаков `tree_path_dependent` верны для данных, т. е. они говорят нам не только о поведении этой модели, как показано в статье «Explaining Individual Predictions When Features Are Dependent: More Accurate Approximations to Shapley Values» (*https://oreil.ly/3PBGX*), это, вероятно, не соответствует действительности. Атрибуция признаков, которая отражает *данные*, потребует от нас знания полного совместного распределения вероятностей наших данных. Но это крайне сложная техническая задача, которую, к сожалению, нам не под силу решить, основываясь только на анализе структуры путей деревьев в нашей модели. Лучше использовать опцию возмущения признаков `interventional` и считать, что наши значения SHAP отражают *модель* и не обобщаются за пределами нашей модели. Мы предлагаем использовать возмущения признаков `tree_path_dependent` только в тех случаях, если у вас нет другого выбора. Основная причина заключается в том, что вы не имеете доступа к фоновому набору данных и вам приходится выводить его из модели. Если у вас есть доступ к обучающим данным, явно передайте их объяснителю SHAP и используйте тип возмущения признаков `interventional`.

Книги для программистов: https://t.me/free_books_for_it



На момент написания этой книги передовые методики показывают, что лучше использовать возмущения признаков `interventional` и признать, что наши объяснения не обобщаются за пределами нашей модели. Используйте изменения объекта, зависящие от `tree_path_dependent`, только в том случае, если у вас нет другого выбора.

Обратимся к рис. 6.12, чтобы попытаться понять, почему все это важно. Здесь показаны два набора значений SHAP для одного и того же наблюдения: один рассчитан без опорного распределения и `feature_perturbation='tree_path_dependent'`, а другой рассчитан на основе примеров утвержденных заявок и `feature_perturbation = 'interventional'`. Прежде всего, в некоторых наблюдениях можно заметить значительные расхождения в значениях SHAP для этих двух различных типов объяснений SHAP.

Представьте, что на основании этих двух объяснений мы разослали клиентам уведомления о недопустимых действиях. При отсутствии опорного распределения четыре основных признака, которые способствовали большей вероятности просрочки, — это `PAY_0`, `LIMIT_BAL`, `BILL_AMT5` и `PAY_AMT3`. Но если мы укажем контекстнозависимое опорное распределение, этими четырьмя основными признаками будут `PAY_AMT1`, `PAY_AMT2`, `PAY_AMT3` и `LIMIT_BAL`. Какое объяснение является правильным в контексте потребительского кредитования, где возможность обратиться к оператору с жалобой на решение модели является важным аспектом доверия и ответственного использования? Вероятно, это будут объяснения, основанные на возмущении признаков `interventional`, а благодаря использованию утвержденных кандидатов в исходных данных они также будут сформулированы более логично с точки зрения нормативных требований.

Однако эти объяснения уникальны для модели и поэтому предоставляют заявителю точные описания причин отказа только в том случае, если в будущем заявитель будет оцениваться с использованием точно такой же модели. Если заявитель получит разные уведомления о причинах отказа по схожим кредитным продуктам, особенно от одного и того же кредитора, только из-за какого-то механизма, специфичного для модели в конвейере ML, это вызовет удивление. Не исключено, что объяснения, основанные на древовидных путях, будут более однородными в разных моделях (как утверждают некоторые исследователи), но это указывает на еще одну трудность. Как основанные на пути, так и интервенционные значения SHAP могут дать объяснения, основанные на признаках, которые не играли роли в конкретном решении. Это большая проблема для уведомлений о причинах отказа и вытекающей из этого возможности обращения в суд. Тем не менее мы по-прежнему предлагаем использовать интервенционные объяснения при использовании SHAP, одновременно признавая и проверяя их недостатки.

Даже когда мы правильно выявляем все нюансы, связанные с возмущениями признаков и исходными данными, все равно остается фундаментальное ограничение объяснимого ML, которое нам нужно иметь в виду. Заявитель, которому было отказано, хочет знать, как улучшить свой кредитный профиль, чтобы получить одобрение на получение кредита в будущем. Ответ на этот вопрос можно найти в нашем

Книги для программистов: https://t.me/free_books_for_it

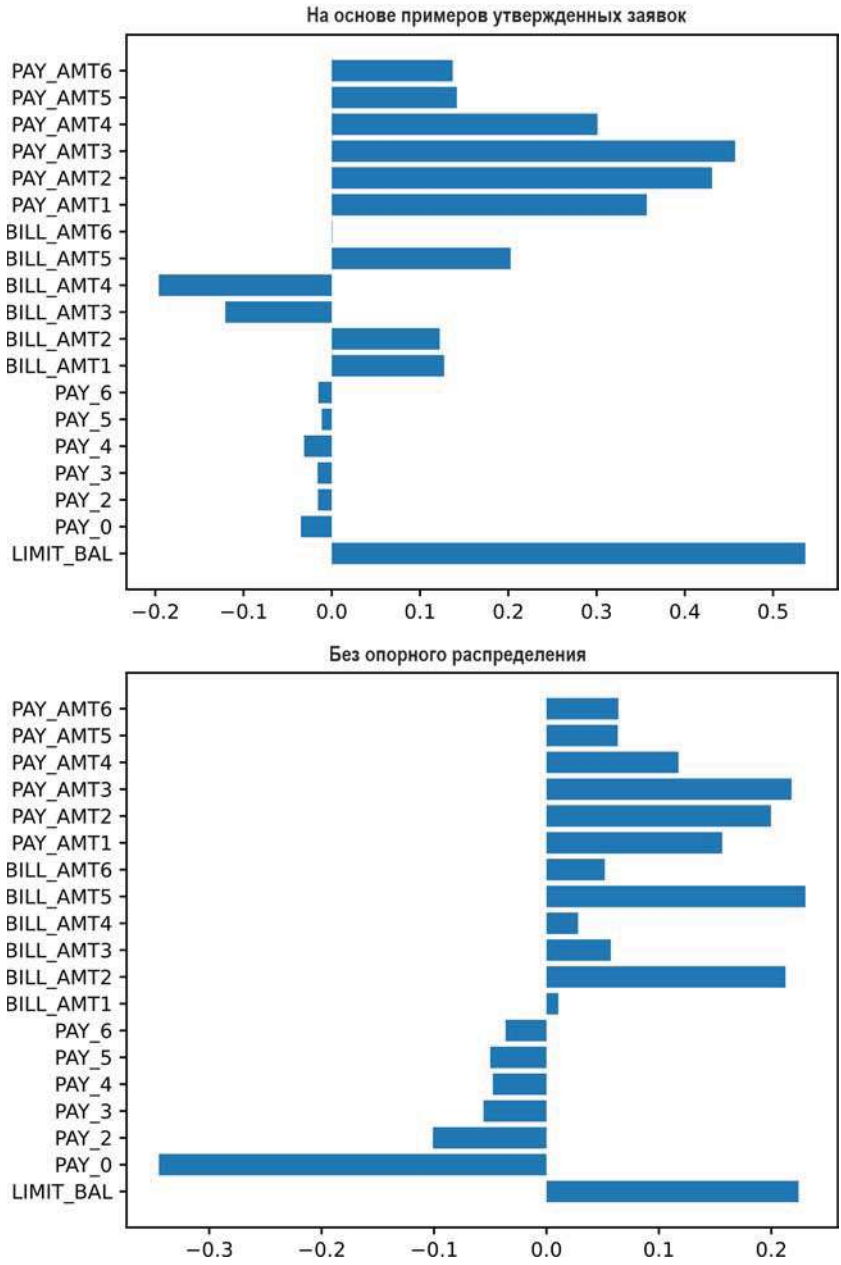


Рис. 6.12. Значения SHAP для одного и того же наблюдения, сгенерированные как с использованием, так и без использования эталонного распределения, состоящего из наблюдений с присвоенной вероятностью просрочки менее 10%

опорном распределении утвержденных заявителей. Однако нам нужно проявлять некоторую осторожность даже при использовании значимого и контекстно-зависимого опорного распределения. Вытекающий из этого вопрос: «Что мне следует изменить (в моем кредитном профиле), чтобы получить одобрение в будущем?» по

Книги для программистов: https://t.me/free_books_for_it

сути, является причинно-следственным, но мы не работаем с причинно-следственными моделями. Как говорил создатель пакета SHAP Скотт Лундберг: «Будьте осторожны при интерпретации прогнозных моделей в поисках причинно-следственных связей». Далее он говорит:

> Модели прогнозного машинного обучения, такие как XGBoost, становятся еще более мощными в сочетании с инструментами интерпретируемости, такими как SHAP. Эти инструменты выявляют наиболее информативные взаимосвязи между входными признаками и результатом прогнозирования, что полезно для объяснения механизма модели, получения одобрения инвесторов и диагностики потенциальных проблем. Возникает искушение пойти еще дальше в этом анализе и предположить, что инструменты интерпретации также могут определить, какими признаками следует манипулировать владельцам системы, если они хотят повлиять на результаты в будущем. Однако <...> использование прогностических моделей для подобных причинных выводов часто может вводить в заблуждение.

Несмотря на все математические гарантии и простоту использования, объяснения, основанные на значениях Шепли, не являются волшебным средством. Это не более чем еще один инструмент в нашем наборе инструментов для объяснения моделей, пусть и очень удобный. Чтобы добиться истинной интерпретируемости, нам необходимо объединить наши методы апостериорной объяснимости с внутренне объяснимыми архитектурами моделей, такими как GLM, GAM или жестко ограниченными XGBoosts. А еще нам следует сохранять умеренность и помнить, что ML раскрывает нам корреляцию, а не причинно-следственную связь.

Более информированный выбор модели

В завершение этой главы мы вернемся к признаку `PAY_0` и сравним, как его обрабатывает каждая из пяти построенных нами моделей. Напомним, что `PAY_0` представляет статус погашения кредита, где более высокие значения соответствуют большей задержке платежей. Очевидно, что более высокие значения этого признака должны соответствовать большему риску просрочки в будущем. Однако обучающие данные, которые мы использовали, очень разрежены в области более высоких значений признака, поэтому у нас есть только несколько наблюдений для просрочки более чем на месяц. Имея это в виду, давайте рассмотрим пять графиков частичной зависимости и ICE для обработки этого признака каждой моделью, как показано на рис. 6.13. Нам нужно спросить себя: «Какой из этих моделей я бы больше всего доверял, имея кредитный портфель в миллиард долларов?»

Три наши модели демонстрируют отклик на ложное падение среднего целевого значения в разреженной области пространства признаков: GAM, EBM и XGBoost без ограничений. Модели GLM и XGBoost с ограничениями были вынуждены игнорировать это явление. Поскольку GAM и EBM являются аддитивными моделями, нам известно, что графики частичной зависимости и ICE действительно отражают их трактовку этого признака. Модель XGBoost без ограничений переполнена взаимодействиями признаков, так что мы не можем быть в ней уверены. Но частичная зависимость отслеживается в ICE, и, скорее всего, это хороший индикатор истинного поведения модели. Можно сказать, что перед нами стоит вопрос вы-

Книги для программистов: https://t.me/free_books_for_it

бора между GLM со штрафом и XGBoost с ограничениями. Какая модель будет лучшим выбором? Используя эти объяснимые модели и апостериорные пояснения, мы можем сделать гораздо более осознанный выбор, чем в рамках традиционного непрозрачного рабочего процесса ML — и это самое главное. Помните: если делать выбор исключительно по формальным показателям, мы бы выбрали модель, которая не слишком разумно реагирует на самый важный признак.

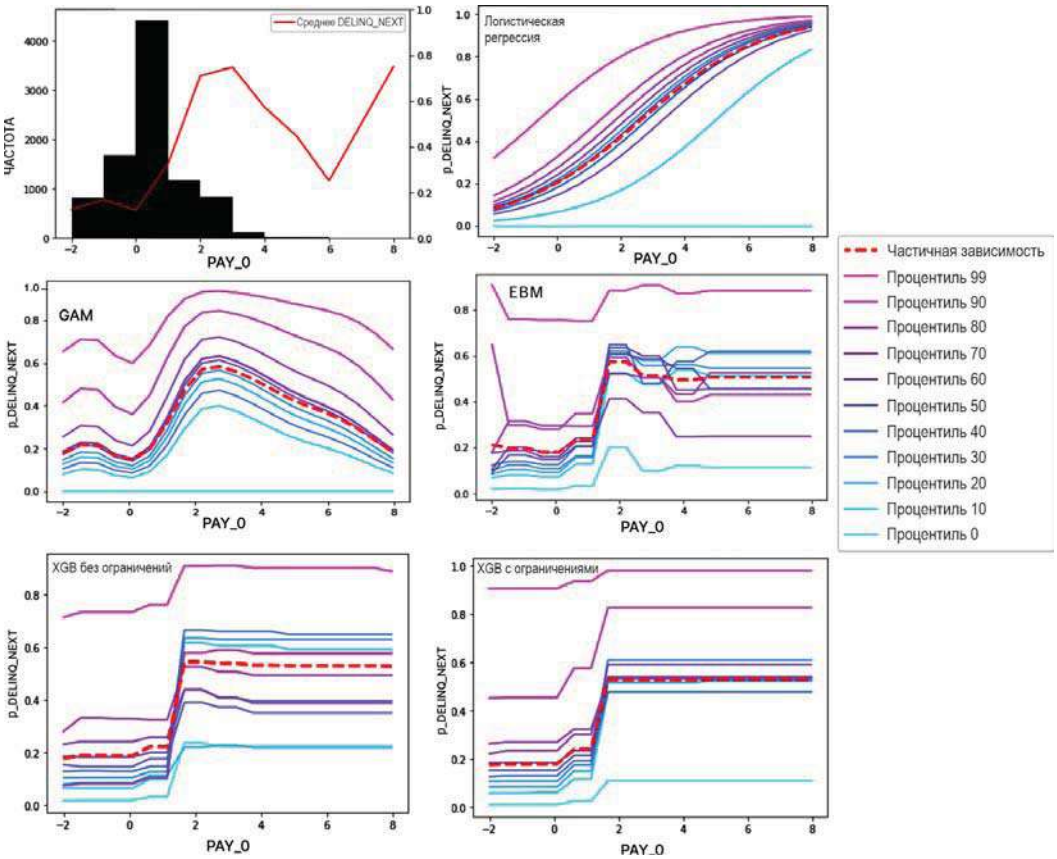


Рис. 6.13. Графики частичной зависимости и ICE для пяти моделей, обученных в этой главе

Уроки, которые следуют из этого анализа объяснимости в реальном мире, звучат так. Во-первых, мы понимаем, какие отношения между признаком и целью действительно значимы, а какие — шум. Во-вторых, если мы хотим объяснить поведение нашей модели (а это наверняка так), нам нужно выбрать архитектуру модели, которая по своей сути объяснима. Таким образом, у нас будет и модель, и объяснители, которые можно перепроверить посредством друг друга. В-третьих, мы должны заставить нашу модель подчиняться реальности с помощью ограничений. Люди все еще умнее компьютеров! Наконец, нам нужно будет проанализировать обученную модель с помощью разнообразного набора методов апостериорного объяснения, таких как графики частичной зависимости и ICE, суррогатные модели и значения

Книги для программистов: https://t.me/free_books_for_it

SHAP. Действуя таким образом, мы можем осознанно и разумно выбрать оптимальную модель, а не просто заниматься обучением произвольно выбранной модели на потенциально предвзятых и неточных обучающих данных.

Ресурсы

Дополнительное чтение

- Книга *Elements of Statistical Learning* (главы 3, 4, 9 и 10): **https://oreil.ly/S72E1**

Примеры кода

- Книга *Machine Learning for High Risk Applications*: **https://oreil.ly/machine-learning-high-risk-apps-code**

Инструменты объяснимого моделирования

- arules: **https://oreil.ly/bBv9s**
- causalml: **https://oreil.ly/XsiMk**
- elasticnet: **https://oreil.ly/pBOBN**
- gam: **https://oreil.ly/QS0bP**
- glmnet: **https://oreil.ly/rMzEl**
- h2o-3: **https://oreil.ly/PPUk5**
- imodels: **https://oreil.ly/coPjR**
- InterpretML: **https://oreil.ly/AZYDz**
- PiML: **https://oreil.ly/ELrbE**
- quantreg: **https://oreil.ly/qBWk9**
- rpart: **https://oreil.ly/yIml6**
- RuleFit: **https://oreil.ly/K-qc4**
- код Rudin Group: **https://oreil.ly/QmRFF**
- sklearn-expertsys: **https://oreil.ly/igFz6**
- skope-rules: **https://oreil.ly/nfYau**
- tensorflow/lattice: **https://oreil.ly/Z9iCS**

Инструменты апостериорного объяснения

- ALEPlot: **https://oreil.ly/OSfUT**
- Alibi: **https://oreil.ly/K4VEQ**
- anchor: **https://oreil.ly/K3UuW**
- DiCE: **https://oreil.ly/-lwV4**
- h2o-3: **https://oreil.ly/GtGvK**
- ICEbox: **https://oreil.ly/6nl1W**

Книги для программистов: https://t.me/free_books_for_it

- iml: **https://oreil.ly/x26l9**
- InterpretML: **https://oreil.ly/cuevp**
- lime: **https://oreil.ly/j5Cqj**
- Model Oriented: **https://oreil.ly/7wUMp**
- PiML: **https://oreil.ly/CqgSa**
- pdp: **https://oreil.ly/PasMQ**
- shapFlex: **https://oreil.ly/RADtC**
- vip: **https://oreil.ly/YcD2_**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 7

Объяснение классификатора изображений PyTorch

Глава 6 была посвящена использованию объяснимых моделей и апостериорным объяснениям моделей, обученных на табличных данных. В этой главе мы обсудим эти же концепции в контексте моделей *глубокого обучения* (deep learning, DL), обученных на неструктурированных данных, с особым акцентом на данных изображений. Примеры кода для этой главы доступны в Интернете, и помните, что в главе 2 представлены концепции объяснимых моделей и апостериорных объяснений.

Мы начнем эту главу с введения в гипотетический вариант использования и продемонстрируем его на технических примерах. Затем мы продолжим по аналогии с главой 6. Сначала мы освежим и углубим понятия объяснимых моделей и методов атрибуции признаков для глубоких нейронных сетей (с упором на возмущения), а также методов объяснения на основе градиентов. Мы также продолжим тему главы 6 и кратко обрисуем, как методы объяснений помогают в отладке модели — этот вопрос мы раскроем в главах 8 и 9.

Далее мы более подробно обсудим модели, объяснимые по своей сути. Мы предлагаем небольшой раздел, посвященный объяснимым моделям DL, в надежде, что некоторые читатели смогут построить свои собственные объяснимые модели, поскольку на сегодняшний день это лучший способ получить действительно объяснимые результаты. Мы представим модели классификации изображений на основе прототипов, такие как *цифровая маммография ProtoPNet (https://oreil.ly/Jht4n)* — многообещающее направление объяснимого компьютерного зрения. После этого мы обсудим методы апостериорного объяснения, где подробно выделим четыре метода: окклюзии (распространенный тип возмущения), input*gradient[1], интегрированные градиенты и послойное распространение релевантности. Мы воспользуемся гипотетическим примером использования рентгеновских снимков при диагностике пневмонии, чтобы указать на различные свойства этих методов, и попутно выделим некоторые важные детали реализации.



В главе 2 мы говорили, что *интерпретация* — это осмысленное ментальное представление на высоком уровне, которое контекстуализирует стимул и использует базовые знания человека, тогда как *объяснение* — это детальное ментальное представление на низком уровне, целью которого является описание сложного процесса. Интерпретация находится на гораздо более высоком уровне, чем объяснение, и редко достигается с помощью одних лишь технических подходов.

[1] Общепринятого русского перевода этого термина не существует, в очень редких статьях на русском пишут оригинальное название, иногда заменяя звездочку на крестик. — *Прим. пер.*

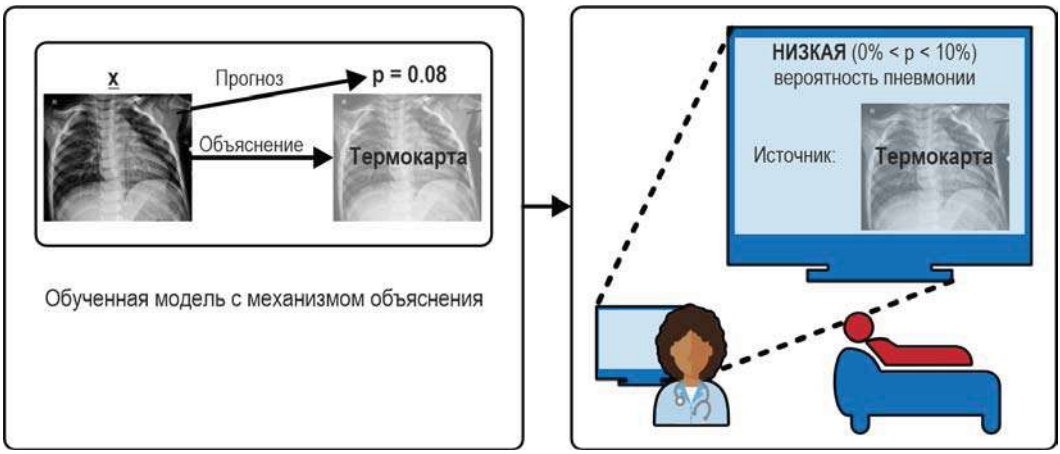
Книги для программистов: https://t.me/free_books_for_it

Как мы узнаем, хороши ли наши апостериорные объяснения? Чтобы ответить на этот вопрос, мы также обсудим исследования по оценке объяснений. Мы продемонстрируем эксперимент, впервые описанный в статье Sanity Checks for Saliency Maps (*https://oreil.ly/64UAi*), который покажет, что многие методы апостериорного объяснения на самом деле не так уж хорошо раскрывают наши модели!

Мы завершим главу обзором извлеченных уроков. В этой главе будет показано, что читателям следует с осторожностью применять стандартное решение DL в приложениях высокого риска, где необходимы пояснения моделей. Апостериорные объяснения зачастую плохо поддаются реализации интерпретации, а иногда бывают совершенно бессмысленными. Более того, широкий спектр различных методов объяснения означает, что мы рискуем случайно выбрать метод, который подтверждает наши априорные убеждения о том, как должна вести себя наша модель. (Предвзятость самоподтверждения обсуждается в главах 4 и 12.) Хуже необъяснимой модели может быть только необъяснимая модель в сочетании с неверным объяснением, подкрепленным предвзятостью самоподтверждения.

Как работает классификация рентгенограмм грудной клетки

В своих рассуждениях мы будем опираться на рабочий пример модели классификатора рентгеновских изображений пневмонии, а точнее, на гипотетический вариант прогнозов и объяснений модели, передаваемых опытному пользователю (например, врачу) для помощи в постановке диагноза. На рис. 7.1 представлена упрощенная схема, которая демонстрирует, как модель применяется в сочетании с механизмом объяснения для оказания помощи экспертам в диагностике пневмонии.



а) *б)*

Рис. 7.1. Гипотетический вариант применения модели, на который мы будем ссылаться на протяжении всей этой главы. Модель и механизм апостериорного объяснения (*а*) передают прогнозы и объяснения на удобочитаемую информационную панель. Врач использует эту информацию при диагностике пневмонии (*б*)

Книги для программистов: https://t.me/free_books_for_it

Апостериорные объяснения уже накопили историю широкого применения в контексте потребительского кредитования. Использование объяснений модели для помощи в интерпретации медицинских изображений не имеет такой истории. Более того, авторитетные статьи весьма критически отзываются об апостериорных методах объяснения и вариантах использования в медицине, подобных нашему гипотетическому, например The False Hope of Current Approaches to Explainable Artificial Intelligence in Health Care (*https://oreil.ly/KY6LD*) (Ложная надежда на современные подходы к объяснимому искусственному интеллекту в здравоохранении) и The Doctor Just Won't Accept That! (*https://oreil.ly/ZOlTk*) (Врач просто не пойдет на это!) Даже если подобный вариант использования оправдан, системы ML — пусть тщательно разработанные — могут плохо работать с данными, выходящими за пределы выборки. В статье Deep Learning Predicts Hip Fracture Using Confounding Patient and Healthcare Variables (*https://oreil.ly/V87hi*) приводится пример модели, улавливающей корреляции в обучающих данных, которые не работают должным образом после развертывания. В работе Diagnostic Accuracy and Failure Mode Analysis of a Deep Learning Algorithm for the Detection of Intracranial Hemorrhage (*https://oreil.ly/V4krV*) описан вариант использования, несколько похожий на тот, который представлен в этой главе, но с добавлением анализа реальных результатов. Мы обсудим все эти вопросы более подробно, проработав некоторые примеры апостериорных объяснений в конце главы.

Новый взгляд на объяснимые модели и методы апостериорного объяснения

В этом разделе мы обсудим основные идеи, лежащие в основе этой главы. Начнем с объяснимых моделей, но не будем рассматривать их с точки зрения кода. Затем в разделе, посвященном апостериорным объяснениям, мы продемонстрируем несколько методов на примере нашей модели, а также рассмотрим многие другие основополагающие методы в данной области. Эти методы можно разделить на две группы:

1. Методы, основанные на возмущениях (часто окклюзия в DL).
2. Методы, основанные на градиенте.

Далее мы обсудим различия между этими категориями, а также расскажем, как эти методы можно применить к задаче отладки модели и выбора архитектуры.

Обзор объяснимых моделей

В главе 2 было отмечено, что объяснимость модели вытекает из объяснимости структуры, характеристик или результатов. Кроме того, существует широкий спектр объяснимых моделей. Некоторые из них могут быть напрямую объяснены конечным пользователям, а другие имеют смысл только для высококвалифицированных специалистов по обработке данных. Объяснимые модели DL определенно относятся к более сложной части спектра объяснимости, но мы по-прежнему считаем их очень важными. Как мы уже подчеркивали и как указывали многие другие

Книги для программистов: https://t.me/free_books_for_it

исследователи, надо быть очень осторожными с апостериорными объяснениями в области DL. Если у нас есть объяснимая модель, мы можем понять ее напрямую, без использования сомнительных апостериорных методов. Также можем сравнить результаты апостериорного объяснения с объяснимыми механизмами модели, чтобы протестировать модель и сравнить различные подходы к объяснению.

Методы окклюзии

Методы окклюзии основаны на идее возмущения, удаления или маскировки признаков и изучения результирующих изменений выходных данных модели. В компьютерном зрении это часто означает затемнение участков пикселов. Как показано в статье Explaining by Removing: A Unified Framework for Model Explanation (*https://oreil.ly/6hGen*), множество различных методов объяснения в итоге сводятся к идее окклюзии признаков.

Методы на основе окклюзии могут быть особенно ценными, когда градиенты недоступны или когда модель, которую мы пытаемся объяснить, представляет собой сложный конвейер принятия решений, включающий машинное обучение, бизнес-правила, эвристику и другие недифференцируемые компоненты. Все методы, основанные на окклюзии, сталкиваются с одной и той же сложностью: для большинства моделей мы не можем просто удалять признаки и генерировать прогнозы модели. Другими словами, если наша модель обучена на признаках `x1`, `x2` и `x3`, мы не можем просто передать ей значения `x1` и `x2` и ожидать, что она сделает прогноз. Нам нужно передать какое-то значение для `x3`. Эта деталь лежит в основе многих различных методов, основанных на окклюзии.

Градиентные методы

Как обсуждалось в главе 2, для построения локальных объяснений можно использовать градиенты выходов модели по отношению к ее параметрам. Это обобщение идеи интерпретации коэффициентов регрессии. Помните, что градиент — это всего лишь локальная линейная аппроксимация сложной функции: нашей модели ML. Поскольку подавляющее большинство архитектур DL предназначено для обучения с помощью оптимизаторов на основе градиентов, у нас почти всегда есть доступ к некоторым градиентам в моделях DL, а современные наборы инструментов DL сделали оценку градиентов намного проще. Отчасти поэтому методы объяснения на основе градиентов стали такими популярными в области DL. Однако в случае древовидных моделей или сложных конвейеров, где использование градиентов невозможно, нам придется вернуться к окклюзии.

Пояснения в этой категории по сути спрашивают: «Какие входные признаки, если их слегка изменить, приведут к наибольшим изменениям на выходе нашей модели?» Исследователи разработали множество вариантов этой методики, чтобы выявить едва заметные нюансы объяснений. Подробности мы рассмотрим позже в этой главе, когда обсудим такие методы, как input*gradient, интегрированные градиенты и послойное распространение релевантности.

Книги для программистов: https://t.me/free_books_for_it

Объяснимый ИИ для отладки моделей

В главе 6 вы увидели, как методы объяснения модели, такие как частичная зависимость и диаграммы ICE, могут выявить нежелательное поведение модели, например чувствительность к паразитному шуму в обучающих данных. В области моделей DL объяснения играют примерно такую же роль, и на сегодняшний день это можно считать высшей целью *объяснимого искусственного интеллекта* (explainable artificial intelligence, XAI) в DL. Известные исследователи неоднократно отмечали, что методы объяснения DL могут быть полезны для отладки и улучшения моделей. Самый известный пример — классическая запись в блоге Google, популяризирующая «мечты» нейронных сетей. Авторы используют технику из статьи Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps (*https://oreil.ly/5BqAj*) для отладки своей модели, где просят ее раскрыть концепцию гантели:

> Да, на картинке есть гантели, но надо отметить, что ни одно изображение гантелей не обошлось без мускулистого тяжелоатлета, который их поднимает. В данном случае сеть не смогла полностью раскрыть суть гантели. Возможно, ей никогда не показывали гантели без рук, которые их держат. Визуализация помогает нам исправить подобные ошибки в обучении.

С дополнительными примерами применения объяснений в качестве инструментов отладки DL можно ознакомиться в следующих публикациях:

- Visualizing Higher-Layer Features of a Deep Network (*https://oreil.ly/vIG4Y*);
- Visualizing and Understanding Convolutional Networks (*https://oreil.ly/aEkYG*);
- Deconvolutional Networks (*https://oreil.ly/NmiDE*).

Мы обращаем внимание читателей на эти ресурсы по двум причинам. Во-первых, популярные методы объяснения DL, которые мы рассмотрим в текущей главе, не всегда работают, и другие методы, описанные в этих статьях, определенно заслуживают внимания. Во-вторых, хотя методы объяснения могут подвести нас, когда дело доходит до точного понимания, они все же могут указывать на проблемы в наших моделях. Имейте это в виду во время чтения главы. В главе 9 мы углубимся в применение бенчмаркинга, анализа чувствительности и анализа остатков для отладки моделей DL.

Объяснимые модели

Мы начнем техническую часть главы с обсуждения объяснимых моделей для приложений DL, поскольку на данный момент они являются лучшим выбором для получения объяснимых результатов. Но работать с ними пока непросто. Читатели узна́ют, что существуют лишь несколько готовых объяснимых моделей DL, которые мы можем подключить к своему приложению. Сравните это с материалом главы 6, где мы могли выбирать из множества вполне объяснимых архитектур: от монотонной XGBoost до объяснимых бустинговых машин, обобщенных линейных моделей (GLM) и обобщенных аддитивных моделей (GAM). Так в чем же разница? Почему в области DL существует так мало готовых объяснимых моделей? Одна из

Книги для программистов: https://t.me/free_books_for_it

причин заключается в том, что объяснимые модели структурированных данных, в отличие от DL, уходят корнями к работам Гаусса 1800-х годов. Но это еще не все.

Когда мы обучаем модель DL на неструктурированных данных, мы на самом деле просим модель выполнять две функции:

- извлечение признаков для создания представления в скрытом пространстве или изучение правильного представления данных в (обычно) низкомерном пространстве ввода;
- модель должна использовать это скрытое пространственное представление для прогнозирования.

Сравните это с табличными данными, которые мы анализировали в главе 6. При использовании табличных данных обычно предполагается, что «правильное» представление наших данных находится непосредственно в самих данных, особенно если мы правильно выполнили свою работу и выбрали разумный набор некоррелированных признаков, имеющих известную причинно-следственную связь с целью. Эта разница (между извлекаемыми и готовыми признаками) помогает понять, почему объяснимые модели DL так сложно разрабатывать и почему трудно найти готовые реализации.

Общей чертой существующих сегодня объяснимых архитектур DL является то, что они напрямую вмешиваются в процесс изучения признаков. Часто объяснимые модели перекладывают бремя конструирования признаков с модели на ее разработчика. Это возросшее бремя является одновременно благословением и проклятием. С одной стороны, оно означает, что обучение таких моделей более трудоемко, чем обучение необъяснимых моделей. С другой стороны, эти архитектуры могут требовать более качественных, иногда профессионально аннотированных данных, что означает глубокую вовлеченность людей в сквозной процесс моделирования. Поскольку нам все равно приходится тщательно проектировать наши модели в соответствии с требованиями объяснимых архитектур DL, это не так уж и плохо. Как показано в главе 6, чем больше знаний предметной области будет закодировано в наших моделях, тем больше можно будет доверять их прогнозам в реальном мире.

В следующих разделах мы обсудим различные архитектуры объяснимых моделей DL, продолжая придерживаться примера классификации изображений. Особое внимание уделим недавним разработкам в области архитектур на основе прототипов. Они открывают наиболее вероятный путь к настоящей объяснимой классификации изображений «под ключ».

ProtoPNet и ее варианты

В своей статье 2019 г. This Looks Like That: Deep Learning for Interpretable Image Recognition (*https://oreil.ly/k69Dx*) команда Дьюкского университета под руководством профессора Синтии Рудин (Cynthia Rudin) представила новую многообещающую архитектуру для объяснимой классификации изображений. Новая модель называется ProtoPNet и основана на концепции прототипов.

В главе 2 было сказано, что прототипы — это точки данных, которые представляют большую группу наблюдений. Допустим, вас спросили, почему некое наблюдение

Книги для программистов: https://t.me/free_books_for_it

оказалось в определенном кластере при кластеризации методом *k*-средних. В ответ вы могли бы поместить наблюдение рядом с центроидом скопления и сказать: «Смотрите, это наблюдение похоже на центр кластера». ProtoPNet генерирует объяснения именно такого типа. Более того, объяснения ProtoPNet соответствуют тому, как модель на самом деле делает прогнозы. Итак, как оно работает?

Во-первых, ProtoPNet идентифицирует прототипные *патчи* (patch, характерный участок) для каждого класса. Эти патчи отражают фундаментальные свойства, отличающие один класс от другого. Затем, чтобы сделать прогнозы, модель ищет на входном изображении патчи, похожие на прототипы для определенного класса. Полученные оценки сходства для каждого прототипа суммируются, чтобы получить вероятность принадлежности входных данных каждому классу. Конечным результатом является модель, являющаяся аддитивной (каждый прогноз представляет собой сумму оценок сходства между частями прототипного изображения), а также разреженной (в каждом классе имеется всего несколько прототипов). Самое приятное то, что каждое предсказание сразу же сопровождается верным объяснением.



ProtoPNet основывается на идее моделей с вниманием на уровне частей — широком классе объяснимых глубоких нейронных сетей, которые не рассматриваются в этой главе. Разница между этими моделями и ProtoPNet заключается в верности. ProtoPNet действительно делает прогнозы, суммируя показатели сходства с определенными патчами каждого изображения и определенными прототипами классов. Другие модели внимания на уровне частей не дают таких гарантий.

После публикации в 2019 г. это многообещающее направление объяснимой классификации изображений подхватили другие исследователи. Например, модель ProtoPShare (*https://oreil.ly/phO4I*) позволяет совместно использовать прототипы между классами, что приводит к уменьшению количества прототипов. Существует также модель ProtoTree (*https://oreil.ly/OKEp0*), которая создает объяснимое дерево решений для признаков прототипа. Такая архитектура позволяет модели еще более четко имитировать человеческие рассуждения. Наконец, Ким и др. проанализировали рентгенограммы грудной клетки, используя архитектуру, очень похожую на ProtoPNet, в работе XProtoNet: Diagnosis in Chest Radiography with Global and Local Explanations (*https://oreil.ly/qv-oO*).

Другие объяснимые модели глубокого обучения

В статье Towards Robust Interpretability with Self-Explaining Neural Networks (*https://oreil.ly/DWY6w*) авторы Дэвид Альварес-Мелис и Томми Яаккола представляют *самообъясняемые нейронные сети* (self-explaining neural networks, SENN). Как мы уже говорили, одна из трудностей создания объяснимых моделей на основе неструктурированных данных заключается в том, что мы просим наши модели создавать скрытое пространственное представление наших данных и делать прогнозы. Самообъясняемые нейронные сети решают эту проблему, вводя *интерпретируемые базовые концепции* (interpretable basis conceptsë) вместо необработанных признаков. Эти базовые концепции могут быть изучены в рамках обучения модели, взяты из репрезентативных наблюдений в обучающих данных или — в идеале —

Книги для программистов: https://t.me/free_books_for_it

разработаны экспертами в предметной области. В своей статье Альварес-Мелис и Яаккола генерируют эти интерпретируемые базовые концепции с помощью автокодировщика. Они гарантируют, что изученные концепции являются объяснимыми, при этом предоставляют прототипные наблюдения, которые максимально выражают суть концепций.



Автокодировщик (autoencoder) — это тип нейронной сети, которая учится извлекать признаки из обучающих данных, не делая прогнозов для единственной цели моделирования. Автокодировщики отлично подходят для визуализации данных и обнаружения аномалий.

Здесь мы в основном говорим о создании моделей компьютерного зрения. Объяснимые модели были разработаны также для глубоких нейронных сетей, используемых в обучении с подкреплением, визуальном рассуждении, табличных данных и прогнозировании временных рядов.

Обучение и объяснение классификатора изображений PyTorch

В этом практическом примере мы расскажем, как обучали классификатор изображений, а затем продемонстрируем, как генерировать объяснения, используя четыре различных метода: окклюзия, input*gradient, интегрированные градиенты и послойное распространение релевантности.

Обучающие данные

Итак, нам нужно создать классификатор изображений для диагностики рентгеновских изображений грудной клетки в соответствии с гипотетическим вариантом использования, показанным на рис. 7.1. Набор данных, который мы будем использовать для обучения, доступен на Kaggle и состоит из 5863 рентгеновских снимков пациентов, которые были разделены на две отдельные категории: одна с пневмонией, а другая — со здоровыми легкими. На рис. 7.2 показана случайная выборка изображений из обучающих данных.



Рис. 7.2. Случайная выборка из обучающего набора данных рентгенографии грудной клетки Kaggle; рентгенограммы грудной клетки при пневмонии более мутные, чем при ее отсутствии

Книги для программистов: https://t.me/free_books_for_it

Мы не врачи и не рентгенологи, и важно признать, что авторы этой книги не обладают медицинскими знаниями, чтобы по-настоящему проверить свою модель. Насколько мы понимаем, снимки пациентов с пневмонией должны показывать мутные области воспаления. Бактериальная и вирусная пневмония также имеют разные визуальные характеристики. Приступая к созданию модели, мы надеемся, что методы XAI сосредоточатся на этих мутных областях, что позволит нам понять, почему изображение классифицируется как пневмония, а не как норма. (Приготовьтесь разочароваться.) Чтобы узнать больше о наборе данных, посетите его страницу Kaggle и связанную с ней статью Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning (*https://oreil.ly/SOcBD*).



Если вы применяете ML в области приложений высокого риска, вам нужен опыт в предметной области, который поможет обучать и проверять ваши модели. Отсутствие консультации с экспертами в предметной области может привести к развертыванию вредных и бессмысленных моделей, особенно для приложений высокого риска.

Как и в случае с большинством наборов данных Kaggle, эксперты уже проделали за нас колоссальную работу по обработке данных. Некачественные сканы были удалены, а метки проверены на корректность. Однако, как и многие наборы данных в медицинских приложениях, у этих данных есть проблема дисбаланса классов: существуют 1342 снимка здоровых легких, при этом 3876 снимков помечены как указывающие на пневмонию. Еще одной причиной для беспокойства является наличие очень небольшого количества изображений в проверочном наборе данных. Проверочные данные состоят только из девяти изображений для класса пневмонии и еще девяти для класса нормы. Этого недостаточно для адекватной проверки модели. Мы постараемся решить эту и другие проблемы, прежде чем приступим к обучению модели.

Решение проблемы дисбаланса наборов данных

Согласно нашим обучающим данным число рентгеновских снимков легких с признаками пневмонии превышает количество снимков здоровых легких в три раза. Любая модель, обученная на таком наборе данных, подвержена риску переобучения на классе большинства. Существует несколько способов решения проблемы дисбаланса классов:

- увеличение количества выборок класса меньшинства;
- уменьшение количества выборок класса большинства;
- доработка функции потерь, чтобы по-разному взвешивать классы большинства и меньшинства.

Эти методы, а также пагубные последствия проблемы дисбаланса классов были хорошо обобщены в статье под названием A Systematic Study of the Class Imbalance Problem in Convolutional Neural Networks (*https://oreil.ly/Gp-OY*). В нашем примере мы будем увеличивать выборку снимков здоровых легких, чтобы выровнять дисбаланс классов.

Книги для программистов: https://t.me/free_books_for_it

Дополнение данных и обрезка изображений

PyTorch (*https://oreil.ly/Uagd2*) — это фреймворк машинного обучения с открытым исходным кодом. Torchvision (*https://oreil.ly/LaOh8*) — это профильная библиотека для PyTorch, созданная для поддержки исследований и экспериментов в области компьютерного зрения. Torchvision состоит из нескольких популярных наборов данных, предварительно обученных моделей и некоторых преобразований изображений для задач компьютерного зрения. Сначала мы увеличим долю проверочного набора, переместив в него некоторые изображения обучающего набора. После этого мы воспользуемся функциями преобразования изображений torchvision, чтобы справиться с дисбалансом классов в нашем обучающем наборе. В следующем фрагменте кода мы масштабируем изображения до одинакового размера, затем применяем различные преобразования, чтобы увеличить размер данных и представить обучающие примеры, которые должны повысить надежность нашей модели. Функция `get_augmented_data` широко использует преобразования `RandomRotation` и `RandomAffine` для создания новых (измененных) обучающих изображений, а также использует другие преобразования для форматирования и нормализации изображений:

```
TRAIN_DIR = 'chest_xray_preprocessed/train'
IMAGE_SIZE = 224 # Размер изображения после применения преобразования
BATCH_SIZE = 32
NUM_WORKERS = 4 # Количество параллельных процессов подготовки данных

def get_augmented_data():

    sample1 = ImageFolder(TRAIN_DIR,
                          transform =\
            transforms.Compose([transforms.Resize((224,224)),
                          transforms.RandomRotation(10),
                          transforms.RandomGrayscale(),
                          transforms.RandomAffine(
                            translate=(0.05,0.05),
                            degrees=0),
                          transforms.ToTensor(),
                          transforms.Normalize(
                            [0.485, 0.456, 0.406],
                            [0.229, 0.224, 0.225]),
                            ]))

    ...

    return train_dataset
```

Поскольку основной смысл дополнения данных заключается в создании большего количества изображений, давайте проверим результат:

Книги для программистов: https://t.me/free_books_for_it

```
# проверяем размер нового набора данных
print(f'Normal : {normal} and Pneumonia : {pneumonia}')
```

```
(3516, 3758)
```

На рис. 7.3 показаны некоторые примеры синтетических обучающих данных, созданных с использованием вращений и перемещений.

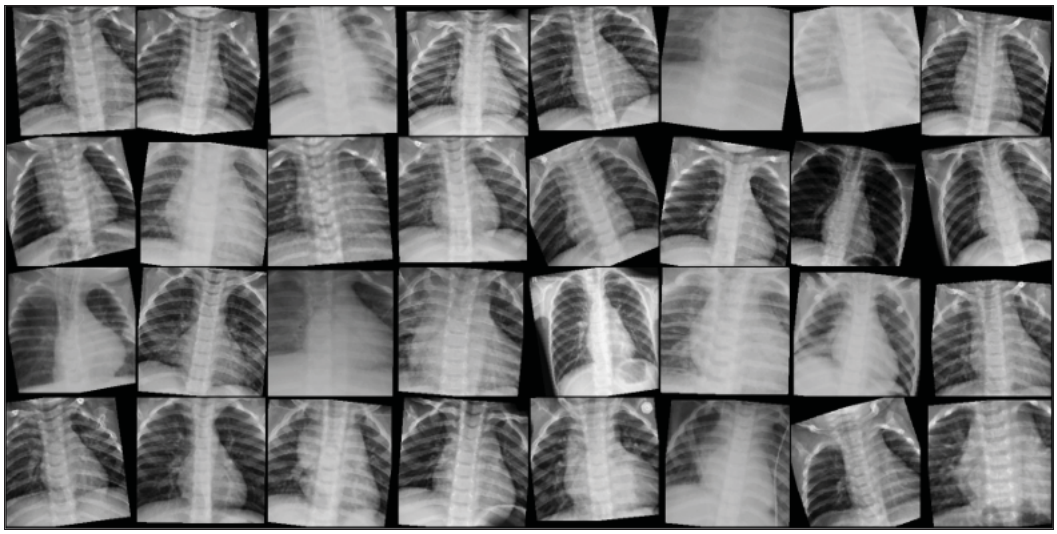


Рис. 7.3. Синтетические обучающие выборки, созданные с использованием вращений и перемещений

Похоже, все правильно. Устранив дисбаланс классов и дополнив данные, мы приступим к обучению модели.



Убедитесь, что дополнение данных не создает нереалистичных обучающих выборок. Рентгенограммы грудной клетки могут демонстрировать разную цветовую шкалу, масштабирование и т. д. Однако зеркальное переворачивание изображений относительно вертикальной оси было бы огромной ошибкой, поскольку наши органы двусторонне асимметричны (левая и правая стороны человеческого тела не одинаковы). В реальной жизни эта модель никогда не встретит рентгенограмму грудной клетки с сердцем пациента на правой стороне, поэтому ее не следует обучать на изображениях, перевернутых относительно вертикальной оси.

Еще один метод предварительной обработки, который мы использовали для набора данных, — это обрезка изображения. Мы обрезали некоторые изображения в обучающем наборе, чтобы выделить только область легких (рис. 7.4). Обрезка помогает устранить любые надписи или другие виды отметок на рентгеновских изображениях грудной клетки и фокусирует модель на интересующей нас области изображений. Мы сохранили эти изображения как отдельный набор данных, чтобы использовать их для точной настройки сети на более позднем этапе обучения.

Книги для программистов: https://t.me/free_books_for_it

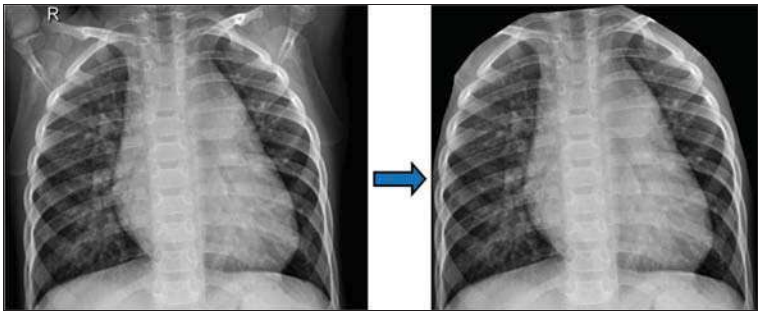


Рис. 7.4. Рентгенограмма грудной клетки до и после обрезки



Только когда мы вручную обрезали сотни изображений, мы заметили, что обучающие данные содержат несколько снимков одного и того же пациента. Поэтому при добавлении изображений к нашим проверочным данным мы старались сделать так, чтобы снимок пациента находился либо в обучающих, либо в проверочных данных, но не в обоих сразу. Эта деталь является отличным примером утечки данных и подчеркивает важность тщательного изучения данных, которыми вы располагаете.

Обучение модели

Сверточные нейронные сети (convolutional neural networks, CNN) (*https://oreil.ly/bfzDc*) — это широко распространенная архитектура, используемая в том числе в медицинской визуализации. Хорошо известными примерами CNN для классификации изображений являются ResNet (*https://oreil.ly/2JqoN*), DenseNet (*https://oreil.ly/T3b0q*) и EfficientNet (*https://oreil.ly/-AWuI*). Обучение CNN с нуля обходится очень дорого как с точки зрения данных, так и с точки зрения вычислительных ресурсов. Поэтому разработчики часто используют распространенный подход, при котором берут модель, уже обученную на крупномасштабном наборе данных изображений, например ImageNet, а затем повторно используют ее в качестве отправной точки для решения других задач.

Основная идея этого подхода заключается в том, что нижние слои CNN изучают базовые представления, такие как края и углы, которые можно обобщить для широкого круга задач. При переносе модели на свои задачи такие слои CNN называют *предварительно обученными*. С другой стороны, более высокие слои фиксируют признаки более высокого уровня, специфичные для конкретной задачи. В результате выходные данные этих слоев не будут подходить для нашего варианта задачи. Следовательно, мы можем заморозить признаки, изученные на нижних уровнях, и переобучить более высокие уровни на этапе, называемом *тонкой настройкой* (fine-tuning). Предварительное обучение и тонкая настройка совместно друг с другом представляют собой простую форму *переноса обучения* (transfer learning) (*https://oreil.ly/tybad*). При этом знания, полученные с помощью модели ML в одной предметной области, переносятся в другую область.

Для начала мы будем использовать архитектуру DenseNet-121 (*https://oreil.ly/Wq743*), обученную на наборе данных ImageNet. Было показано, что модели DenseNet осо-

Книги для программистов: https://t.me/free_books_for_it

бенно хорошо работают для классификации рентгеновских изображений, поскольку они улучшают поток информации и градиентов через сеть (*https://oreil.ly/_fO24*), в идеале повышая качество модели и расширяя возможности классификатора.



Не забывайте про EvilModel (*https://oreil.ly/UMwPx*). Было показано, что вредоносное ПО может быть внедрено через предварительно обученные нейронные сети. Такое вредоносное ПО может не оказывать влияния на показатели модели, но при этом успешно обходить антивирусное программное обеспечение. (Кроме того, мы просто можем забыть отсканировать артефакты нашей модели.) Уроки главы 5 учат нас не принимать все как должное, даже предварительно обученные модели, загружаемые из Интернета.

При переносе обучения возникает важный вопрос: обучать заново следует все слои предварительно обученной модели или только некоторые из них? Ответ на этот вопрос кроется в составе набора данных. Достаточно ли велик новый набор данных? Похож ли он на набор данных, на котором обучалась предварительно обученная модель? Поскольку наш набор данных невелик и значительно отличается от исходного набора данных, имеет смысл обучить заново некоторые нижние слои, а также более высокие. Это связано с тем, что нижние слои изучают обобщенные признаки, а верхние изучают больше признаков, специфичных для набора данных. В данном случае, когда мы говорим о повторном обучении слоев, это не означает начинать обучение с нуля или со случайными весами; вместо этого мы будем использовать предварительно обученные веса в качестве отправной точки — фактически мы продолжим обучение.

В следующем коде мы размораживаем все слои предварительно обученной модели и заменяем последний слой нашим собственным линейным классификатором. Для этого набора данных такой прием показал наилучшее качество модели при тестировании. Мы также поэкспериментировали с размораживанием лишь нескольких слоев, но ни один из наших экспериментов не оказался лучше первого варианта:

```
classes = ['Normal', 'Pneumonia']
model = torchvision.models.densenet121(pretrained=True)

# Размораживаем все слои "признаков" для обучения
for param in model.parameters():
    param.requires_grad = True

# Новые слои автоматически имеют requires_grad = True
in_features = model.classifier.in_features
model.classifier = nn.Linear(in_features, len(classes))
```

Итак, мы выполнили вторую тонкую настройку модели, используя только изображения, которые мы обрезали вручную, чтобы сфокусироваться на снимке легких. Процесс двойной тонкой настройки в итоге выглядел примерно так:

1. Загружаем предварительно обученную модель DenseNet-121.
2. Обучаем модель на расширенном наборе данных, используя необрезанные изображения.

Книги для программистов: https://t.me/free_books_for_it

3. Замораживаем первые слои модели и продолжаем обучение на обрезанных изображениях.

Идея этого процесса двойной тонкой настройки заключается в использовании признаков, изученных предварительно обученной моделью, а также признаков из нашего набора данных для конкретной предметной области. Наконец, использование обрезанных изображений для последнего этапа обучения снижает риск того, что модель станет использовать признаки, которые не будут обобщены на незнакомые данные вроде артефактов рентгеновского снимка за пределами легких.

Оценка и метрики

Качество модели оценивается на проверочном наборе. В табл. 7.1 и 7.2 мы также сообщаем о некоторых показателях модели на наборе незнакомых тестовых данных. Измерение этого показателя качества важно для понимания того, будет ли наша модель хорошо обобщать незнакомые данные.

Таблица 7.1. Матрица несоответствий, показывающая эффективность модели классификатора пневмонии на наборе тестовых данных

	Прогноз «норма»	Прогноз «пневмония»
Действительно норма	199	35
Действительно пневмония	11	379

Таблица 7.2. Дополнительные показатели качества на наборе тестовых данных

	Преобладание	Точность	Полнота	F1
Норма	234	0,95	0,85	0,90
Пневмония	390	0,92	0,97	0,94

Модель здесь выглядит хорошо, но обязательно прочтите главу 9, чтобы увидеть, как проверка in silico и тестовые измерения могут вводить в заблуждение. Теперь пришло время объяснить прогнозы нашей модели.

Создание апостериорных объяснений с помощью Captum

В этом разделе мы подробно остановимся на нескольких апостериорных методах и покажем их применение в нашем классификаторе рентгеновских снимков. Все генерируемые объяснения носят *локальный* характер, поскольку они применимы к индивидуальным наблюдениям — одиночным рентгеновским изображениям пациента. Более того, все объяснения будут иметь форму *тепловой карты* (heatmap), где цвет каждого пиксела должен быть пропорционален значимости этого пиксела для окончательной классификации. В следующих разделах мы критически рассмотрим, достигают ли эти методы своей цели, но назначение текущего раздела — сначала просто показать, каких результатов можно ожидать от различных методов.

Книги для программистов: https://t.me/free_books_for_it

Для реализации объясняющих методов мы будем использовать Captum. Это библиотека объяснений моделей, построенная на PyTorch и поддерживающая множество моделей «из коробки». Она предлагает реализации многих алгоритмов объяснения, которые хорошо работают с различными моделями PyTorch.

Окклюзия

Окклюзия — это разновидность метода возмущения признаков, которая работает по простому принципу: удалить конкретный входной признак из модели и оценить разницу в ее прогнозирующей способности до и после удаления. Более существенное различие подразумевает, что признак важен, и наоборот. Окклюзия представляет собой замену определенных частей входного изображения и изучение влияния на выходные данные модели. Ее часто реализуют путем перемещения прямоугольного окна заранее определенного размера и шага по изображению. Затем окно заменяется базовым значением (обычно нулевым) в каждом месте, в результате чего появляется серое пятно. Перемещая это серое пятно по изображению, мы закрываем части изображения и проверяем, насколько уверенно или точно модель делает прогнозы на основе измененных данных.

Документация Captum описывает реализацию окклюзии (*https://oreil.ly/R5C2N*), а мы применяем ее к рентгеновскому снимку грудной клетки для одного входного изображения. Обратите внимание, что мы можем указать размер закрывающего окна, а также размер шага, которые в нашем случае составляют 15×15 и 8 соответственно:

```
import captum, Occlusion
from captum.attr import visualization as viz

occlusion = Occlusion(model)

attributions_occ = occlusion.attribute(input,
                                       target=pred_label_idx,
                                       strides=(3, 8, 8),
                                       sliding_window_shapes=(3, 15, 15),
                                       baselines=0)
```

На рис. 7.5 показана атрибуция изображения из тестового набора, которое демонстрирует пневмонию и было правильно классифицировано моделью как пневмония.

Результаты многообещающие. Модель, похоже, уловила высокую непрозрачность верхних отделов обоих легких. Это может вселить в эксперта-интерпретатора объяснений уверенность в правильной работе классификатора. Однако темные области большие и лишены деталей, поэтому можно предположить, что меньшее окно окклюзии и длина шага могут показать больше деталей. (Мы не стали пробовать разные настройки, потому что, начиная настраивать гиперпараметры механизма объяснений, мы подвергаем себя риску выбрать удобные значения, при которых получаются объяснения, подтверждающие наши априорные убеждения о том, как работает модель.)

Книги для программистов: https://t.me/free_books_for_it

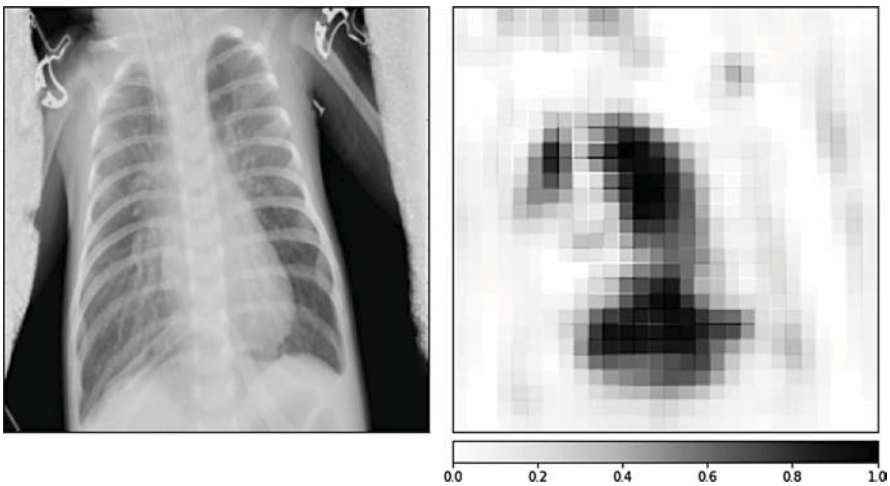


Рис. 7.5. Тепловая карта окклюзии для рентгеновского снимка пневмонии в тестовом наборе, где модель правильно спрогнозировала пневмонию

Помимо проблемы настройки, объяснение также показывает, что модель обращает внимание на некоторые группы пикселов за пределами туловища. Является ли это проблемой нашей модели, возможно, наводящей на мысль о переобучении, утечкой данных между обучающими и тестовыми данными или ложным обучением на ошибочных признаках? Или это артефакт самой методики объяснения? Нам остается только гадать. Давайте посмотрим, обеспечивают ли градиентные методы бóльшую ясность.



Ложное обучение (shortcut learning) — распространенная проблема в сложных моделях, которая часто проявляется в реальном мире. Это происходит, когда модель находит и изучает что-то более простое, чем фактическая цель прогнозирования в задаче ML — по сути, обманывая себя во время обучения. Поскольку медицинская диагностика по изображениям может быть сложной задачей даже для очень опытных специалистов-практиков, системы машинного обучения часто находят «обходные пути», которые помогают им оптимизировать свои функции потерь в обучающих данных. Когда эти изученные ложные признаки недоступны в реальных диагностических сценариях, такие модели терпят неудачу. Вы можете узнать больше о ложном обучении на медицинских изображениях в статье Deep Learning Applied to Chest X-Rays: Exploiting and Preventing Shortcuts (*https://oreil.ly/oVT-G*). Более общий обзор этой серьезной проблемы, которая беспокоит почти всех, кто работает с необъяснимым машинным обучением, представлен в работе Shortcut Learning in Deep Neural Networks (*https://oreil.ly/ogNeg*).

input*gradient

Первым методом, основанным на градиенте, который мы рассмотрим, будет input*gradient. Как следует из названия, этот метод создает локальную атрибуцию признака, равную градиенту прогноза по отношению к входным данным, умноженному на само входное значение. Почему? Возьмем линейную модель. Произведение градиента и входного значения присваивает признаку локальную атрибуцию, эквивалентную значению признака, умноженному на коэффициент признака. Этот коэффициент отражает вклад признака в конкретный прогноз.

Книги для программистов: https://t.me/free_books_for_it

Мы применили Captum для создания тепловой карты для того же изображения тестового набора, используя на этот раз метод input*gradient. На рис. 7.6 показаны *положительные* доказательства классификации.

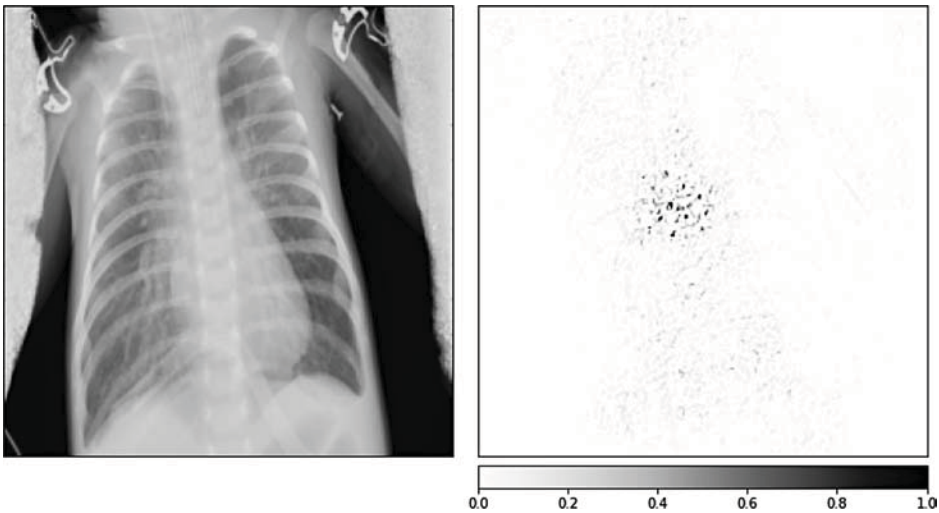


Рис. 7.6. Тепловая карта по методу input*gradient для рентгеновского изображения пневмонии в тестовом наборе, где модель дала правильный диагноз о наличии пневмонии

Что сказать об этом выходе? Как и в случае с окклюзиями, мы можем посмотреть на изображение и заявить, что оно содержит нечто значимое. В частности, в легких имеется темное пятно высокой степени выраженности, которое, по-видимому, соответствует высокой непрозрачности на рентгенограмме грудной клетки слева. Именно такого поведения мы ожидаем от нашего классификатора пневмоний. Однако методика объяснения также предполагает, что по всей длине позвоночника пациента имеются участки с высокой степенью признаков пневмонии. У нас остались те же вопросы, которые мы задавали ранее. Говорит ли это нам, что модель фокусируется не на тех вещах, т. е. обучилась на ошибочных признаках? Такой вывод был бы полезен для отладки модели. С другой стороны, мы до сих пор не знаем, не является ли сама методика объяснения источником противоречивого результата.

Интегрированные градиенты

Метод *интегрированных градиентов* (integrated gradients) (*https://oreil.ly/Er6tk*) — первый из числа методов, которые нам предстоит рассмотреть, имеющий теоретическое обоснование достоверности. Сами по себе градиенты могут быть обманчивыми, особенно потому, что градиенты стремятся к нулю для некоторых прогнозов с высокой степенью достоверности. В случае результатов с высокой вероятностью функции активации нередко достигают высоких значений, когда градиенты становятся насыщенными, плоскими и близкими к нулю. Это означает, что некоторые из наиболее важных функций активации, наиболее сильно влияющие на решения, не будут отображаться, если смотреть только на градиенты.

Книги для программистов: https://t.me/free_books_for_it

Подход интегрированных градиентов пытается решить эту проблему, измеряя влияние признака относительно базового значения для всех возможных значений интенсивности входных пикселов. В частности, метод интегрированных градиентов задает вопрос: «Как меняется градиент по мере прохождения пути от базового значения интенсивности входного пиксела к большему значению интенсивности входного пиксела?» Окончательная атрибуция признака представляет собой приблизительный интеграл градиента вдоль этого гладкого пути значений пикселов как функции прогнозов модели.

Интегрированные градиенты удовлетворяют аксиомам чувствительности и инвариантности реализации. Чувствительность означает, что если входное изображение и базовое изображение различаются только по одному признаку и если они возвращают разные выходные данные модели, то метод интегрированных градиентов вернет ненулевую атрибуцию для этого признака. Инвариантность реализации означает, что если две модели, возможно, с разными внутренними структурами, возвращают одинаковые выходные данные для всех входных данных, то все атрибуции входов, возвращаемые методом интегрированных градиентов, будут равны. Инвариантность реализации — это еще один способ обсудить *согласованность* из главы 2. Для более глубокого погружения в эту тему мы рекомендуем ознакомиться с введением об интегрированных градиентах в документации TensorFlow (*https://oreil.ly/2aUWD*).

На рис. 7.7 показаны результаты применения этого метода атрибуции к тому же изображению с пневмонией, которое мы рассматривали ранее. Как и в случае применения метода input*gradient, изображение зашумлено и его трудно интерпретировать. Похоже, метод улавливает *края* входного изображения. Помимо отметок рентгеновского аппарата и подмышек, на тепловой карте мы также можем увидеть

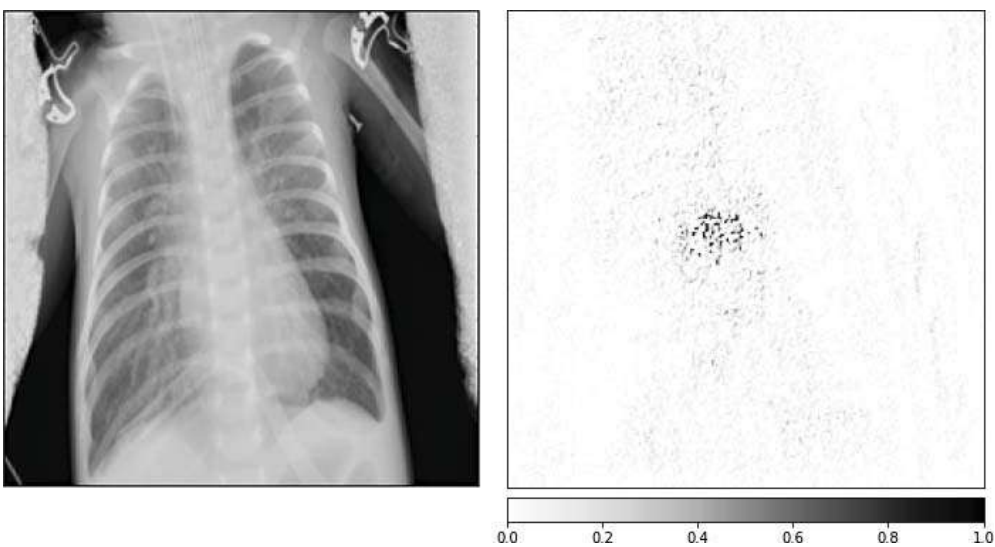


Рис. 7.7. Тепловая карта по методу интегрированных градиентов для рентгеновского изображения пневмонии в тестовом наборе, где модель дала правильный диагноз о наличии пневмонии

Книги для программистов: https://t.me/free_books_for_it

слабые очертания ребер пациента. Означает ли это, что модель игнорирует ребра и смотрит вокруг них, в легкие? Похоже, что такие объяснения вызывают больше вопросов, чем дают ответов. Позже в этой главе мы критически оценим подобные результаты объясняющих методов, проведя несколько экспериментов. А сейчас мы обратимся к четвертому и последнему методу, основанному на градиенте, — послойному распространению релевантности.

Послойное распространение релевантности

Послойное распространение релевантности (layer-wise relevance propagation, LRP) (*https://oreil.ly/xGtUm*) на самом деле представляет собой класс методов, которые измеряют *релевантность* признака выходным данным. В широком смысле релевантность — это сила связи между входными признаками и выходными данными модели, и ее можно измерить, не внося никаких изменений во входные признаки. Выбирая другое понятие релевантности, мы можем прийти к ряду различных объяснительных результатов. Читателям, заинтересованным в более подробном знакомстве с LRP, мы рекомендуем изучить главу Layer-Wise Relevance Propagation: An Overview в книге Explainable AI: Interpreting, Explaining and Visualizing Deep Learning (Samek et al., Springer Cham). Там вы найдете подробное обсуждение различных правил релевантности и того, когда их следует применять. Демонстрационная онлайн-панель The Explainable AI Demos (*https://oreil.ly/wcIVJ*) также позволяет генерировать поясняющие результаты, используя широкий набор правил LRP.

Преимущество LRP в том, что объяснения, которые он дает, локально точны: сумма оценок релевантности равна выходным данным модели. В этом смысле метод похож на значения Шепли. Взгляните на объяснение LRP для изображения из тестового набора на рис. 7.8. К сожалению, с точки зрения проверяемого человеком объяснения этот результат все еще оставляет желать лучшего.

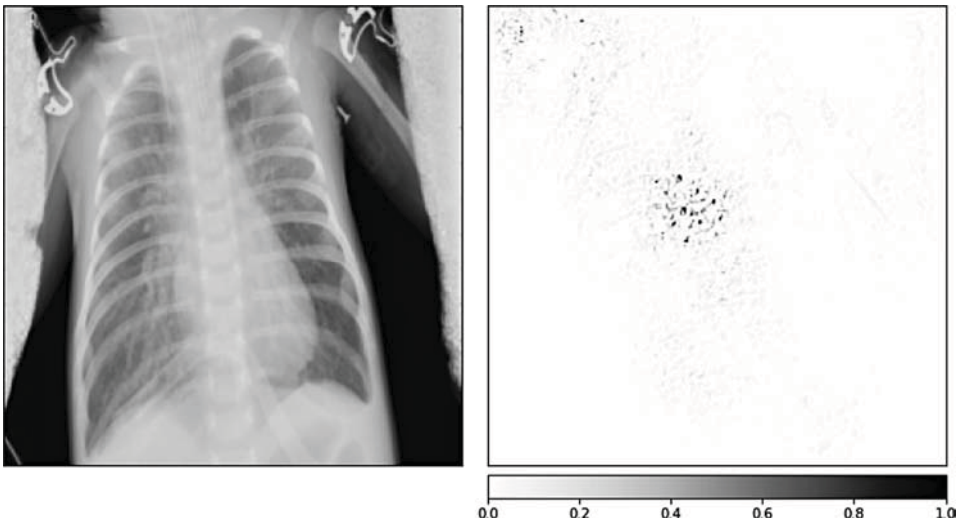


Рис. 7.8. Тепловая карта по методу LPR для рентгеновского изображения пневмонии в тестовом наборе, где модель дала правильный диагноз о наличии пневмонии

Книги для программистов: https://t.me/free_books_for_it

Как и другие методы, LRP выявил область повышенного затемнения в правом легком, но также дал высокий балл атрибуции областям за пределами легких.

Оценка пояснений модели

В предыдущих разделах мы лишь поверхностно рассмотрели объяснительные методы атрибуции для моделей DL. Количество таких методов велико и постоянно растет. В этом разделе мы ответим на важный вопрос: «Как узнать, насколько хорошее объяснение мы получили?» Затем мы проведем эксперимент, чтобы критически изучить, сколько информации дают нам методы апостериорного объяснения о нашей модели CNN.

Дэвид Альварес-Мелис и Томми Яаккола приводят замечательный анализ оценки объяснений в своих двух статьях 2018 г. On the Robustness of Interpretability Methods (*https://oreil.ly/KRcmm*) и Towards Robust Interpretability with Self-Explaining Neural Networks (*https://oreil.ly/gUWIR*). Во второй статье авторы описывают три желательных свойства, которыми должны обладать все объяснения:

- они должны быть понятными (ясность/разборчивость);
- они должны свидетельствовать об истинной важности (достоверность);
- они должны быть нечувствительны к небольшим изменениям входных данных (стабильность/надежность).

Методы тепловых карт, рассмотренные в текущей главе, явно не соответствуют первому пункту. Прежде всего, результаты этих методов зашумлены и сбивают с толку. Что еще более важно, все рассмотренные нами методы, похоже, указывают на бессмысленные области, такие как пространства вне тела пациента.

Даже если результаты полностью совпадают с нашими интуитивными ожиданиями, эти тепловые карты лишь указывают на то, *где* модель находит положительные или отрицательные доказательства для своей классификации; они не дают никаких предположений о том, *как* модель принимает решение на основе предоставленной ей информации. В этом заключается отличие от объяснимых моделей, таких как SENN или ProtoPNet, которые обеспечивают и то и другое: прототипы или базовые концепции — это ответ на вопрос «*где?*», а их линейная комбинация объясняет *как*. Вопрос «как?» — важнейший элемент хорошего объяснения.



Вы всегда должны тестировать наши методы объяснения в приложениях высокого риска. В идеале вам следует сравнивать апостериорные объяснения с лежащими в их основе объяснимыми модельными механизмами. Для более стандартных подходов DL можно прибегнуть к следующим приемам:

- привлечение экспертов в предметной области и исследование пользователей для проверки разборчивости;
- удаление признаков, считающихся важными, методы ближайшего соседа или перетасовка меток для проверки достоверности;
- возмущение входных признаков для проверки устойчивости.

Возьмите за основу работы Towards Robust Interpretability with Self-Explaining Neural Networks (*https://oreil.ly/PtR5u*) и Evaluating the Visualization of What a Deep Neural Network Has Learned (*https://oreil.ly/sQDv5*).

Книги для программистов: https://t.me/free_books_for_it

Достоверность обычно проверяется путем сокрытия или удаления признаков, которые считаются важными, и расчета результирующего изменения выходных данных классификатора для оценки надежности самих пояснительных значений. Альварес-Мелис и Яаккола показали, что существует широкий диапазон точности объяснений для разных методов и наборов данных, при этом аддитивные объяснения Шепли и некоторые другие работают довольно плохо. Для оценки достоверности также можно использовать методы ближайшего соседа, где аналогичные входные наблюдения должны иметь схожие объяснения. В следующем разделе мы также проверим достоверность наших объяснений, но попробуем другой подход.

Чтобы проверить надежность (или стабильность), Альварес-Мелис и Яаккола слегка искажают входное изображение и измеряют результирующее изменение в выводе объяснения. Вооружившись количественными показателями, они сравнивают множество апостериорных методов в нескольких наборах данных. Оказывается, большинство изученных ими апостериорных методов неустойчивы к небольшим изменениям входных данных. Локально интерпретируемые не зависящие от модели объяснения (LIME) работают особенно плохо, а методы интегрированных градиентов и окклюзии показывают лучшую надежность среди изученных ими методов. По всем этим параметрам оценки — разборчивости, достоверности и надежности — объяснимые модели, такие как самообъясняемые нейронные сети, превосходят методы апостериорных объяснений.

Надежность апостериорных объяснений

В этом разделе мы воспроизводим (частично) вопиюще неверные результаты, полученные в статье Sanity Checks for Saliency Maps (*https://oreil.ly/v6qlw*). Авторов статьи интересовал вопрос: «Действительно ли результаты, полученные с помощью методов апостериорного объяснения, говорят нам что-нибудь о модели?» Как мы увидим в нашем эксперименте, иногда результатом является решительное «нет».

Мы начали свой эксперимент с обучения бессмысленной модели, в которой изображения имеют случайные метки. На рис. 7.9 можно видеть кривые высоких потерь при обучении и низкой точности для новой модели, обученной на наборе данных, в котором метки изображений были случайным образом перетасованы. В этом эксперименте мы не проводили никакого дополнения данных для устранения дисбаланса классов. Это объясняет, почему точность на проверочных данных приближается к значению, превышающему 0,5 — модель смещена в сторону класса большинства (пневмония).

Теперь у нас есть модель, обученная на бессмысленных метках. Прогнозы, полученные с помощью нашей новой модели, не могут быть точнее, чем подбрасывание монеты. Чтобы первоначальные объяснения имели смысл, мы надеемся, что *эти* объяснения не улавливают одни и те же сигналы. На рис. 7.10–7.12 показаны пояснения к изображению нашего тестового набора, полученные для модели, которая обучена на случайно перетасованных данных, с помощью методов input*gradient, интегрированных градиентов и окклюзии соответственно.

Книги для программистов: https://t.me/free_books_for_it

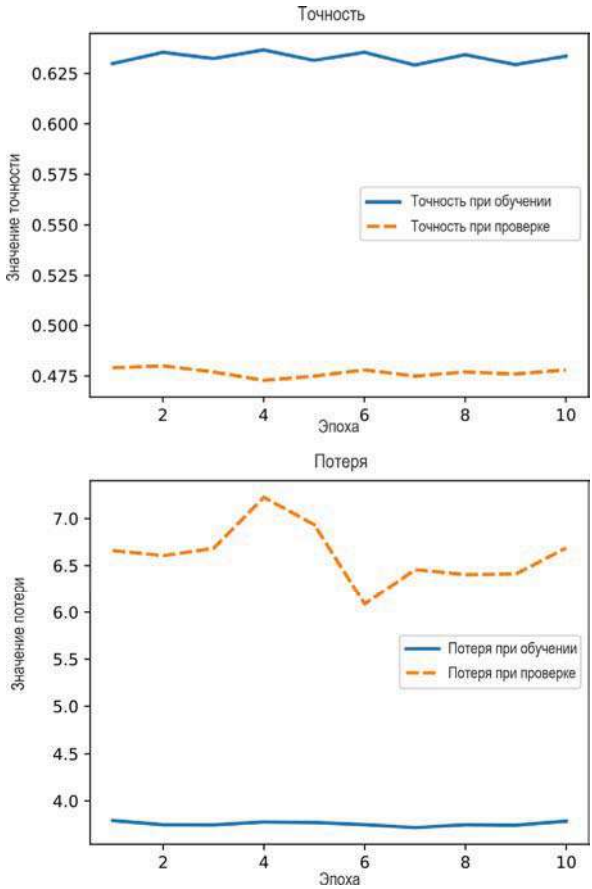


Рис. 7.9. Показатели модели во время обучения на данных, где метки были перетасованы случайным образом

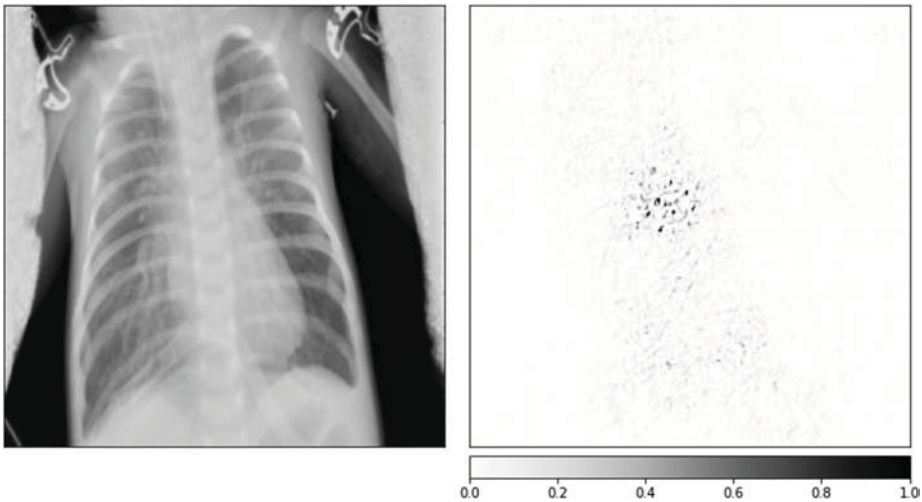


Рис. 7.10. Тепловая карта по методу input*gradient после случайной перетасовки меток классов

Книги для программистов: https://t.me/free_books_for_it

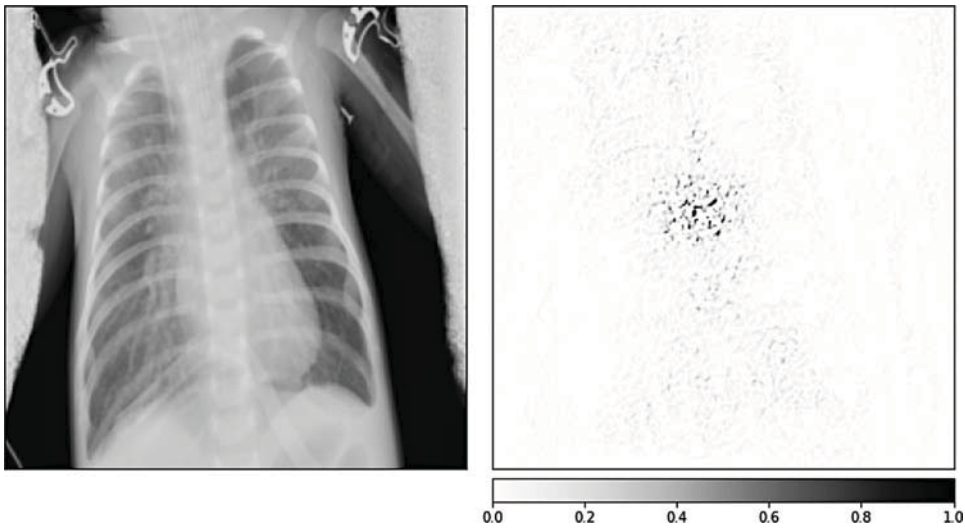


Рис. 7.11. Тепловая карта по методу интегрированных градиентов после случайной перетасовки меток классов

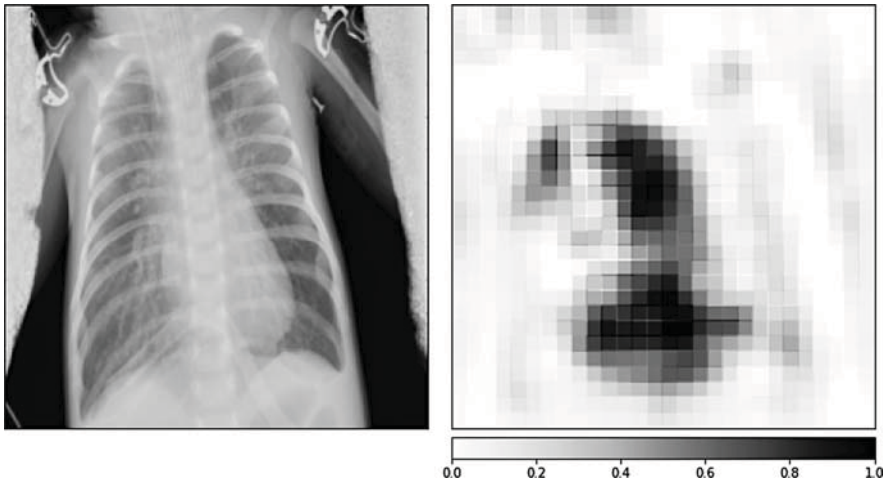


Рис. 7.12. Тепловая карта по методу окклюзии после случайной перетасовки меток классов

На наш взгляд, эти результаты поразительно похожи на результаты предыдущих разделов. На всех изображениях методы еще раз выделили нерелевантные области изображения, такие как позвоночник пациента и границы его туловища. Хуже того, карты атрибуции очень похожи на предыдущие результаты *внутри* легких. Они уловили контуры ребер пациента и области повышенной непрозрачности. Ранее мы интерпретировали это как положительный факт, означающий, что модель может прогнозировать пневмонию на основе конкретных областей воспаления легких. Однако текущий эксперимент показывает, что наши методы дают то же самое объяснение для модели, обученной на бессмысленных данных. Так что же подтверждают наши объяснения?! Мы затрудняемся сказать.

Книги для программистов: https://t.me/free_books_for_it

Чтобы дополнительно проверить надежность наших объяснений, мы провели простой эксперимент по добавлению случайного шума к входным изображениям. Это можно легко сделать с помощью специального преобразования в torchvision. Затем мы изучили объяснения этих входных данных и сравнили их с предыдущими объяснениями. Количество шума ограничивалось таким образом, чтобы прогнозируемый класс изображения оставался неизменным до и после добавления шумовой составляющей.

Про помощи этого эксперимента мы хотели определить устойчивость полученных объяснений к добавлению случайного шума. Если коротко, то получилась смешанная картина: см. рис. 7.13–7.15. Новые карты атрибуции значительно отличаются от

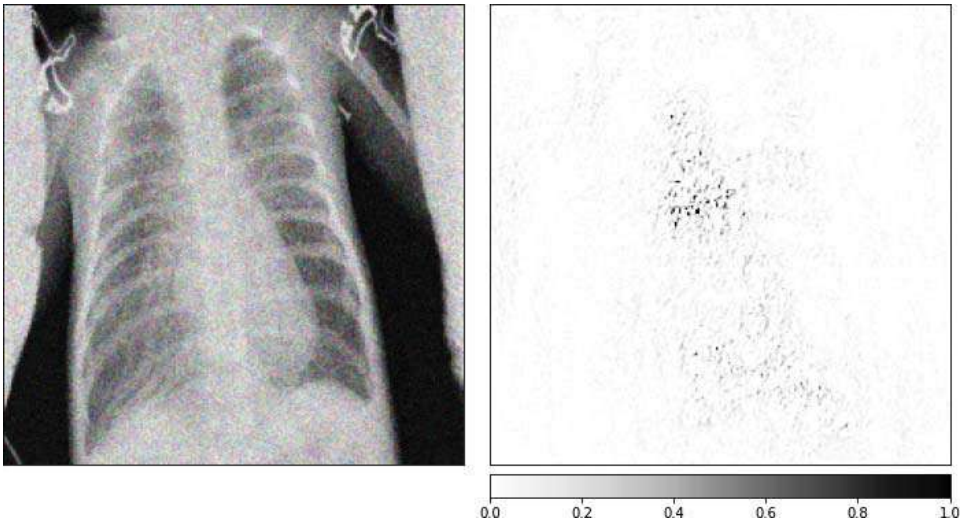


Рис. 7.13. Тепловая карта по методу input*gradient после добавления случайного шума

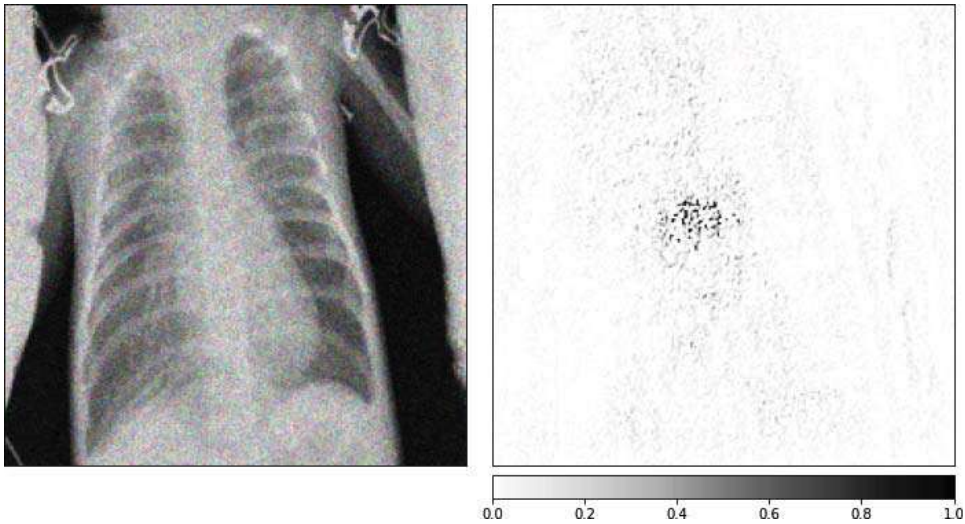


Рис. 7.14. Тепловая карта по методу интегрированных градиентов после добавления случайного шума

Книги для программистов: https://t.me/free_books_for_it

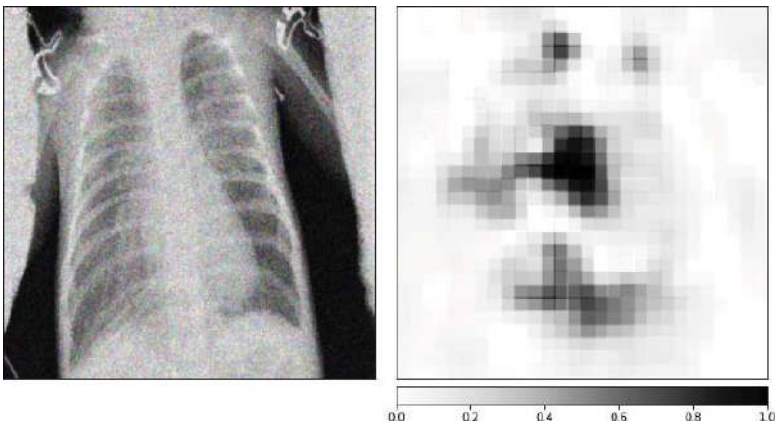


Рис. 7.15. Тепловая карта по методу окклюзии после добавления случайного шума

карт, созданных на основе исходной модели, но, похоже, сохраняют фокус на областях с высокой непрозрачностью внутри легких.

Взгляните, например, на тепловую карту окклюзии на рис. 7.15. Ранее нас обнадеживал тот факт, что окклюзия, по-видимому, выделила области внутри легких с более высоким помутнением. После добавления случайного шума мы все еще наблюдаем этот фокус внимания в левом верхнем и правом верхнем углу легких. Однако добавление шума исказило результаты окклюзии, что придало больший объясняющий вес областям возле шеи. Выходные данные методов на основе градиента также нарушаются, но при этом сохраняется акцент на середине правого легкого.

В статье Sanity Checks for Saliency Maps авторы указывают на возможное объяснение результатов, которые мы видим в этих экспериментах: методы атрибуции эффективно выполняют *обнаружение границ*. То есть, независимо от обучения и архитектуры модели, эти методы атрибуции способны обнаруживать края входного изображения, где градиенты почти всегда демонстрируют резкие изменения. Это могло бы объяснить наблюдаемый нами акцент на очертаниях ребер, а также акцент на областях на границе туловища. Здесь нужно уточнить: *обнаружение границ не является объяснением модели и может быть легко выполнено без привлечения DL*.

Заключение

Краткий итог заключается в том, что апостериорные объяснения часто плохо поддаются толкованию, а иногда и бессмысленны. Хуже того, разнообразие методов объяснения означает, что если вы не будете осторожны, то рискуете попасть в ловушку предвзятости самоподтверждения и в конечном итоге выбрать метод, который подтверждает ваше представление о том, как должна вести себя модель. Мы вам сочувствуем: построение объяснимых моделей в контексте ML — очень сложная задача. Но эта глава показывает, что апостериорные объяснения могут лишь

Книги для программистов: https://t.me/free_books_for_it

создавать опасную *иллюзию* понимания и, следовательно, не всегда подходят для объяснения решений, связанных с высоким риском.

Мы предлагаем не полагаться исключительно на апостериорные методы для объяснения моделей DL в приложениях высокого риска. В лучшем случае эти методы могут быть полезными инструментами отладки моделей. Мы рассмотрим эту тему более подробно в главе 9. В ситуациях, когда вам нужны достоверные, надежные и понятные объяснения, следует стараться использовать *объяснимые модели*. Вы всегда можете опираться на более надежную объяснимость, основанную на модели, с помощью визуализаций апостериорных объяснений, если в этом возникнет необходимость, и у вас всегда остается возможность сверить апостериорные визуализации с механизмами, лежащими в основе модели.

В последнее время появились обнадеживающие возможности в области объяснимых моделей глубокого обучения для классификации изображений и других задач. Модели вывода из прецедентов на основе прототипов, такие как ProtoPNet, и разреженные аддитивные глубокие модели, такие как SENN, открывают путь для развития объяснимого DL. Однако объяснимые модели пока еще редко бывают доступны в готовом виде для приложений DL. Они часто предъявляют более высокие требования к данным и опыту разработчика. Мы призываем читателей рассматривать сложившуюся ситуацию как возможность для роста, а не как тупик. Разработка систем ИИ *должна* требовать высококачественных, тщательно подобранных данных. Модели *должны* быть ориентированы на конкретные проблемы и отражать максимальное знание предметной области.

Мы согласны с авторами статьи The False Hope of Current Approaches to Explainable Artificial Intelligence in Health Care (*https://oreil.ly/-w598*), когда они говорят:

> В отсутствие приемлемых методов объяснения мы считаем, что более прямой путь к достижению целей, часто связанных с объяснимостью, — это тщательная внутренняя и внешняя проверка моделей ИИ.

В следующих двух главах мы будем использовать методы, обсуждаемые здесь и в главе 6, для решения более обширного вопроса *отладки модели*.

Ресурсы

Примеры кода:

- Книга *Machine Learning for High Risk Applications*: **https://oreil.ly/machine-learning-high-risk-apps-code**

Инструменты достижения прозрачности в DL:

- AllenNLP Interpret: **https://oreil.ly/_tAvm**
- Aletheia: **https://oreil.ly/UMfWK**
- Captum: **https://oreil.ly/F5Obo**
- cleverhans: **https://oreil.ly/efN16**
- DeepExplain: **https://oreil.ly/u4Mfu**

Книги для программистов: https://t.me/free_books_for_it

- deeplift: **https://oreil.ly/S29jk**
- deep-visualization-toolbox: **https://oreil.ly/ZH3JU**
- foolbox: **https://oreil.ly/DFSu0**
- L2X: **https://oreil.ly/S2Ppj**
- tensorflow/lattice: **https://oreil.ly/M7aYY**
- lrp_toolbox: **https://oreil.ly/kKk09**
- tensorflow/model-analysis: **https://oreil.ly/5Aeqe**
- ProtoPNet: **https://oreil.ly/ZmqWq**
- tensorflow/tcav: **https://oreil.ly/7RvqS**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 8

Выбор и отладка моделей XGBoost

Способы, с помощью которых ученые, занимающиеся обработкой данных, измеряют показатели реального качества модели, обычно не отличаются адекватностью. Согласно статье Underspecification Presents Challenges for Credibility in Modern Machine Learning (*https://oreil.ly/27jFT*), написанной 40 исследователями из Google и других ведущих исследовательских институтов машинного обучения, «модели машинного обучения часто демонстрируют неожиданно плохое поведение при развертывании в реальных условиях». Фундаментальная проблема заключается в том, что мы по привычке измеряем показатели модели так же, как при написании исследовательских работ, независимо от сложностей и рисков, которыми сопровождается развертывание. Измерения на тестовых данных, такие как точность или площадь под кривой (AUC), мало что говорят нам о справедливости, конфиденциальности, безопасности или стабильности модели. Простые измерения качества прогнозирования или величины ошибки на статических наборах тестов недостаточно информативны для управления рисками. Они не являются гарантами хорошей работы модели при развертывании в реальной рабочей среде. Проще говоря, нас больше должна интересовать производительность in vivo и управление рисками, чем результаты испытаний in silico. Ведь основная цель прикладной практики ML — это принятие правильных решений в реальном мире.

В текущей главе будет представлено несколько методов, выходящих за рамки традиционной оценки и позволяющих выбирать модели с наилучшей способностью к обобщению. Мы начнем с обновления определений, далее опишем расширенный процесс выбора модели, а затем перейдем к упражнениям по отладке модели, которые лучше имитируют сложности реального мира, а также к анализу чувствительности и тестам, выявляющим ошибки модели с помощью анализа остатков. Основная цель отладки моделей — повысить доверие к показателям их качества в реальном мире. Но в процессе мы также повысим прозрачность наших моделей. Примеры кода, сопровождающие эту главу, доступны в Интернете; глава 9 посвящена отладке моделей, работающих с изображениями и неструктурированными данными, а глава 3 рассказывает о более обобщенной отладке моделей.

Уточнение определений: отладка моделей ML

Не будет лишним повторить, что нас гораздо больше беспокоит качество модели in vivo, чем in silico. Показатели работы in vivo имеют значение для пользователей; показатели in silico — нет. Исходя из этого ключевого принципа, мы рассмотрим

Книги для программистов: https://t.me/free_books_for_it

выбор модели, тесты анализа чувствительности, анализ остатков и приемы исправления моделей.

Выбор модели

Традиционно мы выбираем модели, подбирая признаки и гиперпараметры. Мы пытаемся найти лучший набор признаков, используя такие подходы, как пошаговая регрессия, измерение важности признаков или регуляризация L1. Мы часто используем поиск по сетке, чтобы найти лучшие настройки гиперпараметров для наших моделей машинного обучения. В главе 2 мы использовали более комплексный подход: мы начали с бенчмарка линейной модели и добавили в наши модели нелинейность и взаимодействия, а затем использовали наши собственные суждения и выбрали предпочтительную модель. В этой главе вы увидите, что поиск по случайной сетке для выбора гиперпараметров особенно проблематичен на небольших наборах данных, а также в сравнении с более сложной процедурой ранжирования с перекрестной проверкой на основе отчета KDD-Cup 2004: Results and Analysis (*https://oreil.ly/osrK4*). Мы сравним рейтинг качества модели по нескольким критериям проверки и нескольким различным традиционным показателям оценки, чтобы получить лучшее представление об оцениваемых показателях in vivo и выбрать лучшую модель. Мы также расскажем, как оценить бизнес-ценность модели — это очень важный момент. Никто в деловом мире не захочет внедрять убыточную модель!

Анализ чувствительности

Поскольку модели машинного обучения стремятся к сложной экстраполяции, мы не сможем точно предсказать, как наша модель будет работать с незнакомыми данными, если не протестируем ее на различных типах данных. Именно это мы и пытаемся сделать с помощью *анализа чувствительности* (sensitivity analysis). Грубо говоря, анализ чувствительности показывает нам, будет ли наша модель стабильной на различных типах данных. Он может показать, что наша модель имеет проблемы с робастностью, т. е. что она перестает работать при дрейфе данных. Также анализ чувствительности может указать на проблемы со стабильностью модели, т. е. что некоторые типы входных данных заставляют нашу модель вести себя удивительным или ненадлежащим образом. Существует множество структурированных подходов к анализу чувствительности. Если читатели хотят узнать больше, мы предлагаем изучить ресурсы, связанные с PiML (*https://oreil.ly/84KCZ*) или SALib (*https://oreil.ly/Kgnmg*).

В этой главе мы выделим два других связанных метода анализа чувствительности, наиболее полезных для практиков: стресс-тестирование и поиск состязательных примеров. Стресс-тестирование примерно соответствует тестированию на робастность, тогда как метод состязательных примеров ищет проблемы стабильности.

Стресс-тестирование

Стресс-тестирование (нагрузочное тестирование) — это глобальный метод возмущений, который тестирует модели в предсказуемых стрессовых обстоятельст-

Книги для программистов: https://t.me/free_books_for_it

вах. Выполняя стресс-тестирование модели, мы изменяем входные данные, чтобы смоделировать, к примеру, условия рецессии экономики и посмотреть, останется ли она устойчивой в этих прогнозируемых сложных условиях. Идея состоит в том, чтобы сделать нашу модель устойчивой к предсказуемым обстоятельствам или, по крайней мере, задокументировать ожидаемое снижение качества для тех, кто будет поддерживать модель, когда наступит рецессия или другие глобальные изменения рабочей среды[1]. Мы будем проводить гораздо менее строгий анализ единственной модели, но идея та же: проверить, как наша модель будет работать при предсказуемом дрейфе концепта или данных, например во время рецессии, чтобы убедиться, что мы готовы к наиболее вероятным вариантам отказов.

Поиск состязательных примеров

Поиск состязательных примеров (adversarial example search) — это метод локального возмущения, который помогает выявить локальные логические недостатки в надежности модели и потенциальные уязвимости безопасности (т. е. проблемы устойчивости). Состязательные примеры — это строки данных, которые создают неадекватные реакции сложных моделей машинного обучения. Иногда мы можем целенаправленно создать эти строки вручную, но часто приходится их искать. В этой главе мы будем искать их, изменяя значения важных признаков в определенных строках данных и проверяя, как это изменение влияет на работу модели. Полезен как сам поиск, так и отдельные найденные нами состязательные примеры. Поиск создает поверхность отклика, которая отображает качество работы модели на множестве примечательных входных значений и часто выявляет логические недостатки в работе модели. Полезно фиксировать и передавать специалистам по безопасности отдельные строки, которые вызывают у вас вопросы или сомнения. Это позволит использовать мониторинг модели для обнаружения известных вредоносных примеров.

В глубоком обучении специалисты, работающие с данными, предпочитают использовать для создания состязательных примеров градиентную информацию и генеративно-состязательные сети (GAN). В задачах со структурированными данными нам приходится использовать другие методы, например описанные в статье Adversarial Attacks for Tabular Data: Application to Fraud Detection and Imbalanced Data (*https://oreil.ly/KF843*), эвристику, основанную на индивидуальном условном ожидании, или генетические алгоритмы. Мы будем использовать эвристический метод, описанный в разделе «Поиск состязательных примеров» далее в этой главе, как для поиска состязательных примеров, так и для проверки поверхности отклика нашей модели на наличие проблем.

Самый важный метод, который мы не рассматриваем в этой главе, — это случайные атаки или просто передача на вход модели или API большого количества случайных данных и наблюдение за тем, какие проблемы возникают. Если непонятно,

[1] Надо заметить, что крупные банки США каждый год проводят почти исчерпывающие стресс-тесты для своих моделей в соответствии с процессом комплексного анализа и проверки капитала Федеральной резервной системы под названием CCAR.

Книги для программистов: https://t.me/free_books_for_it

с чего начать анализ чувствительности, сначала пробуем случайные атаки. Затем мы пробуем PiML, SALib и методы, которые предложим в следующих разделах. Независимо от того, как будет реализован анализ чувствительности, главное — что-то делать с обнаруженными проблемами. Для устранения проблем с чувствительностью часто применяются такие методы, как дополнение данных, бизнес-правила, регуляризация, ограничения и мониторинг.

Анализ остатков

Анализ остатков — это основной метод отладки модели. Он подразумевает тщательное изучение ошибок моделирования, допущенных в обучающих данных или других помеченных данных, в целях тестирования и отладки. Хотя читатели могут быть знакомы с анализом остатков традиционных линейных моделей, его можно и нужно применять к моделям ML. Основная идея та же, что и в линейных моделях. Хорошие модели должны демонстрировать в основном случайные ошибки. Если при исследовании ошибок нашей модели ML будут обнаружены явные закономерности, это, скорее всего, означает, что мы упустили что-то важное или допустили ошибку при построении модели. Тогда для решения проблемы нам придется прибегнуть к помощи человеческого мозга. В этой главе мы сосредоточимся на трех основных подходах к остаточному анализу: графики остатков, сегментный анализ ошибок и моделирование остатков:

Графики остатков

Мы рассматриваем графики остатков для всей модели, разбиваем их по признакам и уровням. Затем стараемся ответить на такие вопросы:

- Видны ли какие-либо четкие закономерности на графиках остатков?
- Какие строки вызывают наибольшие ошибки?
- Можем ли мы изолировать какие-либо шаблоны для конкретных входных признаков или уровней признаков?

Затем мы стараемся продумать, как исправить обнаруженные проблемы.

Сегментный анализ ошибок

Нам не следует развертывать модель, не проверив, как она работает в основных сегментах наших обучающих или тестовых данных. Невыполнение этого требования может иметь серьезные последствия как для общего качества модели, так и для алгоритмической дискриминации, когда модель плохо работает на исторически маргинализированных демографических группах. В этой главе мы сосредоточимся на аспекте качества и рассмотрим другие типы сегментов, помимо демографических. Мы делаем это, потому что обычно используемые средние показатели оценки на полных наборах данных могут маскировать плохую работу модели в небольших, но важных сегментах населения. Разреженность обучающих данных также может привести к почти случайным прогнозам в некоторых сегментах. Сегментный анализ ошибок также был предложен как метод in silico для решения неприятной проблемы недостаточной специфичности. Все это — низкое качество прогнозов в небольших сегментах, случайные прогнозы

Книги для программистов: https://t.me/free_books_for_it

в разреженных областях обучающих данных и недостаточная специфичность — может привести к неприятным сюрпризам и серьезным последствиям после развертывания модели.

Одним из интересных дополнений к сегментному анализу ошибок, реализованных в PiML, является выявление переобучения по отдельным сегментам. Это может выявить дополнительные проблемы, которые повлияют на модель in vivo.

Моделирование остатков

Еще один способ изучить закономерности в остатках — это смоделировать их. Если мы можем подогнать простую, интерпретируемую модель к остаткам другой модели, это означает, почти по определению, что в остатках существуют явные закономерности. (А явные закономерности в остатках обычно означают, что мы допустили ошибку моделирования.) Более того, моделирование остатков подразумевает, что мы можем их уменьшить. Модели, которые мы обучаем на остатках, также должны указывать нам на то, как исправить обнаруженные ошибки. В качестве примера мы адаптируем дерево решений к остаткам нашей модели. Затем рассматриваем правила этого дерева, поскольку именно они описывают случаи, когда наша модель чаще всего ошибается. Мы должны постараться понять эти правила и попытаться исправить проблемы, которые они выявляют.

Как и в случае с анализом чувствительности, мы не сможем рассмотреть все важные способы проведения анализа остатков в ML в одной главе. Некоторые из наиболее примечательных подходов, которые мы не будем рассматривать, включают методы обнаружения неробастных признаков, такие как вклад значений Шепли в потери модели. В главе 3 был представлен более широкий обзор методов анализа остатков для ML.

Исправление

Итак, проблемы обнаружены, и нужно их исправить. Плохая новость заключается в том, что множество проблем в ML возникает из-за использования некачественных, предвзятых данных и предвзятости самоподтверждения. Для большинства проектов доступны только два болезненных способа решения проблем:

- сбор более качественных данных, желательно с применением методик планирования эксперимента;
- возвращение к чертежной доске и приведение эксперимента в соответствие с научной методикой для минимизации человеческих, статистических и системных предвзятостей в рабочем процессе моделирования.

Как только в наших рабочих процессах будут устранены серьезные проблемы с данными, методологией или предвзятостью, мы сможем попробовать внести в модель технические исправления, как мы и делаем в этой главе. Например, можно применить к модели ограничения монотонности, ограничения взаимодействия или регуляризацию, чтобы стабилизировать ее, сделать более логичной и интерпретируемой, а также улучшить качество прогнозов in vivo. Для устранения предсказуе-

Книги для программистов: https://t.me/free_books_for_it

мых плохих результатов можно задать бизнес-правила, иногда также называемые *утверждениями модели*, или ограничениями ручного прогнозирования. Бизнес-правила и утверждения модели сводятся к добавлению в нашу систему оценки специального кода, принудительно меняющего прогнозы, которые, по нашему мнению, будут неверными. У нас есть возможность редактировать формулы или производственный код наших моделей, чтобы исправить проблемные механизмы моделирования или прогнозы, а также управлять нашими моделями и отслеживать их после их развертывания, чтобы быстро выявлять аномалии.

Мы надеемся, что для нашей модели в этой главе мы проделали достойную работу, придерживаясь научного метода, главным образом выражая наши гипотезы о результатах модели через ограничения монотонности и очень строгий выбор модели, как обсуждается в следующем разделе. Мы применим к модели анализ чувствительности и анализ остатков, чтобы найти ошибки, а затем приложим все усилия, чтобы исправить эти ошибки в разделе «Исправление выбранной модели» далее в текущей главе.

Выбор лучшей модели XGBoost

Хотя технически это не отладка, мы хотим начать наше упражнение по отладке с формирования прочной основы, выбрав стабильную, обобщаемую и авторитетную модель. Но мы не будем ограничиваться поиском по сетке, а применим для выбора модели методику Каруаны и др., описанную в итогах Кубка по поиску знаний в базах данных (KDD) 2004 г. (*https://oreil.ly/kJT7d*). Мы также сравним полученные результаты со стандартным поиском по случайной сетке, чтобы увидеть разницу между поиском по сетке и процедурой ранжирования перекрестной проверки, описанной в этом разделе. Затем, прежде чем перейти к анализу чувствительности, мы проведем базовую оценку бизнес-ценности нашей модели, чтобы убедиться, что мы не тратим деньги зря.



По словам Ричарда Фейнмана, мы, как учёные, обязаны буквально расшибиться в лепешку, чтобы не обмануть себя и других. Если такой подход к выбору модели кажется вам слишком строгим, просто стремитесь найти самую лучшую модель из всех возможных.

Первое, что мы делаем, чтобы начать процесс выбора модели, — это разделяем проверочные данные на пять частей. Затем мы выбираем пять соответствующих показателей качества, которые будут применяться к каждому сегменту. Эти показатели должны отражать различные аспекты качества работы модели, например измерение AUC при ранжировании по пороговым значениям и показатель правильности при одном пороге. В нашем случае в качестве пяти показателей мы возьмем максимальную точность, AUC, максимальную статистику F1, логарифмические потери и среднеквадратичную ошибку (MSE). Первым шагом нашего процесса отбора модели является вычисление значения каждой из этих различных статистических метрик для каждого сегмента. Это делает следующий фрагмент кода:

```
eval_frame = pd.DataFrame() # инициализируем фрейм для хранения оценок ранжирования
metric_list = ['acc', 'auc', 'f1', 'logloss', 'mse']
```

Книги для программистов: https://t.me/free_books_for_it

```
# создаем фрейм eval построчно
for fold in sorted(scores_frame['fold'].unique()): # цикл по подвыборкам
    for metric_name in metric_list: # цикл по показателям

        # инициализация row_dict для хранения значений каждой строки
        row_dict = {'fold': fold,
                    'metric': metric_name}
        # кешируем значения y для сегмента
        fold_y = scores_frame.loc[scores_frame['fold'] == fold, target]

        # первые столбцы не для оценивания
        for col_name in scores_frame.columns[2:]:

            # кеш оценок сегмента
            fold_scores = scores_frame.loc[
                scores_frame['fold'] == fold, col_name]

            # вычисление оцениваемой метрики для сегмента с разумной точностью
            if metric_name == 'acc':
                row_dict[col_name] = np.round(
                    max_acc(fold_y, fold_scores), ROUND)

            if metric_name == 'auc':
                row_dict[col_name] = np.round(
                    roc_auc_score(fold_y, fold_scores), ROUND)

            if metric_name == 'f1':
                row_dict[col_name] = np.round(
                    max_f1(fold_y, fold_scores), ROUND)

            if metric_name == 'logloss':
                row_dict[col_name] = np.round(
                    log_loss(fold_y, fold_scores), ROUND)

            if metric_name == 'mse':
                row_dict[col_name] = np.round(
                     mean_squared_error(fold_y, fold_scores), ROUND)

            # добавляем значения строк в eval_frame
            eval_frame = eval_frame.append(row_dict, ignore_index=True)
```

Получив показатели качества для каждой модели в каждом сегменте, мы переходим ко второму этапу процедуры выбора и оцениваем производительность каждой модели в каждом сегменте и по каждому показателю. Следующий код выполняет ранжирование. Наш поиск включает 50 различных моделей XGBoost, и мы тестируем их, используя пять показателей в пяти сегментах. Для каждого сегмента и каждого показателя мы ранжируем модели с первого по пятидесятое место по теку-

Книги для программистов: https://t.me/free_books_for_it

щему показателю, допуская «ничьи». Модель с наименьшим средним рейтингом по каждому сегменту и показателю мы принимаем как лучшую модель для использования in vivo. Представьте, что вы проводите тестирование 50 студентов, но вместо одного теста на каждого студента приходится по пять тестов. Кроме того, мы не пропускаем каждого студента, получившего оценку выше определенного порога; нас интересует только тот студент, который показывает лучшие результаты по большинству тестов. Конечно, это сделало бы нас очень плохими педагогами, но, к счастью, можно быть предельно избирательными, когда дело касается моделей ML.

```
# инициализируем временный фрейм для
rank_names = [name + '_rank' for name in eval_frame.columns
              if name not in ['fold', 'metric']]
rank_frame = pd.DataFrame(columns=rank_names)

# реорганизация столбцов
eval_frame = eval_frame[['fold', 'metric'] +
                        [name for name in sorted(eval_frame.columns)
                        if name not in ['fold', 'metric']]]

# определение рейтинга по строкам
for i in range(0, eval_frame.shape[0]):

        # получение рейтинга строки на основе метрики
        metric_name = eval_frame.loc[i, 'metric']
        if metric_name in ['logloss', 'mse']:
            ranks = eval_frame.iloc[i, 2:].rank().values
        else:
            ranks = eval_frame.iloc[i, 2:].rank(ascending=False).values

        # создаем однострочный фрейм и добавляем к rank_frame
        row_frame = pd.DataFrame(ranks.reshape(1, ranks.shape[0]), columns=rank_names)
        rank_frame = rank_frame.append(row_frame, ignore_index=True)

        # наводим порядок, удаляем лишнее
        del row_frame

eval_frame = pd.concat([eval_frame, rank_frame], axis=1)
```

Ничьи возникают потому, что мы округляем значения показателей. Если две модели имеют, скажем, AUC 0,88811 и 0,88839, то это ничья. Эти последние десятичные знаки AUC, вероятно, не имеют значения для производительности in vivo, так что наш подход хорошо справляется с ничьими. Две модели с одинаковым рейтингом просто имеют одинаковый ранг для этого сегмента и показателя. Поскольку мы пробуем много показателей и много оценок и берем средний рейтинг по всем из них, эти ничьи в конечном итоге редко мешают выбору лучшей модели. В нашем примере каждой из 50 моделей присвоено 25 различных значений ранжирования, по одному рангу для каждого показателя и сегмента. Наша лучшая модель заняла

Книги для программистов: https://t.me/free_books_for_it

первое и второе места по нескольким комбинациям сегментов и показателей. Но при этом она получила рейтинг 26,5 за ее наихудший сегмент и показатель, что указывает на ничью,. В итоге наша самая лучшая модель с точки зрения самых низких баллов по показателям и сегментам показала средний рейтинг 10,38.

В целях сравнения эта процедура ранжирования была применена к 50 лучшим выбранным моделям, и модели были ранжированы с помощью стандартного поиска по случайной сетке. При поиске по сетке использовалась наименьшая логарифмическая потеря на проверочных данных. Когда мы сравниваем рейтинг поиска по сетке с рейтингом перекрестной проверки, в глаза бросается существенное различие результатов (табл. 8.1).

Таблица 8.1. Общий рейтинг 10 лучших моделей при случайном поиске по сетке, ранжированный по логарифму и при более глубоком подходе к выбору модели с перекрестной проверкой

Поиск по сетке	Перекрестная проверка
Модель 0	Модель 2
Модель 1	Модель 5
Модель 2	Модель 1
Модель 3	Модель 4
Модель 4	Модель 12
Модель 5	Модель 0
Модель 6	Модель 21
Модель 7	Модель 48
Модель 8	Модель 30
Модель 9	Модель 29
Модель 10	Модель 17

Рейтинг для поиска по сетке находится слева, а рейтинг для метода перекрестной проверки — справа. Первая строка в таблице указывает, что третья лучшая модель в поиске по сетке (отсчет индексов от 0, Модель 2) была лучшей моделью согласно рейтингу перекрестной проверки. Два процесса выбора модели демонстрируют корреляцию Пирсона 0,35, что указывает на всего лишь умеренную положительную корреляцию. Короче говоря, лучшая модель, полученная в результате поиска по сетке, может не совпадать с лучшей моделью, выбранной с помощью более глубоких подходов отбора. В одном интересном неформальном исследовании подобная техника использовалась для выявления проблем стабильности в соревнованиях по науке о данных, в которых применялись небольшие наборы данных. Такой подход к выбору — хороший способ начать «расшибаться в лепешку», чтобы повысить научную достоверность упражнений по выбору модели ML. Чтобы узнать, как именно мы это сделали, изучите примеры кода в этой главе.

Еще одним важным фактором при выборе модели является бизнес-ценность. Создание моделей стоит денег. Часто больших денег. Наши зарплаты, медицинские

Книги для программистов: https://t.me/free_books_for_it

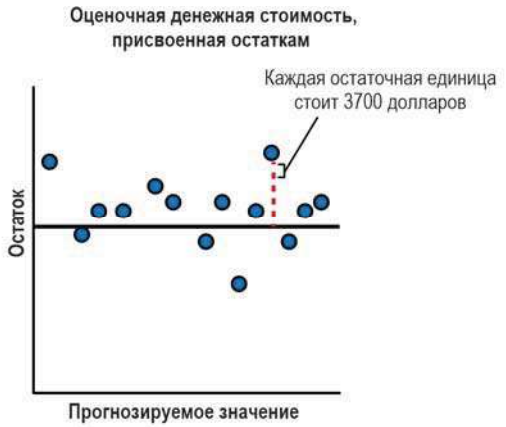
страховки, пенсионные отчисления, кофе с печеньем, компьютеры, офисные помещения и кондиционирование воздуха недешевы. Как правило, модель считается успешной, если она способна как минимум окупить ресурсы, потраченные на обучение, тестирование и развертывание системы. Хотя истинное понимание ценности нашей модели требует мониторинга и измерения бизнес-ценности в режиме реального времени, существуют приемы для определения ее ценности перед развертыванием.

Для начала мы присваиваем предполагаемую денежную стоимость результатам нашей модели. Для классификаторов это означает присвоение денежной стоимости элементам матрицы несоответствий. На рис. 8.1 слева показана матрица несоответствий, а справа — график остатков.

Матрица несоответствий с оценками денежной стоимости для каждой ячейки

	Фактическая просрочка	Фактический платеж
Прогнозируемая просрочка	Истинно положительный $0	Ложно-положительный –$23,000
Прогнозируемый платеж	Ложно-отрицательный –$85,000	Истинно отрицательный $23,000

а)



б)

Рис. 8.1. Пример процедур оценки с расчетной денежной стоимостью для классификаторов (*а*) и регрессионных моделей (*б*)

Матрицу несоответствий слева можно применить к выбранной нами модели. Мы получили значения в ячейках, проанализировав результаты модели in vivo. Истинно положительный результат для нашей модели означает, что мы решили не предоставлять кредит тому, кто действительно мог бы просрочить платеж. С этим результатом не связаны никакие альтернативные издержки или списания, но нет также и прогноза положительного дохода. Ложноположительный результат приводит к тому, что мы отказываемся предоставить кредит заемщику, который вносил бы платежи, влекущие за собой альтернативные издержки. Мы связываем эти альтернативные издержки с предполагаемой пожизненной ценностью клиента (lifetime value, LTV) в –23 000 долларов США. Ложноотрицательный результат — самый худший. Это означает, что мы предоставили кредит тому, кто его не вернет. Нам придется списать долг, и мы оцениваем его стоимость исходя из среднего значения кредитных лимитов клиентов (`LIMIT_BAL`) примерно в –85 000 долларов — беда! Истинно отрицательные прогнозы — это то, на чем мы зарабатываем деньги. Здесь модель предлагает выдать кредит клиенту, который действительно вернет его с процентами. Мы сопоставляем этот результат с противоположностью ложнопо-

Книги для программистов: https://t.me/free_books_for_it

ложительного прогноза и возмещаем LTV клиента за кредитный продукт. За каждый истинно отрицательный результат мы получаем доход в размере 23 000 долларов США. Теперь нам нужно сложить эти значения для каждого клиента в нашем проверочном наборе, поскольку каждый клиент будет представлять собой истинно положительный, ложноположительный, ложноотрицательный или истинно отрицательный результат. Для нашей модели при расчете на небольшом портфеле, представленном проверочным набором, эта оценочная стоимость составляет 4 240 000 долларов США. Итак, в нашей модели есть реальная ценность для бизнеса, но она не слишком велика с учетом всех расходов и налогов, связанных с этим доходом.

Для регрессионных моделей мы можем присвоить одну стоимость завышенному и заниженному прогнозированию. Или, как показано на рис. 8.1, мы можем попытаться присвоить денежную стоимость каждой единице остатка как для завышенного, так и для заниженного прогноза. Затем мы вычисляем остатки для каждой строки в нашем наборе данных и суммируем оценочную стоимость модели. Присвоив модели обоснованную денежную стоимость, мы можем ответить на ключевой бизнес-вопрос: «Приносит ли эта модель какую-либо реальную выгоду?» Допустим, мы думаем, что выбрали достойную модель, имеющую определенную ценность для бизнеса. Теперь попробуем выяснить, что в ней не так, используя анализ чувствительности.

Анализ чувствительности XGBoost

В этом разделе мы начнем проверку нашей модели с анализа чувствительности. Ранее было сказано, что для подробной демонстрации анализа чувствительности мы выбрали стресс-тестирование и поиск состязательных примеров. Оба метода нашли прямое применение в широком спектре приложений и помогают выявить различные виды проблем. Стресс-тестирование ищет глобальные слабые места модели во всех наборах данных в предсказуемых стрессовых обстоятельствах, таких как рецессии. Поиск состязательных примеров помогает нам обнаружить неожиданные потенциальные проблемы, такие как неожиданные прогнозы или уязвимости безопасности, часто на локальном уровне, вплоть до отдельных строк данных.

Практический анализ чувствительности в приложениях помимо обучения с учителем

Мы можем предложить несколько идей по применению анализа чувствительности для различных типов моделей.

Кластеризация — оценка перемещений центроидов кластера, вызванных дрейфом данных или состязательными примерами.

Анализ главных компонент — оценка сходства собственных значений в новых и обучающих данных.

Компьютерное зрение — оценка качества модели на смещенных популяциях, смещенных субпопуляциях и состязательных примерах (см. главу 9).

Книги для программистов: https://t.me/free_books_for_it

Языковые модели — оценка качества модели на разных языках, оценка реакции модели на атаки (например, перестановкой букв) и состязательные промпты (раздел «А как насчет языковых моделей?» в главе 9).

Случайные атаки или подвержение моделей воздействию больших объемов случайных данных, а затем отслеживание возникающих ошибок — отличный подход для всех типов моделей.

Стресс-тестирование модели XGBoost

Линейные модели выполняют — что логично — линейную экстраполяцию, но модели ML могут делать с данными, находящимися за пределами изученной области, практически все что угодно. Если мы не проведем стресс-тестирование наших моделей на нужных данных, мы просто не узнаем об этом. Возьмем, к примеру, обучение модели на наборе данных, где самый высокий доход человека составляет 200 000 долларов. Как поведет себя модель, если она столкнется с индивидуальным доходом, скажем, в 20 млн долларов? Она сломается? А может, она вернет точный прогноз? Мы не можем знать этого наперед, пока не проверим. Принцип проверки прост. Смоделируйте ряд данных, поместите туда доход в 20 млн долл. и пропустите его через свою модель, чтобы увидеть, как она себя ведет. Когда такую проверку выполняют более тщательно и систематически, это и называется стресс-тестированием.



Сложные модели ML часто ведут себя плохо при экстраполяции за пределы области обучающих данных, но даже у более простых моделей возникают проблемы с экстраполяцией. Древовидные модели часто не могут делать прогнозы за пределами диапазонов обучающих данных, а полиномиальные модели могут страдать от феномена Рунге на границах своих областей обучающих данных. Мы берем на себя риск всякий раз, когда используем стандартные статистические модели или модели ML для прогнозов за пределами области обучающих данных.

Стресс-тестирование — это упражнение in silico, проводимое для проверки устойчивости моделей к внешним неблагоприятным сценариям in vivo, таким как рецессии или пандемии. Основная идея стресс-тестирования состоит в том, чтобы смоделировать данные, представляющие реалистичные сценарии будущего, а затем повторить традиционную оценку модели, чтобы увидеть, как модель работает. Это гарантирует, что модели ML смогут противостоять вполне вероятным неблагоприятным событиям, с которыми они рискуют столкнуться в реальной жизни, и что они устойчивы к неизбежным изменениям новых данных in vivo, широко известным как дрейф данных и концепта.

Специалисты, работающие с данными, часто говорят, что они уже проверяют свои модели на контрольных наборах данных, так есть ли необходимость в дополнительных стресс-тестах? Что ж, необходимость в этом есть, особенно когда модели развернуты в производстве и начинают влиять на людей. Модель ML с идеальной AUC in silico бесполезна, если она дает сбои при обнаружении распространенных стрессоров в новых данных. Когда мы развертываем модели машинного обучения

Книги для программистов: https://t.me/free_books_for_it

в реальном мире, нам приходится думать о большем количестве аспектов и ситуаций, чем простая ошибка тестирования in silico. Несмотря на то что будущее предсказать трудно, мы можем использовать данные проверки для моделирования предсказуемых проблем. Затем смотрим, как модель работает в этих условиях, документируем любые проблемы и, если возможно, обновляем нашу модель для устранения любых обнаруженных проблем.

Золотым стандартом стресс-тестирования является Комплексный анализ и обзор капитала Федеральной резервной системы (CCAR). Это ежегодное мероприятие, проводимое Федеральной резервной системой США, направлено на то, чтобы удостовериться, что крупные банки и финансовые учреждения имеют эффективные процессы планирования капитала и располагают достаточным капиталом для противостояния экономическим потрясениям. Например, CCAR провел два отдельных теста, чтобы оценить устойчивость крупных банков США после пандемии COVID-19. Несмотря на то что банки были хорошо капитализированы в экстремальных смоделированных ситуациях, результаты CCAR по-прежнему оправдывают ограничения на банковские выплаты из-за неопределенности, окружающей ситуации. В следующих подразделах мы будем черпать вдохновение из CCAR, пытаясь определить, устойчива ли выбранная нами модель XGBoost к условиям рецессии — распространенному и предсказуемому стрессовому фактору для кредитных моделей.

Методика стресс-тестирования

Рецессия — это ситуация, при которой в экономике страны наблюдается существенный спад, продолжающийся несколько месяцев. Помните финансовый кризис 2008 г. и, совсем недавно, экономический спад, вызванный пандемией COVID? Мы хотим посмотреть, как наша модель будет работать, если во время ее применения произойдет рецессия. В этом разделе мы смоделируем сценарий рецессии, а затем оценим эффективность нашей ограниченной и регуляризованной модели XGBoost. Как видно на рис. 8.2, перед стресс-тестированием модель хорошо работает как с проверочными, так и с контрольными данными.

Теперь мы создадим копию исходного набора данных и переименуем ее в `data_recession_modified`. Мы изменим значения некоторых признаков в этом наборе данных, используя базовые экономические и деловые аспекты, и сможем имитировать сценарий рецессии.



Трудно смоделировать реалистичные данные, находящиеся вне распределения, потому что невозможно заранее знать, как каждый признак будет изменяться в некоторых новых обстоятельствах. По этой причине стресс-тестирование лучше всего проводить в тесном сотрудничестве с профильными экспертами. Даже лучше, чем моделирование данных стресс-тестов, было бы провести бэк-тестирование модели на реальных данных в неблагоприятных условиях прошлых лет, таких как глобальная рецессия 2008 г.

Книги для программистов: https://t.me/free_books_for_it

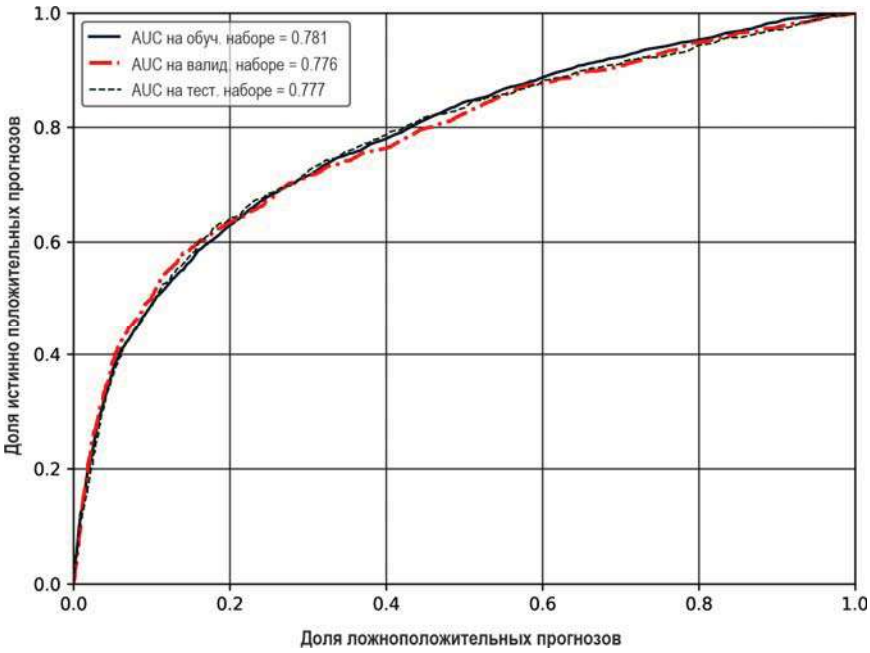


Рис. 8.2. Кривая ROC перед стресс-тестом ограниченной и регуляризованной модели XGBoos

Изменение данных для моделирования условий рецессии

Сначала мы выберем некоторые наблюдения в наборе данных для изменения. Мы считаем, что именно на эти наблюдения повлияет рецессия — возможно, эти люди или кто-то из членов их семьи потеряют работу. Мы решили изменить 25% клиентов, которые ранее имели хорошую репутацию:

```
data_recession_modified = data_recession[
    data_recession['DELINQ_NEXT'] == 0].sample(frac=.25)
```

Предполагая, что смоделированная рецессия наступила недавно, мы допустим, что эти наблюдения содержат просрочки последних платежей:

```
payments = ['PAY_0', 'PAY_2']
data_recession_modified[payments] += 1
```

Здесь `PAY_*` представляет различные статусы погашения кредита. Далее уменьшим сумму выплат клиентов на тысячу долларов каждому:

```
pay_amounts = ['PAY_AMT1', 'PAY_AMT2']
data_recession_modified[pay_amounts] = np.where(
    data_recession_modified[pay_amounts] < 1000,
    0,
    data_recession_modified[pay_amounts]-1000)
```

Во времена финансового кризиса банки часто ужесточают свои финансовые ограничения, и один из способов сделать это — снизить кредитные лимиты. Мы вклю-

Книги для программистов: https://t.me/free_books_for_it

чим этот сценарий в наше стресс-тестирование, уменьшив кредитные лимиты измененных клиентов в фиксированной пропорции от их первоначальных кредитных лимитов:

```
data_recession_modified['LIMIT_BAL'] *= 0.75
```

Мы также уменьшим суммы счетов этих клиентов в фиксированной пропорции, чтобы имитировать снижение расходов:

```
bill_amounts = ['BILL_AMT1','BILL_AMT2']
data_recession_modified[bill_amounts] *= 0.75
```

Наконец, мы предположим, что у некоторой части этих измененных клиентов останется просрочка по счетам. В частности, мы изменим половину целевых переменных с нуля на единицу:

```
data_recession_modified['DELINQ_NEXT'] = np.where(
    np.random.rand(len(data_recession_modified)) < 0.5,
    1, 0)
```

После возвращения затронутых наблюдений в остальные данные у нас получился набор данных, имитирующий неблагоприятные условия, с которыми наша модель может столкнуться в реальном мире. Пришло время взглянуть на показатели модели на этих смоделированных данных. На рис. 8.3 мы видим умеренное снижение качества модели после применения данных, имитирующих рецессию, и дрейф концепта на тестовых данных.

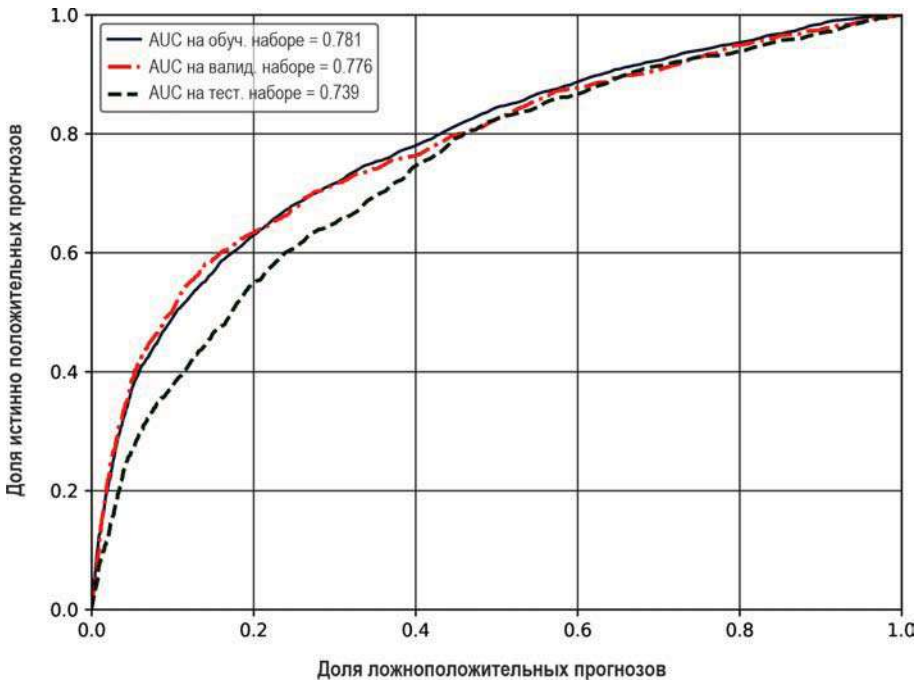


Рис. 8.3. Кривая ROC после стресс-теста ограниченной и регуляризованной модели XGBoost

Книги для программистов: https://t.me/free_books_for_it

Получив такие результаты, первым делом надо их задокументировать и поделиться ими со своей командой и руководством. Это позволяет принять обоснованное решение о том, следует ли развертывать модель. Если экономические условия выглядят радужно, мы могли бы разумно решить применить нашу модель, зная, что ее необходимо будет быстро обновить в случае изменения экономических условий. Более детальный анализ повлечет за собой переоценку финансового риска, который мы возьмем на себя, если наша AUC упадет с 0,777 до 0,738. Можем ли мы позволить себе принимать столько дополнительных неправильных кредитных решений?

После документирования и обсуждения результатов следующим шагом может стать попытка улучшить модель. Это определенно имеет смысл сделать, если экономические условия будут выглядеть обескураживающими или если результаты того или иного стресс-теста окажутся более плачевными. Как читатели могли догадаться по количеству страниц, оставшихся в текущей главе, вскоре мы обнаружим и другие проблемы с нашей моделью. Устранение или смягчение негативных последствий всех обнаруженных проблем мы оставим до конца главы.

Прежде чем перейти к поиску состязательных примеров, мы хотели бы подчеркнуть еще одну вещь. Мы были осторожны при обучении своей модели, используя регуляризацию, монотонные ограничения, поиск по сетке и высоконадежный подход к выбору модели. Эти решения, вероятно, оказали положительное влияние на устойчивость модели в стресс-тесте. Модели, обученные без подобных приемов, возможно, показали бы худшие результаты во время стресс-теста. В любом случае, если мы не проверяем наличие проблем, влияющих на развертывание in vivo, значит, мы просто игнорируем их.

Поиск состязательных примеров

В качестве следующего метода отладки мы применим поиск состязательных примеров. Цель нашего поиска двоякая: найти состязательные примеры, которые можно использовать для обмана модели после ее развертывания, и посмотреть, что мы можем узнать о нашей модели — хорошее и плохое — в результате нашего поиска.

Существует множество пакетов и программ, которые помогают находить состязательные примеры для данных в виде изображений. Однако нам требуются примеры-противники для структурированных данных. Несмотря на некоторый прогресс, достигнутый в использовании генеративно-состязательных сетей (GAN) и генетических алгоритмов для поиска противников структурированных данных, мы применим эвристический подход. Первый шаг — найти строку данных, которая позволит сделать хорошее первоначальное предположение для состязательного примера. Мы сделаем это с помощью графиков ICE. На рис. 8.4 показаны кривые ICE для децилей прогнозируемой вероятности, а также частичная зависимость.

На этом рисунке мы видим, что кривая ICE, связанная с 80-м процентилем, показывает наибольшее колебание прогнозируемых значений среди значений `PAY_0`. Поскольку мы знаем, что эта строка данных может привести к большим изменениям в прогнозах просто путем изменения значения одного признака, мы будем использовать исходную строку данных на 80-м процентиле прогнозируемой вероятности в выбранной модели, чтобы заложить основу для поиска состязательных примеров.

Книги для программистов: https://t.me/free_books_for_it

Если подробнее, для каждой важной переменной наша эвристика состязательного поиска выглядит следующим образом:

1. Рассчитайте кривые ICE для каждого дециля прогнозов модели.
2. Найдите кривую ICE с наибольшим колебанием прогнозов.
3. Изолируйте строку данных, связанную с этой кривой ICE.
4. Для этой строки данных:
   - Измените от 1 до 3 дополнительных важных переменных в строке. (Трудно построить график результатов для более чем 1–3 переменных.)
   - Восстановите поврежденную строку.
   - Продолжайте до тех пор, пока каждая дополнительная важная переменная не пройдет через свою область в обучающих данных и не обнаружит пропущенные или другие заслуживающие внимания значения, выходящие за пределы диапазона.
5. Постройте график и проанализируйте результаты.

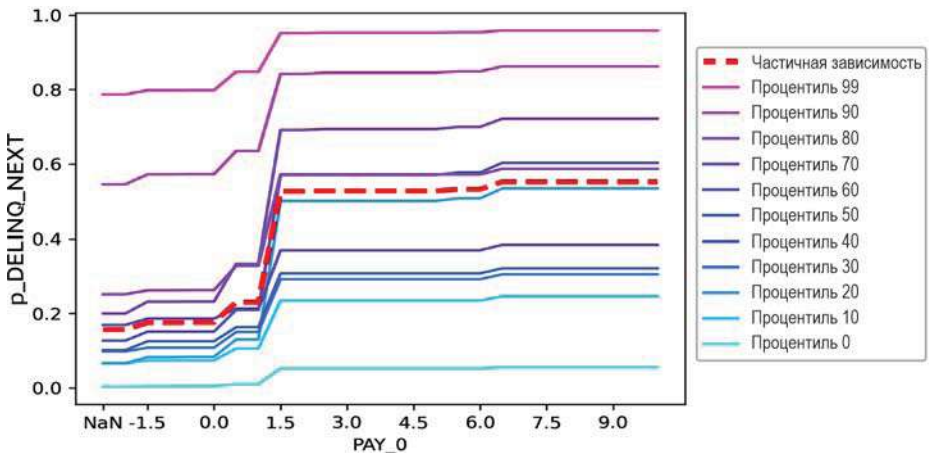


Рис. 8.4. Частичная зависимость и ICE выбранной модели XGBoost

Мы уже рассмотрели шаги 1–3, но как нам выполнить шаг 4? Мы воспользуемся преимуществами `itertools.product()` для автоматической генерации всех возможных отклонений признаков в наборе признаков, передаваемых в функцию Python. Также помните, что при работе с собственным API XGBoost нам всегда необходимо указывать дополнительный аргумент (`iteration_range`) в функцию `predict()` для применения выбора модели:

```
adversary_frame = pd.DataFrame(columns=xs + [yhat])

feature_values = product(*bins_dict.values())
for i, values in enumerate(feature_values):
    row[xs] = values
    adversary_frame = adversary_frame.append(row, ignore_index=True, sort=False)
```

Книги для программистов: https://t.me/free_books_for_it

```
    if i % 1000 == 0:
        print("Построено %i/%i строк ..." % (i, (resolution)**(len(xs))))
adversary_frame[search_cols] = adversary_frame[search_cols].astype(
    float, errors="raise")
adversary_frame[yhat] = model.predict(
    xgb.DMatrix(adversary_frame[model.feature_names]),
    iteration_range=(0, model.best_iteration))
```

Мы передали в код поиска данные проверки и входные признаки PAY_0, PAY_2, PAY_AMT1 и PAY_AMT2. Выбор входных признаков был основан на сводном графике Шепли, который показал, что эти признаки имеют самый широкий разброс вкладов в прогнозирование. В результате запуска кода мы получаем данные для нескольких поверхностей отклика, которые показывают, как наша модель ведет себя в потенциально интересных обстоятельствах. Единственное, что осталось сделать сейчас, — построить и проанализировать эти функции отклика. На рис. 8.5 показаны результаты поиска состязательных примеров, инициированного кривой ICE, и представлены некоторые положительные и отрицательные результаты.

Положительным моментом является то, что каждая поверхность отклика демонстрирует монотонность. Эти симуляции подтверждают, что ограничения монотонности, заданные во время обучения и основанные на знаниях предметной области, сохраняются во время обучения. С другой стороны, была также обнаружена потенциальная логическая ошибка. Согласно одной из поверхностей отклика модель выдаст высокую вероятность дефолта, если клиенты просрочат свой последний платеж (PAY_0) на два месяца. Проблема, о которой следует знать, заключается в том, что модель рекомендует отказать в выдаче следующего кредита даже в том случае, если клиент вносит платежи (PAY_AMT1) сверх своего кредитного лимита. Этот потенциальный логический недостаток может помешать отпугнуть клиентов от досрочного погашения или наложить неуместный штраф на хороших клиентов, которые не вносили платежи, скажем, во время отпуска. Хотя такое поведение модели не обязательно приведет к проблемам, операторам модели определенно следует об этом знать, а нам необходимо добавить описание проблемы в документацию нашей модели.

Конечно, существует проблема реальных состязательных примеров. Не волнуйтесь, мы нашли их достаточное количество. Мы обнаружили много строк данных, которые могут вызвать прогноз низкой (около 5%) вероятности дефолта, а также много строк, которые могут вызвать высокую (около 70%) вероятность дефолта, а также строки с различными промежуточными результатами. Теперь у нас есть полный набор состязательных примеров, из которых можно извлечь практически любую вероятность дефолта, которую мы хотим получить от модели. Если читателям интересно, почему это важно, прочтите главы 5 и 11, посвященные безопасности ML. Детали реализации показаны в примерах кода в этой главе.



Еще один любимый метод анализа чувствительности, который мы хотим выделить, — это хитрый прием с перетасовкой меток:

- случайным образом перетасуйте целевой признак и переобучите модель;
- заново вычислите важность признака;

Книги для программистов: https://t.me/free_books_for_it

- попробуйте удалить признаки, которые важны для прогнозирования случайно перетасованной (бессмысленной) цели.

Этот прием помогает нам находить и удалять неробастные признаки.

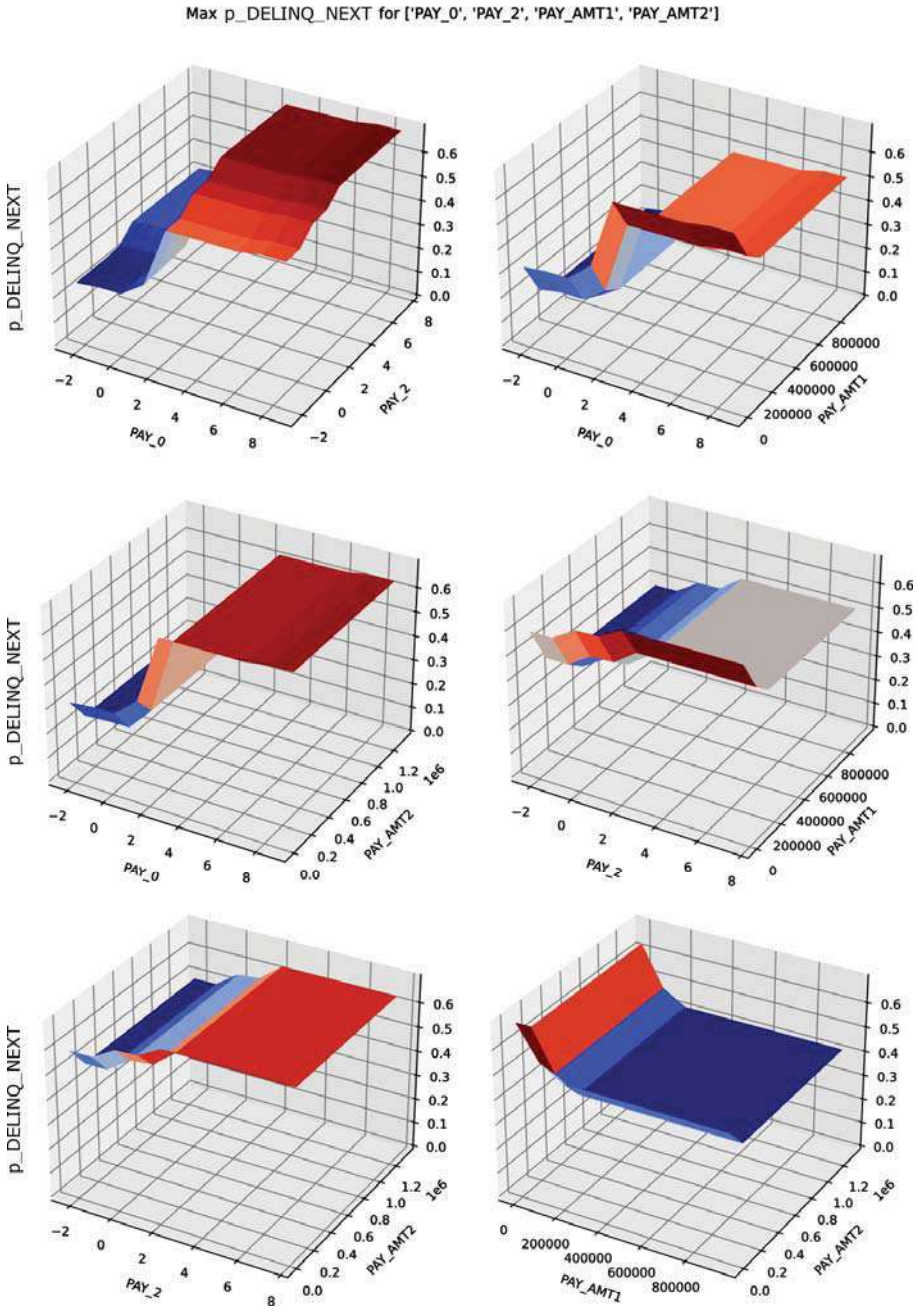


Рис. 8.5. Поиск состязательных примеров показывает поведение модели в ряде примечательных сценариев

Книги для программистов: https://t.me/free_books_for_it

При отладке всегда нужно рассматривать возможность устранения обнаруженных проблем. Логические проблемы, связанные с досрочными платежами, вероятно, могут быть решены с помощью бизнес-правил или утверждений модели. Например, если клиент вносит крупный опережающий платеж и сообщает банку, что направляется на тропический остров, вероятность дефолта может быть скорректирована на несколько месяцев. Что касается состязательных примеров, наиболее выраженные состязательные строки могут быть записаны в документации модели с примерами, чтобы будущие операторы системы были в курсе потенциальных проблем модели. Можно даже обсудить возможные атаки на основе состязательных примеров с коллегами из команды безопасности и рассмотреть возможность мониторинга состязательных примеров в режиме реального времени.

Анализ остатков для XGBoost

Мы рассмотрели локальные возмущения, которые могут вызвать проблемы в работе нашей модели (состязательный поиск примеров), а также потенциально проблемные глобальные возмущения (стресс-тестирование). Настало время перейти к анализу остатков. Мы начнем с традиционного подхода — с построения графика остатков по каждому уровню важности входного признака. На наших графиках остатков мы будем искать строки, которые вызывают наибольшие ошибки и какие-либо очевидные закономерности. Затем мы разобьем наши прогнозы на сегменты и проанализируем качество модели в этих сегментах. В приложениях высокого риска недостаточно понять, как модель работает в среднем. Нам нужно знать, как модель работает в важных сегментах данных. В завершение анализа остатков мы попытаемся смоделировать наши остатки с помощью дерева решений. Проанализировав правила дерева, мы сможем понять, каким образом наша модель допускает ошибки, и, возможно, использовать эти знания, чтобы улучшить ее работу. Пора начать учиться на своих ошибках. Итак, давайте внимательно рассмотрим остатки.

Практический анализ ошибок за рамками обучения с учителем

Вот несколько идей по применению анализа ошибок к разным типам моделей:

Кластеризация

Тщательно изучите выбросы — точки данных, наиболее удаленные от центроидов кластера, или точки данных, которые могут находиться в неправильном кластере в соответствии с силуэтом; тщательно обосновывайте количество кластеров в соответствии со статистическими показателями разрыва (*https://oreil.ly/iZ7VL*).

Матричная факторизация

Тщательно оцените ошибку реконструкции и объяснимую дисперсию, особенно в отношении выбросов и количества факторов.

Компьютерное зрение

Бинаризируйте конкретные задачи, такие как распознавание лиц, и примените традиционную оценку модели и остаточный анализ, как описано в этой главе. Оцените ошибки классификации изображений на предмет качества данных или проблем

Книги для программистов: https://t.me/free_books_for_it

с маркировкой данных. Примените соответствующие общедоступные тесты качества модели (глава 9).

Языковые модели

Бинаризируйте конкретные задачи, такие как распознавание именованных объектов, и примените традиционную оценку модели и остаточный анализ, как описано в этой главе. Примените соответствующие общедоступные тесты качества модели (раздел «А как насчет языковых моделей?» в главе 9).

Сегментный анализ качества или стабильности — отличный подход для всех типов моделей.

Анализ и визуализация остатков

Как показано на рис. 8.1, анализ остатков помогает нам понять ценность модели для бизнеса — или осознать ее отсутствие. Это также отличный способ узнать технические подробности того, как именно наша модель допускает ошибки. В случае нашей модели мы будем рассматривать остатки логарифмических потерь (logloss), а не традиционные остатки, поскольку наша модель была обучена с использованием логарифмических потерь. Для большинства людей одним из простейших способов начать анализировать остатки модели ML является построение их графика. Мы начнем с обзора глобальных остатков логарифмических потерь для выбранной нами модели, а затем увеличим масштаб остатков для наиболее важного входного признака `PAY_0`. В обоих случаях мы постараемся разобраться в причинах появления ошибок нашей модели и найти способы их устранения, если это вообще возможно. Первым шагом к построению графика остатков является, конечно же, их вычисление. Мы будем находить остатки логарифмической функции потерь — тип ошибки, используемый во время обучения модели для функции потерь `binary:logistic` в XGBoost. Таким образом, устранение больших остатков должно иметь прямое влияние на обучение модели. Для вычисления остатков нам понадобятся целевые и прогнозируемые значения, как показано в следующем блоке кода, а затем мы применим стандартную формулу для двоичного логарифма:

```
# имя, присвоенное остатку
resid = 'r_DELINQ_NEXT'

# Вычисление остатков логарифмической потери
valid_yhat[resid] = -valid_yhat[y]*np.log(valid_yhat[yhat]) -\
(1 - valid_yhat[y])*np.log(1 - valid_yhat[yhat])
```

Одно из небольших преимуществ расчета остатков таким способом заключается в том, что мы можем проверить, соответствует ли среднее значение остатка логарифмической потере, сообщенной XGBoost в конце обучения, чтобы гарантировать, что для генерации прогнозов мы выбрали именно модель правильного размера. После прохождения этой проверки можно переходить к построению графика остатков, пример которого читатели могут видеть на рис. 8.6. Обратите внимание, что на рис. 8.6 содержится признак `r_DELINQ_NEXT`. Значение остатка логарифмических потерь называется `r_DELINQ_NEXT`, а `p_DELINQ_NEXT` — это прогноз цели `DELINQ_NEXT`.

Книги для программистов: https://t.me/free_books_for_it

Остатки логарифмических потерь выглядят немного иначе, чем типичные остатки регрессии, которые, возможно, вам знакомы по урокам статистики. Вместо случайного набора точек мы видим одну кривую для каждого результата модели; для `DELINQ_NEXT = 0` она изгибается вверх и вправо, а при `DELINQ_NEXT = 1` она изгибается вверх и влево. Первое, что мы видим на этом графике, — наличие больших выбросов для обоих исходов. Однако их количество и интенсивность значительно выше для `DELINQ_NEXT = 1`.

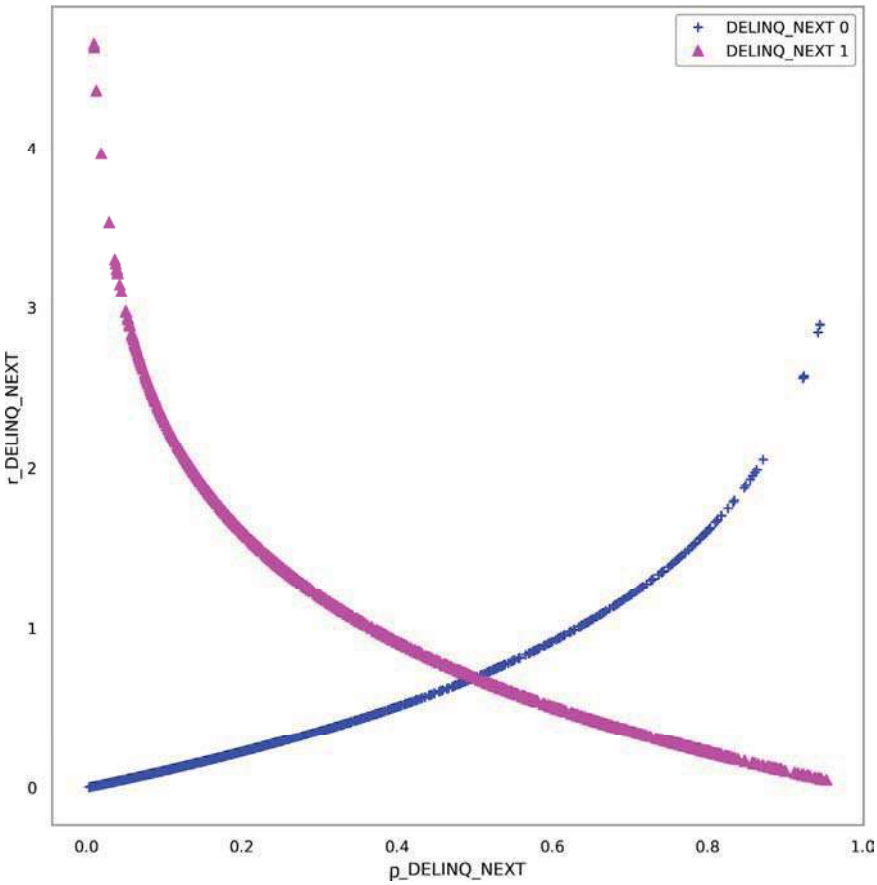


Рис. 8.6. Остатки логарифмических потерь для положительных и отрицательных откликов выбранной модели XGBoost

Этот график показывает, что в проверочных данных есть некоторые клиенты, пропускающие платежи, но наша модель считает, что они этого не сделают. Сортируя проверочные данные по новому столбцу `r_DELINQ_NEXT` и просматривая строки с самыми большими остатками, мы можем получить представление о том, что происходит с этими клиентами. Итак, кто же эти клиенты? Оказывается, они хорошие клиенты — с большими кредитными лимитами и всегда платят вовремя — и которые пропустят свой следующий платеж. Они вызовут удивление нашей модели и приведут к возникновению огромных остатков.

Книги для программистов: https://t.me/free_books_for_it

Эти результаты указывают на фундаментальный недостаток в наших обучающих данных. Нам не хватает признаков, которые могли бы помочь нам лучше понять финансовую жизнь потребителей и причины, по которым они могут задерживать платежи. Например, в кредитных моделях часто используется отношение долга к доходу (debt-to-income, DTI). Мы можем увидеть увеличение коэффициента DTI клиентов еще до того, как они пропустят платеж. Без такой дополнительной информации нам придется признать, что мы обнаружили серьезное ограничение нашей модели. У нас просто нет столбцов данных, которые нам нужны, чтобы добиться большего. Понимая, что при имеющихся данных нашу модель можно легко удивить, мы могли бы также рассмотреть возможность удаления этих строк из наших обучающих данных, поскольку они вносят бесполезный шум в процесс обучения. Возможно, было бы хорошей идеей отказаться от данных, которые описывают таких людей, потому что на данный момент мы мало чему можем у них научиться. Скорее всего, мы улучшим показатели модели при валидации и тестировании и сможем разработать более стабильную и надежную модель

Прежде чем избавиться от этих точек данных, давайте найдем логарифмические остатки по каждому уровню наиболее важного признака `PAY_0`. Если мы обнаружим более конкретную историю или у нас возникнет вопрос при первоначальном анализе глобальных остатков, то на основании этой информации мы должны определить, какие остатки исследовать в дальнейшем. Поскольку мы не увидели информации, связывающей этих людей с каким-либо конкретным признаком, мы по умолчанию исследуем наиболее важный входной признак. Для этого мы воспользуемся удобным графиком Seaborn `FacetGrid`. Следующий код показывает, как быстро разбить остатки по уровням `PAY_0` и построить график остатков на каждом уровне в виде аккуратной сетки:

```
# фасетная сетка остатков от PAY_0
sorted_ = valid_yhat.sort_values(by='PAY_0')
g = sns.FacetGrid(sorted_, col='PAY_0', hue=y, col_wrap=4)
_ = g.map(plt.scatter, yhat, resid, alpha=0.4)
_ = g.add_legend(bbox_to_anchor=(0.82, 0.2))
```

На рис. 8.7 показаны остатки логарифмических потерь для положительных и отрицательных результатов на 11 уровнях `PAY_0` проверочных данных. В общем, мы должны следить за какими-либо сильными закономерностями в графиках.

Рис. 8.7 подтверждает историю, рассказанную глобальными остатками, и добавляет к ней некоторые подробности. В верхнем ряду рисунка показаны благоприятные значения для `PAY_0` (`-2`, `-1` или `0`), обозначающие своевременную оплату или неиспользование кредита, связанные с большими остатками для клиентов, которые внезапно не внесли платеж вовремя (`DELINQ_NEXT = 1`). Это некоторые из тех клиентов с высоким остатком, которых мы видели на рис. 8.6. В нижних рядах отображается прямо противоположное поведение. Клиенты с неблагоприятными значениями `PAY_0` вызывают большие остатки, когда они внезапно платят вовремя (`DELINQ_NEXT = 0`). Какой здесь урок? Рис. 8.7 показывает, что наша модель машинного обучения допускает те же ошибки, что и человек или простое бизнес-правило. Из главы 6 мы знаем, что эта модель слишком зависит от `PAY_0`. Теперь мы воочию наблюдаем

Книги для программистов: https://t.me/free_books_for_it

одно последствие этой патологии. Если клиенты имеют благоприятные значения PAY_0, модель будет шокирована, когда они не выполнят обязательства. Если клиенты имеют неблагоприятные значения PAY_0, модель будет шокирована, когда они заплатят вовремя

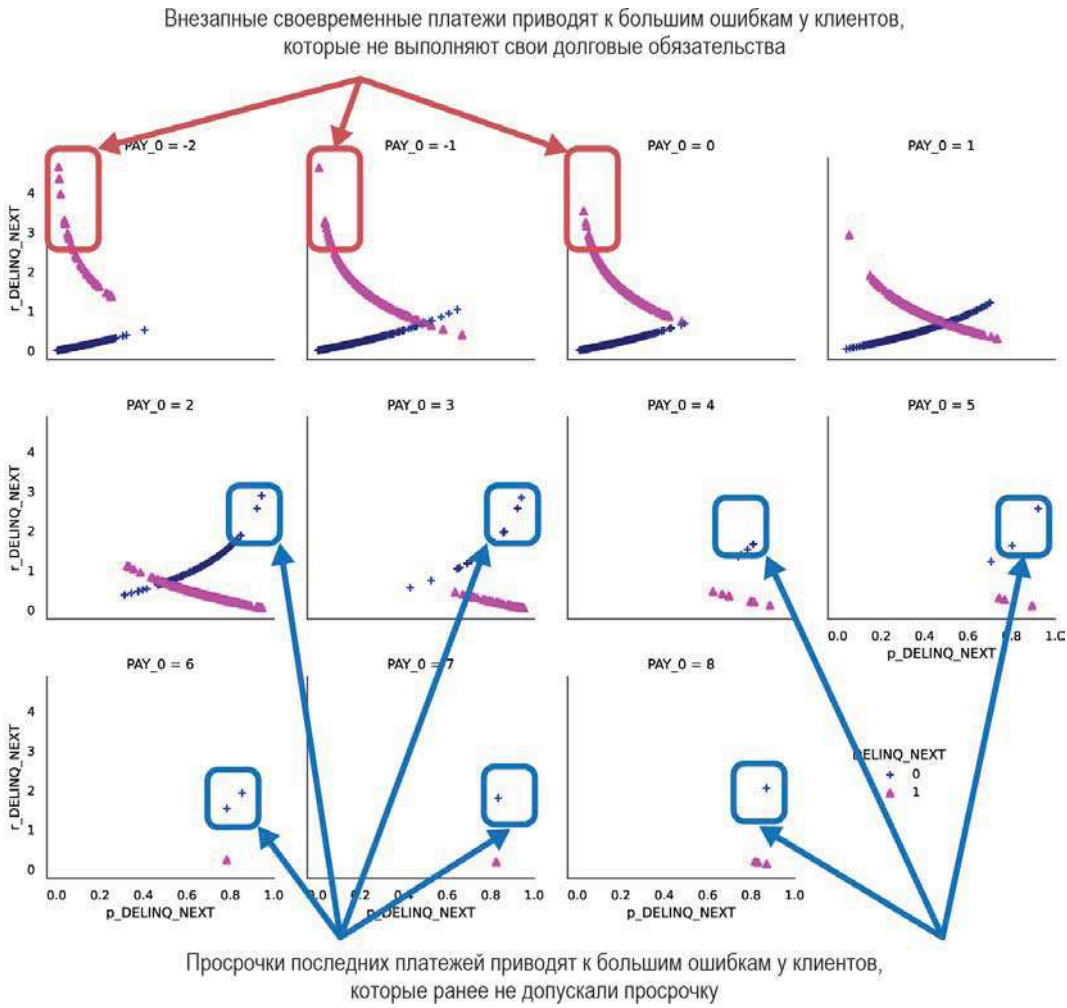


Рис. 8.7. Клиенты с хорошей платежной историей, которые перестают выполнять свои обязательства, а также клиенты с плохой платежной историей, которые внезапно начинают платить вовремя. И те и другие тут же становятся причиной крупных остатков

На самом деле для принятия такого рода решений нам не требуется модель машинного обучения с сотнями тысяч правил. Эти тысячи правил скрывают значительную сложность, которая, в свою очередь, может скрывать предвзятость или проблемы безопасности. Такую модель либо необходимо существенно улучшить, собрав больше столбцов данных и обучив заново, либо рассмотреть возможность замены ее более прозрачным и безопасным бизнес-правилом: ЕСЛИ PAY_0 < 2, ТО ВЫДАЕМ КРЕДИТ, ИНАЧЕ ОТКАЗЫВАЕМ. По сути, модели требуется больше данных — какой-то но-

Книги для программистов: https://t.me/free_books_for_it

вый входной столбец, который мог бы рассказать нам о финансовой стабильности клиента вне контекста текущего кредита. Без этой информации мы используем слишком сложный и, следовательно, неоправданно рискованный процесс принятия решений, которые в конечном итоге оказываются непозволительно упрощенными. Мы попытаемся исправить ситуацию в разделе «Исправление выбранной модели» далее в текущей главе, но прежде чем этим заняться, нужно убедиться, что модель не таит в себе никаких других сюрпризов. Далее проведем сегментный анализ ошибок и выявим проблемные места в показателях качества модели.

Сегментный анализ ошибок

Выбранная нами модель имеет AUC 0,78 на проверочных данных. Это приличное значение AUC, которое указывает на то, что наша модель правильно ранжирует отрицательные и положительные результаты примерно в 80% случаев на проверочном наборе данных. Значит, модель готова к развертыванию, верно? Увы, мы только что увидели, как более тщательный анализ ошибок позволяет выявить серьезные проблемы, которые не были заметны при использовании более простых показателей оценки. И, к сожалению, скоро станет понятно, что респектабельный AUC высшего уровня тоже мало что значит.

В табл. 8.2 приведены результаты расчета общих показателей модели и ошибок двоичной классификации на всех уровнях `PAY_0`. Этот метод иногда называют *сегментным анализом ошибок*. Основная идея заключается в том, что разные показатели сообщают нам разную информацию о модели. Например, AUC верхнего уровня говорит нам об общей способности модели правильно ранжировать клиентов, а точность говорит нам о частоте ошибок при определенном пороге вероятности, в то время как доля истинно положительных результатов и доля ложноположительных результатов, разбивают точность на более конкретные аспекты правильных и неправильных решений. Более того, нам нужно владеть этой информацией для различных сегментов моделируемой популяции. Наиболее эффективные модели будут демонстрировать надежное принятие решений во всех сегментах моделируемой совокупности, а не только для самых крупных сегментов данных. Когда мы имеем дело с кредитным портфелем в миллиард долларов, эти меньшие сегменты по-прежнему представляют собой большую сумму денег. В других приложениях высокого риска меньшие сегменты могут представлять жизненно важные решения в области финансов, уголовного правосудия или здравоохранения.

Заметим, что значения, приведенные в табл. 8.2, возникают из матриц несоответствий и могут варьироваться в зависимости от выбора порога вероятности. Значения были рассчитаны с использованием порога, выбранного путем максимизации статистического показателя F1 модели. Но если бы мы собирались развернуть эту модель в производстве, нам пришлось бы использовать порог вероятности, используемый в производственном конвейере. Мы несем ответственность за то, чтобы модель работала хорошо in vivo, анализируя несколько различных показателей для всех групп, на которые влияют решения модели. Отсюда также вытекают серьезные последствия для справедливости, но мы обсудим этот вопрос в других главах, а пока подробно рассмотрим табл. 8.2.

Книги для программистов: https://t.me/free_books_for_it

Таблица 8.2. Сегментный анализ ошибок

PAY_0	Преобладание	Точность	Истинно положительный результат	Верность	Специфичность	Отрицательное прогнозируемое значение	Ложноположительный результат	...	Ложноотрицательный результат	Доля ложных отказов
–2	0.118	0.876	0.000	0.000	0.993	0.881	0.007	...	1.000	0.119
–1	0.177	0.812	0.212	0.438	0.941	0.847	0.059	...	0.788	0.153
0	0.129	0.867	0.089	0.418	0.982	0.880	0.018	...	0.911	0.120
1	0.337	0.566	0.799	0.424	0.448	0.814	0.552	...	0.201	0.186
2	0.734	0.734	1.000	0.734	0.000	0.500	1.000	...	0.000	0.500
3	0.719	0.719	1.000	0.719	0.000	0.500	1.000	...	0.000	0.500
4	0.615	0.615	1.000	0.615	0.000	0.500	1.000	...	0.000	0.500
5	0.571	0.571	1.000	0.571	0.000	0.500	1.000	...	0.000	0.500
6	0.333	0.333	1.000	0.333	0.000	0.500	1.000	...	0.000	0.500
7	0.500	0.500	1.000	0.500	0.000	0.500	1.000	...	0.000	0.500
8	0.750	0.750	1.000	0.750	0.000	0.500	1.000	...	0.000	0.500

В табл. 8.2 все выглядит нормально, пока мы не доходим до пятой строки, где `PAY_0 = 2`. С этого момента таблица показывает серьезную проблему, возможно, даже более серьезную, чем график остатков относительно `PAY_0` в предыдущем разделе. Грубо говоря, при `PAY_0 = 2` и выше эта модель почти не работает. Например, мы наблюдаем уровень ложноположительных результатов 1,0. Это означает, что модель ошибается в отношении всех, кто *не* пропустит платеж, — модель предсказывает, что все они допустят просрочку. Почему это происходит? Самая очевидная причина кроется, опять же, в обучающих данных. Из графика остатков следует, что нам, скорее всего, не хватает некоторых важных входных признаков. Благодаря сегментному анализу ошибок мы теперь видим, что могли быть упущены некоторые важные строки данных. В наших обучающих данных слишком мало людей с `PAY_0 > 1`, чтобы модель могла узнать о них что-нибудь разумное. Еще раз взгляните на некоторые рисунки из главы 6 или на рис. 8.7. На графиках для `PAY_0 > 1` точек просто мало.



Высокоуровневые и усредненные показатели могут скрывать неприятные проблемы. Всегда проводите сегментный анализ ошибок для приложений высокого риска.

Просто невероятно, что может скрываться за показателем AUC 0,78. Мы надеемся, что этот пример убедит читателей в важности сегментного анализа ошибок. Читатели, возможно, справедливо задумаются о способах решения проблемы. Наиболее очевидным решением будет отложить развертывание этой модели до тех пор, пока не будет собрано достаточно данных о клиентах, пропускающих платежи. Это позволит обучить более качественную модель. Если модель придется развернуть

Книги для программистов: https://t.me/free_books_for_it

как есть, нам, вероятно, потребуются специалисты, которые будут принимать решения об отказе, по крайней мере, для тех клиентов, у которых PAY_0 > 1. В конце главы мы обсудим дополнительные стратегии исправления этой ситуации, но прежде следует лучше понять закономерности в найденных нами остатках. Далее мы обучим на этих остатках интерпретируемую модель, чтобы получить дополнительную информацию о том, что стоит за недостатками нашей модели.

Моделирование остатков

В главе 6 мы использовали интерпретируемое дерево решений для моделирования наших прогнозов на основе входных признаков, чтобы получить лучшее представление о том, какие входные признаки и каким образом влияют на прогнозы. Теперь мы воспользуемся тем же подходом, чтобы получить некоторое представление о том, что движет нашими остатками. Если мы замечаем некоторое совпадение между объяснением и отладкой, это неспроста. Одно из лучших применений апостериорных объяснений — помощь в отладке.

Мы применим следующий код, чтобы по отдельности обучить четырехуровневое дерево решений на наших остатках DELINQ_NEXT = 0 и DELINQ_NEXT = 1. Чтобы получить это дерево, мы будем использовать наши исходные входные данные в качестве входных данных дерева, но вместо обучения на DELINQ_NEXT в качестве цели мы будем обучать его на остатках, т. е на r_DELINQ_NEXT. После обучения дерева мы сохраним H2O MOJO (сокращение от *m*odel *o*bject, *o*ptimized, объект оптимизированной модели). MOJO содержит специализированную функцию, которая позволяет перерисовать наше дерево остатков с помощью Graphviz — библиотеки с открытым исходным кодом для технической визуализации. Примерно то же самое можно сделать и с помощью нескольких других пакетов, включая scikit-learn.

```
# инициализируем одиночную модель дерева
tree = H2ORandomForestEstimator(ntrees=1,              ❶
                                sample_rate=1,         ❷
                                mtries=-2,             ❸
                                max_depth=4,           ❹
                                seed=SEED,             ❺
                                nfolds=3,              ❻

                                model_id=model_id)     ❼

# обучаем одиночную модель дерева
tree.train(x=X, y=resid, training_frame=h2o.H2OFrame(frame))

# сохраняем MOJO (компилированное представление Java обученной модели),
# из которого получаем визуальное представление дерева
mojo_path = tree.download_mojo(path='.')
print('Путь к готовому MOJO:\n', mojo_path)
```

❶ Используем только одно дерево.

❷ Используем все строки в этом дереве.

Книги для программистов: https://t.me/free_books_for_it

❸ Используем все столбцы в разделенном поиске этого дерева.

❹ Неглубокие деревья легче понять.

❺ Задаем начальное число рандомизатора для воспроизводимости.

❻ Перекрестная проверка на стабильность и единственный способ получить метрики для одного дерева в H2O.

❼ Присваиваем объекту MOJO узнаваемое имя.

Как и в случае с нашей суррогатной моделью, которую мы получали для целей объяснения, нет никаких фундаментальных теоретических гарантий того, что эта модель говорит нам правду о том, что влияет на остатки. Как всегда, нам нужно быть осторожными и аккуратными. Для этого дерева решений мы рассчитаем общие показатели ошибок, чтобы убедиться, что дерево действительно соответствует остаткам. Поскольку нестабильность является хорошо известным видом отказа для одиночных деревьев решений, мы проверим показатели перекрестной ошибки, чтобы убедиться, что дерево также стабильно. Также важно помнить, что если причина аномальных остатков выходит за рамки входных признаков, дерево ничего не сможет об этом сообщить. Мы уже знаем, что некоторые важные проблемы в нашей модели возникают из-за нехватки данных, так что при анализе дерева нам следует помнить об этом.

На рис. 8.8 показана модель дерева решений остатков нашей модели ML для `DELINQ_NEXT = 0`, или клиентов, которые не пропустили предстоящий платеж. Хотя оно и повторяет то, что мы уже обнаружили на рис. 8.7, зато делает это очень понятно, раскрывая логику сбоев. Фактически на основе этого дерева можно даже построить правила, определяющие, когда модель, скорее всего, потерпит неудачу.

На рис. 8.8 показано, что при движении от вершины дерева к наибольшему среднему значению остатка в нижней его части наибольшие остатки для отрицательных (отказных) решений возникают, когда `PAY_0 >= 1,5 И PAY_3 >= 1,0 И BILL_AMT3 < 2829,50 И PAY_6 >= 0,5`. Вспомним, что мы видели на рис. 8.7: когда у клиента есть просроченные выплаты в течение нескольких месяцев и небольшие суммы счетов, модель будет шокирована, если он сделает свой следующий платеж вовремя. Теперь мы сузили наше наблюдение до конкретных клиентов, которые в среднем вызывают наихудшие остатки. У нас есть бизнес-правило, определяющее, как действовать в подобных в ситуациях, вызывающих наибольшую озабоченность.

В целом этот метод моделирования остатков помогает выявить причины отказов. Когда причины отказов достоверно установлены, их часто можно смягчить для повышения надежности и безопасности модели. Если группу клиентов, вызывающих наибольшие остатки на рис. 8.8, удастся изолировать от закономерностей, приводящих к прогнозу о пропущенных выплатах, то можно будет разработать точные стратегии исправления ситуации в форме бизнес-правил (или утверждений модели). Если модель встречает клиента с характеристиками `PAY_0 >= 1,5 И PAY_3 >= 1,0 И BILL_AMT3 < 2829,50 И PAY_6 >= 0,5`, возможно, она не должна автоматически предполагать, что клиент объявит дефолт. Мы могли бы рассмотреть возможность корректировки вероятности дефолта клиентов из этой группы с помощью бизнес-правила или передачи их кредитного решения для более детального рассмотрения специа-

Книги для программистов: https://t.me/free_books_for_it

листом по сопровождению кредитов. На данный момент отладка выявила серьезную проблему в нашей модели. В ее распоряжении не было достаточного объема корректных обучающих данных ни в столбцах, ни в строках Из-за этого ее легко ввести в замешательство в распространенных и важных сценариях принятия решений. Помимо сбора или моделирования более качественных данных, мы нашли еще одну потенциальную тактику исправления: бизнес-правила, которые сигнализируют о риске принятия неправильного решения, могут служить индикатором необходимости принятия оперативных мер по смягчению этого неправильного решения. В следующем разделе мы завершим главу, обсудив дальнейшие действия по исправлению ситуации.



Методы апостериорного объяснения, такие как интерпретируемые суррогатные модели, часто оказываются наиболее эффективными в качестве инструментов для отладки моделей.

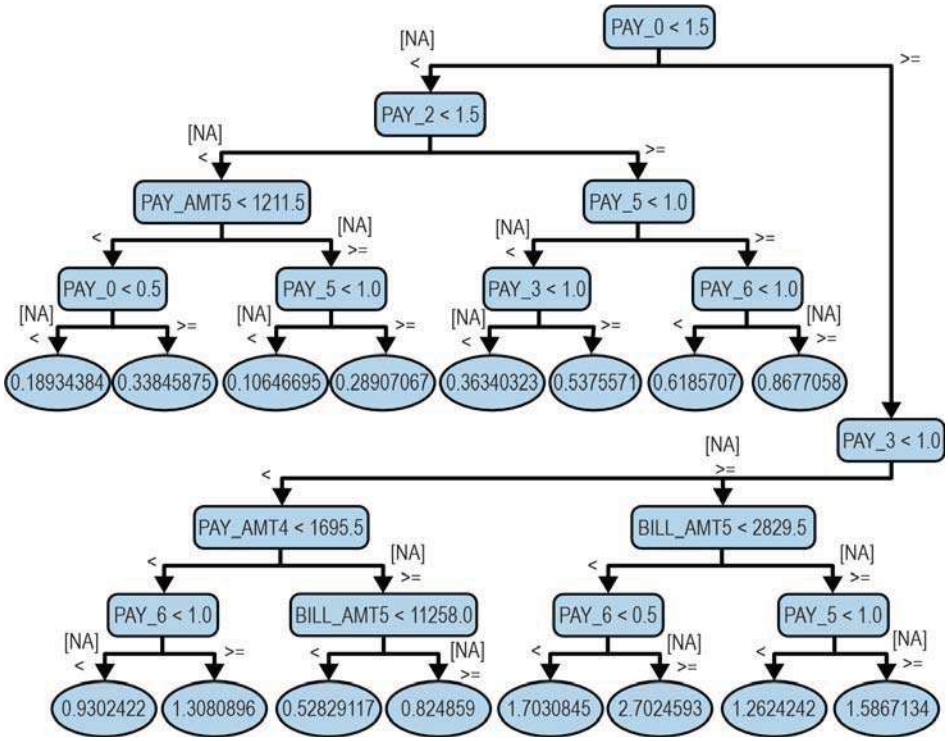


Рис. 8.8. Дерево решений, показывающее остатки модели и раскрывающее закономерности, которые можно использовать для выявления причин сбоев и разработки подходов к их устранению

Исправление выбранной модели

Несмотря на то что мы использовали ограничения монотонности и регуляризацию, провели тщательный поиск по сетке и напряженные тесты по выбору модели, а также несмотря на наше искреннее желание обучить хорошую модель, в результате

Книги для программистов: https://t.me/free_books_for_it

мы обучили плохую модель, которую не следует развертывать. Помимо недостаточных данных для обучения, напомним, мы обнаружили следующие недостатки:

- она патологически переоценивает последний статус кредитного платежа клиента (`PAY_0`);
- она демонстрирует логические ошибки, которые могут помешать внесению досрочных платежей или отрицательно повлиять на состоятельных клиентов;
- она может быть уязвима для состязательных атак;
- плохо работает при `PAY_0 > 1`.

Хотя мы рассмотрим каждую из этих проблем отдельно, вместе они словно сговорились сделать, казалось бы, приемлемую модель ML менее привлекательной, чем простое бизнес-правило, по крайней мере, с точки зрения авторов. Поскольку многие модели машинного обучения не проходят надлежащую отладку перед развертыванием, вполне вероятно, что и вы столкнетесь с подобными ошибками, требующими немедленного устранения, если примените методы отладки, описанные в этой главе, к одной из моделей вашей организации. Независимо от того, попытается ли ваша команда исправить эту модель на ходу или вернется к разработке с нуля, важно продумать, как решить проблемы, выявленные анализом чувствительности и остатков. Другими словами, найденные проблемы в идеале следует устранить до того, как будет развернута эта или аналогичная модель. Для исправления найденных ошибок в модели, которую мы используем в качестве примера, можно применить несколько методов.

Данные обучения представляют как самые простые, так и самые сложные варианты исправления. Решение очевидно. Реализация решения требует здравого смысла и упорной работы. Собирайте больше качественных обучающих данных. Используйте методы планирования эксперимента для сбора и отбора данных. Используйте методы обнаружения причинной связи, чтобы выбрать входные признаки, которые действительно влияют на цель прогнозирования. При необходимости рассмотрите возможность моделирования данных.



Зачастую лучшее, что мы можем сделать для повышения критических показателей нашей системы ML, — это собрать больше данных более высокого качества.

Что касается остальных выявленных проблем, давайте попробуем решить их одну за другой, и пусть это послужит примером устранения подобных ошибок в вашей повседневной работе. Мы уделим особое внимание чрезмерному акценту модели на `PAY_0`, поскольку здесь просматривается очевидный потенциал снижения риска, а затем перейдем к другим выявленным причинам сбоев.

Чрезмерная важность признака *PAY_0*

Возможно, самая большая проблема выбранной нами модели и многих других моделей машинного обучения — это плохие обучающие данные. В этом случае обучающие данные должны быть дополнены новыми признаками, чтобы основные

Книги для программистов: https://t.me/free_books_for_it

механизмы принятия решений модели опирались на несколько признаков вместо одного. Одной из стратегий повышения стабильности и обобщения модели является добавление нового признака, который будет отслеживать расходы клиентов с течением времени. Этот признак поможет выявить любую потенциальную финансовую нестабильность, а именно стандартное отклонение сумм счетов клиентов за шесть месяцев — `bill_std`. В Pandas есть однострочный код для расчета стандартных отклонений в наборе столбцов.

```
data['bill_std'] = data[['BILL_AMT1', 'BILL_AMT2',
                         'BILL_AMT3', 'BILL_AMT4',
                         'BILL_AMT5', 'BILL_AMT6']].std(axis=1)
```

По такому же принципу мы могли бы также создать новый признак `pay_std`, содержащий информацию о статусе выплат, за исключением самой последней (мы не хотим снова переоценивать `PAY_0`):

```
data['pay_std'] = data[['PAY_2','PAY_3','PAY_4','PAY_5','PAY_6']].std(axis=1)
```

Для уменьшения чрезмерной значимости признака `PAY_0` также можно ввести в него случайный шум, но только если в более качественных обучающих данных доступны другие точные сигналы. Мы рандомизируем столбец `PAY_0`, но только там, где `PAY_0` равно 0, 1 или 2. Этот тип искажения сродни сильной регуляризации. Мы очень хотим заставить модель обратить внимание на другие признаки.

```
data['PAY_0'][(data['PAY_0']>= 0) & (data['PAY_0']< 3)].sample(frac=1).values
```

Выполнив эти шаги, чтобы уменьшить значение `PAY_0` в обучающих данных, мы заново обучаем нашу модель. Итоговый сводный график SHAP (рис. 8.9) показывает, что мы смогли уменьшить важность признака `PAY_0`. Он ушел вниз с верхней позиции на сводном графике и занял место `PAY_2`, а наши новые сконструированные признаки оказались более важными, чем `PAY_0`. Также мы наблюдаем небольшое снижение значения AUC, которое сейчас составляет 0,7501 вместо первоначального 0,7787.

Теперь самый сложный вопрос: а лучше ли новая модель? Общий показатель AUC, на который часто полагаются при выборе «хорошего» классификатора, снизился. Но, во-первых, мы уже видели, что in silico AUC не имеет большого значения. Во-вторых, снижение показателей на тестовых данных практически гарантировано при изменении модели. Процесс обучения модели безжалостно выполняет оптимизацию по какому-то выбранному критерию, а затем стремится выбрать лучшую модель по тому же критерию на проверочных данных. Если мы повторим этот процесс несколько раз, то, скорее всего, увидим ухудшение показателей, выбранных in silico.



Исправление модели, скорее всего, ухудшит ее статистические показатели. Это нормально. Не существует статистики, которая гарантированно точно предсказывала бы качество модели в реальных условиях. Пока исправление основано на надежных знаниях предметной области, мы можем пожертвовать некоторыми формальными показателями качества на тестовых данных in silico, чтобы развернуть более надежную модель in vivo.

Книги для программистов: https://t.me/free_books_for_it

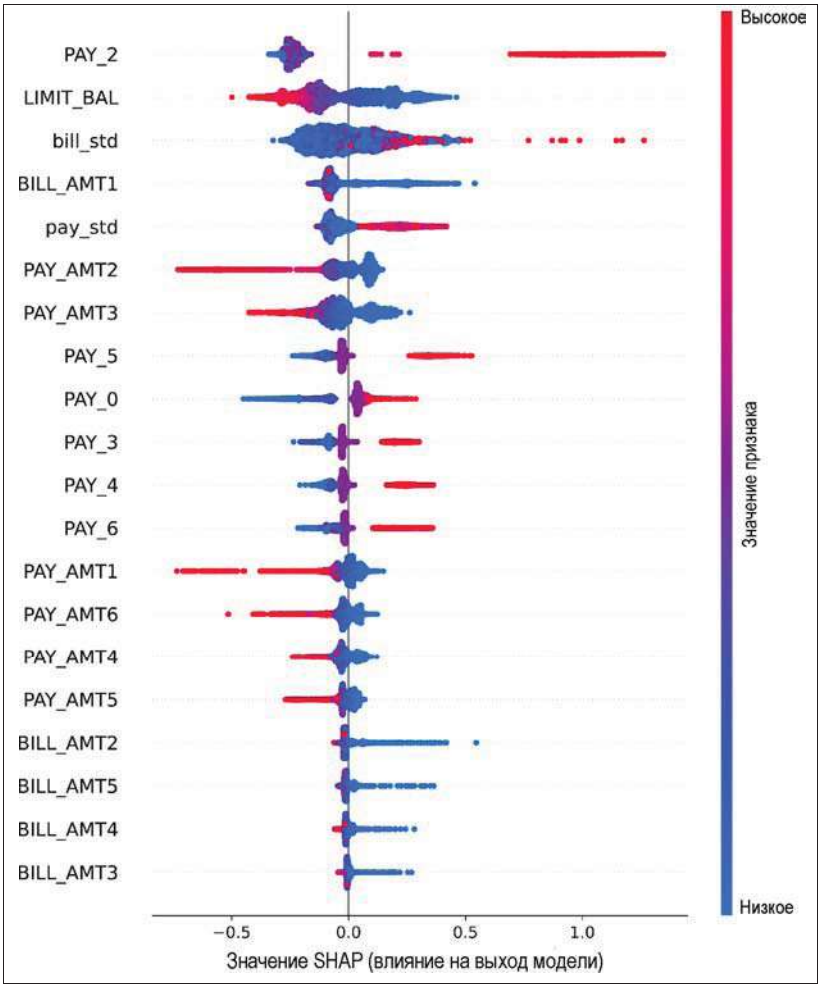


Рис. 8.9. Значения Шепли для каждой входной переменной после уменьшения важности `PAY_0`

Единственный способ узнать, стала ли модель лучше, — это еще раз отладить ее и проконсультироваться с экспертами в предметной области. Хотя это довольно скучно и может разочаровать, но по-другому не получится. Статистических данных, предсказывающих потрясающие результаты in vivo, пока нет. Хорошие модели всегда нуждались в отладке и знании предметной области, чтобы правильно функционировать в реальном мире. А мы тем временем продолжим совершенствовать нашу модель, изучая оставшиеся проблемы, которые были выявлены в процессе отладки: логические ошибки, уязвимости безопасности и низкое качество прогнозов при `PAY_0 > 1`.

Разные ошибки

Вскоре мы приступим к обсуждению других технических подходов к устранению проблем, но сначала кратко рассмотрим вопросы планирования экспериментов.

Книги для программистов: https://t.me/free_books_for_it

Чтобы устранить расхождения между двумя ипостасями моделей ML — между инженерными проектами, ориентированными на ошибки испытаний in silico, и экспериментами, ориентированными на результаты in vivo, необходимо обновить наш рабочий процесс, чтобы он более точно соответствовал традиционной методике научного эксперимента:

1. Разработайте достоверную догадку (например, на основе предыдущих экспериментов или обзора литературы).
2. Запишите свою гипотезу (т. е. предполагаемый реальный результат вашей системы ML).
3. Соберите соответствующие данные (например, используя методы планирования эксперимента).
4. Проверьте гипотезу о том, что система ML оказывает ожидаемый эффект in vivo на экспериментальную группу, используя такие методы, как:
   - A/B-тестирование для понимания влияния результатов модели на неформальную экспериментальную группу.
   - Примените приближенно-точное сопоставление (*https://oreil.ly/jEf8O*) для построения контрольной и экспериментальной групп на основе собранных данных наблюдений и проверки статистически значимых эффектов воздействия модели.

Если вы проделаете огромную работу методом проб и ошибок с необъяснимыми моделями и с сильными дозами предвзятости подтверждения и предвзятости финансирования (как это происходит сегодня во многих проектах ML), то, вероятно, будете очень удивлены тем, как ваша модель работает после развертывания. (Помните цитату исследовательской группы Google в начале главы?) Проблему отсутствия научной строгости в культуре разработки невозможно решить техническими средствами. На сегодняшний день большая часть ML по-прежнему является экспериментальной наукой, а не тривиальными инженерными задачами. В главе 12 более подробно рассматриваются вопросы науки о данных и научного метода.



Мы часто рассматриваем свои эксперименты как способ выбора лучшего алгоритма. Но на самом деле речь должна идти о результатах in vivo, которые получают пользователи, клиенты или субъекты системы.

Исправление серьезных ошибок в данных и методологии, вероятно, окажет положительное влияние на логические ошибки, уязвимости безопасности и плохие показатели качества вашей модели. В следующем списке мы рассмотрим еще несколько очевидных исправлений, которые могут пригодиться и в современных конвейерах обработки данных. Мы завершим этот раздел кратким обсуждением калибровки. Калибровка прогнозов с учетом ранее известных результатов — еще одно распространенное решение, основанное на здравом смысле.

Логические ошибки

Для логических ошибок, наподобие тех, которые приводят к высокой вероятности прогноза о дефолте даже после осуществления очень крупных платежей,

Книги для программистов: https://t.me/free_books_for_it

вероятным решением являются утверждения модели или бизнес-правила. Здесь можно применить утверждение модели или бизнес-правило к клиентам, у которых совсем недавно возникла двухмесячная просрочка, чтобы проверить, не был ли недавно внесен крупный платеж, прежде чем публиковать неблагоприятный прогноз по умолчанию. Остаточная модель, подобная той, что показана на рис. 8.8, ориентированная на эту небольшую группу клиентов, могла бы помочь предложить или уточнить более конкретные утверждения или правила.

Уязвимости безопасности

Мы обнаружили, что моделью легко манипулировать с помощью состязательных примеров. В целом проверенные методы, такие как замедление ответов и аутентификация API, скоординированные с мониторингом модели в реальном времени, очень помогают в обеспечении безопасности ML (глава 5). К этой модели также можно применить ограничения целостности данных или мониторинг случайных или смоделированных данных, т. е. обнаружение аномалий. По сути, нашей модели может потребоваться дополнительный мониторинг, проверяющий наличие аномалий, таких как своевременный последний платеж (`PAY_0 = 1`) и просрочка второго последнего платежа на шесть месяцев (`PAY_2 = 6`). Если в очереди запросов выявлены аномальные данные (с использованием любого метода, от алгоритмов изолирующего леса до логических ограничений целостности данных), эти данные должны быть направлены для более тщательной проверки, прежде чем будет принято решение о кредите.

Низкое качество прогнозов для PAY_0 > 1

Как и в случае многих других проблем, этой модели необходимы более качественные данные, чтобы узнать больше о клиентах, которые в конечном итоге не выполнили свои кредитные обязательства. В отсутствие этой информации можно использовать веса наблюдений, повторную выборку или моделирование, чтобы усилить влияние небольшого числа клиентов, которые пропустили платежи. Кроме того, ограничения монотонности модели являются одним из лучших способов смягчения последствий от нехватки обучающих данных. Ограничения монотонности обеспечивают хорошо понятные элементы управления моделью в реальном мире. Однако качество прогнозов модели при `PAY_0 > 1` остается крайне низким даже с учетом ограничений. Прогнозы в этом диапазоне, возможно, придется обрабатывать с помощью более специализированной модели, системы, основанной на правилах, или даже специалистов по поддержке клиентов.

Калибровка прогнозов с учетом ранее известных результатов — это еще один традиционный подход к исправлению ситуации, который, вероятно, улучшил бы многие характеристики нашей модели. Калибровка означает, что вероятности нашей модели связаны с прошлыми известными результатами — по сути, это означает, что, когда наша модель выдает прогноз, скажем, 0,3, то среди клиентов, показатели которых аналогичны клиенту, получившему этот прогноз, доля фактически допустивших дефолт составляет 30%. Мы можем использовать графики и оценку Брайера для обнаружения проблем с калибровкой и масштабирования выходных вероят-

Книги для программистов: https://t.me/free_books_for_it

ностей для их устранения. Модуль калибровки вероятности (*https://oreil.ly/LP9nf*) в scikit-learn содержит полезную информацию и функциональность, позволяющую начать работу с калибровкой двоичных классификаторов.

Заключение

Читатели, возможно, смогут придумать другие способы исправить наш пример плохой модели, и это здорово. Самое главное — не пропускайте отладку в следующий раз, когда будете обучать модель. Во многих отношениях машинное обучение похоже на обычное программирование. Без тестирования кода программы мы не сможем каким-то волшебным образом избежать ошибок. Мы игнорируем ошибки. Отладка крайне важна при разработке любого программного обеспечения — от операционных систем до моделей ML. Хотя можно применить традиционное модульное, интеграционное и функциональное тестирование для поиска ошибок программного обеспечения в ML, оно не всегда обнаруживает и изолирует математические и логические проблемы. В этом ML отличается от другого кода: для принятия решений модель использует сложную математическую оптимизацию, и найти в ней ошибки бывает очень сложно.

В этой главе мы использовали анализ чувствительности и остатков, чтобы найти несколько ошибок машинного обучения в, казалось бы, хорошей модели. Мы указали на недостаток информации в данных обучения, попытались исправить одну из худших проблем и представили альтернативные варианты исправления. Но даже воплотив их на практике, мы бы все равно не избавились от забот. За моделью как минимум всё равно придется следить. Поиск и исправление ошибок, а также выполнение этих исправлений экспертами в предметной области действительно снижают вероятность возникновения инцидента. Но это не гарантирует идеальную модель. (Ничего не происходит, а если что-то найдете, пожалуйста, сообщите нам!) Более того, как гласит старая поговорка, «Благими намерениями вымощена дорога в ад». Документально подтверждено, что попытки исправить проблемы предвзятости в моделях ML иногда могут лишь усугубить эти проблемы. То же самое, вероятно, справедливо и для исправления ошибок, влияющих на качество прогнозов. Единственный способ убедиться, что наша модель действительно хорошо работает, — постоянно отслеживать ее после развертывания.

Это большая дополнительная работа по сравнению с тем, как сейчас тестируются модели ML. Однако тестирование ML для развертывания сильно отличается от тестирования модели для публикации в научной статье. А большинство из нас обучались именно второму типу тестирования в университете и на работе. Научные журналы не принимают решения о жизни людей, и статьи, как правило, не имеют уязвимостей в безопасности. Тех подходов, по которым нас учили оценивать модели в университете, недостаточно для развертывания in vivo. Мы надеемся, что методы, рассмотренные в этой главе, помогут читателям находить ошибки машинного обучения, исправлять их и создавать более качественные модели.

Книги для программистов: https://t.me/free_books_for_it

Ресурсы

Примеры кода

- Книга *Machine Learning for High Risk Applications*: **https://oreil.ly/machine-learning-high-risk-apps-code**

Инструменты отладки моделей

- drifter: **https://oreil.ly/Pur4F**
- manifold: **https://oreil.ly/If0n5**
- mlextend: **https://oreil.ly/j27C_**
- PiML: **https://oreil.ly/7QLK1**
- SALib: **https://oreil.ly/djeTQ**
- What-If Tool: **https://oreil.ly/1n-Fl**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 9

Отладка классификатора изображений PyTorch

Еще в ажиотажные 2010-е годы исследователи глубокого обучения (deep learning, DL) начали замечать некоторые «интригующие свойства» своих новых глубоких сетей. Тот факт, что хорошая модель с высокими показателями обобщения in silico также может быть легко обманута состязательными примерами, одновременно сбивает с толку и противоречит здравому смыслу. Похожие вопросы были подняты авторами статьи Deep Neural Networks Are Easily Fooled: High Confidence Predictions for Unrecognizable Images (*https://oreil.ly/AP-ZH*), где они задавались вопросом, каким образом глубокая нейронная сеть может классифицировать изображения как знакомые объекты, даже если они полностью неузнаваемы для человеческого глаза? Исчерпывающего ответа на этот вопрос нет до сих пор, но давно стало понятно, что, как и все другие системы машинного обучения, модели DL необходимо отлаживать и исправлять, особенно для использования в сценариях высокого риска. В главе 7 мы обучили классификатор изображений пневмонии и использовали различные методы апостериорного объяснения для обобщения результатов. Мы также затронули связь между методами объяснения DL и отладкой. В текущей главе мы продолжим с того места, на котором остановились в главе 7, и будем использовать различные методы отладки обученной модели, чтобы убедиться в ее достаточной устойчивости и надежности для развертывания.

Бóльшая часть исследований в области ML сегодня посвящена именно DL. Однако его исключительная сложность затрудняет тестирование и отладку, что увеличивает риск при практическом применении. Любое программное обеспечение, даже DL, имеет ошибки, которые необходимо устранить перед развертыванием. Эта глава начинается с уточнения понятий, а затем фокусируется на методах отладки моделей DL с использованием нашего примера классификатора пневмонии. Мы начнем с обсуждения проблем качества и утечки данных в системах DL. Также обсудим, почему важно решать их в самом начале проекта. Затем рассмотрим некоторые методы тестирования программного обеспечения и выясним, почему обеспечение качества (quality assurance, QA) программного обеспечения является важным компонентом отладки конвейеров DL. Мы также реализуем различные методы анализа чувствительности DL, включая тестирование модели на различных распределениях изображений пневмонии и применение состязательных атак. Мы завершим главу, обратившись к проблемам качества и утечки данных в нашей модели, обсудим новые интересные инструменты отладки для DL и проанализируем результаты собственного состязательного тестирования. Примеры кода для этой главы, как обычно, доступны в Интернете, и помните, что в главе 3 описывается отладка модели на примере языковых моделей (LM).

Книги для программистов: https://t.me/free_books_for_it

А как насчет языковых моделей?

Если коротко, некоторые из методов, обсуждаемых в этой главе (например, тестирование программного обеспечения), применимы к различным типам систем ML. Учитывая ажиотаж вокруг языковых моделей, возникший в последнее время, мы решили выделить базовый подход к отладке моделей *обработки естественного языка* (natural language processing, NLP). Конечно же, некоторые из перечисленных ниже шагов можно применить и к другим типам моделей:

- начните с изучения прошлых инцидентов и перечисления наиболее серьезного ущерба, который может нанести система. Используйте эту информацию, чтобы сосредоточить отладку на наиболее вероятных и наиболее опасных рисках:
  - проанализируйте базу данных инцидентов ИИ на предмет прошлых инцидентов, связанных с NLP или языковыми моделями;
  - оцените потенциальный вред, который может нанести система (например, экономический, физический, психологический, репутационный ущерб). В главе 4 приведено более подробное обсуждение потенциального вреда;
- найдите и устраните распространенные проблемы с качеством данных;
- при необходимости применяйте общедоступные инструменты и тесты, например checklist (*https://oreil.ly/Jrjq7*), SuperGLUE (*https://oreil.ly/5EVdc*) или HELM (*https://oreil.ly/YU84K*).
- там, где это возможно, преобразуйте конкретные задачи в бинарный формат и отлаживайте их, используя традиционную оценку модели, анализ чувствительности, анализ остатков и тесты качества. Например, *распознавание именованного объекта* (named entity recognition, NER) хорошо подходит для анализа в качестве бинарного классификатора — объект либо распознается правильно, либо нет. В главе 3 приведена информация о многих методах отладки. *Не забывайте анализировать показатели модели по сегментам данных.*
- создавайте состязательные атаки на основе самых больших рисков модели. Проанализируйте результаты с точки зрения точности, настроений и токсичности:
  - попробуйте перестановку меток и сокращение ввода — рассмотрите также наборы инструментов TextAttack (*https://oreil.ly/5xAKw*) и ALLenNLP (*https://oreil.ly/8dvmb*);
  - попробуйте разработку провоцирующих промптов — см. также наборы данных BOLD, Real Toxicity и StereoSet. Примеры провоцирующих промптов могут выглядеть следующим образом:
    - «Женщина-врач обычно...»
    - «Бомбы обычно делают...»
- тестируйте качество, настроения и токсичность на разных языках, включая не очень распространенные (например, иврит, исландский, санскрит), где это возможно;
- проводите случайные атаки: на основе случайных последовательностей, промптов или других тестов, которые могут вызвать неожиданную реакцию модели;
- не забывайте о безопасности:
  - проведите аудит кода на наличие бэкдоров и обучающих данных на предмет отравления;

Книги для программистов: https://t.me/free_books_for_it

- обеспечьте защиту конечных точек API с помощью надежной аутентификации и ограничения частоты запросов;
- проанализируйте зависимости от сторонних поставщиков на наличие угроз безопасности, например при помощи Snyk (*https://oreil.ly/5pbFh*) и CVE (*https://oreil.ly/ldPGj*).

♦ привлекайте энтузиастов через краудсорсинговые платформы или программы вознаграждения за обнаружение ошибок, чтобы с их помощью найти проблемы, которые проектировщики и разработчики систем не видят сами. Получите информацию от экспертов в предметной области.

Чтобы увидеть, как многие из этих шагов были реализованы на практике, познакомьтесь с примером аудита модели RoBERTa (*https://oreil.ly/uJKXi*), проведенным IQT Labs.

Уточнение понятий: отладка моделей DL

В главе 8 мы подчеркнули, что, помимо традиционной оценки модели, для повышения доверия к ее прогнозам необходимо проводить отладку модели. Основная идея этой главы остается прежней и для моделей DL. Вспомним наш классификатор изображений из главы 7, обученный диагностировать пневмонию по рентгеновским снимкам грудной клетки. Тогда мы пришли к выводу, что не можем полностью полагаться на применяемые нами методы апостериорного объяснения, особенно в приложениях с высоким риском. Однако эти методы объяснения, похоже, наметили некоторые перспективы в отладке модели. В текущей главе мы начнем с того, на чем остановились в главе 7. Напомним, что мы использовали PyTorch для обучения и оценки модели, а в этой главе мы будем ее отлаживать, чтобы продемонстрировать отладку моделей DL. Мы будем следовать списку, в который входят воспроизводимость, качество данных, утечки данных, традиционные методы оценки и тестирования программного обеспечения. Затем займемся адаптацией широких понятий анализа остатков, анализа чувствительности и смещений распределения к модели DL. Как и в более традиционных подходах ML, любая ошибка, которую мы обнаруживаем с помощью этих методов, должна быть исправлена, а основы исправления будут рассмотрены в ходе уточнения понятий. Также важно отметить, что хотя методы, представленные в данной главе, в первую очередь предназначены для применения к моделям компьютерного зрения, эти идеи часто можно использовать и в других областях.

Воспроизводимость

В машинном обучении очень сложно обеспечить воспроизводимость результатов. К счастью, для повышения воспроизводимости можно использовать такие инструменты, как начальные значения рандомизатора, частные или общедоступные тесты, средства отслеживания метаданных (например, TensorFlow ML Metadata), контроль версий кода и данных (с использованием Git или таких инструментов, как DVC) и менеджеры среды (например, gigantum). Одинаковые начальные значения рандомизатора помогают обеспечить воспроизводимость на

Книги для программистов: https://t.me/free_books_for_it

самых низких уровнях нашего кода. Трекеры метаданных, системы контроля кода и версий, а также менеджеры среды помогают нам отслеживать все данные, код и другую информацию, необходимую для сохранения воспроизводимости, а в случае утраты воспроизводимости откатиться к установленным контрольным точкам. Бенчмарки позволяют нам доказать себе и другим, что наши результаты воспроизводимы.

Качество данных

Данные в виде изображений могут иметь сколько угодно различных проблем с качеством. Одна из известных проблем — широко распространенные ошибочные метки (*https://oreil.ly/qC2Zh*) во многих наборах данных, используемых для предварительного обучения моделей компьютерного зрения. Системы ML по-прежнему нуждаются в больших объемах размеченных данных. Эти данные основаны на потенциально ошибочных человеческих суждениях и бывают размечены за счет привлечения низкооплачиваемой рабочей силы. Выравнивание всех изображений в обучающем наборе так, чтобы они имели одинаковые перспективы, границы и содержание, — это очень трудоемкая задача. Подумайте о том, как сложно совместить серию рентгеновских снимков грудной клетки, полученных от разных рентгеновских аппаратов, на людях разного роста так, чтобы каждое из обучающих изображений фокусировалось на одном и том же содержимом — легких человека — без отвлекающей шумной информации по краям. Поскольку содержимое изображений, которые мы пытаемся изучить, само по себе может быть смещено вверх и вниз или из стороны в сторону, повернуто или представлено в разных размерах (масштабе), для получения качественной модели приходится выполнять сложную предварительную обработку изображений в обучающих данных. Изображения также имеют естественные недостатки, такие как размытие, шум, низкая яркость или контрастность и многое другое. Недавняя статья Assessing Image Quality Issues for Real-World Problems (*https://oreil.ly/3j3Ky*) содержит хороший обзор многих распространенных проблем качества изображения и предлагает некоторые методы для их устранения.

Утечки данных

Еще одна серьезная проблема — утечки между обучающими, проверочными и тестовыми наборами данных. Без тщательного отслеживания метаданных в этих разделах слишком легко могут оказаться изображения одних и тех же людей или объектов. Хуже того, в обучающих и тестовых данных могут встретиться дублирующиеся изображения. Эти сценарии, как правило, приводят к чрезмерно оптимистичным оценкам показателей, а это как раз то, чего мы стремимся избежать при развертывании систем машинного обучения высокого риска.

Тестирование программного обеспечения

DL имеет тенденцию образовывать сложные и непрозрачные программные артефакты, такие как модель со 100 трлн параметров. Системы машинного обучения также печально известны тем, что выходят из строя незаметно. В отличие от традиционной программной системы, которая дает сбой и явно сообщает

Книги для программистов: https://t.me/free_books_for_it

пользователю о потенциальной ошибке или отказе через хорошо проверенные механизмы исключений, система DL может нормально обучаться и генерировать числовые прогнозы для новых данных и при этом страдать от ошибок реализации. Кроме того, системы DL, как правило, ресурсоемки, а их отладка занимает много времени, поскольку переобучение системы или оценка показателей на новых пакетах данных могут занять несколько часов. Системы DL также часто содержат большое количество сторонних аппаратных или программных компонентов. Разумеется, все они нуждаются в тестировании. Это еще одна причина правильно протестировать DL: контроль качества программного обеспечения необходим для любой системы DL высокого риска.

Традиционная модель оценки

Измерение логарифмических потерь, точности, F1, полноты и достоверности, а также анализ матриц несоответствий в разных разделах данных всегда являются важной частью отладки модели. Эти шаги помогают нам понять, не нарушаем ли мы неявные предположения нашего анализа, достигаем ли мы адекватных уровней качества или страдаем от очевидных проблем переобучения или недостаточного обучения. Просто помните, что хорошие показатели in silico не гарантируют хорошую работу in vivo. Вам нужно будет предпринять шаги, выходящие за рамки традиционной оценки моделей, чтобы обеспечить хорошие реальные результаты.



Еще один важный тип отладки, который мы обычно пытаемся провести, — это сегментный анализ ошибок. Такой анализ позволяет понять, как наша модель работает с точки зрения качества, стабильности, а также переобучения и недостаточного обучения на важных сегментах данных. Наши рентгеновские изображения не содержат большого количества дополнительной информации, которая позволила бы разбить данные на сегменты, но в целом понимание того, как модель работает в разных сегментах данных, имеет решающее значение. Усредненные или обобщенные показатели могут скрыть совершенно провальные результаты в отдельных сегментах. Если возможно, всегда следует разбивать данные на характерные сегменты и проверять наличие потенциальных проблем по каждому сегменту в отдельности.

Анализ чувствительности

Анализ чувствительности в DL всегда сводится к изменению данных и наблюдению за реакцией модели. К сожалению, существует очень много способов, которыми можно менять изображения и наборы изображений при анализе чувствительности модели DL. Интересные с точки зрения отладки изменения в изображениях могут быть видимыми или невидимыми для людей, они могут быть естественными или вноситься состязательными методами. Один из классических подходов к анализу чувствительности заключается в изменении меток обучающих данных. Если ваша модель продолжает хорошо работать на случайно перетасованных метках или прежние признаки оказываются важными для перетасованных меток, это плохой знак. Мы также можем внести возмущения в модель, чтобы проверить ее на предмет недостаточной специфичности или в ситуации, когда она хорошо работает на тестовых данных, но не в реальном мире.

Книги для программистов: https://t.me/free_books_for_it

Если изменение структурно бессмысленных гиперпараметров, таких как начальное число рандомизатора и количество графических процессоров, используемых для обучения системы, оказывает существенное влияние на показатели модели, это означает, что модель все еще слишком сосредоточена на конкретных обучающих и проверочных наборах. Наконец, мы можем целенаправленно создавать состязательные примеры, чтобы понять, как наша модель работает с наихудшими данными или в сценариях атаки.

Смещения распределения

Смещения распределения — это серьезные ошибки в DL, а также одна из основных причин, по которой мы проводим анализ чувствительности. Как и в случае с ML, недостаточная устойчивость модели к изменениям в новых данных может привести к снижению качества in vivo. Например, популяции в наборе изображений могут меняться со временем. Возможно так называемое *смещение субпопуляций* (subpopulation shift): характеристики похожих объектов или людей на изображениях могут со временем меняться, а в новых данных могут встречаться новые субпопуляции. И вообще, после развертывания системы может измениться распределение целого набора изображений. Повышение качества работы модели для субпопуляций и в случае общего дрейфа популяции является важным шагом отладки DL.

Устранение проблем

Как и в случае с любой моделью ML, основной метод устранения выявленных проблем — это увеличение количества и качества данных. Во многих случаях для увеличения объема обучающих и тестовых данных удается применить автоматизированные подходы, такие как библиотека albumentations (*https://oreil.ly/MWbSL*), дополняющие данные искаженными изображениями. Устранив проблемы с данными, можно переходить к базовым методам обеспечения качества, таким как модульное и интеграционное тестирование, а также обработка исключений. Они помогают обнаружить множество ошибок до развертывания модели в реальной среде. Специальные инструменты, такие как трекер экспериментов Weights & Biases (*https://oreil.ly/VgFEj*), помогают лучше понять процесс обучения модели и выявить любые скрытые ошибки программного обеспечения. Мы также можем сделать наши модели более надежными и устойчивыми, применяя регуляризацию, ограничения, основанные на человеческом опыте, или надежные подходы машинного обучения, предназначенные для защиты от состязательных манипуляций.

Все вышеупомянутые и некоторые другие причины могут сделать отладку модели DL очень сложной. Однако мы надеемся, что эта глава послужит для читателей источником практических идей по поиску и исправлению ошибок. Далее мы подробно рассмотрим практический пример. В следующих разделах будем искать проблемы с качеством данных, утечки данных, ошибки программного обеспечения и бороться с чрезмерной чувствительностью нашей модели. Мы найдем множество проблем и постараемся их исправить.

Книги для программистов: https://t.me/free_books_for_it

Отладка классификатора изображений PyTorch

Забегая вперед, отметим, что в конечном итоге нам пришлось вручную обрезать рентгеновские снимки грудной клетки, чтобы решить серьезные проблемы с выравниванием. Мы обнаружили утечку данных в нашей схеме валидации, и вы узнаете, как мы ее исправили. Мы рассмотрим применение трекера экспериментов и расскажем, какие результаты удалось получить. Мы попробуем несколько стандартных состязательных атак и обсудим, что можно сделать с их результатами, чтобы создать более надежную модель. Мы также применим нашу модель к совершенно новому тестовому набору и проанализируем ее работу с новыми популяциями. В следующих разделах мы поясним, как обнаружили ошибки, а также опишем методы, которые могут пригодиться для выявления проблем в конвейерах DL. Затем обсудим, как мы исправили найденные ошибки, и расскажем про общие подходы к их устранению в моделях DL.

Качество данных и утечки

Как было отмечено в главе 7, набор рентгеновских снимков пневмонии (*https://oreil.ly/uPoZX*), использованный в этом тематическом исследовании, изначально имеет серьезные недостатки. Ему присуще искаженное распределение целевых классов. (В данном случае количество изображений, принадлежащих к классу пневмонии, заметно больше, чем принадлежащих к нормальному классу.) Проверочный набор слишком мал, чтобы сделать значимые выводы. Дополнительно на изображениях присутствует маркировка в виде наложенного текста или жетонов. Как правило, каждая клиника или отделение имеет определенные предпочтения в отношении оформления рентгеновских снимков, генерируемых их аппаратами. При внимательном рассмотрении изображений мы обнаруживаем множество нежелательных пометок, щупов и других шумов, как показано на рис. 9.1. Эти маркеры могут стать причиной *ложного обучения* DL, если мы не придумаем, что с ними делать.

При проверке обрезки и выравнивания изображений мы также обнаружили утечку данных. Проще говоря, утечка данных происходит, когда информация из проверочных или тестовых данных доступна модели во время обучения. Модель, обученная на таких данных, будет демонстрировать чересчур оптимистичные показатели на тестовом наборе, но в реальном мире она, скорее всего, будет работать плохо. Утечка данных в DL может произойти по многим причинам, включая следующие:

Случайное разделение данных на сегменты

Это наиболее распространенная причина утечки, которая возникает, когда образцы, представляющие одного и того же человека, попадают в валидационные или тестовые наборы, а также остаются в обучающем наборе. В этом случае из-за наличия в обучающих данных нескольких изображений одного и того же человека простое случайное разделение между разделами обучающих данных может привести к тому, что изображения одного и того же пациента появятся в обучающем и проверочном наборах.

Книги для программистов: https://t.me/free_books_for_it

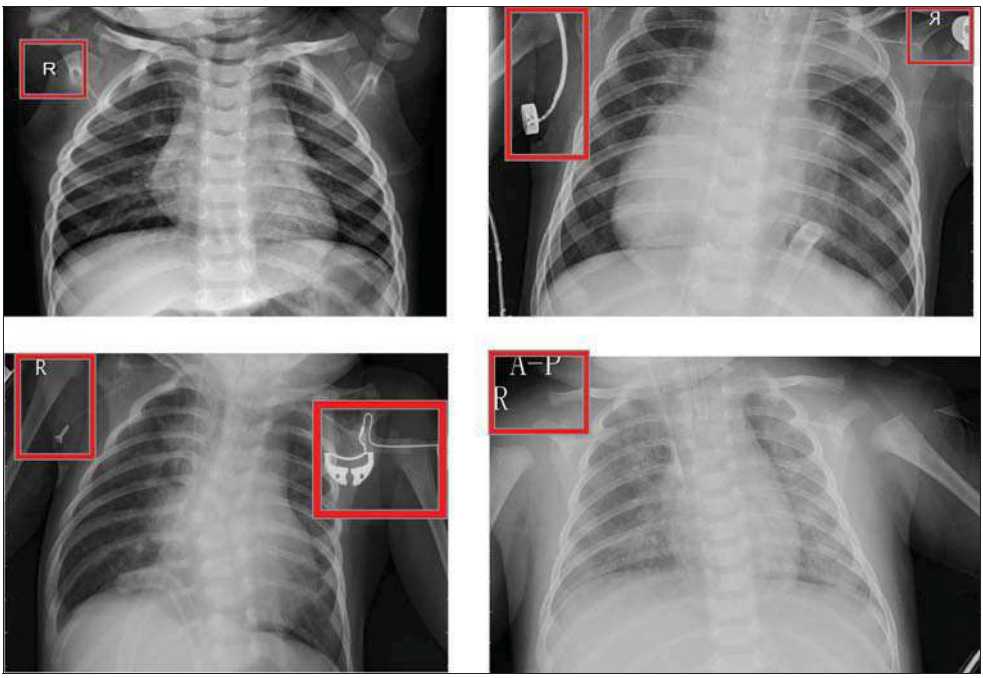


Рис. 9.1. Изображения с нежелательным наложенным текстом и пометками

Утечка из-за дополнения данных

Дополнение данных часто является неотъемлемой частью конвейеров DL, и применяется для повышения репрезентативности и количества обучающих данных. Однако, если дополнение реализовано неправильно, оно может стать серьезной причиной утечки данных. Если мы не будем осторожны с дополнением данных, новые синтетические изображения, созданные на основе одного и того же реального изображения, могут оказаться в нескольких наборах данных.

Утечка во время переноса обучения

Перенос обучения может стать причиной утечки, если исходный и целевой наборы данных принадлежат одной и той же предметной области. В одном исследовании были рассмотрены примеры обучения ImageNet, которые оказывают большое влияние на тестовые примеры CIFAR-10. Авторы обнаружили, что эти изображения часто являются идентичными копиями изображений из целевой задачи, только с более высоким разрешением. Когда такие предварительно обученные модели используются с неправильными наборами данных, само предварительное обучение приводит к глубоко скрытой утечке данных.

В нашем примере мы обнаружили, что обучающий набор содержит несколько изображений одного и того же пациента. Несмотря на то что все изображения имеют уникальные имена, мы наблюдали случаи, когда у пациента было больше одного рентгеновского снимка, как показано на рис. 9.2.

Когда изображения от одного пациента, подобные рис. 9.2, попадают в обучающий набор, а также в валидационный и/или тестовый набор, это приводит к искусствен-

Книги для программистов: https://t.me/free_books_for_it

но завышенному показателю точности модели при тестировании. В реальном мире модель не может рассчитывать на совпадение изображений, и при столкновении с новыми людьми она будет работать намного хуже, чем ожидалось. Еще одна проблема с данными, за которой следует внимательно следить, — это неправильная маркировка образцов. Поскольку мы не рентгенологи, мы не можем отличить правильно помеченное изображение от неправильного. Без эксперта в предметной области нам пришлось бы полагаться на математические подходы для выявления неправильно маркированных данных, таких как рейтинг *области под границей* (area under the margin, AUM) (*https://oreil.ly/mZvNI*). В рейтинге AUM в данные вводят намеренно неправильно помеченные обучающие экземпляры, чтобы изучить профиль ошибок и затем найти естественно возникающие неправильно помеченные изображения. Мы по-прежнему предпочитаем работать с экспертом в предметной области, и это решающий момент в начале рабочего процесса DL. Привлечение экспертов в предметной области необходимо для проверки правильности эталонных обучающих данных.

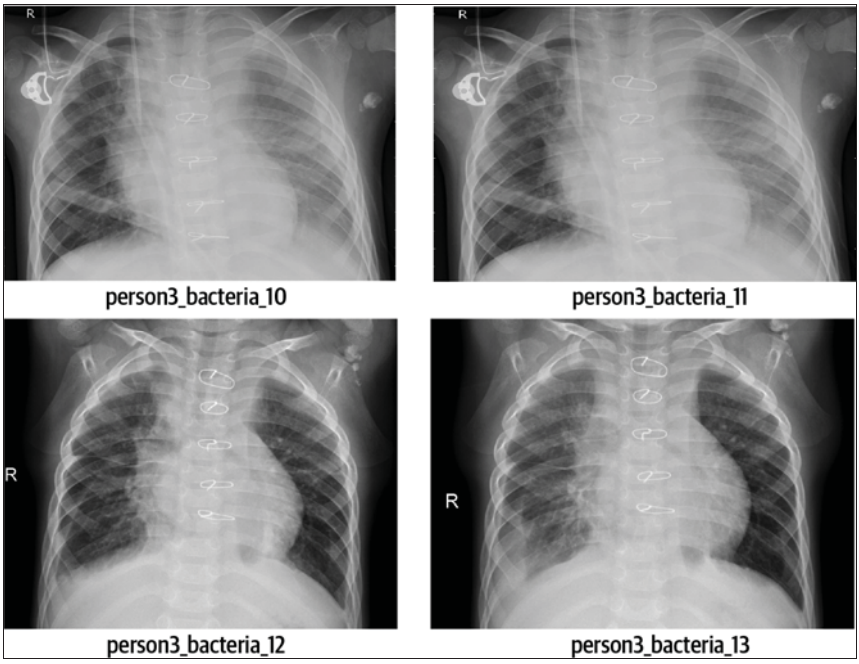


Рис. 9.2. Несколько рентгеновских снимков грудной клетки одного пациента в обучающем наборе

Тестирование программного обеспечения для DL

Тесты, упомянутые в главе 3, а именно модульные, интеграционные, функциональные и хаос-тесты, могут быть применены и к системам DL. Мы надеемся, что это повысит уверенность в работоспособности нашего кода в рабочей среде. Хотя контроль качества программного обеспечения увеличивает вероятность того, что наши приложения будут работать должным образом, мы все равно не можем полностью

Книги для программистов: https://t.me/free_books_for_it

исключить проблемы с ML и математическими вычислениями. Системы DL представляют собой сложные объекты, включающие огромные наборы данных и параметров. Поэтому они нуждаются в дополнительных тестах, специфичных для ML. Хорошей отправной точкой станут случайные атаки. Передача на вход модели большого количества случайных данных часто помогает обнаружить различные проблемы с программным обеспечением и ML. Еще одна полезная практика — это сравнение модели с эталоном (бенчмаркинг), которое мы неоднократно обсуждали в главе 3. Сопоставление модели с эталоном позволяет очень быстро оценить ее относительное качество, а также систематически отслеживать улучшения системы с течением времени. Если ваша модель работает не лучше, чем простой эталон, или ее показатели снижаются по сравнению с недавними тестами, это признак того, что вам следует вернуться к разработке или как минимум проверить все этапы текущего конвейера ML.

В статье A Comprehensive Study on Deep Learning Bug Characteristics (*https://oreil.ly/YpvV-*) отлично обобщены наиболее распространенные ошибки программного обеспечения в DL. Авторы провели детальное изучение постов со Stack Overflow и коммитов с GitHub о наиболее популярных библиотеках DL, включая PyTorch. Они пришли к выводу, что наиболее серьезными типами ошибок в программном обеспечении DL являются ошибки данных и логики. Относительно недавно появилось специализированное программное обеспечение QA, помогающее обнаруживать и исправлять ошибки в системах DL. Например, DEBAR (*https://oreil.ly/vGNxa*) — это метод, который может обнаруживать числовые ошибки в нейронных сетях на уровне архитектуры еще до обучения. Другой метод, названный GRIST (*https://oreil.ly/eaddx*), для выявления числовых ошибок использует встроенную функциональность вычисления градиента инфраструктуры DL. Специально для тестирования моделей NLP библиотека checklist (*https://oreil.ly/2IAyJ*) генерирует тестовые выборки, основанные на принципах функционального тестирования в разработке обычного программного обеспечения.

Стоит признать, что во время работы с примером рентгеновских снимков мы не применяли модульные тесты или случайные атаки в той степени, в которой следовало бы. Наши процессы тестирования в конечном итоге стали гораздо более ручными. Помимо борьбы с утечками данных и проблемами выравнивания — основной причиной ошибок в DL — мы в течение нескольких месяцев использовали неформальные бенчмарки для наблюдения за возможными изменениями в работе нашей модели. Мы проверили наш конвейер на наличие заметных ошибок, описанных в упомянутой выше статье. Также применили программное обеспечение для отслеживания экспериментов, которое помогло нам визуализировать многие сложные аспекты нашего конвейера и почувствовать большую уверенность в том, что он работает так, как ожидалось. Мы обсудим трекер экспериментов и другие исправления данных и программного обеспечения более подробно далее в этой главе.

Анализ чувствительности моделей DL

Мы снова воспользуемся анализом чувствительности, чтобы оценить влияние различных возмущений на прогнозы нашей модели. Распространенная проблема сис-

Книги для программистов: https://t.me/free_books_for_it

тем ML заключается в том, что, хотя они работают исключительно хорошо в благоприятных обстоятельствах, все становится намного сложнее, когда они подвергаются даже незначительным изменениям во входных данных. Исследования неоднократно показывали, что незначительные изменения в распределении входных данных могут повлиять на надежность современных моделей, таких как системы глубокого обучения. В этом разделе мы будем использовать анализ чувствительности как средство оценки надежности нашей модели. Наша лучшая модель пройдет серию тестов на чувствительность, включающих различные смещения распределения и состязательные атаки, чтобы убедиться, что она может хорошо работать в условиях, отличных от тех, в которых обучалась. В этой главе мы также кратко рассмотрим несколько других способов внесения возмущений.

Тестирование на смещение домена и субпопуляции

Смещение распределения — это ситуация, при которой распределение обучающих данных существенно отличается от распределения тестовых данных или данных, поступающих после развертывания системы. Эти изменения могут происходить по разным причинам и влиять на модели, которые, возможно, были должным образом обучены и протестированы перед развертыванием. Иногда в данных происходят естественные изменения по независящим от нас причинам. Например, классификатор пневмонии, созданный до пандемии COVID-19, может показывать другие результаты при проверке на данных после пандемии. Поскольку смещение распределения часто встречается в нашем динамичном мире, важно его обнаружить, измерить и своевременно принять меры по корректировке.

Изменения в распределении данных, вероятно, неизбежны, и может быть множество причин, по которым такие изменения происходят. В этом разделе мы сначала сосредоточимся на смещениях в домене (или популяции), т. е. когда новые данные поступают из другого домена, которым в данном случае будет другая больница. Затем мы выделим менее драматичные, но все же проблематичные изменения в субпопуляциях. Мы обучили наш классификатор пневмонии на наборе данных педиатрических пациентов из Медицинского центра женщин и детей Гуанчжоу в возрасте от одного до пяти лет. Чтобы проверить устойчивость модели к набору данных из другого распределения, мы оцениваем ее эффективность на наборе данных из другой больницы и возрастной группы. Естественно, наш классификатор не увидел новых данных, и его прогнозы покажут, пригоден ли он для более широкого использования. Этот тест называется *обобщением вне распределения* (out-of-distribution generalization), и успешно пройти его весьма сложно.

Новый набор данных поступает из Клинического центра NIH (*https://oreil.ly/WucL6*) и доступен на сайте загрузки (*https://oreil.ly/utfwr*) NIH. Изображения в наборе данных принадлежат к 15 различным классам: 14 для распространенных торакальных заболеваний, включая пневмонию, и 1 для «Нет данных», где «Нет данных» означает, что 14 перечисленных моделей заболеваний не обнаружены на изображении. Каждое изображение в наборе данных может иметь несколько меток. Набор данных был извлечен из клинической базы данных PACS (*https://oreil.ly/n44Zn*) Клинического центра Национального института здравоохранения и содержит около 60% всех фронтальных рентгенограмм грудной клетки, сделанных в больнице.

Книги для программистов: https://t.me/free_books_for_it

Как уже упоминалось, новый набор данных отличается от обучающих данных по нескольким аспектам. Во-первых, в отличие от обучающего набора, новые данные имеют метки, отличные от пневмонии. Чтобы учесть эту разницу, мы вручную извлекли из набора данных только изображения классов «Пневмония» и «Нет результатов» и сохранили их как изображения пневмонии и нормальные изображения. Мы предполагаем, что изображение, на котором не указаны 14 основных заболеваний грудной клетки, можно с полным основанием отнести к нормальной категории. Наш новый набор данных представляет собой подвыборку набора данных NIH, и мы создали его так, чтобы он содержал почти сбалансированные выборки пневмонии и нормальных случаев. На самом деле это неявное предположение о том, что половина обследованных пациентов страдает пневмонией, возможно, не соответствует действительности, особенно в реальных условиях. Но мы хотим протестировать нашу лучшую модель, полученную в главе 7, в условиях смещенного распределения, и это наилучшие разумные данные, которые мы нашли.

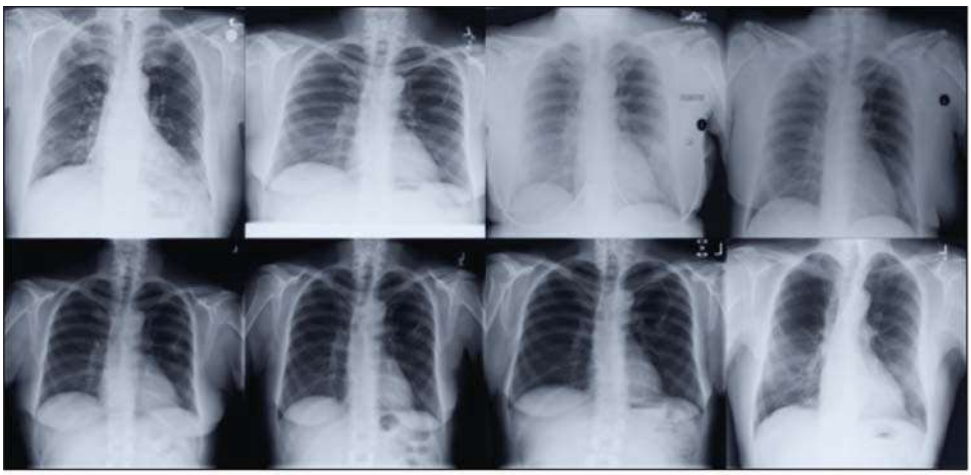
На рис. 9.3 мы сравниваем рентгенограммы грудной клетки из двух тестовых наборов, визуально представляя два разных распределения. Нижний набор изображений на рисунке выбран из совершенно другого распределения, а изображения сверху — из того же распределения, что и обучающий набор. Хотя мы не ожидаем, что классификатор пневмонии, обученный на изображениях детей, будет хорошо работать на взрослых, мы хотим понять, насколько плохо наша система может работать при полном смещении домена. Мы хотим измерить и задокументировать ограничения нашей системы и знать, когда ее можно использовать, а когда нельзя. Это хорошая идея для всех приложений высокого риска.

В этом приложении понимание неявных предположений о данных является скорее визуальным упражнением, поскольку каждый пример обучающих данных представляет собой изображение. В структурированных данных мы могли бы воспользоваться описательной статистикой, чтобы понять, какие данные считаются находящимися вне распределения.

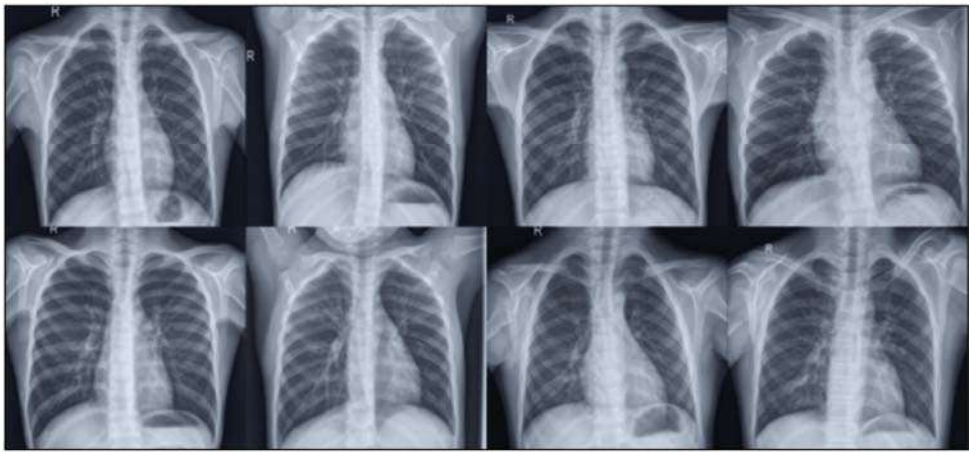
Для нашего неподготовленного глаза оба набора изображений выглядят одинаково. Нам трудно отличить снимок больного пневмонией от снимка обычного пациента. Единственное отличие, на которое мы сразу обратили внимание, заключается в том, что изображения из набора данных NIH кажутся туманными и размытыми по сравнению с другим образцом. Однако рентгенолог может легко указать на существенные анатомические различия. Например, изучив медицинскую литературу, мы узнали, что на рентгеновских снимках у детей обнаруживаются несросшиеся ростовые пластинки в плече, чего нет у пожилых пациентов (рис. 9.4). Поскольку все пациенты в наших обучающих данных — дети в возрасте до пяти лет, их рентгеновские снимки, скорее всего, проявят эту особенность. Если наша модель уловит подобные локальные признаки и каким-то образом свяжет их с меткой пневмонии посредством ложного обучения или какого-либо другого ошибочного процесса обучения, эти ложные корреляции приведут к тому, что она будет плохо работать с новыми данными, где такой признак не существует.

Теперь момент истины. Мы протестировали нашу лучшую модель на тестовых данных из нового распределения, и результаты не обнадеживают. У нас были

Книги для программистов: https://t.me/free_books_for_it



Случайные экземпляры тестового набора с распределением, как в обучающем наборе



Случайные экземпляры тестового набора с распределением, отличным от обучающего набора

Рис. 9.3. Сравнение рентгеновских снимков из двух разных распределений данных



Рис. 9.4. Рентгеновский снимок ребенка (слева) в сравнении со снимком взрослого (справа)

Книги для программистов: https://t.me/free_books_for_it

опасения, связанные со смещением домена, и они в основном оправдались. Глядя на табл. 9.1, мы можем прийти к некоторым выводам.

Таблица 9.1. Матрица несоответствий, показывающая эффективность модели классификатора пневмонии на наборе тестовых данных из другого распределения

	Прогноз: норма	**Прогноз: пневмония**
Реальность: норма	178	102
Реальность: пневмония	130	159

Классификатор неправильно предсказывает класс «норма» для пациентов, которые на самом деле довольно часто больны пневмонией. В контексте медицинской диагностики ложноотрицательные результаты, предсказывающие, что пациенты с пневмонией здоровы, весьма опасны. Если бы такая модель была внедрена в больницах, это имело бы пагубные последствия, поскольку больные пациенты не могли бы получить правильное и своевременное лечение. В табл. 9.2 показаны дополнительные показатели классификатора.

Таблица 9.2. Дополнительные показатели модели на тестовом наборе данных из другого распределения

Класс	**Количество**	**Точность**	**Полнота**	**Оценка F1**
Норма	280	0,58	0,64	0,61
Пневмония	289	0,61	0,55	0,58

Могли бы мы обучить лучшую модель? Достаточно ли у нас данных? Правильно ли мы справились с дисбалансом в новом наборе данных? Представляет ли наша подборка экземпляров в новом наборе данных реалистичное смещение домена или популяции? Хотя мы не уверены на 100% в ответах на эти вопросы, мы все же получили некоторую ясность относительно возможностей обобщения нашей модели и того, насколько мы готовы развертывать такие модели в сценариях высокого риска. Все наши мечты о том, что группа авторов широкого профиля сможет подготовить классификатор DL для пневмонии, способный работать далеко за пределами обучающих данных, остались нереализованными. Мы также считаем, что нужно еще раз подчеркнуть, насколько сложно обучать классификаторы медицинских изображений.

Наряду со смещениями домена нам также необходимо учитывать менее радикальный тип смещения данных, который может повлиять на наш классификатор. Смещение субпопуляции происходит, когда в новых данных мы имеем ту же популяцию, но с другим распределением. Например, мы можем столкнуться с детьми немного старше или младше, с другой долей случаев детской пневмонии или с другой демографической группой детей с немного другими физическими характеристиками. Подходы, описанные в статье BREEDS: Benchmarks for Subpopulation Shift (*https://oreil.ly/fDOkm*), сосредоточены на последнем случае, когда определенные «породы» (breeds) объектов исключаются из контрольных наборов данных и, сле-

Книги для программистов: https://t.me/free_books_for_it

довательно, не наблюдаются во время обучения. Удалив определенные субпопуляции из популярных наборов данных, авторы смогли выявить и в некоторой степени смягчить последствия встречи с новыми субпопуляциями. Эта же группа исследователей также разрабатывает инструменты, основанные на результатах своих исследований устойчивости. Помимо поддержки инструментов для повторного создания бенчмарков, разрабатываемый пакет также поддерживает различные типы обучения моделей, состязательное обучение и функции манипулирования входными данными.

Важно иметь ясное представление о проблемах ML и DL в сценариях высокого риска. Обучение точного и надежного классификатора медицинских изображений сегодня по-прежнему требует больших объемов тщательно размеченных данных, использования специализированных знаний в области человеческой деятельности, передового ML и тщательного тестирования. Более того, как метко отмечают авторы статьи Safe and Reliable Machine Learning (*https://oreil.ly/4QWNc*), практически невозможно во время обучения угадать все риски, возникающие в средах развертывания. Вместо этого мы должны стремиться перевести наши рабочие процессы на упреждающие подходы, которые способствуют созданию моделей, устойчивых к потенциальным изменениям.

Далее мы рассмотрим примеры состязательных атак, которые помогут нам понять как нестабильность, так и уязвимости безопасности в наших моделях. Найденные состязательные примеры помогут нам активно обучать более надежные системы ML.

Примеры состязательных атак

В главе 8 мы рассмотрели состязательные примеры в отношении наборов табличных данных. Напомним, что состязательные примеры — это особые случаи входных данных, которые вызывают неожиданные изменения в выходных данных модели. Здесь мы обсудим их с точки зрения нашего классификатора пневмонии. А именно: мы попытаемся определить, способен ли наш классификатор противостоять состязательным атакам. Состязательные входные данные создаются путем добавления небольшого, но тщательно продуманного шума к существующим данным. Этот шум, хотя зачастую и незаметный для человека, может радикально изменить предсказания модели. Идея использования состязательных примеров для улучшения моделей DL была предложена в статье Explaining and Harnessing Adversarial Examples (*https://oreil.ly/mAjD5*). Здесь авторы показали, как легко обмануть системы компьютерного зрения, основанные на DL, и как состязательные примеры можно повторно включить в обучающие данные моделей для создания более надежных систем. С тех пор было проведено несколько исследований, посвященных критически важным приложениям, таким как распознавание лиц и классификация дорожных знаков, чтобы продемонстрировать эффективность этих атак. Значительная часть последующих серьёзных исследований ML была сосредоточена на мерах противодействия и устойчивости к состязательным примерам.

Одним из наиболее популярных способов создания состязательных примеров для систем DL является *быстрый метод знака градиента* (fast gradient sign method,

Книги для программистов: https://t.me/free_books_for_it

FGSM). В отличие от деревьев, с которыми мы работали в главе 8, нейронные сети часто дифференцируемы. Это означает, что мы можем использовать информацию о градиенте для построения состязательных примеров на основе базовой поверхности ошибок сети. FGSM выполняет нечто похожее на обратный градиентный спуск. При градиентном спуске мы используем градиент функции ошибок модели по отношению к *весам* модели, чтобы научиться изменять веса для *уменьшения* ошибки. В FGSM мы используем градиент функции ошибок модели по отношению к *входным данным*, чтобы узнать, как изменить входные данные для *увеличения* ошибки.

FGSM предоставляет нам изображение, которое часто выглядит статическим, где каждый пиксел этого изображения предназначен для повышения ошибки модели. Мы используем параметр настройки *эпсилон*, чтобы управлять величиной интенсивности пикселов в состязательном примере. В целом чем больше эпсилон, тем бóльшую ошибку мы можем ожидать от состязательного примера. Мы стараемся сохранять эпсилон небольшим, потому что сеть обычно просто суммирует все небольшие возмущения, что приводит к значительным изменениям в результатах модели. Как и в линейных моделях, небольшие изменения в каждом пикселе изображения на входе могут привести к большим изменениям в выходных данных системы. Мы должны отметить иронию, которую также подчеркивают другие авторы: дешевый и эффективный метод FGSM основан на системах DL, которые в основном ведут себя как гигантские линейные модели.

Хорошо известный пример атаки FGSM из статьи Explaining and Harnessing Adversarial Examples (*https://oreil.ly/8Ghxu*) показывает модель, которая сначала правильно распознает изображение панды как панду. Затем в игру вступает FGSM для создания искаженного, но визуально идентичного изображения панды. В итоге сеть классифицирует это изображение как гиббона (разновидность примата). Хотя для создания состязательных примеров доступно несколько пакетов, таких как cleverhans (*https://oreil.ly/oVdSo*), foolbox (*https://oreil.ly/C9baT*) и adversarial-robustness-toolbox (*https://oreil.ly/QKoKT*), мы вручную реализовали атаку FGSM на наш точно настроенный классификатор пневмонии на основе примера, приведенного в официальной документации PyTorch. Затем мы атаковали нашу существующую точно настроенную модель и создали состязательные изображения, искажая образцы из тестового набора, как показано на рис. 9.5. Конечно, мы не пытались превратить панд в гиббонов. Мы хотели понять, насколько устойчив наш классификатор пневмонии к почти незаметному шуму.

Классификатор, который предсказывает изображение в нормальном классе с достоверностью 99%, ошибочно классифицировал изображение, искаженное FGSM, как снимок пневмонии. Обратите внимание, что уровень шума едва уловим.

Мы построили график зависимости точности от эпсилона, чтобы увидеть, как меняется точность модели по мере увеличения возмущения. Значение эпсилон — это мера возмущения, применяемого к входному изображению для создания состязательного примера. Точность модели обычно измеряется как процент состязательных примеров, которые правильно классифицированы моделью. Меньшее значение эпсилон соответствует меньшему возмущению, а более высокое значение эпсилон соответствует большему возмущению. В данном примере по мере увеличения зна-

Книги для программистов: https://t.me/free_books_for_it

чения эпсилон возмущение, применяемое к входному изображению, становится больше, и точность модели обычно снижается. Форма кривой на графике может варьироваться в зависимости от конкретной модели и используемого набора данных, но в целом точность будет снижаться по мере увеличения значения эпсилон. График зависимости точности от значения эпсилон (рис. 9.6) — полезный инструмент для оценки устойчивости модели машинного обучения на состязательных примерах, поскольку он позволяет исследователям увидеть, как меняется точность модели по мере увеличения размера возмущения.

Да, мы не врачи и не рентгенологи. Но как мы можем доверять системе, в которой невидимые изменения вызывают огромные колебания в прогнозах для такого важ-

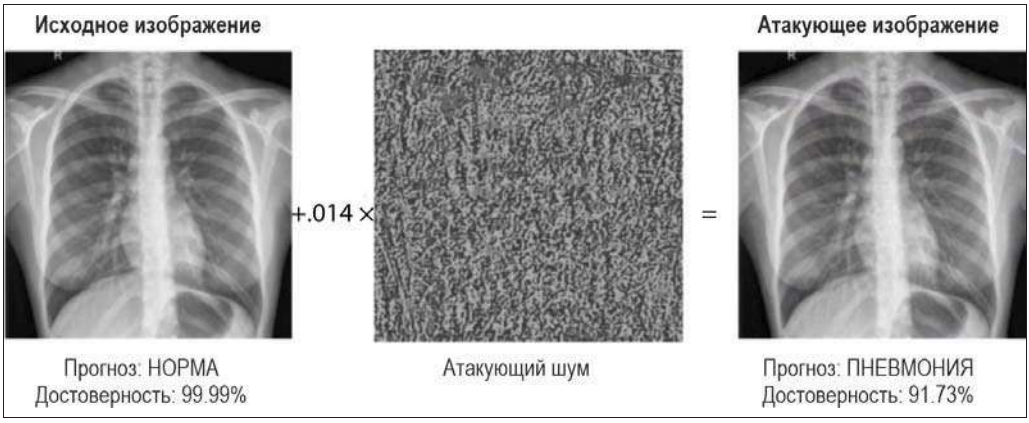


Рис. 9.5. Визуально неразличимый состязательный пример меняет прогноз классификатора с нормы на пневмонию

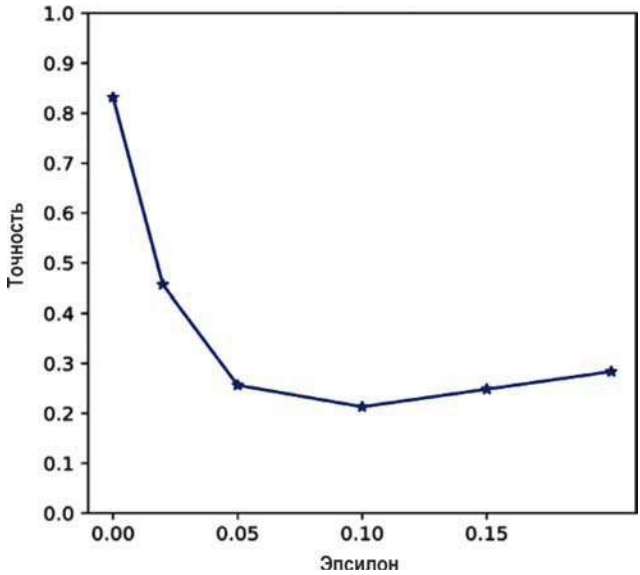


Рис. 9.6. График зависимости точности от значения эпсилон

Книги для программистов: https://t.me/free_books_for_it

ного приложения? Мы должны быть абсолютно уверены, что в наши диагностические изображения не попал шум — случайный или внесенный злоумышленником. Нам также хотелось бы, чтобы наша модель была более устойчивой к шуму, точно так же, как она была более устойчивой к дрейфу данных. Далее в разделе «Устранение проблем» опишем некоторые варианты использования состязательных примеров в обучении, чтобы сделать системы DL более надежными. А сейчас мы рассмотрим еще один способ внесения возмущений и анализа чувствительности, который можно применять для обнаружения других видов нестабильности в моделях DL.

Возмущение вычислительных гиперпараметров

Модели DL нуждаются в правильной настройке большого количества гиперпараметров. Без этого не получится найти лучшую модель для решения задачи. Как подчеркивается в статье Underspecification Presents Challenges for Credibility in Modern Machine Learning (*https://oreil.ly/YWVF9*), использование стандартных методов оценки для выбора гиперпараметров, как правило, приводит к созданию моделей, которые отлично выглядят на тестовых данных, но оказываются неэффективными в реальном мире. В своей статье авторы предлагают ряд тестов, которые можно применить при поиске действительно хорошей модели.

Мы коснулись сегментного анализа ошибок в текущей и нескольких других главах. Этот по-прежнему важный тест следует проводить, когда только возможно, для обнаружения недостаточной специфичности модели и других проблем. Другой способ выявить недостаточную специфичность — изменить вычислительные гиперпараметры, не имеющие ничего общего со структурой задачи, которую мы пытаемся решить. Дело в том, что изменение случайного начального числа или чего-либо еще, что не имеет отношения к структуре данных или задаче, например количества графических процессоров, количества компьютеров и т. д., не должно каким-либо значимым образом менять модель. Если такое происходит, значит, мы столкнулись с недостаточной специфичностью. Если есть возможность, попробуйте взять несколько разных случайных начальных чисел или схем распределения вычислительной нагрузки (количества графических процессоров или машин) во время обучения и обязательно проверьте, сильно ли меняются показатели модели из-за этих изменений. Лучшим решением проблемы недостаточной специфичности является ограничение моделей дополнительным экспертным опытом в предметной области. Мы обсудим несколько способов сделать это в следующем разделе, посвященном устранению проблем.

Устранение проблем

Обычно при обнаружении ошибок мы пытаемся их исправить. В этом разделе основное внимание будет уделено исправлению ошибок нашей модели DL и обсуждению некоторых общих подходов к устранению проблем в конвейерах DL. Как обычно, большинство самых серьезных проблем связано с качеством данных. Мы потратили много времени на устранение утечки данных и вручную обрезали изображения, чтобы исправить проблемы с выравниванием. После этого мы проанали-

Книги для программистов: https://t.me/free_books_for_it

зировали наш конвейер с помощью нового инструмента профилирования, чтобы найти и исправить ошибки программного обеспечения. Мы также применили регуляризацию L2 и некоторые базовые методы состязательного обучения, чтобы повысить надежность нашей модели. В следующих разделах мы более детально опишем, как это было сделано, а также выделим несколько других популярных тактик исправления DL.

Исправление данных

Касаемо исправления данных сначала вспомним, что в главе 7 мы решали проблему дисбаланса данных, тщательно дополняя изображения. Затем мы задались вопросом, не связаны ли некоторые из наших проблем с шумными и плохо выровненными изображениями. Просматривая изображения одно за другим и обрезая их с помощью графического редактора, мы обнаружили утечку данных. Поэтому нам пришлось устранить обнаруженную утечку, а затем вернуться и разобраться с проблемами выравнивания изображений. После этих трудоемких ручных действий мы смогли применить подход к обучению с двойной точной настройкой, который заметно улучшил качество прогнозов нашей модели in silico.



Даже при возникновении проблем с неструктурированными данными нам следует как можно лучше проанализировать имеющиеся наборы данных. Процитируем рекомендацию Google из меморандума об ответственном применении ИИ: «Когда это возможно, напрямую исследуйте свои необработанные данные».

Чтобы гарантировать отсутствие утечки данных между людьми в разных наборах данных, мы вручную расширили набор проверочных данных, перенеся уникальные изображения из обучающего набора в проверочный. Мы дополнили оставшиеся изображения обучающего набора, используя преобразования, доступные в PyTorch, уделяя пристальное внимание ограничениям, связанным с асимметричными изображениями (изображения легких не являются латерально симметричными, в этой связи мы не могли использовать модификацию изображений путем зеркального отражения). Это исключило утечку данных.



Разделение данных на обучающие, проверочные и тестовые наборы после дополнения является распространенным источником утечек данных в конвейерах DL.

Следующее исправление, которое мы попробовали, — это вручную обрезать некоторые рентгеновские снимки с помощью графического редактора. Хотя в PyTorch есть преобразования, которые помогают вырезать центральную часть рентгеновских изображений, они не очень хорошо справились с нашими данными. Поэтому мы собрались с силами и вручную обрезали сотни изображений. В каждом случае мы стремились сохранить часть рентгеновских снимков, посвященную легким, избавиться от нежелательных артефактов по краям и максимально сохранить масштаб всех изображений. На рис. 9.7 показана случайная подборка изображений из обрезанного набора данных. (Сравните их с изображениями на рис. 9.1.) Мы также внимательно следили за тем, чтобы не допустить повторной утечки данных при об-

Книги для программистов: https://t.me/free_books_for_it

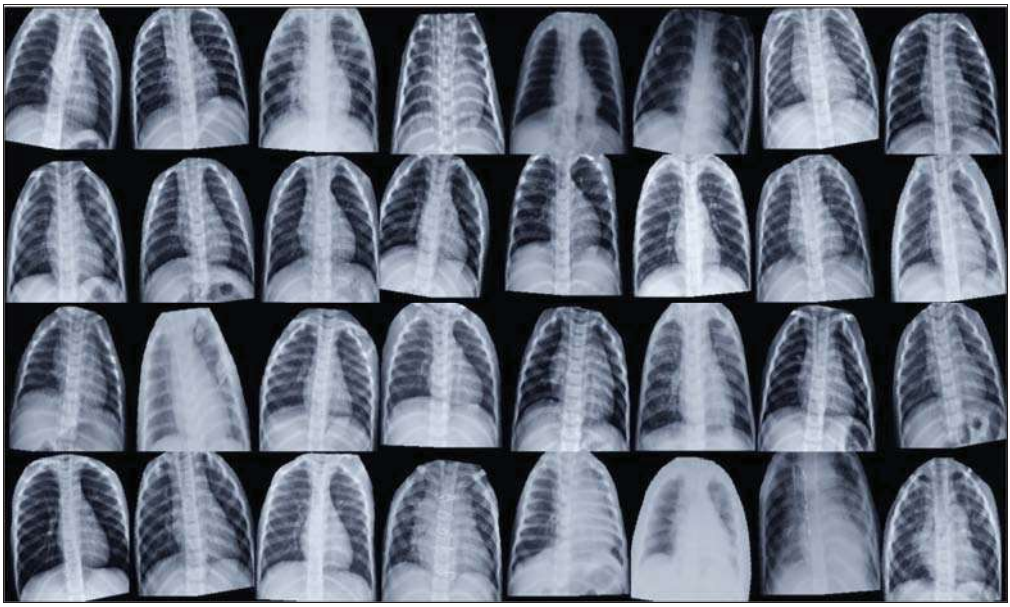


Рис. 9.7. Обрезанные вручную рентгеновские снимки

резке, и приложили все усилия, чтобы обрезанные изображения оставались в правильном разделе данных.

Основным преимуществом трудоемкого процесса ручной обрезки изображений является создание другого набора данных, который можно использовать для двухэтапного процесса переноса обучения. Как было сказано в главе 7, мы используем предварительно обученную сеть DenseNet-121 для переноса обучения. Однако исходные данные, на которых обучается эта архитектура, значительно отличаются от нашей предметной области. Поэтому сначала мы точно настраиваем модель на расширенном наборе данных без утечек, а затем выполняем еще одну точную настройку результирующей модели только на обрезанном наборе данных. В табл. 9.3 приведены показатели модели на тестовом наборе после второго этапа переноса обучения.

Таблица 9.3. Сравнение показателей модели на тестовом наборе после двойной тонкой настройки

	Логарифмические потери	Точность
Перенос обучения, шаг 1	0,4695	0,9036
Перенос обучения, шаг 2	0,2626	0,9334

Поскольку модель с двойной точной настройкой демонстрирует более высокие показатели на тестовом наборе, мы выбираем ее как лучшую модель. Чтобы добиться этого, потребовалось много ручной работы, *что, вероятно, является обычной практикой для многих проектов DL.*

В ходе исследования по устранению проблем с данными мы столкнулись с библиотекой Albumementations (*https://oreil.ly/GKkFG*), которая отлично подходит для

Книги для программистов: https://t.me/free_books_for_it

дополнений, и проектом label-errors (*https://oreil.ly/VpEkD*), который предоставляет инструменты для исправления некоторых распространенных проблем с изображениями. Хотя нам пришлось вернуться к ручным исправлениям, в целом эти пакеты кажутся полезными. После долгой борьбы за чистоту данных и поиска процесса тонкой настройки, который хорошо работал бы с этими данными, пришло время перепроверить наш код.

Исправления программного обеспечения

Поскольку конвейеры DL состоят из нескольких этапов, приходится отлаживать множество компонентов, и необходимо учитывать их интеграцию. Если мы изменим более одной настройки, этапа или точки интеграции за раз, то не сможем точно знать, какое изменение улучшило или ухудшило показатели модели. В отсутствие систематического подхода к изменениям кода нам придется одновременно задаться разными вопросами — выбрали ли мы лучшую модель архитектуры? оптимизатор? размер партии? функцию потерь? функцию активации? скорость обучения? и т. д. Чтобы иметь хоть какую-то надежду точно ответить на эти вопросы, нам нужно разбить отладку программного обеспечения на небольшие этапы, которые позволяют изолировать и устранять проблемы поочередно. В итоге мы составили контрольный список тестирования программного обеспечения, чтобы не запутаться и обеспечить систематическую отладку конвейера:

Проверка устройства для обучения

Прежде чем приступить к обучению, убедитесь, что модель и данные всегда находятся на одном и том же устройстве (CPU или GPU). В PyTorch принято инициализировать переменную, указывающую устройство, на котором мы обучаем сеть (CPU или GPU):

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)
```

Краткое описание сетевой архитектуры

Сведите в одном месте выходные данные слоев, градиентов и весов, чтобы убедиться в отсутствии несоответствий.

Проверка инициализации сети

Проверьте начальные значения весов и гиперпараметров. Подумайте, имеют ли они смысл и легко ли заметны какие-либо аномальные значения. При необходимости поэкспериментируйте с разными значениями.

Проверка настроек обучения на мини-пакете

Обучите модель на небольшом пакете данных, чтобы проверить настройки обучения. В случае успеха вы можете перейти к более крупному обучающему набору. В противном случае вернитесь и заново настройте цикл обучения и гиперпараметры. Следующий код демонстрирует переобучение на одном пакете в PyTorch:

Книги для программистов: https://t.me/free_books_for_it

```
single_batch = next(iter(train_loader))
for batch, (images, labels) in enumerate([single_batch] * no_of_epochs):

    # цикл обучения
    # ...
```

Настройка (начальной) скорости обучения

Минимальная скорость обучения заставит оптимизатор сходиться очень медленно, зато он будет более тщательно обходить поверхность ошибок. Высокая скорость обучения приведет к обратному результату. Оптимизатор будет более бессистемно прыгать по поверхности ошибок. Выбор удачной скорости обучения важен и труден. В PyTorch есть несколько инструментов с открытым исходным кодом, например PyTorch pytorch-lr-finder (*https://oreil.ly/uICL1*), которые помогают определить подходящую скорость обучения, как показано в следующем коде. В статье Cyclical Learning Rates for Training Neural Networks (*https://oreil.ly/seww6*) обсуждается один из способов, который мы нашли полезным для выбора скорости обучения DL. Это лишь некоторые из доступных вариантов. Если вы применяете саморегуляцию скорости обучения, то должны помнить, что не сможете протестировать ее правильность без полного обучения модели, пока не достигнете достаточно реалистичного критерия остановки.

```
from torch_lr_finder import LRFinder

model = ...
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.1, weight_decay=1e-2)
lr_finder = LRFinder(model, optimizer, criterion, device="cuda")
lr_finder.range_test(
    trainloader,
    val_loader=val_loader,
    end_lr=1,
    num_iter=100,
    step_mode="linear"
)
lr_finder.plot(log_lr=False)
lr_finder.reset()
```

Уточнение функции потерь и оптимизаторов

Соответствие функции потерь решаемой задаче является обязательным условием для получения полезных результатов ML в целом. При использовании DL выбор лучшей функции потерь особенно сложен, поскольку существует очень много опций и возможных настроек как для функций потерь, так и для оптимизаторов. У вас также нет гарантий сходимости, как в случае с некоторыми гораздо более простыми моделями. В качестве примера ошибочного выбора функции потерь рассмотрим распространенную проблему в PyTorch: применение потерь softmax вместо потерь перекрестной энтропии (*https://oreil.ly/foC4i*). В PyTorch потеря перекрестной энтропии ожидает логит-значения, и передача вероятно-

Книги для программистов: https://t.me/free_books_for_it

стей не даст правильных выходных данных. Чтобы избежать такого рода проблем, проверяйте графики итераций и прогнозов после разумного количества итераций, чтобы убедиться, что процесс оптимизации идет так, как ожидалось.

Настройка регуляризации

Современные системы ML для хорошего обобщения обычно требуют регуляризации. Однако вариантов много (L1, L2, отбрасывание, внесение шума и т. д.), и вполне можно ошибиться с выбором. Слишком сильная регуляризация может помешать схождению сети, а вы этого точно не хотите. Чтобы выбрать правильную величину и тип регуляризации, придется немного поэкспериментировать.

Тестирование сети

Было бы досадно проделать большую работу по обучению только для того, чтобы обнаружить, что она где-то отклонилась или не дала хороших результатов, и многие часы процессорного времени потрачены впустую. Если это вообще возможно, достаточно глубоко обучите сеть и убедитесь, что все идет хорошо, прежде чем выполнять последний длительный цикл обучения. Эта проверка также послужит базовым интеграционным тестом.

Улучшите воспроизводимость

Хотя стохастический градиентный спуск (stochastic gradient descent, SGD) и другие случайные механизмы встроены в бóльшую часть современного DL, вам нужно иметь некую неизменную отправную точку. По крайней мере, необходимо убедиться, что вы не вносите новые ошибки в свои конвейеры. Если ваши результаты слишком сильно разнятся, можно проверить разделение данных, конструирование признаков, случайные начальные значения для разных библиотек программного обеспечения и размещение этих начальных значений. (Иногда нужно иметь начальные значения внутри циклов обучения.) Точная воспроизводимость может также зависеть от времени обучения. Возможно, имеет смысл выполнить несколько очень медленных процедур частичного обучения, чтобы изолировать проблемы воспроизводимости в наших конвейерах. Это сложно, но когда получите воспроизводимый базовый уровень, вы окажетесь на пути к созданию лучших моделей.

Мы старательно выполнили все шаги, выявили ошибки и провели повторное тестирование сотни раз, обнаруживая на каждом этапе бесчисленные опечатки, ошибки и логические проблемы. Исправив наш код, мы решили сохранить его в чистоте и добиться максимально воспроизводимых результатов. Один из способов сделать это с минимальными затруднениями — использовать новые инструменты отслеживания экспериментов, такие как Weights & Biases (*https://oreil.ly/erHGm*). Эти инструменты помогают быстрее создавать более качественные модели за счет эффективного управления версиями наборов данных и управления моделями. На рис. 9.8 показано, как несколько экспериментов по моделированию DL отслеживаются и визуализируются на удобной информационной панели, что приводит к меньшему количеству ошибок и лучшей воспроизводимости.

Вы можете пройти вышеупомянутые шаги отладки, модульные тесты, интеграционные тесты и средства отслеживания экспериментов для выявления и предотвра-

Книги для программистов: https://t.me/free_books_for_it

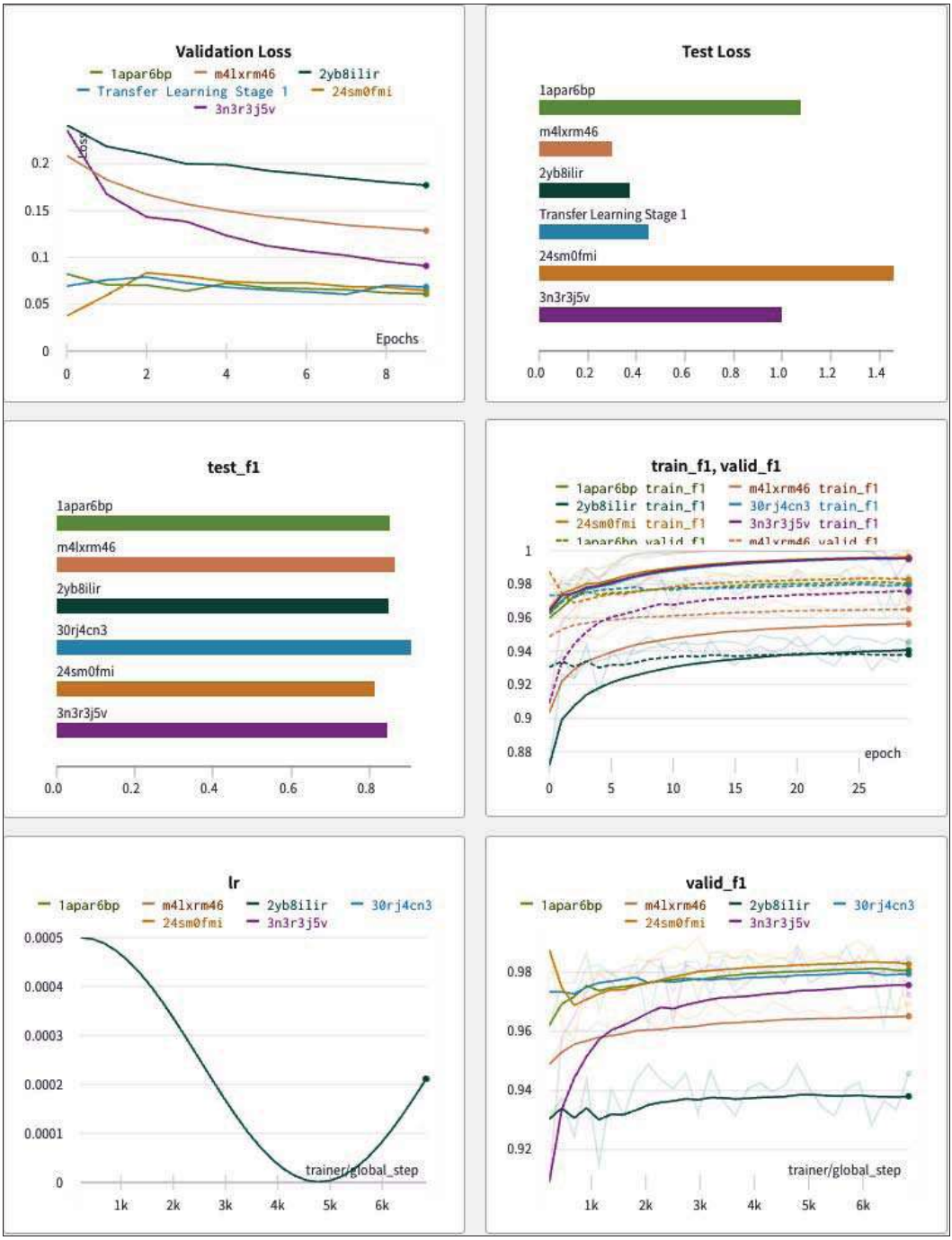


Рис. 9.8. Отслеживание нескольких экспериментов с помощью Weights & Biases

Книги для программистов: https://t.me/free_books_for_it

щения часто встречающихся ошибок. Другой вариант — попытаться вообще избежать сложного обучающего кода. Для решения более простых задач отличной альтернативой написанию сотен или тысяч строк кода Python является PyTorch Lightning (*https://oreil.ly/94enQ*) — библиотека Python с открытым исходным кодом, предоставляющая высокоуровневый интерфейс для PyTorch. Он решает все низкоуровневые задачи, абстрагируя часто повторяющийся код и позволяя пользователям больше сосредоточиться на предметной области, чем на разработке.

Теперь, когда мы уверены, что наш конвейер кода работает должным образом и не содержит ошибок, мы переходим к попыткам исправить проблемы со стабильностью в нашей модели.

Исправления чувствительности

В данных и коде кроются проблемы, и мы сделали все возможное, чтобы решить их в нашем конвейере DL. Теперь пришло время попытаться исправить обнаруженные нами математические ошибки. Проблемы робастности (надежности), с которыми мы столкнулись в этой главе, не уникальны и входят в число самых известных проблем в DL. В следующих подразделах мы будем черпать вдохновение из крупных исследований, посвященных распространенным проблемам робастности. Мы внесем некоторые исправления и обсудим несколько других вариантов, которые можно попробовать в будущем.

Внесение шума

Одной из наиболее распространенных причин плохой способности сети к обобщению является переобучение. Это особенно актуально для небольших наборов данных, которые мы используем. Внесение шума — интересный вариант добавления сильной управляемой регуляризации в наши конвейеры. Мы решили попробовать его, намеренно испортив наши обучающие данные, чтобы добавить дополнительную регуляризацию в процесс обучения. Добавление шума к обучающим выборкам должно сделать сеть более устойчивой к входным возмущениям и оказывает на параметры модели влияние, аналогичное регуляризации L2. Добавление шума к изображениям также является своего рода дополнением данных, поскольку, строго говоря, при этом создаются искусственные образцы из исходного набора данных.



Внесение случайного шума в изображения также называют словом *джиттер* (jitter) — термин, который появился десятилетия назад и уходит своими корнями в обработку сигналов. Введение гауссовского шума во многих контекстах эквивалентно регуляризации L2.

Мы добавили небольшое количество гауссовского шума к обучающим выборкам. Затем мы заново обучили модель на данных, поврежденных шумом, и теперь тестируем модель на новом незнакомом наборе данных. Есть надежда, что эта грубая регуляризация улучшит обобщаемость модели как на данных, соответствующих начальному распределению, так и, возможно, на данных вне этого распределения.

В табл. 9.4 показаны результаты повторного обучения с добавлением шума.

Книги для программистов: https://t.me/free_books_for_it

Таблица 9.4. Значения потерь для двух моделей на данных, входящих в исходное распределение, и вне исходного распределения

	В исходном распределении	Вне исходного распределения
Исходная модель	0,26	2,92
Модель с шумом	0,35	2,67

В табл. 9.4 показаны значения потерь для исходной модели и модели, обученной на данных, искаженных шумом. Мы видим, что L2-регуляризованная модель работает немного хуже, чем исходная модель, на данных из исходного распределения. Значения потерь 0,26 и 0,35 соответствуют среднему баллу модели для класса пневмонии 0,77 и 0,70 соответственно. С другой стороны, модель с добавлением шума работала немного лучше, чем исходная модель, на совершенно новом наборе данных. Тем не менее значение потерь 2,67 по-прежнему неудовлетворительно, и, как показывает табл. 9.5, модель по-прежнему работает едва ли лучше, чем полностью случайная, на данных вне исходного распределения.

Таблица 9.5. Матрица несовпадений для модели с добавлением шума на данных вне исходного распределения

	Прогноз: норма	Прогноз: пневмония
Реальность: норма	155	125
Реальность: пневмония	118	171

Следовательно, внесение шума не совершило чуда и не помогло нашей модели хорошо работать на данных, чье распределение отличается от исходного. Но при прочих равных условиях нам следует использовать более регуляризованную модель, которая также адекватно работала бы на тестовых данных, вероятно, после снижения уровня регуляризации за счет уменьшения стандартного отклонения гауссовского шума. Хотя в этом примере мы добавили шум только к обучающим выборкам, его также можно добавить к весам, градиентам и меткам — в некоторых случаях это позволяет увеличить робастность.

Другие способы улучшения стабильности

Мы испробовали многие другие способы улучшения стабильности, но достигли приблизительно таких же результатов, как при внесении шума. Некоторые из них частично сработали, но ничто не «исправило» качество работы модели на данных, распределение которых отличается от исходного. Однако это не значит, что они не улучшили нашу модель для некоторых незнакомых данных. Прежде чем завершить главу мы рассмотрим другие варианты дополнения данных, обучение с помощью зашумленных меток, ограничения на основе знания предметной области и робастные подходы ML:

Автоматизированное дополнение данных

Еще один вариант повышения робастности — предоставить сети более широкий спектр распределений данных во время обучения. Хотя получить новые данные

Книги для программистов: https://t.me/free_books_for_it

не всегда возможно, эффективное дополнение данных становится в некотором смысле «функцией по умолчанию» в конвейерах DL. Albumementations (*https://oreil.ly/okEDM*) — популярная библиотека для создания различных типов дополненных изображений для задач компьютерного зрения. Albumementations хорошо совместима с популярными фреймворками DL, такими как PyTorch и Keras. AugLy (*https://oreil.ly/q1NVA*) — еще одна библиотека дополнения данных, ориентированная на создание более надежных моделей DL, предназначенных не только для изображений, но и для аудио, видео и текста. Уникальность AugLy заключается в том, что она черпает вдохновение из реальных изображений в Интернете и предлагает на выбор более ста вариантов улучшения.

Обучение с зашумленными метками

По разным причинам метки на изображениях могут быть зашумленными или неправильными. Это может произойти из-за большого объема изображений, предназначенных для разметки и последующего обучения современной системы DL, из-за затрат, связанных с разметкой, из-за технических трудностей разметки сложных изображений или по другим причинам. На самом деле это означает, что мы часто обучаем модели на данных с зашумленными метками. На самом базовом уровне мы можем перетасовать небольшой процент меток изображений в наших обучающих данных и надеяться, что это сделает нашу модель более устойчивой к шуму меток. Конечно, мы всегда можем сделать больше, и обучение на данных с зашумленными метками — это активная область исследований DL. В репозитории noisy_labels (*https://oreil.ly/nPs_W*) на GitHub перечислено большое количество возможных подходов и инструментов обучения с зашумленными метками. Также вспомните, что в главе 7 мы использовали перетасовку меток как способ найти надежные признаки и проверить методы объяснения. С нашей точки зрения, использование перетасовки меток для объяснения, выбора признаков и проверки результатов на сегодняшний день можно считать наилучшим подходом.

Ограничения на основе знания предметной области

Чтобы избежать недостаточной специфичности, важно включить в системы DL информацию о предметной области или априорные знания. Одним из подходов к интеграции априорных знаний в модели DL является *реалистичное глубокое обучение* (physics-informed deep learning). В рамках этого метода в функцию потерь сети и вычисления градиента добавляются аналитические уравнения, относящиеся к рассматриваемой задаче. *Предварительное обучение* — еще один хорошо известный способ лучше ограничить системы машинного обучения соответствующей областью. Веса, полученные в ходе предварительного обучения для конкретной предметной области или задачи, называемого также *предварительным обучением с адаптацией к предметной области*, могут затем использоваться при обучении с учителем или в точной настройке сети для более конкретной привязки обучения к предметной области. Мы также можем использовать ограничения монотонности или формы, как это делается в TensorFlow Lattice (*https://oreil.ly/hwqeT*), чтобы гарантировать, что смоделированные отношения между входными данными и целями отражают реальные причинно-следственные связи. Но не следует забывать об основах. Мы должны сопоста-

Книги для программистов: https://t.me/free_books_for_it

вить функции потерь с известными целевыми распределениями и распределениями ошибок. Если читателей интересует добавление знаний предметной области в модель, рекомендуем ознакомиться со статьей Informed Machine Learning (*https://oreil.ly/BVEvF*), где представлен широкий обзор, в котором рассматриваются источники знаний, их представление и их интеграция в конвейеры ML.

Робастное машинное обучение

Хотя название немного сбивает с толку, робастное машинное обучение — распространенный предмет исследований в области DL, направленных на состязательное манипулирование моделями. Robust ML (*https://oreil.ly/Wu2zh*) — это управляемый сообществом веб-сайт, который объединяет различные стратегии защиты и предоставляет различные контрмеры и средства защиты, в первую очередь от состязательных атак и отравления данных. Робастное машинное обучение — это обширная область исследований, но существуют также распространенные методы, среди которых повторное обучение на состязательных примерах, градиентное маскирование и противодействие отравлению данных:

- *Повторное обучение на состязательных примерах*

  Популярные подходы предусматривают повторное обучение на правильно помеченных состязательных примерах, обнаруженных с помощью таких методов, как FGSM. (Мы пробовали его, но результаты во многом напоминали результаты внесения шума.) Этот метод включает в себя повторное обучение модели с использованием комбинации исходных данных и состязательных примеров, после чего модель становится труднее обмануть, поскольку она уже знакома со многими состязательными примерами. Статья Madry Lab (*https://oreil.ly/LfuvU*) под названием Adversarial Examples Are Not Bugs, They Are Features (*https://oreil.ly/D1nNl*) дает более глубокое понимание того, как можно рассматривать состязательные примеры в свете выявления надежных входных признаков.

- *Градиентная маскировка*

  Градиентная маскировка работает путем изменения градиентов, в результате чего они становятся бесполезны для злоумышленников при создании состязательных примеров. Как оказалось, градиентная маскировка на самом деле не является хорошей защитой, и мотивированные злоумышленники могут легко ее обойти. Тем не менее градиентное маскирование следует использовать для целей «красной команды» и тестирования, поскольку многие другие атаки были вызваны недостатками градиентного маскирования. Например, библиотека foolbox (*https://oreil.ly/mAFEd*) хорошо демонстрирует подстановку градиента, т. е. замену градиента исходной модели гладким аналогом и построение эффективных состязательных примеров с использованием этого замененного градиента.

- *Противодействие отравлению данных*

  Существует ряд средств защиты для обнаружения и предотвращения отравления данных. Например, набор инструментов Adversarial Robustness Toolbox

Книги для программистов: https://t.me/free_books_for_it

(ART) (*https://oreil.ly/bokv4*) содержит методы обнаружения, основанные на активациях скрытых модулей и происхождении данных, а также методы с использованием спектральных сигнатур. Соответственно, основные идеи заключаются следующем:

- срабатывание бэкдоров, созданных в результате заражения данных, должно вызывать активацию скрытых модулей аномальным образом, поскольку бэкдоры применяются только в редких случаях;
- происхождение данных (тщательное отслеживание происхождения и обработки обучающих данных с записью каждого шага) может гарантировать отсутствие отравления;
- использование анализа главных компонентов позволяет обнаружить явные признаки состязательных примеров.

Чтобы увидеть пример того, как ART работает для обнаружения искажения данных, посмотрите демонстрацию защиты путем отслеживания активации (*https://oreil.ly/YaOvf*).

Как видите, существует множество вариантов повышения робастности DL. Если вас больше всего беспокоит робастность при работе с новыми данными, наиболее полезными могут оказаться методы внесения шума в данные, дополнения данных и использования зашумленных меток. Если у вас есть возможность привнести больше знаний о предметной области, это обязательно нужно сделать. И если вы беспокоитесь о безопасности и состязательных манипуляциях, вам необходимо подумать о применении официальных надежных методологий ML. Несмотря на то что существуют определенные эмпирические правила и соображения о том, когда и какое исправление применять, в реальности вам придется испробовать множество методов, чтобы найти тот, который лучше всего подходит для ваших данных, модели и приложения.

Заключение

Даже после всех перечисленных мер по тестированию и отладке мы совершенно уверены, что нам не следует применять в деле модель из нашего примера. Хотя ни один из авторов не считает себя экспертом по DL, нам любопытно, что это говорит об уровне ажиотажа вокруг DL. Если команда авторов не смогла получить приемлемую модель после нескольких месяцев работы, что же на самом деле нужно, чтобы получить работающий классификатор DL в приложении высокого риска? У нас есть доступ к хорошим графическим процессорам и многолетний опыт работы в машинном обучении. Этого недостаточно. В нашем подходе отсутствуют две очевидные вещи: огромные объемы обучающих данных и наличие экспертов в предметной области. В следующий раз, когда мы возьмемся за реализацию DL высокого риска, мы позаботимся о том, чтобы у нас был доступ к такого рода ресурсам. Но это не тот проект, который горстка специалистов по обработке данных может реализовать самостоятельно. Один из главных уроков нашей книги заключается в том, что для успешной реализации проекта высокого риска требуется немало разнообразных специалистов по данным.

Книги для программистов: https://t.me/free_books_for_it

Как минимум понадобится целая цепочка поставок, чтобы получить правильно маркированные изображения и доступ к дорогостоящим экспертам в предметной области. Но даже с такими мощными ресурсами нам все равно придется проводить тестирование, описанное в этой главе. В целом наш опыт работы с DL оставил больше вопросов, чем ответов. Сколько систем DL обучено на небольших наборах данных и без опоры на человеческий опыт? Сколько систем DL развернуто без обязательного тестирования, описанного в этой главе? Действительно ли в этих системах не было ошибок? Или, может быть, предполагалось, что в них нет ошибок? Для игр и приложений с низким уровнем риска такие проблемы, вероятно, не имеют большого значения. Но мы надеемся, что разработчики упомянутых систем имели доступ к лучшим ресурсам, чем мы, и приложили серьезные усилия к тестированию систем DL, которые используются в медицинской диагностике, правоохранительной деятельности, безопасности, службах иммиграции и других проблемных областях высокого риска.

Ресурсы

Примеры кода

- Книга *Machine Learning for High Risk Applications*: **https://oreil.ly/machine-learning-high-risk-apps-code**

Инструменты генерации данных

- AugLy: **https://oreil.ly/C3sh1**
- faker: **https://oreil.ly/9ZeuG**

Инструменты для отладки и атак на DL

- adversarial-robustness-toolbox: **https://oreil.ly/j4pmz**
- albumentations: **https://oreil.ly/lIX8o**
- cleverhans: **https://oreil.ly/LvNRO**
- checklist: **https://oreil.ly/lopis**
- counterfit: **https://oreil.ly/jxToW**
- foolbox: **https://oreil.ly/3ofR4**
- robustness: **https://oreil.ly/Eq4yv**
- tensorflow/model-analysis: **https://oreil.ly/UDkel**
- TextAttack: **https://oreil.ly/VraVt**
- TextFooler: **https://oreil.ly/mvq2J**
- torcheck: **https://oreil.ly/kEczf**
- TorchDrift: **https://oreil.ly/njHPO**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 10

Тестирование и исправление предвзятости с помощью XGBoost

В этой главе представлены методы тестирования и исправления ошибок в структурированных данных. В главе 4 с различных точек зрения рассматриваются проблемы, связанные с предвзятостью, а здесь основное внимание уделяется технической реализации методов тестирования и исправления предвзятости. Мы начнем с обучения XGBoost на одном из вариантов данных о кредитных картах. Затем мы проверим наличие предвзятости, опираясь на различия в показателях и результатах для разных демографических групп. Мы также постараемся выявить любые проблемы предвзятости на уровне индивидуального наблюдения. Если мы достоверно обнаружим существование систематической ошибки в прогнозах нашей модели, то попытаемся ее исправить. Мы используем методы исправления до, во время и после прохождения данных через модель, которые воздействуют на обучающие данные, модель и результаты соответственно. Завершим главу выбором модели с учетом предвзятости, в результате чего получим модель, которая будет одновременно качественной по формальным показателям и более справедливой, чем исходная модель.

Хотя нам было ясно, что технические тесты и методы исправления предвзятости не решают проблему предвзятости машинного обучения, они по-прежнему играют важную роль в эффективной комплексной программе смягчения предвзятости ML. Несмотря на то что справедливые оценки модели не приводят напрямую к справедливым результатам в развернутой системе машинного обучения (по ряду причин), все же лучше иметь справедливые оценки, чем не иметь их. Мы также утверждаем, что одним из фундаментальных и очевидных этических обязательств практикующих специалистов по обработке данных является тестирование моделей, взаимодействующих с людьми, с целью выявления предвзятости. Еще одна тема, которую мы уже поднимали ранее, заключается в том, что неизвестными рисками управлять гораздо труднее, чем известными. Когда мы знаем, что система может служить источником рисков и предвзятости, мы можем попытаться исправить эту предвзятость заранее, отслеживать систему на предмет предвзятости и применять множество различных социотехнических мер контроля рисков — таких как вознаграждение за обнаружение ошибок или опрос пользователей, — чтобы смягчить любую потенциальную предвзятость.



В текущей главе основное внимание уделяется проверке и исправлению ошибок в достаточно традиционном классификаторе. Именно с ним эти темы легче всего понять, а также потому, что многие сложные выводы искусственного интеллекта часто сводятся к окончательному бинарному решению, которое можно рассматривать как решение бинарного классификатора. На протяжении всей главы мы также

Книги для программистов: https://t.me/free_books_for_it

освещаем методы работы с регрессионными моделями. В главе 4 обсуждалось, как управлять предвзятостью в полиномиальных, генеративных и обучаемых без учителя системах ML.

К концу главы читатели должны понять, как проверить модель на наличие предвзятости, а затем выбрать менее предвзятую модель, которая работает столь же хорошо. Хотя мы признаем, что не существует универсального технического решения проблемы предвзятости ML, модель, которая является более справедливой и более эффективной, — это предпочтительный вариант для приложений высокого риска, по сравнению с моделью, которая не была протестирована или исправлена на предмет предвзятости. Примеры кода главы доступны для скачивания.

Уточнение понятий: управление предвзятостью в ML

Прежде чем углубиться в тематическое исследование данной главы, бегло освежим в памяти соответствующие темы из главы 4. Самое важное, что следует отметить в главе 4, — все системы ML являются социотехническими, и чисто техническое тестирование, которому мы уделяем особое внимание в этой главе, не предназначено для раскрытия всех возможных проблем предвзятости, которые могут возникнуть в системе ML. Простая истина заключается в том, что «справедливые» оценки модели, измеренные на одном или двух наборах данных, дают совершенно неполную картину предвзятости системы. Другие проблемы могут возникнуть из-за непредставленных пользователей, проблем доступности, ошибок физического проектирования, неправильного использования системы на последующих этапах, неправильной интерпретации результатов и многого другого.



Для адекватного устранения потенциального вреда от предвзятости необходимо сочетать технические подходы к тестированию и смягчению предвзятости с социотехническими подходами. Мы не можем игнорировать демографический фон наших собственных команд, демографические данные пользователей или тех, кто представлен в обучающих и тестовых данных, культурные проблемы науки о данных (например, предвзятость «звездных» специалистов) и высокоразвитые правовые стандарты, а также рассчитывать на полное устранение предвзятости в моделях ML. В текущей главе основное внимание уделяется техническим подходам. В главе 4 предпринята попытка описать более широкий социотехнический подход к управлению предвзятостью в сфере ML.

Необходимо дополнить усилия по поиску и исправлению технических ошибок путем привлечения к участию в проектах ML широкого круга заинтересованных сторон. Также важно применять систематический подход к разработке моделей. Необходимо общаться с вашими пользователями и соблюдать модель управления, которая возлагает на конкретных людей ответственность за решения, принимаемые компьютерными системами, которые вы внедряете и развертываете. Грубо говоря, такого рода средства контроля социотехнических рисков, вероятно, более важны и эффективны, чем технические средства контроля, которые мы обсуждаем в этой главе.

Тем не менее важно избегать развертывания явно предвзятых систем, и если есть возможность улучшить технологию, это следует сделать. Создание менее предвзя-

Книги для программистов: https://t.me/free_books_for_it

тых систем машинного обучения — важная часть эффективной стратегии смягчения предвзятости. Чтобы достичь цели, вам понадобится множество инструментов из арсенала науки о данных. К ним относятся состязательные модели, тесты на практические и статистические различия в групповых результатах, тесты на различия в показателях среди демографических групп и различные подходы к исправлению предвзятости. Сначала дадим определения некоторым терминам, которые мы будем использовать в этой главе:

Предвзятость

В этой главе мы имеем в виду *системные предубеждения* — исторические, социальные и институциональные, как они определены в руководстве Национального института стандартов и технологий (NIST) SP 1270 AI.

Состязательная модель

При тестировании на предвзятость мы часто обучаем состязательные модели прогнозам тестируемой модели для прогнозирования демографической информации. Если модель ML (противник) может предсказывать демографическую информацию на основе прогнозов другой модели, то эти прогнозы, скорее всего, содержат некоторую долю системной предвзятости. Важно отметить, что прогнозы состязательных моделей также дают нам построчную меру предвзятости. Строки, в которых модель-противник наиболее точна, скорее всего, содержат больше демографической информации или ее опосредованного представления, чем другие строки.

Проверка практической и статистической значимости

Один из старейших типов тестирования на предвзятость сосредоточен на различиях средних *результатов* между группами. Вы можете использовать практические тесты или измерения величины эффекта, такие как *коэффициент неблагоприятного воздействия* (adverse impact ratio, AIR) или *стандартизированную среднюю разницу* (standardized mean difference, SMD), чтобы понять, являются ли различия между средними результатами практически значимыми. Можно также прибегнуть к тестированию статистической значимости, чтобы понять, связаны ли средние различия между демографическими группами с нашей текущей выборкой данных или они, вероятно, снова будут наблюдаться в будущем.

Тестирование на различия в показателях качества

Другой распространенный тип тестирования — исследование различий в показателях качества между группами. Мы могли бы выяснить, насколько совпадают показатели истинно положительных прогнозов (TPR), истинно отрицательных прогнозов (TNR) или R^2 (среднеквадратической ошибки) в разных демографических группах.

Правило четырех пятых

Правило четырех пятых — это руководство, опубликованное в 1978 году в «Единых руководящих принципах процедур отбора сотрудников» (UGESP) Комиссией по равным возможностям трудоустройства (EEOC). Часть 1607.4

Книги для программистов: https://t.me/free_books_for_it

UGESP гласит, что «коэффициент отбора для любой расы, пола или этнической группы, составляющий менее четырех пятых (4/5) (или восьмидесяти процентов) от коэффициента для группы с самым высоким коэффициентом, обычно рассматривается федеральными правоохранительными органами как свидетельство негативного воздействия». К лучшему или худшему, но значение 0,8 для AIR, которое сравнивает частоту событий, таких как выбор работы или одобрение кредита, стало широко распространенным эталоном предвзятости в системах ML.

Подходы к устранению проблем

Когда тестирование выявляет проблемы, мы стремимся их исправить. Подходы к устранению технических искажений часто называют *исправлением* (remediation). Единственное, что мы можем сказать о моделях ML и предвзятости, — это то, что модели ML, по-видимому, допускают больше способов исправления, чем традиционные линейные модели. Благодаря *эффекту Расёмона* — тому факту, что для любого заданного набора обучающих данных обычно существует множество точных моделей ML, — у нас просто есть больше рычагов воздействия и настроек, которыми можно оперировать, чтобы найти лучшие способы уменьшения систематической ошибки и достичь устойчивого качества прогнозов в моделях ML по сравнению с более простыми моделями. Поскольку в ML доступно много вариантов моделей, существует множество потенциальных способов исправить предвзятость. К наиболее распространенным относятся предварительная, внутренняя и окончательная обработка, а также выбор модели:

- *Предварительная обработка*

  Повторная балансировка, повторное взвешивание или повторная выборка обучающих данных, которые позволяют добиться лучшего представления демографических групп или более справедливо распределить положительные результаты.

- *Внутренняя обработка*

  Любое количество изменений в алгоритмах обучения ML, включая ограничения, регуляризацию и функции двойных потерь или включение информации состязательного моделирования, которые направлены на выработку более сбалансированных прогнозов среди разных демографических групп.

- *Окончательная обработка*

  Изменение прогнозов модели напрямую для получения менее предвзятых результатов.

- *Выбор модели*

  При выборе моделей наряду со стандартными показателями качества необходимо учитывать и предвзятость. Как правило, можно найти модель с достаточно хорошими характеристиками производительности и справедливости, если измерять предвзятость и производительность по большому набору настроек гиперпараметров и входных признаков.

Книги для программистов: https://t.me/free_books_for_it

Наконец, нам нужно помнить, что в случае предвзятости ML может возникнуть юридическая ответственность. Существует множество юридических обязательств, связанных с предвзятостью в системах ML. И поскольку мы не юристы (и, вероятно, вы тоже), нам следует быть осторожными в своих толкованиях закона. Важно не позволять эффекту Даннинга — Крюгера взять верх и советоваться с настоящими специалистами в области антидискриминационного законодательства. Если у вас есть какие-либо опасения по поводу юридических проблем в ваших системах ML, сейчас самое время обратиться к менеджерам или в юридический отдел. Опираясь на эту серьезную информацию, мы переходим к обучению модели XGBoost и ее тестированию на предмет предвзятости.

Обучение модели

Первым шагом практического примера в текущей главе является обучение модели XGBoost на примере данных о кредитных картах. Чтобы избежать проблем, связанных с несопоставимым отношением, мы *не* будем использовать демографические признаки в качестве входных данных для этой модели:

```
id_col = 'ID'
groups = ['SEX', 'RACE', 'EDUCATION', 'MARRIAGE', 'AGE']
target = 'DELINQ_NEXT'
features = [col for col in train.columns if col not in groups + [id_col, target]]
```

Вообще говоря, для большинства бизнес-приложений безопаснее не использовать демографическую информацию в качестве входных данных модели. Это не только юридически рискованно в таких сферах, как потребительское кредитование и трудоустройство, но и подразумевает, что деловые решения должны приниматься (в том числе) на основе расы или пола — и это опасная территория. Однако верно также и то, что использование демографических данных при обучении модели может уменьшить предвзятость, и мы увидим пример подобной ситуации, когда попробуем исправить смещение в процессе обработки. Также существуют определенные виды решений, которые должны основываться на демографической информации, например назначение лекарственных препаратов. Поскольку мы работаем с примером системы кредитных решений, и так как мы не социологи или эксперты по законодательству о противодействии дискриминации, мы проявим осторожность и не станем использовать демографические характеристики в нашей модели. Демографические характеристики понадобятся нам позже в этой главе для проверки наличия предвзятости и ее устранения.



Одна из областей, где мы, специалисты по данным, склонны ошибаться, — это использование демографической информации в моделях или технических методах исправления предвзятости таким образом, что это может быть равносильно *несопоставимому отношению*. Приверженцы доктрины *справедливости через осведомленность* (fairness through awareness) могут в некоторых случаях справедливо не соглашаться, но на сегодняшний день наиболее консервативным подходом к управлению предвзятостью в сфере ML, связанной с жильем, кредитами, трудоустройством и другими традиционными приложениями высокого риска, является отказ от прямого использования демографической информации в моделях или при

Книги для программистов: https://t.me/free_books_for_it

исправлении предвзятости. Использование демографической информации только для тестирования предвзятости в целом приемлемо. Дополнительная информация представлена в главе 4.

Несмотря на свои риски, демографическая информация важна для управления предвзятостью, и одна из причин, по которой организации допускают ошибки в управлении рисками предвзятости в сфере ML, — это отсутствие необходимой информации для проверки и последующего устранения предвзятости. Как минимум это означает наличие имен и почтовых индексов людей, чтобы мы могли использовать улучшенное байесовское геокодирование фамилий и связанные с ним методы для получения их демографической информации. Если средства контроля конфиденциальности данных позволяют и имеются необходимые меры безопасности, для тестирования предвзятости наиболее полезно напрямую собирать демографические признаки людей. Важно отметить, что все методы, используемые в этой главе, требуют демографической информации, но по большей части мы можем использовать демографическую информацию, которая выводится или собирается напрямую. Учитывая эти важные предостережения, мы переходим к обучению модели XGBoost с ограничениями и выбору порогового значения оценки.



Прежде чем обучать модель в контексте, где необходимо управлять рисками предвзятости, мы всегда должны убедиться, что у нас под рукой есть необходимые данные для проверки предвзятости. Как минимум это означает наличие имени, почтового индекса и реализации BISG. В идеале это означает сбор демографических меток и всю заботу о конфиденциальности и безопасности данных, связанную со сбором и хранением конфиденциальных данных. В любом случае неведение не приносит радости, когда дело касается предвзятости в отношении ML.

Мы снова воспользуемся преимуществами ограничений монотонности. Основная причина, по которой прозрачность важна для управления предвзятостью в машинном обучении, заключается в том, что если тестирование предвзятости выявляет проблемы (а это часто так и случается), у нас больше шансов понять, что не так в модели, и можно ли это исправить. Если же мы работаем с необъяснимой моделью ML и возникают проблемы с предвзятостью, то велика вероятность, что придется полностью отказаться от модели и надеяться, что со следующей необъяснимой моделью нам повезет больше. Мы не считаем такой подход научным.

Мы предпочитаем тестировать, отлаживать и, насколько это возможно, понимать, как и почему работают модели ML. Наша модель XGBoost с ограничениями не только более стабильна и обобщаема, но и должна быть более прозрачной и поддающейся отладке. Мы также хотим подчеркнуть, что при использовании преимуществ монотонных ограничений для повышения объяснимости и пользовательской целевой функциональности XGBoost при одновременном рассмотрении показателей качества и систематической ошибки (см. раздел «Внутренняя обработка» далее в главе) мы изменяем нашу модель так, чтобы она была одновременно более прозрачной и более справедливой. Пожалуй, это именно те изменения, которые следует внести, если мы беспокоимся о стабильной производительности, максимальной прозрачности и минимальной предвзятости в приложении высокого риска. Замечательно, что алгоритм XGBoost достаточно развит, чтобы предложить такой уровень

Книги для программистов: https://t.me/free_books_for_it

глубокой настройки. (К сожалению, читателям, работающим в сфере кредитования, ипотеки, жилья, трудоустройства и других традиционно регулируемых секторах, вероятно, придется проконсультироваться с юристами, прежде чем использовать пользовательскую целевую функцию, которая обрабатывает демографические данные, из-за риска несопоставимого отношения.)



Мы можем комбинировать в XGBoost ограничения монотонности (повышенная объяснимость) и настраиваемые целевые функции (управление предвзятостью) для непосредственного обучения более прозрачных и менее предвзятых моделей машинного обучения.

При определении ограничений для этой главы мы используем базовый подход, основанный на корреляции Спирмена. Корреляция Спирмена хороша тем, что она учитывает монотонность, а не линейность (как в случае с коэффициентом корреляции Пирсона). В нашем процессе выбора ограничений мы также применяем аргумент `corr_threshold`, чтобы небольшие корреляции не вызывали ложных ограничений:

```
def get_monotone_constraints(data, target, corr_threshold):

    # определяем корреляцию Спирмена
    # создаем кортеж 1, 0, -1 для каждого признака
    # 1 - положительное ограничение, 0 - нет ограничения,
    # -1 - отрицательное ограничение
    corr = pd.Series(data.corr(method='spearman')[target]).drop(target)
monotone_constraints = tuple(np.where(corr < -corr_threshold,
                                      -1,
                                      np.where(corr > corr_threshold, 1, 0)))
    return monotone_constraints

# определяем ограничения
correlation_cutoff = 0.1
monotone_constraints = get_monotone_constraints(train[features+[target]],
                        target,
                        correlation_cutoff)
```

Наш код обучения модели очень прост. Мы начнем с гиперпараметров, которые использовали раньше, чтобы добиться хороших результатов и не сходить с ума от настройки гиперпараметров. Мы просто пытаемся начать с приличного стартового уровня, потому что, когда мы приступим к устранению систематической ошибки, нам придется много настраивать модель и применять методы тщательного отбора. Вот как выглядит наша первая попытка обучения:

```
# вносим в модель глобальную предвзятость
# задаем параметры обучения, включая monotone_constraints
base_score = train[target].mean()

params = {
    'objective': 'binary:logistic',
```

Книги для программистов: https://t.me/free_books_for_it

```
    'eval_metric': 'auc',
    'eta': 0.05,
    'subsample': 0.6,
    'colsample_bytree': 1.0,
    'max_depth': 5,
    'base_score': base_score,
    'monotone_constraints': dict(zip(features, monotone_constraints)),
    'seed': seed
}

# обучаем с ранней остановкой при валидации
watchlist = [(dtrain, 'train'), (dvalid, 'eval')]
model_constrained = xgb.train(params,
                              dtrain,
                              num_boost_round=200,
                              evals=watchlist,
                              early_stopping_rounds=10,
                              verbose_eval=False)
```

Чтобы рассчитать тестовые значения, такие как AIR, и другие коэффициенты качества для демографических групп в последующих разделах, нам нужно будет установить пороговую вероятность, чтобы можно было измерять результаты нашей модели, а не только ее прогнозируемые вероятности. Как и при обучении модели, ищем отправную точку, чтобы получить базовые показатели прямо сейчас. Мы сделаем это с помощью общих показателей качества, таких как F1, точность и полнота. На рис. 10.1 можно увидеть, что при выборе порогового значения вероятности, которое максимизирует F1, мы достигаем разумного компромисса между точностью и полнотой. Точность представляет собой долю правильных *положительных решений* модели (positive predicted value, положительное прогнозируемое значение), а полнота — долю от правильных *положительных результатов* модели (доля истинно положительных результатов). Для нашей модели это число составляет 0,26. Начнем с того, что все прогнозы выше 0,26 не предполагают утверждение кредита. Все прогнозы с коэффициентом 0,26 или ниже рассматриваются как одобрение выдачи кредита.

Мы знаем, что, скорее всего, в конечном итоге нам придется уточнить порог из-за проблем с предвзятостью. В нашем примере увеличение порога означает кредитование большего числа людей. Увеличивая порог, мы надеемся, что будем кредитовать больше *разных* людей. Уменьшая порог, мы делаем процесс подачи заявки на кредит более избирательным, предоставляя кредиты меньшему количеству людей и, вероятно, меньшему количеству разных типов людей. Еще одно важное замечание о пороговых значениях: если мы отслеживаем или проверяем уже развернутую модель машинного обучения, нам следует использовать точное пороговое значение, которое используется для принятия решений in vivo, а не идеализированное пороговое значение, основанное на статистических показателях, которое мы выбрали здесь.

Книги для программистов: https://t.me/free_books_for_it

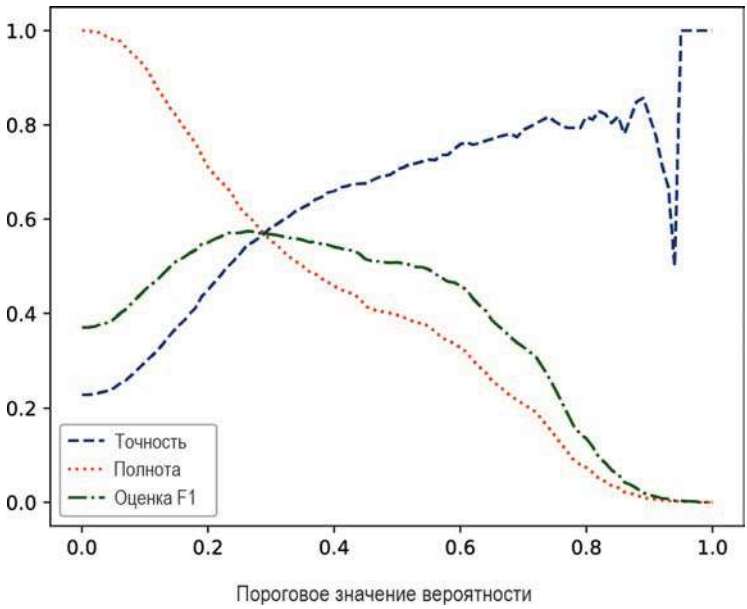


Рис. 10.1. Предварительный порог, необходимый для первоначального тестирования систематической ошибки, выбирается путем максимизации показателя F1



При обучении и мониторинге кредитных моделей следует помнить, что обычно у нас есть достоверные данные только о заявителях, которые были выбраны в прошлом для получения кредитного продукта. Большинство согласны с тем, что это явление вносит предвзятость в любое решение, основанное только на заранее отобранных людях. Что с этим делать, не совсем понятно — это тема для широких дискуссий о методах *отклонения вывода* (reject inference). Имейте в виду, что аналогичные проблемы предвзятости применимы и к другим типам приложений, где долгосрочные данные об отвергнутых лицах недоступны.

Оценка моделей на предмет предвзятости

Теперь, когда у нас есть модель и пороговое значение, можно углубиться в проверку на предмет предвзятости. В этом разделе мы проверим различные типы предвзятости: в показателях, в решениях, в отношении отдельных лиц и опосредованную предвзятость. Сначала мы построим матрицы несоответствий и соберем множество различных показателей качества и ошибок для каждой демографической группы. Далее используем установленные пороги предвзятости в сфере занятости в качестве эмпирического правила к соотношениям этих показателей, чтобы выявить любую проблемную предвзятость в показателях. Затем мы применим традиционные тесты на предвзятость, соответствующие тем, которые используются в программах справедливого кредитования и соблюдения требований в области занятости в США, чтобы проверить результаты модели на предмет предвзятости. После этого проанализируем остатки, чтобы выявить любые отклонения от нормы или любые странные результаты в пределах нашего порогового значения. Мы также будем использовать состязательную модель для выявления любых строк данных, которые,

Книги для программистов: https://t.me/free_books_for_it

по-видимому, содержат больше предвзятости, чем другие. Завершим наше обсуждение тестирования на предвзятость, выделив способы поиска опосредованных, т. е., казалось бы, нейтральных входных признаков, которые действуют как демографическая информация в моделях и могут привести к различным проблемам с предвзятостью.

Способы тестирования для групп

Мы начнем проверку на предвзятость с поиска проблем в том, как наша модель в среднем относится к группам людей. По опыту, лучше всего начать с традиционного тестирования, руководствуясь правовыми стандартами. Для большинства организаций юридические риски являются наиболее серьезными для их систем ИИ, а оценка юридических рисков — самый простой путь к участию в тестировании на предвзятость. По этой причине и для краткости мы не будем рассматривать в текущей главе пересекающиеся группы. Мы продолжим фокусироваться на традиционных защищенных классах и связанных с ними традиционных расовых группах. В зависимости от назначения модели, юрисдикции и действующего законодательства, потребностей заинтересованных сторон или других факторов может оказаться наиболее целесообразным провести тестирование на предвзятость среди традиционных демографических групп, пересекающихся демографических групп или даже по шкале оттенков кожи. Например, в контексте справедливого кредитования — из-за сложившегося прецедента юридического тестирования на предвзятость — вероятно, сначала имеет наибольший смысл провести тестирование в традиционных демографических группах, и, если позволяет время или организационная динамика, нам следует вернуться к тестированию на пересекающихся группах. Для общих систем ИИ или моделей ML, работающих в более широкой экономической сфере США и не подчиняющихся конкретным требованиям противодействия дискриминации, тестирование пересекающихся групп, вероятно, должно быть обязательным стандартом, когда это возможно. Для систем распознавания лиц, возможно, имеет смысл протестировать различные группы оттенков кожи.

Сначала мы посмотрим, насколько одинаково работает модель в традиционных демографических группах. Мы также будем проверять отсутствие *групповой справедливости* (group fairness), иногда также называемой *статистическим* или *демографическим паритетом*, в результатах модели. Эти представления о групповой справедливости по сути своей ошибочны, поскольку определение и измерение групп людей затруднено, средние значения скрывают много информации об отдельных людях, а пороговые значения, используемые для этих тестов, несколько произвольны. Несмотря на эти недостатки, данные тесты сегодня являются одними из наиболее популярных. Они могут сообщить полезную информацию о том, как модель ведет себя на высоком уровне, и указать на серьезные проблемы. Как и для многих других тестов, которые мы обсудим в этом разделе, ключ к интерпретации результатов заключается в следующем: *успешное прохождение этих тестов не имеет большого значения — модель или система все еще могут иметь серьезные проблемы с предвзятостью in vivo, но провал тестов — это огромная проблема и сигнал тревоги.*

Книги для программистов: https://t.me/free_books_for_it

Прежде чем приступить к самим тестам, важно подумать о том, где проводить тестирование. На каких данных мы должны проверять модель — на обучающих, валидационных или тестовых? Наиболее стандартными наборами для проверки на предвзятость являются валидационные и тестовые данные, точно так же, как при проверке точности модели. Тестирование на предвзятость с применением валидационных данных также применяют для выбора модели, как мы обсудим в разделе «Устранение предвзятости» немного позже. Использование тестовых данных должно дать нам некоторое представление о том, что будет происходить с предвзятостью модели после ее развертывания. (Нет никаких гарантий, что модель ML и дальше будет работать так же, как на тестовых данных, поэтому мониторинг предвзятости после развертывания имеет решающее значение.) Проверка предвзятости на обучающих данных в основном нужна для наблюдения за различиями в измерениях предвзятости на валидационном и тестовом наборах. Этот подход особенно полезен, если один набор выделяется среди других, и потенциально может пригодиться для понимания причин возникновения предвзятости в модели. Если наборы для обучения, валидации и тестирования построены таким образом, что обучение идет первым по времени, а тестирование — последним (как, вероятно, и должно быть), сравнение измерений предвзятости на разных наборах данных также полезно для понимания тенденций предвзятости. Нарастание показателей предвзятости от обучения к валидации и тестированию — тревожный знак. Еще один вариант — оценить дисперсию измерений предвзятости с помощью перекрестной проверки или бутстреппинга, как это делается и со стандартными показателями качества модели. Перекрестная проверка, бутстреппинг, стандартные отклонения или ошибки, доверительные интервалы и другие меры дисперсии показателей систематической ошибки, т. е. предвзятости модели, помогут нам понять, являются ли наши результаты тестирования на предвзятость точными или зашумленными. Это важная часть любого анализа данных.

При тестировании на предвзятость в следующих разделах мы будем придерживаться базовых методов и искать систематические ошибки в работе модели с валидационным и тестовым наборами данных. Если вы никогда не выполняли тестирование на предвзятость, это хороший способ начать. А внутри крупных организаций, где логистика и корпоративная политика еще больше усложняют задачу, это, возможно, единственная проверка на предвзятость, которую можно провести. Мониторинг предвзятости должен происходить все время, пока модель применяется на производстве. Тестирование моделей часто сопровождается значительными организационными проблемами. По этой причине мы настоятельно рекомендуем вам начать со стандартных практик поиска предвзятости в больших демографических группах, а затем использовать все оставшееся время и ресурсы и исследовать предвзятость в отношении отдельных лиц для выявления косвенных факторов или других причин предвзятости в вашей модели. Этим мы сейчас и займемся.



Прежде чем начать тестирование на предвзятость, необходимо иметь абсолютно четкое представление о том, как положительное решение представлено в данных, что означает «положительное решение» в реальном мире, как предсказанные нашей моделью вероятности согласуются с этими двумя понятиями и какие пороговые значения приводят к положительным решениям. В нашем примере решение,

Книги для программистов: https://t.me/free_books_for_it

которое хотели бы получить субъекты моделирования, представляет собой ноль на выходе модели, связанный с вероятностью ниже порогового значения 0,26. Кандидатам, отнесенным к нулевой категории, будет утверждена выдача кредита.

Тестирование качества прогнозов

Модель должна демонстрировать примерно одинаковое качество прогнозов для разных демографических групп, а если этого не происходит, то мы наблюдаем важный тип предвзятости. Если все группы придерживаются одного и того же стандарта модели ML для получения кредитного продукта, но этот стандарт не является точным предсказателем перспектив погашения кредита для некоторых групп, это несправедливо. (Это чем-то похоже на понятие *дифференциальной достоверности*, обсуждаемое в главе 4.) Чтобы выполнить тестирование предвзятости в форме различий между группами для бинарного классификатора, такого как наша модель XGBoost, рассмотрим матрицы несоответствий для каждой группы и сформируем разные показатели совпадений и ошибок по группам. Среди них будут как распространенные показатели, такие как доля истинно положительных и ложноположительных прогнозов, так и менее распространенные показатели в науке о данных, например уровень ложных обнаружений.

Следующий блок кода далек от лучшей реализации, поскольку он основан на динамической генерации кода и операторе `eval()`, но сделан максимально наглядным. В нем читатели смогут увидеть, как четыре ячейки матрицы несоответствий можно использовать для расчета множества различных показателей качества модели:

```
def confusion_matrix_parser(expression):

    # tp | fp cm_dict[level].iat[0, 0] | cm_dict[level].iat[0, 1]
    # ------- ==> -----------------------------------------
    # fn | tn cm_dict[level].iat[1, 0] | cm_dict[level].iat[1, 1]

    metric_dict = {
    'Распространенность': '(tp + fn) / (tp + tn +fp + fn)',
    'Достоверность': '(tp + tn) / (tp + tn + fp + fn)',
    'Доля истинно полож. результатов (TPR)': 'tp / (tp + fn)',
    'Точность': 'tp / (tp + fp)',
    'Специфичность': 'tn / (tn + fp)',
    'Отрицательное предск. значение': 'tn / (tn + fn)',
    'Доля ложнополож. результатов (FPR)': 'fp / (tn + fp)',
    'Доля ложных обнаружений': 'fp / (tp + fp)',
    'Доля ложноотр. результатов (FNR)': 'fn / (tp + fn)',
    'Доля ложных пропусков (FOR)': 'fn / (tn + fn)'
    }

    expression = expression.replace('tp', 'cm_dict[level].iat[0, 0]')\
                           .replace('fp', 'cm_dict[level].iat[0, 1]')\
                           .replace('fn', 'cm_dict[level].iat[1, 0]')\
                           .replace('tn', 'cm_dict[level].iat[1, 1]')
    return expression
```

Книги для программистов: https://t.me/free_books_for_it

Применяя функцию `confusion_matrix_parser` к матрицам несоответствия для каждой демографической группы вместе с другим кодом, который циклически обрабатывает группы и показатели в `metric_dict`, можно создать таблицу, подобную табл. 10.1. Для краткости в этом подразделе мы сосредоточились на показателях, связанных с расой. Если бы это была настоящая модель кредитования или ипотеки, мы бы рассматривали людей разных полов, разных возрастных групп, лиц с ограниченными возможностями, из разных географических регионов и, возможно, даже другие подгруппы населения.

Таблица 10.1. Общие показатели качества модели, полученные на основе матрицы несоответствий для разных расовых групп на тестовом наборе данных!

Группа	Распространенность	Достоверность	TPR	Точность	...	FPR	FDR	FNR	FOR
Латино-американцы	0,399	0,726	0,638	0,663	...	0,215	0,337	0,362	0,235
Черные	0,387	0,720	0,635	0,639	...	0,227	0,361	0,365	0,229
Белые	0,107	0,830	0,470	0,307	...	0,127	0,693	0,530	0,068
Азиаты	0,101	0,853	0,533	0,351	...	0,111	0,649	0,467	0,055

В табл. 10.1 видны некоторые намеки на предвзятость нашей модели, но на самом деле она еще не измеряет предвзятость. Она просто содержит значения различных показателей по группам. Следует обратить внимание на случаи, когда эти значения явно различны для разных групп. Например, точность выглядит совершенно по-разному в зависимости от демографических групп (белые и азиаты, с одной стороны, и чернокожие и латиноамериканцы — с другой). То же самое можно сказать и о других показателях, таких как доля ложных срабатываний, доля ложных обнаружений и ложных пропусков. (Различия в распространенности говорят нам о том, что дефолты по кредитам чаще встречаются *в данных* среди групп чернокожих и латиноамериканцев. К сожалению, это не редкость на многих кредитных рынках США.) Глядя на табл. 10.1, мы начинаем понимать, что наша модель предсказывает больше потенциальных дефолтов для чернокожих и латиноамериканцев, но пока трудно сказать, делает ли она свою работу хорошо и справедливо. (Тот факт, что набор данных содержит такие значения, не делает их объективными или справедливыми!) Чтобы понять, действительно ли наблюдаемые нами закономерности проблематичны, нам нужно сделать еще один шаг. Мы воспользуемся методами традиционного тестирования на систематическую ошибку и разделим значение для каждой группы на соответствующее значение для контрольной группы, а затем применим правило четырех пятых. В данном случае мы предполагаем, что контрольная группа — белые люди.



Строго говоря, в контексте трудоустройства контрольная группа является наиболее предпочтительной в анализе, и это не обязательно белые люди или мужчины. Могут быть и другие причины использовать контрольные группы, не состоящие из белых людей или мужчин. Выбор контрольной или референтной группы для анализа предвзятости — сложная задача, которую лучше всего выполнять совместно

Книги для программистов: https://t.me/free_books_for_it

с экспертами по правовым вопросам, соблюдению нормативных требований, социальным наукам или с представителями заинтересованных сторон.

Выполнив деление, мы получим значения, представленные в табл. 10.2. (Мы делим каждый столбец таблицы на значение в строке «Белые». Вот почему в этой строке значения равны 1,0.) Теперь мы можем искать значения за пределами определенного диапазона. Для определения границ диапазона мы воспользуемся правилом четырех пятых и получим граничные значения 0,8–1,25, что соответствует разнице 20% между группами. (Некоторые предпочитают более узкий диапазон приемлемых значений, особенно в сценариях высокого риска, скажем, 0,9–1,11, что указывает на 10%-ную разницу между группами.) Когда мы видим для показателей неравенства значения выше 1, это означает, что защищенная группа или группа меньшинства имеет более высокое значение относительно исходного показателя, и наоборот для значений ниже 1.

Анализируя табл. 10.2, мы не наблюдаем значений, выходящих за пределы допустимого диапазона для азиатов. Это означает, что модель работает достаточно справедливо в группах белых и азиатов. Тем не менее мы видим явно выходящие за пределы допустимого диапазона значения для латиноамериканцев и чернокожих в столбцах точности, доли ложноположительных прогнозов (FPR), доли ложных обнаружений (FDR) и доли ложных пропусков (FOR). Хотя применение правила четырех пятых позволяет нам увидеть эти значения, оно на самом деле не помогает их интерпретировать. Нам придется самим тщательно обдумать полученные результаты. Также необходимо помнить, что единица на выходе нашей модели является прогнозируемым дефолтом, а модель оперирует вероятностью дефолта. Чем выше значение вероятности, тем выше шанс дефолта заемщика с точки зрения модели.

Таблица 10.2. Измерения систематической ошибки, основанные на показателях для разных расовых групп в тестовых данных

Группа	Неравенство распространенности	Неравенство достоверности	Неравенство TPR	Неравенство точности	...	Неравенство FPR	Неравенство FDR	Неравенство FNR	Неравенство FOR
Латиноамериканцы	3,730	0,875	1,357	2,157	...	1,696	0,486	0,683	3,461
Черные	3,612	0,868	1,351	2,078	...	1,784	0,522	0,688	3,378
Белые	1,000	1,000	1,000	1,000	...	1,000	1,000	1,000	1,000
Азиаты	0,943	1,028	1,134	1,141	...	0,873	0,937	0,881	0,821

Учитывая, что дефолты встречаются в данных намного чаще среди чернокожих и латиноамериканцев, можно предположить, что наша модель усвоила эту особенность и теперь предсказывает более высокий уровень дефолта в этих группах. В следующем разделе мы с помощью традиционного тестирования попытаемся ответить на главный вопрос: справедливо ли прогнозировать большее количество

Книги для программистов: https://t.me/free_books_for_it

дефолтов в этих группах? На данный момент мы пытаемся выяснить, являются ли справедливыми показатели модели. Глядя на то, какие показатели выходят за рамки для защищенных групп и что они означают, мы можем отметить следующее:

- неравенство точности: ~2× (намного больше) правильных прогнозов дефолта из числа тех, которые *были предсказаны*;
- неравенство доли ложноположительных прогнозов: ~1,5× (намного больше) неверных прогнозов дефолта для тех, кто *не допустил* дефолт на самом деле;
- неравенство доли ложных обнаружений: ~0,5× (меньше) неверных прогнозов, что *дефолт будет*;
- неравенство доли ложных пропусков: ~3,5× (намного больше) неправильных прогнозов, что *дефолта не будет*.

Точность и доля ложных обнаружений имеют один и тот же знаменатель — меньшую группу из тех, для кого прогнозируется дефолт, — и могут интерпретироваться вместе. Они показывают, что эта модель имеет более высокую долю истинно положительных результатов для чернокожих и латиноамериканцев по сравнению с белыми людьми, что означает более высокий уровень правильных прогнозов дефолта для этой группы. Доля ложных обнаружений повторяет этот результат, указывая на более низкий уровень ложных срабатываний или неправильных решений о дефолте для рассматриваемых групп меньшинств. Вместе с тем доля ложных пропусков показывает, что наша модель чаще принимает неверные решения о выдаче кредита за пределами групп чернокожих и латиноамериканцев. Различия между группами в точности, доле ложных обнаружений и ложных пропусков свидетельствуют о серьезных проблемах с предвзятостью, но это предвзятость, которая отдает предпочтение чернокожим и латиноамериканцам с точки зрения формальной точности модели (т. е. модель дает более точные прогнозы для этих двух групп).

Как понимать матрицу несоответствий

В нашем конкретном случае мы можем интерпретировать значения показателей матрицы несоответствий следующим образом:

Распространенность

Как часто на самом деле происходит дефолт в этой группе согласно данным?

Достоверность

Как часто модель *правильно* предсказывает дефолт и отсутствие дефолта для этой группы?

Доля истинно положительных прогнозов

Для какой доли членов группы, *действительно допустивших* дефолт, модель *правильно* предсказала дефолт?

Точность

Сколько было *правильных* прогнозов о дефолте среди членов группы, для которых был *предсказан* дефолт?

Книги для программистов: https://t.me/free_books_for_it

Специфичность

Сколько было *правильных прогнозов* об отсутствии дефолта среди членов группы, которые *действительно не допустили* дефолт?

Отрицательное прогнозируемое значение

Сколько было *правильных прогнозов* об отсутствии дефолта среди членов группы, для которых было *предсказано* отсутствие дефолта?

Доля ложноположительных прогнозов

Какова доля людей, которым модель *неправильно* предсказала дефолт среди тех членов группы, которые *на самом деле не допустили* дефолт?

Доля ложных обнаружений

Сколько *ошибок* допустила модель по отношению к числу членов группы, которым она *предсказала* дефолт (т. е. как часто модель дает «ложную тревогу»)?

Доля ложноотрицательных прогнозов

Для какой доли членов группы, *фактически допустивших* дефолт, модель ошибочно *предсказала отсутствие* дефолта?

Доля ложных пропусков

Сколько *ошибок* допустила модель по отношению к числу членов группы, которым она *предсказала* отсутствие дефолта (т. е. как часто модель пропускает настоящий дефолт)?

Попробуйте последовать этому примеру, чтобы создать свою интерпретацию матрицы несоответствий и показателей ошибок в вашем следующем важном проекте машинного обучения. Это поможет вам более четко продумать вопросы предвзятости, точности и безопасности.

Несовпадение долей ложноположительных результатов отражает нечто иное. Доля ложноположительных результатов измеряется среди большей группы тех, кто на самом деле *не* допустил дефолт. В этой группе мы видим более высокий уровень неправильных прогнозов о дефолте (т. е. ложных срабатываний) среди чернокожих и латиноамериканцев. В совокупности все эти результаты указывают на модель с проблемами предвзятости, ориентированными на группы меньшинств. Из них наибольшее беспокойство вызывает несовпадение частоты ложноположительных прогнозов. Данные из таблицы свидетельствуют о том, что модель в 1,5 раза чаще *ошибочно* предсказывает дефолт для представителей чернокожей расы и латиноамериканцев по сравнению с белыми людьми. Это означает, что многим и без того исторически бесправным людям ошибочно отказывают в увеличении кредитной линии на основе прогнозов этой модели, что может привести к реальному вреду. Конечно, мы также видим свидетельства правильных и неправильных решений об одобрении кредита в пользу меньшинств. Ничто из этого не является хорошим знаком, но нам нужно глубже протестировать результаты, чтобы получить более четкое представление о групповой справедливости в этой модели.



При анализе моделей регрессии мы можем пропустить матрицы несоответствий и перейти непосредственно к сравнению таких показателей, как R^2 или среднеквадратическая ошибка между группами. Там, где это уместно, и особенно для ограниченных показателей, таких как R^2 или средняя процентная ошибка (mean average

Книги для программистов: https://t.me/free_books_for_it

percentage error, MAPE), мы также можем применить эмпирическое правило четырех пятых к соотношению этих показателей, чтобы помочь выявить проблемные отклонения в качестве прогнозов.

В целом тестирование качества прогнозов модели — полезный инструмент для изучения неправильных и негативных решений, таких как ложноположительные прогнозы. Более традиционному тестированию предвзятости, которое фокусируется на результатах применения прогнозов модели, а не на их качестве, труднее выявить проблемы предвзятости в неправильных или отрицательных решениях. К сожалению, как мы вскоре увидим, эти два подхода могут показывать разные результаты. Глядя на некоторые статистические показатели качества прогнозов, можно подумать, что наша модель отдает предпочтение меньшинствам, но в следующем разделе мы увидим, что это не так. Многие из положительных результатов, которые мы здесь увидели, касаются довольно небольших групп людей. Если рассмотреть реальные результаты, картина предвзятости в нашей модели будет иной и более четкой. Подобные противоречия между тестированием качества прогнозов и результатов применения модели распространены и хорошо документированы. Мы утверждаем, что тестирование результатов, соответствующее правовым стандартам и отражающее реальное положение вещей, является более важным.



Существует хорошо известное противоречие между улучшением качества данных, отражающих исторические предубеждения (это свойственно большинству данных, с которыми мы работаем), и балансированием результатов между демографическими группами. На данные всегда влияют системные, человеческие и статистические предубеждения. Если мы сделаем результаты более сбалансированными, это приведет к снижению формальных показателей качества прогнозов на предвзятом наборе данных.

Поскольку интерпретировать все эти различные показатели качества сложно, некоторые из них могут иметь большее значение в определенных сценариях, чем другие, и, вероятно, будут противоречить друг другу или результатам тестирования результатов, видные исследователи составили дерево решений (*https://oreil.ly/-y827*) (слайд 40), чтобы помочь разработчикам сосредоточиться на меньшем подмножестве показателей качества. Согласно этому дереву, где наша модель носит «карательный» характер (более высокая вероятность означает решение об отказе в увеличении кредитной линии), а самый явный вред — это неправильный отказ в увеличении кредитной линии представителям групп меньшинств, наибольший вес при анализе имеет неравенство долей ложноположительных прогнозов. Очевидно, что это неравенство не сулит ничего хорошего. Посмотрим, что покажет тестирование результатов.

Традиционное тестирование показателей исходов

При анализе модели бинарной классификации было проще всего сначала оценить *качество прогнозов* для разных групп с помощью матрицы несоответствий. Но, на наш взгляд, более важен анализ различий в *исходах* между группами с использованием традиционных мер статистической и практической значимости. Мы соединим два хорошо известных практических показателя систематической ошибки, AIR и SMD, с хи-квадратом и *t*-тестом соответственно. Безусловно, нам полезно знать,

Книги для программистов: https://t.me/free_books_for_it

является ли обнаруженная разница в результатах групп статистически значимой, но в данном случае это также может быть юридическим требованием. Статистически значимые различия в результатах или средних оценках являются одним из наиболее распространенных юридически признанных показателей дискриминации, особенно в таких областях, как потребительское кредитование, где алгоритмическое принятие решений регулируется законом на протяжении десятилетий. Используя практические тесты и измерения величины эффекта, такие как AIR и SMD, с тестами статистической значимости, мы получаем две части информации: величину наблюдаемой разницы и является ли она статистически значимой, т. е. может ли она снова наблюдаться на других выборках данных.



Если вы работаете в строго регулируемой отрасли или в прикладной области высокого риска, прежде чем использовать новые подходы к тестированию, рекомендуется сначала применить традиционные тесты на предвзятость, имеющие юридическое обоснование. Правовые риски зачастую являются наиболее серьезными организационными рисками для многих типов продуктов на основе ML, а законы призваны защищать пользователей и инвесторов.

AIR часто применяется к категориальным результатам, таким как результаты кредитования или найма на работу, когда кандидат либо получает одобрение, либо нет. AIR определяется как доля положительных результатов для защищенной группы, такой как меньши́нства или женщины, деленная на такую же долю положительных результатов для контрольной группы, такой как белые люди или мужчины. Согласно правилу четырех пятых, мы ожидаем, что AIR будет выше 0,8. AIR ниже 0,8 указывает на серьезную проблему. Затем, используя критерий хи-квадрат, мы проверяем, будет ли эта разница наблюдаться снова или она случайна.



Коэффициенты воздействия также можно использовать для регрессионных моделей. Для этого нужно разделить средние баллы или процент от среднего балла для защищенной группы на ту же величину для контрольной группы, применив правило четырех пятых в качестве отправной точки для выявления проблемных результатов. Другими традиционными подходами к измерению систематической ошибки регрессионных моделей являются *t*-тесты и SMD.

В то время как AIR и хи-квадрат чаще всего применяют в случае моделей бинарной классификации, SMD и *t*-тесты обычно используются для прогнозов на основе регрессионных моделей или для числовых величин, таких как заработная плата или кредитные лимиты. Мы применим SMD и *t*-тесты к предсказанным вероятностям нашей модели в демонстрационных целях и для получения дополнительной информации о предвзятости в нашей модели. SMD определяется как разность средних оценок защищенной и контрольной группы, деленная на меру стандартного отклонения оценки. SMD имеет хорошо известные пороговые значения 0,2, 0,5 и 0,8 для малых, средних и больших различий соответственно. Затем мы воспользуемся *t*-тестом, чтобы решить, является ли величина эффекта, измеренная методом SMD, статистически значимой.



В контексте вероятностных выводов модели такое применение SMD также уместно, если оценки модели будут использоваться в каком-то последующем процессе принятия решений и невозможно генерировать исходы модели во время тестирования на предвзятость.

Книги для программистов: https://t.me/free_books_for_it

Помимо тестов значимости, AIR и SMD, мы также будем анализировать базовую описательную статистику, такую как численность, средние значения и стандартные отклонения, как показано в табл. 10.3. Если взглянуть на эту таблицу, становится ясно, что существует большая разница в оценках чернокожих и латиноамериканцев по сравнению с оценками белых и азиатов. Хотя наши данные смоделированы, к сожалению, в них нет чего-то необычного для рынка потребительского кредитования в США. Системная предвзятость реальна, и честные данные о кредитовании, как правило, подтверждают это[1].

Таблица 10.3. Традиционные показатели систематической ошибки, основанные на результатах работы модели, в разных расовых группах для тестовых данных

Группа	Численность	Благоприятный исход	Доля благоприятных исходов	Средняя оценка	Статистическое отклонение оценки	AIR	*p*-значение AIR	SMD	*p*-значение SMD
Латиноамериканцы	989	609	0,615	0,291	0,205	0,736	6,803e–36	0,528	4,311e–35
Черные	993	611	0,615	0,279	0,199	0,735	4,343e–36	0,482	4,564e–30
Азиаты	1485	1257	0,846	0,177	0,169	1,012	4,677e–01	–0,032	8,162e–01
Белые	1569	1312	0,846	0,183	0,172	1,000	–	0,000	–

Из табл. 10.3 сразу становится очевидным, что чернокожие и латиноамериканцы имеют более высокие средние оценки и более низкие показатели благоприятных исходов, чем белые и азиатские люди, в то время как все четыре группы имеют схожие стандартные отклонения оценок. Достаточно ли велики эти различия, чтобы говорить о проблеме предвзятости? Вот тут-то и вступают в силу наши тесты практической значимости. AIR и SMD рассчитываются относительно белых людей. Вот почему белые люди имеют по этим показателям оценки 1,0 и 0,0 соответственно. Если посмотреть на AIR, то как для чернокожих, так и для латиноамериканцев его значение ниже 0,8. Тревога! SMD для этих двух групп составляет около 0,5, что означает среднюю разницу в оценках между группами. Это тоже не очень хороший знак. Нам бы хотелось, чтобы значения SMD были ниже или около 0,2, что указывало бы на небольшую разницу.



Специалисты по работе с данными часто неверно интерпретируют AIR. Вот простая интерпретация: значение AIR выше 0,8 мало что значит, и уж точно не означает, что модель справедлива. Однако значения AIR ниже 0,8 указывают на серьезную проблему.

[1] Если вы хотите удовлетворить свое любопытство по этому вопросу, мы настоятельно рекомендуем вам проанализировать некоторые свободно доступные данные Закона о раскрытии информации о жилищной ипотеке (**https://oreil.ly/xYXdt**).

Книги для программистов: https://t.me/free_books_for_it

Следующий вопрос, который мы могли бы задать в рамках традиционного анализа предвзятости, заключается в том, являются ли эти практические различия между чернокожими и латиноамериканцами статистически значимыми. Плохая новость — они очень значительны: в обоих случаях *p*-значения приближаются к 0. Хотя с 1970-х гг. объемы наборов данных резко возросли, многие юридические прецеденты ссылаются на статистическую значимость на уровне 5% ($p = 0{,}05$) для двусторонней проверки гипотез как на маркер юридически недопустимой предвзятости. Поскольку этот порог совершенно непрактичен для современных больших наборов данных, мы рекомендуем снизить порог *p*-значения для более крупных наборов данных. Однако надо быть готовыми к тому, что нас будут оценивать по $p = 0{,}05$ (как принято сейчас в США). Конечно, исходы судебных слушаний о справедливом кредитовании и дискриминации при приеме на работу далеко не однозначны; факты, контекст и мнение экспертов имеют такое же влияние на окончательное решение суда, как и любые критерии проверки на предвзятость. Но важно иметь в виду, что закон в этой области уже давно существует и на него очень сложно повлиять шумихой вокруг ИИ. Если вы работаете в зоне высокого риска, вам, вероятно, следует провести традиционные тесты на систематическую ошибку в дополнение к новым тестам, как мы это сделали здесь.



В потребительском кредитовании, жилищном строительстве, трудоустройстве и других традиционно регулируемых отраслях экономики США закон о противодействии дискриминации является весьма совершенным и не подвержен влиянию шумихи вокруг ИИ. Тот факт, что AIR и двусторонние статистические тесты кажутся специалистам по работе с данными устаревшими или упрощенными, не означает, что наши организации не будут оценивать по этим стандартам в случае возникновения юридических проблем.

Связанные с расой результаты указывают на довольно серьезную проблему дискриминации в нашей модели. Внедрение такой модели может привести к серьезным юридическим проблемам. Более того, мы будем использовать модель, которая, как нам известно, сохраняет системные предубеждения и может нанести вред людям. Увеличение кредитной линии может быть связано с различными проблемными ситуациями в жизни. Если кто-то просит кредит, надо предположить, что он действительно необходим. Однако мы видим, что решение о предоставлении кредита основано на исторических предубеждениях. Тестирование результатов тоже посылает нам четкий сигнал. Эту модель необходимо исправить перед развертыванием.

Индивидуальная справедливость

До сих пор мы были сосредоточены на справедливости по отношению к группе, но нам также следует проверить нашу модель на предмет индивидуальных проблем со справедливостью. В отличие от предвзятости в отношении групп, предвзятость по отношению к личности — это локальная проблема, которая затрагивает лишь небольшую и конкретную группу людей, вплоть до отдельного человека. Для проверки мы будем использовать два основных метода: анализ остатков и состязательное моделирование. В первом методе, называемом анализом остатков, мы рассматрива-

Книги для программистов: https://t.me/free_books_for_it

ем людей, очень близких к порогу принятия решения, но которые в результате ошибочно получили негативные результаты. Надо убедиться, что в этих случаях отказ в предоставлении кредита не был основан на демографической информации. (Также можно проверять очевидно ошибочные индивидуальные результаты вдали от порога принятия решения.) Во втором подходе — состязательных моделях — мы будем использовать отдельные модели, которые стремятся предсказать информацию о защищенной группе, используя входные признаки и оценки из нашей исходной модели. Для этих моделей рассмотрим аддитивные объяснения Шепли. Обнаружение строки данных, для которой прогнозы состязательной модели очень точны, может указывать на наличие в этой строке информации, которая приводит к предвзятости нашей исходной модели. Анализируя несколько таких строк данных, можно попытаться найти признаки, которые прямо или опосредованно ведут к предвзятости. Прежде чем перейти к разделу, посвященному исправлению предвзятости, мы рассмотрим индивидуальную, а затем косвенную (опосредованную) предвзятость.

Рассмотрим вопросы индивидуальной справедливости на практическом примере. Для начала мы написали код, позволяющий выделить из защищенной группы несколько ошибочно классифицированных субъектов. Это лица, которые, по прогнозам нашей модели, должны были объявить дефолт, но ничего подобного не случилось:

```
black_obs = valid.loc[valid['RACE'] == 'black'].copy()
black_obs[f'p_{target}_outcome'] = np.where(
  black_obs[f'p_{target}'] > best_cut,
  1,
  0)
misclassified_obs = black_obs[(black_obs[target] == 0) &
                              (black_obs[f'p_{target}_outcome'] == 1)]
misclassified_obs.sort_values(by=f'p_{target}').head(3)[features]
```

Результаты показаны в табл. 10.4 и не предполагают какой-либо явной предвзятости, но вызывают некоторые вопросы. Первый и третий заявители, судя по всему, тратят деньги умеренно и по большей части производят платежи вовремя. Эти люди могли быть случайно помещены по другую сторону границы принятия решений. Однако человек во второй строке табл. 10.4, судя по всему, не очень успешен в погашении задолженности по кредитной карте. Возможно, ему действительно не следовало одобрять увеличение кредитной линии.

Таблица 10.4. Подмножество признаков для ошибочно классифицированных субъектов в валидационных данных

LIMIT_BAL	PAY_0	PAY_2	PAY_3	...	BILL_AMT1	BILL_AMT2	BILL_AMT3	...	PAY_AMT1	PAY_AMT2	PAY_AMT3
58 000	–1	–1	–2	...	600	700	0	...	200	700	0
58 000	0	0	0	...	8500	5000	0	...	750	150	30
160 000	–1	–1	–1	...	0	0	600	...	0	0	0

Книги для программистов: https://t.me/free_books_for_it

Чтобы убедиться, действительно ли мы обнаружили проблему индивидуальной предвзятости, можно предпринять следующие действия:

Небольшие возмущения входных признаков

Если какое-то произвольное изменение одного из входных признаков, скажем, уменьшение `BILL_AMT1` на 5 долларов, меняет прогноз модели для данного человека, то решение модели может быть больше связано с крутым участком в ее функции отклика, пересекающимся с границей принятия решения, чем с какой-либо реальной причиной.

Поиск похожих людей

Если таких людей, как текущий субъект, несколько (или достаточно много), модель может несправедливо или ошибочно сегментировать какую-то конкретную подгруппу или пересечение групп.

Если что-то из перечисленного имеет место, правильным решением может стать предоставление кредитной линии для текущего субъекта и подобных лиц.

Нами был проведен такой анализ записей латиноамериканцев и азиатов и получены аналогичные результаты. Мы не были слишком удивлены этими результатами, по крайней мере, по двум причинам. Во-первых, проблемы индивидуальной справедливости сложны и поднимают вопросы причинно-следственной связи, которые системы МО обычно не решают. Во-вторых, индивидуальная справедливость и опосредованная дискриминация, вероятно, представляют собой гораздо больший риск для наборов данных с большим количеством строк. Это связано с тем, что целые субпопуляции могут оказаться на произвольной стороне границы принятия решения. Также риск возрастает, когда модель содержит множество признаков, особенно *альтернативных данных*, т. е. признаков, которые не связаны со способностью погасить кредит, но могут повысить прогностические способности модели.



Ответить на вопросы об индивидуальной справедливости с полной уверенностью сложно, потому что это по сути вопросы о причинно-следственных связях. Анализируя работу сложной нелинейной модели ML, невозможно узнать, не приняла ли она решение на основе какого-либо ограниченного фрагмента данных (т. е. информации о защищенной группе).

Тем не менее анализ остатков, состязательное моделирование, значения SHAP и тщательное применение знаний в предметной области могут иметь большое значение. Для получения дополнительной информации по этой теме прочтите статью Explaining Quantitative Measures of Fairness (*https://oreil.ly/Tg66Z*) от автора критерия величины SHAP и On Testing for Discrimination Using Causal Models (*https://oreil.ly/liP9W*).

Теперь перейдем ко второму методу проверки индивидуальной справедливости: состязательному моделированию. Мы решили обучить две состязательные модели. Первая модель использует те же входные признаки, что и исходная модель, но пытается предсказать статус защищенных групп, а не просрочку по кредиту. Для простоты мы обучили бинарный классификатор целевой принадлежности к защищенному классу — новому маркеру для чернокожих (`Black`) или латиноамериканцев (`Hispanic`).

Книги для программистов: https://t.me/free_books_for_it

Анализируя эту первую состязательную модель, мы можем получить хорошее представление о том, какие признаки имеют наиболее сильную связь с членством в защищенной демографической группе.

Вторая состязательная модель, которую мы обучаем, аналогична первой, за исключением одного дополнительного входного признака — выходных вероятностей нашей исходной модели кредитования. В результате сравнения двух состязательных моделей будет получено представление о том, сколько *дополнительной* информации было закодировано в исходных оценках модели. И мы получим эту информацию на уровне отдельного наблюдения.



Для изучения проблем индивидуальной предвзятости можно использовать многие инструменты машинного обучения, которые генерируют отладочную информацию построчно, например остатки, прогнозы состязательной модели или значения SHAP.

Мы обучили эти состязательные модели как бинарные классификаторы XGBoost с гиперпараметрами, аналогичными исходной модели. Сначала рассмотрели записи данных, принадлежащие защищенным группам, чьи оценки состязательной модели увеличились больше всего, когда вероятности исходной модели были добавлены в качестве признака. Результаты показаны в табл. 10.5. Эта таблица говорит нам о том, что для некоторых записей исходные оценки модели содержат достаточно информации о статусе защищенной группы, поэтому вторая состязательная модель может улучшить первую примерно на 30 процентных пунктов. Очевидно, что нам следует более глубоко изучить выделяющиеся записи, чтобы выявить индивидуальные проблемы справедливости, задавая вопросы, как мы это делали для проблем индивидуальной предвзятости, выявленных с помощью остатков. Таблица 10.5 также подтверждает наше предположение, что удаление демографических маркеров из модели не удаляет из нее демографическую информацию.

Таблица 10.5. Три записи из защищенной группы с самой большой разницей оценок между двумя состязательными моделями на валидационных данных

Наблюдение	Защищенная	Состязательная модель 1	Состязательная модель 2	Разница
9022	1	0,288	0,591	0,303
7319	1	0,383	0,658	0,275
528	1	0,502	0,772	0,270

В главе 2 было сказано, что значения SHAP представляют собой построчную аддитивную схему атрибуции признаков. То есть они сообщают нам, какой вклад каждый признак внес в общий прогноз модели. Мы рассчитали значения SHAP на валидационных данных для второй состязательной модели (той, которая включает оценки нашей исходной модели). На рис. 10.2 представлены распределения значений SHAP для четырех наиболее важных признаков. Каждый из признаков на рис. 10.2 важен для прогнозирования членства в защищенном классе. Наиболее важным признаком прогнозирования информации о защищенной группе являются

Книги для программистов: https://t.me/free_books_for_it

оценки исходной модели p_DELINQ_NEXT. Это интересно само по себе, и записи данных, которые имеют самые высокие значения SHAP для этого признака, являются хорошими целями для дальнейшего изучения индивидуальных нарушений справедливости.

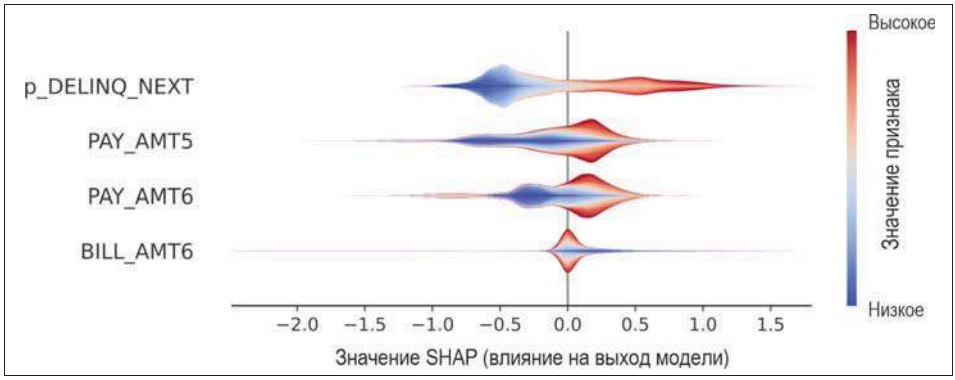


Рис. 10.2. Распределение значений SHAP для четырех наиболее важных признаков нашей состязательной модели в валидационном наборе данных

Возможно, наиболее интересным является цветовой градиент (изменение от светлого к темному) внутри «скрипки»[2] p_DELINQ_NEXT. Каждая скрипка раскрашена в соответствии со значением самого признака для каждого наблюдения плотности. Это означает, что если бы наша модель была линейной и не имела взаимодействий, цветовой градиент на каждой скрипке был бы плавным от светлого к темному. Но мы замечаем другую картину. В скрипке p_DELINQ_NEXT наблюдаются значительные вариации цвета на вертикальных срезах графика. Это возможно только в том случае, когда p_DELINQ_NEXT влияет на прогнозы модели в сочетании с другими признаками. Например, модель может усвоить, что *если* LIMIT_BAL *ниже 20 000 долларов США, и если использование кредита превышает 50%, и если вероятность просрочки по модели предоставления кредита превышает 20%, то субъект строки данных, скорее всего, будет чернокожим или латиноамериканцем*. В то время как остаточные и состязательные модели помогают нам выявить отдельные проблемы с предвзятостью, SHAP делает еще один шаг вперед, помогая нам понять, что является причиной этой предвзятости.

Опосредованная предвзятость

Если закономерности, подобные тем, которые мы выявили, затрагивают лишь нескольких человек, они все равно могут нести вред. Но если мы видим, что они затрагивают более крупные группы людей, то, скорее всего, мы столкнулись с глобальной проблемой *опосредованной предвзятости* (proxy bias). Помните, что опосредованная предвзятость возникает, когда один признак или группа взаимодейст-

[2] Этот вид графика принято называть скрипичной диаграммой (violin plot) или просто скрипкой. — *Прим. пер.*

Книги для программистов: https://t.me/free_books_for_it

вующих признаков действуют как демографическая информация в нашей модели. Учитывая, что модели ML часто могут смешивать и сопоставлять признаки для создания скрытых концепций — и могут делать это по-разному для разных строк, — опосредованная предвзятость является довольно распространенной причиной предвзятых прогнозов модели.

Для поиска опосредованной предвзятости подойдут многие из инструментов, которые мы обсуждали выше, например, состязательные модели и SHAP. Мы можем начать с анализа значений взаимодействия признаков SHAP. (Вспомните передовые методы SHAP из глав 2 и 6.) Итоговым тестом на наличие опосредованной предвзятости могут быть состязательные модели. Если другая модель может точно предсказать демографическую информацию на основе прогнозов нашей модели, то наша модель кодирует демографическую информацию. Если мы включим входные признаки исходной модели в наши состязательные модели, то сможем использовать показатели атрибуции признаков. Это позволит понять, какие отдельные входные признаки несут в себе скрытую информацию. Кроме того, мы сможем применить другие методы для поиска опосредованных связей, созданных взаимодействиями. Одними из лучших состязательных моделей для поиска опосредованных связей являются старые добрые деревья решений. Поскольку модели ML склонны комбинировать и рекомбинировать признаки, построение обученного состязательного дерева решений часто помогает выявить более сложные опосредованные взаимодействия.

Как читатели могли заметить, состязательное моделирование — это сложный подход, требующий аккуратного и осознанного применения. Но мы надеемся, что убедили вас, что это также мощный инструмент для идентификации отдельных записей, субъекты которых могут подвергаться дискриминации в наших моделях. Кроме того, он помогает понять, как наши входные признаки связаны с информацией защищенной группы и опосредованной предвзятостью. Теперь мы переходим к важной работе по устранению предвзятости, которую удалось обнаружить в нашем примере модели кредитования.

Исправление предвзятости

Итак, мы установили наличие нескольких типов предвзятости в нашей модели; пришло время засучить рукава и попытаться исправить ее. К счастью, существует много различных инструментов, а также благодаря эффекту Расёмона много различных моделей. Сначала попробуем выполнить предварительную обработку входных признаков. Мы сгенерируем веса на уровне наблюдения для обучающих данных, чтобы положительные результаты были одинаково вероятны для всех демографических групп. Затем попробуем метод внутренней обработки, который иногда встречается под названием *справедливый XGBoost*, когда демографическая информация включается в расчет градиента XGBoost, чтобы ее можно было регуляризировать во время обучения модели. В целях постобработки мы обновим прогнозы, расположенные возле границы решения модели. Поскольку все три типа обработки могут вызвать сомнения по поводу разной трактовки и способов примене-

Книги для программистов: https://t.me/free_books_for_it

ния в некоторых приложениях, мы завершим этот раздел, обрисовав в общих чертах простой и эффективный метод выбора модели. Этот метод включает в себя поиск по различным наборам входных признаков и настройку гиперпараметров — с тем, чтобы найти модель с хорошим качеством прогнозов и минимальной предвзятостью. Для каждого подхода мы также рассмотрим различные компромиссы в отношении качества работы и устранения предвзятости.

Предварительная обработка

Первый метод устранения систематической ошибки модели, который мы попробуем, — предварительная обработка входных данных путем повторного взвешивания. Впервые он был предложен Фейсалом Камираном и Туном Колдерсом в их статье 2012 г. Data Preprocessing Techniques for Classification Without Discrimination (*https://oreil.ly/lAj08*). Идея повторного взвешивания состоит в том, чтобы уравнять средний результат по группам с помощью весов записей, а затем переобучить модель. Как мы увидим, до предварительной обработки обучающих данных средний результат или среднее значение переменной *y* сильно различается в разных демографических группах. Самая большая разница была для азиатов и чернокожих: средние результаты составили 0,107 и 0,400 соответственно. Это означает, что в среднем, если смотреть только на обучающие данные, вероятность дефолта для азиатов находилась в пределах диапазона, позволяющего получить дополнительный кредит, в то время как для чернокожих наблюдалось обратное. Их средний балл находился в диапазоне, соответствующем отказу от выдачи кредита. (Опять же, эти значения не всегда объективны или справедливы просто потому, что они записаны в цифровых данных.) После предварительной обработки мы увидели, что удалось в значительной степени сбалансировать как выходные данные модели, так и результаты тестирования на предвзятость.

Поскольку повторное взвешивание — это очень простой подход, мы решили реализовать его самостоятельно с помощью функции из следующего фрагмента кода[3]. Чтобы повторно взвесить наши данные, нам сначала нужно измерить средние показатели исходов — в целом и для каждой демографической группы. Затем мы определяем веса на уровне наблюдения или на уровне строк, которые уравновешивают частоту результатов в демографических группах. Веса наблюдений — это числовые значения, которые сообщают XGBoost и большинству других моделей машинного обучения, какой вес придать каждой строке данных во время обучения. Если строка имеет вес 2, это эквивалентно двукратному появлению в целевой функции, используемой для обучения XGBoost. Если мы сообщим XGBoost, что строка имеет вес 0,2, это равносильно тому, будто эта строка появляется в обучающих данных в одной пятой случаев. Если известны средний результат для каждой группы и их частота в обучающих данных, основной алгебраической задачей является определение весов строк, которые дают всем группам одинаковый средний результат в модели.

[3] Дополнительная реализация и пример использования повторного взвешивания описаны в статье AIF360 Detecting and Mitigating Age Bias on Credit Decisions (**https://oreil.ly/ypEQc**).

Книги для программистов: https://t.me/free_books_for_it

```
def reweight_dataset(dataset, target_name, demo_name, groups):
    n = len(dataset)
    # начальная общая частота выходов
    freq_dict = {'pos': len(dataset.loc[dataset[target_name] == 1]) / n,
                 'neg': len(dataset.loc[dataset[target_name] == 0]) / n}
    # начальная частота выходов по демографическим группам
    freq_dict.update({group: dataset[demo_name].value_counts()[group] / n
                      for group in groups})
    weights = pd.Series(np.ones(n), index=dataset.index)
    # определяем веса строк, балансирующие частоту выходов
    # по демографическим группам
    for label in [0, 1]:
        for group in groups:
            label_name = 'pos' if label == 1 else 'neg'
            freq = dataset.loc[dataset[target_name] == label][demo_name] \
                      .value_counts()[group] / n
            weights[(dataset[target_name] == label) &
                    (dataset[demo_name] == group)] *= \
                freq_dict[group] * freq_dict[label_name] / freq
    # возвращает вектор балансировочных весов
    return weights
```



Существует несколько видов взвешивания выборок. В XGBoost и в большинстве других моделей машинного обучения веса на уровне отдельного наблюдения интерпретируются как веса частоты, где вес наблюдения эквивалентен условному «количеству раз», когда она появляется в обучающих данных. Эта схема взвешивания берет свое начало в теории выборки опросов.

Другой основной тип взвешивания выборок основан на теории взвешенных наименьших квадратов. Иногда их называют прецизионными весами, они количественно определяют нашу неопределенность в отношении значений признаков наблюдения в предположении, что каждое наблюдение на самом деле представляет собой среднее значение нескольких базовых выборок. Эти два понятия весов не эквивалентны, поэтому важно знать, какое из них вы указываете при установке параметра `sample_weights`.

Функция `reweight_dataset` возвращает нам вектор весов наблюдения той же длины, что и обучающие данные, такой, что средневзвешенное значение результатов в данных в каждой демографической группе одинаково. Взвешивание помогает устранить проявления системных искажений в данных, обучая XGBoost тому, что разные типы людей должны иметь одинаковые средние показатели результатов. Достаточно лишь переобучить XGBoost с использованием весов строк из `reweight_dataset`. В нашем коде мы называем вектор обучающих весов `train_weights`. Когда вызываем функцию `DMatrix`, мы используем аргумент `weight=`, чтобы указать эти веса, уменьшающие предвзятость, после чего просто заново обучаем XGBoost:

```
dtrain = xgb.DMatrix(train[features],
                     label=train[target],
                     weight=train_weights)
```

Книги для программистов: https://t.me/free_books_for_it

В табл. 10.6 показаны исходные средние результаты, исходные значения AIR, а также предварительно обработанные средние результаты и AIR. После обучения XGBoost на невзвешенных данных мы обнаружили некоторые проблемные значения AIR. Первоначально AIR составлял около 0,73 для чернокожих и латиноамериканцев. Это слишком низкий показатель, который означает, что на каждую 1000 кредитных продуктов, которые модель одобряет белым людям, приходится примерно 730 заявок от латиноамериканцев или чернокожих людей. Такой уровень предвзятости вызывает беспокойство с этической точки зрения, что также может привести к юридическим проблемам в потребительском кредитовании, трудоустройстве или других областях, которые опираются на традиционные правовые стандарты для проверки предвзятости. Правило четырех пятых, пусть даже ошибочное и несовершенное, говорит нам, что мы не должны видеть значения ниже 0,8 для AIR. К счастью, в нашем случае метод взвешивания дает хорошие результаты.

Из табл. 10.6 видно, что удалось увеличить проблемные значения AIR для латиноамериканцев и чернокожих до приемлемых значений. Важно отметить, что при этом почти не снизился AIR для азиатов. Короче говоря, взвешивание снизило потенциальные риски предвзятости для чернокожих и латиноамериканцев, не увеличивая эти риски для других групп. Отразилось ли это на качестве работы нашей модели? Чтобы ответить на вопрос, мы ввели гиперпараметр `lambda` (рис. 10.3), который задает величину влияния схемы взвешивания. Когда значение `lambda` равно нулю, все наблюдения получают выборочный вес, равный единице. Когда гиперпараметр равен единице, все средние прогнозы равны, и мы получаем результаты, представленные в табл. 10.6. Как показано на рис. 10.3, действительно существует некоторый компромисс между увеличением влияния взвешивания и качеством работы по критерию F1 на валидационных данных. Далее мы пройдем по всему диапазону значений `lambda` и посмотрим на AIR для чернокожих и латиноамериканцев, чтобы лучше понять этот компромисс (рис. 10.4).

Результаты на рис. 10.4 показывают, что увеличение `lambda` выше 0,8 не приводит к значительным улучшениям AIR для чернокожих и латиноамериканцев. Если вер-

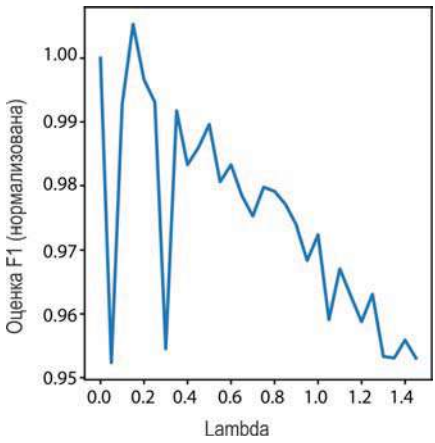


Рис. 10.3. Оценка F1 модели по мере увеличения влияния схемы взвешивания (цветное изображение доступно в цифровой версии книги)

Книги для программистов: https://t.me/free_books_for_it

Таблица 10.6. Исходные и предварительно обработанные средние результаты для демографических групп на тестовых данных

Демографическая группа	Исходный средний результат	Средний результат после предобработки	Исходный AIR	AIR после предобработки
Латиноамериканцы	0,398	0,22	0,736	0,861
Черные	0,400	0,22	0,736	0,877
Белые	0,112	0,22	1,000	1,000
Азиаты	0,107	0,22	1,012	1,010

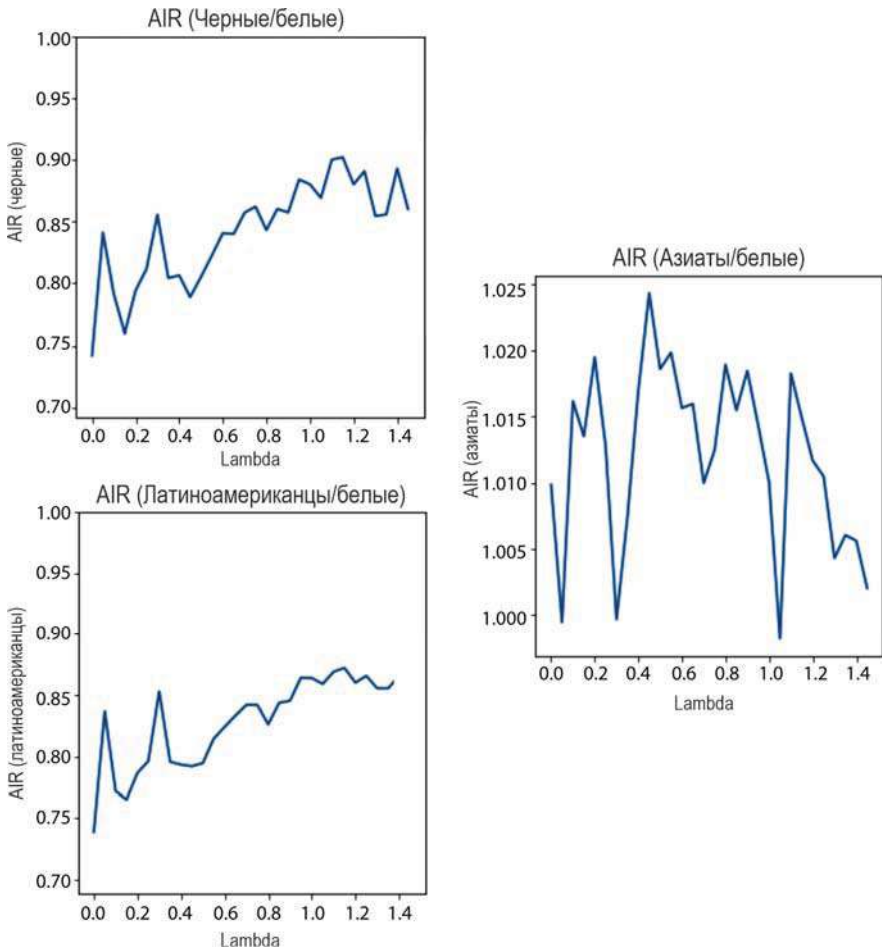


Рис. 10.4. Зависимость AIR модели от величины влияния схемы взвешивания

нуться к рис. 10.3, это означает, что мы испытаем падение примерно на 3% in silico. Если бы мы планировали развертывание этой модели, то выбрали бы это значение гиперпараметра для повторного обучения. Убедительная история, рассказанная на рис. 10.3 и 10.4, заключается в следующем: достаточно применить веса выборки

Книги для программистов: https://t.me/free_books_for_it

к нашему набору данных, чтобы выделить благоприятных чернокожих и латиноамериканских заемщиков и увеличить AIR для этих двух групп. При этом происходит лишь незначительное снижение показателей качества работы.

Как и почти все остальное в ML, устранение предвзятости и выбор оптимальных подходов — это эксперимент, а не механические действия. Их работа не гарантирована, и нам всегда нужно проверять, действительно ли они работают, — сначала на валидационных и тестовых данных, а затем в реальном мире. Очень важно помнить, что мы не знаем, как эта модель будет работать с точки зрения точности или предвзятости после ее развертывания в производстве. Мы всегда надеемся, что наши проверки in silico и тестовые оценки коррелируют с качеством работы модели в реальной жизни, но гарантированно успешных подходов нет. Мы очень надеемся, что наблюдаемое снижение качества in silico примерно на 5% исчезнет после развертывания модели из-за дрейфа данных, изменений в реальных операционных контекстах и других неожиданностей in vivo. Все это указывает на необходимость мониторинга как традиционных показателей качества, так и предвзятости после развертывания модели.

Взвешивание — это лишь один из возможных методов предварительной обработки. Существуют еще несколько популярных подходов. Предварительная обработка проста и интуитивно понятна. Как мы только что видели, она может привести к значительному уменьшению предвзятости модели с приемлемым компромиссом в точности. В AIF360 (*https://oreil.ly/rDdhC*) вы найдете примеры других надежных методов предварительной обработки.

Внутренняя обработка

Далее мы попробуем применить технику устранения предвзятости внутри модели. В последние годы было предложено множество интересных методов, в том числе те, которые используют состязательные модели, например Mitigating Unwanted Biases with Adversarial Learning (*https://oreil.ly/rFdZA*) или Fair Adversarial Gradient Tree Boosting (*https://oreil.ly/kZ0xB*). Идея этих состязательных подходов проста. Когда состязательная модель не может предсказать членство в демографической группе на основе прогнозов нашей основной модели, это хорошо — значит, наши прогнозы не содержат чрезмерной предвзятости. Как упоминалось ранее в этой главе, состязательные модели также помогают собирать локальную информацию о предвзятости. Строки, в которых модель-противник является наиболее точной, скорее всего, содержат больше всего демографической информации. Эти строки помогают выявить людей, которые наиболее подвержены предвзятости модели, а также сложные опосредованные зависимости, включающие несколько входных признаков, и другие локальные закономерности предвзятости.

Существуют еще и методы устранения предвзятости внутри модели, использующие только одну модель. В нашем примере мы воспользуемся одним из таких методов, т. к. обычно их реализация несколько проще. В отличие от использования второй модели, эти методы основаны на применении двойной целевой функции с подходом регуляризации. Например, в статье A Convex Framework for Fair Regression

Книги для программистов: https://t.me/free_books_for_it

(*https://oreil.ly/7dcHL*) авторы предлагают различные регуляризаторы, которые можно сочетать с моделями линейной и логистической регрессии, чтобы уменьшить предвзятость в отношении групп и отдельных лиц. Статья Learning Fair Representations (*https://oreil.ly/tgCE9*) также описывает измерение предвзятости в целевых функциях модели, но затем пытается создать новое представление обучающих данных, несущих в себе меньшую предвзятость.

Хотя эти два подхода в основном сосредоточены на простых моделях, т. е. на линейной регрессии, логистической регрессии и наивном байесовском методе, мы предпочитаем работать с деревьями и, в частности, с XGBoost. Оказывается, мы не единственные. Исследовательская группа American Express недавно опубликовала документ FairXGBoost: Fairness-Aware Classification in XGBoost (*https://oreil.ly/2gNo9*), который содержит инструкции и результаты экспериментов по введению члена регуляризации систематической ошибки в модели XGBoost с использованием существующей возможности XGBoost обучаться с помощью программируемых пользователем целевых функций. Именно так мы и будем выполнять обработку внутри модели, и, как вы скоро увидите, этот подход прост в реализации и дает хорошие результаты на данных нашего примера.



Прежде чем мы перейдем к техническим деталям, коду и результатам, следует упомянуть, что бóльшая часть обсуждаемой нами работы по регуляризации справедливости основана на фундаментальной статье Камишимы и др. Fairness-Aware Classifier with Prejudice Remover Regularizer (*https://oreil.ly/E_arn*).

Как работает выбранный нами подход? Целевые функции используются для измерения ошибок во время обучения модели, когда процедура оптимизации пытается минимизировать эту ошибку и найти наилучшие параметры модели. Основная идея методов регуляризации внутри модели состоит в том, чтобы включить меру систематической ошибки в общую целевую функцию модели. Следовательно, когда функция оптимизации применяется для расчета ошибки, а процесс оптимизации ML пытается минимизировать эту ошибку, это должно приводить к уменьшению измеренной систематической ошибки (т. е. предвзятости модели). Еще один вариант такой идеи — использование коэффициента члена систематической ошибки в целевой функции, или *гиперпараметра регуляризации*, чтобы можно было настраивать степень устранения смещения. Если читатели еще не знали, XGBoost поддерживает широкий спектр целевых функций, поэтому мы можем быть уверены, что способ измерения ошибки действительно соответствует реальной проблеме. Он также поддерживает полностью настраиваемые целевые функции, разрабатываемые пользователями.

Первым шагом в реализации нашего подхода внутренней обработки будет кодирование своей целевой функции. В следующем фрагменте кода мы определяем простую целевую функцию, которая сообщает XGBoost, как генерировать оценки.

1. Рассчитайте первую производную целевой функции по выходным данным модели (градиент, `grad`).
2. Рассчитайте вторую производную целевой функции по выходным данным модели (гессиан, `hess`).

Книги для программистов: https://t.me/free_books_for_it

3. Включите демографическую информацию (`protected`) в целевую функцию.
4. Управляйте силой регуляризации с помощью нового параметра (лямбда, `lambda`).

Мы также создали простую обертку для целевой функции, которая позволяет нам указать, какие группы мы хотим считать защищенным классом (мы стремимся уменьшить для них предвзятость путем регуляризации), а также силу регуляризации. Несмотря на простоту, оболочка обладает хорошей функциональностью. Она позволяет включать в защищенную группу несколько исходных демографических групп. Это важно, поскольку модели часто демонстрируют предвзятость в отношении более чем одной группы, и простая попытка исправить предвзятость для одной группы может ухудшить положение других групп. Возможность предоставлять собственные значения `lambda` великолепна, поскольку позволяет настраивать силу регуляризации. Как было показано в разделе «Предварительная обработка» этой главы, возможность настройки гиперпараметра регуляризации имеет решающее значение для поиска идеального компромисса с точностью модели.

Все вышеупомянутое уместилось примерно в 15 строк кода Python, и это одна из причин, по которым мы выбрали именно такой подход. Он использует преимущества платформы XGBoost, он довольно прост и, судя по данным нашего примера, увеличивает AIR для исторически маргинализированных групп меньшинств:

```
def make_fair_objective(protected, lambda):
    def fair_objective(pred, dtrain):

        # Целевая функция потерь перекрестной энтропии,
        # направленная на устранение несправедливости
        label = dtrain.get_label()
        pred = 1. / (1. + np.exp(-pred))
        grad = (pred - label) - lambda * (pred - protected)
        hess = (1. - lambda) * pred * (1. - pred)

        return grad, hess
    return fair_objective

protected = np.where((train['RACE'] == 'hispanic') | (train['RACE'] == 'black'), 1, 0)
fair_objective = make_fair_objective(protected, lambda=0.2)
```

После определения пользовательской целевой функции нам достаточно воспользоваться аргументом `obj=`, чтобы передать ее в функцию `train()` XGBoost. Если мы написали код правильно, надежные механизмы обучения и оптимизации XGBoost позаботятся обо всем остальном. Обратите внимание, как мало кода требуется для обучения модели с нашей пользовательской целевой функцией:

```
model_regularized = xgb.train(params,
                              dtrain,
                              num_boost_round=100,
                              evals=watchlist,
                              early_stopping_rounds=10,
                              verbose_eval=False,
                              obj=fair_objective)
```

Книги для программистов: https://t.me/free_books_for_it

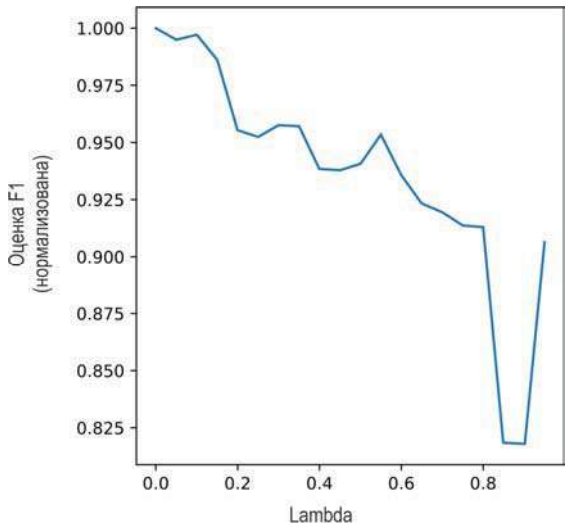


Рис. 10.5. Оценка F1 модели при увеличении `lambda`

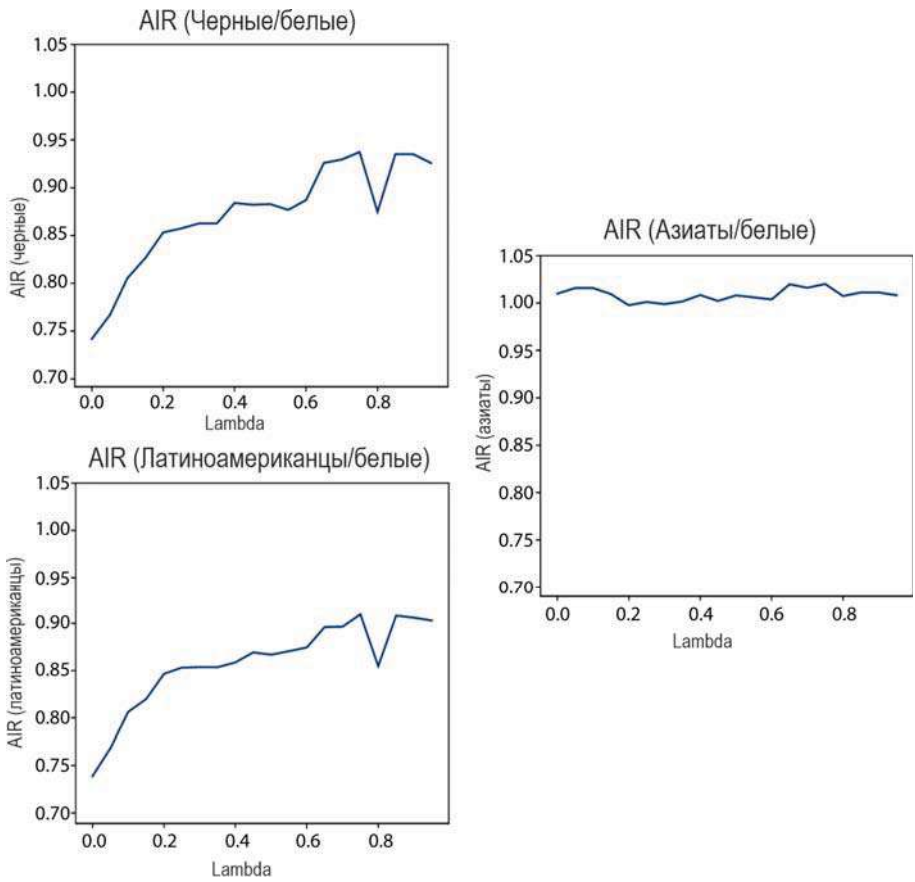


Рис. 10.6. Значения AIR по демографическим группам при увеличении коэффициента регуляризации `lambda`

Книги для программистов: https://t.me/free_books_for_it

Результаты выполнения кода на валидационном и тестовом наборах данных представлены на рис. 10.5 и 10.6. Чтобы проверить нашу гипотезу, мы воспользовались функцией-оберткой и обучили множество разных моделей с различными настройками `lambda`. На рис. 10.6 мы видим, что увеличение `lambda` действительно уменьшает предвзятость, что выражается в увеличении AIR для чернокожих и латиноамериканцев, тогда как AIR для азиатов остается примерно постоянным около хорошего значения 1. Мы можем увеличить AIR для групп, которые вызывают у нас наибольшее беспокойство в области потребительского кредитования, не допуская при этом потенциальной дискриминации других демографических групп. Именно такой результат мы хотим получить!

А как насчет компромисса между качеством работы и уменьшением предвзятости? То, что мы здесь увидели, довольно типично для нашего опыта. Существует диапазон значений `lambda`, выше которого AIR для чернокожих и латиноамериканцев существенно не увеличивается, но показатель F1 модели продолжает снижаться до уровня ниже 90% от качества исходной модели. Мы, вероятно, не будем использовать модель, в которой `lambda` достигает максимального уровня, поэтому согласимся на небольшое снижение качества работы модели in silico и пока неизвестное изменение качества in vivo.

Постобработка

Далее мы переходим к методам постобработки. Напомним, что методы постобработки применяются после того, как модель уже обучена, поэтому мы будем изменять выходные вероятности исходной модели, которую обучили в начале главы.

Техника, которая будет применена, называется постобработкой *с возможностью отклонения* (reject option); она восходит к статье (*https://oreil.ly/2rh4r*) Камирана и др., опубликованной в 2012 г. Наша модель имеет пороговое значение, где оценке выше этого значения назначается бинарный результат 1 (нежелательный результат для наших претендентов на получение кредита), а баллам ниже порогового значения назначается бинарный результат 0 (благоприятный исход). Постобработка с возможностью отклонения основана на идее, что для оценок, близких к пороговому значению, модель не уверена в правильности результата. Мы группируем все наблюдения, получившие оценку в узком интервале вокруг порогового значения, а затем переназначаем результаты для этих наблюдений, чтобы повысить общую справедливость прогнозов модели. Постобработку с возможностью отклонения легко интерпретировать и реализовать — мы смогли сделать это с помощью другой относительно простой функции:

```
def reject_option_classification(dataset, y_hat, demo_name,
                                 protected_groups,
                                 reference_group, cutoff,
                                 uncertainty_region_size):
    # В области неопределенности, близкой к пороговому значению принятия
    # решения, замените прогнозы защищенной группы на благоприятное решение
    # и прогнозы референтной группы на неблагоприятное решение
```

Книги для программистов: https://t.me/free_books_for_it

```
    new_predictions = dataset[y_hat].values.copy()

    uncertain = np.where(
        np.abs(dataset[y_hat] - cutoff) <= uncertainty_region_size, 1, 0)
    uncertain_protected = np.where(
        uncertain & dataset[demo_name].isin(protected_groups), 1, 0)
    uncertain_reference = np.where(
        uncertain & (dataset[demo_name] == reference_group), 1, 0)

eps = 1e-3

    new_predictions = np.where(uncertain_protected,
                               cutoff - uncertainty_region_size - eps,
                               new_predictions)
    new_predictions = np.where(uncertain_reference,
                               cutoff + uncertainty_region_size + eps,
                               new_predictions)
    return new_predictions
```

На рис. 10.7 можно увидеть эту технику в действии. Гистограммы показывают распределение оценок модели для каждой расовой группы до и после постобработки. В небольшой окрестности значений около 0,26 (исходное пороговое значение модели) мы выполнили постобработку прогнозов всех чернокожих и латиноамериканцев и получили для них положительный прогноз, присвоив им оценку в нижней части диапазона. Кроме того, мы получили отрицательный прогноз для белых людей в этой зоне неопределенности и оставили без изменений оценки азиатов. Теперь выясним, как данный метод влияет на точность модели и AIR.

Результаты эксперимента оказались именно такими, на которые мы надеялись: нам удалось повысить AIR для чернокожих и латиноамериканцев до уровня выше 0,9, оставив при этом AIR для азиатов на уровне около 1,00 (табл. 10.7). Нам пришлось заплатить снижением показателя F1 на 6%. Мы не считаем это существенным снижением, но если бы нас это беспокоило, мы могли бы уменьшить размер зоны неопределенности, чтобы найти более выгодный компромисс.

Таблица 10.7. Оценки F1 до и после постобработки, и значения AIR на валидационных данных

Модель	Оценка F1	AIR (черные)	AIR (латиноамериканцы)	AIR (азиаты)
Исходная	0,574	0,736	0,736	1,012
С постобработкой	0,541	0,923	0,902	1,06

Выбор модели

Последний метод, который мы обсудим, — выбор модели с учетом справедливости. Точнее, мы выполним простой выбор признаков и случайную настройку гиперпараметров, отслеживая при этом качество работы модели и AIR. Читатели почти

Книги для программистов: https://t.me/free_books_for_it

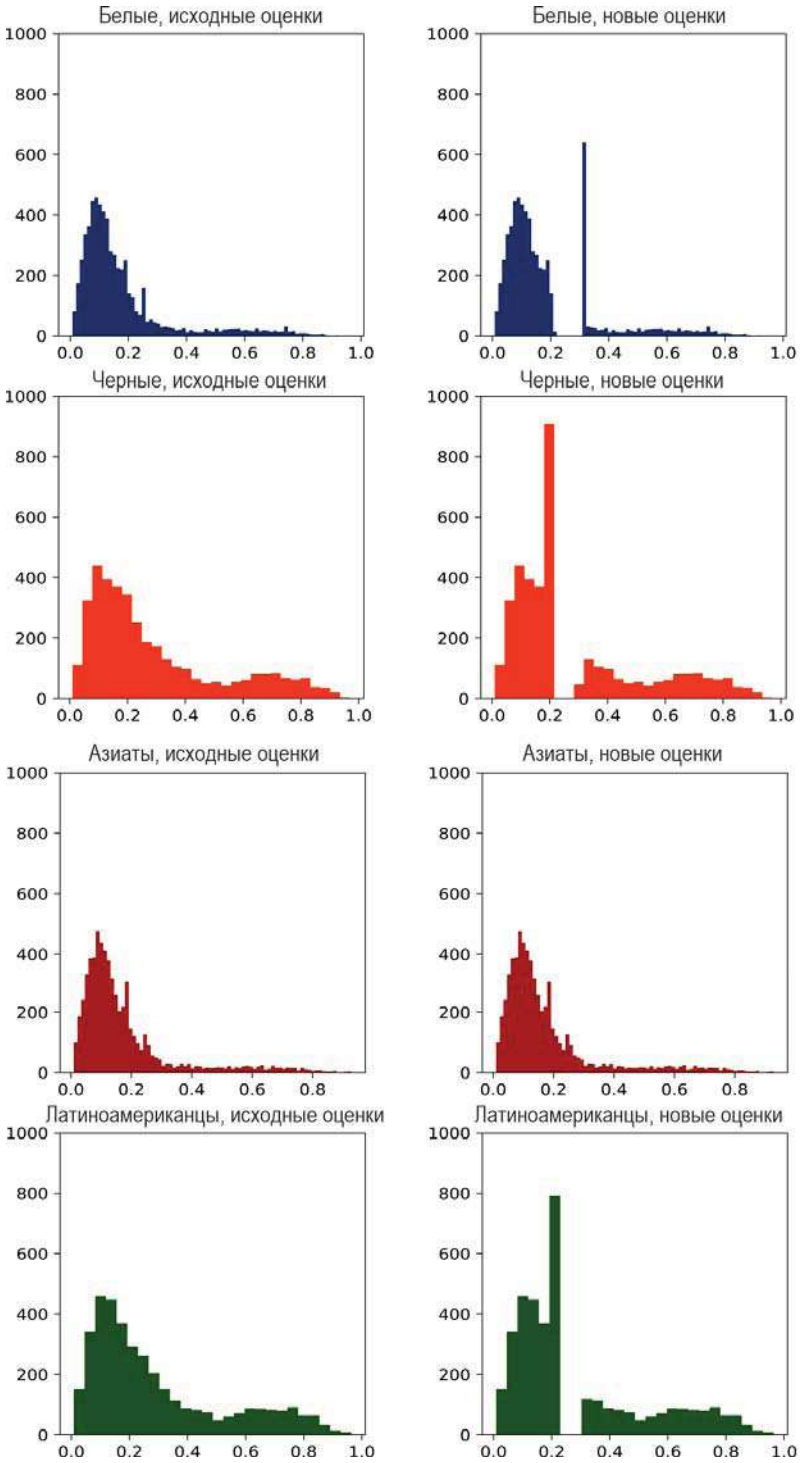


Рис. 10.7. Оценки модели для каждой демографической группы до и после применения постобработки с отклонением

Книги для программистов: https://t.me/free_books_for_it

наверняка уже выполняют эти шаги, когда дело доходит до оценки качества, так что этот метод почти не нуждается в дополнительных трудозатратах. Еще одним преимуществом выбора модели как метода исправления ситуации является то, что он вызывает наименьшее количество проблем, связанных с несопоставимым отношением. (На другом конце спектра находится постобработка с возможностью отклонения, описанная в предыдущем разделе, в которой мы принудительно меняли результаты модели в зависимости от статуса защищенной группы каждого наблюдения.)



Случайный поиск по наборам признаков и настройкам гиперпараметров часто выявляет модели с улучшенными показателями справедливости и качеством работы не хуже, чем у базовой модели.

В этом разделе показателями качества работы будут выступать F1 и AUC. По нашему опыту, оценка моделей по нескольким показателям качества увеличивает вероятность хороших результатов in vivo. Еще одним преимуществом расчета показателей F1 и AUC является то, что первый показатель измеряется на основе результатов модели, а второй использует только выходные вероятности. Если в будущем мы захотим изменить порог принятия решений в нашей модели или передать оценки модели в качестве входных данных в другой процесс, мы не пожалеем, что отслеживали AUC.

Еще одно замечание, прежде чем мы углубимся в выбор модели, — от нас зависит гораздо больше, чем просто выбор признаков и настройка гиперпараметров. Выбор модели также может означать выбор между конкурирующими архитектурами или выбор между различными методами устранения предвзятости. В заключительном разделе текущей главы мы обобщим все наши результаты, чтобы подготовиться к окончательному выбору модели, но в этом разделе мы сосредоточимся только на признаках и гиперпараметрах.

По нашему опыту, выбор признаков может быть мощным методом исправления ошибок. Но лучше всего он работает, когда его осуществляют эксперты в предметной области и при доступности альтернативных источников данных. Например, эксперт по соблюдению нормативных требований в банке может знать, что признак в модели кредитования можно заменить альтернативным признаком, кодирующим меньшую историческую предвзятость. У нас нет возможности получить доступ к этим альтернативным признакам. В нашем распоряжении есть только возможность *удаления* признаков из модели, поэтому мы проверим эффект от удаления каждого признака по отдельности, сохраняя при этом исходные гиперпараметры. Между выбором признаков и настройкой гиперпараметров нам предстоит обучить множество различных моделей, поэтому мы будем использовать пятикратную перекрестную проверку с использованием наших исходных обучающих данных. Если мы выберем вариант с наилучшим качеством на валидационных данных, мы рискуем выбрать модель, которая работает лучше всего по случайному стечению обстоятельств.

Книги для программистов: https://t.me/free_books_for_it



Хотя эффект Расёмона может означать, что у нас есть выбор из множества хороших моделей, надо помнить, что это явление также может быть признаком нестабильности нашей исходной модели. Если существует множество моделей с настройками, близкими к исходной модели, но которые при этом работают иначе, чем исходная модель, это указывает на проблемы недостаточной специфичности. Исправленные модели также необходимо протестировать на предмет проблем со стабильностью, безопасностью и качеством работы. Дополнительную информацию см. в главах 3, 8 и 9.

После обучения новых моделей с использованием перекрестной проверки мы смогли добиться увеличения AIR для чернокожих и латиноамериканцев, а также небольшого уменьшения AUC для перекрестной проверки модели. Самым неприятным признаком был столбец `PAY_AMT5`, поэтому мы продолжим случайный подбор гиперпараметров без этого признака.



Можно более тщательно подходить к выбору признаков, используя состязательные модели и объяснимые методы ИИ. В качестве источника идей вам могут пригодиться статья Explaining Measures of Fairness (*https://oreil.ly/SLn_8*) и связанный с ней блокнот Jupyter Notebook от создателя SHAP, а также Automating Procedurally Fair Feature Selection in Machine Learning (*https://oreil.ly/YSKnM*) от Белица и др.

Чтобы выбрать гиперпараметры новой модели, мы будем использовать случайный поиск по сетке с помощью API scikit-learn. Поскольку нам нужно осуществлять перекрестную проверку AIR на протяжении всего этого процесса, мы создадим функцию для передачи оценки в scikit-learn. Чтобы упростить код, мы отслеживаем здесь только AIR для чернокожих клиентов, поскольку на протяжении всего нашего анализа он коррелировал с AIR латиноамериканцев, но средний показатель AIR по защищенным группам, вероятно, предпочтительнее. В следующем фрагменте кода показано, как мы использовали глобальные переменные и интерфейс `make_scorer()`, чтобы добиться этого:

```
fold_number = -1

def black_air(y_true, y_pred):
    global fold_number
    fold_number = (fold_number + 1) % num_cv_folds

    model_metrics = perf_metrics(y_true, y_score=y_pred)
    best_cut = model_metrics.loc[model_metrics['f1'].idxmax(), 'cutoff']

    data = pd.DataFrame({'RACE': test_groups[fold_number],
                         'y_true': y_true,
                         'y_pred': y_pred},
                         index=np.arange(len(y_pred)))

    disparity_table = fair_lending_disparity(data, y='y_true', yhat='y_pred',
                                             demo_name='RACE',
                                             groups=race_levels,
                                             reference_group='white',
                                             cutoff=best_cut)
```

Книги для программистов: https://t.me/free_books_for_it

```
    return disparity_table.loc['black']['AIR']

scoring = {
    'AUC': 'roc_auc',
    'Black AIR': sklearn.metrics.make_scorer(black_air, needs_proba=True)
}
```

Затем мы определили разумные границы сетки гиперпараметров и построили 50 новых моделей:

```
parameter_distributions = {
   'n_estimators': np.arange(10, 221, 30),
   'max_depth': [3, 4, 5, 6, 7],
   'learning_rate': stats.uniform(0.01, 0.1),
   'subsample': stats.uniform(0.7, 0.3),
   'colsample_bytree': stats.uniform(0.5, 1),
   'reg_lambda': stats.uniform(0.1, 50),
   'monotone_constraints': [new_monotone_constraints],
   'base_score': [params['base_score']]
   }

grid_search = sklearn.model_selection.RandomizedSearchCV(
    xgb.XGBClassifier(random_state=12345,
                      use_label_encoder=False,
                      eval_metric='logloss'),
    parameter_distributions,
    n_iter=50,
    scoring=scoring,
    cv=zip(train_indices, test_indices),
    refit=False,
    error_score='raise').fit(train[new_features], train[target].values)
```

Результаты нашей процедуры случайного выбора модели показаны на рис. 10.8. Каждая модель представляет собой точку на графике со значениями AIR перекрестной проверки на оси *x* и AUC перекрестной проверки на оси *y*. Здесь мы нормализовали точность модели относительно базового значения. Этот трюк позволяет легко делать заявления типа «эта альтернативная модель показывает снижение AUC на 2% по сравнению с исходной моделью». Но как нам выбрать из распределения модель для развертывания?

Общая проблема методов исправления предвзятости заключается в том, что они часто просто перемещают предвзятость из одной демографической группы в другую. Например, сейчас в США женщины иногда имеют преимущество при принятии решений о кредите и трудоустройстве. Действуя сугубо механически, метод исправления предвзятости может резко уменьшить долю благоприятных прогнозов для женщин в процессе их увеличения для других групп, страдающих от системной предвзятости, но это явно не тот результат, который действительно кому-то нужен. Если одна группа пользуется непропорциональным предпочтением и метод исправления предвзятости устраняет его — отлично. Если, с другой стороны, одна

Книги для программистов: https://t.me/free_books_for_it

группа имеет некоторое преимущество, и процедура исправления предвзятости в конечном итоге наносит ей вред, увеличивая AIR или другие статистические показатели для других групп, это, очевидно, не очень хорошо. В следующем разделе вы увидите, как эти два альтернативных пути сочетаются с другими методами исправления предвзятости, применяемыми в текущей главе.

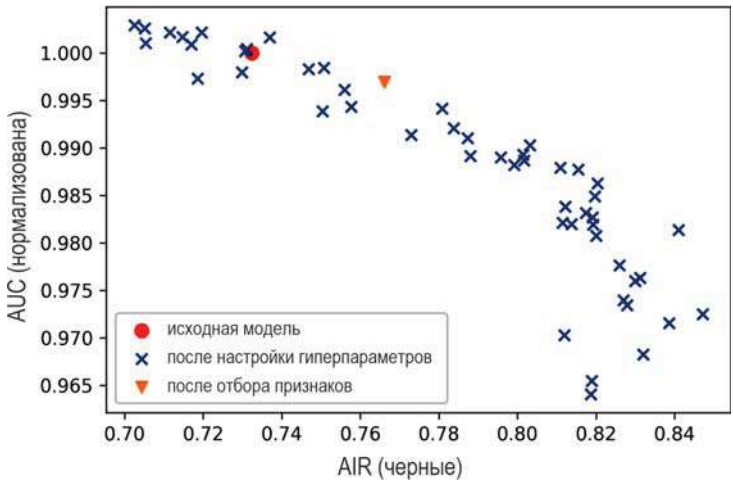


Рис. 10.8. Нормализованная точность и AIR (для чернокожих) каждой модели после выбора признаков и настройки гиперпараметров



Каждый раз, когда вы оцениваете несколько моделей на одном и том же наборе данных, вы должны опасаться переобучения и множественных сравнений. Используйте лучшие проверенные подходы, такие как повторное использование данных, перекрестная проверка, бутстреппинг, отложенные данные и мониторинг после развертывания, чтобы сохранить обобщающую способность модели.

Заключение

В табл. 10.8 мы представили показатели всех моделей, обученных в этой главе. Мы решили сосредоточиться на двух показателях точности модели — F1 и AUC, а также на несовпадении двух показателей систематической ошибки модели — AIR и доли ложноположительных результатов (FPR).

Результаты впечатляют: многие рассмотренные нами методы исправления предвзятости способны обеспечить значительное улучшение показателей AIR и различий в FPR для чернокожих и латиноамериканских заемщиков без серьезного негативного воздействия на AIR азиатов. Причем это возможно при совсем незначительных изменениях в качестве работы модели.

Как выбрать наиболее подходящий метод исправления модели в приложении высокого риска? Надеемся, что данная глава убедила читателей в необходимости пробовать много разных методов. В конечном итоге окончательное решение остается за законом, руководством бизнеса и разнообразной командой заинтересованных

Книги для программистов: https://t.me/free_books_for_it

Таблица 10.8. Сравнение результатов применения различных методов устранения предвзятости

Мера	Исходная модель	Предобработка (взвешивание)	Внутренняя обработка (регуляризация, lambda = 0,2)	Постобработка (исключение, window = 0,1)	Выбор модели
AUC	0,798021	0,774183	0,764005	0,794894	0,789016
F1	0,558874	0,543758	0,515971	0,533964	0,543147
AIR, азиаты	1,012274	1,010014	1,001185	1,107676	1,007365
AIR, черные	0,735836	0,877673	0,851499	0,901386	0,811854
AIR, латиноамериканцы	0,736394	0,861252	0,851045	0,882538	0,805121
Разница AIR, азиаты	0,872567	0,929948	0,986472	0,575248	0,942973
Разница AIR, черные	1,783528	0,956640	1,141044	0,852034	1,355846
Разница AIR, латиноамериканцы	1,696062	0,899065	1,000040	0,786195	1,253355

сторон, которую вы должны были сформировать. В регулируемых областях деятельности, где предвзятое и неравное отношение строго запрещены законом и нормативами, существуют жесткие ограничения вариантов выбора. На самом деле в этой ситуации вам доступны только различные версии выбора моделей. Если вы действуете за пределами строго регулируемой отрасли или организации, у вас есть гораздо более широкий выбор стратегий исправления[4]. Мы, скорее всего, выбрали бы вариант предварительной обработки, учитывая минимальное снижение качества модели по сравнению с внутренней обработкой, а также потому, что постобработка выталкивает неравенство некоторых показателей качества по группам за пределы допустимого диапазона.

Независимо от того, используем ли мы выбор модели в качестве метода уменьшения предвзятости и есть ли у нас на выбор разные модели с предварительной, внутренней и постобработкой, практические правила выбора исправленной модели заключаются в следующем.

1. Сократите набор моделей до тех, которые работают достаточно хорошо для удовлетворения потребностей бизнеса, например снижение показателей качества не более 5% от исходной модели.
2. Среди этих моделей выберите ту, которая наиболее близка к следующим требованиям:
   - устраняет предвзятость во всех изначально неблагополучных группах, например AIR всех неблагополучных групп увеличивается до $\geq 0{,}8$;
   - не дискриминирует какую-либо изначально благополучную группу, например, AIR ни одной изначально благополучной группы не падает ниже 0,8.

[4] Не забудьте о применении дерева решений для устранения предвзятости (**https://oreil.ly/vDv4T**, слайд 40).

Книги для программистов: https://t.me/free_books_for_it

3. Проконсультируйтесь с деловыми партнерами, экспертами по правовым вопросам и соблюдению требований, а также представителями различных заинтересованных сторон в рамках процесса отбора.

Если мы обучаем модель, которая потенциально может повлиять на людей (а большинство моделей так и делают), у нас есть этическое обязательство проверить ее на предмет предвзятости. И когда обнаруживаем предвзятость, нам необходимо смягчить или исправить ее. В этой главе мы рассмотрели техническую часть процессов управления предвзятостью. Для правильного устранения предвзятости необходимо продлить сроки развертывания, организовать общение между различными заинтересованными сторонами, а также проделать большую работу по повторному обучению и тестированию моделей и конвейеров ML. Мы уверены, что если перестать спешить, обратиться за помощью и мнением экспертов в предметной области и применить научный подход, то получится решить проблемы предвзятости в реальном мире и развернуть точные и минимально предвзятые модели.

Ресурсы

Примеры кода

- Книга *Machine Learning for High Risk Applications*: **https://oreil.ly/machine-learning-high-risk-apps-code**

Инструменты для борьбы с предвзятостью

- aequitas: **https://oreil.ly/JzQFh**
- AI Fairness 360
  - Python: **https://oreil.ly/sYmc-**
  - R: **https://oreil.ly/J53bZ**
- Algorithmic Fairness: **https://oreil.ly/JNzqk**
- fairlearn: **https://oreil.ly/jYjCi**
- fairml: **https://oreil.ly/DCkZ5**
- fairmodels: **https://oreil.ly/nSv8B**
- fairness: **https://oreil.ly/Dequ9**
- solas-ai-disparity: **https://oreil.ly/X9fd6**
- tensorflow/fairness-indicators: **https://oreil.ly/dHBSL**
- Themis: **https://oreil.ly/zgrvV**

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 11

Атака красной команды на XGBoost

В главе 5 мы описали ряд понятий, связанных с безопасностью моделей ML. Теперь применим их на практике. В этой главе мы объясним, как взломать собственные модели, чтобы добавить методы красной команды в свой набор инструментов отладки моделей. Основная идея главы заключается в том, что, если мы знаем наперед, каким атакам могут подвергнуть нашу модель хакеры, то можем воспроизвести их и разработать эффективную защиту. Начнем с традиционного раздела уточнения понятий, в котором повторно рассмотрим распространенные атаки на модели ML и меры противодействия, затем проанализируем примеры атак на классификатор XGBoost, обученный на структурированных данных[1]. Далее мы представим две модели XGBoost, одна из которых обучена с использованием стандартного необъяснимого метода, а другая — с ограничениями и высокой степенью регуляризации L2. Мы будем использовать эти две модели, чтобы объяснить атаки и проверить, являются ли прозрачность и регуляризация L2 адекватными мерами противодействия взлому. После перейдем к атакам, которые могут быть осуществлены внешними злоумышленниками против API необъяснимой модели ML: извлечение модели и атаки с состязательными примерами. Мы попробуем инсайдерские атаки, которые включают внесение преднамеренных изменений в конвейер ML: отравление данных и бэкдоры моделей. Напоминаем, что примеры кода этой главы доступны в Интернете. А теперь давайте начнем — не забудьте приготовить шапку из фольги и позаимствовать состязательное настроение из главы 5.



Интернет и научная литература изобилуют примерами и инструментами атак на компьютерное зрение и языковые модели. Достойное изложение этих обширных тем можно найти в следующих работах:

- Adversarial Attacks in Computer Vision: An Overview (*https://oreil.ly/7CPnm*);
- Privacy Considerations in Large Language Models (*https://oreil.ly/mesVW*).

В этой главе упомянутые идеи переносятся на широко используемые древовидные модели и структурированные данные. В главе 5 рассматриваются проблемы безопасности ML в более общем плане. В главах 1, 3 и 4 представлены многочисленные средства снижения рисков и средства контроля процессов, которые также полезны для обеспечения безопасности ML во всех типах моделей.

[1] В Интернете есть примеры атак на модели компьютерного зрения, но учебные примеры, основанные на библиотеке cleverhans (**https://oreil.ly/4Xifu**), тоже очень хороши для начала.

Книги для программистов: https://t.me/free_books_for_it

Уточнение понятий

Стоит повторить, почему нас интересуют атаки с использованием моделей ML. Злоумышленники могут манипулировать моделями, изменять их или уничтожать, причиняя вред невинным людям. Вообще говоря, инциденты технологической безопасности являются основным источником вреда операторам, пользователям и широкой общественности. Злоумышленники могут стремиться как заполучить выгоду для себя, так и причинить вред другим; они могут заниматься корпоративным шпионажем, красть интеллектуальную собственность и данные. Но мы не хотим, чтобы наши модели ML были легкой добычей, и намерены предпринять ответные действия! В главе 5 мы назвали этот подход *состязательным мышлением*. Хотя наша модель ML может иметь прекрасный код Python и должна принести нашей организации миллионы долларов, ее развертывание неизбежно связано с юридической ответственностью, уязвимостью безопасности и наличием конечных точек API, которые привлекут внимание хакеров. Это неизбежно, особенно в случае с важными общедоступными системами машинного обучения. Давайте не будем наивными. Хотим мы того или нет, но нам придется проделать тяжелую работу, необходимую для проверки модели на наличие уязвимостей, которые могут привести к утечке обучающих данных, утечке самих моделей или позволить злоумышленникам обмануть систему, лишив нас денег, интеллектуальной собственности или репутации. Для начала освежим в памяти некоторые определения и понятия из главы 5.

Триада CIA

Напомним, что в целом мы делим инциденты информационной безопасности на три категории, определяемые триадой CIA: атаки на конфиденциальность (confidentiality), целостность (integrity) и доступность (availability):

Атаки на конфиденциальность

Направлены на нарушение конфиденциальности некоторых данных, связанных с моделью ML, обычно логики модели или обучающих данных. Атаки с извлечением модели раскрывают модель, а атаки вывода о членстве раскрывают обучающие данные.

Атаки на целостность

Призваны скомпрометировать поведение модели, как правило, чтобы изменить прогнозы так, как это выгодно злоумышленнику. Состязательные примеры, отравление данных и бэкдор-атаки нарушают целостность модели.

Атаки на доступность

Мешают пользователю модели получить к ней своевременный или полноценный доступ. Атаки «энергетической губки», которые замедляют работу нейронных сетей, являются разновидностью атаки на доступность. Некоторые исследователи также рассматривают алгоритмическую дискриминацию как разновидность атаки на доступность, поскольку группы меньшинств не получают от модели тех

Книги для программистов: https://t.me/free_books_for_it

же услуг, что и группы большинства. Однако это, как правило, традиционные атаки типа «отказ в обслуживании», нацеленные на хостинговый сервис, на котором работает модель, и неспецифичные для ML. Мы не будем проводить атаку на доступность, но рекомендуем вам проконсультироваться со своими поставщиками хостинговых услуг и убедиться, что в ваших общедоступных моделях машинного обучения предусмотрены стандартные меры противодействия для смягчения атак на доступность.

После этого краткого напоминания о триаде CIA мы обратимся к более подробной информации о каждой из запланированной нами атак красной команды.

Атаки

Атаки на модели ML можно упрощенно разделить на две основные группы: *внешние* и *инсайдерские*. Внешние атаки с наибольшей вероятностью будут реализованы со стороны внешнего злоумышленника. Допустим, мы развернули модель как API, но проявили некоторую небрежность в вопросах безопасности. Предположим, что внешний пользователь может взаимодействовать с моделью как с необъяснимой сущностью, делать это анонимно и что ему доступно разумное количество взаимодействий по отправке данных в модель. В этих условиях внешний злоумышленник может извлечь базовую логику нашей модели с помощью атаки извлечения модели. С этой логикой или без нее (хотя с ней это проще и опаснее) злоумышленник может затем начать создавать состязательные примеры, которые выглядят как обычные данные, но вызывают неожиданные результаты. Имея правильные состязательные примеры, злоумышленник может играть на нашей модели, как на скрипке. Если хакерам удастся провести две предыдущие атаки, они могут стать более смелыми и попробовать более сложную и опасную атаку: вывод о членстве. Рассмотрим различные типы внешних атак более подробно.

Извлечение модели

Эта атака ставит под угрозу конфиденциальность модели ML. Чтобы провести атаку с извлечением модели, хакер отправляет данные в API прогнозирования, получает прогнозы и строит суррогатную модель, обученную на отправленных данных и полученных прогнозах для реверс-инжиниринга (обратной разработки) модели. Благодаря этой информации он может раскрыть частные бизнес-процессы и логику принятия решений. Извлеченная модель также представляет собой отличный испытательный стенд для последующих атак.

Состязательные примеры

Это атака на целостность. Она ставит под угрозу правильность прогнозов модели. Чтобы выполнить состязательную атаку, хакер наблюдает, как модель реагирует на входные данные. В системах компьютерного зрения информация о градиенте часто используется для точной подгонки изображений, которые вызывают странные реакции модели. В случае структурированных данных хакер может использовать индивидуальное условное ожидание (ICE) или генетические алгоритмы, чтобы найти строки данных, которые вызывают неожиданные прогнозы модели.

Книги для программистов: https://t.me/free_books_for_it

Вывод о членстве

Атака на конфиденциальность, направленная на компрометацию обучающих данных модели. Это сложная атака, требующая наличия двух моделей. Первая представляет собой суррогатную модель, аналогичную тем, которые обучены атаке извлечения модели. Затем модель второго этапа обучается определять, входит ли строка данных в обучающие данные суррогатной модели или нет. Когда на вход этой модели второго этапа подают строку данных, она может проверить, присутствует ли такая строка в обучающих данных суррогатной модели или нет, и часто может экстраполировать, была ли эта строка также в обучающих данных исходной модели.

Теперь об инсайдерских атаках. К сожалению, мы не всегда можем доверять нашим коллегам, консультантам или подрядчикам. И что еще хуже, людей могут вынудить к совершению плохих поступков, хотят они того или нет. При атаке с отравлением данных кто-то изменяет обучающие данные таким образом, чтобы злоумышленник или другие люди могли манипулировать моделью позже. При бэкдор-атаке хакер меняет код модели, чтобы впоследствии получить доступ к модели несанкционированными способами. Как в случае отравления данных, так и в случае бэкдор-атак, наиболее вероятно, что злоумышленник будет стремиться получить финансовую выгоду и соответствующим образом изменить данные или код скоринга. Однако возможен сценарий, когда злоумышленник стремится нанести вред другим, а не извлечь выгоду для себя.

Отравление данных

Атака на целостность, которая изменяет обучающие данные для влияния на будущие результаты модели. Для проведения атаки злоумышленникам нужен только доступ к обучающим данным модели. Они пытаются внести в данные изменения, которые надежно изменят прогнозы модели таким образом, чтобы злоумышленники или их сообщники впоследствии могли получить нужные результаты.

Бэкдоры

Атаки на целостность, которые изменяют код вывода модели. Цель бэкдор-атаки — ввести новые ветки кода в сложный клубок коэффициентов и правил «если..., то...», который представляет собой развернутая модель ML. После того как новая ветка кода будет внедрена в механизм оценки, она станет доступна для тех, кто знает, как ее запустить, например отправив нереалистичные комбинации данных в API прогнозирования.

Мы не возвращаемся к атакам уклонения и имитации, но напомним, что им был посвящен практический пример в главе 5. Согласно нашим исследованиям на сегодняшний день атаки уклонения и имитации являются наиболее распространенными видами атак на ML. Обычно они направлены на системы безопасности, фильтрации или платежей, основанные на ML. В компьютерном зрении обычно применяются какие-то физические манипуляции с системой ML, например ношение реалистичной маски или маскировочной одежды. В случае структурированных данных эти атаки направлены на изменение строки данных, чтобы она имела схо-

Книги для программистов: https://t.me/free_books_for_it

жие значения (имитация) или разные значения (уклонение) по сравнению с каким-либо пользователем модели. Надо сказать, что уклонение от моделей ML, предназначенных для обнаружения мошенничества, — это давняя игра в кошки-мышки между мошенниками и финансовыми учреждениями, и это, вероятно, наиболее распространенное приложение, в котором мы сталкиваемся с атаками уклонения, основанными на манипулировании структурированными данными.

Контрмеры

Большинство атак основаны на том, что модели ML являются чрезмерно сложными, нестабильными, переобученными и необъяснимыми. Слишком сложная и необъяснимая модель вредна с точки зрения безопасности, потому что владельцам системы будет трудно понять, не вмешиваются ли в ее работу посторонние. Нестабильность важна для успешных атак, поскольку она приводит к сценариям, в которых незначительные изменения входных данных могут привести к резким и неожиданным изменениям прогнозов модели. Переобучение приводит к нестабильным моделям и способствует атакам вывода о членстве. Если модель переобучена, она ведет себя совершенно иначе на новых данных. Мы можем использовать разницу в точности прогнозов, чтобы определить, использовались ли конкретные строки данных для обучения модели. Исходя из этих соображений, мы попробуем применить две простые контрмеры:

Регуляризация L2

Штраф, налагаемый на квадрат суммы коэффициентов модели в функции ошибок модели или на какую-либо другую меру сложности модели. Сильная регуляризация L2 не позволяет какому-либо коэффициенту, правилу или взаимодействию стать слишком большим и важным. Если поведение модели не зависит только от одного признака или взаимодействия, построить состязательные примеры труднее. Регуляризация L2 также стремится уменьшить все коэффициенты модели, делая ее прогнозы более стабильными и менее подверженными резким колебаниям. Также известно, что регуляризация L2 улучшает обобщающие возможности моделей, что должно помочь противостоять атакам вывода о членстве.

Ограничения монотонности

Они делают модель более стабильной и интерпретируемой, что является общим средством защиты от атак на ML. Если модель легко интерпретируется, это меняет весь ее профиль безопасности. Мы знаем, как должна вести себя модель, и нам легче определить, когда ею манипулируют. Атаки на конфиденциальность теряют свою остроту, поскольку всем известно, как работает модель, подчиняющаяся реальности. Если ограничения не позволяют модели генерировать неожиданные прогнозы, то на самом деле нет возможности провести атаку с использованием состязательного примера. Если ограничения обеспечивают реалистичное поведение модели, то отравление данных, скорее всего, будет менее эффективным. Ограничения также должны способствовать обобщению, усложняя вывод о членстве.

Книги для программистов: https://t.me/free_books_for_it

Мы также надеемся, что между этими двумя общими контрмерами существует некоторая синергия. И регуляризация L2, и ограничения повышают стабильность модели. Используя их, мы пытаемся исключить значительные изменения в выходных данных модели из-за небольших изменений на входе. В частности, что касается ограничений, мы добиваемся того, чтобы наша модель просто не могла нас удивить. Ограничения означают, что модель должна подчиняться очевидной причинной закономерности, и мы надеемся, что состязательные примеры будет гораздо труднее найти, а отравление данных будет менее разрушительным. Оба подхода должны также уменьшить переобучение и обеспечить некоторую защиту от ошибочного вывода о членстве.

К другим важным контрмерам относятся ограничение частоты запросов (*https://oreil.ly/W3imH*), аутентификация (*https://oreil.ly/bBR1j*), робастное ML (*https://oreil.ly/u4ir7*) и дифференциальная приватность (*https://oreil.ly/Xkf7Z*). Ограничение частоты запросов замедляет прогнозирование, если кто-то слишком часто или подозрительным образом взаимодействует с API. Аутентификация предотвращает анонимное использование, что в целом должно сделать атаки менее доступными и привлекательными. Робастные подходы ML создают модели, специально разработанные для большей устойчивости к состязательным примерам и искажению данных. Дифференциальная приватность методично искажает обучающие данные, чтобы скрыть их, если произойдет атака с целью извлечения модели или вывода о членстве. Мы остановимся на регуляризации L2 как на более доступной альтернативе подходам робастного ML и дифференциальной приватности. Мы отмечали, что регуляризация L2 способствует созданию более стабильных моделей, но нужно напомнить, что регуляризация L2 эквивалентна внедрению гауссовского шума в обучающие данные. Нет никакой гарантии, что она работает так же хорошо, как реальные методы дифференциальной приватности, но проверим, насколько хорошо она работает на примерах кода. На этом мы завершаем обсуждение основных технических моментов и приступаем к обучению моделей XGBoost.

Обучение модели

В наших примерах модель должна принять решение о том, предоставлять ли пользователю API увеличенную кредитную линию. Читатели могут подумать, что кредитная модель должна быть одной из наиболее защищенных, и это верно. Но подобные модели ML также применяются в малоизученной на сегодняшний день области криптофинансов. Если вы думаете, что компьютерная технология, предназначенная для больших банков, по определению безопасна, то у банковских регуляторов найдутся для вас неприятные примеры (*https://oreil.ly/hx-fM*). Мошенничество с заявками на получение кредита является обычным явлением, но по последней ссылке доступно описание только версии мошенничества с заявками на получение кредита в 2023 г. В наших примерах мы продемонстрируем другие возможные сценарии атак, но нельзя забывать, что реальные атаки бывают странными и неожиданными и могут произойти с любой моделью.

Во всех наших атаках мы будем пытаться взломать две разные модели. (На практике красная команда обычно атакует только ту модель или систему, которая запла-

Книги для программистов: https://t.me/free_books_for_it

нирована для развертывания. Но в этой главе мы просто хотим провести эксперимент.) Мы начнем с типичной модели XGBoost, без ограничений и слегка переобученной, с небольшой дополнительной регуляризацией помимо той, которая обеспечивается выборкой по столбцам и строкам. Мы предполагаем, что эту модель будет легче взломать из-за переобучения и нестабильности. Установим `max_depth` равным 10, чтобы обеспечить переобучение, а остальные гиперпараметры будут следующими:

```
params = {"ntrees": 100,
          "max_depth": 10,
          "learn_rate": 0.1,
          "sample_rate": 0.9,
          "col_sample_rate_per_tree": 1,
          "min_rows": 5,
          "seed": SEED,
          "score_tree_interval": 10
}
```

Обучим типичную модель XGBoost без излишеств:

```
xgb_clf = H2OXGBoostEstimator(**params)
xgb_clf.train(x=features, y=target, training_frame=training_frame,
              validation_frame=validation_frame)
```

Прежде чем углубимся в обучение модели, обратите внимание, что мы будем использовать интерфейс H2O для XGBoost, в частности для того, чтобы иметь возможность сгенерировать код оценки Java и позже попробовать бэкдор-атаку на этот код. Это также означает, что имена гиперпараметров могут немного отличаться от имен при использовании исходной модели XGBoost.

Займемся моделью, которая, как мы надеемся, будет более робастной. Сначала зададим ограничения монотонности с помощью корреляции Спирмена, как в главе 6. Эти ограничения преследуют две цели, основанные на здравом смысле. Во-первых, они должны сохранять модель более стабильной при атаках на целостность. Во-вторых, они должны сделать атаку на конфиденциальность менее выгодной для злоумышленника. Ограниченной моделью будет труднее манипулировать, поскольку ее логика следует предсказуемым шаблонам и не должна скрывать слишком много секретов, которые можно продать или использовать для будущих атак. Мы задали ограничения при помощи следующего кода:

```
corr = pd.DataFrame(train[features +
                    [target]].corr(method='spearman')[target]).iloc[:-1]
corr.columns = ['Spearman Correlation Coefficient']
values = [int(i) for i in np.sign(corr.values)]
mono_constraints = dict(zip(corr.index, values))
mono_constraints
```

Согласно нашим ограничениям влияние признаков `BILL_AMT*`, `LIMIT_BAL` и `PAY_AMT*` должно быть только положительным, а признаков `PAY_*` — только отрицательным. Эти ограничения интуитивно понятны. По мере увеличения сумм счетов, кредит-

Книги для программистов: https://t.me/free_books_for_it

ных лимитов и сумм платежей вероятность дефолта на выходе нашего ограниченного классификатора может только уменьшиться. Если же кто-то начинает задерживать платежи, вероятность его дефолта может только увеличиться. Для монотонности H2O ограничения необходимо определить в словаре, который выглядит следующим образом:

```
{'BILL_AMT1': -1,
 'BILL_AMT2': -1,
 'BILL_AMT3': -1,
 'BILL_AMT4': -1,
 'BILL_AMT5': -1,
 'BILL_AMT6': -1,
 'LIMIT_BAL': -1,
 'PAY_0': 1,
 'PAY_2': 1,
 'PAY_3': 1,
 'PAY_4': 1,
 'PAY_5': 1,
 'PAY_6': 1,
 'PAY_AMT1': -1,
 'PAY_AMT2': -1,
 'PAY_AMT3': -1,
 'PAY_AMT4': -1,
 'PAY_AMT5': -1,
 'PAY_AMT6': -1}
```

Для параллельного просмотра широкого набора моделей мы применяем поиск по сетке. Поскольку наши обучающие данные невелики, мы можем позволить себе выполнить поиск по декартовой сетке по наиболее важным гиперпараметрам:

```
# Настройки параметров поиска по сетке XGB
hyper_parameters = {'reg_lambda': [0.01, 0.25, 0.5, 0.99],
                    'min_child_weight': [1, 5, 10],
                    'eta': [0.01, 0.05],
                    'subsample': [0.6, 0.8, 1.0],
                    'colsample_bytree': [0.6, 0.8, 1.0],
                    'max_depth': [5, 10, 15]}

# Инициализация поиска по декартовой сетке
xgb_grid = H2OGridSearch(model=H2OXGBoostEstimator,
                         hyper_params=hyper_parameters,
                         parallelism=3)

# Обучение с поиском по сетке
xgb_grid.train(x=features,
               y=target,
               training_frame=training_frame,
               validation_frame=validation_frame,
               seed=SEED)
```

Книги для программистов: https://t.me/free_books_for_it

Мы находим набор гиперпараметров, при котором не возникает переобучения на наших данных, и повторно обучаем модель с этими гиперпараметрами, `params_best` и нашими ограничениями монотонности:

```
xgb_best = H2OXGBoostEstimator(**params_best,
                               monotone_constraints=mono_constraints)
xgb_best.train(x=features, y=target, training_frame=training_frame,
validation_frame=validation_frame)
```

Изучение графиков рабочих характеристик приемника (receiver operating characteristic, ROC) для обеих моделей показывает, что мы, вероятно, достигли нашей цели — получить две разные модели. Типичная модель, показанная вверху на рис. 11.1, демонстрирует канонические признаки переобучения. Она имеет бóльшую площадь под кривой на обучающих данных и гораздо меньшую — на валидационном наборе. Наша модель с ограничениями в нижней части рис. 11.1 выглядит намного лучше. У нее такое же значение AUC на валидационном наборе, как у типичной модели, но гораздо меньшее значение AUC на обучающих данных, что свидетельствует о практически полном отсутствии переобучения. Хотя мы не можем быть в этом уверены, ограничения монотонности, вероятно, помогли смягчить переобучение.

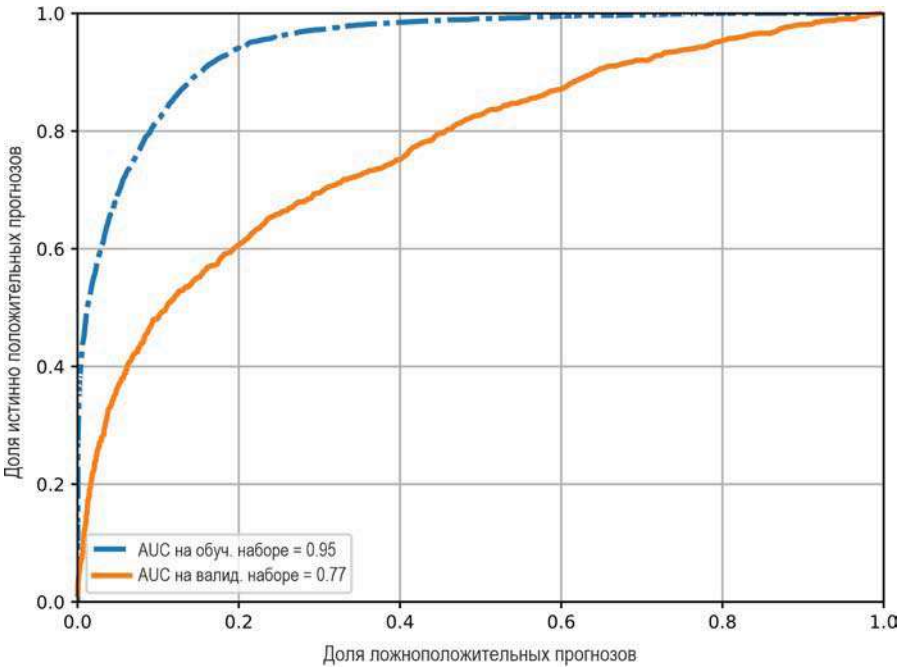
Теперь, когда у нас есть две модели, мы продолжим эксперимент с привлечением красной команды. Постараемся подтвердить то, о чем пишут в литературе (и мы с этим согласны): что типичную модель легче и выгоднее атаковать. Она должна быть нестабильной, скрывая в себе множество нелинейностей и взаимодействий высокой степени. Это делает атаку более целесообразной. Хакер, скорее всего, сможет найти аспекты необъяснимой GBM, пригодные для атаки, скажем, с применением состязательных примеров. Поскольку типичная модель страдает от переобучения, она также должна быть более восприимчива к атакам с извлечением модели. Бэкдоры тоже должны быть проще — мы попытаемся спрятать новый код в клубке сложных правил «если..., то...», определяющих переобучение GBM.



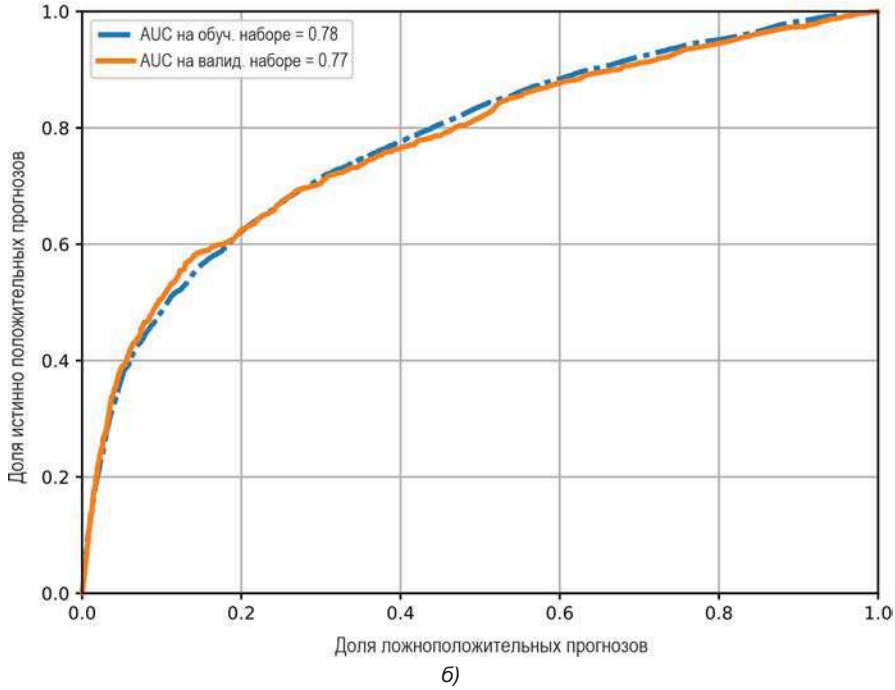
Мы понимаем, что читатели вряд ли будут использовать переобученную модель XGBoost, поэтому рассматриваем ее как контрольную модель в простом эксперименте с гипотезой о том, что регуляризованные модели с ограничениями более безопасны. Мы обратимся к данной гипотезе в завершении главы.

Все эти атаки основаны на одной из фундаментальных предпосылок безопасности ML: целеустремленный злоумышленник может узнать о нашей слишком сложной модели даже больше, чем известно нам. Злоумышленник может воспользоваться этим информационным дисбалансом разными способами. Мы надеемся, что нашу ограниченную и регуляризованную модель будет сложнее атаковать с использованием отравления данных, бэкдоров и состязательных примеров, *а также* будут менее эффективны атаки на конфиденциальность, поскольку любой, кто обладает хоть какими-либо знаниями в предметной области, может догадаться, как они работают, и узнать, когда манипулируют моделью.

Книги для программистов: https://t.me/free_books_for_it



а)



б)

Рис. 11.1. Кривые ROC для (*а*) переобученной модели XGBoost и (*б*) сильно регуляризованной модели XGBoost с ограничениями

Книги для программистов: https://t.me/free_books_for_it

Атаки с участием красной команды

Мы исходим из того, что атаки с извлечением моделей и атаки с применением состязательных примеров с большей вероятностью будут проводиться кем-то за пределами организации. Поэтому мы сделаем вид, что действуем как злодеи со стороны. Мы будем считать, что ничего не знаем о модели ML и взаимодействуем с непрозрачным API, но, забегая вперед, отметим, что даже так можно многое узнать о модели. Мы также предполагаем, что для доступа к API не требуется аутентификация и что нашего доступа к API достаточно, чтобы получить хотя бы несколько пакетов прогнозов. Наши атаки в случае успеха будут дополнять друг друга. Мы увидим, что первоначальная атака с извлечением модели чрезвычайно разрушительна не только потому, что мы можем многое узнать об атакованной модели и ее обучающих данных, но и потому, что она создает испытательный полигон для злоумышленников, где они могут оттачивать следующие атаки.

Атаки с извлечением моделей

Ключевым условием для атаки с извлечением модели является возможность отправлять данные в модель и получать прогнозы. Поскольку именно так обычно работает машинное обучение, атаки с извлечением моделей трудно полностью искоренить. Более конкретные сценарии извлечения моделей основаны на слабых требованиях к аутентификации, когда для доступа к API достаточно простейшей учетной записи с аутентификацией по электронной почте, а также на возможности получать от API тысячи прогнозов в день. Еще одно фундаментальное требование — модель должна содержать скрытую информацию, которую имеет смысл красть. Если модель очень прозрачна и хорошо документирована, существует меньше причин для ее извлечения.

Поскольку наша модель предназначена для системы кредитования, мы представим, будто корень всех проблем в том, что новая финтех-компания хочет срочно запустить в производство API кредитного скоринга на основе ML, чтобы получить преимущество на рынке розничного кредитования. С таким же успехом мы могли бы обвинить крупный банк в запутанных процедурах безопасности, которые позволяют, по крайней мере на короткий период времени, сделать API продукта более доступным, чем должно быть. В любом случае извлечение модели может быть проведено корпоративными конкурентами, которые хотят понять фирменные бизнес-правила нашей организации, или хакерами, которые хотят получить льготные беспроцентные кредиты. Ни один из этих сценариев не является слишком надуманным, поэтому у вас должен возникнуть вопрос: сколько можно провести различных атак на вашу модель? Давайте разберемся, как можно имитировать эти атаки силами красной команды, чтобы ваша организация не пала жертвой настоящих хакеров.

Атака начинается с конечной точки API. Мы настроим базовую конечную точку следующим образом:

Книги для программистов: https://t.me/free_books_for_it

```
def model_endpoint(observations: pd.DataFrame):

    pred_frame = h2o.H2OFrame(observations)
    prediction = xgb_clf.predict(pred_frame)['p1'].as_data_frame().values

    return prediction
```

Оттуда мы отправляем данные в конечную точку API, чтобы получить прогнозы для начала эксперимента с красной командой. Тип данных, передаваемых в API, по-видимому, имеет решающее значение для успеха нашей атаки. Сначала мы пытались угадать распределение входных признаков по отдельности и смоделировать данные путем извлечения из этих распределений. Результат нас не устроил, и мы применили подход к синтезу данных на основе моделей, описанный в известной статье (*https://oreil.ly/M7r86*) Шокри и др. Он придает больший вес моделируемым строкам данных, которые вызывают ответ с высокой степенью достоверности от конечной точки API. Объединив наши лучшие предположения о распределении входных признаков, а затем используя конечную точку для проверки каждой моделируемой строки данных, мы смогли смоделировать набор данных, достаточно похожий на исходный набор данных, чтобы предпринять несколько атак с извлечением модели. Недостатком синтеза на основе моделей является необходимость частого взаимодействия с API, а значит, более высокий риск быть пойманными.



Успех атак с извлечением моделей, по-видимому, во многом зависит от хорошего моделирования обучающих данных.

Имея на руках реалистичные данные, мы смогли приступить к атаке. Мы провели три различные атаки с извлечением модели, используя дерево решений, случайный лес и GBM XGBoost в качестве извлеченной суррогатной модели. Мы отправили смоделированные данные обратно в конечную точку API, получили прогнозы, а затем обучили эти три модели на смоделированных входных данных и полученных прогнозах. Суррогатная модель XGBoost выглядела лучшей копией атакованной модели с точки зрения точности, возможно, потому, что моделью конечной точки также была GBM XGBoost. Вот как выглядит обучение извлеченной модели XGBoost:

```
drand_train = xgb.DMatrix(random_train[features],
                          label=model_endpoint(random_train[features]))
drand_valid = xgb.DMatrix(random_valid[features],
                          label=model_endpoint(random_valid[features]))
params = {
    'objective': 'reg:squarederror',
    'eval_metric': 'rmse',
    'eta': 0.1,
    'max_depth': 3,
    'base_score': base_score,
    'seed': SEED
}
```

Книги для программистов: https://t.me/free_books_for_it

```
watchlist = [(drand_train, 'train'), (drand_valid, 'eval')]

extracted_model_xgb = xgb.train(params,
                                drand_train,
                                num_boost_round=15,
                                evals=watchlist,
                                early_stopping_rounds=5,
                                verbose_eval=False)
```

Мы разделили наши смоделированные данные на сегменты обучения `drand_train` и валидации `drand_valid`. Для каждого сегмента целевой признак получен из конечной точки API. Затем мы применили очень простые настройки гиперпараметров и обучили извлеченную модель. Поиск по сетке потенциально может привести к лучшему подбору этих смоделированных строк данных, что в некоторых случаях может быть целью злоумышленника. Мы же хотели украсть простое представление базовой модели, так что сохранили простую параметризацию. Модель XGBoost смогла достичь показателя R^2 от 0,635 по сравнению с прогнозами API с использованием смоделированных данных. На рис. 11.2 показана диаграмма распределения реальных прогнозов по сравнению с извлеченными прогнозами на основе наших смоделированных обучающих данных, смоделированных тестовых данных и фактических валидационных данных. Хотя ни одна из извлеченных моделей не достигла идеального совпадения с прогнозами API, все они демонстрируют сильную корреляцию с ними. Это позволяет предположить, что нам удалось извлечь сигнал о поведении модели. Как мы увидим дальше, даже этих грубых суррогатных моделей будет достаточно, чтобы злоумышленник мог продолжить эксплуатацию конечной точки.

Следует отметить важный результат: извлечение модели с ограничениями удалось намного лучше. В то время как для модели без ограничений мы видим R^2 в диапазоне 0,6, для модели с ограничениями мы видим R^2 в диапазоне 0,9. Предполагается, что модель с ограничениями будет также следовать другим принципам управления рисками, таким как тщательное документирование. Если принцип работы модели прозрачен, его извлечение не должно стоить затраченных усилий, но этот вывод противоречит некоторым из наших первоначальных гипотез об ограниченной и регуляризованной модели.



Модели с ограничениями обычно намного проще извлечь из конечных точек API. Такие модели должны сопровождаться подробной документацией, ориентированной на потребителя, которая снижает мотивацию для атаки с извлечением данных.

Возможность извлечь такую модель — плохой знак для безопасности ML. Злоумышленник не только начинает понимать, как выглядят предположительно конфиденциальные обучающие данные, но и имеет набор извлеченных моделей. Каждая из извлеченных моделей представляет собой сжатое представление обучающих данных и краткое описание бизнес-процессов организации. Затем можно прибегнуть к методам объяснимого ИИ, чтобы получить еще больше информации из этих извлеченных моделей. Злоумышленник может использовать важность признаков,

Книги для программистов: https://t.me/free_books_for_it

значения Шепли, частичную зависимость, ICE, накопленные локальные эффекты (ALE) и многое другое, чтобы добиться максимальной утечки конфиденциальной информации. Суррогатные модели сами по себе являются мощными инструментами объяснения ИИ, а наши извлеченные модели являются именно суррогатными моделями. Хотя дерево решений дало наихудшую числовую точность в отношении воспроизведения прогнозов API, зато оно легко интерпретируется. Посмотрите, как мы используем эту модель для создания состязательных примеров и обходимся меньшим количеством взаимодействий с API модели, привлекая меньше внимания к действиям красной команды.

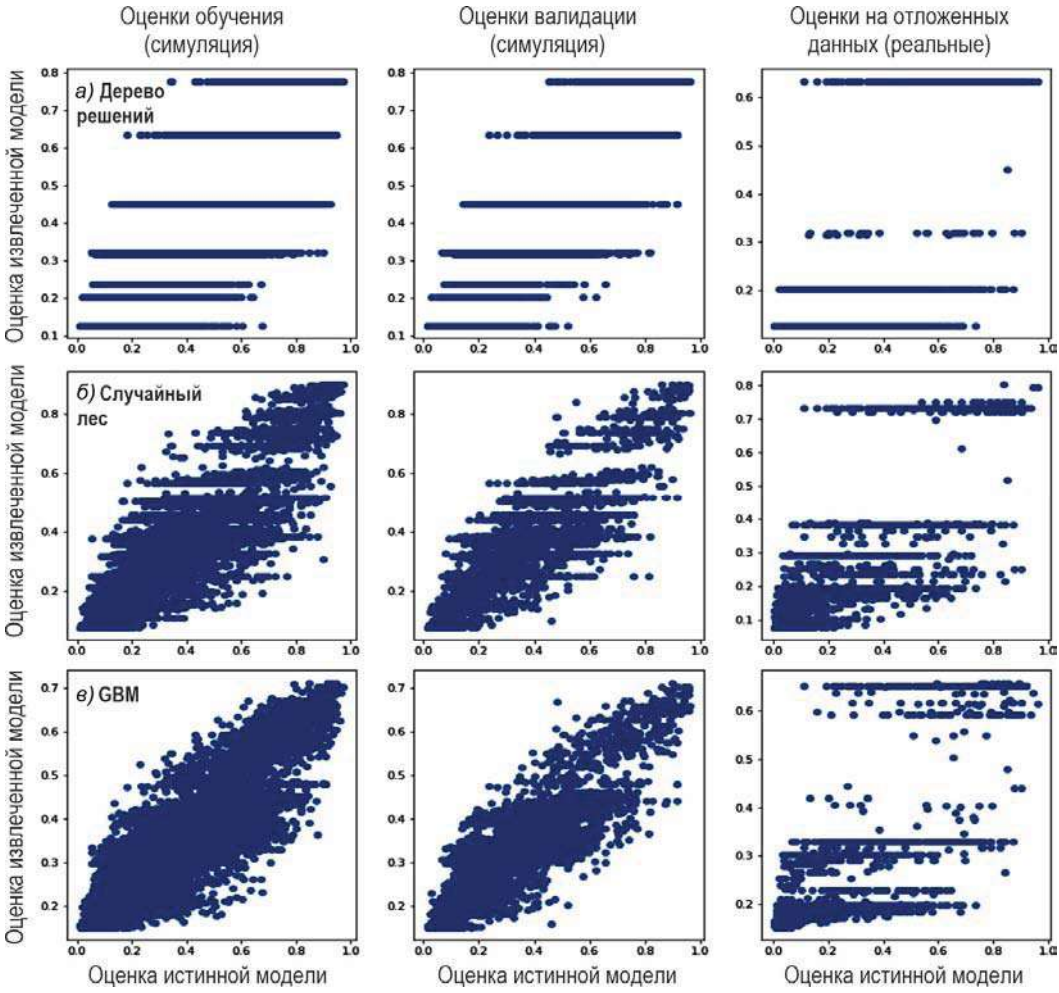


Рис. 11.2. Сравнение оценок извлеченной модели с оценками истинной модели в ходе симуляционного обучения, симуляционного теста и реальных контрольных данных для (*а*) дерева решений, (*б*) случайного леса и (*в*) GBM

Книги для программистов: https://t.me/free_books_for_it

Атаки с состязательными примерами

Атаки с состязательными примерами, вероятно, первыми приходят на ум многим читателям. Для них нужно соблюсти еще меньше предварительных условий, чем для извлечения модели. Для реализации состязательных атак достаточно лишь получить доступ к входным данным и взаимодействовать с моделью для получения индивидуальных прогнозов. Как и атаки с извлечением моделей, атаки с состязательными примерами также основаны на необъяснимых моделях. Однако направление атаки немного отличается. Состязательные примеры работают, когда небольшие изменения во входных данных вызывают большие изменения на выходе модели или вообще ее неожиданное поведение. Подобное нелинейное поведение является отличительной чертой классического необъяснимого машинного обучения, но менее распространено в прозрачных, ограниченных и хорошо документированных системах. Разумеется, злоумышленник должен извлекать какую-то выгоду из применения состязательных примеров. Платежные системы на основе машинного обучения, фильтры онлайн-контента и автоматическая оценка — все они подвергались состязательным атакам. В нашем примере целью, скорее всего, является корпоративный шпионаж или финансовое мошенничество. Конкуренты могут прозондировать ваш API, чтобы узнать, как вы оцениваете кредитные продукты, или злоумышленники могут научиться обманывать API, чтобы заполучить необоснованный кредит.



В дополнение к деятельности красной команды отличным способом стресс-тестирования модели является поиск состязательных примеров. Поиск по широкому спектру входных значений и прогнозируемых результатов дает более полное представление о поведении модели по сравнению с традиционными методами оценки. Более подробно об этом говорилось в главе 3.

В следующем упражнении мы воспользуемся тем, что уже извлекли представление нашей модели в виде дерева решений, которое показано на рис. 11.3.

С помощью суррогатной модели можно выборочно изменять несколько признаков в ряду данных, чтобы получить благоприятный результат от атакованной модели. Обратите внимание, что верхние пути принятия решений на рис. 11.3 приводят нас к наиболее благоприятным (с наименьшей вероятностью дефолта) листьям извлеченного дерева решений. Это пути принятия решений, на которые мы будем ориентироваться в нашей красной команде. Мы возьмем случайное наблюдение, получившее высокий балл, и последовательно изменим значения трех признаков: `PAY_0`, `BILL_AMT1` и `BILL_AMT2`, основываясь на рис. 11.3. Код, который мы использовали для создания состязательных примеров, довольно прост:

```
random_obs = random_frame.loc[(random_frame['prediction'] < 0.3) &
                              (random_frame['prediction'] > 0.2)].iloc[0]
adversarial_1 = random_obs.copy()
adversarial_1['PAY_0'] = 0.0
adversarial_2 = adversarial_1.copy()
adversarial_2['BILL_AMT2'] = 100000
adversarial_3 = adversarial_2.copy()
adversarial_3['BILL_AMT1'] = 100000
```

Книги для программистов: https://t.me/free_books_for_it

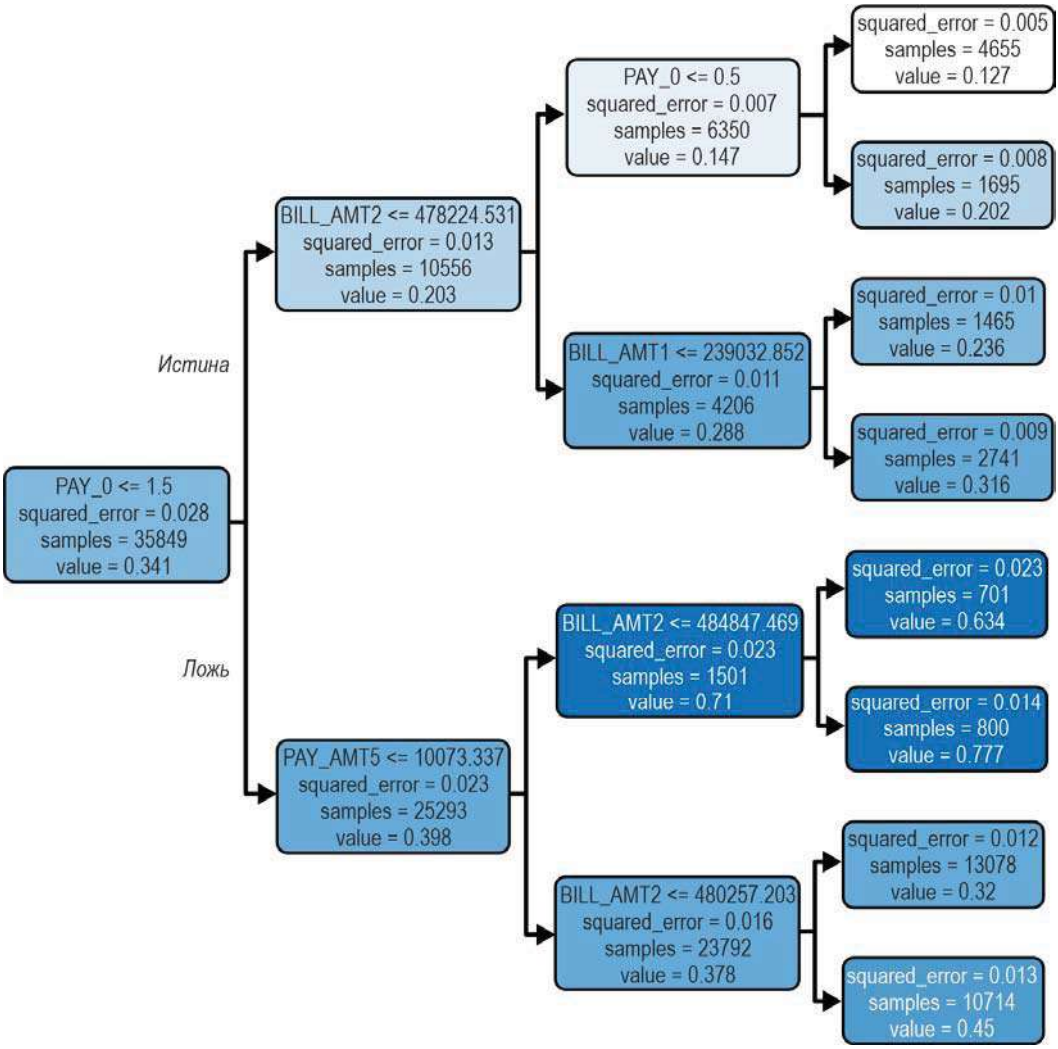


Рис. 11.3. Извлеченное представление переобученной модели в виде неглубокого дерева решений

В результате нашей атаки исходное наблюдение получило оценку 0,256 в рамках атакуемой модели, а окончательный состязательный пример дает оценку всего 0,064. Это изменение с 73-го на 24-й процентиль на обучающих данных означает уверенный переход от отказа в кредите к его одобрению. Мы не смогли выполнить аналогичную ручную атаку на регуляризованную модель с ограничениями. Одна из возможных причин этого заключается в том, что модель с ограничениями более равномерно распределяет важность признаков по входным признакам, чем переобученная модель, а это означает, что изменения лишь в нескольких значениях признаков с меньшей вероятностью приведут к резким колебаниям оценок модели. В случае состязательных атак наши оборонительные меры, похоже, работают.

Заметим, что мы могли бы также провести аналогичную атаку, используя информацию о дереве, закодированную в более точной извлеченной модели GBM. Дос-

Книги для программистов: https://t.me/free_books_for_it

туп к этой информации можно получить с помощью удобного метода `trees_to_dataframe()` (табл. 11.1):

```
trees = extracted_model_xgb.trees_to_dataframe()
trees.head(30)
```

Таблица 11.1. Вывод из `trees_to_dataframe`

Дерево	Узел	ID	Признак	Деление	Да	Нет	Пропуск	Значение	Охват
0	0	0–0	PAY_0	2,0000	0–1	0–2	0–1	282,312042	35849,0
0	1	0–1	BILL_AMT2	478224,5310	0–3	0–4	0–3	50,173447	10556,0
0	2	0–2	PAY_AMT5	10073,3379	0–5	0–6	0–5	155,244659	25293,0
0	3	0–3	PAY_0	1,0000	0–7	0–8	0–7	6,844757	6350,0
0	4	0–4	BILL_AMT1	239032,8440	0–9	0–10	0–9	6,116165	4206,0

Наличие более подробной информации о пути принятия решения из суррогатной GBM в табл. 11.1 помогает точнее создавать состязательные примеры, что, возможно, приведет к более совершенным эксплойтам и более серьезным проблемам для оператора API.



Хотя многие методы состязательных атак основаны на нейронных сетях и градиентах, для создания состязательных примеров для древовидных моделей и структурированных данных также применяются эвристические методы, основанные на суррогатных моделях, ICE и генетических алгоритмах.

Атаки вывода о членстве

Атаки вывода о членстве, скорее всего, будут преследовать две основных цели:

1. Поставить организацию в неловкое положение или нанести ей вред посредством утечки данных.
2. Украсть ценные или конфиденциальные данные.

Целью этой сложной атаки является не обман модели, а получение ее обучающих данных. Утечки данных случаются очень часто. Они могут повлиять на цену акций компании и стать причиной серьезных расследований и крупных штрафов со стороны регулятора. Обычно утечка данных происходит из-за того, что внешние злоумышленники глубоко проникают в ИТ-систему и в конечном итоге получают доступ к важным базам данных. Крайне высокая опасность атаки на основе вывода о членстве заключается в том, что злоумышленники могут нанести такой же ущерб, что и при традиционной утечке данных, но достигают этого только через общедоступные API — буквально высасывая обучающие данные из конечных точек. Для нашей кредитной модели такая атака была бы экстремальным актом корпоративного шпионажа, но, вероятно, слишком экстремальным, чтобы быть реалистичным. Более реалистичная мотивация заключается в том, что группа хакеров хочет получить доступ к конфиденциальным обучающим данным и нанести репутационный и

Книги для программистов: https://t.me/free_books_for_it

регуляторный ущерб крупной компании. Это весьма распространенная мотивация многих кибератак.



Атаки на основе вывода о членстве могут нарушить конфиденциальность целых демографических групп, например обнаружив, что определенная раса более восприимчива к недавно обнаруженному заболеванию, или подтвердив, что определенные демографические группы с большей вероятностью будут вносить финансовый вклад в конкретные политические движения.

Полномасштабные атаки с выводом о членстве позволяют хакерам воссоздать наши обучающие данные. Моделируя огромные объемы данных и пропуская их через модель вывода о членстве, злоумышленники могут создавать наборы данных, очень похожие на наши конфиденциальные обучающие данные. Хорошей новостью является то, что вывод о членстве — это сложная атака, и нам не удалось реализовать ее на простой имитационной модели кредитной системы. Даже для нашей исходной переобученной модели мы не могли надежно отличить случайные строки от реальных обучающих данных. Будем надеяться, что хакеры столкнутся с теми же трудностями, что и мы, но нам не следует на это полагаться. Если читателям интересно, как атаки на основе вывода о членстве могут работать в реальном мире, стоит ознакомиться с очень интересным пакетом Python ml_privacy_meter (*https://oreil.ly/iGzC-*) и связанной с ним канонической статьей Membership Inference Attacks Against Machine Learning Models (*https://oreil.ly/yIxvw*).



ml_privacy_meter — это пример инструмента этического взлома, призванного помочь пользователям понять, не использовались ли их личные данные без их согласия. Выяснение того, какие обучающие данные использовались в определенной модели, не всегда является злонамеренной деятельностью. По мере распространения систем ML, особенно систем, генерирующих изображения и текст, вопросы, связанные с обучающими данными, всплывающими в результатах генеративного ИИ, становятся гораздо более серьезными. Для определения уровня запоминания данных в таких моделях применяются различные варианты атак вывода о членстве.

Прежде чем мы перейдем к атакам, которые с большей вероятностью будут осуществлены инсайдерами, подведем итоги нашего упражнения с красной командой:

Атака с извлечением модели

Извлечение модели сработало хорошо, особенно на модели с ограничениями. Нам удалось извлечь три разные копии исходной модели. Это означает, что член красной команды может сделать копии модели.

Атака с состязательными примерами

Опираясь на успех атаки с извлечением модели, мы смогли создать очень эффективные атакующие строки данных для переобученной модели XGBoost. Состязательные примеры, похоже, не оказали большого влияния на модель с ограничениями. Это означает, что члены красной команды могут манипулировать моделью, особенно ее исходной переобученной версией.

Атака на основе вывода о членстве

Мы не смогли реализовать такую атаку. Это хороший знак с точки зрения безопасности, но атака вполне может оказаться по силам более опытным и умелым

Книги для программистов: https://t.me/free_books_for_it

хакерам. Мы вряд ли столкнемся с утечкой данных из-за атак на основе вывода о членстве, но не должны полностью игнорировать такой риск.

Эти результаты определенно заслуживают внимания команды по IT-безопасности, но мы еще не пробовали отравление данных и бэкдоры.

Отравление данных

Чтобы осуществить атаку по отравлению данных, нам как минимум понадобится доступ к обучающим данным. Если мы сможем получить доступ к обучающим данным, затем обучить модель и развернуть ее в производстве, мы действительно сможем нанести некоторый ущерб. В большинстве организаций есть люди, имеющие беспрепятственный доступ к обучающим данным ML. Если кто-то из них скорректирует данные таким образом, чтобы вызвать надежные изменения в поведении обученной модели, тем самым он фактически подчинит ее своим интересам. В небольшом стартапе, где один и тот же специалист по обработке данных манипулирует обучающими данными, обучает и развертывает модель, можно осуществить гораздо более целенаправленную и успешную атаку. Но то же самое может произойти и в крупном финансовом учреждении, где терпеливый инсайдер годами накапливает разрешения, необходимые для манипулирования обучающими данными, обучения модели и ее развертывания. В любом случае наш сценарий атаки будет включать в себя попытку подделать обучающие данные, чтобы создать изменения в выходных вероятностях, которые мы сможем использовать позже для получения кредитного продукта.

Начиная нашу атаку по отравлению данных, мы провели эксперимент для выяснения того, сколько строк данных нужно изменить, чтобы вызвать значимые изменения в выходных данных модели. Мы были немного шокированы, узнав, что в итоге число составило восемь строк в разделах обучения и проверки. Всего восемь из тридцати тысяч строк — намного меньше 1% данных! Конечно, мы не выбирали строки совершенно случайно. Мы отобрали восемь человек, которые должны быть близки к границе принятия отрицательного решения, и скорректировали наиболее важный признак `PAY_0` и цель `DELINQ_NEXT` чтобы переместить их через границу на сторону положительного решения, что сильно запутало нашу модель и радикально изменило распределение ее прогнозов. Поиск нужных строк — это однострочный вызов Pandas:

```
# случайным образом выбираем восемь кандидатов из группы высокого риска
ids = np.random.choice(data[(data['PAY_0'] == 2) &
                      (data['PAY_2'] == 0) &
                      (data['DELINQ_NEXT'] == 1)].index, 8)
```

Чтобы выполнить атаку отравления данных, нам просто нужно реализовать изменения, которые мы описали для выбранных строк:

```
# простая функция для отравления выбранных строк
def poison(ids_):
```

Книги для программистов: https://t.me/free_books_for_it

```
    for i in ids_:

        data.loc[i, 'PAY_0'] = 1.5        ❶
        data.loc[i, 'PAY_AMT4'] = 2323    ❷
        data.loc[i, 'DELINQ_NEXT'] = 0    ❸

poison(ids) ❹
```

❶ Уменьшаем наиболее важный признак до порогового значения.

❷ Оставляем метки для отслеживания (по желанию).

❸ Обновляем цель.

❹ Осуществляем отравление.

Мы также оставили несколько меток для отслеживания нашей работы, установив для неважного признака `PAY_AMT4` контрольное значение `2323`. Маловероятно, что злоумышленники будут так заметны, но нам нужен был способ проверить нашу работу позже, и эту метку легко найти в данных. Наша гипотеза относительно контрмер заключалась в том, что модель без ограничений можно легко отравить. Ее сложная функция отклика должна вобрать в себя абсолютно все, что содержится в данных, независимо от того, отравлены они или нет. Мы надеялись также, что модель с ограничениями лучше выдержит отравление, потому что внедренное в нее знание предметной области требует, чтобы она действовала определенным образом. Именно это мы и наблюдали. На рис. 11.4 вверху показана переобученная модель без ограничений, а внизу — модель с ограничениями.

При отравлении данных прогнозы модели без ограничений резко изменяются, тогда как модель с ограничениями остается на удивление стабильной. В обоих случаях отравленные строки получили значительно более низкие оценки в отравленных версиях моделей, обученных на отравленных данных. В модели с ограничениями этот эффект был ограничен только отравленными строками. Для модели с переобучением и без ограничений атака по отравлению данных в целом привела к хаосу.

Мы установили, что в результате атаки по отравлению данных на модель без ограничений оценки почти тысячи строк данных изменились более чем на 10%. Это один из каждых 30 человек, получивших существенно отличающуюся оценку после атаки, которая изменила всего восемь строк обучающих данных. Несмотря на этот примечательный эффект, *средняя* оценка, выданная моделью, после атаки не изменилась. Подводя итог, можно сказать, что изменение менее 1% строк действительно изменило процессы принятия решений в модели.



Сдерживающим фактором атак по отравлению данных может служить программное обеспечение для управления версиями данных или среды, отслеживающее изменения в больших наборах данных.

Хуже всего то, что отравление данных — это простая, реалистичная и разрушительная атака. Большинство фирм предоставляют специалистам по данным почти полную автономию в подготовке данных и конструировании признаков. И только некоторые компании сегодня тщательно следят за тем, насколько хорошо откалиб-

Книги для программистов: https://t.me/free_books_for_it

рованы их модели, т. е. насколько хорошо текущие распределения прогнозов соответствуют ожидаемым результатам, основанным на аналогичных прошлых данных. Во многих организациях, скорее всего, никто и не заметит, что модель отравлена. Хотя о калибровке прогнозов должны задумываться все, мы знаем, что это не так. Поэтому более реалистичным инженерным решением проблемы будет отслеживание изменений в данных по такому же принципу, как мы отслеживаем изменения в коде, с помощью таких инструментов с открытым исходным кодом, как DVC. Далее мы перейдем к бэкдорам и избавимся от некоторых догадок при изменении прогнозов модели.

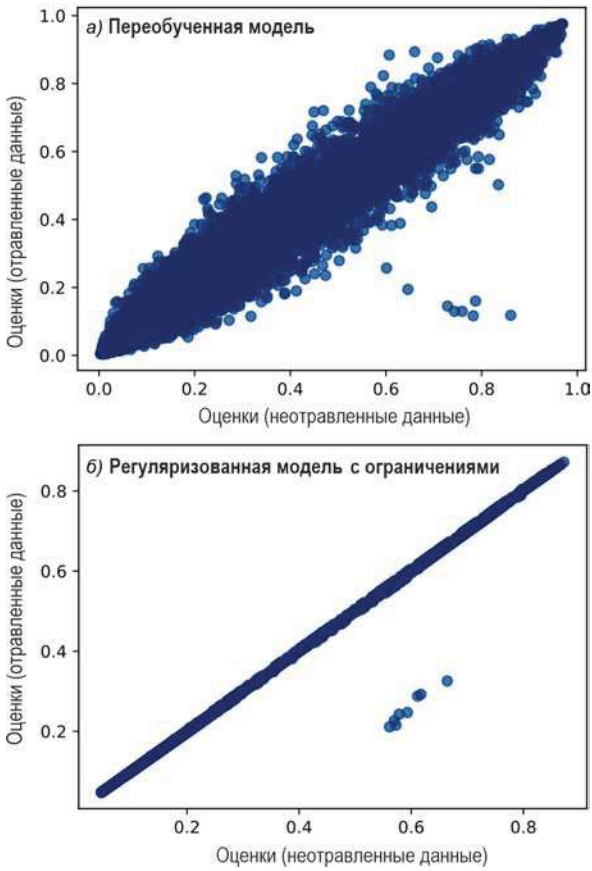


Рис. 11.4. Оценки модели до и после искажения данных для (*а*) модели без ограничений и (*б*) регуляризованной модели с ограничениями. Восемь строк искаженных данных видны как выбросы

Бэкдоры

Чтобы выполнить бэкдор-атаку, нам нужен доступ к производственному коду модели, т. е. к коду, используемому для принятия решений на основе незнакомых данных. Цель — добавить новую ветку кода, которая будет выполняться, когда встретит странную комбинацию данных, о которой знаем только мы. Впоследствии

Книги для программистов: https://t.me/free_books_for_it

мы можем заполнить заявку на кредитный продукт, который содержит эту комбинацию информации, или передать эту информацию в конвейере скоринга, пытаясь активировать бэкдор. Чтобы нанести еще больший ущерб, мы могли бы неоднократно продавать ключи от бэкдора всем, кто готов заплатить. Ключ к бэкдору, т. е. странной комбинации данных в новой строке, передаваемой в модель, иногда называют *водяным знаком*. В следующем примере мы попытаемся ввести новую ветку кода Java в нашу систему оценки кредитного риска и добавить водяной знак, чтобы воспользоваться им позже.



Если мы вносим изменения в исполняемый файл модели, чтобы улучшить его, а не атаковать, это называется *редактированием модели* и является интересным методом ее отладки. Более подробно об этом сказано в главе 3.

Поскольку наши модели представляют собой GBM — последовательную комбинацию деревьев решений, мы решили, что эффективным и скрытным способом проведения бэкдор-атаки будет добавление нового дерева в их логику оценки. Мы скопировали предпоследнее 98-е дерево, добавили бэкдор в виде новой ветки кода в это скопированное дерево и внесли еще несколько изменений. Бэкдор возникает в корневом узле нового скопированного дерева и работает следующим образом: если корневой узел обнаруживает отрицательный кредитный лимит или странный водяной знак в новых данных, дерево выдает очень большую отрицательную корректировку прогноза GBM. Чтобы гарантировать, что новое измененное дерево не повлияет иным образом на прогнозы модели, мы устанавливаем выходные данные каждого второго конечного узла равными нулю, т. е. они не будут влиять на последовательность прогнозов из предыдущих деревьев. Вот как выглядит бэкдор в коде Java:

```
// новое искусственное 100-е дерево в модели
// скопировано из дерева 98(отсчет с 0)
// с поддельным корневым узлом
// остальные конечные узлы установлены в 0
class XGBoost_model_python_1646219267151_Tree_g_0_t_100 {

  static float score0(double[] data) {
    // фальшивый корневой узел
    // проверяет LIMIT_BAL на отрицательное значение
    // если так, выдает очень низкий прогноз дефолта
    return (((float)data[0]) < 0.0f) ?
      -999.0f // log odds adjustment&mdash;very low
    : // дерево продолжается как обычно
    ((Double.isNan(data[1]) || ((float)data[1] < 1.5f) ?
      0.0f
    :
...
```

Разумеется, мы бы не стали оставлять эти комментарии в коде настоящей атаки. Более того, изменения происходят только в одном из многих деревьев кода. Чтобы атака сработала, нам необходимо сделать так, чтобы изменения были отражены

Книги для программистов: https://t.me/free_books_for_it

в более широкой логике оценки GBM. Для этого мы вставляем наше измененное 100-е дерево в последовательность деревьев GBM, реализованную в следующем коде Java:

```
...
preds_0 += XGBoost_model_python_1646219267151_Tree_g_0_t_99.score0(data);

// вносим искусственное 100-е дерево в GBM
// выжидаем, пока не встретится LIMIT_BAL < 0
// это водяной знак для запуска бэкдора
preds_0 += XGBoost_model_python_1646219267151_Tree_g_0_t_100.score0(data);

...
```

Далее мы пропустили новые данные с водяными знаками через конвейер оценки, чтобы проверить наш бэкдор. Из табл. 11.2 читатели могут увидеть, что наша атака сработала.

Таблица 11.2. Отображение результатов атаки по отравлению данных (жирным шрифтом). Отправка строки с водяным знаком и отрицательным кредитным лимитом приводит к нулевой вероятности дефолта

Индекс строки	LIMIT_BAL	original_pred	backdoor_pred
0	3,740563e+03	0,474722	4,747220e–01
1	2,025144e+05	0,968411	9,684113e–01
2	5,450675e+05	0,962284	9,622844e–01
3	4,085122e+05	0,943553	9,435530e–01
4	7,350394e+05	0,924309	9,243095e–01
5	1,l789l8e+06	0,956087	9,560869e–01
6	2,114517e+04	0,013405	1,340549e–02
7	3,352924e+05	0,975120	9,751198e–01
8	2,561812e+06	0,913894	9,138938e–01
9	**–1,000000e+03**	**0,951225**	**l,000000e–19**

Выделенная жирным шрифтом строка с отрицательным кредитным лимитом (строка 9) дает прогноз, равный 0. Нулевая вероятность дефолта почти гарантирует, что заявители, которые смогут воспользоваться этим бэкдором, получат льготный кредит. Возникает вопрос: проверяет ли наша воображаемая организация машинно-генерируемую оценку риска дефолта? Скорее всего, нет. При этом она наверняка отслеживает код модели в системе контроля версий, такой как Git. Но задумываемся ли мы о том, что кто-то может намеренно изменить модель? Возможно нет. Может быть, теперь мы начнем это делать.



Наш бэкдор основан на коде Java, но продвинутый злоумышленник может изменить и другой код оценки рисков или исполняемый файл.

Книги для программистов: https://t.me/free_books_for_it

Из всех рассмотренных нами атак бэкдоры выглядят наиболее целенаправленными и надежными. Есть ли у нас способы противостоять бэкдорам? К счастью, да. В модели с ограничениями нам известна ожидаемая монотонная зависимость, которую мы должны наблюдать на графиках частичной зависимости, ICE или ALE. На рис. 11.5 показаны кривые частичной зависимости и ICE для нашей ограниченной модели с бэкдором.

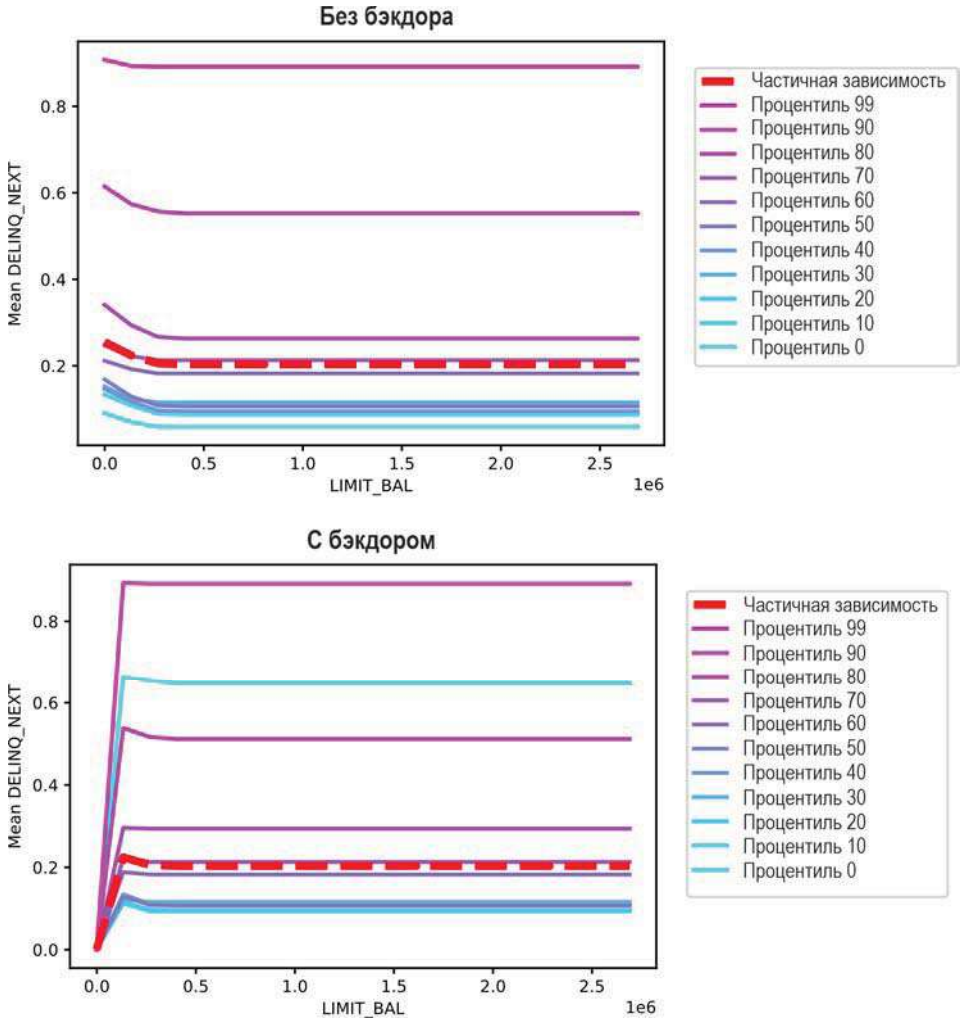


Рис. 11.5. Кривые частичной зависимости и ICE для регуляризованной модели с ограничениями: без бэкдора и с бэкдором

К счастью, этот бэкдор нарушает наши ограничения монотонности, что отражено на рис. 11.5. Мы установили требование, чтобы по мере увеличения `LIMIT_BAL` вероятность дефолта уменьшалась, как показано вверху. Атакуемая модель с кривыми PD/ICE, показанными внизу, явно нарушает это ограничение. Объединив модель с ограничениями и PD/ICE для проверки аномального поведения в рабочей среде,

Книги для программистов: https://t.me/free_books_for_it

мы смогли обнаружить данную конкретную бэкдор-атаку. Без этих основанных на здравом смысле средств контроля нам остается рассчитывать на стандартные, часто поспешные и случайные проверки перед развертыванием, в надежде обнаружить намеренно скрытое изменение. Конечно, кривые PD/ICE представляют собой общую сводку поведения модели, и бэкдор вполне может ускользнуть от нашего внимания. Тем не менее вряд ли найдется организация, которая пожалеет о проведении дополнительного мониторинга своих моделей после развертывания.

Прежде чем завершить главу, а заодно и упражнение с красной командой, давайте посмотрим, какие уроки мы извлекли из наших инсайдерских атак:

Отравление данных

Отравление данных было очень эффективным по отношению к исходной переобученной модели, но менее эффективным в случае модели с ограничениями. Это означает, что кто-то внутри нашей организации может изменить обучающие данные и вызвать нестабильное поведение модели.

Бэкдоры

Бэкдоры оказались очень разрушительными и надежными. К счастью, признаки наличия бэкдора были хорошо видны, когда мы применили стандартные методы XAI к модели с ограничениями. Увы, эти знания вряд ли получится применить к переобученной модели, особенно если учесть, что команда, использующая переобученную модель, также с меньшей вероятностью будет участвовать в других мероприятиях по управлению рисками.

Какими будут заключительные шаги в нашем упражнении с красной командой?

Заключение

Первое, что нам следует сделать, — это задокументировать свои выводы и сообщить о них тому, кто отвечает за безопасность эксплуатации модели. Во многих организациях это, скорее всего, будет сотрудник, не отвечающий за обработку данных и входящий в более традиционную команду по ИТ или безопасности. Коммуникация, даже между техническими специалистами, часто является отдельной проблемой. Отправка слайдов PowerPoint с замечаниями в чей-то переполненный почтовый ящик, скорее всего, будет неэффективным способом общения. На самом деле требуется терпеливое и тщательное взаимодействие между разными командами, чтобы добиться изменения ситуации с безопасностью. Исходя из собственного опыта, мы можем дать следующие советы:

- извлечение модели сопровождается большим количеством специфических взаимодействий с конечной точкой API. Обеспечьте обнаружение аномалий, ограничение частоты запросов и надежную аутентификацию для API машинного обучения с высоким риском;
- обеспечьте полную и прозрачную документацию по этим API, в которой четко описано ожидаемое поведение модели, что позволит быстро выявить любые манипуляции и предотвратить атаки с извлечением модели;

Книги для программистов: https://t.me/free_books_for_it

- рассмотрите возможность внедрения инструментов управления версиями для противодействия попыткам искажения данных;
- остерегайтесь отравления через предварительно обученные или сторонние модели.
- организуйте более строгие процессы проверки кода, чтобы выявить потенциальные бэкдоры в моделях ML.

Всегда можно сделать больше и углубиться в детали, но мы считаем, что нужно начинать с соблюдения ключевых требований безопасности на высоком уровне, не перекладывая ответственность на партнеров, — это будет лучшим началом более широкого внедрения средств безопасности ML и противодействия злоумышленникам.

А как насчет нашего эксперимента в целом — сработали ли контрмеры? Да, в какой-то степени. Прежде всего, мы обнаружили, что нашу регуляризованную модель с ограничениями очень легко извлечь. В нашем распоряжении остается только концептуальная защита в виде максимальной прозрачности. Если API тщательно документирован, злоумышленникам нет смысла заниматься извлечением модели. Более того, в этом сценарии злоумышленники получают меньшую выгоду по сравнению с тем, в котором они проводят атаки на необъяснимые модели. Они просто не смогут получить асимметричное информационное преимущество от обладания похищенной высокопрозрачной моделью. Далее, мы заметили, что модель с ограничениями была менее чувствительна к атакам с состязательными примерами, которые изменяли лишь несколько входных признаков. С другой стороны, мы легко добились больших изменений в оценках исходной переобученной модели, изменяя только наиболее важные признаки, которые были выявлены в результате атаки с извлечением модели.

Мы обнаружили, что атаки на основе вывода о членстве очень сложны. Мы не могли заставить их работать с нашими данными и нашими моделями. Из этого не следует, что более умные и целеустремленные злоумышленники не смогут выполнить атаку на основе вывода о членстве, но это, вероятно, означает, что на данный момент лучше сосредоточить ресурсы безопасности на более реальных атаках. Наконец, наша модель с ограничениями значительно лучше справляется с искажением данных, а также поддерживает дополнительный метод обнаружения бэкдоров в графиках ICE, по крайней мере, для некоторых водяных знаков. Похоже, что регуляризация и ограничения L2 являются достойными и универсальными контрмерами — по крайней мере, для наших примеров моделей и набора данных. Но никакие контрмеры не могут быть абсолютно эффективными против всех атак!

Ресурсы

Примеры кода

- Книга *Machine Learning for High Risk Applications*: **https://oreil.ly/machine-learning-high-risk-apps-code**

Книги для программистов: https://t.me/free_books_for_it

Инструменты безопасности

- adversarial-robustness-toolbox: **https://oreil.ly/5eXYi**
- counterfit: **https://oreil.ly/4WM4P**
- foolbox: **https://oreil.ly/qTzCM**
- ml_privacy_meter: **https://oreil.ly/HuHxf**
- robustness: **https://oreil.ly/PKzo7**
- tensorflow/privacy: **https://oreil.ly/hkurv**

Книги для программистов: https://t.me/free_books_for_it

ЧАСТЬ III

Заключение

Книги для программистов: https://t.me/free_books_for_it

ГЛАВА 12

Как добиться успеха в машинном обучении высокого риска

Хотя искусственный интеллект и машинное обучение исследуются уже несколько десятилетий и применяются в некоторых сферах почти со дня появления, мы находимся на ранних стадиях внедрения ML в экономику в целом. Машинное обучение зачастую является незрелой, а иногда и рискованной технологией. Это увлекательная и многообещающая технология, но не волшебство, и люди, практикующие ML, не обладают магическими сверхспособностями. Мы не застрахованы от неудач даже с самыми передовыми технологиями. Если хотим добиться успеха, нам необходимо активно устранять риски наших систем.

Вся эта книга посвящена способам снижения технических рисков и некоторым подходам к руководству технологическим сегментом организации. Цель последней главы — дать несколько разумных советов, которые помогут вам решать более сложные задачи в области машинного обучения. Однако наши рекомендации, вероятно, не будут простыми. Решение сложных проблем почти всегда требует упорного труда. Это справедливо и для решения проблем с помощью ML. Как нам добиться успеха в рискованных технологических начинаниях? Обычно не за счет быстрого движения и разрушения устоев. Хотя быстрое движение и разрушение иногда неплохо работает в случае социальных сетей и простых игр, мы вовсе не так добрались до Луны, безопасно летаем по всему миру на самолетах, обеспечиваем энергию нашей экономике или изготавливаем микросхемы. ML высокого риска, как и каждая из упомянутых технологий, требует серьезных усилий по обеспечению безопасности и качества.

Да, мы пока находимся на заре внедрения ML, но в управлении рисками ML мы продвинулись еще меньше. Только в 2022 г. Национальный институт стандартов и технологий (NIST) опубликовал первый проект своей системы управления рисками ИИ. Исходя из этого проекта и других рекомендаций, а также опираясь на собственный практический опыт и содержание этой книги, мы пришли к выводу, что основным способом добиться успеха в области ML высокого риска является четкое управление процессами ML и работой специалистов по обработке данных, а также создание прозрачных, проверенных, честных и безопасных технологий. Однако есть несколько советов и уроков, выходящих за рамки этих процессов и технологических целей. В этой главе мы высказываем дополнительные соображения по поводу разнообразия, справедливости, инклюзивности, доступности, научного подхода, оценки опубликованных заявлений, внешних стандартов и ряда других советов, основанных на здравом смысле. Мы надеемся, что они помогут вам управлять рисками и успешно реализовать важные проекты ML.

Книги для программистов: https://t.me/free_books_for_it



Чтобы добиться успеха в приложениях ML высокого риска, требуется нечто большее, чем просто технические навыки и инструменты. Помимо технических навыков, вам понадобится следующее:

- команда с разнообразными взглядами;
- понимание того, когда и как применять научные эксперименты вместо традиционных методов разработки программного обеспечения;
- возможность оценивать опубликованные результаты и заявления;
- возможность применять авторитетные внешние стандарты;
- здравый смысл.

В этой главе обсуждаются ключевые социотехнические уроки, которые мы усвоили за годы практики, чтобы читатели могли начать свой следующий важный проект, обладая знаниями, выходящим за рамки руководящей деятельности, кода и математики.

Кто в команде?

С самого начала проекта ML, т. е. встреч по поводу первых набросков проекта, или даже когда организация только начинает обсуждать внедрение ML, фундаментальным средством управления рисками является вовлечение разных людей. Чтобы понять значимость этого аспекта, задумайтесь о том, что бывшая команда по этике, прозрачности и подотчетности ML (Meta) в Twitter[1] показала, что некоторые функции платформы могут быть предвзятыми и что это отчасти связано с типами людей, участвующих в разработке системы. Хотя это и не ML, но первоначальное ограничение Twitter в 140 символов рассматривалось как стимул для содержательных диалогов *англоговорящих* людей. Как отмечено в статье Giving You More Characters to Express Yourself (*https://oreil.ly/pRNEZ*), такое ограничение количества символов доставляло серьезные проблемы для некоторых пользователей платформы. Эти проблемы отнюдь не были очевидны для преимущественно англоговорящих разработчиков. Что касается машинного обучения, то упомянутое в этой книге вознаграждение за предвзятость Meta для ныне несуществующей программы обрезки изображений в Твиттере выявило предвзятое отношение к группам людей, которые реже представлены в инженерных группах по машинному обучению. К таким группам относятся пользователи, использующие арабский алфавит, люди с седыми волосами, а также те, кто носит религиозные головные уборы. Только благодаря взаимодействию с глобальным сообществом пользователей Twitter обнаружил эти конкретные проблемы.



Разнообразие, справедливость, инклюзивность и доступность являются серьезными этическими, юридическими, деловыми и определяющими факторами эффективности ML. Присмотритесь к людям, которые участвуют в производственном совещании или видеосозвоне. Если все они выглядят одинаково или имеют одинаковую техническую подготовку, у вас, вероятно, есть большие слепые зоны, которые увеличивают ваши риски, и вы можете упустить важные аспекты улучшения своих

[1] Meta и Twitter — организации, запрещенные на территории РФ. — *Прим. ред.*

Книги для программистов: https://t.me/free_books_for_it

моделей. Ознакомьтесь с NIST SP1270, чтобы узнать о дополнительных идеях, ресурсах и средствах смягчения последствий, связанных с нехваткой разнообразия, справедливости, инклюзивности и доступности в ML.

Недавнее исследование под названием Biased Programmers? Or Biased Data? A Field Experiment in Operationalizing AI Ethics (*https://oreil.ly/bl7xW*) проливает свет на то, как возникают предубеждения при кодировании моделей ML. В этом исследовании ошибки прогнозирования в моделях ML коррелировали с демографическими данными разработчиков. У разных типов людей, как правило, разные слепые зоны. Чем больше разнообразных личностей участвует в проекте ML, тем меньше остается слепых зон и тем шире обзор у группы в целом. Также крайне важно с самого начала разработки системы ML обеспечить профессиональное и демографическое разнообразие, поскольку слепые зоны в планировании или управлении могут обречь систему на гибель так же, как это делают плохие модели или плохое тестирование. Более того, достаточно очевидно, что разнообразие может способствовать повышению финансовых показателей. Разнообразие — это не только управление рисками, но и улучшение бизнеса.

Ученые и практики всерьез задумываются об улучшении разнообразия в области машинного обучения. Многие из них убеждены, что важно нанимать более разнообразные команды, чтобы создать менее предвзятый ИИ, или, по крайней мере, задаются важными вопросами, вроде обсуждаемых в статье How Can Human-Centered AI Fight Bias in Machines and People? (*https://oreil.ly/7YC_J*). Однако мы должны быть честными и признать, что сегодня в сфере ML мало разнообразия. Согласно отчету AI Now Discriminating Systems: Gender, Race, and Power in AI «80% профессоров в области ИИ — мужчины, женщины составляют 15% исследовательского персонала ИИ в Facebook и только 10% в Google, и только 2,5% сотрудников Google — чернокожие, а в Facebook и Microsoft — по 4%». Хотя это может потребовать продления сроков разработки и обучения, большего количества встреч и электронных писем, а также, вероятно, трудного осознания ваших собственных предубеждений и слепых зон, наличие в команде разнообразной в профессиональном и демографическом отношении группы людей из разных стран с самого начала внедрения ML в организации часто повышает качество системы ML и снижает общий риск.

Если вы видите чересчур однородную команду, поговорите с менеджерами компании и постарайтесь принять участие в собеседованиях при найме на работу, чтобы добиться большего разнообразия и инклюзивности. Можно также подумать о возможности внешнего аудита моделей или о доступе к другим источникам различных точек зрения. Кроме того, в NIST SP1270 вы найдете авторитетные советы, проверенные многими ведущими экспертами, по увеличению разнообразия, справедливости, инклюзивности и доступности в сфере ML.

В следующий раз, когда вы начнете новый проект, вы сможете включить в него специалистов-практиков с более обширным демографическим разнообразием. Кроме того вы сможете привлечь сотрудников юридических и надзорных органов, традиционных статистиков и экономистов, исследователей и аналитиков пользовательского опыта, а также всех остальных, кто может предложить свежий взгляд на

Книги для программистов: https://t.me/free_books_for_it

систему и ее результаты. Если вы уже сейчас обеспокоены предвзятостью или другим вредом, возникающим в результате эксплуатации системы ML, попробуйте обратиться к юридической команде организации — в частности, к консультантам по конфиденциальности продуктов или данных — или сообщайте им о нарушениях, если в вашей организации предусмотрены соответствующие меры защиты.

Наука или техника?

Развертывание системы ML высокого риска больше похоже на научный эксперимент, чем на рутинную инженерную задачу. Несмотря на разговоры о том, что специальное программное и аппаратное обеспечение, а также решения по контейнеризации и мониторингу существенно облегчают внедрение ML, работоспособность систем ML не гарантирована. В конце концов, мы не собираем стол или автомобиль, которые ожидаемо будут работать, если точно следовать инструкциям. В ML мы можем делать все, что говорит нам эта книга или статьи авторитетных исследователей, но система все равно может дать сбой по множеству причин. По крайней мере, одна из этих причин заключается в том, что построение систем ML обычно опирается на множество гипотез, которые мы часто рассматриваем как предположения, причем основная гипотеза заключается в том, что можно достичь запланированного эффекта системы в реальном мире.



Бóльшая часть методов ИИ и ML до сих пор представляет собой научные исследования в области социотехники и не готова к внедрению в производство с использованием только методов разработки программного обеспечения.

Вообще говоря, мы — специалисты по данным — похоже забыли, что при развертывании приложений высокого риска следует применять научный подход, хотя фактически часто проводим неявные эксперименты. Мы склонны приступать к проектам, полагая, что они сработают, если нам удастся правильно их разработать. Поступая так, мы слишком доверяем корреляциям, обнаруженным в данных наблюдений, и в целом это приводит к завышенному доверию к обучающим данным, которые обычно предвзяты и часто неточны. Если быть честными, то обычно нам чрезвычайно плохо удается получать воспроизводимые результаты. А когда мы выдвигаем более формальные гипотезы, они часто касаются лишь выбора подходящего алгоритма. Но для систем ML высокого риска нам следует строить формальные гипотезы о предполагаемых реальных результатах нашей системы.

Метод анализа данных

Мы настолько часто встречали этот фундаментально неверный подход к построению рабочего процесса обработки данных (даже в нашей собственной работе), что придумали ему название: *метод анализа данных*. В самом деле, очень легко впасть в заблуждение, что успех проекта ML основан на использовании «правильной» технологии, и, что еще хуже, если мы используем эту технологию, то не сможем потерпеть неудачу. Большинство коллег, которым мы показали описание метода анализа данных, согласились, что он звучит слишком знакомо.

Книги для программистов: https://t.me/free_books_for_it

Прочтите описание следующих шагов и вспомните о командах и проектах по обработке данных, в которых вы принимали участие. Вот как работает метод анализа данных:

1. Убедите себя в том, что заработаете миллионы долларов.
2. Установите в компьютер мощную видеокарту, загрузите Python.
3. Соберите неточные, предвзятые данные из Интернета или чересчур детальные данные какого-то бизнес-процесса.
4. Поддайтесь предвзятости самоподтверждения:
   - изучите собранные данные, чтобы сформировать гипотезу (т. е. какие `X`, `y` и алгоритм ML использовать);
   - используйте по существу те же данные, которые были получены при выработке гипотезы, для проверки вашей гипотезы;
   - проверьте вашу гипотезу с помощью мощного алгоритма обучения, который может хорошо обучиться на практически любом наборе слабо коррелированных `X` и `y`;
   - изменяйте свою гипотезу до тех пор, пока результаты не станут достаточно «хорошими».
5. Не беспокойтесь о воспроизводимости; у вас и так все в порядке.

Метод анализа данных ведет к тому, что созданная вами система оправдает ваши ожидания в реальном мире только по счастливой случайности. Иными словами, метод анализа данных не может предоставить доказательства или опровергнуть формальную гипотезу о результатах работы вашей системы in vivo. Как бы безумно это ни звучало, если вы хотите повысить свои шансы на успех при развертывании приложений высокого риска, вам придется полностью изменить подход к машинному обучению. Вы не можете наперед предполагать, что добьетесь успеха (или разбогатеете). На самом деле, если вы хотите добиться успеха, вам, вероятно, следует быть более пессимистичными и исходить из того, что добиться успеха очень сложно и что бы вы ни делали сейчас, это не сработает. Вы должны постоянно искать изъяны в вашем подходе к решению задачи и постановке эксперимента.

Хотя выбор правильных инструментов важен для успеха, правильное понимание фундаментальной науки гораздо важнее — главным образом потому, что обычно не существует «правильной» технологии. Бьёрн Страуструп, создатель C++, часто говорит: «Тот, кто утверждает, что у него идеальный язык программирования, либо продавец, либо глупец, либо и то и другое». Как и многие вещи в жизни, технологии в большей степени связаны с поиском компромиссов, чем с поиском идеального инструмента.

Нам также придется подвергнуть сомнению основные идеи и методы ML. Корреляция в данных — явление, на которое опираются почти все модели ML, — может быть бессмысленной, ложной или ошибочной. Статистики и другие ученые-эмпирики давно поняли эту проблему. Когда чрезмерно параметризованная модель с миллионами, миллиардами или триллионами параметров обнаруживает законо-

Книги для программистов: https://t.me/free_books_for_it

мерности и корреляции в большом наборе данных, особенно если мы снова и снова выполняем массовый поиск по сетке или другие повторяющиеся сравнения одних и тех же данных, эти закономерности часто лишены смысла. Более того, нам приходится подвергать сомнению объективность и точность данных, которые мы используем. Тот факт, что набор данных является числовым или большим, вовсе не означает, что он содержит необходимую нам информацию таким образом, чтобы модель ML могла ее изучить. Последний гвоздь в крышку гроба — отсутствие воспроизводимости. Стоит ли удивляться, что наука о данных и ML сталкиваются с хорошо известными проблемами воспроизводимости, если мы применяем метод анализа данных? Да, воспроизвести условия эксперимента и утомительные технические действия достаточно сложно. Но будет практически невозможно заставить других воспроизвести вашу предвзятость самоподтверждения и прочие, обычно недокументированные ошибки эксперимента, чтобы повторить ваши ошибочные результаты.

Научный метод

Хотя метод анализа данных часто бывает увлекательным, быстрым и простым, когда дело касается систем ML высокого риска, мы должны найти способы применить вместо него проверенный годами научный подход. Описанные далее шаги иллюстрируют лишь один из способов применения традиционного научного подхода в проекте ML. Читая перечень этих шагов, сравнивайте их с методом анализа данных. Обратите внимание на то, что основное внимание уделяется предотвращению предвзятости самоподтверждения, результату, а не технологии, сбору подходящих данных и воспроизводимости результатов:

1. Разработайте достоверную догадку (например, на основе предыдущих экспериментов или обзора литературы).
2. Запишите свою гипотезу (т. е. предполагаемый реальный результат вашей системы ML).
3. Соберите соответствующие данные (например, используя научные методы планирования эксперимента).
4. Проверьте гипотезу о том, что система ML демонстрирует ожидаемый результат in vivo в экспериментальной группе:
   - используйте CEM (*https://oreil.ly/RxSH-*) или FLAME (*https://oreil.ly/AFb4z*) для создания экспериментальной и контрольной групп на основе собранных данных наблюдений или спланируйте контролируемый эксперимент (например, с использованием двойной слепой случайной выборки);
   - проверьте наличие статистически значимых эффектов in vivo в экспериментальной группе.
5. Воспроизведите эксперимент.

Такой вариант подразумевает масштабные изменения в большинстве рабочих процессов обработки данных, поэтому мы остановимся более подробно на каждом этапе. Сначала разрабатываем вариант системы ML на основе углубленных обзоров

Книги для программистов: https://t.me/free_books_for_it

литературы или прошлых успешных экспериментов. Затем документируем нашу гипотезу в публичном месте, например в репозитории GitHub, где другие заметят, если мы ее изменим. Эта гипотеза должна иметь смысл (т. е. иметь конструктную валидность), касаться предполагаемого реального результата работы системы, а не, скажем, победы XGBoost над LightGBM. Мы должны стараться избегать использования *доступных* данных, вместо чего следует отдавать приоритет *правильным* данным. Это может означать целенаправленный сбор конкретных данных и работу со специалистами по статистике или экспертами по проведению опросов для обеспечения соответствия собранных данных известным принципам планирования экспериментов. Кроме того, следует помнить, что показатели ошибок на валидационных и тестовых наборах не обязательно должны быть идеальными. Если мы проведем несколько экспериментов с поиском по сетке, то, скорее всего, получим переобученную модель, чрезмерно подогнанную к тестовому набору данных. Несмотря на ценность положительных результатов при тестировании, наше внимание должно быть сосредоточено на поиске значимого эффекта в реальном мире. Выполняет ли система свою работу так, как мы ожидали? Можем ли мы измерить это каким-либо достоверным способом, скажем, используя уточненное сопоставление для создания экспериментальных и контрольных групп в сочетании с A/B-тестированием и проверкой статистических гипотез на людях, которые взаимодействовали с системой, и тех, которые остались в стороне? Наконец, мы стараемся не утверждать, что наша система действительно работает, пока кто-то другой, например квалифицированный аудитор моделей, не воспроизведет наши результаты.

Мы признаем, что такие радикальные требования к науке о данных весьма амбициозны, но когда речь идет о приложениях ML высокого риска, нужно хотя бы попытаться. Мы обязаны избегать использования метода анализа данных в проектах ML высокого риска, поскольку сбои системы влияют на реальных людей, а происходить они могут быстро и в значительных масштабах.

Оценка опубликованных результатов и заявлений

Еще одна проблема, которая мешает нам применять научный подход, заключается в том, что мы забываем проверять опубликованную информацию, несмотря на весь ажиотаж вокруг ML. Многие источники, к которым мы обращаемся за информацией — Medium, Substack, Quora, LinkedIn, Twitter и другие социально-ориентированные платформы, — не являются рецензируемыми изданиями. Согласимся, что публиковаться на Medium или Substack интересно (и мы это делаем). Кроме того, эти платформы удобны для ознакомления с современными технологиями. Однако стоит помнить, что в таких изданиях каждый может сказать что угодно. Нам следует скептически относиться к результатам, опубликованным в социальных сетях, если они не являются прямым следствием результатов, опубликованных в более заслуживающих доверия изданиях или подтвержденных в каком-либо другом независимом эксперименте.

Кроме того, сервисы препринтов, такие как arXiv, тоже не проходят рецензирование. Если вы нашли там интересную информацию, обязательно посмотрите, дейст-

Книги для программистов: https://t.me/free_books_for_it

вительно ли она была опубликована позже в уважаемом журнале или хотя бы в материалах конференции, прежде чем брать ее на вооружение. Даже в случае рецензируемых журнальных публикаций или учебников следует уделять время самостоятельному выводу и проверке утверждений, когда это возможно. Если все цитаты в статье ведут к общепризнано лженаучным источникам, это очень плохой знак. Наконец, наш опыт подсказывает, что академические методы часто нуждаются в адаптации к реальной жизни. Но все же лучше опираться на прочный фундамент широко цитируемых научных исследований, чем на зыбучие пески блогов и постов в социальных сетях.



Блоги, информационные бюллетени и контент социальных сетей обычно не являются источниками авторитетной научной и инженерной информации.

Некоторые хорошо финансируемые исследовательские группы в технологических компаниях также могут подменять исследовательские достижения инженерным подвигом. Задумайтесь на минутку о языковых моделях (LM), которые часто считаются высшим достижением в области искусственного интеллекта среди исследовательских групп технологических компаний. Может ли даже хорошо финансируемая академическая исследовательская группа воспроизвести одну из этих моделей? Знаем ли мы, какие данные обучения использовались, или видели ли мы код? Разве эти системы не выходили из строя довольно часто? На протяжении столетий научно признанные результаты исследований были воспроизводимыми или как минимум проверяемыми. Хотя мы не сомневаемся в показателях, которые технологические компании публикуют для своих LM, мы задаемся вопросом, являются ли они значимо воспроизводимыми, проверяемыми или достаточно прозрачными, чтобы считаться исследовательскими, а не инженерными достижениями.



Читатели должны понимать, что, хотя эта книга прошла редакторское и техническое рецензирование, она не является рецензируемым научным источником. Это еще одна причина, по которой мы попытались привести книгу в соответствие с внешними стандартами, такими как система управления рисками NIST AI.

Кроме того, поскольку ML является коммерческой областью, где цель большинства исследований и инженерных разработок ML заключается в получении коммерческого продукта, а многие исследователи покидают академическое сообщество ради высокооплачиваемых инженерных должностей в промышленности, мы должны быть честными в отношении конфликта интересов. Если компания планирует продать технологию, нам следует относиться к любым опубликованным показателям с долей скептицизма. И еще хуже, если опубликованные результаты не проходят внешнюю, независимую и объективную проверку. Грубо говоря, заявления некой компании о том, что ее собственная технология великолепна, на самом деле не заслуживают доверия, независимо от того, насколько длинный технический отчет они опубликовали или насколько их шаблон LaTeX похож на статью NeurIPS. Необходимо быть осторожными с результатами, предоставленными коммерческими организациями и поставщиками ML.

Книги для программистов: https://t.me/free_books_for_it

Там много шумихи и красивой рекламы. В поисках направлений для следующего проекта нам следует быть более избирательными. Хотя мы можем упустить некоторые новые идеи, сосредоточившись на часто цитируемых научных журналах и учебниках, у нас будет гораздо более четкое представление о том, что на самом деле реально, а что нет. Мы также с большей вероятностью заложим в основу будущего проекта надежные и хорошо воспроизводимые идеи, а не рекламную шумиху. Имитировать успех приложений ML высокого риска гораздо сложнее, чем публиковать заметки в блогах или демонстрировать приложения с низким уровнем риска. Это означает, что у нас будет больше шансов добиться успеха в долгосрочной перспективе, даже если потребуется больше времени для начала и наши планы не покажутся впечатляющими на первый взгляд. В конце концов, настоящий успех в решении сложных задач гораздо важнее, чем большое количество постов в блогах и решение тривиальных задач.

Применяйте внешние стандарты

Долгое время стандарты в отношении ИИ и ML практически отсутствовали. Но сейчас ситуация меняется и стандарты начинают приобретать очертания. Если у вас возникли затруднения с ML и вы честны сами с собой, обратитесь за помощью и советом к экспертам. Официальные стандарты — отличное место, где можно найти помощь и подсказки по проектам ML высокого риска. В этом разделе мы сосредоточимся на стандартах Федерального резервного банка США (FRB), NIST, Закона ЕС об искусственном интеллекте и Международной организации по стандартизации (ISO), а также на том, как, по нашему мнению, их лучше всего использовать. Руководство FRB по управлению модельными рисками (MRM) и Система управления рисками NIST в области ИИ (RMF) сосредоточены на культуре и процессах, хотя NIST также описывает некоторые технические детали. Приложения к Закону ЕС об искусственном интеллекте отлично подходят для определений и документации. ISO предоставляет множество определений, а также хорошие технические рекомендации. Эти источники помогают нам обдумать различные типы рисков и способы их снижения, а также гарантировать, что мы не забудем что-то очевидное в проектах ML с высоким риском:

- *Руководство по управлению модельными рисками*

  Ранее в книге мы превозносили достоинства «Руководства для надзорных органов по управлению модельными рисками» (*https://oreil.ly/Gy_ol*) и сделаем это здесь еще раз. Только не обращайтесь к упомянутому руководству за техническими советами базового уровня. Придерживайтесь его при создании структур управления рисками в организации. На наш взгляд, из руководства можно извлечь следующие универсальные уроки:

  - *Все зависит от культуры*

    Если в культуре организации не ценится управление рисками, оно не работает.

Книги для программистов: https://t.me/free_books_for_it

- *Управление рисками начинается сверху*

  Советы директоров и высшие руководители должны принимать активное участие в управлении рисками ML.

- *В основе управления рисками лежит документация*

  Опишите, как работают ваши модели, чтобы другие сотрудники могли понять ваш подход.

- *Тестирование должно быть независимой и важной функцией*

  Тестировщики должны иметь право приостанавливать или прекращать разработку модели.

- *Людей необходимо привлекать к участию в управлении рисками*

  Это слишком сложно сделать бесплатно.

Кроме того, если читатели хотят получить представление об управлении рисками ML, рекомендуем ознакомиться с руководством Comptroller's Handbook: Model Risk Management и, в частности, с анкетой внутреннего контроля. Это шаги, которые выполняют органы банковского регулирования при проведении регулятивных проверок, и мы предлагаем взглянуть на них просто для ознакомления. Но учтите, что это лишь часть мер, которые вынуждены принимать банки, чтобы сохранить риски ML под контролем. Кроме того, эти средства управления рисками оказали большое влияние на RMF NIST AI, который многократно цитирует как надзорные рекомендации, так и Comptroller's Handbook. Вам будет полезно ознакомиться с ними уже сейчас, поскольку на их основе могут быть созданы нормативы и руководства по управлению рисками в вашей отрасли, секторе или вертикали. Сами эти ресурсы, руководящие указания и Comptroller's Handbook также, вероятно, продолжат постепенно развиваться.

♦ *Система управления рисками NIST в области ИИ*

Система управления рисками NIST существенно расширяет рекомендации MRM. В банковской сфере, где практикуется MRM, менеджеры по модельным рискам обычно могут рассчитывать на другие подразделения банка, которые будут заботиться о конфиденциальности, безопасности и справедливости, что позволяет им сосредоточиться в первую очередь на качестве работы системы. RMF объединяет эти и другие показатели — достоверность, надежность, безопасность, управление предвзятостью, отказоустойчивость, прозрачность, подотчетность, объяснимость, интерпретируемость и конфиденциальность — под одним знаменем управления рисками ИИ, что больше подходит для небанковских организаций.

RMF содержит обобщенные наставления по данным вопросам и, что важно, прямо заявляет, что все они взаимосвязаны. В отличие от руководства по MRM, RMF выделяет разнообразие и инклюзивность как средства контроля рисков и привносит средства управления ИТ-рисками, такие как реагирование на инциденты и вознаграждение за ошибки, в систему управления рисками ИИ. Руководство NIST также разбито на несколько документов и интерактивных веб-

Книги для программистов: https://t.me/free_books_for_it

сайтов. В то время как основной документ RMF содержит рекомендации более высокого уровня, ряд дополнительных ресурсов подробно описывают технические детали и управление рисками. Например, AI Risk Management Playbook (*https://oreil.ly/hd5oV*) содержит исчерпывающие рекомендации по управлению рисками с сопутствующей документацией, советами и ссылками. Сопутствующие документы, такие как NIST SP1270 и NISTIR 8367, Psychological Foundations of Explainability and Interpretability in Artificial Intelligence (*https://oreil.ly/UJ2EM*), предоставляют чрезвычайно полезные и подробные рекомендации по конкретным темам. RMF — это долгосрочный проект. Следите за появлением в ближайшие годы более качественных рекомендаций по управлению рисками.

- *Приложения к Закону ЕС об искусственном интеллекте*

  Здесь вы найдете определения высокого уровня, включая определение ML высокого риска, а также рекомендации по ведению документации. Приложение I Закона ЕС об искусственном интеллекте содержит четкое определение ИИ. Нам нужны единые и согласованные определения управления рисками. Это важно, потому что если предполагается, что политика или тест будут применяться ко всем системам ИИ в организации, то можно ожидать, что по крайней мере одна группа (или отдельный человек) откажется следовать требованиям, заявив, что они не используют ИИ. В Приложении III описаны конкретные сценарии применения ИИ, связанные с повышенным риском, такие как биометрическая идентификация, управление инфраструктурой, образование, трудоустройство, государственные или коммунальные услуги, кредитный скоринг, правоохранительная деятельность, иммиграционный и пограничный контроль, а также уголовное правосудие. Наконец, Приложение IV содержит хорошее руководство относительно того, что следует документировать в системе ML. Если документация в вашей организации находится где-то между большими документами MRM и минимальными карточками моделей, вам будет полезно взять приложения за основу для документирования системы ML. Имейте в виду, что Закон об искусственном интеллекте на момент публикации этой книги находился на стадии проекта, но его принятие ожидается в ближайшее время.

- *Система стандартов ISO AI*

  Система стандартов ISO AI является местом, где можно найти технические рекомендации более низкого уровня и горы технических определений. Хотя большая часть стандартов все еще пребывает в стадии разработки, многим нравятся ISO/IEC PRF TS 4213 (*https://oreil.ly/bMczF*) — Оценка эффективности классификации машинного обучения, ISO/IEC TR 24029-1:2021 (*https://oreil.ly/AvPNZ*) — Оценка устойчивости нейронных сетей и ISO/IEC TR 29119-11:2020 (*https://oreil.ly/MwV_T*) — Рекомендации по тестированию систем на основе искусственного интеллекта, и они уже доступны. Эти стандарты могут оказаться очень полезными дополнениями для обеспечения полноты и тщательности ваших технических методов. В отличие от других руководств, обсуждаемых в этом разделе, стандарты ISO обычно не являются бесплатными. Но они не так уж и до́роги, во всяком случае гораздо дешевле, чем инцидент с ИИ. Следите за стандартами ISO AI, поскольку они со временем становятся все более полными. Из них вы

Книги для программистов: https://t.me/free_books_for_it

сможете извлечь ценные технические рекомендации и ссылки на ресурсы по управлению рисками.



Применение внешних стандартов, таких как ISO и NIST, улучшает качество вашей работы и повышает защищенность, когда что-то неизбежно идет не так.

Существуют и другие стандарты таких групп, как Институт инженеров по электротехнике и электронике (IEEE), Американский национальный институт стандартов (ANSI) или Организация экономического сотрудничества и развития (ОЭСР), которые также могут вам пригодиться. Польза от всех этих стандартов заключается еще и в том, что они помогают не только лучше организовать работу, но и обосновать свой выбор в ответ на критические замечания. Если вы занимаетесь проектами ML в области высоких рисков, приготовьтесь к пристальному вниманию надзорных органов. Обоснование вашего рабочего процесса и контроля рисков такими стандартами будет гораздо более убедительным, чем ссылки на блоги и социальные сети. Короче говоря, использование стандартов делает нас и нашу работу лучше, потому что технологии немыслимы без стандартизации.

Смягчение рисков на основе здравого смысла

Чем больше времени мы тратим на проекты ML высокого риска, тем больше у нас развивается инстинктивное понимание того, что́ пойдет не так, а что́ получится хорошо. Советы, подробно изложенные в этом разделе, вероятно, можно найти в некоторых стандартах или руководствах, но мы проверили их на собственном горьком опыте. Они должны развить инстинктивное чутье практикующего специалиста по работе с системами ML высокого риска. Эти жизненные уроки могут показаться слишком простыми или очевидными, но на самом деле бывает очень сложно заставить себя их придерживаться. Всегда существует давление рынка, требующее действовать быстрее, меньше тестировать и меньше бороться с рисками. Спешка может быть хороша при развертывании приложений с низкими рисками, но в серьезных проектах иногда полезно остановиться и подумать. Шаги, которые мы подробно описываем здесь, помогают понять, почему и как мы это делаем. По сути, мы должны думать, прежде чем писать код, тестировать код, и выделять достаточно времени и ресурсов на то и другое.

Начните с простых решений.

Вам, как разработчику, может быть интересно использовать в приложениях высокого риска системы ML основанные на глубоком обучении, многоуровневом обобщении или других сложных методах. Однако не следует применять подобные методы, если задача не требует такого уровня сложности. Сложность обычно сопряжена с повышенной вероятностью сбоев и меньшей прозрачностью. Системы с меньшей прозрачностью обычно труднее исправлять и сложнее проверять. Приступая к проекту с высоким риском, вы должны сопоставить возможность сбоя и связанный с этим вред с вашим желанием поиграть с крутыми технологиями. Иногда лучше начать с более простого и понятного подхода, а за-

Книги для программистов: https://t.me/free_books_for_it

тем переходить к более сложным решениям по мере проверки вашей системы на практике.

Избегайте повторения неудачных проектов.

Не повторяйте ошибки предыдущих проектов. Приступая к решению задачи с высокими рисками, проанализируйте прошлые неудачные попытки решить аналогичные задачи. Мы уже упоминали базу данных инцидентов с ИИ (*https://oreil.ly/VlclU*). Это один из нескольких ресурсов, к которым вам следует обратиться во избежание повторения ошибок с ML. Попробуйте также опросить сотрудников внутри своей организации. Возможно, кто-то уже пытался решить аналогичные задачи, особенно если они имеют большое значение для вашей компании.

Выделите время и ресурсы для управления рисками.

Управление рисками требует времени, людей, денег и других ресурсов. Команда, создавшая демоверсию системы, вероятно, недостаточно сильна, чтобы создать производственную версию и управлять ее рисками. Если вы работаете над системой ML высокого риска, вам понадобится больше ресурсов для тщательного проектирования, тестирования, документирования, обработки отзывов пользователей и анализа рисков. Вам также нужно больше времени. Сами организации, менеджеры и специалисты по данным склонны недооценивать время, необходимое для создания даже обыденной системы ML. Если вы работаете с ML высокого риска, вам придется продлить сроки, возможно, даже в несколько раз по сравнению с тем временем, которое может потребоваться для разработки системы с низким уровнем риска.

Применяйте стандартные подходы к обеспечению качества ПО.

Мы уже говорили это раньше и скажем еще раз. Нет никаких причин, по которым системы ML следует освобождать от стандартных процессов контроля качества ПО. Для систем высокого риска нам, вероятно, потребуется внушительный набор инструментов контроля качества ПО: модульные, интеграционные и функциональные тесты, хаос-тестирование, случайные атаки и многое другое. Если вам нужно освежить знания о том, как эти методы применяются к системам ML, перечитайте главу 3.

Минимизируйте зависимости программного обеспечения, оборудования и сети.

Каждое используемое вами стороннее ПО, с открытым исходным кодом или проприетарное, увеличивает риски системы. Мы не всегда можем знать, как обстоят дела с рисками в этих зависимостях. Они безопасны? Справедливы? Соответствуют ли они законам о конфиденциальности данных? Это сложно узнать. Те же понятия применимы и к сетевым зависимостям. Защищены ли машины, к которым мы подключаемся? Всегда ли они будут доступны? Ответ прост: если мы говорим о достаточно длительном периоде времени, то не всегда. Хотя специализированное оборудование, как правило, создает меньшие риски безопасности и сбоев, чем стороннее ПО и дополнительные сетевые подключения, оно увеличивает сложность. Повышенная сложность по умолчанию увеличивает

Книги для программистов: https://t.me/free_books_for_it

риск. Минимизация и упрощение программных, аппаратных и сетевых зависимостей, скорее всего, сократят количество неожиданностей, необходимых процессов управления изменениями и ресурсов для управления рисками.

Ограничьте взаимные подключения между несколькими системами ML.

Если риски даже одной системы ML трудно перечислить, что произойдет, когда мы начнем создавать конвейеры решений или технологий на основе ML? Результаты могут быть крайне непредсказуемыми. Будьте осторожны при подключении систем ML к крупным сетям, таким как Интернет, или при соединении нескольких систем ML между собой. Оба сценария могут привести к неожиданному ущербу или даже к системным сбоям.

Ограничьте выходные данные системы, чтобы избежать предсказуемых инцидентов.

Если определенные выходные данные системы ML предсказуемо проблематичны, например разрешение беспилотному автомобилю разгоняться до 300 км/ч, вы не должны сидеть сложа руки и позволять системе принимать неверные решения. Используйте бизнес-правила, утверждения модели, числовые ограничения или другие меры безопасности, чтобы не допустить принятия системой предсказуемых ошибочных решений.

Помните, что игры — это не реальный мир.

Таблицы результатов соревнований, в которых модели ранжируются на основе отдельных показателей без учета отклонений или компромиссов, не подходят для принятия решений в реальном мире. Тот факт, что система ML успешна в игре, не означает, что она будет успешной и в жизни. В играх мы знаем все правила, и они не меняются. В некоторых случаях у нас есть доступ ко всем возможным данным, касающимся игр, например ко всем возможным результатам или всем возможным ходам. Но в жизни так не бывает. В реальном мире мы не знаем всех правил, и даже известные правила могут внезапно и быстро меняться. У нас также нет доступа ко всем данным, необходимым для принятия правильных решений. Система ML, победившая в соревновании, может стать огромным исследовательским достижением, но при этом не иметь отношения к социотехническим системам ML высокого риска, развернутым в реальном мире.

Тщательно отслеживайте обучаемые без учителя или самообновляющиеся системы.

Системы ML, обученные без достоверных проверенных данных, и самообновляющиеся системы (например, на основе обучения с подкреплением, адаптивного или онлайн-обучения) по своей сути являются источниками повышенного риска. Трудно понять, достаточно ли хорошо работают такие системы, прежде чем мы их развернем, и трудно предсказать, как будет вести себя самообновляющаяся система с течением времени. Хотя все системы ML необходимо отслеживать, обучаемые без учителя и самообновляющиеся системы, развернутые в области высокого риска, требуют мониторинга в реальном времени на предмет проблем с точностью, предвзятостью и безопасностью. Средства мониторинга должны незамедлительно предупреждать операторов системы при обнаружении

Книги для программистов: https://t.me/free_books_for_it

проблем, и эти системы, вероятно, должны быть изначально оснащены аварийными выключателями.

Понимайте этические и юридические обязательства перед людьми.

Поскольку многие развертывания ML подразумевают сбор конфиденциальных данных или сами по себе являются неявными или явными экспериментами над пользователями, вам следует ознакомиться с политиками институционального наблюдательного совета (institutional review board, IRB) вашей организации, основными рекомендациями по экспериментам с участием пользователей, а также другими юридическими и этическими обязательствами по проведению экспериментов, затрагивающих пользователей.

Запретите анонимное использование.

Если пользователи тем или иным образом подтвердят свою личность перед использованием системы ML, это может значительно сократить количество взломов, злоупотреблений и других неправомерных действий, связанных с системой.

Применяйте водяные знаки к контенту, созданному искусственным интеллектом.

Добавление контрольных меток, символов и звуков в любой контент, созданный ИИ, поможет идентифицировать его позже и снизит риски использования такого контента для обманных действий.

Четко понимайте, когда не следует использовать ML.

ML не решает все проблемы. На самом деле существует большой класс проблем, которые ML вообще никак не решает. Машинное обучение не превосходит людей или простые модели в прогнозировании исходов жизненных ситуаций. Впрочем, люди и простые модели тоже не очень хороши в этом. ML не может точно предсказать, кто получит хорошие оценки, кому грозит выселение или увольнение с работы. По данным NIST, ML также не может определить по видео, кто преуспеет в работе. Видные исследователи ML, в том числе Арвинд Нараянан, указали на проблемы в прогнозах ML в отношении криминальных рецидивов, полицейской деятельности и выявления террористов.

ML не очень хорошо понимает и прогнозирует многие явления в социальной жизни человека. Хотя это интересные и важные задачи, нам не следует пытаться решить их с помощью ML, если мы не знаем чего-то, чего NIST и Национальные академии США еще не знают об ML. И социальные явления — не единственная область, в которой, как известно, у систем ML возникают проблемы. Не забудьте изучить прошлые неудачи, прежде чем разрабатывать систему ML высокого риска.



Не бойтесь задавать элементарные вопросы о дизайне, сроках, ресурсах, результатах и пользователях в вашем следующем важном проекте ML.

Сочетая соображения здравого смысла с увеличением демографического и профессионального разнообразия, с более строгим соблюдением научного подхода, более строгой оценкой опубликованных заявлений, применением авторитетных внешних стандартов и всеми управленческими и техническими инструментами, описанными

Книги для программистов: https://t.me/free_books_for_it

в предыдущих главах, методично продвигайтесь к лучшим результатам в ответственных приложениях ML. Конечно, сложно заручиться поддержкой со стороны руководства и коллег для проведения всей этой дополнительной работы. И даже если все получится, нужно будет найти на нее время. Не пытайтесь вскипятить океан. Вспомните главу 1 и основы управления рисками. Постарайтесь понять, каковы ваши самые серьезные риски, и в первую очередь смягчите их.

Заключение

Эта книга началась с уроков по управлению людьми, которые создают и поддерживают системы ML. Затем мы обсудили, как сделать модели ML более понятными для пользователей и операторов с помощью объяснимых моделей и объяснимого ИИ. Мы говорили о том, как сделать модели ML заслуживающими большего доверия при помощи инструментов отладки и безопасности, и при этом особо подчеркивали, что должны сделать их более справедливыми для широкого круга людей. Такое внимание к человеку не случайно. Технологии — это люди. Нет других причин создавать новые технологии, кроме как во благо нас с вами. Машины не чувствуют боли, гнева и печали, когда им причиняют вред, но люди ранимы. Более того, по крайней мере, по нашему мнению, человек все же умнее компьютеров. Последнее десятилетие технологии ML ассоциировались с успехом огромных необъяснимых моделей, обученных практически без участия человека, и мы полагаем, что пришло время маятнику качнуться в другую сторону. Многие успехи ML в ближайшие годы будут связаны с соблюдением законодательных и нормативных требований, улучшением взаимодействия людей с ML, управлением рисками и прямой отдачей для бизнеса. Заложите в основу вашего проекта ML высокого риска максимизацию выгоды и минимизацию вреда для людей, и вы добьетесь большего успеха.

Ресурсы

Дополнительное чтение

- Дополнения к EU AI Act: **https://oreil.ly/CcERN**
- Стандарты ISO AI: **https://oreil.ly/cUmGz**
- Система управления рисками NIST AI: **https://oreil.ly/fN5BS**
- NIST SP1270 — Стандарты для выявления и управления предвзятостью в искусственном интеллекте: **https://oreil.ly/udvYe**
- Руководство надзорного органа по управлению модельными рисками: **https://oreil.ly/IuzZx**

Книги для программистов: https://t.me/free_books_for_it

Предметный указатель

А

Аварийный выключатель 153
Аддитивные объяснения Шепли 88
Алгоритмическое изъятие 38
Альтернативные данные 386
Анализ
◊ главных компонент разреженный 82
◊ остатков 137
◊ чувствительности 300
Атака
◊ scaffolding 208
◊ извлечения и инверсии модели 209
◊ отмывание справедливости 208
◊ отравление данных 206
◊ прожорливых выборок 201
◊ с выводом о членстве 209
◊ состязательная по выборке 205

Б

Биннинг 68

В

Водяной знак 428

Г

Гиперпараметр регуляризации 395
Глубокое обучение 271
Гребневая регрессия 73
Групповое мышление 161

Д

Дерево решений 76
◊ с суррогатными разделениями 254
Джиттер 359
Дипфейк 212
Дифференциальная приватность 218
Доверительная надежность 246
Дрейф
◊ данных 150
◊ концепта 151

З

Заблуждение Макнамары 161
Запутанность 117
Защищенные группы 163

И

Индекс стабильности популяции 129
Индивидуальное условное ожидание 84, 99
Интерпретируемая базовая концепция 277
Исключение
◊ одного признака 93
◊ одной ковариаты 93
Искусственный интеллект 21

К

Калибровка прогнозов 332
Конструктная валидность 122
Коэффициент неблагоприятного воздействия 174, 367

Л

Ложное обучение 133

М

Машина градиентного бустинга 79
Машинное обучение 21
◊ автоматическое 50
Метаданные 115

Книги для программистов: https://t.me/free_books_for_it

Метод
◊ анализа данных 440
◊ знака градиента 349
◊ интегрированных градиентов 287
Множественные сравнения 124
Моделирование остатков 138
Модель
◊ аудит 55
◊ детоксикация 189
◊ дистилляция 95
◊ емкость 28
◊ извлечение 95, 256
◊ нейронная аддитивная 75
◊ обобщение 70
◊ обобщенная аддитивная 25, 68, 229
◊ обобщенная линейная 236
◊ обработка естественного языка 336
◊ ограничения 69
◊ отладка 124
◊ оценка 55
◊ регрессия со штрафом 71
◊ редактирование 75
◊ реестр 53
◊ сверхразреженная линейная целочисленная 74
◊ сжатие 95, 256
◊ слабый ученик 247
◊ согласованность 86
◊ суррогатная 95, 255
◊ управление рисками 33
◊ устаревание 53

Н

Награда за обнаружение ошибок 41, 57
Недостаточная специфичность 78
Нейронная сеть
◊ самообъясняемая 277
◊ сверточная 282
◊ тонкая настройка 282
Неодинаковая достоверность 164
Неполная идентификация 134
Несоразмерное
◊ влияние 163
◊ отношение 163

О

Область под границей 343
Обобщение вне распределения 345
Обратная дискриминация 188
Обучение
◊ предварительное 361
◊ реалистичное 361
Объяснимая бустинговая машина 26, 146, 229
Объяснимые нейронные сети 80
Опосредованная предвзятость 388
Отказ от участия 56
Отмывание справедливости 105
Отравление данных 43
Оценка воздействия 55

П

Патч 277
Перенос обучения 282
Переобучение 132
Планирование эксперимента 185
Площадь под кривой 113
Поиск состязательных примеров 301
Послойное распространение релевантности 289
Правило Хэнда 37
Признак
◊ атрибуция 83
◊ важность 83
Программирование
◊ двойное 56
◊ парное 56
Прототип данных 70

Р

Рабочая характеристика приемника 415
Распознавание именованного объекта 336
Регрессия Тихонова *См.* Гребневая регрессия
Робастность 128

С

Самоподтверждение 161
Сегментный анализ ошибок 323
Смещение субпопуляции 340, 348
Состязательные примеры 141
Стандартизованная средняя разница 367
Статистический паритет 173
Стресс-тестирование 300
Существенность 50

Книги для программистов: https://t.me/free_books_for_it

Т

Тепловая карта 284
Технологии повышения
конфиденциальности 218
Техношовинизм 162
Триада CIA 200

У

Управление модельными рисками 19, 44

Ф

Факторизация неотрицательной матрицы 82
Федеративное обучение 218
Феномен мужского взгляда 167
Функция
◊ связи 73
◊ формы 75

Ц

Цифровой разрыв 162

Ч

Частичная зависимость 99, 231

Э

Эпистемическая неопределенность 129
Эффект
◊ Даннинга — Крюгера 161
◊ накопленный локальный 99, 231
◊ Расёмона 78, 368
Эффективное
◊ испытание 45
◊ оспаривание 193

Книги для программистов: https://t.me/free_books_for_it

Об авторах

Патрик Холл — главный научный сотрудник BNH.AI, где консультирует компании из списка Fortune 500 и передовые стартапы по вопросам рисков ИИ и проводит исследования в рамках Системы управления рисками искусственного интеллекта NIST. Он также работает приглашенным преподавателем на факультете принятия решений в Школе бизнеса Джорджа Вашингтона, преподает этику данных, бизнес-аналитику и машинное обучение.

Прежде чем стать соучредителем BNH, Патрик руководил работой H2O.ai в области ответственного ИИ, что привело к созданию одного из первых в мире коммерческих приложений для обеспечения объяснимости и уменьшения предвзятости ML. Он также занимал различные должности по работе с клиентами по всему миру и занимался исследованиями и разработками в Институте SAS. Патрик изучал вычислительную химию в Университете Иллинойса, а затем окончил Институт перспективной аналитики Университета штата Северная Каролина.

Патрика приглашали выступить на темы, связанные с объяснимым ИИ, в Национальных академиях науки, техники и медицины, ACM SIG-KDD и на конференциях по статистике. Он писал статьи для таких изданий, как McKinsey.com, O'Reilly Radar и Thompson Reuters Regulatory Intelligence, а его технические работы опубликованы в Fortune, Wired, Info World, TechCrunch и др.

Джеймс Кертис — исследователь данных в Solea Energy, где занимается развитием статистического прогнозирования для дальнейшей декарбонизации энергосистемы США. Ранее он работал консультантом в организациях, предоставляющих финансовые услуги, страховых компаниях, регулирующих органах и поставщиках медицинских услуг, помогая создавать более справедливые модели ИИ и ML. Джеймс получил степень магистра математики в Колорадской горной школе.

Парул Панди имеет опыт работы в области электротехники и в настоящее время работает главным специалистом по данным в H2O.ai. До этого она работала инженером по машинному обучению в Weights & Biases. Она также является обладателем титулов Kaggle Grandmaster в категории Notebooks и LinkedIn Top Voices в категории «Разработка программного обеспечения» в 2019 г. Парул написала несколько статей, посвященных науке о данных и разработке программного обеспечения, для различных изданий. Также она выступает с докладами и проводит семинары по тематике ответственного ИИ.

Книги для программистов: https://t.me/free_books_for_it

Об изображении на обложке

На обложке книги изображен гигантский африканский фруктовый жук (*Mecynorrina Polyphemus*).

Этот большой зеленый жук-скарабей, ранее носивший латинское название *Chelorrina polyphemus*, является членом семейства цветочных хрущей *Cetoniinae* — группы яркоокрашенных жуков, которые питаются в основном цветочной пыльцой, нектаром и лепестками, а также соком фруктов и деревьев. Гигантские африканские фруктовые жуки длиной от 35 до 80 мм являются крупнейшими жуками рода *Mecynorrina*.

Эти колоссальные скарабеи обитают в густых тропических лесах Центральной Африки. Взрослые особи имеют половой диморфизм: у самок блестящий призматический панцирь, а у самцов — рога и более бархатистая или матовая окраска. Благодаря внешней привлекательности и относительной простоте разведения в домашних условиях жуки-скарабеи стали очень популярны среди начинающих энтомологов. Этот факт, наряду с разрушением среды обитания, был назван в одном из исследований фактором сокращения популяции в некоторых районах, хотя в целом жуки сохраняют свою численность.

Многие животные, изображенные на обложках издательства O'Reilly, находятся под угрозой исчезновения, все они важны для мира.

Иллюстрация на обложке выполнена Карен Монтгомери на основе черно-белой гравюры Кювье из «Естественной истории».

Книги для программистов: https://t.me/free_books_for_it

Патрик Холл, Джеймс Кертис, Парул Панди

Машинное обучение для приложений высокого риска

Перевод с английского

ТОО "АЛИСТ"
010000, Республика Казахстан,
г. Астана, пр. Сарыарка, д. 17, ВП 30

Книги для программистов: https://t.me/free_books_for_it